

```
=====
FILE: data\output\canonical_model.json
ENCODING: utf-8
=====
```

```
{
  "records": [
    {
      "name": "COVERAGE",
      "primary_key": null,
      "record_id": null,
      "location_mode": null,
      "fields": [
        {
          "name": "SELECTION_DATE_0400",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "TERMINATION_DATE_0400",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "TYPE_0400",
          "datatype": "CHAR",
          "length": 1,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "INS_PLAN_CODE_0400",
          "datatype": "VARCHAR",
          "length": 3,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        }
      ]
    },
    {
      "name": "DENTAL_CLAIM",
      "primary_key": null,
      "record_id": null,
      "location_mode": null,
      "fields": [
        {
          "name": "CLAIM_DATE_0405",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "PATIENT_BIRTH_DATE_0405",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "SERVICE_DATE_0405",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        }
      ]
    }
  ]
}
```

```

},
{
  "name": "PATIENT_FIRST_NAME_0405",
  "datatype": "VARCHAR",
  "length": 10,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "PATIENT_LAST_NAME_0405",
  "datatype": "VARCHAR",
  "length": 15,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "PATIENT_SEX_0405",
  "datatype": "CHAR",
  "length": 1,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "RELATION_TO_EMPLOYEE_0405",
  "datatype": "VARCHAR",
  "length": 10,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DENTIST_FIRST_NAME_0405",
  "datatype": "VARCHAR",
  "length": 10,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DENTIST_LAST_NAME_0405",
  "datatype": "VARCHAR",
  "length": 15,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DENTIST_STREET_0405",
  "datatype": "VARCHAR",
  "length": 20,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DENTIST_CITY_0405",
  "datatype": "VARCHAR",
  "length": 15,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DENTIST_STATE_0405",
  "datatype": "VARCHAR",
  "length": 2,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DENTIST_ZIP_FIRST_FIVE_0405",
  "datatype": "VARCHAR",

```

```

        "length": 5,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DENTIST_ZIP_LAST_FOUR_0405",
        "datatype": "VARCHAR",
        "length": 4,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DENTIST_LICENSE_NUMBER_0405",
        "datatype": "SMALLINT",
        "length": null,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "NUMBER_OF_PROCEDURES_0405",
        "datatype": "DECIMAL",
        "length": 1,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "TOOTH_NUMBER_0405",
        "datatype": "SMALLINT",
        "length": null,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "PROCEDURE_CODE_0405",
        "datatype": "SMALLINT",
        "length": null,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DESCRIPTION_OF_SERVICE_0405",
        "datatype": "VARCHAR",
        "length": 60,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "FEE_0405",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "DEPARTMENT",
    "primary_key": "DEPT_ID_0410",
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "DEPT_ID_0410",
            "datatype": "SMALLINT",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        }
    ]
}

```

```

    },
    {
      "name": "DEPT_NAME_0410",
      "datatype": "VARCHAR",
      "length": 45,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "DEPT_HEAD_ID_0410",
      "datatype": "SMALLINT",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
},
{
  "name": "EMPLOYEE",
  "primary_key": "EMP_ID_0415",
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "START_DATE_0415",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "TERMINATION_DATE_0415",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "BIRTH_DATE_0415",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "EMP_ID_0415",
      "datatype": "SMALLINT",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "EMP_FIRST_NAME_0415",
      "datatype": "VARCHAR",
      "length": 10,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "EMP_LAST_NAME_0415",
      "datatype": "VARCHAR",
      "length": 15,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "EMP_STREET_0415",
      "datatype": "VARCHAR",

```

```

        "length": 20,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_CITY_0415",
        "datatype": "VARCHAR",
        "length": 15,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_STATE_0415",
        "datatype": "VARCHAR",
        "length": 2,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_ZIP_FIRST_FIVE_0415",
        "datatype": "VARCHAR",
        "length": 5,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_ZIP_LAST_FOUR_0415",
        "datatype": "VARCHAR",
        "length": 4,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_PHONE_0415",
        "datatype": "SMALLINT",
        "length": null,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "STATUS_0415",
        "datatype": "VARCHAR",
        "length": 2,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "SS_NUMBER_0415",
        "datatype": "SMALLINT",
        "length": null,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "EMPOSITION",
    "primary_key": null,
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "START_DATE_0420",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        }
    ]
}

```

```

    },
    {
      "name": "FINISH_DATE_0420",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "SALARY_GRADE_0420",
      "datatype": "SMALLINT",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "SALARY_AMOUNT_0420",
      "datatype": "DECIMAL",
      "length": 2,
      "scale": 2,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "BONUS_PERCENT_0420",
      "datatype": "DECIMAL",
      "length": 3,
      "scale": 3,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "COMMISSION_PERCENT_0420",
      "datatype": "DECIMAL",
      "length": 3,
      "scale": 3,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "OVERTIME_RATE_0420",
      "datatype": "DECIMAL",
      "length": 3,
      "scale": 2,
      "occurs": false,
      "occurs_max": null
    }
  ]
},
{
  "name": "EXPERTISE",
  "primary_key": null,
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "EXPERTISE_DATE_0425",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "SKILL_LEVEL_0425",
      "datatype": "VARCHAR",
      "length": 2,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
}
],
{

```

```

"name": "HOSPITAL_CLAIM",
"primary_key": null,
"record_id": null,
"location_mode": null,
"fields": [
  {
    "name": "CLAIM_DATE_0430",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_BIRTH_DATE_0430",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "ADMIT_DATE_0430",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DISCHARGE_DATE_0430",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_FIRST_NAME_0430",
    "datatype": "VARCHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_LAST_NAME_0430",
    "datatype": "VARCHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_SEX_0430",
    "datatype": "CHAR",
    "length": 1,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "RELATION_TO_EMPLOYEE_0430",
    "datatype": "VARCHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSPITAL_NAME_0430",
    "datatype": "VARCHAR",
    "length": 25,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  }
]

```

```

},
{
  "name": "HOSP_STREET_0430",
  "datatype": "VARCHAR",
  "length": 20,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "HOSP_CITY_0430",
  "datatype": "VARCHAR",
  "length": 15,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "HOSP_STATE_0430",
  "datatype": "VARCHAR",
  "length": 2,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "HOSP_ZIP_FIRST_FIVE_0430",
  "datatype": "VARCHAR",
  "length": 5,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "HOSP_ZIP_LAST_FOUR_0430",
  "datatype": "VARCHAR",
  "length": 4,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DIAGNOSIS_0430",
  "datatype": "VARCHAR",
  "length": 120,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "WARD_DAYS_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 0,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "WARD_RATE_0430",
  "datatype": "DECIMAL",
  "length": 2,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "WARD_TOTAL_0430",
  "datatype": "DECIMAL",
  "length": 2,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "SEMI_DAYS_0430",
  "datatype": "DECIMAL",

```



```

        "length": 5,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "SEMI_RATE_0430",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "SEMI_TOTAL_0430",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DELIVERY_COST_0430",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "ANESTHESIA_COST_0430",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "LAB_COST_0430",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "INSURANCE_PLAN",
    "primary_key": "INS_PLAN_CODE_0435",
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "INS_PLAN_CODE_0435",
            "datatype": "VARCHAR",
            "length": 3,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "INS_CO_NAME_0435",
            "datatype": "VARCHAR",
            "length": 45,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "INS_CO_STREET_0435",
            "datatype": "VARCHAR",
            "length": 20,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        }
    ]
}

```

```

},
{
  "name": "INS_CO_CITY_0435",
  "datatype": "VARCHAR",
  "length": 15,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "INS_CO_STATE_0435",
  "datatype": "VARCHAR",
  "length": 2,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "INS_CO_ZIP_FIRST_FIVE_0435",
  "datatype": "VARCHAR",
  "length": 5,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "INS_CO_ZIP_LAST_FOUR_0435",
  "datatype": "VARCHAR",
  "length": 4,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "INS_CO_PHONE_0435",
  "datatype": "SMALLINT",
  "length": null,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "GROUP_NUMBER_0435",
  "datatype": "SMALLINT",
  "length": null,
  "scale": null,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DEDUCT_0435",
  "datatype": "DECIMAL",
  "length": 2,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "MAXIMUM_LIFE_COST_0435",
  "datatype": "DECIMAL",
  "length": 2,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "FAMILY_COST_0435",
  "datatype": "DECIMAL",
  "length": 2,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DEP_COST_0435",
  "datatype": "DECIMAL",

```

```

        "length": 2,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "JOB",
    "primary_key": "JOB_ID_0440",
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "JOB_ID_0440",
            "datatype": "SMALLINT",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "TITLE_0440",
            "datatype": "VARCHAR",
            "length": 20,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "DESCRIPTION_LINE_0440",
            "datatype": "VARCHAR",
            "length": 120,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "REQUIREMENT_LINE_0440",
            "datatype": "VARCHAR",
            "length": 120,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "MINIMUM_SALARY_0440",
            "datatype": "DECIMAL",
            "length": 2,
            "scale": 2,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "MAXIMUM_SALARY_0440",
            "datatype": "DECIMAL",
            "length": 2,
            "scale": 2,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "SALARY_GRADES_0440",
            "datatype": "SMALLINT",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "NUMBER_OF_POSITIONS_0440",
            "datatype": "SMALLINT",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        }
    ]
}

```

```

    },
    {
      "name": "NUMBER_OPEN_0440",
      "datatype": "SMALLINT",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
},
{
  "name": "NON_HOSP_CLAIM",
  "primary_key": null,
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "CLAIM_DATE_0445",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_BIRTH_DATE_0445",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "SERVICE_DATE_0445",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_FIRST_NAME_0445",
      "datatype": "VARCHAR",
      "length": 10,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_LAST_NAME_0445",
      "datatype": "VARCHAR",
      "length": 15,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_SEX_0445",
      "datatype": "CHAR",
      "length": 1,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "RELATION_TO_EMPLOYEE_0445",
      "datatype": "VARCHAR",
      "length": 10,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PHYSICIAN_FIRST_NAME_0445",
      "datatype": "VARCHAR",

```

```

    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_LAST_NAME_0445",
    "datatype": "VARCHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_STREET_0445",
    "datatype": "VARCHAR",
    "length": 20,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_CITY_0445",
    "datatype": "VARCHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_STATE_0445",
    "datatype": "VARCHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_ZIP_FIRST_FIVE_0445",
    "datatype": "VARCHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_ZIP_LAST_FOUR_0445",
    "datatype": "VARCHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_ID_0445",
    "datatype": "SMALLINT",
    "length": null,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DIAGNOSIS_0445",
    "datatype": "VARCHAR",
    "length": 120,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "NUMBER_OF_PROCEDURES_0445",
    "datatype": "DECIMAL",
    "length": 1,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  }

```

```

    },
    {
      "name": "PROCEDURE_CODE_0445",
      "datatype": "SMALLINT",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "DESCRIPTION_OF_SERVICE_0445",
      "datatype": "VARCHAR",
      "length": 60,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "FEE_0445",
      "datatype": "DECIMAL",
      "length": 2,
      "scale": 2,
      "occurs": false,
      "occurs_max": null
    }
  ]
},
{
  "name": "OFFICE",
  "primary_key": "OFFICE_CODE_0450",
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "OFFICE_CODE_0450",
      "datatype": "VARCHAR",
      "length": 3,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "OFFICE_STREET_0450",
      "datatype": "VARCHAR",
      "length": 20,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "OFFICE_CITY_0450",
      "datatype": "VARCHAR",
      "length": 15,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "OFFICE_STATE_0450",
      "datatype": "VARCHAR",
      "length": 2,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "OFFICE_ZIP_FIRST_FIVE_0450",
      "datatype": "VARCHAR",
      "length": 5,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "OFFICE_ZIP_LAST_FOUR_0450",
      "datatype": "VARCHAR",

```

```

        "length": 4,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "OFFICE_PHONE_0450",
        "datatype": "SMALLINT",
        "length": null,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "OFFICE_AREA_CODE_0450",
        "datatype": "VARCHAR",
        "length": 3,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "SPEED_DIAL_0450",
        "datatype": "VARCHAR",
        "length": 3,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "SKILL",
    "primary_key": "SKILL_ID_0455",
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "SKILL_ID_0455",
            "datatype": "SMALLINT",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "SKILL_NAME_0455",
            "datatype": "VARCHAR",
            "length": 12,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "SKILL_DESCRIPTION_0455",
            "datatype": "VARCHAR",
            "length": 60,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        }
    ]
},
{
    "name": "STRUCTURE",
    "primary_key": null,
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "STRUCTURE_DATE_0460",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        }
    ]
}

```

```

    },
    {
      "name": "STRUCTURE_CODE_0460",
      "datatype": "VARCHAR",
      "length": 2,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
},
"sets": [
  {
    "name": "DEPT-EMPLOYEE",
    "owner_record": "DEPARTMENT",
    "member_record": "EMPLOYEE",
    "order_mode": null
  },
  {
    "name": "MANAGES",
    "owner_record": "EMPLOYEE",
    "member_record": "STRUCTURE",
    "order_mode": null
  },
  {
    "name": "OFFICE-EMPLOYEE",
    "owner_record": "OFFICE",
    "member_record": "EMPLOYEE",
    "order_mode": null
  },
  {
    "name": "EMP-EXPERTISE",
    "owner_record": "EMPLOYEE",
    "member_record": "EXPERTISE",
    "order_mode": null
  },
  {
    "name": "SKILL-EXPERTISE",
    "owner_record": "SKILL",
    "member_record": "EXPERTISE",
    "order_mode": null
  },
  {
    "name": "EMP-EMPOSITION",
    "owner_record": "EMPLOYEE",
    "member_record": "EMPOSITION",
    "order_mode": null
  },
  {
    "name": "REPORTS-TO",
    "owner_record": "EMPLOYEE",
    "member_record": "STRUCTURE",
    "order_mode": null
  },
  {
    "name": "EMP-COVERAGE",
    "owner_record": "EMPLOYEE",
    "member_record": "COVERAGE",
    "order_mode": null
  },
  {
    "name": "JOB-EMPOSITION",
    "owner_record": "JOB",
    "member_record": "EMPOSITION",
    "order_mode": null
  },
  {
    "name": "COVERAGE-CLAIMS",
    "owner_record": "COVERAGE",
    "member_record": "DENTAL_CLAIM",
    "order_mode": null
  },
  {
    "name": "COVERAGE-CLAIMS",
    "owner_record": "COVERAGE",

```



```

    "member_record": "HOSPITAL_CLAIM",
    "order_mode": null
  },
  {
    "name": "COVERAGE-CLAIMS",
    "owner_record": "COVERAGE",
    "member_record": "NON_HOSP_CLAIM",
    "order_mode": null
  }
],
"relationships": [
  {
    "set_name": "DEPT-EMPLOYEE",
    "parent_record": "DEPARTMENT",
    "child_record": "EMPLOYEE",
    "cardinality": "1:N"
  },
  {
    "set_name": "MANAGES",
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "cardinality": "1:N"
  },
  {
    "set_name": "OFFICE-EMPLOYEE",
    "parent_record": "OFFICE",
    "child_record": "EMPLOYEE",
    "cardinality": "1:N"
  },
  {
    "set_name": "EMP-EXPERTISE",
    "parent_record": "EMPLOYEE",
    "child_record": "EXPERTISE",
    "cardinality": "1:N"
  },
  {
    "set_name": "SKILL-EXPERTISE",
    "parent_record": "SKILL",
    "child_record": "EXPERTISE",
    "cardinality": "1:N"
  },
  {
    "set_name": "EMP-EMPOSITION",
    "parent_record": "EMPLOYEE",
    "child_record": "EMPOSITION",
    "cardinality": "1:N"
  },
  {
    "set_name": "REPORTS-TO",
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "cardinality": "1:N"
  },
  {
    "set_name": "EMP-COVERAGE",
    "parent_record": "EMPLOYEE",
    "child_record": "COVERAGE",
    "cardinality": "1:N"
  },
  {
    "set_name": "JOB-EMPOSITION",
    "parent_record": "JOB",
    "child_record": "EMPOSITION",
    "cardinality": "1:N"
  },
  {
    "set_name": "COVERAGE-CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "DENTAL_CLAIM",
    "cardinality": "1:N"
  },
  {
    "set_name": "COVERAGE-CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "HOSPITAL_CLAIM",
    "cardinality": "1:N"
  }
]

```

```

    },
    {
        "set_name": "COVERAGE-CLAIMS",
        "parent_record": "COVERAGE",
        "child_record": "NON_HOSP_CLAIM",
        "cardinality": "1:N"
    }
]
}

```

```

=====
FILE: data\output\converted_db2_cobol.cb1
ENCODING: utf-8
=====

```

```

*RETRIEVAL
*DMLIST
*NO-ACTIVITY-LOG
*SCHEMA-COMMENTS
IDENTIFICATION DIVISION.
PROGRAM-ID. DEPTRPT.
AUTHOR. CA.
DATE-WRITTEN. APRIL 1995.
REMARKS. THIS PROGRAM DEMONSTRATES
CA IDMS DATABASE ACCESS USING
COBOL DML STATEMENTS. IT READS
DEPARTMENT ID NUMBERS AND RETRIEVES
RELATED RECORD OCCURRENCES,
PRINTING A REPORT THAT INCLUDES
DEPARTMENT, EMPLOYEE, JOB, AND
OFFICE INFORMATION.
*****
ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
SELECT DEPT-FILE-IN ASSIGN TO INFILE.
SELECT DEPT-FILE-OUT ASSIGN TO OUTFILE.
SELECT ERR-FILE-OUT ASSIGN TO ERRFILE.
*****

DATA DIVISION.

FILE SECTION.
FD DEPT-FILE-IN
RECORD CONTAINS 80
BLOCK CONTAINS 80 CHARACTERS
RECORDING MODE IS F
LABEL RECORDS ARE OMITTED.
01 DEPT-REC-IN.
02 DEPT-ID-IN PIC 9(4).
02 DEPT-IN-FILLER PIC X(76).
FD DEPT-FILE-OUT
RECORD CONTAINS 133
BLOCK CONTAINS 133 CHARACTERS
RECORDING MODE IS F
LABEL RECORDS ARE OMITTED.
01 DEPT-REC-OUT.
02 CC PIC X.
02 PRINT-LINE PIC X(132).
FD ERR-FILE-OUT
RECORD CONTAINS 133
BLOCK CONTAINS 133 CHARACTERS
RECORDING MODE IS F
LABEL RECORDS ARE OMITTED.
01 ERR-REC-OUT.
02 ERR-CC PIC X.
02 ERR-LINE PIC X(132).
*****
WORKING-STORAGE SECTION.
01 EOF-SW PIC X VALUE 'N'.
88 END-OF-FILE VALUE 'Y'.
01 LINE-COUNT PIC 99 VALUE 0.
01 ERR-LINE-COUNT PIC 99 VALUE 0.
01 LINE-MAX PIC 99 VALUE 50.
*****

```

```

01 DEPT-HEADER.
05 FILLER PIC X(30) VALUE SPACES.
05 FILLER PIC X(13) VALUE 'DEPARTMENT ID'.
05 FILLER PIC X(10) VALUE SPACES.
05 FILLER PIC X(9) VALUE 'DEPT NAME'.
05 FILLER PIC X(70) VALUE SPACES.
01 DEPT-DETAIL-LINE.
05 FILLER PIC X(33) VALUE SPACES.
05 DEPT-ID-OUT PIC X(4).
05 FILLER PIC X(16) VALUE SPACES.
05 DEPT-NAME-OUT PIC X(45).
05 FILLER PIC X(34) VALUE SPACES.
01 EMP-HEADER.
05 FILLER PIC X(5) VALUE SPACES.
05 FILLER PIC X(6) VALUE 'EMP ID'.
05 FILLER PIC X(2) VALUE SPACES.
05 FILLER PIC X(9) VALUE 'LAST NAME'.
05 FILLER PIC X(8) VALUE SPACES.
05 FILLER PIC X(10) VALUE 'FIRST NAME'.
05 FILLER PIC X(3) VALUE SPACES.
05 FILLER PIC X(10) VALUE 'START DATE'.
05 FILLER PIC X(2) VALUE SPACES.
05 FILLER PIC X(9) VALUE 'JOB TITLE'.
05 FILLER PIC X(13) VALUE SPACES.
05 FILLER PIC X(14) VALUE 'OFFICE ADDRESS'.
05 FILLER PIC X(42) VALUE SPACES.
01 EMP-DETAIL-LINE.
05 FILLER PIC X(5) VALUE SPACES.
05 ID-OUT PIC X(4).
05 FILLER PIC X(4) VALUE SPACES.
05 LAST-OUT PIC X(15).
05 FILLER PIC X(2) VALUE SPACES.
05 FIRST-OUT PIC X(10).
05 FILLER PIC X(3) VALUE SPACES.
05 SD-OUT.
10 SD-MM PIC XX.
10 FILLER PIC X VALUE '/'.
10 SD-DD PIC XX.
10 FILLER PIC X VALUE '/'.
10 SD-YY PIC XX.
05 FILLER PIC X(4) VALUE SPACES.
05 TITLE-OUT PIC X(20).
05 FILLER PIC X(2) VALUE SPACES.
05 OFF-ADDRESS-OUT.
10 STREET-OUT PIC X(20).
10 FILLER PIC XX VALUE SPACES.
10 CITY-OUT PIC X(15).
10 FILLER PIC XX VALUE SPACES.
10 STATE-OUT PIC XX.
10 FILLER PIC XX VALUE SPACES.
10 ZIP-OUT PIC X(5).
05 FILLER PIC X(8) VALUE SPACES.
01 ERR-HEADER-1.
05 FILLER PIC X(40) VALUE SPACES.
05 FILLER PIC X(12) VALUE 'ERROR REPORT'.
05 FILLER PIC X(80) VALUE SPACES.
01 ERR-HEADER-2.
05 FILLER PIC X(10) VALUE SPACES.
05 FILLER PIC X(4) VALUE '*** '.
05 FILLER PIC X(51)
VALUE 'THIS REPORT LISTS EMPTY AND NONEXISTENT DEPARTMENTS'.
05 FILLER PIC X(4) VALUE ' ***'.
05 FILLER PIC X(63) VALUE SPACES.
01 ERR-HEADER-3.
05 FILLER PIC X(20) VALUE SPACES.
05 FILLER PIC X(7) VALUE 'DEPT ID'.
05 FILLER PIC X(9) VALUE SPACES.
05 FILLER PIC X(7) VALUE 'MESSAGE'.
05 FILLER PIC X(89) VALUE SPACES.
01 ERR-DETAIL-LINE.
05 FILLER PIC X(20) VALUE SPACES.
05 ERR-ID-OUT PIC X(4).
05 FILLER PIC X(12) VALUE SPACES.
05 ERR-MESS-OUT PIC X(15).
05 FILLER PIC X(79) VALUE SPACES.
*****

```

```

01 MESSAGES.
05 NO-JOB-MESSAGE.
10 FILLER PIC X(20)
VALUE 'NO JOB ASSIGNED'.
05 NO-OFFICE-MESSAGE.
10 FILLER PIC X(20)
VALUE 'NO OFFICE ASSIGNED'.
05 NO-DEPT-MESSAGE.
10 FILLER PIC X(15)
VALUE 'DOES NOT EXIST'.
05 NO-EMP-MESSAGE.
10 FILLER PIC X(15)
VALUE 'IS EMPTY'.
05 NO-INPUT-MESSAGE.
10 FILLER PIC XX VALUE SPACES.
10 FILLER PIC X(11)
VALUE '=====> '.
10 FILLER PIC X(8)
VALUE 'NO INPUT'.
10 FILLER PIC X(11)
VALUE ' <=====' .
10 FILLER PIC X(100)
VALUE SPACES.

```

EJECT

```

*****
* DB2 SQLCA, HOST VARIABLES, AND CURSORS
*****

```

```

EXEC SQL
    INCLUDE SQLCA
END-EXEC.
EXEC SQL
    BEGIN DECLARE SECTION
END-EXEC.

```

01 HV-DEPARTMENT.

```

05 HV-DEPARTMENT-DEPT-ID-0410          PIC S9(4) COMP.
05 HV-DEPARTMENT-DEPT-NAME-0410        PIC X(45).
05 HV-DEPARTMENT-DEPT-HEAD-ID-0410     PIC S9(4) COMP.

```

01 HV-EMPLOYEE.

```

05 HV-EMPLOYEE-START-DATE-0415         PIC X(10).
05 HV-EMPLOYEE-TERMINATION-DATE-0415   PIC X(10).
05 HV-EMPLOYEE-BIRTH-DATE-0415        PIC X(10).
05 HV-EMPLOYEE-EMP-ID-0415             PIC S9(4) COMP.
05 HV-EMPLOYEE-EMP-FIRST-NAME-0415     PIC X(10).
05 HV-EMPLOYEE-EMP-LAST-NAME-0415      PIC X(15).
05 HV-EMPLOYEE-EMP-STREET-0415         PIC X(20).
05 HV-EMPLOYEE-EMP-CITY-0415           PIC X(15).
05 HV-EMPLOYEE-EMP-STATE-0415          PIC X(2).
05 HV-EMPLOYEE-EMP-ZIP-FIRST-FIVE-0415 PIC X(5).
05 HV-EMPLOYEE-EMP-ZIP-LAST-FOUR-0415  PIC X(4).
05 HV-EMPLOYEE-EMP-PHONE-0415          PIC S9(4) COMP.
05 HV-EMPLOYEE-STATUS-0415             PIC X(2).
05 HV-EMPLOYEE-SS-NUMBER-0415          PIC S9(4) COMP.
05 HV-EMPLOYEE-DEPT-ID-0410            PIC S9(4) COMP.
05 HV-EMPLOYEE-OFFICE-CODE-0450        PIC X(3).

```

01 HV-EMPOSITION.

```

05 HV-EMPOSITION-START-DATE-0420       PIC X(10).
05 HV-EMPOSITION-FINISH-DATE-0420      PIC X(10).
05 HV-EMPOSITION-SALARY-GRADE-0420     PIC S9(4) COMP.
05 HV-EMPOSITION-SALARY-AMOUNT-0420    PIC S9(1)V9(2) COMP-3.
05 HV-EMPOSITION-BONUS-PERCENT-0420    PIC S9(1)V9(3) COMP-3.
05 HV-EMPOSITION-COMMISSION-PERCENT-0420 PIC S9(1)V9(3) COMP-3.
05 HV-EMPOSITION-OVERTIME-RATE-0420    PIC S9(1)V9(2) COMP-3.
05 HV-EMPOSITION-EMP-ID-0415           PIC S9(4) COMP.
05 HV-EMPOSITION-JOB-ID-0440           PIC S9(4) COMP.

```

01 HV-JOB.

```

05 HV-JOB-JOB-ID-0440                  PIC S9(4) COMP.
05 HV-JOB-TITLE-0440                   PIC X(20).
05 HV-JOB-DESCRIPTION-LINE-0440        PIC X(120).
05 HV-JOB-REQUIREMENT-LINE-0440       PIC X(120).
05 HV-JOB-MINIMUM-SALARY-0440          PIC S9(1)V9(2) COMP-3.
05 HV-JOB-MAXIMUM-SALARY-0440          PIC S9(1)V9(2) COMP-3.

```

05 HV-JOB-SALARY-GRADES-0440	PIC S9(4) COMP.
05 HV-JOB-NUMBER-OF-POSITIONS-0440	PIC S9(4) COMP.
05 HV-JOB-NUMBER-OPEN-0440	PIC S9(4) COMP.
01 HV-OFFICE.	
05 HV-OFFICE-OFFICE-CODE-0450	PIC X(3).
05 HV-OFFICE-OFFICE-STREET-0450	PIC X(20).
05 HV-OFFICE-OFFICE-CITY-0450	PIC X(15).
05 HV-OFFICE-OFFICE-STATE-0450	PIC X(2).
05 HV-OFFICE-OFFICE-ZIP-FIRST-FIVE-0450	PIC X(5).
05 HV-OFFICE-OFFICE-ZIP-LAST-FOUR-0450	PIC X(4).
05 HV-OFFICE-OFFICE-PHONE-0450	PIC S9(4) COMP.
05 HV-OFFICE-OFFICE-AREA-CODE-0450	PIC X(3).
05 HV-OFFICE-SPEED-DIAL-0450	PIC X(3).
01 NI-DEPARTMENT-DEPT-NAME-0410	PIC S9(4) COMP.
01 NI-DEPARTMENT-DEPT-HEAD-ID-0410	PIC S9(4) COMP.
01 NI-EMPLOYEE-TERMINATION-DATE-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-BIRTH-DATE-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-FIRST-NAME-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-LAST-NAME-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-STREET-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-CITY-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-STATE-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-ZIP-FIRST-FIVE-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-ZIP-LAST-FOUR-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-PHONE-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-STATUS-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-SS-NUMBER-0415	PIC S9(4) COMP.
01 NI-EMPLOYEE-DEPT-ID-0410	PIC S9(4) COMP.
01 NI-EMPLOYEE-OFFICE-CODE-0450	PIC S9(4) COMP.
01 NI-EMPOSITION-FINISH-DATE-0420	PIC S9(4) COMP.
01 NI-EMPOSITION-SALARY-GRADE-0420	PIC S9(4) COMP.
01 NI-EMPOSITION-SALARY-AMOUNT-0420	PIC S9(4) COMP.
01 NI-EMPOSITION-BONUS-PERCENT-0420	PIC S9(4) COMP.
01 NI-EMPOSITION-COMMISSION-PERCENT-0420	PIC S9(4) COMP.
01 NI-EMPOSITION-OVERTIME-RATE-0420	PIC S9(4) COMP.
01 NI-EMPOSITION-EMP-ID-0415	PIC S9(4) COMP.
01 NI-EMPOSITION-JOB-ID-0440	PIC S9(4) COMP.
01 NI-JOB-TITLE-0440	PIC S9(4) COMP.
01 NI-JOB-DESCRIPTION-LINE-0440	PIC S9(4) COMP.
01 NI-JOB-REQUIREMENT-LINE-0440	PIC S9(4) COMP.
01 NI-JOB-MINIMUM-SALARY-0440	PIC S9(4) COMP.
01 NI-JOB-MAXIMUM-SALARY-0440	PIC S9(4) COMP.
01 NI-JOB-SALARY-GRADES-0440	PIC S9(4) COMP.
01 NI-JOB-NUMBER-OF-POSITIONS-0440	PIC S9(4) COMP.
01 NI-JOB-NUMBER-OPEN-0440	PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-STREET-0450	PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-CITY-0450	PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-STATE-0450	PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-ZIP-FIRST-FIVE-0450	PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-ZIP-LAST-FOUR-0450	PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-PHONE-0450	PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-AREA-CODE-0450	PIC S9(4) COMP.
01 NI-OFFICE-SPEED-DIAL-0450	PIC S9(4) COMP.
EXEC SQL	
END DECLARE SECTION	
END-EXEC.	
EXEC SQL	
DECLARE C-DEPT-EMPLOYEE CURSOR FOR	
SELECT START_DATE_0415,	
TERMINATION_DATE_0415,	
BIRTH_DATE_0415,	
EMP_ID_0415,	
EMP_FIRST_NAME_0415,	
EMP_LAST_NAME_0415,	
EMP_STREET_0415,	
EMP_CITY_0415,	
EMP_STATE_0415,	
EMP_ZIP_FIRST_FIVE_0415,	
EMP_ZIP_LAST_FOUR_0415,	
EMP_PHONE_0415,	
STATUS_0415,	
SS_NUMBER_0415,	
DEPT_ID_0410,	
OFFICE_CODE_0450	

```

        FROM EMPLOYEE
        WHERE DEPT_ID_0410 = :HV-DEPARTMENT-DEPT-ID-0410
        ORDER BY DEPT_ID_0410
    END-EXEC.

PROCEDURE DIVISION.
*****
* PROCEDURE DIVISION GENERAL STRATEGY: *
* *
* 1) READ DEPT-ID-IN, WHICH CONTAINS THE *
* DEPT-ID NUMBER *
* *
* 2) ACCESS THE DATABASE USING THE DEPT-ID NUMBER *
* WITH AN OBTAIN CALC ON THE DEPARTMENT RECORD *
* *
* 3) ACCESS ALL EMPLOYEES IN THE DEPT-EMPLOYEE SET *
* AND RETRIEVE RELATED JOB AND OFFICE DATA *
* *
* 4) PRINT A REPORT FOR EACH DEPARTMENT *
* *
* 5) PRINT AN ERROR REPORT FOR EMPTY DEPARTMENTS *
* AND NONEXISTENT DEPARTMENTS (NO MATCHING *
* DEPT-ID) *
*****
FETCH-DEPT-EMPLOYEE.
    EXEC SQL
        FETCH C-DEPT-EMPLOYEE
        INTO :HV-EMPLOYEE-START-DATE-0415,
            :HV-EMPLOYEE-TERMINATION-DATE-0415 :NI-EMPLOYEE-TERMINATION-DATE-0415,
            :HV-EMPLOYEE-BIRTH-DATE-0415 :NI-EMPLOYEE-BIRTH-DATE-0415,
            :HV-EMPLOYEE-EMP-ID-0415,
            :HV-EMPLOYEE-EMP-FIRST-NAME-0415 :NI-EMPLOYEE-EMP-FIRST-NAME-0415,
            :HV-EMPLOYEE-EMP-LAST-NAME-0415 :NI-EMPLOYEE-EMP-LAST-NAME-0415,
            :HV-EMPLOYEE-EMP-STREET-0415 :NI-EMPLOYEE-EMP-STREET-0415,
            :HV-EMPLOYEE-EMP-CITY-0415 :NI-EMPLOYEE-EMP-CITY-0415,
            :HV-EMPLOYEE-EMP-STATE-0415 :NI-EMPLOYEE-EMP-STATE-0415,
            :HV-EMPLOYEE-EMP-ZIP-FIRST-FIVE-0415 :NI-EMPLOYEE-EMP-ZIP-FIRST-FIVE-0415,
            :HV-EMPLOYEE-EMP-ZIP-LAST-FOUR-0415 :NI-EMPLOYEE-EMP-ZIP-LAST-FOUR-0415,
            :HV-EMPLOYEE-EMP-PHONE-0415 :NI-EMPLOYEE-EMP-PHONE-0415,
            :HV-EMPLOYEE-STATUS-0415 :NI-EMPLOYEE-STATUS-0415,
            :HV-EMPLOYEE-SS-NUMBER-0415 :NI-EMPLOYEE-SS-NUMBER-0415,
            :HV-EMPLOYEE-DEPT-ID-0410 :NI-EMPLOYEE-DEPT-ID-0410,
            :HV-EMPLOYEE-OFFICE-CODE-0450 :NI-EMPLOYEE-OFFICE-CODE-0450
    END-EXEC.
    IF SQLCODE NOT = 0 AND SQLCODE NOT = 100
        PERFORM SQL-ERROR
    END-IF.

MAIN-LINE.
    PERFORM INIT-FILES.
    IF END-OF-FILE
        PERFORM EMPTY-INPUT-PROCESSING
    ELSE
        PERFORM INIT-DB2-SETUP
        PERFORM U220-ERR-HEADER
        PERFORM DEPT-PROCESSING THRU DEPT-PROCESSING-EXIT
UNTIL END-OF-FILE.
    PERFORM END-PROCESSING.
    GOBACK.

INIT-DB2-SETUP.
    CONTINUE.

INIT-FILES.
    OPEN INPUT DEPT-FILE-IN.
    OPEN OUTPUT DEPT-FILE-OUT.
    OPEN OUTPUT ERR-FILE-OUT.
    MOVE SPACES TO PRINT-LINE.
    MOVE SPACES TO ERR-LINE.
    READ DEPT-FILE-IN
        AT END MOVE 'Y' TO EOF-SW.
EMPTY-INPUT-PROCESSING.
    MOVE NO-INPUT-MESSAGE TO PRINT-LINE.
    MOVE '1' TO CC.
    PERFORM U000-WRITE-LINE.
*****
* THIS PARAGRAPH ACCESSES THE DATABASE USING THE DEPT-ID-0415 *
* CALCKEY VALUE. *
*****

```

```

DEPT-PROCESSING.
  MOVE DEPT-ID-IN TO HV-DEPARTMENT-DEPT-ID-0410.
  EXEC SQL
    SELECT DEPT_ID_0410,
           DEPT_NAME_0410,
           DEPT_HEAD_ID_0410
  INTO :HV-DEPARTMENT-DEPT-ID-0410,
       :HV-DEPARTMENT-DEPT-NAME-0410 :NI-DEPARTMENT-DEPT-NAME-0410,
       :HV-DEPARTMENT-DEPT-HEAD-ID-0410 :NI-DEPARTMENT-DEPT-HEAD-ID-0410
  FROM DEPARTMENT
  WHERE DEPT_ID_0410 = :HV-DEPARTMENT-DEPT-ID-0410
END-EXEC.
IF SQLCODE = 100
  PERFORM NO-DEPT-PROCESSING
ELSE
  IF SQLCODE NOT = 0
    PERFORM SQL-ERROR
  ELSE
    EXEC SQL
      OPEN C-DEPT-EMPLOYEE
    END-EXEC.
  IF SQLCODE NOT = 0
    PERFORM SQL-ERROR
  ELSE
    PERFORM FETCH-DEPT-EMPLOYEE

  IF SQLCODE = 100
    PERFORM EMPTY-SET
  ELSE
    PERFORM U020-VALID-HEADER
    MOVE HV-DEPARTMENT-DEPT-ID-0410 TO DEPT-ID-OUT
    MOVE HV-DEPARTMENT-DEPT-NAME-0410 TO DEPT-NAME-OUT
    MOVE DEPT-DETAIL-LINE TO PRINT-LINE
    PERFORM U000-WRITE-LINE
    PERFORM U030-EMP-HEADERS

  PERFORM UNTIL SQLCODE = 100
    PERFORM SET-WALK THRU SET-WALK-EXIT
    PERFORM FETCH-DEPT-EMPLOYEE
  END-PERFORM
END-IF.
EXEC SQL
  CLOSE C-DEPT-EMPLOYEE
END-EXEC.
END-IF.
END-IF.
END-IF.
READ DEPT-FILE-IN
  AT END MOVE 'Y' TO EOF-SW.
DEPT-PROCESSING-EXIT.
EXIT.
*****
* THIS PARAGRAPH RETRIEVES EMPLOYEE, JOB, AND OFFICE DATA *
* FOR EACH EMPLOYEE IN THE DEPT-EMPLOYEE SET. *
*****
SET-WALK.
  CONTINUE.
  MOVE HV-EMPLOYEE-EMP-ID-0415 TO ID-OUT.
  MOVE HV-EMPLOYEE-EMP-LAST-NAME-0415 TO LAST-OUT.
  MOVE HV-EMPLOYEE-EMP-FIRST-NAME-0415 TO FIRST-OUT.
  MOVE HV-EMPLOYEE-START-DATE-0415(3:2) TO SD-YY.
  MOVE HV-EMPLOYEE-START-DATE-0415(6:2) TO SD-MM.
  MOVE HV-EMPLOYEE-START-DATE-0415(9:2) TO SD-DD.
  EXEC SQL
    SELECT START_DATE_0420,
           FINISH_DATE_0420,
           SALARY_GRADE_0420,
           SALARY_AMOUNT_0420,
           BONUS_PERCENT_0420,
           COMMISSION_PERCENT_0420,
           OVERTIME_RATE_0420,
           EMP_ID_0415,
           JOB_ID_0440
  INTO :HV-EMPOSITION-START-DATE-0420,
       :HV-EMPOSITION-FINISH-DATE-0420 :NI-EMPOSITION-FINISH-DATE-0420,
       :HV-EMPOSITION-SALARY-GRADE-0420 :NI-EMPOSITION-SALARY-GRADE-0420,

```

```

:HV-EMPOSITION-SALARY-AMOUNT-0420 :NI-EMPOSITION-SALARY-AMOUNT-0420,
:HV-EMPOSITION-BONUS-PERCENT-0420 :NI-EMPOSITION-BONUS-PERCENT-0420,
:HV-EMPOSITION-COMMISSION-PERCENT-0420 :NI-EMPOSITION-COMMISSION-PERCENT-0420,
:HV-EMPOSITION-OVERTIME-RATE-0420 :NI-EMPOSITION-OVERTIME-RATE-0420,
:HV-EMPOSITION-EMP-ID-0415 :NI-EMPOSITION-EMP-ID-0415,
:HV-EMPOSITION-JOB-ID-0440 :NI-EMPOSITION-JOB-ID-0440
FROM EMPOSITION
WHERE EMP_ID_0415 = :HV-EMPLOYEE-EMP-ID-0415
ORDER BY EMP_ID_0415
FETCH FIRST 1 ROW ONLY
END-EXEC.
IF SQLCODE = 100
MOVE NO-JOB-MESSAGE TO TITLE-OUT
ELSE
IF SQLCODE NOT = 0
PERFORM SQL-ERROR
ELSE
EXEC SQL
    SELECT JOB_ID_0440,
    TITLE_0440,
    DESCRIPTION_LINE_0440,
    REQUIREMENT_LINE_0440,
    MINIMUM_SALARY_0440,
    MAXIMUM_SALARY_0440,
    SALARY_GRADES_0440,
    NUMBER_OF_POSITIONS_0440,
    NUMBER_OPEN_0440
INTO :HV-JOB-JOB-ID-0440,
:HV-JOB-TITLE-0440 :NI-JOB-TITLE-0440,
:HV-JOB-DESCRIPTION-LINE-0440 :NI-JOB-DESCRIPTION-LINE-0440,
:HV-JOB-REQUIREMENT-LINE-0440 :NI-JOB-REQUIREMENT-LINE-0440,
:HV-JOB-MINIMUM-SALARY-0440 :NI-JOB-MINIMUM-SALARY-0440,
:HV-JOB-MAXIMUM-SALARY-0440 :NI-JOB-MAXIMUM-SALARY-0440,
:HV-JOB-SALARY-GRADES-0440 :NI-JOB-SALARY-GRADES-0440,
:HV-JOB-NUMBER-OF-POSITIONS-0440 :NI-JOB-NUMBER-OF-POSITIONS-0440,
:HV-JOB-NUMBER-OPEN-0440 :NI-JOB-NUMBER-OPEN-0440
FROM JOB
WHERE JOB_ID_0440 = :HV-EMPOSITION-JOB-ID-0440
END-EXEC.
IF SQLCODE = 100
MOVE NO-JOB-MESSAGE TO TITLE-OUT
ELSE
IF SQLCODE NOT = 0
PERFORM SQL-ERROR
ELSE
CONTINUE.
END-IF.
END-IF.
END-IF.
END-IF.
MOVE HV-JOB-TITLE-0440 TO TITLE-OUT.
IF NI-EMPLOYEE-OFFICE-CODE-0450 < 0
MOVE NO-OFFICE-MESSAGE TO STREET-OUT
MOVE SPACES TO CITY-OUT
MOVE SPACES TO STATE-OUT
MOVE SPACES TO ZIP-OUT
ELSE
EXEC SQL
    SELECT OFFICE_CODE_0450,
    OFFICE_STREET_0450,
    OFFICE_CITY_0450,
    OFFICE_STATE_0450,
    OFFICE_ZIP_FIRST_FIVE_0450,
    OFFICE_ZIP_LAST_FOUR_0450,
    OFFICE_PHONE_0450,
    OFFICE_AREA_CODE_0450,
    SPEED_DIAL_0450
INTO :HV-OFFICE-OFFICE-CODE-0450,
:HV-OFFICE-OFFICE-STREET-0450 :NI-OFFICE-OFFICE-STREET-0450,
:HV-OFFICE-OFFICE-CITY-0450 :NI-OFFICE-OFFICE-CITY-0450,
:HV-OFFICE-OFFICE-STATE-0450 :NI-OFFICE-OFFICE-STATE-0450,
:HV-OFFICE-OFFICE-ZIP-FIRST-FIVE-0450 :NI-OFFICE-OFFICE-ZIP-FIRST-FIVE-0450,
:HV-OFFICE-OFFICE-ZIP-LAST-FOUR-0450 :NI-OFFICE-OFFICE-ZIP-LAST-FOUR-0450,
:HV-OFFICE-OFFICE-PHONE-0450 :NI-OFFICE-OFFICE-PHONE-0450,
:HV-OFFICE-OFFICE-AREA-CODE-0450 :NI-OFFICE-OFFICE-AREA-CODE-0450,
:HV-OFFICE-SPEED-DIAL-0450 :NI-OFFICE-SPEED-DIAL-0450

```



```

        FROM OFFICE
        WHERE OFFICE_CODE_0450 = :HV-EMPLOYEE-OFFICE-CODE-0450
END-EXEC.
IF SQLCODE = 0
CONTINUE.
ELSE
MOVE NO-OFFICE-MESSAGE TO STREET-OUT
MOVE SPACES TO CITY-OUT
MOVE SPACES TO STATE-OUT
MOVE SPACES TO ZIP-OUT
END-IF.
END-IF.
MOVE HV-OFFICE-OFFICE-STREET-0450 TO STREET-OUT
MOVE HV-OFFICE-OFFICE-CITY-0450 TO CITY-OUT
MOVE HV-OFFICE-OFFICE-STATE-0450 TO STATE-OUT
MOVE HV-OFFICE-OFFICE-ZIP-FIRST-FIVE-0450 TO ZIP-OUT
MOVE EMP-DETAIL-LINE TO PRINT-LINE.
PERFORM U000-WRITE-LINE.
SET-WALK-EXIT.
EXIT.
END-PROCESSING.
CLOSE DEPT-FILE-OUT.
CLOSE ERR-FILE-OUT.
CLOSE DEPT-FILE-IN.
EMPTY-SET.
MOVE SPACES TO ERR-LINE.
MOVE HV-DEPARTMENT-DEPT-ID-0410 TO ERR-ID-OUT.
MOVE NO-EMP-MESSAGE TO ERR-MESS-OUT.
MOVE ERR-DETAIL-LINE TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.
NO-DEPT-PROCESSING.
MOVE DEPT-ID-IN TO ERR-ID-OUT.
MOVE NO-DEPT-MESSAGE TO ERR-MESS-OUT.
MOVE ERR-DETAIL-LINE TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.
U000-WRITE-LINE.
WRITE DEPT-REC-OUT AFTER POSITIONING CC.
IF CC = '1'
THEN MOVE 0 TO LINE-COUNT
ELSE IF CC = ' '
THEN ADD 1 TO LINE-COUNT
ELSE IF CC = '0'
THEN ADD 2 TO LINE-COUNT.
IF LINE-COUNT > LINE-MAX
THEN PERFORM U010-NEW-PAGE-ROUTINE.
U010-NEW-PAGE-ROUTINE.
PERFORM U020-VALID-HEADER.
MOVE DEPT-DETAIL-LINE TO PRINT-LINE.
PERFORM U000-WRITE-LINE.
PERFORM U030-EMP-HEADERS.
U020-VALID-HEADER.
MOVE DEPT-HEADER TO PRINT-LINE.
MOVE '1' TO CC.
PERFORM U000-WRITE-LINE.
MOVE ' ' TO CC.
U030-EMP-HEADERS.
MOVE '0' TO CC.
MOVE EMP-HEADER TO PRINT-LINE.
PERFORM U000-WRITE-LINE.
MOVE SPACES TO PRINT-LINE.
MOVE ' ' TO CC.
PERFORM U000-WRITE-LINE.
U200-WRITE-ERR-LINE.
WRITE ERR-REC-OUT AFTER POSITIONING ERR-CC.
IF ERR-CC = '1'
THEN MOVE 0 TO ERR-LINE-COUNT
ELSE IF ERR-CC = ' '
THEN ADD 1 TO ERR-LINE-COUNT
ELSE IF ERR-CC = '0'
THEN ADD 2 TO ERR-LINE-COUNT.
IF ERR-LINE-COUNT > LINE-MAX
THEN
PERFORM U220-ERR-HEADER.
U220-ERR-HEADER.
MOVE ERR-HEADER-1 TO ERR-LINE.
MOVE '1' TO ERR-CC.

```

```

PERFORM U200-WRITE-ERR-LINE.
MOVE '0' TO ERR-CC.
MOVE ERR-HEADER-2 TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.
MOVE ERR-HEADER-3 TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.
MOVE SPACES TO ERR-LINE.
MOVE ' ' TO ERR-CC.
PERFORM U200-WRITE-ERR-LINE.

```

SQL-ERROR.

```

MOVE SPACES TO ERR-LINE.
MOVE 'SQL ERROR' TO ERR-MESS-OUT.
MOVE ERR-DETAIL-LINE TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.

```

SQL-ERROR-EXIT.

EXIT.

```

=====
FILE: data\output\db2_ddl.sql
ENCODING: utf-8
=====

```

CREATE TABLE COVERAGE

```

(
    COVERAGE_ID BIGINT NOT NULL,
    SELECTION_DATE_0400 DATE,
    TERMINATION_DATE_0400 DATE,
    TYPE_0400 CHAR(1),
    INS_PLAN_CODE_0400 VARCHAR(3),
    EMP_ID_0415 SMALLINT,
    PRIMARY KEY (COVERAGE_ID)
);

```

CREATE TABLE DENTAL_CLAIM

```

(
    CLAIM_DATE_0405 DATE,
    PATIENT_BIRTH_DATE_0405 DATE,
    SERVICE_DATE_0405 DATE,
    PATIENT_FIRST_NAME_0405 VARCHAR(10),
    PATIENT_LAST_NAME_0405 VARCHAR(15),
    PATIENT_SEX_0405 CHAR(1),
    RELATION_TO_EMPLOYEE_0405 VARCHAR(10),
    DENTIST_FIRST_NAME_0405 VARCHAR(10),
    DENTIST_LAST_NAME_0405 VARCHAR(15),
    DENTIST_STREET_0405 VARCHAR(20),
    DENTIST_CITY_0405 VARCHAR(15),
    DENTIST_STATE_0405 VARCHAR(2),
    DENTIST_ZIP_FIRST_FIVE_0405 VARCHAR(5),
    DENTIST_ZIP_LAST_FOUR_0405 VARCHAR(4),
    DENTIST_LICENSE_NUMBER_0405 SMALLINT,
    NUMBER_OF_PROCEDURES_0405 DECIMAL(1,0),
    TOOTH_NUMBER_0405 SMALLINT,
    PROCEDURE_CODE_0405 SMALLINT,
    DESCRIPTION_OF_SERVICE_0405 VARCHAR(60),
    FEE_0405 DECIMAL(2,2),
    COVERAGE_ID BIGINT
);

```

CREATE TABLE DEPARTMENT

```

(
    DEPT_ID_0410 SMALLINT NOT NULL,
    DEPT_NAME_0410 VARCHAR(45),
    DEPT_HEAD_ID_0410 SMALLINT,
    PRIMARY KEY (DEPT_ID_0410)
);

```

CREATE TABLE EMPLOYEE

```

(
    START_DATE_0415 DATE,
    TERMINATION_DATE_0415 DATE,
    BIRTH_DATE_0415 DATE,
    EMP_ID_0415 SMALLINT NOT NULL,
    EMP_FIRST_NAME_0415 VARCHAR(10),

```

```

EMP_LAST_NAME_0415 VARCHAR(15),
EMP_STREET_0415 VARCHAR(20),
EMP_CITY_0415 VARCHAR(15),
EMP_STATE_0415 VARCHAR(2),
EMP_ZIP_FIRST_FIVE_0415 VARCHAR(5),
EMP_ZIP_LAST_FOUR_0415 VARCHAR(4),
EMP_PHONE_0415 SMALLINT,
STATUS_0415 VARCHAR(2),
SS_NUMBER_0415 SMALLINT,
DEPT_ID_0410 SMALLINT,
OFFICE_CODE_0450 VARCHAR(3),
PRIMARY KEY (EMP_ID_0415)
);

CREATE TABLE EMPOSITION
(
    START_DATE_0420 DATE,
    FINISH_DATE_0420 DATE,
    SALARY_GRADE_0420 SMALLINT,
    SALARY_AMOUNT_0420 DECIMAL(2,2),
    BONUS_PERCENT_0420 DECIMAL(3,3),
    COMMISSION_PERCENT_0420 DECIMAL(3,3),
    OVERTIME_RATE_0420 DECIMAL(3,2),
    EMP_ID_0415 SMALLINT,
    JOB_ID_0440 SMALLINT
);

CREATE TABLE EXPERTISE
(
    EXPERTISE_DATE_0425 DATE,
    SKILL_LEVEL_0425 VARCHAR(2),
    EMP_ID_0415 SMALLINT,
    SKILL_ID_0455 SMALLINT
);

CREATE TABLE HOSPITAL_CLAIM
(
    CLAIM_DATE_0430 DATE,
    PATIENT_BIRTH_DATE_0430 DATE,
    ADMIT_DATE_0430 DATE,
    DISCHARGE_DATE_0430 DATE,
    PATIENT_FIRST_NAME_0430 VARCHAR(10),
    PATIENT_LAST_NAME_0430 VARCHAR(15),
    PATIENT_SEX_0430 CHAR(1),
    RELATION_TO_EMPLOYEE_0430 VARCHAR(10),
    HOSPITAL_NAME_0430 VARCHAR(25),
    HOSP_STREET_0430 VARCHAR(20),
    HOSP_CITY_0430 VARCHAR(15),
    HOSP_STATE_0430 VARCHAR(2),
    HOSP_ZIP_FIRST_FIVE_0430 VARCHAR(5),
    HOSP_ZIP_LAST_FOUR_0430 VARCHAR(4),
    DIAGNOSIS_0430 VARCHAR(120),
    WARD_DAYS_0430 DECIMAL(5,0),
    WARD_RATE_0430 DECIMAL(2,2),
    WARD_TOTAL_0430 DECIMAL(2,2),
    SEMI_DAYS_0430 DECIMAL(5,0),
    SEMI_RATE_0430 DECIMAL(2,2),
    SEMI_TOTAL_0430 DECIMAL(2,2),
    DELIVERY_COST_0430 DECIMAL(2,2),
    ANESTHESIA_COST_0430 DECIMAL(2,2),
    LAB_COST_0430 DECIMAL(2,2),
    COVERAGE_ID BIGINT
);

CREATE TABLE INSURANCE_PLAN
(
    INS_PLAN_CODE_0435 VARCHAR(3) NOT NULL,
    INS_CO_NAME_0435 VARCHAR(45),
    INS_CO_STREET_0435 VARCHAR(20),
    INS_CO_CITY_0435 VARCHAR(15),
    INS_CO_STATE_0435 VARCHAR(2),
    INS_CO_ZIP_FIRST_FIVE_0435 VARCHAR(5),
    INS_CO_ZIP_LAST_FOUR_0435 VARCHAR(4),
    INS_CO_PHONE_0435 SMALLINT,
    GROUP_NUMBER_0435 SMALLINT,
    DEDUCT_0435 DECIMAL(2,2),

```

```

        MAXIMUM_LIFE_COST_0435 DECIMAL(2,2),
        FAMILY_COST_0435 DECIMAL(2,2),
        DEP_COST_0435 DECIMAL(2,2),
        PRIMARY KEY (INS_PLAN_CODE_0435)
    );

CREATE TABLE JOB
(
    JOB_ID_0440 SMALLINT NOT NULL,
    TITLE_0440 VARCHAR(20),
    DESCRIPTION_LINE_0440 VARCHAR(120),
    REQUIREMENT_LINE_0440 VARCHAR(120),
    MINIMUM_SALARY_0440 DECIMAL(2,2),
    MAXIMUM_SALARY_0440 DECIMAL(2,2),
    SALARY_GRADES_0440 SMALLINT,
    NUMBER_OF_POSITIONS_0440 SMALLINT,
    NUMBER_OPEN_0440 SMALLINT,
    PRIMARY KEY (JOB_ID_0440)
);

CREATE TABLE NON_HOSP_CLAIM
(
    CLAIM_DATE_0445 DATE,
    PATIENT_BIRTH_DATE_0445 DATE,
    SERVICE_DATE_0445 DATE,
    PATIENT_FIRST_NAME_0445 VARCHAR(10),
    PATIENT_LAST_NAME_0445 VARCHAR(15),
    PATIENT_SEX_0445 CHAR(1),
    RELATION_TO_EMPLOYEE_0445 VARCHAR(10),
    PHYSICIAN_FIRST_NAME_0445 VARCHAR(10),
    PHYSICIAN_LAST_NAME_0445 VARCHAR(15),
    PHYSICIAN_STREET_0445 VARCHAR(20),
    PHYSICIAN_CITY_0445 VARCHAR(15),
    PHYSICIAN_STATE_0445 VARCHAR(2),
    PHYSICIAN_ZIP_FIRST_FIVE_0445 VARCHAR(5),
    PHYSICIAN_ZIP_LAST_FOUR_0445 VARCHAR(4),
    PHYSICIAN_ID_0445 SMALLINT,
    DIAGNOSIS_0445 VARCHAR(120),
    NUMBER_OF_PROCEDURES_0445 DECIMAL(1,0),
    PROCEDURE_CODE_0445 SMALLINT,
    DESCRIPTION_OF_SERVICE_0445 VARCHAR(60),
    FEE_0445 DECIMAL(2,2),
    COVERAGE_ID BIGINT
);

CREATE TABLE OFFICE
(
    OFFICE_CODE_0450 VARCHAR(3) NOT NULL,
    OFFICE_STREET_0450 VARCHAR(20),
    OFFICE_CITY_0450 VARCHAR(15),
    OFFICE_STATE_0450 VARCHAR(2),
    OFFICE_ZIP_FIRST_FIVE_0450 VARCHAR(5),
    OFFICE_ZIP_LAST_FOUR_0450 VARCHAR(4),
    OFFICE_PHONE_0450 SMALLINT,
    OFFICE_AREA_CODE_0450 VARCHAR(3),
    SPEED_DIAL_0450 VARCHAR(3),
    PRIMARY KEY (OFFICE_CODE_0450)
);

CREATE TABLE SKILL
(
    SKILL_ID_0455 SMALLINT NOT NULL,
    SKILL_NAME_0455 VARCHAR(12),
    SKILL_DESCRIPTION_0455 VARCHAR(60),
    PRIMARY KEY (SKILL_ID_0455)
);

CREATE TABLE STRUCTURE
(
    STRUCTURE_DATE_0460 DATE,
    STRUCTURE_CODE_0460 VARCHAR(2),
    MANAGES_EMP_ID_0415 SMALLINT,
    REPORTS_TO_EMP_ID_0415 SMALLINT
);

```

```

ALTER TABLE COVERAGE ADD FOREIGN KEY (EMP_ID_0415) REFERENCES EMPLOYEE (EMP_ID_0415);
ALTER TABLE COVERAGE ADD FOREIGN KEY (INS_PLAN_CODE_0400) REFERENCES INSURANCE_PLAN (INS_PLAN_CODE_0435);
ALTER TABLE DENTAL_CLAIM ADD FOREIGN KEY (COVERAGE_ID) REFERENCES COVERAGE (COVERAGE_ID);
ALTER TABLE EMPLOYEE ADD FOREIGN KEY (DEPT_ID_0410) REFERENCES DEPARTMENT (DEPT_ID_0410);
ALTER TABLE EMPLOYEE ADD FOREIGN KEY (OFFICE_CODE_0450) REFERENCES OFFICE (OFFICE_CODE_0450);
ALTER TABLE EMPOSITION ADD FOREIGN KEY (EMP_ID_0415) REFERENCES EMPLOYEE (EMP_ID_0415);
ALTER TABLE EMPOSITION ADD FOREIGN KEY (JOB_ID_0440) REFERENCES JOB (JOB_ID_0440);
ALTER TABLE EXPERTISE ADD FOREIGN KEY (EMP_ID_0415) REFERENCES EMPLOYEE (EMP_ID_0415);
ALTER TABLE EXPERTISE ADD FOREIGN KEY (SKILL_ID_0455) REFERENCES SKILL (SKILL_ID_0455);
ALTER TABLE HOSPITAL_CLAIM ADD FOREIGN KEY (COVERAGE_ID) REFERENCES COVERAGE (COVERAGE_ID);
ALTER TABLE NON_HOSP_CLAIM ADD FOREIGN KEY (COVERAGE_ID) REFERENCES COVERAGE (COVERAGE_ID);
ALTER TABLE STRUCTURE ADD FOREIGN KEY (MANAGES_EMP_ID_0415) REFERENCES EMPLOYEE (EMP_ID_0415);
ALTER TABLE STRUCTURE ADD FOREIGN KEY (REPORTS_TO_EMP_ID_0415) REFERENCES EMPLOYEE (EMP_ID_0415);

```

```

=====
FILE: data\output\phase2_metadata.json
ENCODING: utf-8
=====

```

```

{
  "version": "1.0",
  "generated_at": "2026-07-23T11:24:59.689661Z",
  "source": "PHASE1",
  "schema_source": "PHASE1_CANONICAL_DB2_METADATA",
  "record_table_map": {
    "COVERAGE": "COVERAGE",
    "DENTAL_CLAIM": "DENTAL_CLAIM",
    "DEPARTMENT": "DEPARTMENT",
    "EMPLOYEE": "EMPLOYEE",
    "EMPOSITION": "EMPOSITION",
    "EXPERTISE": "EXPERTISE",
    "HOSPITAL_CLAIM": "HOSPITAL_CLAIM",
    "INSURANCE_PLAN": "INSURANCE_PLAN",
    "JOB": "JOB",
    "NON_HOSP_CLAIM": "NON_HOSP_CLAIM",
    "OFFICE": "OFFICE",
    "SKILL": "SKILL",
    "STRUCTURE": "STRUCTURE"
  },
  "records": [
    {
      "name": "COVERAGE",
      "table": "COVERAGE",
      "primary_key": "COVERAGE_ID",
      "fields": [
        {
          "name": "COVERAGE_ID",
          "column": "COVERAGE_ID",
          "datatype": "BIGINT",
          "length": 18,
          "scale": 0,
          "nullable": false,
          "primary_key": true
        }
      ],
      {
        "name": "SELECTION_DATE_0400",
        "column": "SELECTION_DATE_0400",
        "datatype": "DATE",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
      },
      {
        "name": "TERMINATION_DATE_0400",
        "column": "TERMINATION_DATE_0400",
        "datatype": "DATE",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
      },
      {
        "name": "TYPE_0400",
        "column": "TYPE_0400",

```

```

        "datatype": "CHAR(1)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "INS_PLAN_CODE_0400",
        "column": "INS_PLAN_CODE_0400",
        "datatype": "VARCHAR(3)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "EMP_ID_0415",
        "column": "EMP_ID_0415",
        "datatype": "SMALLINT",
        "length": 4,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    }
]
},
{
    "name": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "primary_key": null,
    "fields": [
        {
            "name": "CLAIM_DATE_0405",
            "column": "CLAIM_DATE_0405",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "PATIENT_BIRTH_DATE_0405",
            "column": "PATIENT_BIRTH_DATE_0405",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "SERVICE_DATE_0405",
            "column": "SERVICE_DATE_0405",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "PATIENT_FIRST_NAME_0405",
            "column": "PATIENT_FIRST_NAME_0405",
            "datatype": "VARCHAR(10)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "PATIENT_LAST_NAME_0405",
            "column": "PATIENT_LAST_NAME_0405",
            "datatype": "VARCHAR(15)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        }
    ],

```

```

{
  "name": "PATIENT_SEX_0405",
  "column": "PATIENT_SEX_0405",
  "datatype": "CHAR(1)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "RELATION_TO_EMPLOYEE_0405",
  "column": "RELATION_TO_EMPLOYEE_0405",
  "datatype": "VARCHAR(10)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "DENTIST_FIRST_NAME_0405",
  "column": "DENTIST_FIRST_NAME_0405",
  "datatype": "VARCHAR(10)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "DENTIST_LAST_NAME_0405",
  "column": "DENTIST_LAST_NAME_0405",
  "datatype": "VARCHAR(15)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "DENTIST_STREET_0405",
  "column": "DENTIST_STREET_0405",
  "datatype": "VARCHAR(20)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "DENTIST_CITY_0405",
  "column": "DENTIST_CITY_0405",
  "datatype": "VARCHAR(15)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "DENTIST_STATE_0405",
  "column": "DENTIST_STATE_0405",
  "datatype": "VARCHAR(2)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "DENTIST_ZIP_FIRST_FIVE_0405",
  "column": "DENTIST_ZIP_FIRST_FIVE_0405",
  "datatype": "VARCHAR(5)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "DENTIST_ZIP_LAST_FOUR_0405",
  "column": "DENTIST_ZIP_LAST_FOUR_0405",
  "datatype": "VARCHAR(4)",

```

```

        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "DENTIST_LICENSE_NUMBER_0405",
        "column": "DENTIST_LICENSE_NUMBER_0405",
        "datatype": "SMALLINT",
        "length": 4,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "NUMBER_OF_PROCEDURES_0405",
        "column": "NUMBER_OF_PROCEDURES_0405",
        "datatype": "DECIMAL(1,0)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "TOOTH_NUMBER_0405",
        "column": "TOOTH_NUMBER_0405",
        "datatype": "SMALLINT",
        "length": 4,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "PROCEDURE_CODE_0405",
        "column": "PROCEDURE_CODE_0405",
        "datatype": "SMALLINT",
        "length": 4,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "DESCRIPTION_OF_SERVICE_0405",
        "column": "DESCRIPTION_OF_SERVICE_0405",
        "datatype": "VARCHAR(60)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "FEE_0405",
        "column": "FEE_0405",
        "datatype": "DECIMAL(2,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "COVERAGE_ID",
        "column": "COVERAGE_ID",
        "datatype": "BIGINT",
        "length": 18,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    }
]
},
{
    "name": "DEPARTMENT",
    "table": "DEPARTMENT",
    "primary_key": "DEPT_ID_0410",
    "fields": [
        {

```



```

        "name": "DEPT_ID_0410",
        "column": "DEPT_ID_0410",
        "datatype": "SMALLINT",
        "length": 4,
        "scale": 0,
        "nullable": false,
        "primary_key": true
    },
    {
        "name": "DEPT_NAME_0410",
        "column": "DEPT_NAME_0410",
        "datatype": "VARCHAR(45)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "DEPT_HEAD_ID_0410",
        "column": "DEPT_HEAD_ID_0410",
        "datatype": "SMALLINT",
        "length": 4,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    }
]
},
{
    "name": "EMPLOYEE",
    "table": "EMPLOYEE",
    "primary_key": "EMP_ID_0415",
    "fields": [
        {
            "name": "START_DATE_0415",
            "column": "START_DATE_0415",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "TERMINATION_DATE_0415",
            "column": "TERMINATION_DATE_0415",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "BIRTH_DATE_0415",
            "column": "BIRTH_DATE_0415",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "EMP_ID_0415",
            "column": "EMP_ID_0415",
            "datatype": "SMALLINT",
            "length": 4,
            "scale": 0,
            "nullable": false,
            "primary_key": true
        },
        {
            "name": "EMP_FIRST_NAME_0415",
            "column": "EMP_FIRST_NAME_0415",
            "datatype": "VARCHAR(10)",
            "length": null,
            "scale": null,
            "nullable": true,

```

```

    "primary_key": false
  },
  {
    "name": "EMP_LAST_NAME_0415",
    "column": "EMP_LAST_NAME_0415",
    "datatype": "VARCHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "EMP_STREET_0415",
    "column": "EMP_STREET_0415",
    "datatype": "VARCHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "EMP_CITY_0415",
    "column": "EMP_CITY_0415",
    "datatype": "VARCHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "EMP_STATE_0415",
    "column": "EMP_STATE_0415",
    "datatype": "VARCHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "EMP_ZIP_FIRST_FIVE_0415",
    "column": "EMP_ZIP_FIRST_FIVE_0415",
    "datatype": "VARCHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "EMP_ZIP_LAST_FOUR_0415",
    "column": "EMP_ZIP_LAST_FOUR_0415",
    "datatype": "VARCHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "EMP_PHONE_0415",
    "column": "EMP_PHONE_0415",
    "datatype": "SMALLINT",
    "length": 4,
    "scale": 0,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "STATUS_0415",
    "column": "STATUS_0415",
    "datatype": "VARCHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "SS_NUMBER_0415",

```

```

        "column": "SS_NUMBER_0415",
        "datatype": "SMALLINT",
        "length": 4,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "DEPT_ID_0410",
        "column": "DEPT_ID_0410",
        "datatype": "SMALLINT",
        "length": 4,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "OFFICE_CODE_0450",
        "column": "OFFICE_CODE_0450",
        "datatype": "VARCHAR(3)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    }
]
},
{
    "name": "EMPOSITION",
    "table": "EMPOSITION",
    "primary_key": null,
    "fields": [
        {
            "name": "START_DATE_0420",
            "column": "START_DATE_0420",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "FINISH_DATE_0420",
            "column": "FINISH_DATE_0420",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "SALARY_GRADE_0420",
            "column": "SALARY_GRADE_0420",
            "datatype": "SMALLINT",
            "length": 4,
            "scale": 0,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "SALARY_AMOUNT_0420",
            "column": "SALARY_AMOUNT_0420",
            "datatype": "DECIMAL(2,2)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "BONUS_PERCENT_0420",
            "column": "BONUS_PERCENT_0420",
            "datatype": "DECIMAL(3,3)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        }
    ]
}

```

```

    },
    {
      "name": "COMMISSION_PERCENT_0420",
      "column": "COMMISSION_PERCENT_0420",
      "datatype": "DECIMAL(3,3)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false
    },
    {
      "name": "OVERTIME_RATE_0420",
      "column": "OVERTIME_RATE_0420",
      "datatype": "DECIMAL(3,2)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false
    },
    {
      "name": "EMP_ID_0415",
      "column": "EMP_ID_0415",
      "datatype": "SMALLINT",
      "length": 4,
      "scale": 0,
      "nullable": true,
      "primary_key": false
    },
    {
      "name": "JOB_ID_0440",
      "column": "JOB_ID_0440",
      "datatype": "SMALLINT",
      "length": 4,
      "scale": 0,
      "nullable": true,
      "primary_key": false
    }
  ]
},
{
  "name": "EXPERTISE",
  "table": "EXPERTISE",
  "primary_key": null,
  "fields": [
    {
      "name": "EXPERTISE_DATE_0425",
      "column": "EXPERTISE_DATE_0425",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false
    },
    {
      "name": "SKILL_LEVEL_0425",
      "column": "SKILL_LEVEL_0425",
      "datatype": "VARCHAR(2)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false
    },
    {
      "name": "EMP_ID_0415",
      "column": "EMP_ID_0415",
      "datatype": "SMALLINT",
      "length": 4,
      "scale": 0,
      "nullable": true,
      "primary_key": false
    },
    {
      "name": "SKILL_ID_0455",
      "column": "SKILL_ID_0455",
      "datatype": "SMALLINT",
      "length": 4,

```

```

        "scale": 0,
        "nullable": true,
        "primary_key": false
    }
]
},
{
    "name": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "primary_key": null,
    "fields": [
        {
            "name": "CLAIM_DATE_0430",
            "column": "CLAIM_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "PATIENT_BIRTH_DATE_0430",
            "column": "PATIENT_BIRTH_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "ADMIT_DATE_0430",
            "column": "ADMIT_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "DISCHARGE_DATE_0430",
            "column": "DISCHARGE_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "PATIENT_FIRST_NAME_0430",
            "column": "PATIENT_FIRST_NAME_0430",
            "datatype": "VARCHAR(10)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "PATIENT_LAST_NAME_0430",
            "column": "PATIENT_LAST_NAME_0430",
            "datatype": "VARCHAR(15)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "PATIENT_SEX_0430",
            "column": "PATIENT_SEX_0430",
            "datatype": "CHAR(1)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "RELATION_TO_EMPLOYEE_0430",

```

```

    "column": "RELATION_TO_EMPLOYEE_0430",
    "datatype": "VARCHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "HOSPITAL_NAME_0430",
    "column": "HOSPITAL_NAME_0430",
    "datatype": "VARCHAR(25)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "HOSP_STREET_0430",
    "column": "HOSP_STREET_0430",
    "datatype": "VARCHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "HOSP_CITY_0430",
    "column": "HOSP_CITY_0430",
    "datatype": "VARCHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "HOSP_STATE_0430",
    "column": "HOSP_STATE_0430",
    "datatype": "VARCHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "HOSP_ZIP_FIRST_FIVE_0430",
    "column": "HOSP_ZIP_FIRST_FIVE_0430",
    "datatype": "VARCHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "HOSP_ZIP_LAST_FOUR_0430",
    "column": "HOSP_ZIP_LAST_FOUR_0430",
    "datatype": "VARCHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "DIAGNOSIS_0430",
    "column": "DIAGNOSIS_0430",
    "datatype": "VARCHAR(120)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "WARD_DAYS_0430",
    "column": "WARD_DAYS_0430",
    "datatype": "DECIMAL(5,0)",
    "length": null,
    "scale": null,

```

```

    "nullable": true,
    "primary_key": false
  },
  {
    "name": "WARD_RATE_0430",
    "column": "WARD_RATE_0430",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "WARD_TOTAL_0430",
    "column": "WARD_TOTAL_0430",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "SEMI_DAYS_0430",
    "column": "SEMI_DAYS_0430",
    "datatype": "DECIMAL(5,0)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "SEMI_RATE_0430",
    "column": "SEMI_RATE_0430",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "SEMI_TOTAL_0430",
    "column": "SEMI_TOTAL_0430",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "DELIVERY_COST_0430",
    "column": "DELIVERY_COST_0430",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "ANESTHESIA_COST_0430",
    "column": "ANESTHESIA_COST_0430",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "LAB_COST_0430",
    "column": "LAB_COST_0430",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {

```

```

        "name": "COVERAGE_ID",
        "column": "COVERAGE_ID",
        "datatype": "BIGINT",
        "length": 18,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    }
]
},
{
    "name": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "primary_key": "INS_PLAN_CODE_0435",
    "fields": [
        {
            "name": "INS_PLAN_CODE_0435",
            "column": "INS_PLAN_CODE_0435",
            "datatype": "VARCHAR(3)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true
        },
        {
            "name": "INS_CO_NAME_0435",
            "column": "INS_CO_NAME_0435",
            "datatype": "VARCHAR(45)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "INS_CO_STREET_0435",
            "column": "INS_CO_STREET_0435",
            "datatype": "VARCHAR(20)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "INS_CO_CITY_0435",
            "column": "INS_CO_CITY_0435",
            "datatype": "VARCHAR(15)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "INS_CO_STATE_0435",
            "column": "INS_CO_STATE_0435",
            "datatype": "VARCHAR(2)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "INS_CO_ZIP_FIRST_FIVE_0435",
            "column": "INS_CO_ZIP_FIRST_FIVE_0435",
            "datatype": "VARCHAR(5)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "INS_CO_ZIP_LAST_FOUR_0435",
            "column": "INS_CO_ZIP_LAST_FOUR_0435",
            "datatype": "VARCHAR(4)",
            "length": null,
            "scale": null,
            "nullable": true,

```



```

    "primary_key": false
  },
  {
    "name": "INS_CO_PHONE_0435",
    "column": "INS_CO_PHONE_0435",
    "datatype": "SMALLINT",
    "length": 4,
    "scale": 0,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "GROUP_NUMBER_0435",
    "column": "GROUP_NUMBER_0435",
    "datatype": "SMALLINT",
    "length": 4,
    "scale": 0,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "DEDUCT_0435",
    "column": "DEDUCT_0435",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "MAXIMUM_LIFE_COST_0435",
    "column": "MAXIMUM_LIFE_COST_0435",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "FAMILY_COST_0435",
    "column": "FAMILY_COST_0435",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "DEP_COST_0435",
    "column": "DEP_COST_0435",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  }
]
},
{
  "name": "JOB",
  "table": "JOB",
  "primary_key": "JOB_ID_0440",
  "fields": [
    {
      "name": "JOB_ID_0440",
      "column": "JOB_ID_0440",
      "datatype": "SMALLINT",
      "length": 4,
      "scale": 0,
      "nullable": false,
      "primary_key": true
    },
    {
      "name": "TITLE_0440",
      "column": "TITLE_0440",
      "datatype": "VARCHAR(20)",

```

```

    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "DESCRIPTION_LINE_0440",
    "column": "DESCRIPTION_LINE_0440",
    "datatype": "VARCHAR(120)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "REQUIREMENT_LINE_0440",
    "column": "REQUIREMENT_LINE_0440",
    "datatype": "VARCHAR(120)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "MINIMUM_SALARY_0440",
    "column": "MINIMUM_SALARY_0440",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "MAXIMUM_SALARY_0440",
    "column": "MAXIMUM_SALARY_0440",
    "datatype": "DECIMAL(2,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "SALARY_GRADES_0440",
    "column": "SALARY_GRADES_0440",
    "datatype": "SMALLINT",
    "length": 4,
    "scale": 0,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "NUMBER_OF_POSITIONS_0440",
    "column": "NUMBER_OF_POSITIONS_0440",
    "datatype": "SMALLINT",
    "length": 4,
    "scale": 0,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "NUMBER_OPEN_0440",
    "column": "NUMBER_OPEN_0440",
    "datatype": "SMALLINT",
    "length": 4,
    "scale": 0,
    "nullable": true,
    "primary_key": false
  }
]
},
{
  "name": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "primary_key": null,
  "fields": [
    {

```

```

    "name": "CLAIM_DATE_0445",
    "column": "CLAIM_DATE_0445",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PATIENT_BIRTH_DATE_0445",
    "column": "PATIENT_BIRTH_DATE_0445",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "SERVICE_DATE_0445",
    "column": "SERVICE_DATE_0445",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PATIENT_FIRST_NAME_0445",
    "column": "PATIENT_FIRST_NAME_0445",
    "datatype": "VARCHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PATIENT_LAST_NAME_0445",
    "column": "PATIENT_LAST_NAME_0445",
    "datatype": "VARCHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PATIENT_SEX_0445",
    "column": "PATIENT_SEX_0445",
    "datatype": "CHAR(1)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "RELATION_TO_EMPLOYEE_0445",
    "column": "RELATION_TO_EMPLOYEE_0445",
    "datatype": "VARCHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PHYSICIAN_FIRST_NAME_0445",
    "column": "PHYSICIAN_FIRST_NAME_0445",
    "datatype": "VARCHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PHYSICIAN_LAST_NAME_0445",
    "column": "PHYSICIAN_LAST_NAME_0445",
    "datatype": "VARCHAR(15)",
    "length": null,

```

```

    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PHYSICIAN_STREET_0445",
    "column": "PHYSICIAN_STREET_0445",
    "datatype": "VARCHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PHYSICIAN_CITY_0445",
    "column": "PHYSICIAN_CITY_0445",
    "datatype": "VARCHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PHYSICIAN_STATE_0445",
    "column": "PHYSICIAN_STATE_0445",
    "datatype": "VARCHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PHYSICIAN_ZIP_FIRST_FIVE_0445",
    "column": "PHYSICIAN_ZIP_FIRST_FIVE_0445",
    "datatype": "VARCHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PHYSICIAN_ZIP_LAST_FOUR_0445",
    "column": "PHYSICIAN_ZIP_LAST_FOUR_0445",
    "datatype": "VARCHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "PHYSICIAN_ID_0445",
    "column": "PHYSICIAN_ID_0445",
    "datatype": "SMALLINT",
    "length": 4,
    "scale": 0,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "DIAGNOSIS_0445",
    "column": "DIAGNOSIS_0445",
    "datatype": "VARCHAR(120)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
  {
    "name": "NUMBER_OF_PROCEDURES_0445",
    "column": "NUMBER_OF_PROCEDURES_0445",
    "datatype": "DECIMAL(1,0)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false
  },
},

```

```

{
  "name": "PROCEDURE_CODE_0445",
  "column": "PROCEDURE_CODE_0445",
  "datatype": "SMALLINT",
  "length": 4,
  "scale": 0,
  "nullable": true,
  "primary_key": false
},
{
  "name": "DESCRIPTION_OF_SERVICE_0445",
  "column": "DESCRIPTION_OF_SERVICE_0445",
  "datatype": "VARCHAR(60)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "FEE_0445",
  "column": "FEE_0445",
  "datatype": "DECIMAL(2,2)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false
},
{
  "name": "COVERAGE_ID",
  "column": "COVERAGE_ID",
  "datatype": "BIGINT",
  "length": 18,
  "scale": 0,
  "nullable": true,
  "primary_key": false
}
]
},
{
  "name": "OFFICE",
  "table": "OFFICE",
  "primary_key": "OFFICE_CODE_0450",
  "fields": [
    {
      "name": "OFFICE_CODE_0450",
      "column": "OFFICE_CODE_0450",
      "datatype": "VARCHAR(3)",
      "length": null,
      "scale": null,
      "nullable": false,
      "primary_key": true
    },
    {
      "name": "OFFICE_STREET_0450",
      "column": "OFFICE_STREET_0450",
      "datatype": "VARCHAR(20)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false
    },
    {
      "name": "OFFICE_CITY_0450",
      "column": "OFFICE_CITY_0450",
      "datatype": "VARCHAR(15)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false
    },
    {
      "name": "OFFICE_STATE_0450",
      "column": "OFFICE_STATE_0450",
      "datatype": "VARCHAR(2)",
      "length": null,
      "scale": null,

```

```

        "nullable": true,
        "primary_key": false
    },
    {
        "name": "OFFICE_ZIP_FIRST_FIVE_0450",
        "column": "OFFICE_ZIP_FIRST_FIVE_0450",
        "datatype": "VARCHAR(5)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "OFFICE_ZIP_LAST_FOUR_0450",
        "column": "OFFICE_ZIP_LAST_FOUR_0450",
        "datatype": "VARCHAR(4)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "OFFICE_PHONE_0450",
        "column": "OFFICE_PHONE_0450",
        "datatype": "SMALLINT",
        "length": 4,
        "scale": 0,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "OFFICE_AREA_CODE_0450",
        "column": "OFFICE_AREA_CODE_0450",
        "datatype": "VARCHAR(3)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    },
    {
        "name": "SPEED_DIAL_0450",
        "column": "SPEED_DIAL_0450",
        "datatype": "VARCHAR(3)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    }
]
},
{
    "name": "SKILL",
    "table": "SKILL",
    "primary_key": "SKILL_ID_0455",
    "fields": [
        {
            "name": "SKILL_ID_0455",
            "column": "SKILL_ID_0455",
            "datatype": "SMALLINT",
            "length": 4,
            "scale": 0,
            "nullable": false,
            "primary_key": true
        },
        {
            "name": "SKILL_NAME_0455",
            "column": "SKILL_NAME_0455",
            "datatype": "VARCHAR(12)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "SKILL_DESCRIPTION_0455",
            "column": "SKILL_DESCRIPTION_0455",

```

```

        "datatype": "VARCHAR(60)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false
    }
]
},
{
    "name": "STRUCTURE",
    "table": "STRUCTURE",
    "primary_key": null,
    "fields": [
        {
            "name": "STRUCTURE_DATE_0460",
            "column": "STRUCTURE_DATE_0460",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "STRUCTURE_CODE_0460",
            "column": "STRUCTURE_CODE_0460",
            "datatype": "VARCHAR(2)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "MANAGES_EMP_ID_0415",
            "column": "MANAGES_EMP_ID_0415",
            "datatype": "SMALLINT",
            "length": 4,
            "scale": 0,
            "nullable": true,
            "primary_key": false
        },
        {
            "name": "REPORTS_TO_EMP_ID_0415",
            "column": "REPORTS_TO_EMP_ID_0415",
            "datatype": "SMALLINT",
            "length": 4,
            "scale": 0,
            "nullable": true,
            "primary_key": false
        }
    ]
}
],
"relationships": [
    {
        "set_name": "DEPT_EMPLOYEE",
        "parent_record": "DEPARTMENT",
        "child_record": "EMPLOYEE",
        "parent_key": "DEPT_ID_0410",
        "child_fk": "DEPT_ID_0410",
        "cardinality": "1:N",
        "order_by": [
            "DEPT_ID_0410"
        ]
    },
    {
        "set_name": "MANAGES",
        "parent_record": "EMPLOYEE",
        "child_record": "STRUCTURE",
        "parent_key": "EMP_ID_0415",
        "child_fk": null,
        "cardinality": "1:N",
        "order_by": []
    },
    {
        "set_name": "OFFICE_EMPLOYEE",
        "parent_record": "OFFICE",

```

```

    "child_record": "EMPLOYEE",
    "parent_key": "OFFICE_CODE_0450",
    "child_fk": "OFFICE_CODE_0450",
    "cardinality": "1:N",
    "order_by": [
        "OFFICE_CODE_0450"
    ]
},
{
    "set_name": "EMP_EXPERTISE",
    "parent_record": "EMPLOYEE",
    "child_record": "EXPERTISE",
    "parent_key": "EMP_ID_0415",
    "child_fk": "EMP_ID_0415",
    "cardinality": "1:N",
    "order_by": [
        "EMP_ID_0415"
    ]
},
{
    "set_name": "SKILL_EXPERTISE",
    "parent_record": "SKILL",
    "child_record": "EXPERTISE",
    "parent_key": "SKILL_ID_0455",
    "child_fk": "SKILL_ID_0455",
    "cardinality": "1:N",
    "order_by": [
        "SKILL_ID_0455"
    ]
},
{
    "set_name": "EMP_EMPOSITION",
    "parent_record": "EMPLOYEE",
    "child_record": "EMPOSITION",
    "parent_key": "EMP_ID_0415",
    "child_fk": "EMP_ID_0415",
    "cardinality": "1:N",
    "order_by": [
        "EMP_ID_0415"
    ]
},
{
    "set_name": "REPORTS_TO",
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "parent_key": "EMP_ID_0415",
    "child_fk": null,
    "cardinality": "1:N",
    "order_by": []
},
{
    "set_name": "EMP_COVERAGE",
    "parent_record": "EMPLOYEE",
    "child_record": "COVERAGE",
    "parent_key": "EMP_ID_0415",
    "child_fk": "EMP_ID_0415",
    "cardinality": "1:N",
    "order_by": [
        "EMP_ID_0415"
    ]
},
{
    "set_name": "JOB_EMPOSITION",
    "parent_record": "JOB",
    "child_record": "EMPOSITION",
    "parent_key": "JOB_ID_0440",
    "child_fk": "JOB_ID_0440",
    "cardinality": "1:N",
    "order_by": [
        "JOB_ID_0440"
    ]
},
{
    "set_name": "COVERAGE_CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "DENTAL_CLAIM",

```



```

    "parent_key": "COVERAGE_ID",
    "child_fk": "COVERAGE_ID",
    "cardinality": "1:N",
    "order_by": [
        "COVERAGE_ID"
    ]
},
{
    "set_name": "COVERAGE_CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "HOSPITAL_CLAIM",
    "parent_key": "COVERAGE_ID",
    "child_fk": "COVERAGE_ID",
    "cardinality": "1:N",
    "order_by": [
        "COVERAGE_ID"
    ]
},
{
    "set_name": "COVERAGE_CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "NON_HOSP_CLAIM",
    "parent_key": "COVERAGE_ID",
    "child_fk": "COVERAGE_ID",
    "cardinality": "1:N",
    "order_by": [
        "COVERAGE_ID"
    ]
}
],
"field_map": {
    "TYPE_0400": {
        "host": "HV-COVERAGE-TYPE",
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "TYPE_0400",
        "legacy_field": "TYPE_0400"
    },
    "TYPE": {
        "host": "HV-COVERAGE-TYPE",
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "TYPE_0400",
        "legacy_field": "TYPE_0400"
    },
    "INS_PLAN_CODE_0400": {
        "host": "HV-COVERAGE-INS-PLAN-CODE",
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "INS_PLAN_CODE_0400",
        "legacy_field": "INS_PLAN_CODE_0400"
    },
    "INS_PLAN_CODE": {
        "host": "HV-INSURANCE-PLAN-INS-PLAN-CODE",
        "record": "INSURANCE_PLAN",
        "table": "INSURANCE_PLAN",
        "column": "INS_PLAN_CODE_0435",
        "legacy_field": "INS_PLAN_CODE_0435"
    },
    "PATIENT_FIRST_NAME_0405": {
        "host": "HV-DENTAL-CLAIM-PATIENT-FIRST-NAME",
        "record": "DENTAL_CLAIM",
        "table": "DENTAL_CLAIM",
        "column": "PATIENT_FIRST_NAME_0405",
        "legacy_field": "PATIENT_FIRST_NAME_0405"
    },
    "PATIENT_FIRST_NAME": {
        "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRST-NAME",
        "record": "NON_HOSP_CLAIM",
        "table": "NON_HOSP_CLAIM",
        "column": "PATIENT_FIRST_NAME_0445",
        "legacy_field": "PATIENT_FIRST_NAME_0445"
    },
    "PATIENT_LAST_NAME_0405": {
        "host": "HV-DENTAL-CLAIM-PATIENT-LAST-NAME",
        "record": "DENTAL_CLAIM",

```

```

    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LAST_NAME_0405",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT_LAST_NAME": {
    "host": "HV-NON-HOSP-CLAIM-PATIENT-LAST-NAME",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_LAST_NAME_0445",
    "legacy_field": "PATIENT_LAST_NAME_0445"
  },
  "PATIENT_SEX_0405": {
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_0405",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENT_SEX": {
    "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_SEX_0445",
    "legacy_field": "PATIENT_SEX_0445"
  },
  "RELATION_TO_EMPLOYEE_0405": {
    "host": "HV-DENTAL-CLAIM-RELATION-TO-EMPLOYEE",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TO_EMPLOYEE_0405",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION_TO_EMPLOYEE": {
    "host": "HV-NON-HOSP-CLAIM-RELATION-TO-EMPLOYEE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "RELATION_TO_EMPLOYEE_0445",
    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "DENTIST_FIRST_NAME_0405": {
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRST-NAME",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRST_NAME_0405",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST_FIRST_NAME": {
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRST-NAME",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRST_NAME_0405",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST_LAST_NAME_0405": {
    "host": "HV-DENTAL-CLAIM-DENTIST-LAST-NAME",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LAST_NAME_0405",
    "legacy_field": "DENTIST_LAST_NAME_0405"
  },
  "DENTIST_LAST_NAME": {
    "host": "HV-DENTAL-CLAIM-DENTIST-LAST-NAME",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LAST_NAME_0405",
    "legacy_field": "DENTIST_LAST_NAME_0405"
  },
  "DENTIST_STREET_0405": {
    "host": "HV-DENTAL-CLAIM-DENTIST-STREET",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STREET_0405",
    "legacy_field": "DENTIST_STREET_0405"
  },
  "DENTIST_STREET": {
    "host": "HV-DENTAL-CLAIM-DENTIST-STREET",

```

```

    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STREET_0405",
    "legacy_field": "DENTIST_STREET_0405"
  },
  "DENTIST_CITY_0405": {
    "host": "HV-DENTAL-CLAIM-DENTIST-CITY",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_CITY_0405",
    "legacy_field": "DENTIST_CITY_0405"
  },
  "DENTIST_CITY": {
    "host": "HV-DENTAL-CLAIM-DENTIST-CITY",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_CITY_0405",
    "legacy_field": "DENTIST_CITY_0405"
  },
  "DENTIST_STATE_0405": {
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_0405",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST_STATE": {
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_0405",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST_ZIP_FIRST_FIVE_0405": {
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIP-FIRST-FIVE",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIP_FIRST_FIVE_0405",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTIST_ZIP_FIRST_FIVE": {
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIP-FIRST-FIVE",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIP_FIRST_FIVE_0405",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTIST_ZIP_LAST_FOUR_0405": {
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIP-LAST-FOUR",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIP_LAST_FOUR_0405",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_ZIP_LAST_FOUR": {
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIP-LAST-FOUR",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIP_LAST_FOUR_0405",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_LICENSE_NUMBER_0405": {
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSE-NUMBER",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSE_NUMBER_0405",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "DENTIST_LICENSE_NUMBER": {
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSE-NUMBER",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSE_NUMBER_0405",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "NUMBER_OF_PROCEDURES_0405": {

```

```

    "host": "HV-DENTAL-CLAIM-NUMBER-OF-PROCEDURES",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "NUMBER_OF_PROCEDURES_0405",
    "legacy_field": "NUMBER_OF_PROCEDURES_0405"
  },
  "NUMBER_OF_PROCEDURES": {
    "host": "HV-NON-HOSP-CLAIM-NUMBER-OF-PROCEDURES",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "NUMBER_OF_PROCEDURES_0445",
    "legacy_field": "NUMBER_OF_PROCEDURES_0445"
  },
  "TOOTH_NUMBER_0405": {
    "host": "HV-DENTAL-CLAIM-TOOTH-NUMBER",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "TOOTH_NUMBER_0405",
    "legacy_field": "TOOTH_NUMBER_0405"
  },
  "TOOTH_NUMBER": {
    "host": "HV-DENTAL-CLAIM-TOOTH-NUMBER",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "TOOTH_NUMBER_0405",
    "legacy_field": "TOOTH_NUMBER_0405"
  },
  "PROCEDURE_CODE_0405": {
    "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PROCEDURE_CODE_0405",
    "legacy_field": "PROCEDURE_CODE_0405"
  },
  "PROCEDURE_CODE": {
    "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PROCEDURE_CODE_0445",
    "legacy_field": "PROCEDURE_CODE_0445"
  },
  "DESCRIPTION_OF_SERVICE_0405": {
    "host": "HV-DENTAL-CLAIM-DESCRIPTION-OF-SERVICE",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DESCRIPTION_OF_SERVICE_0405",
    "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
  },
  "DESCRIPTION_OF_SERVICE": {
    "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OF-SERVICE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DESCRIPTION_OF_SERVICE_0445",
    "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
  },
  "FEE_0405": {
    "host": "HV-DENTAL-CLAIM-FEE",
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "FEE_0405",
    "legacy_field": "FEE_0405"
  },
  "FEE": {
    "host": "HV-NON-HOSP-CLAIM-FEE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_0445",
    "legacy_field": "FEE_0445"
  },
  "DEPT_ID_0410": {
    "host": "HV-DEPARTMENT-DEPT-ID",
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_0410",
    "legacy_field": "DEPT_ID_0410"
  },

```

```

"DEPT_ID": {
  "host": "HV-DEPARTMENT-DEPT-ID",
  "record": "DEPARTMENT",
  "table": "DEPARTMENT",
  "column": "DEPT_ID_0410",
  "legacy_field": "DEPT_ID_0410"
},
"DEPT_NAME_0410": {
  "host": "HV-DEPARTMENT-DEPT-NAME",
  "record": "DEPARTMENT",
  "table": "DEPARTMENT",
  "column": "DEPT_NAME_0410",
  "legacy_field": "DEPT_NAME_0410"
},
"DEPT_NAME": {
  "host": "HV-DEPARTMENT-DEPT-NAME",
  "record": "DEPARTMENT",
  "table": "DEPARTMENT",
  "column": "DEPT_NAME_0410",
  "legacy_field": "DEPT_NAME_0410"
},
"DEPT_HEAD_ID_0410": {
  "host": "HV-DEPARTMENT-DEPT-HEAD-ID",
  "record": "DEPARTMENT",
  "table": "DEPARTMENT",
  "column": "DEPT_HEAD_ID_0410",
  "legacy_field": "DEPT_HEAD_ID_0410"
},
"DEPT_HEAD_ID": {
  "host": "HV-DEPARTMENT-DEPT-HEAD-ID",
  "record": "DEPARTMENT",
  "table": "DEPARTMENT",
  "column": "DEPT_HEAD_ID_0410",
  "legacy_field": "DEPT_HEAD_ID_0410"
},
"EMP_ID_0415": {
  "host": "HV-EMPLOYEE-EMP-ID",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ID_0415",
  "legacy_field": "EMP_ID_0415"
},
"EMP_ID": {
  "host": "HV-EMPLOYEE-EMP-ID",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ID_0415",
  "legacy_field": "EMP_ID_0415"
},
"EMP_FIRST_NAME_0415": {
  "host": "HV-EMPLOYEE-EMP-FIRST-NAME",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_FIRST_NAME_0415",
  "legacy_field": "EMP_FIRST_NAME_0415"
},
"EMP_FIRST_NAME": {
  "host": "HV-EMPLOYEE-EMP-FIRST-NAME",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_FIRST_NAME_0415",
  "legacy_field": "EMP_FIRST_NAME_0415"
},
"EMP_LAST_NAME_0415": {
  "host": "HV-EMPLOYEE-EMP-LAST-NAME",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_LAST_NAME_0415",
  "legacy_field": "EMP_LAST_NAME_0415"
},
"EMP_LAST_NAME": {
  "host": "HV-EMPLOYEE-EMP-LAST-NAME",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_LAST_NAME_0415",
  "legacy_field": "EMP_LAST_NAME_0415"
}

```

```

},
"EMP_STREET_0415": {
  "host": "HV-EMPLOYEE-EMP-STREET",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STREET_0415",
  "legacy_field": "EMP_STREET_0415"
},
"EMP_STREET": {
  "host": "HV-EMPLOYEE-EMP-STREET",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STREET_0415",
  "legacy_field": "EMP_STREET_0415"
},
"EMP_CITY_0415": {
  "host": "HV-EMPLOYEE-EMP-CITY",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_0415",
  "legacy_field": "EMP_CITY_0415"
},
"EMP_CITY": {
  "host": "HV-EMPLOYEE-EMP-CITY",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_0415",
  "legacy_field": "EMP_CITY_0415"
},
"EMP_STATE_0415": {
  "host": "HV-EMPLOYEE-EMP-STATE",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_0415",
  "legacy_field": "EMP_STATE_0415"
},
"EMP_STATE": {
  "host": "HV-EMPLOYEE-EMP-STATE",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_0415",
  "legacy_field": "EMP_STATE_0415"
},
"EMP_ZIP_FIRST_FIVE_0415": {
  "host": "HV-EMPLOYEE-EMP-ZIP-FIRST-FIVE",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIP_FIRST_FIVE_0415",
  "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
},
"EMP_ZIP_FIRST_FIVE": {
  "host": "HV-EMPLOYEE-EMP-ZIP-FIRST-FIVE",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIP_FIRST_FIVE_0415",
  "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
},
"EMP_ZIP_LAST_FOUR_0415": {
  "host": "HV-EMPLOYEE-EMP-ZIP-LAST-FOUR",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIP_LAST_FOUR_0415",
  "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
},
"EMP_ZIP_LAST_FOUR": {
  "host": "HV-EMPLOYEE-EMP-ZIP-LAST-FOUR",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIP_LAST_FOUR_0415",
  "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
},
"EMP_PHONE_0415": {
  "host": "HV-EMPLOYEE-EMP-PHONE",
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_PHONE_0415",

```

```

    "legacy_field": "EMP_PHONE_0415"
  },
  "EMP_PHONE": {
    "host": "HV-EMPLOYEE-EMP-PHONE",
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_0415",
    "legacy_field": "EMP_PHONE_0415"
  },
  "STATUS_0415": {
    "host": "HV-EMPLOYEE-STATUS",
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_0415",
    "legacy_field": "STATUS_0415"
  },
  "STATUS": {
    "host": "HV-EMPLOYEE-STATUS",
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_0415",
    "legacy_field": "STATUS_0415"
  },
  "SS_NUMBER_0415": {
    "host": "HV-EMPLOYEE-SS-NUMBER",
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_0415",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SS_NUMBER": {
    "host": "HV-EMPLOYEE-SS-NUMBER",
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_0415",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SALARY_GRADE_0420": {
    "host": "HV-EMPOSITION-SALARY-GRADE",
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_GRADE_0420",
    "legacy_field": "SALARY_GRADE_0420"
  },
  "SALARY_GRADE": {
    "host": "HV-EMPOSITION-SALARY-GRADE",
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_GRADE_0420",
    "legacy_field": "SALARY_GRADE_0420"
  },
  "SALARY_AMOUNT_0420": {
    "host": "HV-EMPOSITION-SALARY-AMOUNT",
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_0420",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARY_AMOUNT": {
    "host": "HV-EMPOSITION-SALARY-AMOUNT",
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_0420",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "BONUS_PERCENT_0420": {
    "host": "HV-EMPOSITION-BONUS-PERCENT",
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_0420",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUS_PERCENT": {
    "host": "HV-EMPOSITION-BONUS-PERCENT",
    "record": "EMPOSITION",
    "table": "EMPOSITION",

```

```

    "column": "BONUS_PERCENT_0420",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "COMMISSION_PERCENT_0420": {
    "host": "HV-EMPOSITION-COMMISSION-PERCENT",
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_0420",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSION_PERCENT": {
    "host": "HV-EMPOSITION-COMMISSION-PERCENT",
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_0420",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "OVERTIME_RATE_0420": {
    "host": "HV-EMPOSITION-OVERTIME-RATE",
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "OVERTIME_RATE_0420",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "OVERTIME_RATE": {
    "host": "HV-EMPOSITION-OVERTIME-RATE",
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "OVERTIME_RATE_0420",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "SKILL_LEVEL_0425": {
    "host": "HV-EXPERTISE-SKILL-LEVEL",
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_0425",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "SKILL_LEVEL": {
    "host": "HV-EXPERTISE-SKILL-LEVEL",
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_0425",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "PATIENT_FIRST_NAME_0430": {
    "host": "HV-HOSPITAL-CLAIM-PATIENT-FIRST-NAME",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "PATIENT_FIRST_NAME_0430",
    "legacy_field": "PATIENT_FIRST_NAME_0430"
  },
  "PATIENT_LAST_NAME_0430": {
    "host": "HV-HOSPITAL-CLAIM-PATIENT-LAST-NAME",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "PATIENT_LAST_NAME_0430",
    "legacy_field": "PATIENT_LAST_NAME_0430"
  },
  "PATIENT_SEX_0430": {
    "host": "HV-HOSPITAL-CLAIM-PATIENT-SEX",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "PATIENT_SEX_0430",
    "legacy_field": "PATIENT_SEX_0430"
  },
  "RELATION_TO_EMPLOYEE_0430": {
    "host": "HV-HOSPITAL-CLAIM-RELATION-TO-EMPLOYEE",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "RELATION_TO_EMPLOYEE_0430",
    "legacy_field": "RELATION_TO_EMPLOYEE_0430"
  },
  "HOSPITAL_NAME_0430": {
    "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME",
    "record": "HOSPITAL_CLAIM",

```



```

    "table": "HOSPITAL_CLAIM",
    "column": "HOSPITAL_NAME_0430",
    "legacy_field": "HOSPITAL_NAME_0430"
  },
  "HOSPITAL_NAME": {
    "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSPITAL_NAME_0430",
    "legacy_field": "HOSPITAL_NAME_0430"
  },
  "HOSP_STREET_0430": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-STREET",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STREET_0430",
    "legacy_field": "HOSP_STREET_0430"
  },
  "HOSP_STREET": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-STREET",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STREET_0430",
    "legacy_field": "HOSP_STREET_0430"
  },
  "HOSP_CITY_0430": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_0430",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP_CITY": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_0430",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP_STATE_0430": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_0430",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP_STATE": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_0430",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP_ZIP_FIRST_FIVE_0430": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIP-FIRST-FIVE",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIP_FIRST_FIVE_0430",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSP_ZIP_FIRST_FIVE": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIP-FIRST-FIVE",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIP_FIRST_FIVE_0430",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSP_ZIP_LAST_FOUR_0430": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIP-LAST-FOUR",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIP_LAST_FOUR_0430",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP_ZIP_LAST_FOUR": {
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIP-LAST-FOUR",

```

```

    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIP_LAST_FOUR_0430",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "DIAGNOSIS_0430": {
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_0430",
    "legacy_field": "DIAGNOSIS_0430"
  },
  "DIAGNOSIS": {
    "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DIAGNOSIS_0445",
    "legacy_field": "DIAGNOSIS_0445"
  },
  "WARD_DAYS_0430": {
    "host": "HV-HOSPITAL-CLAIM-WARD-DAYS",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_DAYS_0430",
    "legacy_field": "WARD_DAYS_0430"
  },
  "WARD_DAYS": {
    "host": "HV-HOSPITAL-CLAIM-WARD-DAYS",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_DAYS_0430",
    "legacy_field": "WARD_DAYS_0430"
  },
  "WARD_RATE_0430": {
    "host": "HV-HOSPITAL-CLAIM-WARD-RATE",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_RATE_0430",
    "legacy_field": "WARD_RATE_0430"
  },
  "WARD_RATE": {
    "host": "HV-HOSPITAL-CLAIM-WARD-RATE",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_RATE_0430",
    "legacy_field": "WARD_RATE_0430"
  },
  "WARD_TOTAL_0430": {
    "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_TOTAL_0430",
    "legacy_field": "WARD_TOTAL_0430"
  },
  "WARD_TOTAL": {
    "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_TOTAL_0430",
    "legacy_field": "WARD_TOTAL_0430"
  },
  "SEMI_DAYS_0430": {
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_0430",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI_DAYS": {
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_0430",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI_RATE_0430": {

```

```

    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_0430",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI_RATE": {
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_0430",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI_TOTAL_0430": {
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_0430",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMI_TOTAL": {
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_0430",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "DELIVERY_COST_0430": {
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_0430",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERY_COST": {
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_0430",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "ANESTHESIA_COST_0430": {
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_0430",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIA_COST": {
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_0430",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "LAB_COST_0430": {
    "host": "HV-HOSPITAL-CLAIM-LAB-COST",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_0430",
    "legacy_field": "LAB_COST_0430"
  },
  "LAB_COST": {
    "host": "HV-HOSPITAL-CLAIM-LAB-COST",
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_0430",
    "legacy_field": "LAB_COST_0430"
  },
  "INS_PLAN_CODE_0435": {
    "host": "HV-INSURANCE-PLAN-INS-PLAN-CODE",
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLAN_CODE_0435",
    "legacy_field": "INS_PLAN_CODE_0435"
  },

```

```

"INS_CO_NAME_0435": {
  "host": "HV-INSURANCE-PLAN-INS-CO-NAME",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_NAME_0435",
  "legacy_field": "INS_CO_NAME_0435"
},
"INS_CO_NAME": {
  "host": "HV-INSURANCE-PLAN-INS-CO-NAME",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_NAME_0435",
  "legacy_field": "INS_CO_NAME_0435"
},
"INS_CO_STREET_0435": {
  "host": "HV-INSURANCE-PLAN-INS-CO-STREET",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_STREET_0435",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_CO_STREET": {
  "host": "HV-INSURANCE-PLAN-INS-CO-STREET",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_STREET_0435",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_CO_CITY_0435": {
  "host": "HV-INSURANCE-PLAN-INS-CO-CITY",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_CITY_0435",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_CO_CITY": {
  "host": "HV-INSURANCE-PLAN-INS-CO-CITY",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_CITY_0435",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_CO_STATE_0435": {
  "host": "HV-INSURANCE-PLAN-INS-CO-STATE",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_STATE_0435",
  "legacy_field": "INS_CO_STATE_0435"
},
"INS_CO_STATE": {
  "host": "HV-INSURANCE-PLAN-INS-CO-STATE",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_STATE_0435",
  "legacy_field": "INS_CO_STATE_0435"
},
"INS_CO_ZIP_FIRST_FIVE_0435": {
  "host": "HV-INSURANCE-PLAN-INS-CO-ZIP-FIRST-FIVE",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_ZIP_FIRST_FIVE_0435",
  "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
},
"INS_CO_ZIP_FIRST_FIVE": {
  "host": "HV-INSURANCE-PLAN-INS-CO-ZIP-FIRST-FIVE",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_ZIP_FIRST_FIVE_0435",
  "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
},
"INS_CO_ZIP_LAST_FOUR_0435": {
  "host": "HV-INSURANCE-PLAN-INS-CO-ZIP-LAST-FOUR",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_ZIP_LAST_FOUR_0435",
  "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
}

```

```

},
"INS_CO_ZIP_LAST_FOUR": {
  "host": "HV-INSURANCE-PLAN-INS-CO-ZIP-LAST-FOUR",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_ZIP_LAST_FOUR_0435",
  "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
},
"INS_CO_PHONE_0435": {
  "host": "HV-INSURANCE-PLAN-INS-CO-PHONE",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_PHONE_0435",
  "legacy_field": "INS_CO_PHONE_0435"
},
"INS_CO_PHONE": {
  "host": "HV-INSURANCE-PLAN-INS-CO-PHONE",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CO_PHONE_0435",
  "legacy_field": "INS_CO_PHONE_0435"
},
"GROUP_NUMBER_0435": {
  "host": "HV-INSURANCE-PLAN-GROUP-NUMBER",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "GROUP_NUMBER_0435",
  "legacy_field": "GROUP_NUMBER_0435"
},
"GROUP_NUMBER": {
  "host": "HV-INSURANCE-PLAN-GROUP-NUMBER",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "GROUP_NUMBER_0435",
  "legacy_field": "GROUP_NUMBER_0435"
},
"DEDUCT_0435": {
  "host": "HV-INSURANCE-PLAN-DEDUCT",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEDUCT_0435",
  "legacy_field": "DEDUCT_0435"
},
"DEDUCT": {
  "host": "HV-INSURANCE-PLAN-DEDUCT",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEDUCT_0435",
  "legacy_field": "DEDUCT_0435"
},
"MAXIMUM_LIFE_COST_0435": {
  "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFE-COST",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "MAXIMUM_LIFE_COST_0435",
  "legacy_field": "MAXIMUM_LIFE_COST_0435"
},
"MAXIMUM_LIFE_COST": {
  "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFE-COST",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "MAXIMUM_LIFE_COST_0435",
  "legacy_field": "MAXIMUM_LIFE_COST_0435"
},
"FAMILY_COST_0435": {
  "host": "HV-INSURANCE-PLAN-FAMILY-COST",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_0435",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILY_COST": {
  "host": "HV-INSURANCE-PLAN-FAMILY-COST",
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_0435",

```

```

    "legacy_field": "FAMILY_COST_0435"
  },
  "DEP_COST_0435": {
    "host": "HV-INSURANCE-PLAN-DEP-COST",
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEP_COST_0435",
    "legacy_field": "DEP_COST_0435"
  },
  "DEP_COST": {
    "host": "HV-INSURANCE-PLAN-DEP-COST",
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEP_COST_0435",
    "legacy_field": "DEP_COST_0435"
  },
  "JOB_ID_0440": {
    "host": "HV-JOB-JOB-ID",
    "record": "JOB",
    "table": "JOB",
    "column": "JOB_ID_0440",
    "legacy_field": "JOB_ID_0440"
  },
  "JOB_ID": {
    "host": "HV-JOB-JOB-ID",
    "record": "JOB",
    "table": "JOB",
    "column": "JOB_ID_0440",
    "legacy_field": "JOB_ID_0440"
  },
  "TITLE_0440": {
    "host": "HV-JOB-TITLE",
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_0440",
    "legacy_field": "TITLE_0440"
  },
  "TITLE": {
    "host": "HV-JOB-TITLE",
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_0440",
    "legacy_field": "TITLE_0440"
  },
  "DESCRIPTION_LINE_0440": {
    "host": "HV-JOB-DESCRIPTION-LINE",
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_0440",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "DESCRIPTION_LINE": {
    "host": "HV-JOB-DESCRIPTION-LINE",
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_0440",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "REQUIREMENT_LINE_0440": {
    "host": "HV-JOB-REQUIREMENT-LINE",
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_0440",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENT_LINE": {
    "host": "HV-JOB-REQUIREMENT-LINE",
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_0440",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "MINIMUM_SALARY_0440": {
    "host": "HV-JOB-MINIMUM-SALARY",
    "record": "JOB",
    "table": "JOB",

```

```

    "column": "MINIMUM_SALARY_0440",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUM_SALARY": {
    "host": "HV-JOB-MINIMUM-SALARY",
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_0440",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MAXIMUM_SALARY_0440": {
    "host": "HV-JOB-MAXIMUM-SALARY",
    "record": "JOB",
    "table": "JOB",
    "column": "MAXIMUM_SALARY_0440",
    "legacy_field": "MAXIMUM_SALARY_0440"
  },
  "MAXIMUM_SALARY": {
    "host": "HV-JOB-MAXIMUM-SALARY",
    "record": "JOB",
    "table": "JOB",
    "column": "MAXIMUM_SALARY_0440",
    "legacy_field": "MAXIMUM_SALARY_0440"
  },
  "SALARY_GRADES_0440": {
    "host": "HV-JOB-SALARY-GRADES",
    "record": "JOB",
    "table": "JOB",
    "column": "SALARY_GRADES_0440",
    "legacy_field": "SALARY_GRADES_0440"
  },
  "SALARY_GRADES": {
    "host": "HV-JOB-SALARY-GRADES",
    "record": "JOB",
    "table": "JOB",
    "column": "SALARY_GRADES_0440",
    "legacy_field": "SALARY_GRADES_0440"
  },
  "NUMBER_OF_POSITIONS_0440": {
    "host": "HV-JOB-NUMBER-OF-POSITIONS",
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OF_POSITIONS_0440",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBER_OF_POSITIONS": {
    "host": "HV-JOB-NUMBER-OF-POSITIONS",
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OF_POSITIONS_0440",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBER_OPEN_0440": {
    "host": "HV-JOB-NUMBER-OPEN",
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OPEN_0440",
    "legacy_field": "NUMBER_OPEN_0440"
  },
  "NUMBER_OPEN": {
    "host": "HV-JOB-NUMBER-OPEN",
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OPEN_0440",
    "legacy_field": "NUMBER_OPEN_0440"
  },
  "PATIENT_FIRST_NAME_0445": {
    "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRST-NAME",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_FIRST_NAME_0445",
    "legacy_field": "PATIENT_FIRST_NAME_0445"
  },
  "PATIENT_LAST_NAME_0445": {
    "host": "HV-NON-HOSP-CLAIM-PATIENT-LAST-NAME",
    "record": "NON_HOSP_CLAIM",

```

```

    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_LAST_NAME_0445",
    "legacy_field": "PATIENT_LAST_NAME_0445"
  },
  "PATIENT_SEX_0445": {
    "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_SEX_0445",
    "legacy_field": "PATIENT_SEX_0445"
  },
  "RELATION_TO_EMPLOYEE_0445": {
    "host": "HV-NON-HOSP-CLAIM-RELATION-TO-EMPLOYEE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "RELATION_TO_EMPLOYEE_0445",
    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "PHYSICIAN_FIRST_NAME_0445": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRST-NAME",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRST_NAME_0445",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_FIRST_NAME": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRST-NAME",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRST_NAME_0445",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_LAST_NAME_0445": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LAST-NAME",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LAST_NAME_0445",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_LAST_NAME": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LAST-NAME",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LAST_NAME_0445",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_STREET_0445": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_0445",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN_STREET": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_0445",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN_CITY_0445": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_0445",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN_CITY": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_0445",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN_STATE_0445": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE",

```



```

    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_0445",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN_STATE": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_0445",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN_ZIP_FIRST_FIVE_0445": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIP-FIRST-FIVE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIP_FIRST_FIVE_0445",
    "legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
  },
  "PHYSICIAN_ZIP_FIRST_FIVE": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIP-FIRST-FIVE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIP_FIRST_FIVE_0445",
    "legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
  },
  "PHYSICIAN_ZIP_LAST_FOUR_0445": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIP-LAST-FOUR",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIP_LAST_FOUR_0445",
    "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
  },
  "PHYSICIAN_ZIP_LAST_FOUR": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIP-LAST-FOUR",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIP_LAST_FOUR_0445",
    "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
  },
  "PHYSICIAN_ID_0445": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ID_0445",
    "legacy_field": "PHYSICIAN_ID_0445"
  },
  "PHYSICIAN_ID": {
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ID_0445",
    "legacy_field": "PHYSICIAN_ID_0445"
  },
  "DIAGNOSIS_0445": {
    "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DIAGNOSIS_0445",
    "legacy_field": "DIAGNOSIS_0445"
  },
  "NUMBER_OF_PROCEDURES_0445": {
    "host": "HV-NON-HOSP-CLAIM-NUMBER-OF-PROCEDURES",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "NUMBER_OF_PROCEDURES_0445",
    "legacy_field": "NUMBER_OF_PROCEDURES_0445"
  },
  "PROCEDURE_CODE_0445": {
    "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PROCEDURE_CODE_0445",
    "legacy_field": "PROCEDURE_CODE_0445"
  },
  "DESCRIPTION_OF_SERVICE_0445": {

```

```

    "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OF-SERVICE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DESCRIPTION_OF_SERVICE_0445",
    "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
  },
  "FEE_0445": {
    "host": "HV-NON-HOSP-CLAIM-FEE",
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_0445",
    "legacy_field": "FEE_0445"
  },
  "OFFICE_CODE_0450": {
    "host": "HV-OFFICE-OFFICE-CODE",
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_0450",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE_CODE": {
    "host": "HV-OFFICE-OFFICE-CODE",
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_0450",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE_STREET_0450": {
    "host": "HV-OFFICE-OFFICE-STREET",
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_0450",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE_STREET": {
    "host": "HV-OFFICE-OFFICE-STREET",
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_0450",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE_CITY_0450": {
    "host": "HV-OFFICE-OFFICE-CITY",
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_0450",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE_CITY": {
    "host": "HV-OFFICE-OFFICE-CITY",
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_0450",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE_STATE_0450": {
    "host": "HV-OFFICE-OFFICE-STATE",
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_0450",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE_STATE": {
    "host": "HV-OFFICE-OFFICE-STATE",
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_0450",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE_ZIP_FIRST_FIVE_0450": {
    "host": "HV-OFFICE-OFFICE-ZIP-FIRST-FIVE",
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIP_FIRST_FIVE_0450",
    "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
  },

```

```

"OFFICE_ZIP_FIRST_FIVE": {
  "host": "HV-OFFICE-OFFICE-ZIP-FIRST-FIVE",
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_ZIP_FIRST_FIVE_0450",
  "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
},
"OFFICE_ZIP_LAST_FOUR_0450": {
  "host": "HV-OFFICE-OFFICE-ZIP-LAST-FOUR",
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_ZIP_LAST_FOUR_0450",
  "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
},
"OFFICE_ZIP_LAST_FOUR": {
  "host": "HV-OFFICE-OFFICE-ZIP-LAST-FOUR",
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_ZIP_LAST_FOUR_0450",
  "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
},
"OFFICE_PHONE_0450": {
  "host": "HV-OFFICE-OFFICE-PHONE",
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_0450",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE_PHONE": {
  "host": "HV-OFFICE-OFFICE-PHONE",
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_0450",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE_AREA_CODE_0450": {
  "host": "HV-OFFICE-OFFICE-AREA-CODE",
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREA_CODE_0450",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICE_AREA_CODE": {
  "host": "HV-OFFICE-OFFICE-AREA-CODE",
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREA_CODE_0450",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"SPEED_DIAL_0450": {
  "host": "HV-OFFICE-SPEED-DIAL",
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_0450",
  "legacy_field": "SPEED_DIAL_0450"
},
"SPEED_DIAL": {
  "host": "HV-OFFICE-SPEED-DIAL",
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_0450",
  "legacy_field": "SPEED_DIAL_0450"
},
"SKILL_ID_0455": {
  "host": "HV-SKILL-SKILL-ID",
  "record": "SKILL",
  "table": "SKILL",
  "column": "SKILL_ID_0455",
  "legacy_field": "SKILL_ID_0455"
},
"SKILL_ID": {
  "host": "HV-SKILL-SKILL-ID",
  "record": "SKILL",
  "table": "SKILL",
  "column": "SKILL_ID_0455",
  "legacy_field": "SKILL_ID_0455"
}

```

```

    },
    "SKILL_NAME_0455": {
        "host": "HV-SKILL-SKILL-NAME",
        "record": "SKILL",
        "table": "SKILL",
        "column": "SKILL_NAME_0455",
        "legacy_field": "SKILL_NAME_0455"
    },
    "SKILL_NAME": {
        "host": "HV-SKILL-SKILL-NAME",
        "record": "SKILL",
        "table": "SKILL",
        "column": "SKILL_NAME_0455",
        "legacy_field": "SKILL_NAME_0455"
    },
    "SKILL_DESCRIPTION_0455": {
        "host": "HV-SKILL-SKILL-DESCRIPTION",
        "record": "SKILL",
        "table": "SKILL",
        "column": "SKILL_DESCRIPTION_0455",
        "legacy_field": "SKILL_DESCRIPTION_0455"
    },
    "SKILL_DESCRIPTION": {
        "host": "HV-SKILL-SKILL-DESCRIPTION",
        "record": "SKILL",
        "table": "SKILL",
        "column": "SKILL_DESCRIPTION_0455",
        "legacy_field": "SKILL_DESCRIPTION_0455"
    },
    "STRUCTURE_CODE_0460": {
        "host": "HV-STRUCTURE-STRUCTURE-CODE",
        "record": "STRUCTURE",
        "table": "STRUCTURE",
        "column": "STRUCTURE_CODE_0460",
        "legacy_field": "STRUCTURE_CODE_0460"
    },
    "STRUCTURE_CODE": {
        "host": "HV-STRUCTURE-STRUCTURE-CODE",
        "record": "STRUCTURE",
        "table": "STRUCTURE",
        "column": "STRUCTURE_CODE_0460",
        "legacy_field": "STRUCTURE_CODE_0460"
    }
},
"calc_key_map": {
    "DEPARTMENT": {
        "record": "DEPARTMENT",
        "table": "DEPARTMENT",
        "key": "DEPT_ID_0410",
        "host": "HV-DEPARTMENT-DEPT-ID"
    },
    "EMPLOYEE": {
        "record": "EMPLOYEE",
        "table": "EMPLOYEE",
        "key": "EMP_ID_0415",
        "host": "HV-EMPLOYEE-EMP-ID"
    },
    "INSURANCE_PLAN": {
        "record": "INSURANCE_PLAN",
        "table": "INSURANCE_PLAN",
        "key": "INS_PLAN_CODE_0435",
        "host": "HV-INSURANCE-PLAN-INS-PLAN-CODE"
    },
    "JOB": {
        "record": "JOB",
        "table": "JOB",
        "key": "JOB_ID_0440",
        "host": "HV-JOB-JOB-ID"
    },
    "OFFICE": {
        "record": "OFFICE",
        "table": "OFFICE",
        "key": "OFFICE_CODE_0450",
        "host": "HV-OFFICE-OFFICE-CODE"
    },
    "SKILL": {

```

```

    "record": "SKILL",
    "table": "SKILL",
    "key": "SKILL_ID_0455",
    "host": "HV-SKILL-SKILL-ID"
  }
},
"set_ordering_map": {
  "DEPT_EMPLOYEE": {
    "order_by": [
      "DEPT_ID_0410"
    ]
  },
  "MANAGES": {
    "order_by": []
  },
  "OFFICE_EMPLOYEE": {
    "order_by": [
      "OFFICE_CODE_0450"
    ]
  },
  "EMP_EXPERTISE": {
    "order_by": [
      "EMP_ID_0415"
    ]
  },
  "SKILL_EXPERTISE": {
    "order_by": [
      "SKILL_ID_0455"
    ]
  },
  "EMP_EMPOSITION": {
    "order_by": [
      "EMP_ID_0415"
    ]
  },
  "REPORTS_TO": {
    "order_by": []
  },
  "EMP_COVERAGE": {
    "order_by": [
      "EMP_ID_0415"
    ]
  },
  "JOB_EMPOSITION": {
    "order_by": [
      "JOB_ID_0440"
    ]
  },
  "COVERAGE_CLAIMS": {
    "order_by": [
      "COVERAGE_ID"
    ]
  }
},
"navigation_intent": {
  "DEPT_EMPLOYEE": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "DEPARTMENT",
    "child_record": "EMPLOYEE",
    "parent_key": "DEPT_ID_0410",
    "child_fk": "DEPT_ID_0410",
    "order_by": [
      "DEPT_ID_0410"
    ]
  },
  "MANAGES": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "parent_key": "EMP_ID_0415",
    "child_fk": null,
    "order_by": []
  },
  "OFFICE_EMPLOYEE": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "OFFICE",

```

```

    "child_record": "EMPLOYEE",
    "parent_key": "OFFICE_CODE_0450",
    "child_fk": "OFFICE_CODE_0450",
    "order_by": [
        "OFFICE_CODE_0450"
    ]
},
"EMP_EXPERTISE": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "EMPLOYEE",
    "child_record": "EXPERTISE",
    "parent_key": "EMP_ID_0415",
    "child_fk": "EMP_ID_0415",
    "order_by": [
        "EMP_ID_0415"
    ]
},
"SKILL_EXPERTISE": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "SKILL",
    "child_record": "EXPERTISE",
    "parent_key": "SKILL_ID_0455",
    "child_fk": "SKILL_ID_0455",
    "order_by": [
        "SKILL_ID_0455"
    ]
},
"EMP_EMPOSITION": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "EMPLOYEE",
    "child_record": "EMPOSITION",
    "parent_key": "EMP_ID_0415",
    "child_fk": "EMP_ID_0415",
    "order_by": [
        "EMP_ID_0415"
    ]
},
"REPORTS_TO": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "parent_key": "EMP_ID_0415",
    "child_fk": null,
    "order_by": []
},
"EMP_COVERAGE": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "EMPLOYEE",
    "child_record": "COVERAGE",
    "parent_key": "EMP_ID_0415",
    "child_fk": "EMP_ID_0415",
    "order_by": [
        "EMP_ID_0415"
    ]
},
"JOB_EMPOSITION": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "JOB",
    "child_record": "EMPOSITION",
    "parent_key": "JOB_ID_0440",
    "child_fk": "JOB_ID_0440",
    "order_by": [
        "JOB_ID_0440"
    ]
},
"COVERAGE_CLAIMS": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "COVERAGE",
    "child_record": "NON_HOSP_CLAIM",
    "parent_key": "COVERAGE_ID",
    "child_fk": "COVERAGE_ID",
    "order_by": [
        "COVERAGE_ID"
    ]
}
},

```

```

"nullable_fk_map": {
  "COVERAGE.EMP_ID_0415": true,
  "COVERAGE.INS_PLAN_CODE_0400": true,
  "DENTAL_CLAIM.COVERAGE_ID": true,
  "EMPLOYEE.DEPT_ID_0410": true,
  "EMPLOYEE.OFFICE_CODE_0450": true,
  "EMPOSITION.EMP_ID_0415": true,
  "EMPOSITION.JOB_ID_0440": true,
  "EXPERTISE.EMP_ID_0415": true,
  "EXPERTISE.SKILL_ID_0455": true,
  "HOSPITAL_CLAIM.COVERAGE_ID": true,
  "NON_HOSP_CLAIM.COVERAGE_ID": true,
  "STRUCTURE.MANAGES_EMP_ID_0415": true,
  "STRUCTURE.REPORTS_TO_EMP_ID_0415": true
},
"date_part_map": {
  "SELECTION_YEAR_0400": {
    "host": "HV-COVERAGE-SELECTION-DATE",
    "table": "COVERAGE",
    "column": "SELECTION_DATE_0400",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "SELECTION_MONTH_0400": {
    "host": "HV-COVERAGE-SELECTION-DATE",
    "table": "COVERAGE",
    "column": "SELECTION_DATE_0400",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "SELECTION_DAY_0400": {
    "host": "HV-COVERAGE-SELECTION-DATE",
    "table": "COVERAGE",
    "column": "SELECTION_DATE_0400",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "TERMINATION_YEAR_0400": {
    "host": "HV-COVERAGE-TERMINATION-DATE",
    "table": "COVERAGE",
    "column": "TERMINATION_DATE_0400",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "TERMINATION_MONTH_0400": {
    "host": "HV-COVERAGE-TERMINATION-DATE",
    "table": "COVERAGE",
    "column": "TERMINATION_DATE_0400",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "TERMINATION_DAY_0400": {
    "host": "HV-COVERAGE-TERMINATION-DATE",
    "table": "COVERAGE",
    "column": "TERMINATION_DATE_0400",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "CLAIM_YEAR_0405": {
    "host": "HV-DENTAL-CLAIM-CLAIM-DATE",
    "table": "DENTAL_CLAIM",
    "column": "CLAIM_DATE_0405",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "CLAIM_MONTH_0405": {
    "host": "HV-DENTAL-CLAIM-CLAIM-DATE",
    "table": "DENTAL_CLAIM",
    "column": "CLAIM_DATE_0405",

```

```

    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "CLAIM_DAY_0405": {
    "host": "HV-DENTAL-CLAIM-CLAIM-DATE",
    "table": "DENTAL_CLAIM",
    "column": "CLAIM_DATE_0405",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "PATIENT_BIRTH_YEAR_0405": {
    "host": "HV-DENTAL-CLAIM-PATIENT-BIRTH-DATE",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_BIRTH_DATE_0405",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "PATIENT_BIRTH_MONTH_0405": {
    "host": "HV-DENTAL-CLAIM-PATIENT-BIRTH-DATE",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_BIRTH_DATE_0405",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "PATIENT_BIRTH_DAY_0405": {
    "host": "HV-DENTAL-CLAIM-PATIENT-BIRTH-DATE",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_BIRTH_DATE_0405",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "SERVICE_YEAR_0405": {
    "host": "HV-DENTAL-CLAIM-SERVICE-DATE",
    "table": "DENTAL_CLAIM",
    "column": "SERVICE_DATE_0405",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "SERVICE_MONTH_0405": {
    "host": "HV-DENTAL-CLAIM-SERVICE-DATE",
    "table": "DENTAL_CLAIM",
    "column": "SERVICE_DATE_0405",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "SERVICE_DAY_0405": {
    "host": "HV-DENTAL-CLAIM-SERVICE-DATE",
    "table": "DENTAL_CLAIM",
    "column": "SERVICE_DATE_0405",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "START_YEAR_0415": {
    "host": "HV-EMPLOYEE-START-DATE",
    "table": "EMPLOYEE",
    "column": "START_DATE_0415",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "START_MONTH_0415": {
    "host": "HV-EMPLOYEE-START-DATE",
    "table": "EMPLOYEE",
    "column": "START_DATE_0415",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },

```



```

"START_DAY_0415": {
  "host": "HV-EMPLOYEE-START-DATE",
  "table": "EMPLOYEE",
  "column": "START_DATE_0415",
  "date_part": "DAY",
  "substring_start": 9,
  "substring_length": 2
},
"TERMINATION_YEAR_0415": {
  "host": "HV-EMPLOYEE-TERMINATION-DATE",
  "table": "EMPLOYEE",
  "column": "TERMINATION_DATE_0415",
  "date_part": "YEAR",
  "substring_start": 3,
  "substring_length": 2
},
"TERMINATION_MONTH_0415": {
  "host": "HV-EMPLOYEE-TERMINATION-DATE",
  "table": "EMPLOYEE",
  "column": "TERMINATION_DATE_0415",
  "date_part": "MONTH",
  "substring_start": 6,
  "substring_length": 2
},
"TERMINATION_DAY_0415": {
  "host": "HV-EMPLOYEE-TERMINATION-DATE",
  "table": "EMPLOYEE",
  "column": "TERMINATION_DATE_0415",
  "date_part": "DAY",
  "substring_start": 9,
  "substring_length": 2
},
"BIRTH_YEAR_0415": {
  "host": "HV-EMPLOYEE-BIRTH-DATE",
  "table": "EMPLOYEE",
  "column": "BIRTH_DATE_0415",
  "date_part": "YEAR",
  "substring_start": 3,
  "substring_length": 2
},
"BIRTH_MONTH_0415": {
  "host": "HV-EMPLOYEE-BIRTH-DATE",
  "table": "EMPLOYEE",
  "column": "BIRTH_DATE_0415",
  "date_part": "MONTH",
  "substring_start": 6,
  "substring_length": 2
},
"BIRTH_DAY_0415": {
  "host": "HV-EMPLOYEE-BIRTH-DATE",
  "table": "EMPLOYEE",
  "column": "BIRTH_DATE_0415",
  "date_part": "DAY",
  "substring_start": 9,
  "substring_length": 2
},
"START_YEAR_0420": {
  "host": "HV-EMPOSITION-START-DATE",
  "table": "EMPOSITION",
  "column": "START_DATE_0420",
  "date_part": "YEAR",
  "substring_start": 3,
  "substring_length": 2
},
"START_MONTH_0420": {
  "host": "HV-EMPOSITION-START-DATE",
  "table": "EMPOSITION",
  "column": "START_DATE_0420",
  "date_part": "MONTH",
  "substring_start": 6,
  "substring_length": 2
},
"START_DAY_0420": {
  "host": "HV-EMPOSITION-START-DATE",
  "table": "EMPOSITION",
  "column": "START_DATE_0420",

```

```

    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "FINISH_YEAR_0420": {
    "host": "HV-EMPOSITION-FINISH-DATE",
    "table": "EMPOSITION",
    "column": "FINISH_DATE_0420",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "FINISH_MONTH_0420": {
    "host": "HV-EMPOSITION-FINISH-DATE",
    "table": "EMPOSITION",
    "column": "FINISH_DATE_0420",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "FINISH_DAY_0420": {
    "host": "HV-EMPOSITION-FINISH-DATE",
    "table": "EMPOSITION",
    "column": "FINISH_DATE_0420",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "EXPERTISE_YEAR_0425": {
    "host": "HV-EXPERTISE-EXPERTISE-DATE",
    "table": "EXPERTISE",
    "column": "EXPERTISE_DATE_0425",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "EXPERTISE_MONTH_0425": {
    "host": "HV-EXPERTISE-EXPERTISE-DATE",
    "table": "EXPERTISE",
    "column": "EXPERTISE_DATE_0425",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "EXPERTISE_DAY_0425": {
    "host": "HV-EXPERTISE-EXPERTISE-DATE",
    "table": "EXPERTISE",
    "column": "EXPERTISE_DATE_0425",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "CLAIM_YEAR_0430": {
    "host": "HV-HOSPITAL-CLAIM-CLAIM-DATE",
    "table": "HOSPITAL_CLAIM",
    "column": "CLAIM_DATE_0430",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "CLAIM_MONTH_0430": {
    "host": "HV-HOSPITAL-CLAIM-CLAIM-DATE",
    "table": "HOSPITAL_CLAIM",
    "column": "CLAIM_DATE_0430",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "CLAIM_DAY_0430": {
    "host": "HV-HOSPITAL-CLAIM-CLAIM-DATE",
    "table": "HOSPITAL_CLAIM",
    "column": "CLAIM_DATE_0430",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },

```

```

"PATIENT_BIRTH_YEAR_0430": {
  "host": "HV-HOSPITAL-CLAIM-PATIENT-BIRTH-DATE",
  "table": "HOSPITAL_CLAIM",
  "column": "PATIENT_BIRTH_DATE_0430",
  "date_part": "YEAR",
  "substring_start": 3,
  "substring_length": 2
},
"PATIENT_BIRTH_MONTH_0430": {
  "host": "HV-HOSPITAL-CLAIM-PATIENT-BIRTH-DATE",
  "table": "HOSPITAL_CLAIM",
  "column": "PATIENT_BIRTH_DATE_0430",
  "date_part": "MONTH",
  "substring_start": 6,
  "substring_length": 2
},
"PATIENT_BIRTH_DAY_0430": {
  "host": "HV-HOSPITAL-CLAIM-PATIENT-BIRTH-DATE",
  "table": "HOSPITAL_CLAIM",
  "column": "PATIENT_BIRTH_DATE_0430",
  "date_part": "DAY",
  "substring_start": 9,
  "substring_length": 2
},
"ADMIT_YEAR_0430": {
  "host": "HV-HOSPITAL-CLAIM-ADMIT-DATE",
  "table": "HOSPITAL_CLAIM",
  "column": "ADMIT_DATE_0430",
  "date_part": "YEAR",
  "substring_start": 3,
  "substring_length": 2
},
"ADMIT_MONTH_0430": {
  "host": "HV-HOSPITAL-CLAIM-ADMIT-DATE",
  "table": "HOSPITAL_CLAIM",
  "column": "ADMIT_DATE_0430",
  "date_part": "MONTH",
  "substring_start": 6,
  "substring_length": 2
},
"ADMIT_DAY_0430": {
  "host": "HV-HOSPITAL-CLAIM-ADMIT-DATE",
  "table": "HOSPITAL_CLAIM",
  "column": "ADMIT_DATE_0430",
  "date_part": "DAY",
  "substring_start": 9,
  "substring_length": 2
},
"DISCHARGE_YEAR_0430": {
  "host": "HV-HOSPITAL-CLAIM-DISCHARGE-DATE",
  "table": "HOSPITAL_CLAIM",
  "column": "DISCHARGE_DATE_0430",
  "date_part": "YEAR",
  "substring_start": 3,
  "substring_length": 2
},
"DISCHARGE_MONTH_0430": {
  "host": "HV-HOSPITAL-CLAIM-DISCHARGE-DATE",
  "table": "HOSPITAL_CLAIM",
  "column": "DISCHARGE_DATE_0430",
  "date_part": "MONTH",
  "substring_start": 6,
  "substring_length": 2
},
"DISCHARGE_DAY_0430": {
  "host": "HV-HOSPITAL-CLAIM-DISCHARGE-DATE",
  "table": "HOSPITAL_CLAIM",
  "column": "DISCHARGE_DATE_0430",
  "date_part": "DAY",
  "substring_start": 9,
  "substring_length": 2
},
"CLAIM_YEAR_0445": {
  "host": "HV-NON-HOSP-CLAIM-CLAIM-DATE",
  "table": "NON_HOSP_CLAIM",
  "column": "CLAIM_DATE_0445",

```

```

    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "CLAIM_MONTH_0445": {
    "host": "HV-NON-HOSP-CLAIM-CLAIM-DATE",
    "table": "NON_HOSP_CLAIM",
    "column": "CLAIM_DATE_0445",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "CLAIM_DAY_0445": {
    "host": "HV-NON-HOSP-CLAIM-CLAIM-DATE",
    "table": "NON_HOSP_CLAIM",
    "column": "CLAIM_DATE_0445",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "PATIENT_BIRTH_YEAR_0445": {
    "host": "HV-NON-HOSP-CLAIM-PATIENT-BIRTH-DATE",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_BIRTH_DATE_0445",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "PATIENT_BIRTH_MONTH_0445": {
    "host": "HV-NON-HOSP-CLAIM-PATIENT-BIRTH-DATE",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_BIRTH_DATE_0445",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "PATIENT_BIRTH_DAY_0445": {
    "host": "HV-NON-HOSP-CLAIM-PATIENT-BIRTH-DATE",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_BIRTH_DATE_0445",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "SERVICE_YEAR_0445": {
    "host": "HV-NON-HOSP-CLAIM-SERVICE-DATE",
    "table": "NON_HOSP_CLAIM",
    "column": "SERVICE_DATE_0445",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },
  "SERVICE_MONTH_0445": {
    "host": "HV-NON-HOSP-CLAIM-SERVICE-DATE",
    "table": "NON_HOSP_CLAIM",
    "column": "SERVICE_DATE_0445",
    "date_part": "MONTH",
    "substring_start": 6,
    "substring_length": 2
  },
  "SERVICE_DAY_0445": {
    "host": "HV-NON-HOSP-CLAIM-SERVICE-DATE",
    "table": "NON_HOSP_CLAIM",
    "column": "SERVICE_DATE_0445",
    "date_part": "DAY",
    "substring_start": 9,
    "substring_length": 2
  },
  "STRUCTURE_YEAR_0460": {
    "host": "HV-STRUCTURE-STRUCTURE-DATE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_DATE_0460",
    "date_part": "YEAR",
    "substring_start": 3,
    "substring_length": 2
  },

```

```

"STRUCTURE_MONTH_0460": {
  "host": "HV-STRUCTURE-STRUCTURE-DATE",
  "table": "STRUCTURE",
  "column": "STRUCTURE_DATE_0460",
  "date_part": "MONTH",
  "substring_start": 6,
  "substring_length": 2
},
"STRUCTURE_DAY_0460": {
  "host": "HV-STRUCTURE-STRUCTURE-DATE",
  "table": "STRUCTURE",
  "column": "STRUCTURE_DATE_0460",
  "date_part": "DAY",
  "substring_start": 9,
  "substring_length": 2
}
},
"output_semantics": {
  "generated_by": "idms-db2-modernizer-phase1",
  "usage": "IDMS retrieval COBOL to DB2 embedded SQL conversion",
  "physical_types_source": "DB2_DDL",
  "logical_mapping_source": "PHASE1_CANONICAL_METADATA"
},
"paragraph_operation_graph": {},
"validation_messages": []
}

```

```

=====
FILE: installed.txt
ENCODING: utf-8
=====

```

```

aistudio-sdk==0.3.8
altair==6.2.1
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.14.0
ast-serialize==0.5.0
astor==0.8.1
attrs==26.1.0
bce-python-sdk==0.9.72
blinker==1.9.0
cachetools==7.1.4
certifi==2026.6.17
cffi==2.0.0
cfgv==3.5.0
chardet==7.4.3
charset-normalizer==3.4.7
click==8.4.1
colorama==0.4.6
colorlog==6.10.1
coverage==7.14.1
crc32c==2.8
cryptography==49.0.0
decorator==5.3.1
distlib==0.4.3
easyocr==1.7.2
et-xmlfile==2.0.0
filelock==3.29.4
fsspec==2026.6.0
future==1.0.0
gitdb==4.0.12
gitpython==3.1.50
graphviz==0.21
h11==0.16.0
hf-xet==1.5.1
httpcore==1.0.9
httptools==0.8.0
httpx==0.28.1
huggingface-hub==1.19.0
identify==2.6.19
idna==3.18
imageio==2.37.3
imagesize==2.0.0
iniconfig==2.3.0

```

itsdangerous==2.2.0
jinja2==3.1.6
jsonschema==4.26.0
jsonschema-specifications==2025.9.1
lazy-loader==0.5
librt==0.11.0
markdown-it-py==4.2.0
markupsafe==3.0.3
mdurl==0.1.2
modelscope==1.37.1
mpmath==1.3.0
mypy==2.1.0
mypy-extensions==1.1.0
narwhals==2.22.1
networkx==3.6.1
ninja==1.13.0
nodeenv==1.10.0
numpy==2.4.6
opencv-contrib-python==4.10.0.84
opencv-python==4.13.0.92
opencv-python-headless==4.13.0.92
openpyxl==3.1.5
opt-einsum==3.3.0
packaging==26.2
paddleocr==3.4.1
paddlepaddle==3.0.0
paddlex==3.4.3
pandas==3.0.3
pathspec==1.1.1
pdfminer-six==20260107
pdfplumber==0.11.10
pillow==12.2.0
platformdirs==4.10.0
pluggy==1.6.0
pre-commit==4.6.0
prettytable==3.17.0
protobuf==7.35.1
psutil==7.2.2
py-cpuinfo==9.0.0
pyarrow==24.0.0
pyclipper==1.4.0
pycparser==3.0
pycryptodome==3.23.0
pydantic==2.13.4
pydantic-core==2.46.4
pydantic-settings==2.14.1
pydeck==0.9.2
pygments==2.20.0
pymupdf==1.27.2.3
pypdfium2==5.10.1
pytest==9.1.0
pytest-cov==7.1.0
pytest-mock==3.15.1
python-bidi==0.6.10
python-dateutil==2.9.0.post0
python-discovery==1.4.2
python-dotenv==1.2.2
python-multipart==0.0.32
pyyaml==6.0.2
referencing==0.37.0
reportlab==4.5.1
requests==2.34.2
rich==15.0.0
rpds-py==2026.5.1
ruamel-yaml==0.19.1
ruff==0.15.17
safetensors==0.8.0
scikit-image==0.26.0
scipy==1.17.1
setuptools==81.0.0
shapely==2.1.2
shellingham==1.5.4
six==1.17.0
smmap==5.0.3
starlette==1.3.1
streamlit==1.58.0

```

structlog==26.1.0
sympy==1.14.0
tenacity==9.1.4
tiffiffle==2026.6.1
toml==0.10.2
torch==2.12.1
torchvision==0.27.1
tqdm==4.68.3
typer==0.25.1
typing-extensions==4.15.0
typing-inspection==0.4.2
tzdata==2026.2
ujson==5.13.0
urllib3==2.7.0
uvicorn==0.49.0
virtualenv==21.5.1
watchdog==6.0.0
wcwidth==0.8.1
websockets==16.0

```

```

=====
FILE: main.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.services.metadata_service import (
    MetadataService
)

from idms_modernizer.services.validation_service import (
    ValidationService
)

from idms_modernizer.ui.metadata_page import (
    render_metadata
)

from idms_modernizer.ui.upload_page import (
    render_upload_page
)

metadata_service = MetadataService()

validator = ValidationService()

pdf_path = render_upload_page()

if pdf_path:
    metadata = (
        metadata_service.build_metadata(
            pdf_path
        )
    )

    validator.validate_metadata(
        metadata
    )

    render_metadata(
        metadata
    )

```

```

=====
FILE: pyproject.toml
ENCODING: utf-8
=====

```

```

[project]
name = "idms-db2-modernizer"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12"

```

```

dependencies = [
    "graphviz>=0.21",
    "networkx>=3.6.1",
    "pdfplumber>=0.11.10",
    "pydantic>=2.13.4",
    "pydantic-settings>=2.14.1",
    "streamlit>=1.58.0",
]

[tool.setuptools]
package-dir = {"" = "src"}

[tool.setuptools.packages.find]
where = ["src"]

[dependency-groups]
dev = [
    "pytest>=9.1.0",
    "pdfminer.six>=20250506",
]

=====
FILE: readme
ENCODING: utf-8
=====

set PYTHONPATH=src
python -m streamlit run src\idms_modernizer\app.py --server.port 8502

.venv\Scripts\Activate
python -m streamlit run app\streamlit_app.py

python src\tools\audit_hardcoding.py

python -m streamlit run app\streamlit_app.py --server.port 8502

streamlit run src\idms_modernizer\ui\schema_listing_only_app.py

--

=====
FILE: README.md
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_converter\__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_converter\exceptions.py
ENCODING: utf-8
=====

class ConversionError(Exception):
    """Raised when IDMS to DB2 conversion fails."""

```



```
=====
FILE: src\idms_db2_converter\generators\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_converter\generators\cobol_builder.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Protocol
```

```
class CobolNode(Protocol):
```

```
    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:
        ...
```

```
def pad(
    indent: int
) -> str:

    return " " * indent
```

```
def ensure_period(
    value: str
) -> str:

    value = value.rstrip()

    if value.endswith("."):
        return value

    return value + "."
```

```
@dataclass
class PreRenderedBlock:
```

```
    lines: list[str]

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result = list(
            self.lines
        )

        if final and result:
            result[-1] = ensure_period(
                result[-1]
            )

        return result
```

```
@dataclass
class RawLines:
```

```
    lines: list[str]
```

```

def render(
    self,
    indent: int = 7,
    final: bool = False
) -> list[str]:

    result: list[str] = []

    for line in self.lines:
        if line.strip():
            result.append(
                pad(indent) + line.strip()
            )
        else:
            result.append("")

    if final and result:
        result[-1] = ensure_period(
            result[-1]
        )

    return result

```

```

@dataclass
class Move:

    source: str
    target: str

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        return [
            f"{pad(indent)}MOVE {self.source} TO {self.target}."
        ]

```

```

@dataclass
class Perform:

    paragraph: str

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        return [
            f"{pad(indent)}PERFORM {self.paragraph}."
        ]

```

```

@dataclass
class Continue:

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        return [
            f"{pad(indent)}CONTINUE."
        ]

```

```

@dataclass
class ReadAtEndMove:

    file_name: str
    source: str

```

```

target: str

def render(
    self,
    indent: int = 7,
    final: bool = False
) -> list[str]:

    source = self.source

    if (
        len(source) == 1
        and not source.startswith('"')
        and not source.endswith('"')
    ):
        source = f"'{source}'"

    return [
        f"{pad(indent)}READ {self.file_name} AT END MOVE {source} TO {self.target}."
    ]

```

```

@dataclass
class Write:

    record: str
    source: str | None = None

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        if self.source:
            return [
                f"{pad(indent)}WRITE {self.record} FROM {self.source}."
            ]

        return [
            f"{pad(indent)}WRITE {self.record}."
        ]

```

```

@dataclass
class ExecSql:

    sql: str

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        normalized_lines = self._normalize_sql_lines(
            self.sql
        )

        result: list[str] = []

        for line in normalized_lines:
            if line.strip():
                result.append(
                    pad(indent) + line.rstrip()
                )
            else:
                result.append("")

        if final and result:
            result[-1] = ensure_period(
                result[-1]
            )

        return result

```

```

def _normalize_sql_lines(
    self,
    sql: str
) -> list[str]:

    raw_lines = sql.splitlines()

    stripped_lines = [
        line.rstrip()
        for line in raw_lines
        if line.strip()
    ]

    if not stripped_lines:
        return []

    min_indent = min(
        len(line) - len(line.lstrip(" "))
        for line in stripped_lines
    )

    normalized: list[str] = []

    for line in raw_lines:
        if line.strip():
            normalized.append(
                line[min_indent:].rstrip()
            )
        else:
            normalized.append("")

    return normalized

```

```

@dataclass
class IfBlock:

    condition: str
    then_body: list[CobolNode] = field(
        default_factory=list
    )
    else_body: list[CobolNode] = field(
        default_factory=list
    )

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = [
            f"{pad(indent)}IF {self.condition}"
        ]

        result.extend(
            render_nodes(
                self.then_body,
                indent + 3
            )
        )

        if self.else_body:
            result.append(
                f"{pad(indent)}ELSE"
            )

            result.extend(
                render_nodes(
                    self.else_body,
                    indent + 3
                )
            )

        terminator = "END-IF."

```

```

        if not final:
            terminator = "END-IF"

        result.append(
            f"{pad(indent)}{terminator}"
        )

        return result

def with_prefix(
    self,
    prefix: CobolNode
) -> PrefixedBlock:

    return PrefixedBlock(
        prefix=prefix,
        block=self
    )

```

```

@dataclass
class PrefixedBlock:

    prefix: CobolNode
    block: CobolNode

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = []

        result.extend(
            self.prefix.render(
                indent=indent
            )
        )

        result.append("")

        result.extend(
            self.block.render(
                indent=indent,
                final=final
            )
        )

        return result

```

```

@dataclass
class PerformUntil:

    condition: str
    body: list[CobolNode] = field(
        default_factory=list
    )

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = [
            f"{pad(indent)}PERFORM UNTIL {self.condition}"
        ]

        result.extend(
            render_nodes(
                self.body,
                indent + 3
            )
        )

        return result

```

```

        terminator = "END-PERFORM."

    if not final:
        terminator = "END-PERFORM"

    result.append(
        f"{pad(indent)}{terminator}"
    )

    return result

```

```

@dataclass
class Paragraph:

    name: str
    body: list[CobolNode] = field(
        default_factory=list
    )

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = [
            f"{self.name}."
        ]

        if self.body:
            result.extend(
                render_nodes(
                    self.body,
                    indent,
                    terminate_last=True
                )
            )

        return result

```

```

def render_nodes(
    nodes: list[CobolNode],
    indent: int = 7,
    terminate_last: bool = False
) -> list[str]:

    result: list[str] = []

    for index, node in enumerate(nodes):
        is_last = (
            terminate_last
            and index == len(nodes) - 1
        )

        result.extend(
            node.render(
                indent=indent,
                final=is_last
            )
        )

    return result

```

```

def render_paragraph(
    name: str,
    nodes: list[CobolNode]
) -> str:

    paragraph = Paragraph(
        name=name,
        body=nodes
    )

```

```

return "\n".join(
    paragraph.render()
)

```

```

=====
FILE: src\idms_db2_converter\generators\cursors.py
ENCODING: utf-8
=====

```

```

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.formatting import comma_block
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

```

```

class CursorGenerator:

```

```

    """
    Generates DECLARE CURSOR statements for IDMS set navigation.

```

```

    This version selects all child columns from the final schema so the fetch
    paragraph can populate all host variables needed later by field rewriting.
    """

```

```

def generate(
    self,
    schema: SchemaModel,
    used_sets: list[str]
) -> str:

```

```

    print("USING CURSOR_GENERATOR VERSION ALL-COLUMNS-2026-07-01")

```

```

    blocks: list[str] = []
    seen: set[str] = set()

```

```

    for set_name in used_sets:
        set_name = set_name.upper()

```

```

        if set_name in seen:
            continue

```

```

        seen.add(set_name)

```

```

        relationship = schema.relationships.get(set_name)

```

```

        if not relationship:
            raise ConversionError(
                f"Set {set_name} missing from schema relationships."
            )

```

```

        if not relationship.parent_key or not relationship.child_fk:
            raise ConversionError(
                f"Set {set_name} missing parent_key or child_fk."
            )

```

```

        child_record = schema.records.get(
            self._physical_record_name(
                schema,
                relationship.child_record
            )
        )

```

```

        if not child_record:
            raise ConversionError(
                f"Child record {relationship.child_record} missing."
            )

```

```

        columns = list(child_record.fields.keys())

```

```

        parent_host = Naming.hv(
            relationship.parent_record,
            relationship.parent_key
        )

```

```

        lines = [

```

```

        "            EXEC SQL",
        f"            DECLARE {Naming.cursor(set_name)} CURSOR FOR"
    ]

    lines.extend(
        comma_block(
            items=columns,
            first_prefix="        SELECT ",
            next_prefix="        "
        )
    )

    lines.append(
        f"            FROM {self._table_name(schema, relationship.child_record)}"
    )

    lines.append(
        f"            WHERE {relationship.child_fk} = :{parent_host}"
    )

    if relationship.order_by:
        lines.append(
            f"            ORDER BY {'', '.join(relationship.order_by)}"
        )

    lines.append(
        "        END-EXEC."
    )

    blocks.append(
        "\n".join(lines)
    )

    return "\n\n".join(blocks)

def _table_name(
    self,
    schema: SchemaModel,
    record_name: str
) -> str:

    mapped = schema.record_table_map.get(
        record_name
    )

    if mapped:
        return mapped

    normalized = record_name.replace(
        "-",
        "_",
    ).upper()

    mapped = schema.record_table_map.get(
        normalized
    )

    if mapped:
        return mapped

    return normalized

def _physical_record_name(
    self,
    schema: SchemaModel,
    logical_record_name: str
) -> str:

    logical_record_name = logical_record_name.upper()

    mapped = schema.record_table_map.get(
        logical_record_name
    )

    if mapped and mapped in schema.records:
        return mapped

```



```

normalized = logical_record_name.replace(
    "-",
    " "
)

mapped = schema.record_table_map.get(
    normalized
)

if mapped and mapped in schema.records:
    return mapped

if logical_record_name in schema.records:
    return logical_record_name

if normalized in schema.records:
    return normalized

return logical_record_name

```

```

=====
FILE: src\idms_db2_converter\generators\fetch_paragraphs.py
ENCODING: utf-8
=====

```

```

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

```

```

class FetchParagraphGenerator:
    """
    Generates complete FETCH paragraphs for cursor-based IDMS set walking.

    Guarantees:
    - FETCH statement has END-EXEC.
    - SQLCODE handling is emitted after FETCH.
    - INTO list matches every selected column in the cursor.
    """

    def __init__(
        self,
        schema: SchemaModel
    ):
        self.schema = schema

    def paragraph_name(
        self,
        set_name: str
    ) -> str:

        return f"FETCH-{set_name}".upper()

    def generate(
        self,
        used_sets: list[str]
    ) -> str:

        print(
            "USING FETCH_PARAGRAPH_GENERATOR VERSION COMPLETE-SQLCODE-2026-07-01"
        )

        paragraphs: list[str] = []
        seen: set[str] = set()

        for set_name in used_sets:
            set_name = set_name.upper()

            if set_name in seen:
                continue

            seen.add(set_name)

            if set_name not in self.schema.relationships:
                continue

```

```

        paragraph = self._generate_one(
            set_name
        )

        if paragraph.strip():
            paragraphs.append(
                paragraph
            )

    return "\n\n".join(
        paragraphs
    )

def _generate_one(
    self,
    set_name: str
) -> str:

    relationship = self.schema.relationships[
        set_name
    ]

    child_record_name = self._physical_record_name(
        relationship.child_record
    )

    child_record = self.schema.records.get(
        child_record_name
    )

    if child_record is None:
        return ""

    into_items: list[str] = []

    for column_name, column in child_record.fields.items():
        host = (
            f":{Naming.hv(relationship.child_record, column_name)}"
        )

        if column.nullable:
            host += (
                f":{Naming.ni(relationship.child_record, column_name)}"
            )

        into_items.append(
            host
        )

    lines: list[str] = [
        f"{self.paragraph_name(set_name)}. ",
        "    EXEC SQL",
        f"        FETCH {Naming.cursor(set_name)}"
    ]

    for index, item in enumerate(into_items):
        suffix = "," if index < len(into_items) - 1 else ""

        if index == 0:
            lines.append(
                f"        INTO {item}{suffix}"
            )
        else:
            lines.append(
                f"        {item}{suffix}"
            )

    lines.extend(
        [
            "    END-EXEC.",
            "",
            "    IF SQLCODE NOT = 0 AND SQLCODE NOT = 100",
            "        PERFORM SQL-ERROR",
            "    END-IF."
        ]
    )

```

```

    )

    return "\n".join(
        lines
    )

def _physical_record_name(
    self,
    logical_record_name: str
) -> str:

    logical_record_name = logical_record_name.upper()

    mapped = self.schema.record_table_map.get(
        logical_record_name
    )

    if mapped and mapped in self.schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        " "
    )

    mapped = self.schema.record_table_map.get(
        normalized
    )

    if mapped and mapped in self.schema.records:
        return mapped

    if logical_record_name in self.schema.records:
        return logical_record_name

    if normalized in self.schema.records:
        return normalized

    return logical_record_name

```

```

=====
FILE: src\idms_db2_converter\generators\formatting.py
ENCODING: utf-8
=====

```

```

def comma_block(
    items: list[str],
    first_prefix: str = "",
    next_prefix: str = "",
) -> list[str]:
    """
    Format a list of SQL items as comma-separated lines.

    Example:
        comma_block(
            items=["EMP_ID", "EMP_NAME", "DEPT_ID"],
            first_prefix="SELECT ",
            next_prefix="      "
        )

    Returns:
        [
            "SELECT EMP_ID,",
            "      EMP_NAME,",
            "      DEPT_ID"
        ]
    """

    if not items:
        return []

    lines: list[str] = []

    for index, item in enumerate(items):
        prefix = first_prefix if index == 0 else next_prefix

```

```

        suffix = "," if index < len(items) - 1 else ""

        lines.append(
            f"{prefix}{item}{suffix}"
        )

    return lines

```

```

=====
FILE: src\idms_db2_converter\generators\host_variables.py
ENCODING: utf-8
=====

```

```

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

```

```

class HostVariableGenerator:

```

```

    """

```

```

    Generates COBOL host variables for DB2 embedded SQL.

```

```

    Rules:

```

- Generates SQLCA.
- Generates BEGIN DECLARE SECTION and END DECLARE SECTION.
- Generates all columns for each used record.
- Uses Naming.normalize for group names and field names.
- Converts underscores to hyphens for COBOL data names.
- Generates null indicators for nullable columns.

```

    """

```

```

    def generate(

```

```

        self,
        schema: SchemaModel,
        used_records: list[str],

```

```

    ) -> str:

```

```

        lines: list[str] = [
            "      EXEC SQL",
            "          INCLUDE SQLCA",
            "      END-EXEC.",
            "      EXEC SQL",
            "          BEGIN DECLARE SECTION",
            "      END-EXEC.",
            "",
        ]

```

```

        null_indicators: list[str] = []
        emitted_null_indicators: set[str] = set()

```

```

        physical_records = self.resolve_used_records(
            schema=schema,
            used_records=used_records,
        )

```

```

        for logical_record_name, physical_record_name in physical_records:
            if physical_record_name not in schema.records:
                continue

```

```

            record = schema.records[physical_record_name]
            logical_group_name = Naming.normalize(
                logical_record_name,
            )

```

```

            lines.append(
                f"      01 HV-{logical_group_name}."
            )

```

```

            for column_name, column in record.fields.items():
                host_variable = Naming.hv(
                    logical_record_name,
                    column_name,
                )

```

```

            lines.extend(
                self.column_to_cobol(

```

```

        hv=host_variable,
        datatype=column.datatype,
        length=column.length,
        scale=column.scale,
    )
)

if column.nullable:
    null_indicator = Naming.ni(
        logical_record_name,
        column_name,
    )

    if null_indicator not in emitted_null_indicators:
        emitted_null_indicators.add(
            null_indicator,
        )

        null_indicators.append(
            f"      01 {null_indicator:<45} PIC S9(4) COMP."
        )

    lines.append(
        "",
    )

if null_indicators:
    lines.extend(
        null_indicators,
    )

lines.extend(
    [
        "      EXEC SQL",
        "      END DECLARE SECTION",
        "      END-EXEC.",
    ]
)

return "\n".join(lines)

def resolve_used_records(
    self,
    schema: SchemaModel,
    used_records: list[str],
) -> list[tuple[str, str]]:
    result: list[tuple[str, str]] = []
    seen: set[tuple[str, str]] = set()

    for logical_record_name in used_records:
        if not logical_record_name:
            continue

        physical_record_name = self.physical_record_name(
            schema=schema,
            logical_record_name=logical_record_name,
        )

        key = (
            logical_record_name.upper(),
            physical_record_name.upper(),
        )

        if key in seen:
            continue

        seen.add(
            key,
        )

        result.append(
            (
                logical_record_name.upper(),
                physical_record_name,
            )
        )

```

```

        return result

def physical_record_name(
    self,
    schema: SchemaModel,
    logical_record_name: str,
) -> str:
    logical_record_name = logical_record_name.upper()

    mapped = schema.record_table_map.get(
        logical_record_name,
    )

    if mapped and mapped in schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        "_",
    )

    mapped = schema.record_table_map.get(
        normalized,
    )

    if mapped and mapped in schema.records:
        return mapped

    if logical_record_name in schema.records:
        return logical_record_name

    if normalized in schema.records:
        return normalized

    hyphen_name = logical_record_name.replace(
        "-",
        "-",
    )

    if hyphen_name in schema.records:
        return hyphen_name

    return logical_record_name

def column_to_cobol(
    self,
    hv: str,
    datatype: str | None,
    length: int | None,
    scale: int | None,
) -> list[str]:
    datatype = (
        datatype
        or "CHAR"
    ).upper()

    if datatype in {
        "CHAR",
        "CHARACTER",
    }:
        return [
            f"          05 {hv:<45} PIC X({length or 1})."
        ]

    if datatype in {
        "VARCHAR",
        "LONG VARCHAR",
    }:
        return [
            f"          05 {hv:<45} PIC X({length or 255})."
        ]

    if datatype in {
        "DATE",
    }:

```

```

        return [
            f"          05 {hv:<45} PIC X(10)."

```

```

        scale
        if scale is not None
        else 0
    )

    integer_digits = max(
        actual_precision - actual_scale,
        1,
    )

    if actual_scale > 0:
        return [
            f"          05 {hv:<45} PIC S9({integer_digits})V9({actual_scale}) COMP-3."
        ]

    return [
        f"          05 {hv:<45} PIC S9({actual_precision}) COMP-3."
    ]

```

```

=====
FILE: src\idms_db2_converter\generators\naming.py
ENCODING: utf-8
=====

```

```
import re
```

```

class Naming:
    @staticmethod
    def normalize(
        value: str | None,
    ) -> str:
        if not value:
            return ""

        normalized = str(value).strip().upper()
        normalized = normalized.replace("_", "-")
        normalized = normalized.replace(" ", "-")
        normalized = re.sub(
            r"^[A-Z0-9-]",
            "-",
            normalized,
        )
        normalized = re.sub(
            r"-+",
            "-",
            normalized,
        )

        return normalized.strip("-")

    @staticmethod
    def hv(
        record: str,
        field: str,
    ) -> str:
        record_name = Naming.normalize(
            record,
        )

        field_name = Naming.normalize(
            field,
        )

        return f"HV-{record_name}-{field_name}"

    @staticmethod
    def ni(
        record: str,
        field: str,
    ) -> str:
        record_name = Naming.normalize(
            record,
        )

```



```

        field_name = Naming.normalize(
            field,
        )

        return f"NI-{record_name}-{field_name}"

    @staticmethod
    def cursor(
        set_name: str,
    ) -> str:
        return f"C-{Naming.normalize(set_name)}"

```

```

=====
FILE: src\idms_db2_converter\generators\sql_snippets.py
ENCODING: utf-8
=====

```

```

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.formatting import comma_block
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

```

```

class SqlSnippets:
    """
    Generates embedded SQL snippets with full SELECT and INTO lists.
    """

    def __init__(
        self,
        schema: SchemaModel
    ):
        self.schema = schema

    def select_for_record_by_pk(
        self,
        record_name: str
    ) -> str:

        physical_record_name = self.physical_record_name(
            record_name
        )

        record = self.schema.records.get(
            physical_record_name
        )

        if not record:
            raise ConversionError(
                f"Record {record_name} missing from schema."
            )

        if not record.primary_key:
            raise ConversionError(
                f"Record {record_name} has no primary key."
            )

        columns = list(record.fields.keys())

        into_items = [
            self.host_with_indicator(
                logical_record_name=record_name,
                column_name=column_name,
                nullable=record.fields[column_name].nullable
            )
            for column_name in columns
        ]

        lines = [
            "EXEC SQL"
        ]

        lines.extend(
            comma_block(
                items=columns,
            )
        )

```

```

        first_prefix="    SELECT ",
        next_prefix="        "
    )
)

lines.extend(
    comma_block(
        items=into_items,
        first_prefix="        INTO ",
        next_prefix="        "
    )
)

lines.append(
    f"    FROM {physical_record_name}"
)

lines.append(
    f"    WHERE {record.primary_key} = :{Naming.hv(record_name, record.primary_key)}"
)

lines.append(
    "END-EXEC."
)

return "\n".join(lines)

def select_first_child_for_set(
    self,
    set_name: str
) -> str:

    set_name = set_name.upper()

    relationship = self.schema.relationships.get(
        set_name
    )

    if not relationship:
        raise ConversionError(
            f"Set {set_name} missing from schema relationships."
        )

    child_record_name = self.physical_record_name(
        relationship.child_record
    )

    child_record = self.schema.records.get(
        child_record_name
    )

    if not child_record:
        raise ConversionError(
            f"Child record {relationship.child_record} missing from schema."
        )

    columns = list(child_record.fields.keys())

    into_items = [
        self.host_with_indicator(
            logical_record_name=relationship.child_record,
            column_name=column_name,
            nullable=child_record.fields[column_name].nullable
        )
        for column_name in columns
    ]

    parent_host = Naming.hv(
        relationship.parent_record,
        relationship.parent_key
    )

    lines = [
        "EXEC SQL"
    ]

```

```

        lines.extend(
            comma_block(
                items=columns,
                first_prefix="    SELECT ",
                next_prefix="    "
            )
        )

        lines.extend(
            comma_block(
                items=into_items,
                first_prefix="    INTO ",
                next_prefix="    "
            )
        )

        lines.append(
            f"    FROM {child_record_name}"
        )

        lines.append(
            f"    WHERE {relationship.child_fk} = :{parent_host}"
        )

        if relationship.order_by:
            lines.append(
                f"    ORDER BY {'', ' '.join(relationship.order_by)}"
            )

        lines.append(
            "    FETCH FIRST 1 ROW ONLY"
        )

        lines.append(
            "END-EXEC."
        )

        return "\n".join(lines)

def select_for_owner(
    self,
    set_name: str
) -> str:

    set_name = set_name.upper()

    relationship = self.schema.relationships.get(
        set_name
    )

    if not relationship:
        raise ConversionError(
            f"Set {set_name} missing from schema relationships."
        )

    parent_record_name = self.physical_record_name(
        relationship.parent_record
    )

    parent_record = self.schema.records.get(
        parent_record_name
    )

    if not parent_record:
        raise ConversionError(
            f"Parent record {relationship.parent_record} missing from schema."
        )

    columns = list(parent_record.fields.keys())

    into_items = [
        self.host_with_indicator(
            logical_record_name=relationship.parent_record,
            column_name=column_name,
            nullable=parent_record.fields[column_name].nullable
        )
    ]

```

```

        for column_name in columns
    ]

    child_host = Naming.hv(
        relationship.child_record,
        relationship.child_fk
    )

    lines = [
        "EXEC SQL"
    ]

    lines.extend(
        comma_block(
            items=columns,
            first_prefix="    SELECT ",
            next_prefix="        "
        )
    )

    lines.extend(
        comma_block(
            items=into_items,
            first_prefix="        INTO ",
            next_prefix="            "
        )
    )

    lines.append(
        f"    FROM {parent_record_name}"
    )

    lines.append(
        f"    WHERE {relationship.parent_key} = :{child_host}"
    )

    lines.append(
        "END-EXEC."
    )

    return "\n".join(lines)

def fetch_for_set(
    self,
    set_name: str
) -> str:

    set_name = set_name.upper()

    relationship = self.schema.relationships.get(
        set_name
    )

    if not relationship:
        raise ConversionError(
            f"Set {set_name} missing from schema relationships."
        )

    child_record_name = self.physical_record_name(
        relationship.child_record
    )

    child_record = self.schema.records.get(
        child_record_name
    )

    if not child_record:
        raise ConversionError(
            f"Child record {relationship.child_record} missing from schema."
        )

    into_items = [
        self.host_with_indicator(
            logical_record_name=relationship.child_record,
            column_name=column_name,
            nullable=column.nullable

```

```

        )
        for column_name, column in child_record.fields.items()
    ]

    lines = [
        "EXEC SQL",
        f"      FETCH {Naming.cursor(set_name)}"
    ]

    lines.extend(
        comma_block(
            items=into_items,
            first_prefix="      INTO ",
            next_prefix="      "
        )
    )

    lines.append(
        "END-EXEC."
    )

    return "\n".join(lines)

def nullable_fk_indicator_for_set(
    self,
    set_name: str
) -> str | None:

    relationship = self.schema.relationships.get(
        set_name.upper()
    )

    if not relationship:
        return None

    child_record_name = self.physical_record_name(
        relationship.child_record
    )

    child_record = self.schema.records.get(
        child_record_name
    )

    if not child_record:
        return None

    child_fk = relationship.child_fk

    if not child_fk:
        return None

    column = child_record.fields.get(
        child_fk
    )

    if not column:
        return None

    if not column.nullable:
        return None

    return Naming.ni(
        relationship.child_record,
        child_fk
    )

def physical_record_name(
    self,
    logical_record_name: str
) -> str:

    logical_record_name = logical_record_name.upper()

    mapped = self.schema.record_table_map.get(
        logical_record_name
    )

```

```

        if mapped and mapped in self.schema.records:
            return mapped

        normalized = logical_record_name.replace(
            "-",
            " ",
            "-"
        )

        mapped = self.schema.record_table_map.get(
            normalized
        )

        if mapped and mapped in self.schema.records:
            return mapped

        if logical_record_name in self.schema.records:
            return logical_record_name

        if normalized in self.schema.records:
            return normalized

        return logical_record_name

def host_with_indicator(
    self,
    logical_record_name: str,
    column_name: str,
    nullable: bool
) -> str:

    host = f":{Naming.hv(logical_record_name, column_name)}"

    if nullable:
        host = f"{host} :{Naming.ni(logical_record_name, column_name)}"

    return host

```

```

=====
FILE: src\idms_db2_converter\models.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field
from typing import Dict, List, Optional

```

```

@dataclass
class Column:
    name: str
    datatype: str
    length: Optional[int] = None
    scale: Optional[int] = None
    nullable: bool = True
    primary_key: bool = False
    generated: bool = False
    source_kind: str = ""

@dataclass
class Record:
    name: str
    primary_key: Optional[str] = None
    fields: Dict[str, Column] = field(default_factory=dict)

    # New composite PK support.
    # Existing code can continue using primary_key.
    primary_keys: List[str] = field(default_factory=list)

    def effective_primary_keys(self) -> List[str]:
        keys: List[str] = []

        if self.primary_keys:
            keys.extend(self.primary_keys)

```

```

        if self.primary_key and self.primary_key not in keys:
            keys.append(self.primary_key)

    return keys

def set_primary_keys(self, keys: List[str]) -> None:
    cleaned: List[str] = []

    for key in keys or []:
        if not key:
            continue

        normalized = str(key).upper()

        if normalized in cleaned:
            continue

        cleaned.append(normalized)

    self.primary_keys = cleaned
    self.primary_key = cleaned[0] if cleaned else None

    for key in cleaned:
        if key in self.fields:
            self.fields[key].primary_key = True
            self.fields[key].nullable = False

@dataclass
class Relationship:
    set_name: str
    parent_record: str
    child_record: str
    cardinality: Optional[str] = None

    # Backward-compatible single key fields.
    parent_key: Optional[str] = None
    child_fk: Optional[str] = None

    # New composite relationship support.
    parent_keys: List[str] = field(default_factory=list)
    child_fks: List[str] = field(default_factory=list)

    order_by: List[str] = field(default_factory=list)

    def effective_parent_keys(self) -> List[str]:
        keys: List[str] = []

        if self.parent_keys:
            keys.extend(self.parent_keys)

        if self.parent_key and self.parent_key not in keys:
            keys.append(self.parent_key)

        return keys

    def effective_child_fks(self) -> List[str]:
        keys: List[str] = []

        if self.child_fks:
            keys.extend(self.child_fks)

        if self.child_fk and self.child_fk not in keys:
            keys.append(self.child_fk)

        return keys

@dataclass
class SchemaModel:
    records: Dict[str, Record] = field(default_factory=dict)
    relationships: Dict[str, Relationship] = field(default_factory=dict)

    field_map: Dict[str, dict] = field(default_factory=dict)
    record_table_map: Dict[str, str] = field(default_factory=dict)
    calc_key_map: Dict[str, dict] = field(default_factory=dict)

```

```

# Existing Phase 2 metadata maps.
set_ordering_map: Dict[str, dict] = field(default_factory=dict)
navigation_intent: Dict[str, dict] = field(default_factory=dict)
nullable_fk_map: Dict[str, dict] = field(default_factory=dict)
date_part_map: Dict[str, dict] = field(default_factory=dict)
output_semantics: Dict[str, dict] = field(default_factory=dict)
paragraph_operation_graph: Dict[str, list] = field(default_factory=dict)
validation_messages: List[str] = field(default_factory=list)

# New key metadata from Phase 1.
table_key_map: Dict[str, dict] = field(default_factory=dict)

schema_source: Optional[str] = None

def has_schema_objects(self) -> bool:
    return bool(self.records or self.relationships)

def is_empty(self) -> bool:
    return not self.has_schema_objects()

def add_validation_message(self, message: str) -> None:
    if not hasattr(self, "validation_messages"):
        self.validation_messages = []

    self.validation_messages.append(message)

@dataclass
class CobolAnalysis:
    program_id: Optional[str] = None

    idms_records: List[str] = field(default_factory=list)
    obtain_calc_records: List[str] = field(default_factory=list)
    obtain_next: List[tuple[str, str]] = field(default_factory=list)
    obtain_owner_sets: List[str] = field(default_factory=list)
    find_first_sets: List[str] = field(default_factory=list)

    store_records: List[str] = field(default_factory=list)
    modify_records: List[str] = field(default_factory=list)
    erase_records: List[str] = field(default_factory=list)
    ready_update_areas: List[str] = field(default_factory=list)

    field_references: List[str] = field(default_factory=list)
    idms_fields: List[str] = field(default_factory=list)

@dataclass
class Paragraph:
    name: str
    text: str

=====
FILE: src\idms_db2_converter\parsers\__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_converter\parsers\canonical_parser.py
ENCODING: utf-8
=====

import json

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import Column, Record, Relationship, SchemaModel

class CanonicalParser:
    def parse(self, text: str) -> SchemaModel:
        try:
            payload = json.loads(text)

```



```

except json.JSONDecodeError as exc:
    raise ConversionError(f"Invalid canonical JSON: {exc}") from exc

schema = SchemaModel()

self._parse_records(payload, schema)
self._parse_relationships(payload, schema)
self._parse_field_map(payload, schema)

return schema

def _parse_records(self, payload: dict, schema: SchemaModel) -> None:
    for record_json in payload.get("records", []):
        record_name = record_json["name"].upper()
        primary_key = self._upper_or_none(record_json.get("primary_key"))

        fields = {}

        for field_json in record_json.get("fields", []):
            field_name = field_json["name"].upper()

            nullable = field_json.get("nullable")
            if nullable is None:
                nullable = field_name != primary_key

            fields[field_name] = Column(
                name=field_name,
                datatype=field_json["datatype"].upper(),
                length=field_json.get("length"),
                scale=field_json.get("scale"),
                nullable=bool(nullable),
            )

        schema.records[record_name] = Record(
            name=record_name,
            primary_key=primary_key,
            fields=fields,
        )

def _parse_relationships(self, payload: dict, schema: SchemaModel) -> None:
    for rel_json in payload.get("relationships", []):
        set_name = rel_json["set_name"].upper()
        parent_record = rel_json["parent_record"].upper()
        child_record = rel_json["child_record"].upper()

        parent_key = self._upper_or_none(rel_json.get("parent_key"))
        child_fk = self._upper_or_none(rel_json.get("child_fk"))

        if not parent_key:
            parent_key = self._resolve_parent_key(schema, parent_record)

        if not child_fk:
            child_fk = self._resolve_child_fk(
                schema=schema,
                parent_record=parent_record,
                child_record=child_record,
            )

        order_by = [
            value.upper()
            for value in rel_json.get("order_by", [])
        ]

        if not order_by and child_fk:
            order_by = [child_fk]

        schema.relationships[set_name] = Relationship(
            set_name=set_name,
            parent_record=parent_record,
            child_record=child_record,
            cardinality=rel_json.get("cardinality"),
            parent_key=parent_key,
            child_fk=child_fk,
            order_by=order_by,
        )

```

```

def _parse_field_map(self, payload: dict, schema: SchemaModel) -> None:
    schema.field_map = {
        key.upper(): value
        for key, value in payload.get("field_map", {}).items()
    }

def _resolve_parent_key(
    self,
    schema: SchemaModel,
    parent_record: str,
) -> str | None:
    record = schema.records.get(parent_record)

    if not record:
        return None

    return record.primary_key

def _resolve_child_fk(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
) -> str | None:
    parent = schema.records.get(parent_record)
    child = schema.records.get(child_record)

    if not parent or not child or not parent.primary_key:
        return None

    if parent.primary_key in child.fields:
        return parent.primary_key

    return None

def _upper_or_none(self, value: str | None) -> str | None:
    return value.upper() if value else None

```

```

=====
FILE: src\idms_db2_converter\parsers\cobol_parser.py
ENCODING: utf-8
=====

```

```
import re
```

```
from idms_db2_converter.models import CobolAnalysis, Paragraph
```

```
class CobolParser:
```

```
    """
```

```
    Parses IDMS COBOL.
```

```
    Captures:
```

- PROGRAM-ID
- COPY IDMS RECORD usage
- OBTAIN CALC record usage
- OBTAIN NEXT record WITHIN set
- OBTAIN OWNER WITHIN set
- FIND FIRST WITHIN set
- STORE record usage
- MODIFY record usage
- ERASE record usage
- READY AREA ... USAGE-MODE IS UPDATE
- Paragraphs
- IDMS schema-suffixed field references in PROCEDURE DIVISION

```
    Important:
```

- This parser does not hard-code record suffix to record-name mappings.
- Record/field inference should come from schema metadata, not from demo data.

```
    """
```

```

    PROGRAM_ID = re.compile(
        r"^\s*PROGRAM-ID\.\s*([A-Z0-9-]+\.\.?)",
        re.IGNORECASE | re.MULTILINE,
    )

```

```

COPY_RECORD = re.compile(
    r"^\\s*(?:01\\s+)?COPY\\s+IDMS\\s+RECORD\\s+([A-Z0-9-]+)\\.?.?",
    re.IGNORECASE | re.MULTILINE,
)

OBTAIN_CALC = re.compile(
    r"^\\s*OBTAIN\\s+CALC\\s+([A-Z0-9-]+)\\.?.?\\s*$",
    re.IGNORECASE | re.MULTILINE,
)

OBTAIN_NEXT = re.compile(
    r"^\\s*OBTAIN\\s+NEXT\\s+([A-Z0-9-]+)\\s+WITHIN\\s+([A-Z0-9-]+)\\.?.?\\s*$",
    re.IGNORECASE | re.MULTILINE,
)

OBTAIN_OWNER = re.compile(
    r"^\\s*OBTAIN\\s+OWNER\\s+WITHIN\\s+([A-Z0-9-]+)\\.?.?\\s*$",
    re.IGNORECASE | re.MULTILINE,
)

FIND_FIRST = re.compile(
    r"^\\s*FIND\\s+FIRST\\s+WITHIN\\s+([A-Z0-9-]+)\\.?.?\\s*$",
    re.IGNORECASE | re.MULTILINE,
)

STORE_RECORD = re.compile(
    r"^\\s*STORE\\s+([A-Z0-9-]+)\\.?.?\\s*$",
    re.IGNORECASE | re.MULTILINE,
)

MODIFY_RECORD = re.compile(
    r"^\\s*MODIFY\\s+([A-Z0-9-]+)\\.?.?\\s*$",
    re.IGNORECASE | re.MULTILINE,
)

ERASE_RECORD = re.compile(
    r"^\\s*ERASE\\s+([A-Z0-9-]+)\\.?.?\\s*$",
    re.IGNORECASE | re.MULTILINE,
)

READY_UPDATE_AREA = re.compile(
    r"""
    ^\\s*READY\\s+AREA\\s+([A-Z0-9-]+)\\.?.?\\s*
    (?:\\n\\s*USAGE-MODE\\s+IS\\s+UPDATE\\.?.?\\s*)?
    """,
    re.IGNORECASE | re.MULTILINE | re.VERBOSE,
)

PARAGRAPH_HEADER = re.compile(
    r"^\\s*([A-Z0-9-]+)\\.?.?\\s*$",
    re.IGNORECASE | re.MULTILINE,
)

IDMS_SCHEMA_FIELD = re.compile(
    r"\\b[A-Z][A-Z0-9]*(?:-[A-Z0-9]+)*-\\d{4}\\b",
    re.IGNORECASE,
)

PROCEDURE_DIVISION = re.compile(
    r"^\\s*PROCEDURE\\s+DIVISION\\.?.?\\s*$",
    re.IGNORECASE | re.MULTILINE,
)

def parse(
    self,
    cobol: str,
) -> CobolAnalysis:
    source = self.remove_comment_lines(
        cobol,
    )

    program_match = self.PROGRAM_ID.search(
        source,
    )

```

```

analysis = CobolAnalysis(
    program_id=self._upper_or_none(
        program_match.group(1)
        if program_match
        else None
    ),
    idms_records=self._unique(
        self.COPY_RECORD.findall(source),
    ),
    obtain_calc_records=self._unique(
        self.OBTAIN_CALC.findall(source),
    ),
    obtain_next=[
        (
            record.upper(),
            set_name.upper(),
        )
        for record, set_name in self.OBTAIN_NEXT.findall(source)
    ],
    obtain_owner_sets=self._unique(
        self.OBTAIN_OWNER.findall(source),
    ),
    find_first_sets=self._unique(
        self.FIND_FIRST.findall(source),
    ),
    store_records=self._unique(
        self.STORE_RECORD.findall(source),
    ),
    modify_records=self._unique(
        self.MODIFY_RECORD.findall(source),
    ),
    erase_records=self._unique(
        self.ERASE_RECORD.findall(source),
    ),
    ready_update_areas=self._unique(
        self.READY_UPDATE_AREA.findall(source),
    ),
)

self._attach_field_references(
    analysis=analysis,
    cobol=source,
)

return analysis

def paragraphs(
    self,
    cobol: str,
) -> list[Paragraph]:
    source = self.remove_comment_lines(
        cobol,
    )

    matches = list(
        self.PARAGRAPH_HEADER.finditer(source),
    )

    paragraphs: list[Paragraph] = []

    for index, match in enumerate(matches):
        name = match.group(1).upper()
        start = match.start()

        if index + 1 < len(matches):
            end = matches[index + 1].start()
        else:
            end = len(source)

        paragraphs.append(
            Paragraph(
                name=name,
                text=source[start:end],
            )
        )

```

```

    return paragraphs

def remove_comment_lines(
    self,
    cobol: str,
) -> str:
    lines: list[str] = []

    for line in cobol.splitlines():
        stripped = line.strip()

        if not stripped:
            lines.append(line)
            continue

        if stripped.startswith("*"):
            continue

        lines.append(line)

    return "\n".join(lines)

def _attach_field_references(
    self,
    analysis: CobolAnalysis,
    cobol: str,
) -> None:
    procedure_text = self._procedure_division_text(
        cobol,
    )

    field_references = self._unique(
        self.IDMS_SCHEMA_FIELD.findall(
            procedure_text,
        )
    )

    try:
        analysis.field_references = field_references
    except Exception:
        pass

    try:
        analysis.idms_fields = field_references
    except Exception:
        pass

    record_names = set(
        analysis.idms_records,
    )

    for record in analysis.obtain_calc_records:
        record_names.add(
            record,
        )

    for record, _ in analysis.obtain_next:
        record_names.add(
            record,
        )

    for record in getattr(
        analysis,
        "store_records",
        [],
    ):
        record_names.add(
            record,
        )

    for record in getattr(
        analysis,
        "modify_records",
        [],
    ):
        record_names.add(

```

```

        record,
    )

    for record in getattr(
        analysis,
        "erase_records",
        [],
    ):
        record_names.add(
            record,
        )

    try:
        analysis.idms_records = sorted(
            record_names,
        )
    except Exception:
        pass

def _procedure_division_text(
    self,
    cobol: str,
) -> str:
    match = self.PROCEDURE_DIVISION.search(
        cobol,
    )

    if not match:
        return cobol

    return cobol[match.start():]

def _unique(
    self,
    values,
) -> list[str]:
    seen = set()
    result: list[str] = []

    for value in values:
        upper_value = value.upper()

        if upper_value not in seen:
            seen.add(
                upper_value,
            )

            result.append(
                upper_value,
            )

    return result

def _upper_or_none(
    self,
    value: str | None,
) -> str | None:
    return value.upper() if value else None

```

```

=====
FILE: src\idms_db2_converter\parsers\ddl_parser.py
ENCODING: utf-8
=====

```

```

import re

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import (
    Column,
    Record,
    Relationship,
    SchemaModel
)

```

```

class DDLParser:
    """
    Robust DB2 DDL parser for Phase 2.

    This version uses manual datatype parsing instead of fragile regex-only
    datatype parsing.

    Fixes:
    - VARCHAR(3) being skipped
    - VARCHAR(45) being skipped
    - CHAR(1) being skipped
    - DECIMAL(18,0) being skipped
    - Missing columns in final Phase 2 schema
    """

    DEBUG = True

    INLINE_PRIMARY_KEY = re.compile(
        r"\bPRIMARY\s+KEY\b",
        re.IGNORECASE
    )

    INLINE_NOT_NULL = re.compile(
        r"\bNOT\s+NULL\b",
        re.IGNORECASE
    )

    TABLE_PRIMARY_KEY = re.compile(
        r"\bPRIMARY\s+KEY\s*$(?P<columns>.*?)$",
        re.IGNORECASE | re.DOTALL
    )

    TABLE_FOREIGN_KEY = re.compile(
        r"""
        \bFOREIGN\s+KEY\s*
        $(?P<child_cols>.*?)$
        \s+REFERENCES\s+
        (?P<parent_table>(?:[A-Z0-9_]+\.)?[A-Z0-9_]+)
        \s*
        $(?P<parent_cols>.*?)$
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE
    )

    ALTER_TABLE_FK = re.compile(
        r"""
        \bALTER\s+TABLE\s+
        (?P<child_table>(?:[A-Z0-9_]+\.)?[A-Z0-9_]+)
        .*?
        \bFOREIGN\s+KEY\s*
        $(?P<child_cols>.*?)$
        \s+REFERENCES\s+
        (?P<parent_table>(?:[A-Z0-9_]+\.)?[A-Z0-9_]+)
        \s*
        $(?P<parent_cols>.*?)$
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE
    )

    CONSTRAINT_STARTERS = {
        "CONSTRAINT",
        "PRIMARY",
        "FOREIGN",
        "UNIQUE",
        "CHECK"
    }

    def parse(
        self,
        ddl: str
    ) -> SchemaModel:

        self._debug(
            "USING DDL_PARSER VERSION MANUAL-DATATYPE-2026-07-01"
        )

```

```

schema = SchemaModel()
schema.schema_source = "DDL"

if ddl is None:
    ddl = ""

self._debug(
    f"DDL INPUT LENGTH: {len(ddl)}"
)

self._debug(
    f"DDL HAS CREATE TABLE: {'CREATE TABLE' in ddl.upper()}"
)

ddl = self.remove_comments(
    ddl
)

create_statements = self._extract_create_table_statements(
    ddl
)

self._debug(
    f"DDL PARSER CREATE TABLE COUNT: {len(create_statements)}"
)

for statement in create_statements:
    table_name = statement["table_name"]
    body = statement["body"]

    self._debug(
        f"DDL PARSER START TABLE: {table_name}"
    )

    record = Record(
        name=table_name,
        primary_key=None,
        fields={}
    )

    schema.record_table_map[
        table_name
    ] = table_name

    entries = self.split_entries(
        body
    )

    self._debug(
        f"DDL PARSER TABLE {table_name} ENTRY COUNT: {len(entries)}"
    )

    pending_foreign_keys: list[dict] = []

    for entry in entries:
        entry = entry.strip()

        if not entry:
            continue

        first_word = self._first_word(
            entry
        )

        if first_word in self.CONSTRAINT_STARTERS:
            self._parse_constraint_entry(
                record=record,
                entry=entry,
                pending_foreign_keys=pending_foreign_keys
            )
            continue

        column = self._parse_column_entry(
            entry=entry,
            table_name=table_name
        )

```



```

        if column is None:
            continue

        record.fields[
            column.name
        ] = column

        if self.INLINE_PRIMARY_KEY.search(entry):
            record.primary_key = column.name
            column.nullable = False

    schema.records[
        table_name
    ] = record

    self._debug(
        f"DDL PARSER TABLE {table_name}: "
        f"{len(record.fields)} columns, PK={record.primary_key}"
    )

    for field_name, field in record.fields.items():
        self._debug(
            f" COLUMN {field_name}: "
            f"{field.datatype}({field.length},{field.scale}) "
            f"NULLABLE={field.nullable}"
        )

    for foreign_key in pending_foreign_keys:
        self._add_relationship(
            schema=schema,
            child_table=table_name,
            child_fk=foreign_key["child_fk"],
            parent_table=foreign_key["parent_table"],
            parent_key=foreign_key["parent_key"]
        )

    self._parse_alter_table_foreign_keys(
        ddl=ddl,
        schema=schema
    )

    self._validate(
        schema
    )

    if "CREATE TABLE" in ddl.upper() and not schema.records:
        raise ConversionError(
            "DDL text was provided and contains CREATE TABLE, but no tables were parsed."
        )

    return schema

def remove_comments(
    self,
    ddl: str
) -> str:

    ddl = re.sub(
        r"--.*?$",
        "",
        ddl,
        flags=re.MULTILINE
    )

    ddl = re.sub(
        r"/\*.*?\*/",
        "",
        ddl,
        flags=re.DOTALL
    )

    return ddl

def _extract_create_table_statements(
    self,

```

```

ddl: str
) -> list[dict]:

    results: list[dict] = []

    pattern = re.compile(
        r"\bCREATE\s+TABLE\s+(?P<table>(?:[A-Z0-9_]+\.)?[A-Z0-9_]+)",
        re.IGNORECASE
    )

    for match in pattern.finditer(ddl):
        table_name = self._normalize_name(
            match.group("table")
        )

        open_index = ddl.find(
            "(",
            match.end()
        )

        if open_index < 0:
            self._debug(
                f"DDL PARSER TABLE {table_name}: no opening parenthesis found"
            )
            continue

        close_index = self._find_matching_parenthesis(
            text=ddl,
            open_index=open_index
        )

        if close_index < 0:
            self._debug(
                f"DDL PARSER TABLE {table_name}: no closing parenthesis found"
            )
            continue

        body = ddl[
            open_index + 1:close_index
        ]

        results.append(
            {
                "table_name": table_name,
                "body": body
            }
        )

    return results

def _find_matching_parenthesis(
    self,
    text: str,
    open_index: int
) -> int:

    depth = 0
    quote = None

    for index in range(open_index, len(text)):
        char = text[index]

        if quote:
            if char == quote:
                quote = None
                continue

        if char in {
            '"',
            "'",
        }:
            quote = char
            continue

        if char == "(":
            depth += 1

```

```

        continue

    if char == ")":
        depth -= 1

        if depth == 0:
            return index

    return -1

def split_entries(
    self,
    body: str
) -> list[str]:

    entries: list[str] = []
    current: list[str] = []
    depth = 0
    quote = None

    for char in body:
        if quote:
            current.append(char)

            if char == quote:
                quote = None

            continue

        if char in {
            "'",
            '"',
        }:
            quote = char
            current.append(char)
            continue

        if char == "(":
            depth += 1
            current.append(char)
            continue

        if char == ")":
            depth -= 1
            current.append(char)
            continue

        if char == "," and depth == 0:
            entry = "".join(current).strip()

            if entry:
                entries.append(entry)

            current = []
            continue

        current.append(char)

    entry = "".join(current).strip()

    if entry:
        entries.append(entry)

    return entries

def _parse_constraint_entry(
    self,
    record: Record,
    entry: str,
    pending_foreign_keys: list[dict]
) -> None:

    primary_key_match = self.TABLE_PRIMARY_KEY.search(
        entry
    )

```

```

if primary_key_match:
    primary_key = self._first_column(
        primary_key_match.group("columns")
    )

    if primary_key:
        record.primary_key = primary_key

        if primary_key in record.fields:
            record.fields[
                primary_key
            ].nullable = False

    self._debug(
        f"DDL PARSER PRIMARY KEY FOR {record.name}: {record.primary_key}"
    )

    return

foreign_key_match = self.TABLE_FOREIGN_KEY.search(
    entry
)

if foreign_key_match:
    child_fk = self._first_column(
        foreign_key_match.group("child_cols")
    )

    parent_table = self._normalize_name(
        foreign_key_match.group("parent_table")
    )

    parent_key = self._first_column(
        foreign_key_match.group("parent_cols")
    )

    if child_fk and parent_table and parent_key:
        pending_foreign_keys.append(
            {
                "child_fk": child_fk,
                "parent_table": parent_table,
                "parent_key": parent_key
            }
        )

        self._debug(
            f"DDL PARSER FK: {child_fk} -> {parent_table}({parent_key})"
        )

    return

def _parse_alter_table_foreign_keys(
    self,
    ddl: str,
    schema: SchemaModel
) -> None:

    for match in self.ALTER_TABLE_FK.finditer(ddl):
        child_table = self._normalize_name(
            match.group("child_table")
        )

        child_fk = self._first_column(
            match.group("child_cols")
        )

        parent_table = self._normalize_name(
            match.group("parent_table")
        )

        parent_key = self._first_column(
            match.group("parent_cols")
        )

        if not child_table:
            continue

```

```

        if not child_fk:
            continue

        if not parent_table:
            continue

        if not parent_key:
            continue

        self._add_relationship(
            schema=schema,
            child_table=child_table,
            child_fk=child_fk,
            parent_table=parent_table,
            parent_key=parent_key
        )

def _add_relationship(
    self,
    schema: SchemaModel,
    child_table: str,
    child_fk: str,
    parent_table: str,
    parent_key: str
) -> None:

    child_table = self._normalize_name(child_table)
    child_fk = self._normalize_name(child_fk)
    parent_table = self._normalize_name(parent_table)
    parent_key = self._normalize_name(parent_key)

    set_name = f"{parent_table}-{child_table}"

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=parent_table,
        child_record=child_table,
        cardinality="1:N",
        parent_key=parent_key,
        child_fk=child_fk,
        order_by=[child_fk]
    )

def _parse_column_entry(
    self,
    entry: str,
    table_name: str
) -> Column | None:

    entry = entry.strip()

    if not entry:
        return None

    match = re.match(
        r"^(?P<name>[A-Z0-9_]+)\s+(?P<datatype>.+)$",
        entry,
        flags=re.IGNORECASE | re.DOTALL
    )

    if not match:
        self._debug(
            f"DDL PARSER SKIPPED ENTRY IN {table_name}: no column match. ENTRY={repr(entry)}"
        )
        return None

    column_name = self._normalize_name(
        match.group("name")
    )

    datatype_text = match.group("datatype").strip()

    datatype, length, scale = self._parse_datatype(
        datatype_text
    )

```

```

if datatype is None:
    self._debug_unrecognized_datatype(
        table_name=table_name,
        column_name=column_name,
        datatype_text=datatype_text
    )
    return None

nullable = not bool(
    self.INLINE_NOT_NULL.search(entry)
)

return Column(
    name=column_name,
    datatype=datatype,
    length=length,
    scale=scale,
    nullable=nullable
)

def _parse_datatype(
    self,
    datatype_text: str
) -> tuple[str | None, int | None, int | None]:

    text = self._normalize_datatype_text(
        datatype_text
    )

    if not text:
        return (
            None,
            None,
            None
        )

    datatype_name = self._leading_word(
        text
    )

    if datatype_name in {
        "VARCHAR",
        "CHAR",
        "CHARACTER"
    }:
        length = self._manual_first_parenthesized_int(
            text
        )

        if datatype_name == "CHARACTER":
            datatype_name = "CHAR"

        if length is None and datatype_name == "CHAR":
            length = 1

        if length is not None:
            return (
                datatype_name,
                length,
                None
            )

    if datatype_name in {
        "DECIMAL",
        "NUMERIC"
    }:
        precision_scale = self._manual_precision_scale(
            text
        )

        if precision_scale is not None:
            precision, scale = precision_scale

            return (
                "DECIMAL",

```

```

        precision,
        scale
    )

if datatype_name == "SMALLINT":
    return (
        "SMALLINT",
        4,
        0
    )

if datatype_name in {
    "INTEGER",
    "INT"
}:
    return (
        "INTEGER",
        9,
        0
    )

if datatype_name == "BIGINT":
    return (
        "BIGINT",
        18,
        0
    )

if datatype_name == "DATE":
    return (
        "DATE",
        None,
        None
    )

if datatype_name == "TIMESTAMP":
    return (
        "TIMESTAMP",
        None,
        None
    )

if datatype_name == "TIME":
    return (
        "TIME",
        None,
        None
    )

return (
    None,
    None,
    None
)

def _normalize_datatype_text(
    self,
    datatype_text: str
) -> str:

    text = datatype_text or ""

    text = str(text).strip().upper()

    text = text.replace(" ", "(")
    text = text.replace(" ", ")")
    text = text.replace(" ", ",")
    text = text.replace("\u00a0", " ")

    text = re.sub(
        r"\s+",
        " ",
        text
    )

    text = re.sub(

```

```

        r"\bNOT\s+NULL\b",
        "",
        text,
        flags=re.IGNORECASE
    )

    text = re.sub(
        r"\bPRIMARY\s+KEY\b",
        "",
        text,
        flags=re.IGNORECASE
    )

    text = re.sub(
        r"\bDEFAULT\b.*$",
        "",
        text,
        flags=re.IGNORECASE
    )

    return text.strip()

def _leading_word(
    self,
    text: str
) -> str:

    result = []

    for char in text:
        if char.isalpha():
            result.append(char)
            continue

        break

    return "".join(result).upper()

def _manual_first_parenthesized_int(
    self,
    text: str
) -> int | None:

    open_index = text.find("(")

    if open_index < 0:
        return None

    close_index = text.find(")", open_index + 1)

    if close_index < 0:
        return None

    content = text[
        open_index + 1:close_index
    ].strip()

    if not content.isdigit():
        return None

    return int(content)

def _manual_precision_scale(
    self,
    text: str
) -> tuple[int, int] | None:

    open_index = text.find("(")

    if open_index < 0:
        return None

    close_index = text.find(")", open_index + 1)

    if close_index < 0:
        return None

```



```

        content = text[
            open_index + 1:close_index
        ].strip()

        parts = [
            part.strip()
            for part in content.split(",")
        ]

        if len(parts) != 2:
            return None

        if not parts[0].isdigit():
            return None

        if not parts[1].isdigit():
            return None

        return (
            int(parts[0]),
            int(parts[1])
        )

def _validate(
    self,
    schema: SchemaModel
) -> None:

    errors: list[str] = []

    for record in schema.records.values():
        if record.primary_key and record.primary_key not in record.fields:
            errors.append(
                f"{record.name} primary key {record.primary_key} is not declared as a column."
            )

        for field_name, field in record.fields.items():
            if not field.datatype:
                errors.append(
                    f"{record.name}.{field_name} has no datatype."
                )

    if errors:
        raise ConversionError(
            "\n".join(errors)
        )

def _first_word(
    self,
    entry: str
) -> str:

    tokens = entry.strip().split()

    if not tokens:
        return ""

    return self._normalize_name(
        tokens[0]
    )

def _first_column(
    self,
    text: str
) -> str | None:

    if not text:
        return None

    first = text.split(",")[0].strip()

    if not first:
        return None

    return self._normalize_name(

```

```

        first
    )

def _normalize_name(
    self,
    value: str | None
) -> str:

    if not value:
        return ""

    value = str(value).strip()
    value = value.strip("'")
    value = value.strip('"')
    value = value.strip("`")
    value = value.strip("[")
    value = value.strip("]")

    if "." in value:
        value = value.split(".")[1]

    return value.upper()

def _debug(
    self,
    message: str
) -> None:

    if self.DEBUG:
        print(message)

def _debug_unrecognized_datatype(
    self,
    table_name: str,
    column_name: str,
    datatype_text: str
) -> None:

    if not self.DEBUG:
        return

    normalized = self._normalize_datatype_text(
        datatype_text
    )

    char_codes = [
        ord(char)
        for char in datatype_text
    ]

    print(
        "DDL PARSER SKIPPED COLUMN "
        f"{table_name}.{column_name}: "
        f"raw={repr(datatype_text)} "
        f"normalized={repr(normalized)} "
        f"leading={repr(self._leading_word(normalized))} "
        f"char_codes={char_codes}"
    )

```

```

=====
FILE: src\idms_db2_converter\parsers\idms_schema_parser.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import Column, Record, Relationship, SchemaModel

```

```

class IdmsSchemaParser:
    RECORD_HEADER = re.compile(
        r"^RECORD NAME\.\.+s+([A-Z0-9-]+)",
        re.IGNORECASE | re.MULTILINE,
    )

```

```

LOCATION_CALC = re.compile(
    r"LOCATION MODE\.\s+CALC USING\s+([A-Z0-9-]+)",
    re.IGNORECASE,
)

DBKEY_SET_ROLE = re.compile(
    r"^\s+([A-Z0-9-]+)\s+(?:INDEX\s+)?(OWNER|MEMBER)\b",
    re.IGNORECASE | re.MULTILINE,
)

DATA_LINE_PREFIX = re.compile(
    r"^\s*(0[2-5])\s+([A-Z0-9-]+)\s+(.*)$",
    re.IGNORECASE,
)

PICTURE_FIND = re.compile(
    r"S?9$\d+$V9$\d+$|"
    r"S?9$\d+$V9+|"
    r"S?9+V9$\d+$|"
    r"S?9+V9+|"
    r"S?V9$\d+$|"
    r"S?V9+|"
    r"S?9$\d+$|"
    r"S?9+|"
    r"X$\d+$|"
    r"X+",
    re.IGNORECASE,
)

DATE_PARTS = {
    "YEAR": {
        "substring_start": 1,
        "substring_length": 4,
    },
    "MONTH": {
        "substring_start": 5,
        "substring_length": 2,
    },
    "DAY": {
        "substring_start": 7,
        "substring_length": 2,
    },
}

def parse(self, text: str) -> SchemaModel:
    schema = SchemaModel()
    schema.schema_source = "IDMS_SCHEMA"

    blocks = self._record_blocks(text)
    set_roles: dict[str, dict[str, str]] = {}

    for record_name, block in blocks:
        self._parse_record(schema, record_name, block)
        self._collect_set_roles(set_roles, record_name, block)

    self._build_relationships(schema, set_roles)

    if hasattr(schema, "add_validation_message"):
        schema.add_validation_message("Schema built from IDMS schema listing.")

    return schema

def _record_blocks(self, text: str) -> list[tuple[str, str]]:
    matches = list(self.RECORD_HEADER.finditer(text))
    result = []

    for index, match in enumerate(matches):
        record_name = match.group(1).upper()
        start = match.start()
        end = matches[index + 1].start() if index + 1 < len(matches) else len(text)

        result.append((record_name, text[start:end]))

    return result

```

```

def _parse_record(
    self,
    schema: SchemaModel,
    record_name: str,
    block: str,
) -> None:
    primary_key = None

    calc_match = self.LOCATION_CALC.search(block)

    if calc_match:
        primary_key = self._normalize_column(calc_match.group(1))

    record = Record(
        name=record_name,
        primary_key=primary_key,
        fields={},
    )

    schema.record_table_map[record_name] = self._normalize_table(record_name)

    active_date_groups: dict[int, dict[str, str]] = {}

    for line in block.splitlines():
        parsed = self._parse_data_item_line(line)

        if not parsed:
            continue

        level = parsed["level"]
        raw_field_name = parsed["field_name"]
        body = parsed["body"]
        storage_length = parsed["length"]

        self._close_inactive_date_groups(active_date_groups, level)

        if raw_field_name == "FILLER":
            continue

        usage = self._extract_usage(body)
        picture = self._extract_picture(body)
        column_name = self._normalize_column(raw_field_name)

        if self._is_date_group(raw_field_name, picture, storage_length):
            if column_name not in record.fields:
                record.fields[column_name] = Column(
                    name=column_name,
                    datatype="CHAR",
                    length=8,
                    scale=None,
                    nullable=False,
                )

            host = Naming.hv(record_name, column_name)
            self._add_field_map(schema, raw_field_name, host)

            active_date_groups[level] = {
                "raw_field_name": raw_field_name,
                "column_name": column_name,
                "host": host,
            }

            continue

        date_group = self._nearest_date_group(active_date_groups)

        if date_group and self._is_date_part(raw_field_name):
            self._add_date_part_map(
                schema=schema,
                idms_field=raw_field_name,
                host=date_group["host"],
            )
            continue

        if not picture:
            continue

```

```

        datatype, column_length, scale = self._map_picture(
            picture=picture,
            usage=usage,
            storage_length=storage_length,
        )

        if column_name not in record.fields:
            record.fields[column_name] = Column(
                name=column_name,
                datatype=datatype,
                length=column_length,
                scale=scale,
                nullable=False,
            )

        self._add_field_map(
            schema=schema,
            idms_field=raw_field_name,
            host=Naming.hv(record_name, column_name),
        )

    if primary_key and primary_key not in record.fields:
        record.fields[primary_key] = Column(
            name=primary_key,
            datatype="CHAR",
            length=12,
            scale=None,
            nullable=False,
        )

    schema.records[record_name] = record

def _parse_data_item_line(self, line: str) -> dict | None:
    match = self.DATA_LINE_PREFIX.match(line)

    if not match:
        return None

    level = int(match.group(1))
    field_name = match.group(2).upper()
    remainder = match.group(3).strip()

    if field_name == "FILLER":
        return None

    parts = remainder.split()

    if len(parts) < 2:
        return None

    if not parts[-1].isdigit() or not parts[-2].isdigit():
        return None

    length = int(parts[-1])
    start = int(parts[-2])
    body = " ".join(parts[:-2])

    return {
        "level": level,
        "field_name": field_name,
        "body": body,
        "start": start,
        "length": length,
    }

def _collect_set_roles(
    self,
    set_roles: dict[str, dict[str, str]],
    record_name: str,
    block: str,
) -> None:
    for match in self.DBKEY_SET_ROLE.finditer(block):
        set_name = match.group(1).upper()
        role = match.group(2).upper()

```

```

        if set_name == "CALC":
            continue

        set_roles.setdefault(set_name, {})

        if role == "OWNER":
            set_roles[set_name]["owner"] = record_name

        if role == "MEMBER":
            set_roles[set_name]["member"] = record_name

def _build_relationships(
    self,
    schema: SchemaModel,
    set_roles: dict[str, dict[str, str]],
) -> None:
    for set_name, roles in set_roles.items():
        owner = roles.get("owner")
        member = roles.get("member")

        if not owner or not member:
            continue

        if owner not in schema.records or member not in schema.records:
            continue

        owner_record = schema.records[owner]
        member_record = schema.records[member]

        parent_key = owner_record.primary_key

        if not parent_key:
            continue

        child_fk = parent_key

        if child_fk not in member_record.fields:
            parent_column = owner_record.fields.get(parent_key)

            member_record.fields[child_fk] = Column(
                name=child_fk,
                datatype=parent_column.datatype if parent_column else "CHAR",
                length=parent_column.length if parent_column else 12,
                scale=parent_column.scale if parent_column else None,
                nullable=True,
            )

        schema.relationships[set_name] = Relationship(
            set_name=set_name,
            parent_record=owner,
            child_record=member,
            cardinality="1:N",
            parent_key=parent_key,
            child_fk=child_fk,
            order_by=[child_fk],
        )

def _extract_usage(self, body: str) -> str:
    upper = body.upper()

    if "COMP-3" in upper:
        return "COMP-3"

    if re.search(r"\bCOMP\b", upper):
        return "COMP"

    return "DISPLAY"

def _extract_picture(self, body: str) -> str | None:
    upper = body.upper()
    matches = list(self.PICTURE_FIND.finditer(upper))

    if not matches:
        return None

    valid_matches = []

```

```

for match in matches:
    value = match.group(0).upper()
    start = match.start()
    end = match.end()

    before = upper[start - 1] if start > 0 else " "
    after = upper[end] if end < len(upper) else " "

    if before.isalnum() or before in {"-", "_":
        continue

    if after.isalnum() or after in {"-", "_":
        continue

    valid_matches.append(value)

if not valid_matches:
    return None

return valid_matches[-1]

def _map_picture(
    self,
    picture: str,
    usage: str,
    storage_length: int,
) -> tuple[str, int | None, int | None]:
    picture = picture.upper()
    usage = usage.upper()

    if picture.startswith("X"):
        length_match = re.search(r"X$(\d+)$", picture)

        if length_match:
            return "CHAR", int(length_match.group(1)), None

        return "CHAR", len(picture), None

    if "9" in picture:
        precision = self._numeric_precision(picture)
        scale = self._numeric_scale(picture)

        if usage == "DISPLAY":
            return "CHAR", storage_length, None

        if usage == "COMP":
            return "INTEGER", None, None

        return "DECIMAL", precision, scale

    return "CHAR", storage_length, None

def _numeric_precision(self, picture: str) -> int:
    picture = picture.upper().replace("S", "")

    if "V" in picture:
        before_v, after_v = picture.split("V", 1)
    else:
        before_v = picture
        after_v = ""

    return self._count_9_digits(before_v) + self._count_9_digits(after_v)

def _numeric_scale(self, picture: str) -> int:
    picture = picture.upper().replace("S", "")

    if "V" not in picture:
        return 0

    after_v = picture.split("V", 1)[1]

    return self._count_9_digits(after_v)

def _count_9_digits(self, value: str) -> int:
    total = 0

```

```

for match in re.finditer(r"9$(\d+)$|9", value):
    if match.group(1):
        total += int(match.group(1))
    else:
        total += 1

return total

def _is_date_group(
    self,
    raw_field_name: str,
    picture: str | None,
    storage_length: int,
) -> bool:
    normalized = self._normalize_column(raw_field_name)

    return (
        picture is None
        and normalized.endswith("DATE")
        and storage_length == 8
    )

def _is_date_part(self, raw_field_name: str) -> bool:
    normalized = self._normalize_column(raw_field_name)

    return (
        normalized.endswith("_YEAR")
        or normalized.endswith("_MONTH")
        or normalized.endswith("_DAY")
    )

def _add_date_part_map(
    self,
    schema: SchemaModel,
    idms_field: str,
    host: str,
) -> None:
    normalized = self._normalize_column(idms_field)

    if normalized.endswith("_YEAR"):
        part = "YEAR"
    elif normalized.endswith("_MONTH"):
        part = "MONTH"
    elif normalized.endswith("_DAY"):
        part = "DAY"
    else:
        return

    schema.date_part_map[idms_field.upper()] = {
        "host": host,
        "substring_start": self.DATE_PARTS[part]["substring_start"],
        "substring_length": self.DATE_PARTS[part]["substring_length"],
    }

def _add_field_map(
    self,
    schema: SchemaModel,
    idms_field: str,
    host: str,
) -> None:
    schema.field_map[idms_field.upper()] = {
        "host": host,
    }

def _close_inactive_date_groups(
    self,
    active_date_groups: dict[int, dict[str, str]],
    current_level: int,
) -> None:
    closed_levels = [
        level
        for level in active_date_groups
        if level >= current_level
    ]

```



```

        for level in closed_levels:
            del active_date_groups[level]

    def _nearest_date_group(
        self,
        active_date_groups: dict[int, dict[str, str]],
    ) -> dict[str, str] | None:
        if not active_date_groups:
            return None

        nearest_level = max(active_date_groups.keys())

        return active_date_groups[nearest_level]

    def _normalize_table(self, record_name: str) -> str:
        return record_name.upper().replace("-", "_")

    def _normalize_column(self, field_name: str) -> str:
        value = field_name.upper()
        value = re.sub(r"-\d{4}$", "", value)

        return value.replace("-", "_")

```

```

=====
FILE: src\idms_db2_converter\parsers\phase2_metadata_parser.py
ENCODING: utf-8
=====

```

```

import json

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import (
    Column,
    Record,
    Relationship,
    SchemaModel,
)

class Phase2MetadataParser:
    """
    Parses Phase 2 metadata JSON produced by Phase 1.

    Generic behavior only:
    - No hardcoded DB2 table names.
    - No hardcoded DB2 column names.
    - No hardcoded business record names.
    - Uses only names present in payload:
      records, fields, relationships, table_key_map, field_map, etc.

    Supports:
    - records / tables / db2_tables
    - primary_key and primary_keys
    - relationships / sets / db2_relationships
    - parent_key / parent_keys
    - child_fk / child_fks
    - record_table_map
    - field_map
    - calc_key_map
    - table_key_map
    - set_ordering_map
    - navigation_intent
    - nullable_fk_map
    - date_part_map
    - output_semantics
    - paragraph_operation_graph
    - validation_messages
    """

    def parse_as_schema(
        self,
        text: str,
    ) -> SchemaModel:
        schema = SchemaModel()

```

```

if not text or not text.strip():
    return schema

try:
    payload = json.loads(text)
except json.JSONDecodeError as exc:
    raise ConversionError(
        f"Invalid Phase 2 metadata JSON: {exc}"
    ) from exc

if not isinstance(payload, dict):
    raise ConversionError(
        "Invalid Phase 2 metadata JSON: root must be an object."
    )

self._parse_records(
    payload=payload,
    schema=schema,
)

self._merge_table_key_map(
    payload=payload,
    schema=schema,
)

self._parse_relationships(
    payload=payload,
    schema=schema,
)

self._merge_record_table_map(
    payload=payload,
    schema=schema,
)

self._merge_field_map(
    payload=payload,
    schema=schema,
)

self._merge_calc_key_map(
    payload=payload,
    schema=schema,
)

self._merge_table_key_map_to_schema(
    payload=payload,
    schema=schema,
)

self._merge_set_ordering_map(
    payload=payload,
    schema=schema,
)

self._merge_navigation_intent(
    payload=payload,
    schema=schema,
)

self._merge_nullable_fk_map(
    payload=payload,
    schema=schema,
)

self._merge_date_part_map(
    payload=payload,
    schema=schema,
)

self._merge_output_semantics(
    payload=payload,
    schema=schema,
)

self._merge_paragraph_operation_graph(

```

```

        payload=payload,
        schema=schema,
    )

    self._merge_validation_messages(
        payload=payload,
        schema=schema,
    )

    self._apply_key_flags(
        schema=schema,
    )

    schema.schema_source = "PHASE2_METADATA"

    return schema

def _parse_records(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    record_items = (
        payload.get("records")
        or payload.get("tables")
        or payload.get("db2_tables")
        or []
    )

    if isinstance(record_items, dict):
        record_items = list(record_items.values())

    for record_json in record_items:
        if not isinstance(record_json, dict):
            continue

        record_name = self.normalize_name(
            self._get_first_non_empty(
                source=record_json,
                keys=[
                    "name",
                    "record",
                    "table",
                    "table_name",
                    "record_name",
                ],
            )
        )

        if not record_name:
            continue

        primary_keys = self._extract_primary_keys(
            source=record_json,
        )

        fields = self._parse_fields(
            record_json=record_json,
            primary_keys=primary_keys,
        )

        primary_key = primary_keys[0] if primary_keys else None

        record = Record(
            name=record_name,
            primary_key=primary_key,
            primary_keys=primary_keys,
            fields=fields,
        )

        record.set_primary_keys(
            keys=primary_keys,
        )

        schema.records[record_name] = record

```

```

def _parse_fields(
    self,
    record_json: dict,
    primary_keys: list[str],
) -> dict[str, Column]:
    fields: dict[str, Column] = {}

    field_items = (
        record_json.get("fields")
        or record_json.get("columns")
        or []
    )

    if isinstance(field_items, dict):
        field_items = list(field_items.values())

    primary_key_set = {
        self.normalize_name(primary_key)
        for primary_key in primary_keys
        if primary_key
    }

    for field_json in field_items:
        if not isinstance(field_json, dict):
            continue

        field_name = self.normalize_name(
            self._get_first_non_empty(
                source=field_json,
                keys=[
                    "name",
                    "field",
                    "column",
                    "column_name",
                ],
            )
        )

        if not field_name:
            continue

        datatype = (
            self._get_first_non_empty(
                source=field_json,
                keys=[
                    "datatype",
                    "data_type",
                    "type",
                ],
            )
            or "CHAR"
        )

        datatype = str(datatype).upper()

        nullable = field_json.get("nullable")

        if nullable is None:
            nullable = field_name not in primary_key_set

        primary_key_flag = bool(
            field_json.get("primary_key", False)
            or field_name in primary_key_set
        )

        fields[field_name] = Column(
            name=field_name,
            datatype=datatype,
            length=self._safe_int_or_none(
                field_json.get("length"),
            ),
            scale=self._safe_int_or_none(
                field_json.get("scale"),
            ),
            nullable=bool(nullable) if not primary_key_flag else False,
            primary_key=primary_key_flag,

```

```

        generated=bool(field_json.get("generated", False)),
        source_kind=str(field_json.get("source_kind", "") or ""),
    )

    return fields

def _parse_relationships(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    relationship_items = (
        payload.get("relationships")
        or payload.get("sets")
        or payload.get("db2_relationships")
        or []
    )

    if isinstance(relationship_items, dict):
        relationship_items = list(relationship_items.values())

    for rel_json in relationship_items:
        if not isinstance(rel_json, dict):
            continue

        set_name = self.normalize_name(
            self._get_first_non_empty(
                source=rel_json,
                keys=[
                    "set_name",
                    "name",
                    "set",
                ],
            )
        )

        parent_record = self.normalize_name(
            self._get_first_non_empty(
                source=rel_json,
                keys=[
                    "parent_record",
                    "owner_record",
                    "parent",
                    "owner",
                    "parent_table",
                ],
            )
        )

        child_record = self.normalize_name(
            self._get_first_non_empty(
                source=rel_json,
                keys=[
                    "child_record",
                    "member_record",
                    "child",
                    "member",
                    "child_table",
                ],
            )
        )

        if not set_name or not parent_record or not child_record:
            continue

        parent_keys = self._extract_key_list(
            source=rel_json,
            plural_keys=[
                "parent_keys",
                "owner_keys",
            ],
            single_keys=[
                "parent_key",
                "owner_key",
            ],
        )

```

```

        child_fks = self._extract_key_list(
            source=rel_json,
            plural_keys=[
                "child_fks",
                "member_fks",
                "foreign_keys",
            ],
            single_keys=[
                "child_fk",
                "member_fk",
                "foreign_key",
            ],
        )

    if not parent_keys:
        parent_key = self._resolve_parent_key(
            schema=schema,
            parent_record=parent_record,
        )

        if parent_key:
            parent_keys = [parent_key]

    if not child_fks:
        child_fk = self._resolve_child_fk(
            schema=schema,
            parent_record=parent_record,
            child_record=child_record,
            parent_keys=parent_keys,
        )

        if child_fk:
            child_fks = [child_fk]

    order_by = [
        self.normalize_name(value)
        for value in rel_json.get("order_by", []) or []
        if value
    ]

    if not order_by and child_fks:
        order_by = child_fks.copy()

    parent_key_single = parent_keys[0] if parent_keys else None
    child_fk_single = child_fks[0] if child_fks else None

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=parent_record,
        child_record=child_record,
        cardinality=rel_json.get("cardinality") or "1:N",
        parent_key=parent_key_single,
        child_fk=child_fk_single,
        parent_keys=parent_keys,
        child_fks=child_fks,
        order_by=order_by,
    )

def _merge_table_key_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    table_key_map = payload.get("table_key_map") or {}

    if not isinstance(table_key_map, dict):
        return

    for table_name_raw, key_payload in table_key_map.items():
        table_name = self.normalize_name(table_name_raw)

        if not table_name:
            continue

        if not isinstance(key_payload, dict):

```

```

        continue

    primary_keys = self._extract_key_list(
        source=key_payload,
        plural_keys=[
            "primary_keys",
            "pk_columns",
        ],
        single_keys=[
            "primary_key",
            "pk",
        ],
    )

    if not primary_keys:
        continue

    record = schema.records.get(table_name)

    if record is None:
        continue

    record.set_primary_keys(
        keys=primary_keys,
    )

def _merge_table_key_map_to_schema(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    table_key_map = payload.get("table_key_map") or {}

    if not isinstance(table_key_map, dict):
        return

    schema.table_key_map = {}

    for table_name, value in table_key_map.items():
        normalized_table_name = self.normalize_name(table_name)

        if not normalized_table_name:
            continue

        if isinstance(value, dict):
            schema.table_key_map[normalized_table_name] = self._normalize_nested(
                value=value,
            )
        else:
            schema.table_key_map[normalized_table_name] = value

def _merge_record_table_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    for key, value in (payload.get("record_table_map") or {}).items():
        schema.record_table_map[self.normalize_name(key)] = self.normalize_name(value)

def _merge_field_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    for key, value in (payload.get("field_map") or {}).items():
        schema.field_map[self.normalize_name(key)] = value

def _merge_calc_key_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    for key, value in (payload.get("calc_key_map") or {}).items():
        schema.calc_key_map[self.normalize_name(key)] = value

def _merge_set_ordering_map(

```

```

self,
payload: dict,
schema: SchemaModel,
) -> None:
    for key, value in (payload.get("set_ordering_map") or {}).items():
        schema.set_ordering_map[self.normalize_name(key)] = value

        relationship = schema.relationships.get(
            self.normalize_name(key),
        )

        if relationship is None:
            continue

        if isinstance(value, dict):
            parent_keys = self._extract_key_list(
                source=value,
                plural_keys=[
                    "parent_keys",
                ],
                single_keys=[
                    "parent_key",
                ],
            )

            child_fks = self._extract_key_list(
                source=value,
                plural_keys=[
                    "child_fks",
                ],
                single_keys=[
                    "child_fk",
                ],
            )

            if parent_keys:
                relationship.parent_keys = parent_keys
                relationship.parent_key = parent_keys[0]

            if child_fks:
                relationship.child_fks = child_fks
                relationship.child_fk = child_fks[0]

            order_by = value.get("order_by") or []

            if order_by:
                relationship.order_by = [
                    self.normalize_name(item)
                    for item in order_by
                    if item
                ]

def _merge_navigation_intent(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("navigation_intent") or {}

    if isinstance(value, dict):
        schema.navigation_intent = value

def _merge_nullable_fk_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("nullable_fk_map") or {}

    if isinstance(value, dict):
        schema.nullable_fk_map = value

def _merge_date_part_map(
    self,
    payload: dict,
    schema: SchemaModel,

```



```

) -> None:
    value = payload.get("date_part_map") or {}

    if isinstance(value, dict):
        schema.date_part_map = {
            self.normalize_name(key): item
            for key, item in value.items()
        }

def _merge_output_semantics(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("output_semantics") or {}

    if isinstance(value, dict):
        schema.output_semantics = value

def _merge_paragraph_operation_graph(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("paragraph_operation_graph") or {}

    if isinstance(value, dict):
        schema.paragraph_operation_graph = value

def _merge_validation_messages(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    messages = payload.get("validation_messages") or []

    if isinstance(messages, list):
        schema.validation_messages.extend(
            str(message)
            for message in messages
        )

def _apply_key_flags(
    self,
    schema: SchemaModel,
) -> None:
    for record in schema.records.values():
        primary_keys = record.effective_primary_keys()

        for primary_key in primary_keys:
            if primary_key not in record.fields:
                continue

            record.fields[primary_key].primary_key = True
            record.fields[primary_key].nullable = False

        if not record.primary_key and primary_keys:
            record.primary_key = primary_keys[0]

def _resolve_parent_key(
    self,
    schema: SchemaModel,
    parent_record: str,
) -> str | None:
    record = schema.records.get(parent_record)

    if not record:
        return None

    primary_keys = record.effective_primary_keys()

    if primary_keys:
        return primary_keys[0]

    return record.primary_key

```

```

def _resolve_child_fk(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
    parent_keys: list[str],
) -> str | None:
    child = schema.records.get(child_record)

    if not child:
        return None

    for parent_key in parent_keys or []:
        if parent_key in child.fields:
            return parent_key

    parent = schema.records.get(parent_record)

    if parent is None:
        return None

    parent_effective_keys = parent.effective_primary_keys()

    for parent_key in parent_effective_keys:
        if parent_key in child.fields:
            return parent_key

    parent_base_keys = {
        self.remove_record_suffix(parent_key): parent_key
        for parent_key in parent_effective_keys
        if parent_key
    }

    for child_field_name in child.fields:
        child_base_name = self.remove_record_suffix(child_field_name)

        if child_base_name in parent_base_keys:
            return child_field_name

    return None

def _extract_primary_keys(
    self,
    source: dict,
) -> list[str]:
    return self._extract_key_list(
        source=source,
        plural_keys=[
            "primary_keys",
            "pk_columns",
        ],
        single_keys=[
            "primary_key",
            "pk",
        ],
    )

def _extract_key_list(
    self,
    source: dict,
    plural_keys: list[str],
    single_keys: list[str],
) -> list[str]:
    values: list[str] = []

    for key in plural_keys:
        raw_value = source.get(key)

        if raw_value is None:
            continue

        if isinstance(raw_value, list):
            for item in raw_value:
                normalized = self.normalize_name(item)

                if normalized and normalized not in values:

```

```

        values.append(normalized)

    elif isinstance(raw_value, str):
        normalized = self.normalize_name(raw_value)

        if normalized and normalized not in values:
            values.append(normalized)

for key in single_keys:
    raw_value = source.get(key)

    if raw_value is None:
        continue

    normalized = self.normalize_name(raw_value)

    if normalized and normalized not in values:
        values.append(normalized)

return values

def _get_first_non_empty(
    self,
    source: dict,
    keys: list[str],
) -> str | None:
    for key in keys:
        value = source.get(key)

        if value is None:
            continue

        if isinstance(value, str):
            if value.strip():
                return value

        else:
            return str(value)

    return None

def _normalize_nested(
    self,
    value,
):
    if isinstance(value, dict):
        return {
            self.normalize_name(key): self._normalize_nested(item)
            for key, item in value.items()
        }

    if isinstance(value, list):
        return [
            self._normalize_nested(item)
            for item in value
        ]

    if isinstance(value, str):
        return self.normalize_name(value)

    return value

def _safe_int_or_none(
    self,
    value,
) -> int | None:
    try:
        if value is None:
            return None

        return int(value)
    except Exception:
        return None

def normalize_name(
    self,

```

```

        value,
    ) -> str:
        if value is None:
            return ""

        text = str(value).strip().upper()

        if not text:
            return ""

        text = text.replace("-", "_")
        text = text.replace(" ", "_")

        while "__" in text:
            text = text.replace("__", "_")

        return text.strip("_")

def remove_record_suffix(
    self,
    value: str,
) -> str:
    text = self.normalize_name(value)

    if not text:
        return ""

    parts = text.rsplit("_", 1)

    if len(parts) == 2 and parts[1].isdigit() and len(parts[1]) == 4:
        return parts[0]

    return text

```

```

=====
FILE: src\idms_db2_converter\services\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_converter\services\conversion_service.py
ENCODING: utf-8
=====

```

```

from copy import deepcopy
from time import perf_counter
from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.parsers.cobol_parser import CobolParser
from idms_db2_converter.parsers.canonical_parser import CanonicalParser
from idms_db2_converter.parsers.ddl_parser import DDLParser
from idms_db2_converter.parsers.phase2_metadata_parser import Phase2MetadataParser
from idms_db2_converter.parsers.idms_schema_parser import IdmsSchemaParser
from idms_db2_converter.services.relationship_resolver import RelationshipResolver
from idms_db2_converter.services.schema_merger import SchemaMerger
from idms_db2_converter.transformers.retrieval_transformer import RetrievalTransformer
from idms_db2_converter.validators.output_validator import OutputValidator
from idms_db2_converter.validators.schema_validator import SchemaValidator

```

```

class ConversionService:
    DEBUG_SCHEMA = False

    def convert_retrieval(
        self,
        cobol_text: str,
        canonical_json: str,
        phase2_metadata_json: str | None = None,
        ddl_text: str | None = None,
        idms_schema_text: str | None = None,
        relationship_overrides_json: str | None = None,
        target_program: str | None = None,
    ) -> str:

```

```

) -> tuple[str, list[str]]:
    timings: list[str] = []
    total_start = perf_counter()
    step_start = perf_counter()

    def add_timing(
        label: str,
    ) -> None:
        nonlocal step_start

        elapsed = perf_counter() - step_start
        message = f"Phase 2 - {label}: {elapsed:.2f} seconds"

        print(
            message,
        )

        timings.append(
            message,
        )

        step_start = perf_counter()

    analysis = CobolParser().parse(
        cobol_text,
    )

    add_timing(
        "COBOL parse",
    )

    schema_builder = (
        getattr(
            self,
            "_build_schema_from_available_sources",
            None,
        )
        or getattr(
            self,
            "build_schema_from_available_sources",
        )
    )

    schema = schema_builder(
        canonical_json=canonical_json,
        phase2_metadata_json=phase2_metadata_json,
        ddl_text=ddl_text,
        idms_schema_text=idms_schema_text,
    )

    add_timing(
        "Build schema from available sources",
    )

    if self.DEBUG_SCHEMA:
        self.debug_schema(
            schema,
        )

    add_timing(
        "Debug schema",
    )

    if (
        relationship_overrides_json
        and relationship_overrides_json.strip()
    ):
        schema = RelationshipResolver().apply_overrides(
            schema=schema,
            relationship_overrides_json=relationship_overrides_json,
        )

    add_timing(
        "Apply relationship overrides",
    )

```

```

schema = RelationshipResolver().resolve_cobol_sets(
    schema=schema,
    analysis=analysis,
)

add_timing(
    "Resolve COBOL sets",
)

schema_errors = SchemaValidator().validate(
    schema,
    analysis,
)

add_timing(
    "Schema validation",
)

if schema_errors:
    raise ConversionError(
        "\n".join(schema_errors),
    )

converted = RetrievalTransformer(
    schema,
    analysis,
).transform(
    cobol=cobol_text,
    target_program=target_program,
)

add_timing(
    "Retrieval transformer",
)

output_errors = OutputValidator().validate(
    converted,
)

add_timing(
    "Output validation",
)

validation_messages = []

validation_messages.extend(
    getattr(
        schema,
        "validation_messages",
        [],
    )
)

validation_messages.extend(
    output_errors,
)

total_elapsed = perf_counter() - total_start

timings.append(
    f"Phase 2 - TOTAL: {total_elapsed:.2f} seconds",
)

print("=" * 80)
print("PHASE 2 TIMINGS")
print("=" * 80)

for timing in timings:
    print(
        timing,
    )

validation_messages.extend(
    timings,
)

```

```

        return converted, validation_messages

def _build_schema_from_available_sources(
    self,
    canonical_json: str | None,
    phase2_metadata_json: str | None,
    ddl_text: str | None,
    idms_schema_text: str | None,
) -> SchemaModel:
    canonical_has_text = bool(canonical_json and canonical_json.strip())
    metadata_has_text = bool(phase2_metadata_json and phase2_metadata_json.strip())
    ddl_has_text = bool(ddl_text and ddl_text.strip())
    idms_schema_has_text = bool(idms_schema_text and idms_schema_text.strip())

    canonical_schema = SchemaModel()
    metadata_schema = SchemaModel()
    ddl_schema = SchemaModel()
    idms_schema = SchemaModel()

    if canonical_has_text:
        canonical_schema = CanonicalParser().parse(canonical_json)
        canonical_schema.schema_source = "CANONICAL"

    if metadata_has_text:
        metadata_schema = Phase2MetadataParser().parse_as_schema(
            phase2_metadata_json
        )
        metadata_schema.schema_source = "PHASE2_METADATA"

    if ddl_has_text:
        ddl_schema = DDLParser().parse(ddl_text)

        if self._has_schema_objects(ddl_schema):
            ddl_schema.schema_source = "DDL"
        elif self._looks_like_idms_schema(ddl_text):
            idms_schema = IdmsSchemaParser().parse(ddl_text)
            idms_schema.schema_source = "IDMS_SCHEMA"
        elif "CREATE TABLE" in ddl_text.upper():
            raise ConversionError(
                "DDL text was provided and contains CREATE TABLE, but no tables were parsed."
            )

    if idms_schema_has_text:
        idms_schema = IdmsSchemaParser().parse(idms_schema_text)
        idms_schema.schema_source = "IDMS_SCHEMA"

    if self._has_schema_objects(ddl_schema):
        base_schema = ddl_schema
        base_schema.schema_source = "DDL"

    if self._has_schema_objects(canonical_schema):
        base_schema = self._merge_non_physical_metadata(
            physical_schema=base_schema,
            overlay=canonical_schema,
        )

    if metadata_has_text:
        base_schema = Phase2MetadataParser().parse_into_schema(
            text=phase2_metadata_json,
            schema=base_schema,
        )

    if self._has_schema_objects(idms_schema):
        base_schema = SchemaMerger().merge_physical_with_idms(
            physical_schema=base_schema,
            idms_schema=idms_schema,
        )

    return base_schema

    if self._has_schema_objects(canonical_schema):
        base_schema = canonical_schema

    if metadata_has_text:
        base_schema = Phase2MetadataParser().parse_into_schema(
            text=phase2_metadata_json,

```

```

        schema=base_schema,
    )

    if self._has_schema_objects(idms_schema):
        base_schema = self._merge_schema(base_schema, idms_schema)

    return base_schema

if self._has_schema_objects(metadata_schema):
    base_schema = metadata_schema

    if self._has_schema_objects(idms_schema):
        base_schema = self._merge_schema(base_schema, idms_schema)

    return base_schema

if self._has_schema_objects(idms_schema):
    return idms_schema

raise ConversionError(
    "No usable schema source found. Provide canonical model JSON, "
    "Phase 2 metadata JSON with records/relationships, DB2 DDL, "
    "or IDMS schema listing."
)

def _merge_schema(
    self,
    base: SchemaModel,
    overlay: SchemaModel,
) -> SchemaModel:
    merged = deepcopy(base)

    for record_name, overlay_record in overlay.records.items():
        if record_name in merged.records:
            merged.records[record_name].primary_key = (
                overlay_record.primary_key
                or merged.records[record_name].primary_key
            )

            for field_name, overlay_field in overlay_record.fields.items():
                merged.records[record_name].fields[field_name] = overlay_field
        else:
            merged.records[record_name] = overlay_record

    for set_name, overlay_rel in overlay.relationships.items():
        if set_name not in merged.relationships:
            merged.relationships[set_name] = overlay_rel
            continue

        rel = merged.relationships[set_name]
        rel.parent_record = overlay_rel.parent_record or rel.parent_record
        rel.child_record = overlay_rel.child_record or rel.child_record
        rel.parent_key = overlay_rel.parent_key or rel.parent_key
        rel.child_fk = overlay_rel.child_fk or rel.child_fk
        rel.cardinality = overlay_rel.cardinality or rel.cardinality

        if overlay_rel.order_by:
            rel.order_by = overlay_rel.order_by

    self._merge_dict(merged.field_map, overlay.field_map)
    self._merge_dict(merged.record_table_map, overlay.record_table_map)
    self._merge_dict(merged.calc_key_map, overlay.calc_key_map)
    self._merge_dict(merged.set_ordering_map, overlay.set_ordering_map)
    self._merge_dict(merged.navigation_intent, overlay.navigation_intent)
    self._merge_dict(merged.nullable_fk_map, overlay.nullable_fk_map)
    self._merge_dict(merged.date_part_map, overlay.date_part_map)
    self._merge_dict(merged.output_semantics, overlay.output_semantics)
    self._merge_dict(
        merged.paragraph_operation_graph,
        overlay.paragraph_operation_graph,
    )

    if hasattr(merged, "validation_messages") and hasattr(
        overlay,
        "validation_messages",
    ):

```



```

        merged.validation_messages.extend(overlay.validation_messages)

    if not getattr(merged, "schema_source", None):
        merged.schema_source = getattr(overlay, "schema_source", None)

    return merged

def _merge_non_physical_metadata(
    self,
    physical_schema: SchemaModel,
    overlay: SchemaModel,
) -> SchemaModel:
    merged = deepcopy(physical_schema)

    self._merge_dict(merged.field_map, overlay.field_map)

    for key, value in overlay.record_table_map.items():
        if key not in merged.record_table_map:
            merged.record_table_map[key] = value

    self._merge_dict(merged.calc_key_map, overlay.calc_key_map)
    self._merge_dict(merged.set_ordering_map, overlay.set_ordering_map)
    self._merge_dict(merged.navigation_intent, overlay.navigation_intent)
    self._merge_dict(merged.nullable_fk_map, overlay.nullable_fk_map)
    self._merge_dict(merged.date_part_map, overlay.date_part_map)
    self._merge_dict(merged.output_semantics, overlay.output_semantics)
    self._merge_dict(
        merged.paragraph_operation_graph,
        overlay.paragraph_operation_graph,
    )

    for set_name, relationship in overlay.relationships.items():
        if set_name not in merged.relationships:
            merged.relationships[set_name] = relationship

    if hasattr(merged, "validation_messages") and hasattr(
        overlay,
        "validation_messages",
    ):
        merged.validation_messages.extend(overlay.validation_messages)

    merged.schema_source = "DDL+CANONICAL_METADATA"

    return merged

def _merge_dict(self, target: dict, source: dict) -> None:
    for key, value in source.items():
        target[key] = value

def _has_schema_objects(self, schema: SchemaModel) -> bool:
    if hasattr(schema, "has_schema_objects"):
        return schema.has_schema_objects()

    return bool(schema.records or schema.relationships)

def _looks_like_idms_schema(self, text: str | None) -> bool:
    if not text:
        return False

    upper = text.upper()

    return (
        "RECORD NAME" in upper
        and "DBKEY POSITIONS" in upper
        and "DATA ITEM" in upper
    )

def _debug_schema(self, schema: SchemaModel) -> None:
    print("FINAL SCHEMA SOURCE:", getattr(schema, "schema_source", None))
    print("RECORD COUNT:", len(schema.records))
    print("RELATIONSHIP COUNT:", len(schema.relationships))
    print("RECORD TABLE MAP COUNT:", len(schema.record_table_map))
    print("FIELD MAP COUNT:", len(schema.field_map))
    print("DATE PART MAP COUNT:", len(schema.date_part_map))

    for record_name in sorted(schema.records.keys()):

```

```

        record = schema.records[record_name]
        print(f"RECORD: {record_name} PK={record.primary_key}")

        for field_name in sorted(record.fields.keys()):
            field = record.fields[field_name]
            print(
                "  FIELD:",
                field_name,
                field.datatype,
                field.length,
                field.scale,
                "NULLABLE=",
                field.nullable,
            )

    print("RELATIONSHIPS:")

    for set_name in sorted(schema.relationships.keys()):
        rel = schema.relationships[set_name]
        print(
            "  REL:",
            set_name,
            rel.parent_record,
            rel.parent_key,
            "->",
            rel.child_record,
            rel.child_fk,
        )

```

```

=====
FILE: src\idms_db2_converter\services\relationship_resolver.py
ENCODING: utf-8
=====

```

```
import json
```

```

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import CobolAnalysis, Relationship, SchemaModel

```

```
class RelationshipResolver:
```

```
    """
```

```
    Resolves COBOL IDMS set names to schema relationships.
```

```
    No business-specific set names are hard-coded.
```

```
    Resolution order:
```

1. Use exact set name if already present.
2. Use relationship overrides if provided.
3. Use DDL-derived relationships if a COBOL set can be matched unambiguously.

```
    """
```

```
def apply_overrides(
```

```
    self,
```

```
    schema: SchemaModel,
```

```
    relationship_overrides_json: str,
```

```
) -> SchemaModel:
```

```
    try:
```

```
        payload = json.loads(relationship_overrides_json)
```

```
    except json.JSONDecodeError as exc:
```

```
        raise ConversionError(f"Invalid relationship override JSON: {exc}") from exc
```

```
    self._apply_record_aliases(schema, payload)
```

```
    for item in payload.get("relationships", []):
```

```
        self._apply_relationship_override(schema, item)
```

```
    return schema
```

```
def resolve_cobol_sets(
```

```
    self,
```

```
    schema: SchemaModel,
```

```
    analysis: CobolAnalysis,
```

```
) -> SchemaModel:
```

```
    references = self._collect_set_references(analysis)
```

```

for set_name, child_records in references.items():
    if set_name in schema.relationships:
        continue

    matched = self._find_unambiguous_relationship(
        schema=schema,
        set_name=set_name,
        child_records=child_records,
    )

    if not matched:
        continue

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=matched.parent_record,
        child_record=matched.child_record,
        cardinality=matched.cardinality,
        parent_key=matched.parent_key,
        child_fk=matched.child_fk,
        order_by=matched.order_by,
    )

    schema.add_validation_message(
        f"Set {set_name} resolved from existing schema relationship "
        f"{matched.parent_record}-{matched.child_record}."
    )

return schema

def _collect_set_references(
    self,
    analysis: CobolAnalysis,
) -> dict[str, set[str]]:
    references: dict[str, set[str]] = {}

    for record_name, set_name in analysis.obtain_next:
        references.setdefault(set_name, set()).add(record_name)

    for set_name in analysis.obtain_owner_sets:
        references.setdefault(set_name, set())

    for set_name in analysis.find_first_sets:
        references.setdefault(set_name, set())

    return references

def _apply_record_aliases(self, schema: SchemaModel, payload: dict) -> None:
    for item in payload.get("record_aliases", []):
        record_name = item["record"].upper()
        table_name = item["table"].upper()

        if table_name not in schema.records:
            raise ConversionError(
                f"Record alias {record_name} references unknown table {table_name}."
            )

        schema.records[record_name] = schema.records[table_name]
        schema.record_table_map[record_name] = table_name

def _apply_relationship_override(self, schema: SchemaModel, item: dict) -> None:
    set_name = item["set_name"].upper()

    parent_record = self._upper_or_none(item.get("parent_record"))
    child_record = self._upper_or_none(item.get("child_record"))
    parent_key = self._upper_or_none(item.get("parent_key"))
    child_fk = self._upper_or_none(item.get("child_fk"))
    order_by = [value.upper() for value in item.get("order_by", [])]

    if set_name in schema.relationships:
        rel = schema.relationships[set_name]

        if parent_record:
            rel.parent_record = parent_record

```

```

        if child_record:
            rel.child_record = child_record

        if parent_key:
            rel.parent_key = parent_key

        if child_fk:
            rel.child_fk = child_fk

        if order_by:
            rel.order_by = order_by

    return

if parent_record and child_record:
    self._create_explicit_relationship(
        schema=schema,
        set_name=set_name,
        parent_record=parent_record,
        child_record=child_record,
        parent_key=parent_key,
        child_fk=child_fk,
        order_by=order_by,
    )
    return

matched = self._find_relationship_by_keys(
    schema=schema,
    parent_key=parent_key,
    child_fk=child_fk,
)

if matched:
    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=matched.parent_record,
        child_record=matched.child_record,
        cardinality=matched.cardinality,
        parent_key=parent_key or matched.parent_key,
        child_fk=child_fk or matched.child_fk,
        order_by=order_by or matched.order_by,
    )
    return

raise ConversionError(
    f"Override references set {set_name}, but it could not be resolved. "
    "Provide parent_record and child_record, or provide keys matching a DDL foreign key."
)

def _create_explicit_relationship(
    self,
    schema: SchemaModel,
    set_name: str,
    parent_record: str,
    child_record: str,
    parent_key: str | None,
    child_fk: str | None,
    order_by: list[str],
) -> None:
    if parent_record not in schema.records:
        raise ConversionError(
            f"Set {set_name} parent_record {parent_record} is not present in schema."
        )

    if child_record not in schema.records:
        raise ConversionError(
            f"Set {set_name} child_record {child_record} is not present in schema."
        )

    parent = schema.records[parent_record]
    child = schema.records[child_record]

    resolved_parent_key = parent_key or parent.primary_key

    if not resolved_parent_key:
        raise ConversionError(

```

```

        f"Set {set_name} cannot resolve parent_key."
    )

    resolved_child_fk = child_fk

    if not resolved_child_fk and resolved_parent_key in child.fields:
        resolved_child_fk = resolved_parent_key

    if not resolved_child_fk:
        raise ConversionError(
            f"Set {set_name} cannot resolve child_fk."
        )

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=parent_record,
        child_record=child_record,
        cardinality="1:N",
        parent_key=resolved_parent_key,
        child_fk=resolved_child_fk,
        order_by=order_by or [resolved_child_fk],
    )

def _find_relationship_by_keys(
    self,
    schema: SchemaModel,
    parent_key: str | None,
    child_fk: str | None,
) -> Relationship | None:
    if not parent_key and not child_fk:
        return None

    matches = []

    for rel in schema.relationships.values():
        if parent_key and rel.parent_key != parent_key:
            continue

        if child_fk and rel.child_fk != child_fk:
            continue

        matches.append(rel)

    if len(matches) == 1:
        return matches[0]

    if len(matches) > 1:
        raise ConversionError(
            "Relationship override is ambiguous. Add parent_record and child_record."
        )

    return None

def _find_unambiguous_relationship(
    self,
    schema: SchemaModel,
    set_name: str,
    child_records: set[str],
) -> Relationship | None:
    candidates = list(schema.relationships.values())

    if child_records:
        filtered = [
            rel
            for rel in candidates
            if rel.child_record in child_records
        ]

        if filtered:
            candidates = filtered

    if len(candidates) == 1:
        return candidates[0]

    scored = []

```

```

    for rel in candidates:
        score = self._score_relationship(set_name, rel)

        if score > 0:
            scored.append((score, rel))

    if not scored:
        return None

    scored.sort(key=lambda item: item[0], reverse=True)

    top_score = scored[0][0]
    top_matches = [
        rel
        for score, rel in scored
        if score == top_score
    ]

    if len(top_matches) == 1:
        return top_matches[0]

    return None

def _score_relationship(self, set_name: str, rel: Relationship) -> int:
    tokens = [
        token
        for token in set_name.replace("_", "-").split("-")
        if token
    ]

    searchable_values = [
        rel.parent_record,
        rel.child_record,
        rel.parent_key or "",
        rel.child_fk or "",
    ]

    score = 0

    for token in tokens:
        for value in searchable_values:
            if self._token_matches(token, value):
                score += 1

    return score

def _token_matches(self, token: str, value: str) -> bool:
    token = token.upper()
    value = value.upper()

    return token in value or value in token

def _upper_or_none(self, value: str | None) -> str | None:
    return value.upper() if value else None

```

```

=====
FILE: src\idms_db2_converter\services\schema_merger.py
ENCODING: utf-8
=====

```

```

from copy import deepcopy
import re

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import Relationship, SchemaModel

```

```

class SchemaMerger:
    """
    Merge DB2 DDL schema with IDMS schema listing.

    Rules:
    - DDL wins for physical column definitions.
    - IDMS wins for legacy field names, IDMS set names, and field_map.
    """

```

```

def merge_physical_with_idms(
    self,
    physical_schema: SchemaModel,
    idms_schema: SchemaModel,
) -> SchemaModel:
    merged = deepcopy(physical_schema)

    if not getattr(merged, "schema_source", None):
        merged.schema_source = "DDL"

    merged.schema_source = f"{merged.schema_source}+IDMS_SCHEMA"

    self._build_record_table_map(merged, idms_schema)
    self._build_field_map_from_idms_to_physical(merged, idms_schema)
    self._build_date_part_map_from_idms_to_physical(merged, idms_schema)
    self._preserve_idms_set_names(merged, idms_schema)
    self._merge_validation_messages(merged, idms_schema)

    return merged

def _build_record_table_map(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for idms_record_name in idms_schema.records:
        physical_record_name = self._find_matching_record(
            schema=merged,
            record_name=idms_record_name,
        )

        if physical_record_name:
            merged.record_table_map[idms_record_name] = physical_record_name
        else:
            merged.record_table_map[idms_record_name] = self._table_name(
                idms_record_name
            )

def _build_field_map_from_idms_to_physical(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for idms_record_name in idms_schema.records:
        physical_record_name = merged.record_table_map.get(idms_record_name)

        if not physical_record_name:
            continue

        if physical_record_name not in merged.records:
            continue

        physical_record = merged.records[physical_record_name]

        for idms_field_name, meta in idms_schema.field_map.items():
            idms_host = str(meta.get("host", "")).upper()

            if not self._host_belongs_to_record(idms_host, idms_record_name):
                continue

            normalized_idms_field = self._normalize_field_name(idms_field_name)

            physical_column = self._find_physical_column(
                physical_record=physical_record,
                normalized_idms_field=normalized_idms_field,
                idms_host=idms_host,
                idms_record_name=idms_record_name,
            )

            if not physical_column:
                continue

            merged.field_map[idms_field_name.upper()] = {
                "host": Naming.hv(idms_record_name, physical_column)
            }

```

```

def _build_date_part_map_from_idms_to_physical(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for idms_field_name, meta in idms_schema.date_part_map.items():
        idms_host = str(meta.get("host", "")).upper()

        idms_record_name = self._find_record_for_host(
            host=idms_host,
            record_names=list(idms_schema.records.keys()),
        )

        if not idms_record_name:
            continue

        physical_record_name = merged.record_table_map.get(idms_record_name)

        if not physical_record_name:
            continue

        if physical_record_name not in merged.records:
            continue

        physical_record = merged.records[physical_record_name]

        idms_column = self._column_from_host(idms_host, idms_record_name)

        if not idms_column:
            continue

        physical_column = self._find_physical_column(
            physical_record=physical_record,
            normalized_idms_field=idms_column,
            idms_host=idms_host,
            idms_record_name=idms_record_name,
        )

        if not physical_column:
            continue

        column = physical_record.fields[physical_column]

        substring_start = meta.get("substring_start")
        substring_length = meta.get("substring_length")

        if column.datatype.upper() == "DATE":
            part = self._date_part_name(idms_field_name)

            if part == "YEAR":
                substring_start = 3
                substring_length = 2
            elif part == "MONTH":
                substring_start = 6
                substring_length = 2
            elif part == "DAY":
                substring_start = 9
                substring_length = 2

            merged.date_part_map[idms_field_name.upper()] = {
                "host": Naming.hv(idms_record_name, physical_column),
                "substring_start": substring_start,
                "substring_length": substring_length,
            }

def _preserve_idms_set_names(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for set_name, idms_rel in idms_schema.relationships.items():
        idms_parent = idms_rel.parent_record
        idms_child = idms_rel.child_record

        physical_parent = merged.record_table_map.get(idms_parent)

```



```

physical_child = merged.record_table_map.get(idms_child)

if not physical_parent or not physical_child:
    continue

ddl_rel = self._find_relationship_by_records(
    schema=merged,
    parent_record=physical_parent,
    child_record=physical_child,
)

if ddl_rel:
    parent_key = ddl_rel.parent_key
    child_fk = ddl_rel.child_fk
    order_by = ddl_rel.order_by
else:
    parent_key = self._find_matching_column_name(
        record=merged.records[physical_parent],
        name=idms_rel.parent_key or "",
    )

    child_fk = self._find_matching_column_name(
        record=merged.records[physical_child],
        name=idms_rel.child_fk or idms_rel.parent_key or "",
    )

    order_by = [child_fk] if child_fk else []

if not parent_key or not child_fk:
    continue

merged.relationships[set_name] = Relationship(
    set_name=set_name,
    parent_record=idms_parent,
    child_record=idms_child,
    cardinality=idms_rel.cardinality or "1:N",
    parent_key=parent_key,
    child_fk=child_fk,
    order_by=order_by or [child_fk],
)

def _find_physical_column(
    self,
    physical_record,
    normalized_idms_field: str,
    idms_host: str,
    idms_record_name: str,
) -> str | None:
    direct = self._find_matching_column_name(
        record=physical_record,
        name=normalized_idms_field,
    )

    if direct:
        return direct

    from_host = self._column_from_host(idms_host, idms_record_name)

    if from_host:
        direct = self._find_matching_column_name(
            record=physical_record,
            name=from_host,
        )

        if direct:
            return direct

    singular = self._singularize(normalized_idms_field)

    if singular != normalized_idms_field:
        direct = self._find_matching_column_name(
            record=physical_record,
            name=singular,
        )

    if direct:

```

```

        return direct

    return self._find_flattened_occurs_column(
        physical_record=physical_record,
        name=normalized_idms_field,
    )

def _find_flattened_occurs_column(
    self,
    physical_record,
    name: str,
) -> str | None:
    candidates = [
        column_name
        for column_name in physical_record.fields
        if column_name.upper().startswith(name.upper() + "_")
    ]

    if not candidates:
        singular = self._singularize(name)
        candidates = [
            column_name
            for column_name in physical_record.fields
            if column_name.upper().startswith(singular.upper() + "_")
        ]

    if not candidates:
        return None

    return sorted(candidates)[0]

def _find_matching_record(
    self,
    schema: SchemaModel,
    record_name: str,
) -> str | None:
    record_name = record_name.upper()

    if record_name in schema.records:
        return record_name

    normalized = self._normalize_record_name(record_name)

    for candidate in schema.records:
        if self._normalize_record_name(candidate) == normalized:
            return candidate

    return None

def _find_matching_column_name(
    self,
    record,
    name: str,
) -> str | None:
    normalized = self._normalize_field_name(name)

    if normalized in record.fields:
        return normalized

    for column_name in record.fields:
        if column_name.upper() == normalized:
            return column_name

    return None

def _find_relationship_by_records(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
) -> Relationship | None:
    normalized_parent = self._normalize_record_name(parent_record)
    normalized_child = self._normalize_record_name(child_record)

    for relationship in schema.relationships.values():
        if (

```

```

        self._normalize_record_name(relationship.parent_record)
        == normalized_parent
        and self._normalize_record_name(relationship.child_record)
        == normalized_child
    ):
        return relationship

    return None

def _host_belongs_to_record(
    self,
    host: str,
    record_name: str,
) -> bool:
    normalized_host = self._normalize_token(host)
    normalized_record = self._normalize_token(record_name)

    return normalized_host.startswith(f"HV_{normalized_record}_")

def _find_record_for_host(
    self,
    host: str,
    record_names: list[str],
) -> str | None:
    normalized_host = self._normalize_token(host)

    for record_name in sorted(record_names, key=len, reverse=True):
        normalized_record = self._normalize_token(record_name)

        if normalized_host.startswith(f"HV_{normalized_record}_"):
            return record_name

    return None

def _column_from_host(
    self,
    host: str,
    record_name: str,
) -> str | None:
    normalized_host = self._normalize_token(host)
    normalized_record = self._normalize_token(record_name)

    prefix = f"HV_{normalized_record}_"

    if not normalized_host.startswith(prefix):
        return None

    return normalized_host[len(prefix):].upper()

def _date_part_name(
    self,
    idms_field_name: str,
) -> str | None:
    normalized = self._normalize_field_name(idms_field_name)

    if normalized.endswith("_YEAR"):
        return "YEAR"

    if normalized.endswith("_MONTH"):
        return "MONTH"

    if normalized.endswith("_DAY"):
        return "DAY"

    return None

def _normalize_token(
    self,
    value: str,
) -> str:
    return value.upper().replace("-", "_")

def _normalize_record_name(
    self,
    value: str,
) -> str:

```

```

        return value.upper().replace("-", "_")

def _normalize_field_name(
    self,
    value: str,
) -> str:
    value = value.upper()
    value = re.sub(r"-\d{4}$", "", value)
    return value.replace("-", "_")

def _singularize(
    self,
    value: str,
) -> str:
    value = value.upper()

    if value.endswith("IES"):
        return value[:-3] + "Y"

    if value.endswith("S"):
        return value[:-1]

    return value

def _table_name(
    self,
    record_name: str,
) -> str:
    return record_name.upper().replace("-", "_")

def _merge_validation_messages(
    self,
    merged: SchemaModel,
    overlay: SchemaModel,
) -> None:
    if hasattr(merged, "validation_messages") and hasattr(
        overlay,
        "validation_messages",
    ):
        merged.validation_messages.extend(overlay.validation_messages)

```

```

=====
FILE: src\idms_db2_converter\transformers\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_converter\transformers\cobol_cleanup.py
ENCODING: utf-8
=====

```

```
import re
```

```

class CobolCleanup:
    """
    Final cleanup pass for generated COBOL.

    Goals:
    - Preserve EXEC SQL blocks.
    - Normalize PROCEDURE DIVISION formatting.
    - Fix END-IF, ELSE, CONTINUE, EXIT, GOBACK indentation.
    - Fix SQL indentation for common SQL clauses.
    - Fix READ ... AT END indentation.
    - Fix paragraph header concatenation.
    - Avoid changing DATA DIVISION and WORKING-STORAGE declarations.
    """

    EXEC_SQL_START = re.compile(
        r"^\s*EXEC\s+SQL\b",
        re.IGNORECASE,
    )

```

```

EXEC_SQL_END = re.compile(
    r"^\\s*END-EXEC\\.?\\s*$",
    re.IGNORECASE,
)

PROCEDURE_DIVISION = re.compile(
    r"^\\s*PROCEDURE\\s+DIVISION\\.",
    re.IGNORECASE | re.MULTILINE,
)

PARAGRAPH_HEADER = re.compile(
    r"^\\s*[A-Z0-9-]+\\.\\s*$",
    re.IGNORECASE,
)

COMMENT_LINE = re.compile(
    r"^\\s*\\*",
)

DEBUG_DISPLAY_LINE = re.compile(
    r"^\\s*D\\s+DISPLAY\\b",
    re.IGNORECASE,
)

def clean(
    self,
    text: str,
) -> str:
    text = self.fix_continue_concatenation(
        text,
    )

    text = self.fix_paragraph_header_concatenation(
        text,
    )

    text = self.repair_read_at_end(
        text,
    )

    text = self.remove_end_processing_sqlcode_guard(
        text,
    )

    text = self.normalize_procedure_division(
        text,
    )

    text = self.fix_db2_check_status(
        text,
    )

    text = self.force_procedure_keyword_indentation(
        text,
    )

    text = self.normalize_sql_spacing(
        text,
    )

    text = self.normalize_blank_lines(
        text,
    )

    text = self.trim_trailing_spaces(
        text,
    )

    return text

def normalize_procedure_division(
    self,
    text: str,
) -> str:
    match = self.PROCEDURE_DIVISION.search(

```

```

        text,
    )

    if not match:
        return text

    before = text[:match.start()]
    procedure = text[match.start():]

    procedure = self.normalize_lines(
        procedure,
    )

    return before + procedure

def normalize_lines(
    self,
    text: str,
) -> str:
    lines = text.splitlines()
    output: list[str] = []

    in_exec_sql = False

    for line in lines:
        stripped = line.strip()

        if not stripped:
            output.append("")
            continue

        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True
            output.append(
                "EXEC SQL",
            )
            continue

        if in_exec_sql:
            output.append(
                self.normalize_exec_sql_line(
                    stripped,
                )
            )

            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

            continue

        if self.COMMENT_LINE.match(line):
            output.append(line)
            continue

        if self.DEBUG_DISPLAY_LINE.match(line):
            output.append(stripped)
            continue

        if self.PARAGRAPH_HEADER.match(line):
            output.append(stripped)
            continue

        output.append(
            self.normalize_procedure_line(
                stripped,
            )
        )

    return "\n".join(output)

def normalize_exec_sql_line(
    self,
    stripped: str,
) -> str:
    upper = stripped.upper()

```

```

if upper == "EXEC SQL":
    return "      EXEC SQL"

if upper in {
    "END-EXEC",
    "END-EXEC.",
}:
    return "      END-EXEC."

if upper == "COMMIT":
    return "      COMMIT"

if upper in {
    "(",
    ")"",
}:
    return "      " + stripped

if stripped.startswith(":"):
    return "      " + stripped

if re.match(
    r"^[A-Z0-9_]+,?$",
    stripped,
    flags=re.IGNORECASE,
):
    return "      " + stripped

if re.match(
    r"^(DECLARE|FETCH|OPEN|CLOSE|INSERT|UPDATE|DELETE|INCLUDE|BEGIN|END)\b",
    stripped,
    flags=re.IGNORECASE,
):
    return "      " + stripped

if re.match(
    r"^SELECT\b",
    stripped,
    flags=re.IGNORECASE,
):
    return "      " + stripped

if re.match(
    r"^(INTO|FROM|WHERE|ORDER\s+BY|FETCH\s+FIRST|SET|VALUES)\b",
    stripped,
    flags=re.IGNORECASE,
):
    return "      " + stripped

if re.match(
    r"^[A-Z0-9_]+\s*=",
    stripped,
    flags=re.IGNORECASE,
):
    return "      " + stripped

return "      " + stripped

def normalize_procedure_line(
    self,
    stripped: str,
) -> str:
    upper = stripped.upper()

    if upper == "PROCEDURE DIVISION.":
        return "PROCEDURE DIVISION."

    if upper in {
        "END-IF",
        "END-IF.",
    }:
        return "      END-IF."

    if upper in {
        "END-PERFORM",
        "END-PERFORM.",
    }

```

```

    }:
        return "        END-PERFORM"

    if upper == "ELSE":
        return "        ELSE"

    if upper in {
        "CONTINUE",
        "CONTINUE.",
    }:
        return "        CONTINUE."

    if upper in {
        "EXIT",
        "EXIT.",
    }:
        return "        EXIT."

    if upper in {
        "GOBACK",
        "GOBACK.",
    }:
        return "        GOBACK."

    if upper == "THEN":
        return "        THEN"

    if upper.startswith("THEN "):
        return "        " + stripped

    if upper.startswith("AT END "):
        return "        " + stripped

    if upper.startswith("DISPLAY "):
        return "        " + stripped

    if self.is_common_procedure_statement(
        upper,
    ):
        return "        " + stripped

    return stripped

def is_common_procedure_statement(
    self,
    upper: str,
) -> bool:
    prefixes = (
        "IF ",
        "MOVE ",
        "PERFORM ",
        "READ ",
        "WRITE ",
        "OPEN ",
        "CLOSE ",
        "ADD ",
        "SUBTRACT ",
        "MULTIPLY ",
        "DIVIDE ",
        "COMPUTE ",
        "GO TO ",
        "SET ",
        "CALL ",
        "EVALUATE ",
        "WHEN ",
        "END-EVALUATE",
        "PERFORM UNTIL ",
    )

    return upper.startswith(prefixes)

def fix_db2_check_status(
    self,
    text: str,
) -> str:
    pattern = re.compile(

```



```

        r"""
        ^\s*DB2-CHECK-STATUS\.\s*
        \n\s*IF\s+SQLCODE\s+NOT\s*=\s*\0\s*
        \n\s*DISPLAY\s+'DB2\s+SQL\s+ERROR\s+SQLCODE='\s+SQLCODE\s*
        \n\s*END-IF\.?
        """
        re.IGNORECASE | re.MULTILINE | re.VERBOSE,
    )

    replacement = "\n".join(
        [
            "DB2-CHECK-STATUS.",
            "    IF SQLCODE NOT = 0",
            "    DISPLAY 'DB2 SQL ERROR SQLCODE=' SQLCODE",
            "    END-IF.",
        ]
    )

    return pattern.sub(
        replacement,
        text,
    )

def repair_read_at_end(
    self,
    text: str,
) -> str:
    lines = text.splitlines()
    output: list[str] = []

    in_exec_sql = False

    for line in lines:
        stripped = line.strip()

        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True
            output.append(line)
            continue

        if in_exec_sql:
            output.append(line)

            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

            continue

        if re.match(
            r"^AT\s+END\b",
            stripped,
            flags=re.IGNORECASE,
        ):
            output.append(
                "    " + stripped,
            )
            continue

        output.append(line)

    return "\n".join(output)

def remove_end_processing_sqlcode_guard(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"""
        (\n\s*END-PROCESSING\.\s*)
        \n\s*IF\s+SQLCODE\s+NOT\s*=\s*\0\s*
        \n\s*PERFORM\s+SQL-ERROR\.\?\s*
        \n\s*END-IF\.?
        """
        re.IGNORECASE | re.MULTILINE | re.VERBOSE,
    )

```



```

)

procedure = re.sub(
    r"(?im)^\s*ELSE\s*$",
    "    ELSE",
    procedure,
)

procedure = re.sub(
    r"(?im)^\s*GOBACK\.\?\s*$",
    "    GOBACK.",
    procedure,
)

procedure = re.sub(
    r"(?im)^\s*EXIT\.\?\s*$",
    "    EXIT.",
    procedure,
)

procedure = re.sub(
    r"(?im)^\s*DISPLAY\s+'DB2 SQL ERROR SQLCODE='\s+SQLCODE\s*$",
    "    DISPLAY 'DB2 SQL ERROR SQLCODE=' SQLCODE",
    procedure,
)

procedure = re.sub(
    r"(?im)^\s*PERFORM\s+SQL-ERROR\.\?\s*$",
    "    PERFORM SQL-ERROR",
    procedure,
)

return before + procedure

def normalize_sql_spacing(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(?im)^\s{12,}COMMIT\s*$",
        "    COMMIT",
        text,
    )

    text = re.sub(
        r"(?im)^\s{12,}VALUES\s*$",
        "    VALUES",
        text,
    )

    return text

def normalize_blank_lines(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"\n{3,}",
        "\n\n",
        text,
    )

    text = re.sub(
        r"\n\s+\n",
        "\n\n",
        text,
    )

    return text

def trim_trailing_spaces(
    self,
    text: str,
) -> str:
    return re.sub(
        r"[ \t]+$",

```

```

        "",
        text,
        flags=re.MULTILINE,
    )

```

```

=====
FILE: src\idms_db2_converter\transformers\dml_transformer.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import CobolAnalysis, SchemaModel

```

```
class DmlTransformer:
```

```
    """
```

```
    Converts IDMS update DML to DB2 embedded SQL.
```

```
    Supported mappings:
```

- STORE record -> INSERT
- MODIFY record -> UPDATE
- ERASE record -> DELETE
- READY AREA ... USAGE-MODE IS UPDATE -> CONTINUE
- FINISH -> COMMIT for update programs

```
    Important behavior:
```

- MODIFY updates only fields moved in the same paragraph before MODIFY.
- If changed fields cannot be detected, MODIFY falls back to non-key fields.
- Primary key resolution prefers ID columns such as EMP_ID_0415.

```
    """
```

```

STORE_RECORD = re.compile(
    r"^\s*STORE\s+([A-Z0-9-]+)\.?\s*$",
    re.IGNORECASE | re.MULTILINE,
)

```

```

MODIFY_RECORD = re.compile(
    r"^\s*MODIFY\s+([A-Z0-9-]+)\.?\s*$",
    re.IGNORECASE | re.MULTILINE,
)

```

```

ERASE_RECORD = re.compile(
    r"^\s*ERASE\s+([A-Z0-9-]+)\.?\s*$",
    re.IGNORECASE | re.MULTILINE,
)

```

```

READY_UPDATE_BLOCK = re.compile(
    r"""
    ^\s*READY\s+AREA\s+([A-Z0-9-]+)\.?\s*
    (?:\n\s*USAGE-MODE\s+IS\s+UPDATE\.?\s*)?
    (?:\n\s*PERFORM\s+IDMS-STATUS\.?\s*)?
    """,
    re.IGNORECASE | re.MULTILINE | re.VERBOSE,
)

```

```

READY_AREA_LINE = re.compile(
    r"^\s*READY\s+AREA\s+([A-Z0-9-]+)\.?\s*$",
    re.IGNORECASE | re.MULTILINE,
)

```

```

USAGE_MODE_UPDATE_LINE = re.compile(
    r"^\s*USAGE-MODE\s+IS\s+UPDATE\.?\s*$",
    re.IGNORECASE | re.MULTILINE,
)

```

```

FINISH_LINE = re.compile(
    r"^\s*FINISH\.?\s*$",
    re.IGNORECASE | re.MULTILINE,
)

```

```

IDMS_STATUS_PERFORM = re.compile(
    r"^\s*PERFORM\s+IDMS-STATUS\.?\s*$",

```

```

        re.IGNORECASE | re.MULTILINE,
    )

    PARAGRAPH_HEADER = re.compile(
        r"^\s*[A-Z0-9-]+\.\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    MOVE_TO_TARGET = re.compile(
        r"^\s*MOVE\s+.+?\s+TO\s+([A-Z0-9_-]+\.\s*$",
        re.IGNORECASE,
    )

    def __init__(
        self,
        schema: SchemaModel,
        analysis: CobolAnalysis,
    ) -> None:
        self.schema = schema
        self.analysis = analysis

    def has_update_dml(
        self,
    ) -> bool:
        return bool(
            getattr(self.analysis, "store_records", [])
            or getattr(self.analysis, "modify_records", [])
            or getattr(self.analysis, "erase_records", [])
            or getattr(self.analysis, "ready_update_areas", [])
        )

    def transform(
        self,
        text: str,
    ) -> str:
        if not self.has_update_dml():
            return text

        text = self.remove_ready_update_area(text)
        text = self.replace_store(text)
        text = self.replace_modify(text)
        text = self.replace_erase(text)
        text = self.replace_finish_with_commit(text)
        text = self.remove_residual_idms_status(text)
        text = self.remove_residual_ready_update_lines(text)
        text = self.fix_continue_concatenation(text)
        text = self.fix_status_paragraph_formatting(text)
        text = self.insert_db2_check_status_paragraph(text)
        text = self.normalize_blank_lines(text)

        return text

    def remove_ready_update_area(
        self,
        text: str,
    ) -> str:
        return self.READY_UPDATE_BLOCK.sub(
            "\n        CONTINUE.\n",
            text,
        )

    def remove_residual_ready_update_lines(
        self,
        text: str,
    ) -> str:
        text = self.READY_AREA_LINE.sub(
            "        CONTINUE.",
            text,
        )

        text = self.USAGE_MODE_UPDATE_LINE.sub(
            "",
            text,
        )

        return text

```

```

def replace_store(
    self,
    text: str,
) -> str:
    def replacement(match: re.Match) -> str:
        record_name = match.group(1).upper()
        return self.insert_sql(record_name)

    return self.STORE_RECORD.sub(
        replacement,
        text,
    )

def replace_modify(
    self,
    text: str,
) -> str:
    matches = list(
        self.MODIFY_RECORD.finditer(text),
    )

    if not matches:
        return text

    pieces: list[str] = []
    last_position = 0

    for match in matches:
        record_name = match.group(1).upper()

        pieces.append(
            text[last_position:match.start()],
        )

        changed_columns = self.changed_columns_before_modify(
            text=text,
            modify_position=match.start(),
            logical_record_name=record_name,
        )

        pieces.append(
            self.update_sql(
                logical_record_name=record_name,
                changed_columns=changed_columns,
            ),
        )

        last_position = match.end()

    pieces.append(
        text[last_position:],
    )

    return "".join(pieces)

def replace_erase(
    self,
    text: str,
) -> str:
    def replacement(match: re.Match) -> str:
        record_name = match.group(1).upper()
        return self.delete_sql(record_name)

    return self.ERASE_RECORD.sub(
        replacement,
        text,
    )

def replace_finish_with_commit(
    self,
    text: str,
) -> str:
    return self.FINISH_LINE.sub(
        self.commit_sql(),
        text,
    )

```

```

    )

def remove_residual_idms_status(
    self,
    text: str,
) -> str:
    return self.IDMS_STATUS_PERFORM.sub(
        "",
        text,
    )

def fix_continue_concatenation(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"CONTINUE\\.\\s*PERFORM\\s+IDMS-STATUS\\.?",
        "CONTINUE.",
        text,
        flags=re.IGNORECASE,
    )

    text = re.sub(
        r"CONTINUE\\.([A-Z0-9-]+\\.)",
        r"CONTINUE.\\n\\1",
        text,
        flags=re.IGNORECASE,
    )

    return text

def fix_status_paragraph_formatting(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(?m)^END-IF\\.",
        "        END-IF.",
        text,
    )

    return text

def insert_sql(
    self,
    logical_record_name: str,
) -> str:
    physical_record_name = self.physical_record_name(
        logical_record_name,
    )

    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        raise ConversionError(
            f"STORE references missing record {logical_record_name}."
        )

    columns = list(
        record.fields.keys(),
    )

    if not columns:
        raise ConversionError(
            f"STORE record {logical_record_name} has no DB2 columns."
        )

    value_items = [
        self.host_with_indicator(
            logical_record_name=logical_record_name,
            column_name=column_name,
            nullable=record.fields[column_name].nullable,
        )
        for column_name in columns
    ]

```

```

]

lines: list[str] = [
    "EXEC SQL",
    f"INSERT INTO {physical_record_name}",
    "(",
]

lines.extend(
    self.sql_list_lines(
        items=columns,
        indent="    ",
    )
)

lines.extend(
    [
        "    )",
        "VALUES",
        "(",
    ]
)

lines.extend(
    self.sql_list_lines(
        items=value_items,
        indent="    ",
    )
)

lines.extend(
    [
        "    )",
        "END-EXEC.",
        "PERFORM DB2-CHECK-STATUS.",
    ]
)

return "\n".join(lines)

def update_sql(
    self,
    logical_record_name: str,
    changed_columns: list[str],
) -> str:
    physical_record_name = self.physical_record_name(
        logical_record_name,
    )

    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        raise ConversionError(
            f"MODIFY references missing record {logical_record_name}."
        )

    primary_key = self.resolve_primary_key(
        logical_record_name=logical_record_name,
        physical_record_name=physical_record_name,
    )

    if not primary_key:
        raise ConversionError(
            f"MODIFY record {logical_record_name} has no primary key."
        )

    update_columns = [
        column_name
        for column_name in changed_columns
        if column_name in record.fields
        and column_name != primary_key
    ]

    if not update_columns:

```



```

        update_columns = [
            column_name
            for column_name in record.fields.keys()
            if column_name != primary_key
        ]

    if not update_columns:
        raise ConversionError(
            f"MODIFY record {logical_record_name} has no update columns."
        )

    key_host = Naming.hv(
        logical_record_name,
        primary_key,
    )

    lines: list[str] = [
        "      EXEC SQL",
        f"      UPDATE {physical_record_name}",
    ]

    for index, column_name in enumerate(update_columns):
        column = record.fields[column_name]

        host = self.host_with_indicator(
            logical_record_name=logical_record_name,
            column_name=column_name,
            nullable=column.nullable,
        )

        suffix = "," if index < len(update_columns) - 1 else ""

        if index == 0:
            lines.append(
                f"      SET {column_name} =",
            )
        else:
            lines.append(
                f"      {column_name} =",
            )

        lines.append(
            f"      {host}{suffix}",
        )

    lines.extend(
        [
            f"      WHERE {primary_key} =",
            f"      :{key_host}",
            "      END-EXEC.",
            "      PERFORM DB2-CHECK-STATUS.",
        ]
    )

    return "\n".join(lines)

def delete_sql(
    self,
    logical_record_name: str,
) -> str:
    physical_record_name = self.physical_record_name(
        logical_record_name,
    )

    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        raise ConversionError(
            f"ERASE references missing record {logical_record_name}."
        )

    primary_key = self.resolve_primary_key(
        logical_record_name=logical_record_name,
        physical_record_name=physical_record_name,
    )

```

```

)

if not primary_key:
    raise ConversionError(
        f"ERASE record {logical_record_name} has no primary key."
    )

key_host = Naming.hv(
    logical_record_name,
    primary_key,
)

return "\n".join(
    [
        "      EXEC SQL",
        f"      DELETE FROM {physical_record_name}",
        f"      WHERE {primary_key} =",
        f"      :{key_host}",
        "      END-EXEC.",
        "      PERFORM DB2-CHECK-STATUS.",
    ]
)

def commit_sql(
    self,
) -> str:
    return "\n".join(
        [
            "      EXEC SQL",
            "      COMMIT",
            "      END-EXEC.",
            "      PERFORM DB2-CHECK-STATUS.",
        ]
    )

def insert_db2_check_status_paragraph(
    self,
    text: str,
) -> str:
    if re.search(
        r"^s*DB2-CHECK-STATUS\.\s*$",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    ):
        return text

    paragraph = "\n".join(
        [
            "",
            "DB2-CHECK-STATUS.",
            "      IF SQLCODE NOT = 0",
            "      DISPLAY 'DB2 SQL ERROR SQLCODE=' SQLCODE",
            "      END-IF.",
            "",
        ]
    )

    if re.search(
        r"\n\s*MAIN-LINE\.",
        text,
        flags=re.IGNORECASE,
    ):
        return re.sub(
            r"(?=\n\s*MAIN-LINE\.)",
            paragraph,
            text,
            count=1,
            flags=re.IGNORECASE,
        )

    return text.rstrip() + paragraph

def changed_columns_before_modify(
    self,
    text: str,
    modify_position: int,

```

```

        logical_record_name: str,
    ) -> list[str]:
        paragraph_text = self.current_paragraph_text(
            text=text,
            position=modify_position,
        )

        physical_record_name = self.physical_record_name(
            logical_record_name,
        )

        record = self.schema.records.get(
            physical_record_name,
        )

        if not record:
            return []

        changed_columns: list[str] = []
        seen: set[str] = set()

        for line in paragraph_text.splitlines():
            match = self.MOVE_TO_TARGET.match(
                line,
            )

            if not match:
                continue

            target = match.group(1).upper()

            column_name = self.column_from_move_target(
                logical_record_name=logical_record_name,
                physical_record_name=physical_record_name,
                target=target,
            )

            if not column_name:
                continue

            if column_name not in record.fields:
                continue

            if column_name in seen:
                continue

            seen.add(
                column_name,
            )

            changed_columns.append(
                column_name,
            )

        return changed_columns

    def current_paragraph_text(
        self,
        text: str,
        position: int,
    ) -> str:
        paragraph_start = 0

        for match in self.PARAGRAPH_HEADER.finditer(
            text,
            0,
            position,
        ):
            paragraph_start = match.start()

        return text[paragraph_start:position]

    def column_from_move_target(
        self,
        logical_record_name: str,
        physical_record_name: str,

```

```

        target: str,
    ) -> str | None:
        record = self.schema.records.get(
            physical_record_name,
        )

        if not record:
            return None

        target = target.upper().rstrip(".")
        target_as_column = self.normalize_column_token(
            target,
        )

        direct = self.find_matching_column(
            record_fields=list(record.fields.keys()),
            candidate=target_as_column,
        )

        if direct:
            return direct

        field_map_result = self.column_from_field_map(
            logical_record_name=logical_record_name,
            target=target,
            record_fields=list(record.fields.keys()),
        )

        if field_map_result:
            return field_map_result

        host_result = self.column_from_host_variable(
            logical_record_name=logical_record_name,
            target=target,
            record_fields=list(record.fields.keys()),
        )

        if host_result:
            return host_result

        return None

def column_from_field_map(
    self,
    logical_record_name: str,
    target: str,
    record_fields: list[str],
) -> str | None:
    field_map = getattr(
        self.schema,
        "field_map",
        {},
    )

    meta = field_map.get(
        target.upper(),
    )

    if not meta:
        return None

    column = meta.get(
        "column",
    )

    if column:
        matched = self.find_matching_column(
            record_fields=record_fields,
            candidate=self.normalize_column_token(str(column)),
        )

        if matched:
            return matched

    host = meta.get(
        "host",
    )

```

```

    )

    if host:
        return self.column_from_host_variable(
            logical_record_name=logical_record_name,
            target=str(host).upper(),
            record_fields=record_fields,
        )

    return None

def column_from_host_variable(
    self,
    logical_record_name: str,
    target: str,
    record_fields: list[str],
) -> str | None:
    normalized_target = target.upper().replace("_", "-")
    normalized_record = logical_record_name.upper().replace("_", "-")
    prefix = f"HV-{normalized_record}-"

    if not normalized_target.startswith(prefix):
        return None

    column_part = normalized_target[len(prefix):]
    candidate = self.normalize_column_token(
        column_part,
    )

    return self.find_matching_column(
        record_fields=record_fields,
        candidate=candidate,
    )

def find_matching_column(
    self,
    record_fields: list[str],
    candidate: str,
) -> str | None:
    candidate = candidate.upper()

    for field_name in record_fields:
        if field_name.upper() == candidate:
            return field_name

    candidate_hyphen = candidate.replace("_", "-")

    for field_name in record_fields:
        if field_name.upper().replace("_", "-") == candidate_hyphen:
            return field_name

    return None

def resolve_primary_key(
    self,
    logical_record_name: str,
    physical_record_name: str,
) -> str | None:
    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        return None

    record_fields = list(
        record.fields.keys(),
    )

    preferred_id = self.preferred_id_column(
        logical_record_name=logical_record_name,
        record_fields=record_fields,
    )

    if preferred_id:
        return preferred_id

```

```

    calc_key = self.primary_key_from_calc_key_map(
        logical_record_name=logical_record_name,
        physical_record_name=physical_record_name,
        record_fields=record_fields,
    )

    if calc_key:
        return calc_key

    if record.primary_key:
        matched_primary_key = self.find_matching_column(
            record_fields=record_fields,
            candidate=self.normalize_column_token(record.primary_key),
        )

        if matched_primary_key:
            return matched_primary_key

    if record_fields:
        return record_fields[0]

    return None

def preferred_id_column(
    self,
    logical_record_name: str,
    record_fields: list[str],
) -> str | None:
    id_columns = [
        field_name
        for field_name in record_fields
        if re.search(
            r"^(_|_)ID(_|$)",
            field_name.upper(),
        )
    ]

    if not id_columns:
        return None

    logical_prefix = logical_record_name.upper().replace("-", "_").split("_")[0]

    for field_name in id_columns:
        first_token = field_name.upper().split("_")[0]

        if first_token.startswith(
            logical_prefix[:3],
        ):
            return field_name

    return id_columns[0]

def primary_key_from_calc_key_map(
    self,
    logical_record_name: str,
    physical_record_name: str,
    record_fields: list[str],
) -> str | None:
    calc_key_map = getattr(
        self.schema,
        "calc_key_map",
        {},
    )

    possible_keys = [
        logical_record_name.upper(),
        logical_record_name.upper().replace("-", "_"),
        physical_record_name.upper(),
        physical_record_name.upper().replace("-", "_"),
    ]

    for key in possible_keys:
        meta = calc_key_map.get(
            key,
        )

```

```

        if not meta:
            continue

        calc_key = (
            meta.get("key")
            or meta.get("primary_key")
            or meta.get("column")
        )

        if not calc_key:
            continue

        matched = self.find_matching_column(
            record_fields=record_fields,
            candidate=self.normalize_column_token(str(calc_key)),
        )

        if matched:
            return matched

    return None

def physical_record_name(
    self,
    logical_record_name: str,
) -> str:
    logical_record_name = logical_record_name.upper()

    mapped = self.schema.record_table_map.get(
        logical_record_name,
    )

    if mapped and mapped in self.schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        "_",
    )

    mapped = self.schema.record_table_map.get(
        normalized,
    )

    if mapped and mapped in self.schema.records:
        return mapped

    if logical_record_name in self.schema.records:
        return logical_record_name

    if normalized in self.schema.records:
        return normalized

    return logical_record_name

def host_with_indicator(
    self,
    logical_record_name: str,
    column_name: str,
    nullable: bool,
) -> str:
    host = f":{Naming.hv(logical_record_name, column_name)}"

    if nullable:
        host += f" :{Naming.ni(logical_record_name, column_name)}"

    return host

def normalize_column_token(
    self,
    value: str,
) -> str:
    return (
        value.upper()
        .strip()
    )

```

```

        .rstrip(".")
        .replace("-", "_")
        .replace(" ", "_")
    )

def sql_list_lines(
    self,
    items: list[str],
    indent: str,
) -> list[str]:
    lines: list[str] = []

    for index, item in enumerate(items):
        suffix = "," if index < len(items) - 1 else ""

        lines.append(
            f"{indent}{item}{suffix}",
        )

    return lines

def normalize_blank_lines(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"\n{3,}",
        "\n\n",
        text,
    )

    return text

```

```

=====
FILE: src\idms_db2_converter\transformers\field_rewriter.py
ENCODING: utf-8
=====

```

```

import json
import re

from datetime import datetime
from typing import Any

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import DB2Model, DB2Table, DB2Column
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer

```

```

class Phase2MetadataGenerator:
    """
    Generates Phase 2 metadata JSON from Phase 1 outputs.

```

```

    Supports:
    - Record/table mapping.
    - Field/column host variable mapping.
    - Date part mapping for:
      YEAR / MONTH / DAY
      YR / MO / DY
      Y / M / D
      YY / MM / DD
      YYYY / MM / DD
      DY / DM / DD
    - CALC key mapping.
    - FK/nullability hints.
    - Set relationship payload.
    """

```

```

VERSION = "1.0"

```

```

DATE_PARTS = {
    "YEAR": {
        "substring_start": 3,
        "substring_length": 2,
    }
}

```



```

    },
    "MONTH": {
        "substring_start": 6,
        "substring_length": 2,
    },
    "DAY": {
        "substring_start": 9,
        "substring_length": 2,
    },
}

YEAR_PARTS = {
    "YEAR",
    "YR",
    "Y",
    "YY",
    "YYYY",
    "DY",
}

MONTH_PARTS = {
    "MONTH",
    "MON",
    "MO",
    "M",
    "MM",
    "DM",
}

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}

def generate_json(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata,
) -> str:
    payload = self.generate(
        canonical_schema=canonical_schema,
        db2_model=db2_model,
        metadata=metadata,
    )

    return json.dumps(
        payload,
        indent=2,
    )

def generate(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata,
) -> dict[str, Any]:
    db2_table_lookup = self.build_db2_table_lookup(
        db2_model=db2_model,
    )

    canonical_record_lookup = self.build_canonical_record_lookup(
        canonical_schema=canonical_schema,
    )

    record_table_map = self.build_record_table_map(
        canonical_schema=canonical_schema,
        metadata=metadata,
    )

    records_payload = self.build_records_payload(
        db2_model=db2_model,
    )

    field_map = self.build_field_map(

```

```

        metadata=metadata,
        canonical_schema=canonical_schema,
        db2_table_lookup=db2_table_lookup,
        record_table_map=record_table_map,
    )

    calc_key_map = self.build_calc_key_map(
        canonical_schema=canonical_schema,
        metadata=metadata,
        record_table_map=record_table_map,
    )

    relationships_payload = self.build_relationships_payload(
        canonical_schema=canonical_schema,
        db2_table_lookup=db2_table_lookup,
    )

    set_ordering_map = self.build_set_ordering_map(
        relationships=relationships_payload,
    )

    nullable_fk_map = self.build_nullable_fk_map(
        db2_model=db2_model,
    )

    date_part_map = self.build_date_part_map(
        metadata=metadata,
        db2_table_lookup=db2_table_lookup,
        record_table_map=record_table_map,
    )

    navigation_intent = self.build_navigation_intent(
        relationships=relationships_payload,
    )

    output_semantics = self.build_output_semantics()

    validation_messages = self.build_validation_messages(
        canonical_schema=canonical_schema,
        db2_model=db2_model,
        metadata=metadata,
        field_map=field_map,
    )

    return {
        "version": self.VERSION,
        "generated_at": datetime.utcnow().isoformat() + "Z",
        "records": records_payload,
        "record_table_map": record_table_map,
        "field_map": field_map,
        "calc_key_map": calc_key_map,
        "relationships": relationships_payload,
        "set_ordering_map": set_ordering_map,
        "nullable_fk_map": nullable_fk_map,
        "date_part_map": date_part_map,
        "navigation_intent": navigation_intent,
        "output_semantics": output_semantics,
        "validation_messages": validation_messages,
        "canonical_record_lookup_count": len(canonical_record_lookup),
    }

def build_db2_table_lookup(
    self,
    db2_model: DB2Model,
) -> dict[str, DB2Table]:
    lookup: dict[str, DB2Table] = {}

    for table in getattr(db2_model, "tables", []) or []:
        table_name = NameNormalizer.normalize(
            getattr(table, "name", ""),
        )

        if table_name:
            lookup[table_name] = table

    suffix_removed = self.remove_record_suffix(

```

```

        table_name,
    )

    if suffix_removed:
        lookup[suffix_removed] = table

    return lookup

def build_canonical_record_lookup(
    self,
    canonical_schema: CanonicalSchema,
) -> dict[str, Any]:
    lookup: dict[str, Any] = {}

    for record in getattr(canonical_schema, "records", []) or []:
        record_name = NameNormalizer.normalize(
            getattr(record, "name", ""),
        )

        if record_name:
            lookup[record_name] = record

    return lookup

def build_record_table_map(
    self,
    canonical_schema: CanonicalSchema,
    metadata: SchemaMetadata,
) -> dict[str, str]:
    record_table_map: dict[str, str] = {}

    for record in getattr(canonical_schema, "records", []) or []:
        canonical_name = NameNormalizer.normalize(
            getattr(record, "name", ""),
        )

        if canonical_name:
            record_table_map[canonical_name] = canonical_name

    for record in getattr(metadata, "records", []) or []:
        metadata_name = NameNormalizer.normalize(
            getattr(record, "name", ""),
        )

        if metadata_name and metadata_name not in record_table_map:
            record_table_map[metadata_name] = metadata_name

    return record_table_map

def build_records_payload(
    self,
    db2_model: DB2Model,
) -> list[dict[str, Any]]:
    records_payload: list[dict[str, Any]] = []

    for table in getattr(db2_model, "tables", []) or []:
        fields_payload: list[dict[str, Any]] = []

        for column in getattr(table, "columns", []) or []:
            datatype, length, scale = self.parse_db2_datatype(
                getattr(column, "datatype", ""),
            )

            fields_payload.append(
                {
                    "name": getattr(column, "name", ""),
                    "column": getattr(column, "name", ""),
                    "datatype": datatype,
                    "length": length,
                    "scale": scale,
                    "nullable": getattr(column, "nullable", True),
                    "primary_key": getattr(column, "primary_key", False),
                }
            )

        records_payload.append(

```

```

        {
            "name": getattr(table, "name", ""),
            "table": getattr(table, "name", ""),
            "primary_key": getattr(table, "primary_key", None),
            "fields": fields_payload,
        }
    )

    return records_payload

def build_field_map(
    self,
    metadata: SchemaMetadata,
    canonical_schema: CanonicalSchema,
    db2_table_lookup: dict[str, DB2Table],
    record_table_map: dict[str, str],
) -> dict[str, dict[str, Any]]:
    field_map: dict[str, dict[str, Any]] = {}

    for record in getattr(metadata, "records", []) or []:
        idms_record_name = NameNormalizer.normalize(
            getattr(record, "name", ""),
        )

        if not idms_record_name:
            continue

        table_name = record_table_map.get(
            idms_record_name,
            idms_record_name,
        )

        table = db2_table_lookup.get(
            table_name,
        )

        if table is None:
            continue

        db2_columns = {
            NameNormalizer.normalize(getattr(column, "name", "")): column
            for column in getattr(table, "columns", []) or []
        }

        source_fields = (
            getattr(record, "mapping_fields", None)
            or getattr(record, "fields", [])
            or []
        )

        for field in source_fields:
            legacy_field_name = NameNormalizer.normalize(
                getattr(field, "name", ""),
            )

            if not legacy_field_name:
                continue

            physical_column_name = self.find_matching_column(
                legacy_field_name=legacy_field_name,
                db2_columns=db2_columns,
            )

            if not physical_column_name:
                continue

            host = self.host_variable(
                record_name=idms_record_name,
                column_name=physical_column_name,
                remove_suffix=True,
            )

            field_map[legacy_field_name] = {
                "host": host,
                "record": idms_record_name,
                "table": table.name,
            }

```

```

        "column": physical_column_name,
        "legacy_field": legacy_field_name,
    }

    suffix_removed_legacy = self.remove_record_suffix(
        legacy_field_name,
    )

    if suffix_removed_legacy and suffix_removed_legacy not in field_map:
        field_map[suffix_removed_legacy] = {
            "host": host,
            "record": idms_record_name,
            "table": table.name,
            "column": physical_column_name,
            "legacy_field": legacy_field_name,
        }

    return field_map

def build_calc_key_map(
    self,
    canonical_schema: CanonicalSchema,
    metadata: SchemaMetadata,
    record_table_map: dict[str, str],
) -> dict[str, dict[str, Any]]:
    calc_key_map: dict[str, dict[str, Any]] = {}

    for record in getattr(metadata, "records", []) or []:
        primary_key = getattr(record, "primary_key", None)

        if not primary_key:
            continue

        idms_record_name = NameNormalizer.normalize(
            getattr(record, "name", ""),
        )

        table_name = record_table_map.get(
            idms_record_name,
            idms_record_name,
        )

        normalized_primary_key = NameNormalizer.normalize(
            primary_key,
        )

        calc_key_map[idms_record_name] = {
            "record": idms_record_name,
            "table": table_name,
            "key": normalized_primary_key,
            "primary_key": normalized_primary_key,
            "column": normalized_primary_key,
            "host": self.host_variable(
                record_name=idms_record_name,
                column_name=normalized_primary_key,
                remove_suffix=True,
            ),
        }

    for record in getattr(canonical_schema, "records", []) or []:
        primary_key = getattr(record, "primary_key", None)

        if not primary_key:
            continue

        record_name = NameNormalizer.normalize(
            getattr(record, "name", ""),
        )

        normalized_primary_key = NameNormalizer.normalize(
            primary_key,
        )

        if record_name not in calc_key_map:
            calc_key_map[record_name] = {
                "record": record_name,

```

```

        "table": record_name,
        "key": normalized_primary_key,
        "primary_key": normalized_primary_key,
        "column": normalized_primary_key,
        "host": self.host_variable(
            record_name=record_name,
            column_name=normalized_primary_key,
            remove_suffix=True,
        ),
    }

    return calc_key_map

def build_relationships_payload(
    self,
    canonical_schema: CanonicalSchema,
    db2_table_lookup: dict[str, DB2Table],
) -> list[dict[str, Any]]:
    relationships_payload: list[dict[str, Any]] = []

    for relationship in getattr(canonical_schema, "relationships", []) or []:
        parent_record = NameNormalizer.normalize(
            getattr(relationship, "parent_record", ""),
        )

        child_record = NameNormalizer.normalize(
            getattr(relationship, "child_record", ""),
        )

        set_name = NameNormalizer.normalize(
            getattr(relationship, "set_name", ""),
        )

        parent_table = db2_table_lookup.get(
            parent_record,
        )

        child_table = db2_table_lookup.get(
            child_record,
        )

        parent_key = (
            getattr(parent_table, "primary_key", None)
            if parent_table is not None
            else None
        )

        child_fk = None
        order_by: list[str] = []

        if child_table is not None:
            for foreign_key in getattr(child_table, "foreign_keys", []) or []:
                if (
                    parent_table is not None
                    and NameNormalizer.normalize(getattr(foreign_key, "reference_table", ""))
                    == NameNormalizer.normalize(getattr(parent_table, "name", ""))
                ):
                    child_fk = getattr(foreign_key, "column_name", None)
                    order_by = [child_fk] if child_fk else []
                    break

        relationships_payload.append(
            {
                "set_name": set_name,
                "parent_record": parent_record,
                "child_record": child_record,
                "parent_key": parent_key,
                "child_fk": child_fk,
                "cardinality": getattr(relationship, "cardinality", "1:N") or "1:N",
                "order_by": order_by,
            }
        )

    return relationships_payload

def build_set_ordering_map(

```

```

self,
relationships: list[dict[str, Any]],
) -> dict[str, dict[str, Any]]:
    set_ordering_map: dict[str, dict[str, Any]] = {}

    for relationship in relationships:
        set_name = relationship.get(
            "set_name",
        )

        if not set_name:
            continue

        set_ordering_map[set_name] = {
            "order_by": relationship.get("order_by") or [],
        }

    return set_ordering_map

def build_nullable_fk_map(
    self,
    db2_model: DB2Model,
) -> dict[str, bool]:
    nullable_fk_map: dict[str, bool] = {}

    for table in getattr(db2_model, "tables", []) or []:
        column_lookup = {
            getattr(column, "name", ""): column
            for column in getattr(table, "columns", []) or []
        }

        for foreign_key in getattr(table, "foreign_keys", []) or []:
            fk_column_name = getattr(foreign_key, "column_name", "")
            fk_column = column_lookup.get(fk_column_name)

            nullable_fk_map[f"{table.name}.{fk_column_name}"] = (
                getattr(fk_column, "nullable", True)
                if fk_column is not None
                else True
            )

    return nullable_fk_map

def build_date_part_map(
    self,
    metadata: SchemaMetadata,
    db2_table_lookup: dict[str, DB2Table],
    record_table_map: dict[str, str],
) -> dict[str, dict[str, Any]]:
    date_part_map: dict[str, dict[str, Any]] = {}

    for record in getattr(metadata, "records", []) or []:
        idms_record_name = NameNormalizer.normalize(
            getattr(record, "name", "")
        )

        if not idms_record_name:
            continue

        table_name = record_table_map.get(
            idms_record_name,
            idms_record_name,
        )

        table = db2_table_lookup.get(
            table_name,
        )

        if table is None:
            continue

        date_columns = {
            NameNormalizer.normalize(getattr(column, "name", "")): column
            for column in getattr(table, "columns", []) or []
            if str(getattr(column, "datatype", "")).upper() == "DATE"
        }

```

```

source_fields = (
    getattr(record, "mapping_fields", None)
    or getattr(record, "fields", [])
    or []
)

for field in source_fields:
    legacy_field_name = NameNormalizer.normalize(
        getattr(field, "name", "")
    )

    parsed = self.parse_date_part(
        field_name=legacy_field_name
    )

    if parsed is None:
        continue

    part = parsed["part"]
    candidate_columns = parsed["candidate_columns"]

    physical_date_column = self.find_first_existing_column(
        candidates=candidate_columns,
        columns=date_columns
    )

    if physical_date_column is None:
        continue

    date_part_meta = self.DATE_PARTS[part]

    date_part_map[legacy_field_name] = {
        "host": self.host_variable(
            record_name=idms_record_name,
            column_name=physical_date_column,
            remove_suffix=True
        ),
        "record": idms_record_name,
        "table": table.name,
        "column": physical_date_column,
        "date_part": part,
        "substring_start": date_part_meta["substring_start"],
        "substring_length": date_part_meta["substring_length"]
    }

return date_part_map

def build_navigation_intent(
    self,
    relationships: list[dict[str, Any]],
) -> dict[str, dict[str, Any]]:
    navigation_intent: dict[str, dict[str, Any]] = {}

    for relationship in relationships:
        set_name = relationship.get(
            "set_name",
        )

        if not set_name:
            continue

        navigation_intent[set_name] = {
            "access_pattern": "SET_NAVIGATION",
            "parent_record": relationship.get("parent_record"),
            "child_record": relationship.get("child_record"),
            "parent_key": relationship.get("parent_key"),
            "child_fk": relationship.get("child_fk"),
            "order_by": relationship.get("order_by") or [],
        }

    return navigation_intent

def build_output_semantics(
    self,
) -> dict[str, Any]:

```



```

    return {
        "generated_by": "idms-db2-modernizer-phase1",
        "usage": "IDMS retrieval COBOL to DB2 embedded SQL conversion",
        "physical_types_source": "DB2_DDL",
        "logical_mapping_source": "PHASE1_CANONICAL_METADATA",
    }

def build_validation_messages(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata,
    field_map: dict[str, dict[str, Any]],
) -> list[str]:
    messages: list[str] = []

    if not getattr(canonical_schema, "records", []):
        messages.append("No canonical records found.")

    if not getattr(db2_model, "tables", []):
        messages.append("No DB2 tables found.")

    if not getattr(metadata, "records", []):
        messages.append("No IDMS metadata records found.")

    if not field_map:
        messages.append("No field map entries generated.")

    for table in getattr(db2_model, "tables", []) or []:
        primary_key = getattr(table, "primary_key", None)

        if not primary_key:
            continue

        column_names = {
            NameNormalizer.normalize(getattr(column, "name", ""))
            for column in getattr(table, "columns", []) or []
        }

        if NameNormalizer.normalize(primary_key) not in column_names:
            messages.append(
                f"Table {table.name} primary key {primary_key} is not present as a DB2 column."
            )

    return messages

def find_matching_column(
    self,
    legacy_field_name: str,
    db2_columns: dict[str, DB2Column],
) -> str | None:
    normalized = NameNormalizer.normalize(
        legacy_field_name,
    )

    if normalized in db2_columns:
        return normalized

    suffix_removed = self.remove_record_suffix(
        normalized,
    )

    for column_name in db2_columns:
        if self.remove_record_suffix(column_name) == suffix_removed:
            return column_name

    return None

def parse_date_part(
    self,
    field_name: str,
) -> dict[str, Any] | None:
    tokens = self.split_tokens(
        value=field_name,
    )

```

```

if len(tokens) < 2:
    return None

for index, token in enumerate(tokens):
    part = self.date_part_type(
        token=token,
        tokens=tokens,
    )

    if part is None:
        continue

    base_tokens = tokens[:index] + tokens[index + 1 :]
    date_tokens = tokens.copy()
    date_tokens[index] = "DATE"

    base_name = " ".join(base_tokens)
    date_name = " ".join(date_tokens)

    return {
        "part": part,
        "candidate_columns": self.unique_values(
            [
                date_name,
                base_name,
            ]
        ),
    }

return None

def date_part_type(
    self,
    token: str,
    tokens: list[str],
) -> str | None:
    token = token.upper()

    has_dy_dm_dd = (
        "DY" in tokens
        and "DM" in tokens
        and "DD" in tokens
    )

    if token in {"YEAR", "YR", "Y", "YY", "YYYY"}:
        return "YEAR"

    if token in {"MONTH", "MON", "MO", "M", "MM", "DM"}:
        return "MONTH"

    if token in {"DAY", "D", "DD"}:
        return "DAY"

    if token == "DY":
        if has_dy_dm_dd:
            return "YEAR"

        return "DAY"

    return None

def find_first_existing_column(
    self,
    candidates: list[str],
    columns: dict[str, DB2Column],
) -> str | None:
    for candidate in candidates:
        normalized_candidate = NameNormalizer.normalize(
            candidate,
        )

        if normalized_candidate in columns:
            return normalized_candidate

    suffix_removed_candidate = self.remove_record_suffix(
        normalized_candidate,
    )

```

```

    )

    for column_name in columns:
        if self.remove_record_suffix(column_name) == suffix_removed_candidate:
            return column_name

    return None

def host_variable(
    self,
    record_name: str,
    column_name: str,
    remove_suffix: bool = True,
) -> str:
    record = NameNormalizer.normalize(
        record_name,
    ).replace(
        " ",
        "-",
    )

    column = NameNormalizer.normalize(
        column_name,
    )

    if remove_suffix:
        column = self.remove_record_suffix(
            column,
        )

    column = column.replace(
        " ",
        "-",
    )

    return f"HV-{{record}}-{{column}}"

def parse_db2_datatype(
    self,
    datatype: str,
) -> tuple[str, int | None, int | None]:
    value = (
        datatype
        or "VARCHAR(255)"
    ).strip().upper()

    decimal_match = re.match(
        r"^(DECIMAL|NUMERIC)$($(\d+),\s*(\d+)$)",
        value,
    )

    if decimal_match:
        return (
            "DECIMAL",
            int(decimal_match.group(2)),
            int(decimal_match.group(3)),
        )

    varchar_match = re.match(
        r"^(VARCHAR|CHAR)$($(\d+)$)",
        value,
    )

    if varchar_match:
        return (
            varchar_match.group(1),
            int(varchar_match.group(2)),
            None,
        )

    if value in {"INTEGER", "INT"}:
        return "INTEGER", 9, 0

    if value == "BIGINT":
        return "BIGINT", 18, 0

```

```

    if value == "SMALLINT":
        return "INTEGER", 9, 0

    if value == "DATE":
        return "DATE", None, None

    if value == "TIMESTAMP":
        return "TIMESTAMP", None, None

    return value, None, None

def remove_record_suffix(
    self,
    field_name: str,
) -> str:
    normalized = NameNormalizer.normalize(
        field_name,
    )

    parts = normalized.split()

    if (
        len(parts) > 1
        and parts[-1].isdigit()
        and len(parts[-1]) == 4
    ):
        return " ".join(
            parts[:-1],
        )

    return normalized

def split_tokens(
    self,
    value: str,
) -> list[str]:
    normalized = NameNormalizer.normalize(
        value,
    )

    return [
        token.upper()
        for token in re.split(
            r"[\s_-]+",
            normalized,
        )
        if token
    ]

def unique_values(
    self,
    values: list[str],
) -> list[str]:
    result: list[str] = []
    seen: set[str] = set()

    for value in values:
        normalized = NameNormalizer.normalize(
            value,
        )

        if not normalized:
            continue

        if normalized in seen:
            continue

        seen.add(
            normalized,
        )

        result.append(
            normalized,
        )

    return result

```

```
=====
FILE: src\idms_db2_converter\transformers\metadata_helpers.py
ENCODING: utf-8
=====
```

```
from idms_db2_converter.generators.cobol_builder import Move
from idms_db2_converter.models import SchemaModel
```

```
class MetadataHelpers:
    def __init__(self, schema: SchemaModel):
        self.schema = schema

    def lookup_by_name(self, name: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("name", "").upper() == name.upper():
                return item

        return None

    def lookup_by_owner_set(self, owner_set: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("owner_set", "").upper() == owner_set.upper():
                return item

        return None

    def lookup_by_first_member_set(self, first_member_set: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("first_member_set", "").upper() == first_member_set.upper():
                return item

        return None

    def moves_to_nodes(self, moves: list[dict]) -> list[Move]:
        nodes = []

        for move in moves:
            source = self.map_source(move["from"])
            target = move["to"]
            nodes.append(Move(source=source, target=target))

        return nodes

    def map_source(self, source: str) -> str:
        source_upper = source.upper()

        if source_upper in self.schema.date_part_map:
            meta = self.schema.date_part_map[source_upper]
            return f"{meta['host']}({meta['substring_start']}:{meta['substring_length']})"

        if source_upper in self.schema.field_map:
            return self.schema.field_map[source_upper].get("host", source)

        return source
```

```
=====
FILE: src\idms_db2_converter\transformers\operation_graph_transformer.py
ENCODING: utf-8
=====
```

```
import re

from idms_db2_converter.generators.cobol_builder import (
    Continue,
    ExecSql,
    IfBlock,
    Move,
    Perform,
    PerformUntil,
    PreRenderedBlock,
    ReadAtEndMove,
    Write,
```

```

        render_nodes,
        render_paragraph,
    )
from idms_db2_converter.generators.fetch_paragraphs import FetchParagraphGenerator
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.generators.sql_snippets import SqlSnippets
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.transformers.metadata_helpers import MetadataHelpers

class OperationGraphTransformer:
    def __init__(self, schema: SchemaModel):
        self.schema = schema
        self.sql = SqlSnippets(schema)
        self.meta = MetadataHelpers(schema)
        self.fetch_generator = FetchParagraphGenerator(schema)
        self.used_cursor_sets: set[str] = set()

    def has_graph(self) -> bool:
        return bool(self.schema.paragraph_operation_graph)

    def transform(self, text: str) -> str:
        for paragraph_name, operations in self.schema.paragraph_operation_graph.items():
            rendered = self._render_paragraph(paragraph_name, operations)
            text = self._replace_paragraph(text, paragraph_name, rendered)

        return text

    def _replace_paragraph(self, text: str, paragraph_name: str, replacement: str) -> str:
        pattern = re.compile(
            rf"^\s*{re.escape(paragraph_name)}\.\s*\n.*?(?=\s*[A-Z0-9-]+\.\s*$)",
            re.IGNORECASE | re.MULTILINE | re.DOTALL,
        )

        if pattern.search(text):
            return pattern.sub(replacement + "\n", text, count=1)

        return text + "\n" + replacement + "\n"

    def _render_paragraph(self, paragraph_name: str, operations: list[dict]) -> str:
        nodes = self._render_operations(operations)
        return render_paragraph(paragraph_name, nodes)

    def _render_operations(self, operations: list[dict]) -> list:
        nodes = []

        for operation in operations:
            op_type = operation.get("type", "").upper()

            if op_type == "MOVE":
                nodes.append(
                    Move(
                        source=self.meta.map_source(operation["from"]),
                        target=operation["to"],
                    )
                )

            elif op_type == "MOVE_DATE_PART":
                nodes.append(
                    Move(
                        source=self.meta.map_source(operation["from"]),
                        target=operation["to"],
                    )
                )

            elif op_type == "PERFORM":
                nodes.append(Perform(paragraph=operation["paragraph"]))

            elif op_type == "WRITE":
                nodes.append(
                    Write(
                        record=operation["record"],
                        source=operation.get("source"),
                    )
                )

```

```

        elif op_type == "READ_NEXT_INPUT":
            at_end_move = operation.get("at_end_move", {})
            nodes.append(
                ReadAtEndMove(
                    file_name=operation["file"],
                    source=at_end_move.get("from", "Y"),
                    target=at_end_move.get("to", "EOF-SW"),
                )
            )

        elif op_type == "CONTINUE":
            nodes.append(Continue())

        elif op_type == "OBTAIN_CALC":
            nodes.append(self._node_obtain_calc(operation))

        elif op_type == "CURSOR_LOOP":
            nodes.append(self._node_cursor_loop(operation))

        elif op_type == "FIRST_MEMBER_TO_OWNER_LOOKUP":
            nodes.extend(self._nodes_first_member_to_owner_lookup(operation))

        elif op_type == "OWNER_LOOKUP":
            nodes.extend(self._nodes_owner_lookup(operation))

    return nodes

def _node_obtain_calc(self, operation: dict):
    record = operation["record"].upper()
    select_sql = self.sql.select_for_record_by_pk(record)

    not_found_nodes = self._render_operations(operation.get("not_found", []))
    success_nodes = self._render_operations(operation.get("success", []))

    lines = []
    lines.extend(ExecSql(select_sql).render(indent=7))
    lines.append("")
    lines.extend(
        IfBlock(
            condition="SQLCODE = 100",
            then_body=not_found_nodes,
            else_body=[
                IfBlock(
                    condition="SQLCODE NOT = 0",
                    then_body=[Perform("SQL-ERROR")],
                    else_body=success_nodes,
                )
            ],
        ).render(indent=7, final=True)
    )

    return PreRenderedBlock(lines)

def _node_cursor_loop(self, operation: dict):
    set_name = operation["set"].upper()
    self.used_cursor_sets.add(set_name)

    cursor_name = Naming.cursor(set_name)
    fetch_paragraph = self.fetch_generator.paragraph_name(set_name)

    process_paragraph = operation["process_paragraph"]
    exit_paragraph = operation.get("exit_paragraph")
    empty_nodes = self._render_operations(operation.get("empty", []))
    before_loop_nodes = self._render_operations(operation.get("before_loop", []))

    perform_target = process_paragraph
    if exit_paragraph:
        perform_target = f"{{process_paragraph}} THRU {{exit_paragraph}}"

    cursor_open = ExecSql(
        "\n".join(
            [
                "EXEC SQL",
                f"      OPEN {cursor_name}",
                "END-EXEC.",
            ]
        )
    )

```

```

    )
)

cursor_close = ExecSql(
    "\n".join(
        [
            "EXEC SQL",
            f"      CLOSE {cursor_name}",
            "END-EXEC.",
        ]
    )
)

loop_nodes = before_loop_nodes + [
    PerformUntil(
        condition="SQLCODE = 100",
        body=[
            Perform(perform_target),
            Perform(fetch_paragraph),
        ],
    )
]

return IfBlock(
    condition="SQLCODE NOT = 0",
    then_body=[Perform("SQL-ERROR")],
    else_body=[
        Perform(fetch_paragraph),
        IfBlock(
            condition="SQLCODE = 100",
            then_body=empty_nodes,
            else_body=loop_nodes,
        ),
        cursor_close,
    ],
).with_prefix(cursor_open)

def _nodes_first_member_to_owner_lookup(self, operation: dict) -> list:
    lookup_name = operation.get("name")
    first_member_set = operation["first_member_set"].upper()
    owner_set = operation["owner_set"].upper()

    lookup = self.meta.lookup_by_name(lookup_name) if lookup_name else None
    if not lookup:
        lookup = self.meta.lookup_by_first_member_set(first_member_set)

    not_found_moves = lookup.get("not_found_moves", []) if lookup else []
    success_moves = lookup.get("success_moves", []) if lookup else []

    first_select = ExecSql(self.sql.select_first_child_for_set(first_member_set))
    owner_select = ExecSql(self.sql.select_for_owner(owner_set))

    not_found_nodes = self.meta.moves_to_nodes(not_found_moves)
    owner_not_found_nodes = self.meta.moves_to_nodes(not_found_moves)
    success_nodes = self.meta.moves_to_nodes(success_moves) or [Continue()]

    return [
        first_select,
        IfBlock(
            condition="SQLCODE = 100",
            then_body=not_found_nodes,
            else_body=[
                IfBlock(
                    condition="SQLCODE NOT = 0",
                    then_body=[Perform("SQL-ERROR")],
                    else_body=[
                        owner_select,
                        IfBlock(
                            condition="SQLCODE = 100",
                            then_body=owner_not_found_nodes,
                            else_body=[
                                IfBlock(
                                    condition="SQLCODE NOT = 0",
                                    then_body=[Perform("SQL-ERROR")],
                                    else_body=success_nodes,
                                )
                            ]
                        )
                    ]
                )
            ]
        )
    ]

```



```

        ],
    ),
],
),
],
]

def _nodes_owner_lookup(self, operation: dict) -> list:
    lookup_name = operation.get("name")
    owner_set = operation["owner_set"].upper()

    lookup = self.meta.lookup_by_name(lookup_name) if lookup_name else None
    if not lookup:
        lookup = self.meta.lookup_by_owner_set(owner_set)

    not_found_moves = lookup.get("not_found_moves", []) if lookup else []
    success_moves = lookup.get("success_moves", []) if lookup else []

    not_found_nodes = self.meta.moves_to_nodes(not_found_moves)
    success_nodes = self.meta.moves_to_nodes(success_moves) or [Continue()]
    owner_select = ExecSql(self.sql.select_for_owner(owner_set))

    nullable_meta = self.schema.nullable_fk_map.get(owner_set)
    null_indicator = nullable_meta.get("null_indicator") if nullable_meta else None

    if null_indicator:
        return [
            IfBlock(
                condition=f"{null_indicator} < 0",
                then_body=not_found_nodes,
                else_body=[
                    owner_select,
                    IfBlock(
                        condition="SQLCODE = 0",
                        then_body=success_nodes,
                        else_body=[
                            IfBlock(
                                condition="SQLCODE = 100",
                                then_body=not_found_nodes,
                                else_body=[Perform("SQL-ERROR")],
                            ),
                        ],
                    ),
                ],
            ),
        ],
    ],
)

return [
    owner_select,
    IfBlock(
        condition="SQLCODE = 0",
        then_body=success_nodes,
        else_body=[
            IfBlock(
                condition="SQLCODE = 100",
                then_body=not_found_nodes,
                else_body=[Perform("SQL-ERROR")],
            ),
        ],
    ),
],
)
]

```

```

=====
FILE: src\idms_db2_converter\transformers\paragraph_rewriter.py
ENCODING: utf-8
=====

```

```

import re

from idms_db2_converter.generators.fetch_paragraphs import FetchParagraphGenerator
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.generators.sql_snippets import SqlSnippets
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.transformers.structured_blocks import StructuredBlocks

```

```

class ParagraphRewriter:
    def __init__(self, schema: SchemaModel):
        self.schema = schema
        self.sql = SqlSnippets(schema)
        self.fetch_generator = FetchParagraphGenerator(schema)
        self.controlled_loop_sets: set[str] = set()

    def rewrite(self, text: str) -> str:
        text = self._rewrite_cursor_loop_blocks(text)
        text = self._rewrite_find_first_owner_chain_blocks(text)
        text = self._rewrite_empty_else_obtain_owner_blocks(text)
        text = self._rewrite_empty_else_find_first_blocks(text)
        text = self._rewrite_obtain_calc_if_blocks(text)
        text = self._rewrite_remaining_obtain_calc(text)
        text = self._rewrite_remaining_obtain_next(text)
        text = self._rewrite_remaining_find_first(text)
        text = self._rewrite_remaining_obtain_owner(text)
        text = self._rewrite_status_tokens(text)
        return text

    def _rewrite_cursor_loop_blocks(self, text: str) -> str:
        for set_name in self.schema.relationships:
            text = self._rewrite_cursor_loop_for_set(text, set_name)

        return text

    def _rewrite_cursor_loop_for_set(self, text: str, set_name: str) -> str:
        pattern = re.compile(
            rf"""
            IF\s+{re.escape(set_name)}\s+IS\s+NOT\s+EMPTY\s+THEN
            (?P<before_loop>.*?)
            PERFORM\s+(?P<walk>[A-Z0-9-]+\s+THRU\s+(?P<exit>[A-Z0-9-]+\s+
            \s+UNTIL\s+DB-END-OF-SET
            \s+ELSE
            (?P<empty_body>.*?)
            (?=\.\s*READ|\.\s*[A-Z0-9-]+\.)
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

    def replace(match: re.Match) -> str:
        self.controlled_loop_sets.add(set_name.upper())

        return StructuredBlocks.cursor_loop_block(
            cursor_name=Naming.cursor(set_name),
            fetch_paragraph=self.fetch_generator.paragraph_name(set_name),
            empty_body=self._clean_block(match.group("empty_body")),
            before_loop_body=self._clean_block(match.group("before_loop")),
            walk_paragraph=match.group("walk").upper(),
            walk_exit=match.group("exit").upper(),
            indent=7,
        )

    return pattern.sub(replace, text)

    def _rewrite_find_first_owner_chain_blocks(self, text: str) -> str:
        for first_set in self.schema.relationships:
            for owner_set in self.schema.relationships:
                text = self._rewrite_one_find_first_owner_chain(
                    text=text,
                    first_set=first_set,
                    owner_set=owner_set,
                )

        return text

    def _rewrite_one_find_first_owner_chain(
        self,
        text: str,
        first_set: str,
        owner_set: str,
    ) -> str:
        pattern = re.compile(
            rf"""

```

```

IF\s+{re.escape(first_set)}\s+IS\s+EMPTY
(?P<first_empty>.*?)
ELSE\s+
FIND\s+FIRST\s+WITHIN\s+{re.escape(first_set)}\s+\.?
(?P<after_find>.*?)
IF\s+NOT\s+{re.escape(owner_set)}\s+MEMBER
(?P<owner_missing>.*?)
ELSE\s+
OBTAIN\s+OWNER\s+WITHIN\s+{re.escape(owner_set)}\s+\.?
(?P<success>.*?)
(?:=
    \n\s*IF\s+[A-Z0-9-]+\s+IS\s+EMPTY
    |
    \n\s*MOVE\s+EMP-DETAIL-LINE
    |
    \n\s*PERFORM\s+
    |
    \n\s*[A-Z0-9-]+-EXIT\
)
""",
re.IGNORECASE | re.DOTALL | re.VERBOSE,
)

def replace(match: re.Match) -> str:
    first_empty = self._remove_idms_status_lines(
        self._clean_block(match.group("first_empty"))
    )
    owner_missing = self._remove_idms_status_lines(
        self._clean_block(match.group("owner_missing"))
    )
    success = self._remove_idms_status_lines(
        self._clean_block(match.group("success"))
    )

    if not success:
        success = "CONTINUE"

    return StructuredBlocks.select_first_then_owner_chain(
        first_select_sql=self.sql.select_first_child_for_set(first_set),
        owner_select_sql=self.sql.select_for_owner(owner_set),
        first_not_found_body=first_empty,
        owner_not_found_body=owner_missing,
        success_body=success,
        indent=7,
    )

return pattern.sub(replace, text)

def _rewrite_empty_else_obtain_owner_blocks(self, text: str) -> str:
    for set_name in self.schema.relationships:
        pattern = re.compile(
            rf"""
            IF\s+{re.escape(set_name)}\s+IS\s+EMPTY
            (?P<empty_body>.*?)
            ELSE\s+
            OBTAIN\s+OWNER\s+WITHIN\s+{re.escape(set_name)}\s+\.?
            (?P<success_body>.*?)
            (?:=
                \n\s*MOVE\s+EMP-DETAIL-LINE
                |
                \n\s*PERFORM\s+
                |
                \n\s*[A-Z0-9-]+-EXIT\
            )
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

    def replace(match: re.Match, set_name=set_name) -> str:
        empty_body = self._remove_idms_status_lines(
            self._clean_block(match.group("empty_body"))
        )
        success_body = self._remove_idms_status_lines(
            self._clean_block(match.group("success_body"))
        )

```

```

        if not success_body:
            success_body = "CONTINUE"

        indicator = self.sql.nullable_fk_indicator_for_set(set_name)

        if indicator:
            return StructuredBlocks.nullable_owner_block(
                null_indicator=indicator,
                null_body=empty_body,
                owner_select_sql=self.sql.select_for_owner(set_name),
                success_body=success_body,
                fallback_body=empty_body,
                indent=7,
            )

        return StructuredBlocks.owner_select_block(
            owner_select_sql=self.sql.select_for_owner(set_name),
            success_body=success_body,
            fallback_body=empty_body,
            indent=7,
        )

    text = pattern.sub(replace, text)

    return text

def _rewrite_empty_else_find_first_blocks(self, text: str) -> str:
    for set_name in self.schema.relationships:
        pattern = re.compile(
            rf"""
            IF\s+{re.escape(set_name)}\s+IS\s+EMPTY
            (?P<empty_body>.*?)
            ELSE\s+
            FIND\s+FIRST\s+WITHIN\s+{re.escape(set_name)}\s+\.?
            (?P<success_body>.*?)
            (?=
                \n\s*IF\s+[A-Z0-9-]+\s+IS\s+EMPTY
                |
                \n\s*MOVE\s+EMP-DETAIL-LINE
                |
                \n\s*PERFORM\s+
                |
                \n\s*[A-Z0-9-]+\s+-EXIT\
            )
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

    def replace(match: re.Match, set_name=set_name) -> str:
        empty_body = self._remove_idms_status_lines(
            self._clean_block(match.group("empty_body"))
        )
        success_body = self._remove_idms_status_lines(
            self._clean_block(match.group("success_body"))
        )

        if not success_body:
            success_body = "CONTINUE"

        return StructuredBlocks.sqlcode_select_block(
            sql=self.sql.select_first_child_for_set(set_name),
            not_found_body=empty_body,
            success_body=success_body,
            indent=7,
        )

    text = pattern.sub(replace, text)

    return text

def _rewrite_obtain_calc_if_blocks(self, text: str) -> str:
    pattern = re.compile(
        rf"""
        OBTAIN\s+CALC\s+(?P<record>[A-Z0-9-]+\s+)\.?
        \s+
        IF\s+DB-REC-NOT-FOUND\s+THEN
        """
    )

```

```

        (?P<not_found>.*?)
    ELSE
        (?P<success>.*?)
    (?=\n\s*READ\s+)
    """,
    re.IGNORECASE | re.DOTALL | re.VERBOSE,
)

def replace(match: re.Match) -> str:
    record = match.group("record").upper()
    not_found = self._remove_idms_status_lines(
        self._clean_block(match.group("not_found"))
    )
    success = self._remove_idms_status_lines(
        self._clean_block(match.group("success"))
    )

    return StructuredBlocks.sqlcode_select_block(
        sql=self.sql.select_for_record_by_pk(record),
        not_found_body=not_found,
        success_body=success,
        indent=7,
    )

return pattern.sub(replace, text)

def _rewrite_remaining_obtain_calc(self, text: str) -> str:
    pattern = re.compile(
        r"^\s*OBTAIN\s+CALC\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    return pattern.sub(
        lambda m: self.sql.select_for_record_by_pk(m.group(1).upper()),
        text,
    )

def _rewrite_remaining_obtain_next(self, text: str) -> str:
    pattern = re.compile(
        r"^\s*OBTAIN\s+NEXT\s+([A-Z0-9-]+)\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    def replace(match: re.Match) -> str:
        set_name = match.group(2).upper()

        if set_name in self.controlled_loop_sets:
            return "CONTINUE."

        return f"PERFORM {self.fetch_generator.paragraph_name(set_name)}"

    return pattern.sub(replace, text)

def _rewrite_remaining_find_first(self, text: str) -> str:
    pattern = re.compile(
        r"^\s*FIND\s+FIRST\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    return pattern.sub(
        lambda m: self.sql.select_first_child_for_set(m.group(1).upper()),
        text,
    )

def _rewrite_remaining_obtain_owner(self, text: str) -> str:
    pattern = re.compile(
        r"^\s*OBTAIN\s+OWNER\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    return pattern.sub(
        lambda m: self.sql.select_for_owner(m.group(1).upper()),
        text,
    )

def _rewrite_status_tokens(self, text: str) -> str:

```

```

        text = re.sub(
            r"\bDB-REC-NOT-FOUND\b",
            "SQLCODE = 100",
            text,
            flags=re.IGNORECASE,
        )

        text = re.sub(
            r"\bDB-END-OF-SET\b",
            "SQLCODE = 100",
            text,
            flags=re.IGNORECASE,
        )

        text = re.sub(
            r"IF\s+NOT\s+[A-Z0-9-]+\s+MEMBER",
            "IF SQLCODE = 100",
            text,
            flags=re.IGNORECASE,
        )

    return text

def _remove_idms_status_lines(self, text: str) -> str:
    text = re.sub(
        r"^\s*PERFORM\s+IDMS-STATUS\.\?\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

    text = re.sub(
        r"^\s*IF SQLCODE NOT = 0 AND SQLCODE NOT = 100\s*"
        r"\n\s*PERFORM SQL-ERROR\s*"
        r"\n\s*END-IF\.\?\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

    return text.strip()

def _clean_block(self, value: str) -> str:
    value = value.strip()

    if value.endswith("."):
        value = value[:-1].rstrip()

    return value

```

```

=====
FILE: src\idms_db2_converter\transformers\retrieval_transformer.py
ENCODING: utf-8
=====

```

```

import re

from idms_db2_converter.generators.cursors import CursorGenerator
from idms_db2_converter.generators.fetch_paragraphs import FetchParagraphGenerator
from idms_db2_converter.generators.host_variables import HostVariableGenerator
from idms_db2_converter.models import CobolAnalysis, SchemaModel
from idms_db2_converter.transformers.cobol_cleanup import CobolCleanup
from idms_db2_converter.transformers.dml_transformer import DmlTransformer
from idms_db2_converter.transformers.field_rewriter import FieldRewriter
from idms_db2_converter.transformers.operation_graph_transformer import (
    OperationGraphTransformer,
)
from idms_db2_converter.transformers.paragraph_rewriter import ParagraphRewriter

class RetrievalTransformer:
    """
    Transforms IDMS retrieval/update COBOL into DB2 embedded SQL COBOL.

    Retrieval behavior:
    """

```

- OBTAIN CALC becomes SELECT.
- OBTAIN NEXT set navigation becomes cursor processing.
- FIND FIRST lookup sets do not become declared cursors.
- OBTAIN OWNER becomes keyed SELECT.
- Existing generated DB2 blocks are removed before regeneration.
- Existing incomplete FETCH paragraphs are removed and regenerated.
- INIT-BIND-READY is renamed to INIT-DB2-SETUP.
- PERFORM IDMS-STATUS lines are removed.
- FINISH executable lines are removed unless already converted by DML logic.

Update behavior:

- STORE, MODIFY, ERASE are delegated to DmlTransformer.
- DmlTransformer is called only when update DML exists.

Host-variable optimization:

- Does not add every schema relationship record.
- Adds only records actually used by COBOL DML, set navigation, or field references.

"""

```

PROCEDURE_DIVISION = re.compile(
    r"^\s*PROCEDURE\s+DIVISION\.",
    re.IGNORECASE | re.MULTILINE,
)

def __init__(
    self,
    schema: SchemaModel,
    analysis: CobolAnalysis,
) -> None:
    self.schema = schema
    self.analysis = analysis

def transform(
    self,
    cobol: str,
    target_program: str | None = None,
) -> str:
    text = cobol

    if target_program and self.analysis.program_id:
        text = self._rename_program(
            text=text,
            source=self.analysis.program_id,
            target=target_program,
        )

    text = self._remove_existing_generated_db2_block(text)
    text = self._remove_idms_control_section(text)
    text = self._remove_schema_section(text)
    text = self._remove_idms_copybooks(text)

    operation_graph_transformer = OperationGraphTransformer(
        self.schema,
    )

    if operation_graph_transformer.has_graph():
        text = operation_graph_transformer.transform(text)

        used_cursor_sets = sorted(
            operation_graph_transformer.used_cursor_sets,
        )
    else:
        paragraph_rewriter = ParagraphRewriter(
            self.schema,
        )

        text = paragraph_rewriter.rewrite(text)

        used_cursor_sets = sorted(
            paragraph_rewriter.controlled_loop_sets,
        )

    used_cursor_sets = self._ensure_cursor_sets(
        used_cursor_sets,

```

```

    )

    dml_transformer = DmlTransformer(
        schema=self.schema,
        analysis=self.analysis,
    )

    if dml_transformer.has_update_dml():
        text = dml_transformer.transform(text)

    text = self._remove_existing_fetch_paragraphs(
        text=text,
        used_cursor_sets=used_cursor_sets,
    )

    text = self._insert_sql_declarations(
        text=text,
        used_cursor_sets=used_cursor_sets,
    )

    text = self._insert_fetch_paragraphs(
        text=text,
        used_cursor_sets=used_cursor_sets,
    )

    text = FieldRewriter(
        self.schema,
    ).rewrite(text)

    text = self._replace_init_bind_ready(text)
    text = self._remove_idms_status_perform(text)
    text = self._replace_finish(text)
    text = self._remove_bind_ready_lines(text)
    text = self._clean_end_processing(text)
    text = self._remove_idms_abort_paragraphs(text)
    text = self._remove_obsolete_set_walk_guard(text)
    text = self._cleanup_orphan_sqlcode_blocks(text)
    text = self._fix_paragraph_header_concatenation(text)
    text = self._normalize_formatting(text)

    text = CobolCleanup().clean(text)

    text = self._add_sql_error_paragraph(text)

    text = CobolCleanup().clean(text)

    return text

def _rename_program(
    self,
    text: str,
    source: str,
    target: str,
) -> str:
    return re.sub(
        rf"(PROGRAM-ID\.\\s*){re.escape(source)}(\\.)?",
        rf"\\1{target}.",
        text,
        flags=re.IGNORECASE,
    )

def _remove_existing_generated_db2_block(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"""
        \\n\\s*
        (?:EJECT\\s*)?
        \\*+\\s*
        \\n\\s*\\*\\s*DB2\\s+SQLCA, \\s+HOST\\s+VARIABLES, \\s+AND\\s+CURSORS\\s*
        .*?
        (?:=\\n\\s*PROCEDURE\\s+DIVISION\\. )
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE,
    )

```



```

        return pattern.sub(
            "\n",
            text,
        )

def _remove_idms_control_section(
    self,
    text: str,
) -> str:
    return re.sub(
        r"\n\s*IDMS-CONTROL\s+SECTION\..*?(?=\n\s*DATA\s+DIVISION\.)",
        "\n",
        text,
        flags=re.IGNORECASE | re.DOTALL,
    )

def _remove_schema_section(
    self,
    text: str,
) -> str:
    return re.sub(
        r"\n\s*SCHEMA\s+SECTION\..*?(?=\n\s*FILE\s+SECTION\.)",
        "\n",
        text,
        flags=re.IGNORECASE | re.DOTALL,
    )

def _remove_idms_copybooks(
    self,
    text: str,
) -> str:
    return re.sub(
        r"^\s*(?:01\s+)?COPY\s+IDMS\s+.*?\.\.?s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

def _ensure_cursor_sets(
    self,
    used_cursor_sets: list[str],
) -> list[str]:
    result = set(
        item.upper()
        for item in used_cursor_sets
        if item
    )

    for _, set_name in getattr(self.analysis, "obtain_next", []):
        if set_name:
            result.add(
                set_name.upper(),
            )

    return sorted(result)

def _insert_sql_declarations(
    self,
    text: str,
    used_cursor_sets: list[str],
) -> str:
    used_records = self._used_records(
        used_cursor_sets=used_cursor_sets,
    )

    host_variables = HostVariableGenerator().generate(
        self.schema,
        used_records,
    )

    cursors = CursorGenerator().generate(
        self.schema,
        used_cursor_sets,
    )

```

```

block = "\n".join(
    [
        "",
        "*****",
        " * DB2 SQLCA, HOST VARIABLES, AND CURSORS",
        "*****",
        host_variables,
        cursors,
        "",
    ]
)

return re.sub(
    r"(?=\n\s*PROCEDURE\s+DIVISION\.)",
    block,
    text,
    count=1,
    flags=re.IGNORECASE,
)

def _insert_fetch_paragraphs(
    self,
    text: str,
    used_cursor_sets: list[str],
) -> str:
    if not used_cursor_sets:
        return text

    fetch_paragraphs = FetchParagraphGenerator(
        self.schema,
    ).generate(
        used_cursor_sets,
    )

    if not fetch_paragraphs.strip():
        return text

    return re.sub(
        r"(?=\n\s*MAIN-LINE\.)",
        "\n" + fetch_paragraphs + "\n",
        text,
        count=1,
        flags=re.IGNORECASE,
    )

def _remove_existing_fetch_paragraphs(
    self,
    text: str,
    used_cursor_sets: list[str],
) -> str:
    if not used_cursor_sets:
        return text

    for set_name in used_cursor_sets:
        fetch_name = f"FETCH-{set_name}".upper()

        pattern = re.compile(
            rf"""
            ^\s*{re.escape(fetch_name)}\.\s*
            .*?
            (?=
                ^\s*MAIN-LINE\.\s*$
                |
                ^\s*[A-Z0-9-]+\.\s*$
                |
                \Z
            )
            """,
            re.IGNORECASE | re.MULTILINE | re.DOTALL | re.VERBOSE,
        )

        text = pattern.sub(
            "",
            text,
        )

```

```

return text

def _replace_init_bind_ready(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"\bPERFORM\s+INIT-BIND-READY\b",
        "PERFORM INIT-DB2-SETUP",
        text,
        flags=re.IGNORECASE,
    )

    pattern = re.compile(
        r"""
        ^\s*INIT-BIND-READY\.\s*
        .*?
        (?=
            ^\s*INIT-FILES\.\s*$
            |
            \Z
        )
        """,
        re.IGNORECASE | re.MULTILINE | re.DOTALL | re.VERBOSE,
    )

    replacement = (
        "\n"
        "INIT-DB2-SETUP.\n"
        "        CONTINUE.\n"
    )

    return pattern.sub(
        replacement,
        text,
        count=1,
    )

def _remove_idms_status_perform(
    self,
    text: str,
) -> str:
    return re.sub(
        r"^\s*PERFORM\s+IDMS-STATUS\.\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

def _replace_finish(
    self,
    text: str,
) -> str:
    return re.sub(
        r"^\s*FINISH\.\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

def _remove_bind_ready_lines(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"^\s*BIND\s+[A-Z0-9-]+\.\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

    text = re.sub(
        r"^\s*READY\.\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

```

```

    )

    return text

def _clean_end_processing(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(\n\s*END-PROCESSING\.\s*)\n\s*CONTINUE\.\s*",
        r"\1",
        text,
        flags=re.IGNORECASE,
    )

    return text

def _remove_idms_abort_paragraphs(
    self,
    text: str,
) -> str:
    return re.sub(
        r"\n\s*IDMS-ABORT\.\s*(?:\n\s*EXIT\.\s*)?\n\s*IDMS-ABORT-EXIT\.\s*(?:\n\s*EXIT\.\s*)?",
        "\n",
        text,
        flags=re.IGNORECASE | re.DOTALL,
    )

def _remove_obsolete_set_walk_guard(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"""
        \n\s*CONTINUE\.\s*
        \n\s*IF\s+SQLCODE\s*=\s*100\s*
        \n\s*GO\s+TO\s+[A-Z0-9-]+\s+-EXIT\s*
        \n\s*ELSE\s*
        """,
        re.IGNORECASE | re.VERBOSE,
    )

    return pattern.sub(
        "\n        CONTINUE.\n",
        text,
    )

def _cleanup_orphan_sqlcode_blocks(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"""
        (\n\s*CONTINUE\.\s*)
        \n\s*IF\s+SQLCODE\s+NOT\s*=\s*0\s+AND\s+SQLCODE\s+NOT\s*=\s*100\s*
        \n\s*PERFORM\s+SQL-ERROR\s*
        \n\s*END-IF\.?
        """,
        r"\1",
        text,
        flags=re.IGNORECASE | re.VERBOSE,
    )

    text = re.sub(
        r"""
        (\n\s*END-PROCESSING\.\s*)
        \s*IF\s+SQLCODE\s+NOT\s*=\s*0\s+AND\s+SQLCODE\s+NOT\s*=\s*100\s*
        \n\s*PERFORM\s+SQL-ERROR\s*
        \n\s*END-IF\.?
        """,
        r"\1",
        text,
        flags=re.IGNORECASE | re.VERBOSE,
    )

    return text

```

```

def _fix_paragraph_header_concatenation(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(\b[A-Z0-9-]+\.\.)\s*(IF\s+SQLCODE)",
        r"\1\n        \2",
        text,
        flags=re.IGNORECASE,
    )

    text = re.sub(
        r"(\b[A-Z0-9-]+\.\.)\s*(EXEC\s+SQL)",
        r"\1\n        \2",
        text,
        flags=re.IGNORECASE,
    )

    return text

def _normalize_formatting(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"\n{3,}",
        "\n\n",
        text,
    )

    text = re.sub(
        r"[\t]+\$",
        "",
        text,
        flags=re.MULTILINE,
    )

    return text

def _add_sql_error_paragraph(
    self,
    text: str,
) -> str:
    if re.search(
        r"^\s*SQL-ERROR\.",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    ):
        return text

    return text.rstrip() + ""

SQL-ERROR.
MOVE SPACES TO ERR-LINE.
MOVE 'SQL ERROR' TO ERR-MESS-OUT.
MOVE ERR-DETAIL-LINE TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.

SQL-ERROR-EXIT.
EXIT.
"""

def _used_records(
    self,
    used_cursor_sets: list[str] | None = None,
) -> list[str]:
    records: set[str] = set()

    for record in getattr(self.analysis, "idms_records", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    for record in getattr(self.analysis, "obtain_calc_records", []):

```

```

        self._add_record(
            records=records,
            record_name=record,
        )

    for record, _ in getattr(self.analysis, "obtain_next", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    for record in getattr(self.analysis, "store_records", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    for record in getattr(self.analysis, "modify_records", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    for record in getattr(self.analysis, "erase_records", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    self._add_relationship_records_for_used_sets(
        records=records,
        used_cursor_sets=used_cursor_sets or [],
    )

    self._add_field_reference_records(
        records=records,
    )

    return sorted(records)

def _add_record(
    self,
    records: set[str],
    record_name: str | None,
) -> None:
    if not record_name:
        return

    value = str(record_name).strip().upper()

    if not value:
        return

    records.add(
        value,
    )

def _used_set_names(
    self,
    used_cursor_sets: list[str],
) -> set[str]:
    set_names: set[str] = set()

    for set_name in used_cursor_sets:
        if set_name:
            set_names.add(
                set_name.upper(),
            )

    for _, set_name in getattr(self.analysis, "obtain_next", []):
        if set_name:
            set_names.add(
                set_name.upper(),
            )

    for set_name in getattr(self.analysis, "obtain_owner_sets", []):

```

```

        if set_name:
            set_names.add(
                set_name.upper(),
            )

    for set_name in getattr(self.analysis, "find_first_sets", []):
        if set_name:
            set_names.add(
                set_name.upper(),
            )

    return set_names

def _add_relationship_records_for_used_sets(
    self,
    records: set[str],
    used_cursor_sets: list[str],
) -> None:
    used_set_names = self._used_set_names(
        used_cursor_sets=used_cursor_sets,
    )

    if not used_set_names:
        return

    for set_name in used_set_names:
        relationship = self.schema.relationships.get(
            set_name,
        )

        if not relationship:
            continue

        self._add_record(
            records=records,
            record_name=relationship.parent_record,
        )

        self._add_record(
            records=records,
            record_name=relationship.child_record,
        )

def _add_field_reference_records(
    self,
    records: set[str],
) -> None:
    field_references = (
        getattr(self.analysis, "field_references", [])
        or getattr(self.analysis, "idms_fields", [])
        or []
    )

    for field_name in field_references:
        for record_name in self._record_names_from_field_reference(
            field_name=field_name,
        ):
            self._add_record(
                records=records,
                record_name=record_name,
            )

def _record_names_from_field_reference(
    self,
    field_name: str,
) -> list[str]:
    result: list[str] = []

    if not field_name:
        return result

    field_map = getattr(
        self.schema,
        "field_map",
        {},
    )

```

```

meta = field_map.get(
    field_name.upper(),
)

if not meta:
    return result

record = (
    meta.get("record")
    or meta.get("table")
    or meta.get("record_name")
    or meta.get("table_name")
)

if record:
    result.append(
        str(record).upper(),
    )

host = meta.get(
    "host",
)

if host:
    host_record = self._record_name_from_host(
        str(host),
    )

    if host_record:
        result.append(
            host_record,
        )

column_record = meta.get(
    "logical_record",
)

if column_record:
    result.append(
        str(column_record).upper(),
    )

return result

def _record_name_from_host(
    self,
    host: str,
) -> str | None:
    normalized_host = self._normalize_name(
        host,
    )

    if not normalized_host.startswith("HV-"):
        return None

    candidates: list[str] = []

    candidates.extend(
        list(getattr(self.schema, "record_table_map", {}).keys()),
    )

    candidates.extend(
        list(getattr(self.schema, "records", {}).keys()),
    )

    candidates = sorted(
        set(
            str(candidate).upper()
            for candidate in candidates
            if candidate
        ),
        key=len,
        reverse=True,
    )

```



```

    for candidate in candidates:
        normalized_candidate = self._normalize_name(
            candidate,
        )

        prefix = f"HV-{normalized_candidate}-"

        if normalized_host.startswith(prefix):
            return candidate.upper()

    return None

def _normalize_name(
    self,
    value: str,
) -> str:
    return (
        str(value)
        .strip()
        .upper()
        .replace("_", "-")
        .replace(" ", "-")
    )

```

```

=====
FILE: src\idms_db2_converter\transformers\structured_blocks.py
ENCODING: utf-8
=====

```

```

class StructuredBlocks:
    @staticmethod
    def indent(block: str, spaces: int) -> str:
        pad = " " * spaces
        lines = []

        for line in block.splitlines():
            stripped = line.strip()
            if stripped:
                lines.append(pad + stripped)
            else:
                lines.append("")

        return "\n".join(lines)

    @staticmethod
    def sqlcode_select_block(
        sql: str,
        not_found_body: str,
        success_body: str,
        indent: int = 7,
    ) -> str:
        pad = " " * indent
        inner = " " * (indent + 3)
        inner2 = " " * (indent + 6)

        return "\n".join(
            [
                StructuredBlocks.indent(sql, indent),
                "",
                f"{pad}IF SQLCODE = 100",
                StructuredBlocks.indent(not_found_body, indent + 3),
                f"{pad}ELSE",
                f"{inner}IF SQLCODE NOT = 0",
                f"{inner2}PERFORM SQL-ERROR",
                f"{inner}ELSE",
                StructuredBlocks.indent(success_body, indent + 6),
                f"{inner}END-IF",
                f"{pad}END-IF.",
            ]
        )

    @staticmethod
    def select_first_then_owner_chain(
        first_select_sql: str,
        owner_select_sql: str,
    )

```

```

first_not_found_body: str,
owner_not_found_body: str,
success_body: str,
indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)
    inner2 = " " * (indent + 6)
    inner3 = " " * (indent + 9)

    return "\n".join(
        [
            StructuredBlocks.indent(first_select_sql, indent),
            "",
            f"{pad}IF SQLCODE = 100",
            StructuredBlocks.indent(first_not_found_body, indent + 3),
            f"{pad}ELSE",
            f"{inner}IF SQLCODE NOT = 0",
            f"{inner2}PERFORM SQL-ERROR",
            f"{inner}ELSE",
            StructuredBlocks.indent(owner_select_sql, indent + 6),
            "",
            f"{inner2}IF SQLCODE = 100",
            StructuredBlocks.indent(owner_not_found_body, indent + 9),
            f"{inner2}ELSE",
            f"{inner3}IF SQLCODE NOT = 0",
            f"{inner3}    PERFORM SQL-ERROR",
            f"{inner3}ELSE",
            StructuredBlocks.indent(success_body, indent + 12),
            f"{inner3}END-IF",
            f"{inner2}END-IF",
            f"{inner}END-IF",
            f"{pad}END-IF.",
        ]
    )

```

```

@staticmethod
def nullable_owner_block(
    null_indicator: str,
    null_body: str,
    owner_select_sql: str,
    success_body: str,
    fallback_body: str,
    indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)

    return "\n".join(
        [
            f"{pad}IF {null_indicator} < 0",
            StructuredBlocks.indent(null_body, indent + 3),
            f"{pad}ELSE",
            StructuredBlocks.indent(owner_select_sql, indent + 3),
            "",
            f"{inner}IF SQLCODE = 0",
            StructuredBlocks.indent(success_body, indent + 6),
            f"{inner}ELSE",
            StructuredBlocks.indent(fallback_body, indent + 6),
            f"{inner}END-IF",
            f"{pad}END-IF.",
        ]
    )

```

```

@staticmethod
def owner_select_block(
    owner_select_sql: str,
    success_body: str,
    fallback_body: str,
    indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)

    return "\n".join(
        [

```

```

        StructuredBlocks.indent(owner_select_sql, indent),
        """
        f"{pad}IF SQLCODE = 0",
        StructuredBlocks.indent(success_body, indent + 3),
        f"{pad}ELSE",
        StructuredBlocks.indent(fallback_body, indent + 3),
        f"{pad}END-IF.",
    ]
)

@staticmethod
def cursor_loop_block(
    cursor_name: str,
    fetch_paragraph: str,
    empty_body: str,
    before_loop_body: str,
    walk_paragraph: str,
    walk_exit: str,
    indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)
    inner2 = " " * (indent + 6)

    return "\n".join(
        [
            f"{pad}EXEC SQL",
            f"{pad}    OPEN {cursor_name}",
            f"{pad}END-EXEC.",
            "",
            f"{pad}IF SQLCODE NOT = 0",
            f"{inner}PERFORM SQL-ERROR",
            f"{pad}ELSE",
            f"{inner}PERFORM {fetch_paragraph}",
            "",
            f"{inner}IF SQLCODE = 100",
            StructuredBlocks.indent(empty_body, indent + 6),
            f"{inner}ELSE",
            StructuredBlocks.indent(before_loop_body, indent + 6),
            "",
            f"{inner2}PERFORM UNTIL SQLCODE = 100",
            f"{inner2}    PERFORM {walk_paragraph} THRU {walk_exit}",
            f"{inner2}    PERFORM {fetch_paragraph}",
            f"{inner2}END-PERFORM",
            f"{inner}END-IF",
            "",
            f"{inner}EXEC SQL",
            f"{inner}    CLOSE {cursor_name}",
            f"{inner}END-EXEC",
            f"{pad}END-IF",
        ]
    )

```

```

=====
FILE: src\idms_db2_converter\validators\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_converter\validators\output_validator.py
ENCODING: utf-8
=====

```

```
import re
```

```

class OutputValidator:
    """
    Validates generated DB2 COBOL output.

    This validator is generic:
    - No application record names are hard-coded.

```

- No demo set names are hard-coded.
- Paragraph names like READY-AREA are not treated as IDMS READY commands.
- Multiline SQL column lists are not falsely flagged as empty parentheses.

```

FORBIDDEN_EXECUTABLE_PATTERNS = [
    r"^\\s*OBTAIN\\b",
    r"^\\s*FIND\\b",
    r"^\\s*READY(?:\\s|\\.|\\$)",
    r"^\\s*BIND(?:\\s|\\.|\\$)",
    r"^\\s*FINISH\\.?\\s*$",
    r"^\\s*KEEP\\b",
    r"^\\s*STORE\\b",
    r"^\\s*MODIFY\\b",
    r"^\\s*ERASE\\b",
    r"^\\s*CONNECT\\b",
    r"^\\s*DISCONNECT\\b",
    r"^\\s*GET\\b",
    r"^\\s*IF\\s+DB-[A-Z0-9-]+\\b",
    r"^\\s*PERFORM\\s+IDMS-STATUS\\.?\\s*$",
    r"^\\s*USAGE-MODE\\s+IS\\s+UPDATE\\.?\\s*$",
]

REQUIRED = [
    "EXEC SQL",
    "SQLCA",
    "END-EXEC",
]

EXEC_SQL_START = re.compile(
    r"^\\s*EXEC\\s+SQL\\b",
    re.IGNORECASE,
)

EXEC_SQL_END = re.compile(
    r"^\\s*END-EXEC\\.?\\s*$",
    re.IGNORECASE,
)

COMMENT_LINE = re.compile(
    r"^\\s*\\*",
)

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^\\s*[A-Z0-9-]+\\.\\s*$",
    re.IGNORECASE,
)

UNCONVERTED_IDMS_FIELD_PATTERN = re.compile(
    r"\\b[A-Z][A-Z0-9]*(?:-[A-Z0-9-]+)*-\\d{4}\\b",
    re.IGNORECASE,
)

HOST_VARIABLE_PATTERN = re.compile(
    r"\\b(?:HV|NI)-[A-Z0-9-]+(?:\\$\\d+\\$)?\\b",
    re.IGNORECASE,
)

def validate(
    self,
    text: str,
) -> list[str]:
    errors: list[str] = []

    executable_text = self.executable_text(
        text,
    )

    upper_executable = executable_text.upper()
    upper_all = text.upper()

    self.validate_forbidden_idms_patterns(
        upper=upper_executable,
        errors=errors,
    )

```

```

self.validate_required_db2_tokens(
    upper=upper_all,
    errors=errors,
)

self.validate_no_nested_host_variables(
    upper=upper_all,
    errors=errors,
)

self.validate_no_rewritten_report_fields(
    upper=upper_all,
    errors=errors,
)

self.validate_no_unconverted_idms_schema_fields(
    text=executable_text,
    errors=errors,
)

self.validate_sql_blocks_closed(
    text=text,
    errors=errors,
)

self.validate_sql_punctuation(
    text=text,
    errors=errors,
)

return errors

def executable_text(
    self,
    text: str,
) -> str:
    lines: list[str] = []
    in_exec_sql = False
    in_data_division = False
    in_procedure_division = False

    for line in text.splitlines():
        stripped = line.strip()
        upper = stripped.upper()

        if not stripped:
            continue

        if self.COMMENT_LINE.match(line):
            continue

        if upper.startswith("DATA DIVISION"):
            in_data_division = True
            in_procedure_division = False
            continue

        if upper.startswith("PROCEDURE DIVISION"):
            in_data_division = False
            in_procedure_division = True
            lines.append(line)
            continue

        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True
            lines.append(line)

            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

            continue

        if in_exec_sql:
            lines.append(line)

            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

```

```

        continue

    if in_data_division:
        continue

    if not in_procedure_division:
        continue

    lines.append(line)

    return "\n".join(lines)

def validate_forbidden_idms_patterns(
    self,
    upper: str,
    errors: list[str],
) -> None:
    for pattern in self.FORBIDDEN_EXECUTABLE_PATTERNS:
        if re.search(
            pattern,
            upper,
            flags=re.IGNORECASE | re.MULTILINE,
        ):
            errors.append(
                f"Forbidden IDMS executable pattern remains: {pattern}"
            )

def validate_required_db2_tokens(
    self,
    upper: str,
    errors: list[str],
) -> None:
    for token in self.REQUIRED:
        if token not in upper:
            errors.append(
                f"Required DB2 token missing: {token}"
            )

def validate_no_nested_host_variables(
    self,
    upper: str,
    errors: list[str],
) -> None:
    if re.search(
        r"\bHV-[A-Z0-9-]+-HV-[A-Z0-9-]+\b",
        upper,
    ):
        errors.append(
            "Converted output contains nested host variables like HV-...-HV-...."
        )

    if re.search(
        r"\bNI-[A-Z0-9-]+-NI-[A-Z0-9-]+\b",
        upper,
    ):
        errors.append(
            "Converted output contains nested null indicators like NI-...-NI-...."
        )

def validate_no_rewritten_report_fields(
    self,
    upper: str,
    errors: list[str],
) -> None:
    bad_patterns = [
        r"\bHV-[A-Z0-9-]+-IN\b",
        r"\bHV-[A-Z0-9-]+-OUT\b",
        r"\bNI-[A-Z0-9-]+-IN\b",
        r"\bNI-[A-Z0-9-]+-OUT\b",
    ]

    for pattern in bad_patterns:
        if re.search(
            pattern,
            upper,

```

```

):
    errors.append(
        "Converted output appears to have rewritten input/output report fields."
    )
    return

def validate_no_unconverted_idms_schema_fields(
    self,
    text: str,
    errors: list[str],
) -> None:
    lines = text.splitlines()
    in_exec_sql = False

    for line in lines:
        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True

            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

            continue

        if in_exec_sql:
            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

            continue

        if self.COMMENT_LINE.match(line):
            continue

        if self.PARAGRAPH_HEADER_PATTERN.match(line):
            continue

        scrubbed = self.remove_host_variables(
            line,
        )

        matches = self.UNCONVERTED_IDMS_FIELD_PATTERN.findall(
            scrubbed,
        )

        for match in matches:
            upper_match = match.upper()

            if upper_match.startswith("HV-"):
                continue

            if upper_match.startswith("NI-"):
                continue

            errors.append(
                f"Unconverted IDMS schema field remains in procedure logic: {match}"
            )

def remove_host_variables(
    self,
    line: str,
) -> str:
    return self.HOST_VARIABLE_PATTERN.sub(
        "",
        line,
    )

def validate_sql_blocks_closed(
    self,
    text: str,
    errors: list[str],
) -> None:
    start_count = 0
    end_count = 0

    for line in text.splitlines():
        if self.EXEC_SQL_START.match(line):
            start_count += 1

```

```

        if self.EXEC_SQL_END.match(line):
            end_count += 1

    if start_count != end_count:
        errors.append(
            f"EXEC SQL / END-EXEC block mismatch: EXEC SQL={start_count}, END-EXEC={end_count}"
        )

def validate_sql_punctuation(
    self,
    text: str,
    errors: list[str],
) -> None:
    sql_blocks = self.extract_sql_blocks(
        text,
    )

    for block in sql_blocks:
        self.validate_sql_commas(
            block=block,
            errors=errors,
        )

def extract_sql_blocks(
    self,
    text: str,
) -> list[str]:
    blocks: list[str] = []
    current: list[str] = []
    in_exec_sql = False

    for line in text.splitlines():
        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True
            current = [
                line,
            ]

            if self.EXEC_SQL_END.match(line):
                blocks.append(
                    "\n".join(current),
                )

                current = []
                in_exec_sql = False

            continue

        if in_exec_sql:
            current.append(
                line,
            )

            if self.EXEC_SQL_END.match(line):
                blocks.append(
                    "\n".join(current),
                )

                current = []
                in_exec_sql = False

    return blocks

def validate_sql_commas(
    self,
    block: str,
    errors: list[str],
) -> None:
    if re.search(
        r",\s*",
        block,
    ):
        errors.append(
            "SQL block contains duplicate commas."
        )

```



```

if re.search(
    r",\s*(FROM|WHERE|END-EXEC)",
    block,
    flags=re.IGNORECASE,
):
    errors.append(
        "SQL block contains trailing comma before FROM, WHERE, or END-EXEC."
    )

```

```

=====
FILE: src\idms_db2_converter\validators\schema_validator.py
ENCODING: utf-8
=====

```

```

import re
from idms_db2_converter.models import CobolAnalysis, Column, Relationship, SchemaModel

```

```

class SchemaValidator:

```

```

    """

```

```

    Validates and enriches schema references used by COBOL.

```

```

    This validator intentionally performs generic inference:

```

- Infers missing primary keys from field names.
- Infers missing set relationships from set-name tokens and COBOL usage.
- Does not hard-code application-specific record or set names.

```

    """

```

```

def validate(self, schema: SchemaModel, analysis: CobolAnalysis) -> list[str]:
    errors = []

```

```

    self._ensure_primary_keys(schema)

```

```

    for record in analysis.idms_records:

```

```

        if record not in schema.records:

```

```

            errors.append(
                f"COBOL references record {record}, but schema does not define it."
            )

```

```

    for record in analysis.obtain_calc_records:

```

```

        if record not in schema.records:

```

```

            errors.append(f"OBTAIN CALC references missing record {record}.")
            continue

```

```

        if not schema.records[record].primary_key:

```

```

            self._infer_primary_key_for_record(schema, record)

```

```

        if not schema.records[record].primary_key:

```

```

            errors.append(f"OBTAIN CALC record {record} has no primary key.")

```

```

    for record_name, set_name in analysis.obtain_next:

```

```

        self._validate_set(

```

```

            schema=schema,
            set_name=set_name,
            errors=errors,
            expected_child_record=record_name,
        )

```

```

    for set_name in analysis.obtain_owner_sets:

```

```

        self._validate_set(

```

```

            schema=schema,
            set_name=set_name,
            errors=errors,
            expected_child_record=None,
        )

```

```

    for set_name in analysis.find_first_sets:

```

```

        self._validate_set(

```

```

            schema=schema,
            set_name=set_name,
            errors=errors,
            expected_child_record=None,
        )

```

```

return errors

def _ensure_primary_keys(self, schema: SchemaModel) -> None:
    for record_name in list(schema.records.keys()):
        self._infer_primary_key_for_record(schema, record_name)

def _infer_primary_key_for_record(
    self,
    schema: SchemaModel,
    record_name: str,
) -> None:
    if record_name not in schema.records:
        return

    record = schema.records[record_name]

    if record.primary_key:
        if record.primary_key in record.fields:
            record.fields[record.primary_key].nullable = False
            return

    candidate = self._find_primary_key_candidate(
        record_name=record_name,
        fields=set(record.fields.keys()),
    )

    if candidate:
        record.primary_key = candidate
        record.fields[candidate].nullable = False
        return

    if record.fields:
        first_field = next(iter(record.fields.keys()))
        record.primary_key = first_field
        record.fields[first_field].nullable = False

def _find_primary_key_candidate(
    self,
    record_name: str,
    fields: set[str],
) -> str | None:
    normalized_record = self._normalize(record_name)
    compact_record = self._compact(normalized_record)

    direct_candidates = [
        f"{normalized_record}_ID",
        f"{normalized_record}_CODE",
    ]

    for candidate in direct_candidates:
        if candidate in fields:
            return candidate

    for field in fields:
        normalized_field = self._normalize(field)
        compact_field = self._compact(normalized_field)

        if normalized_field.endswith("_ID") and compact_record in compact_field:
            return field

        if normalized_field.endswith("_CODE") and compact_record in compact_field:
            return field

    id_fields = [
        field
        for field in fields
        if self._normalize(field).endswith("_ID")
    ]

    if len(id_fields) == 1:
        return id_fields[0]

    code_fields = [
        field
        for field in fields
        if self._normalize(field).endswith("_CODE")
    ]

```

```

]

if len(code_fields) == 1:
    return code_fields[0]

return None

def _validate_set(
    self,
    schema: SchemaModel,
    set_name: str,
    errors: list[str],
    expected_child_record: str | None,
) -> None:
    if set_name not in schema.relationships:
        inferred = self._infer_relationship(
            schema=schema,
            set_name=set_name,
            expected_child_record=expected_child_record,
        )

        if inferred:
            schema.relationships[set_name] = inferred
        else:
            errors.append(
                f"COBOL references set {set_name}, but schema relationships do not define it."
            )
            return

    rel = schema.relationships[set_name]

    if rel.parent_record not in schema.records:
        errors.append(
            f"Set {set_name} parent record {rel.parent_record} is missing."
        )
        return

    if rel.child_record not in schema.records:
        errors.append(
            f"Set {set_name} child record {rel.child_record} is missing."
        )
        return

    if not rel.parent_key:
        self._infer_primary_key_for_record(schema, rel.parent_record)
        rel.parent_key = schema.records[rel.parent_record].primary_key

    if not rel.child_fk:
        rel.child_fk = self._infer_child_fk(
            schema=schema,
            parent_record=rel.parent_record,
            child_record=rel.child_record,
            parent_key=rel.parent_key,
        )

    if not rel.parent_key:
        errors.append(f"Set {set_name} has no parent_key.")

    if not rel.child_fk:
        errors.append(f"Set {set_name} has no child_fk.")

    if rel.parent_key and rel.parent_key not in schema.records[rel.parent_record].fields:
        self._add_field_like_parent_key(
            schema=schema,
            target_record=rel.parent_record,
            source_record=rel.parent_record,
            source_key=rel.parent_key,
            target_field=rel.parent_key,
            nullable=False,
        )

    if rel.child_fk and rel.child_fk not in schema.records[rel.child_record].fields:
        self._add_field_like_parent_key(
            schema=schema,
            target_record=rel.child_record,
            source_record=rel.parent_record,

```

```

        source_key=rel.parent_key,
        target_field=rel.child_fk,
        nullable=True,
    )

def _infer_relationship(
    self,
    schema: SchemaModel,
    set_name: str,
    expected_child_record: str | None,
) -> Relationship | None:
    child_record = self._infer_child_record(
        schema=schema,
        set_name=set_name,
        expected_child_record=expected_child_record,
    )

    if not child_record:
        return None

    parent_record = self._infer_parent_record(
        schema=schema,
        set_name=set_name,
        child_record=child_record,
    )

    if not parent_record:
        return None

    self._infer_primary_key_for_record(schema, parent_record)

    parent_key = schema.records[parent_record].primary_key

    if not parent_key:
        return None

    child_fk = self._infer_child_fk(
        schema=schema,
        parent_record=parent_record,
        child_record=child_record,
        parent_key=parent_key,
    )

    if not child_fk:
        child_fk = parent_key

    if child_fk not in schema.records[child_record].fields:
        self._add_field_like_parent_key(
            schema=schema,
            target_record=child_record,
            source_record=parent_record,
            source_key=parent_key,
            target_field=child_fk,
            nullable=True,
        )

    return Relationship(
        set_name=set_name,
        parent_record=parent_record,
        child_record=child_record,
        cardinality="1:N",
        parent_key=parent_key,
        child_fk=child_fk,
        order_by=[child_fk],
    )

def _infer_child_record(
    self,
    schema: SchemaModel,
    set_name: str,
    expected_child_record: str | None,
) -> str | None:
    if expected_child_record and expected_child_record in schema.records:
        return expected_child_record

    set_tokens = self._tokens(set_name)

```

```

best_score = 0
best_record = None

for record_name in schema.records:
    score = self._score_record_against_tokens(record_name, set_tokens)

    if score > best_score:
        best_score = score
        best_record = record_name

return best_record if best_score > 0 else None

def _infer_parent_record(
    self,
    schema: SchemaModel,
    set_name: str,
    child_record: str,
) -> str | None:
    set_tokens = self._tokens(set_name)

    best_score = 0
    best_record = None

    for record_name in schema.records:
        if record_name == child_record:
            continue

        score = self._score_record_against_tokens(record_name, set_tokens)

        if score > best_score:
            best_score = score
            best_record = record_name

    return best_record if best_score > 0 else None

def _infer_child_fk(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
    parent_key: str | None,
) -> str | None:
    if not parent_key:
        return None

    child = schema.records[child_record]
    parent_key_norm = self._normalize(parent_key)

    if parent_key in child.fields:
        return parent_key

    if parent_key_norm in child.fields:
        return parent_key_norm

    parent_tokens = self._tokens(parent_record)
    key_suffixes = ["_ID", "_CODE"]

    for field_name in child.fields:
        normalized_field = self._normalize(field_name)

        if normalized_field == parent_key_norm:
            return field_name

        for token in parent_tokens:
            if normalized_field.startswith(token + "_"):
                for suffix in key_suffixes:
                    if normalized_field.endswith(suffix):
                        return field_name

    parent_compact = self._compact(parent_record)

    for field_name in child.fields:
        compact_field = self._compact(field_name)

        if parent_compact and parent_compact in compact_field:

```

```

        return field_name

    return None

def _add_field_like_parent_key(
    self,
    schema: SchemaModel,
    target_record: str,
    source_record: str,
    source_key: str | None,
    target_field: str,
    nullable: bool,
) -> None:
    if not source_key:
        return

    if source_record not in schema.records:
        return

    source = schema.records[source_record]
    target = schema.records[target_record]

    source_column = source.fields.get(source_key)

    if source_column:
        target.fields[target_field] = Column(
            name=target_field,
            datatype=source_column.datatype,
            length=source_column.length,
            scale=source_column.scale,
            nullable=nullable,
        )
    else:
        target.fields[target_field] = Column(
            name=target_field,
            datatype="CHAR",
            length=12,
            scale=None,
            nullable=nullable,
        )

def _score_record_against_tokens(
    self,
    record_name: str,
    tokens: list[str],
) -> int:
    normalized_record = self._normalize(record_name)
    compact_record = self._compact(record_name)

    score = 0

    for token in tokens:
        normalized_token = self._normalize(token)
        compact_token = self._compact(token)

        if normalized_token == normalized_record:
            score += 20
            continue

        if normalized_record.startswith(normalized_token):
            score += 10
            continue

        if normalized_token.startswith(normalized_record):
            score += 10
            continue

        if compact_token and compact_record.startswith(compact_token):
            score += 6
            continue

        if compact_token and compact_token in compact_record:
            score += 4
            continue

        if normalized_token[:3] and normalized_record.startswith(normalized_token[:3]):

```

```

        score += 2

    return score

def _tokens(self, value: str) -> list[str]:
    normalized = self._normalize(value)

    return [
        token
        for token in normalized.split("_")
        if token
    ]

def _normalize(self, value: str) -> str:
    return value.upper().replace("-", "_")

def _compact(self, value: str) -> str:
    return re.sub(r"^[A-Z0-9]", "", self._normalize(value))

=====
FILE: src\idms_modernizer\__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_modernizer\app.py
ENCODING: utf-8
=====

from pathlib import Path
from time import perf_counter

import streamlit as st

from idms_modernizer.core.config import settings
from idms_modernizer.core.utils import ensure_directory
from idms_modernizer.core.constants import (
    ER_DIAGRAM_FILE,
    PHASE2_METADATA_FILE,
)
from idms_modernizer.parsers.cobol_pdf_extractor import CobolPdfExtractor
from idms_modernizer.services.metadata_service import MetadataService
from idms_modernizer.services.canonical_schema_builder import (
    CanonicalSchemaBuilder,
)
from idms_modernizer.services.canonical_reconciler import (
    CanonicalReconciler,
)
from idms_modernizer.services.db2_model_builder import DB2ModelBuilder
from idms_modernizer.services.phase2_metadata_generator import (
    Phase2MetadataGenerator,
)
from idms_modernizer.services.phase2_conversion_bridge import (
    Phase2ConversionBridge,
)
from idms_modernizer.services.er_diagram_generator import (
    generate_er_diagram,
)
from idms_modernizer.services.excel_sheet_mapping_service import (
    ExcelSheetMappingService,
)
from idms_modernizer.generators.ddl_generator import DDLGenerator

def initialize_directories() -> None:
    ensure_directory(
        settings.input_dir,
    )

    ensure_directory(
        settings.output_dir,
    )

```

```

def initialize_session_state() -> None:
    defaults = {
        "generated": False,
        "schema_path": "",
        "cobol_pdf_path": "",
        "metadata": None,
        "canonical_schema": None,
        "db2_model": None,
        "ddl_text": "",
        "canonical_json": "",
        "phase2_metadata_json": "",
        "cobol_text": "",
        "converted_cobol": "",
        "relationships": [],
        "all_sets": [],
        "record_count": 0,
        "set_count": 0,
        "relationship_count": 0,
        "validation_messages": [],
        "generation_warnings": [],
        "generation_timings": [],
        "er_png_path": "",
        "er_image_bytes": None,
        "output_files": {},
        "excel_mapping_rows": [],
    }

    for key, value in defaults.items():
        if key not in st.session_state:
            st.session_state[key] = value

def reset_outputs() -> None:
    st.session_state.generated = False
    st.session_state.schema_path = ""
    st.session_state.cobol_pdf_path = ""
    st.session_state.metadata = None
    st.session_state.canonical_schema = None
    st.session_state.db2_model = None
    st.session_state.ddl_text = ""
    st.session_state.canonical_json = ""
    st.session_state.phase2_metadata_json = ""
    st.session_state.cobol_text = ""
    st.session_state.converted_cobol = ""
    st.session_state.relationships = []
    st.session_state.all_sets = []
    st.session_state.record_count = 0
    st.session_state.set_count = 0
    st.session_state.relationship_count = 0
    st.session_state.validation_messages = []
    st.session_state.generation_warnings = []
    st.session_state.generation_timings = []
    st.session_state.er_png_path = ""
    st.session_state.er_image_bytes = None
    st.session_state.output_files = {}
    st.session_state.excel_mapping_rows = []

def save_uploaded_file(
    uploaded_file,
    target_folder: str,
) -> str:
    target_dir = Path(
        target_folder,
    )

    target_dir.mkdir(
        parents=True,
        exist_ok=True,
    )

    target_path = target_dir / uploaded_file.name

    with open(

```



```

        target_path,
        "wb",
    ) as output_file:
        output_file.write(
            uploaded_file.getbuffer(),
        )

    return str(
        target_path,
    )

def write_output_file(
    output_dir: Path,
    file_name: str,
    content: str,
    encoding: str = "utf-8",
) -> Path:
    output_dir.mkdir(
        parents=True,
        exist_ok=True,
    )

    path = output_dir / file_name

    path.write_text(
        content,
        encoding=encoding,
    )

    return path

def log_timing(
    timings: list[str],
    label: str,
    step_start: float,
) -> float:
    elapsed = perf_counter() - step_start

    timings.append(
        f"{label}: {elapsed:.2f} seconds",
    )

    return perf_counter()

def build_sets_summary(
    metadata,
) -> list[str]:
    all_sets = set()

    for record in metadata.records:
        for membership in record.set_memberships:
            set_name = getattr(
                membership,
                "set_name",
                "",
            )

            if set_name:
                all_sets.add(
                    set_name,
                )

    return sorted(
        all_sets,
    )

def build_relationship_rows(
    relationships,
) -> list[dict[str, str]]:
    rows = []

    for relationship in relationships:

```

```

rows.append(
    {
        "Owner Record": getattr(
            relationship,
            "owner_record",
            "",
        ),
        "Member Record": getattr(
            relationship,
            "member_record",
            "",
        ),
        "Set Name": getattr(
            relationship,
            "set_name",
            "",
        ),
    }
)

return rows

def build_mapping_row_count_by_table(
    rows: list[dict],
    include_total: bool = True,
) -> list[dict[str, int | str]]:
    """
    Builds table-level counts from the actual Excel Sheet Mapping rows.

    This count matches the detailed mapping table because it counts the
    generated mapping rows, not only DB2 physical columns.

    Output column order:
    - Column Field Count
    - Table Name
    """
    table_counts: dict[str, int] = {}

    for row in rows or []:
        table_name = (
            row.get("New DB2 Record")
            or row.get("Cobol Record IDMS")
            or ""
        )

        table_name = str(
            table_name,
        ).strip()

        if not table_name:
            continue

        table_counts[table_name] = table_counts.get(
            table_name,
            0,
        ) + 1

    output_rows: list[dict[str, int | str]] = []

    total_count = 0

    for table_name in sorted(table_counts):
        count = table_counts[table_name]

        output_rows.append(
            {
                "Column Field Count": count,
                "Table Name": table_name,
            }
        )

        total_count += count

    if include_total:
        output_rows.append(

```

```

        {
            "Column Field Count": total_count,
            "Table Name": "TOTAL",
        }
    )

    return output_rows

def build_db2_physical_column_count_rows(
    db2_model,
    include_total: bool = True,
) -> list[dict[str, int | str]]:
    """
    Builds table-level physical DB2 column counts.

    This is separate from mapping row counts because COBOL YEAR, MONTH,
    and DAY fields can map to one DB2 DATE column.
    """
    table_rows: list[dict[str, int | str]] = []

    if db2_model is None:
        return table_rows

    total_count = 0

    for table in getattr(
        db2_model,
        "tables",
        [],
    ) or []:
        table_name = getattr(
            table,
            "name",
            "",
        )

        columns = getattr(
            table,
            "columns",
            [],
        ) or []

        count = len(
            columns,
        )

        table_rows.append(
            {
                "Column Field Count": count,
                "Table Name": table_name,
            }
        )

        total_count += count

    if include_total:
        table_rows.append(
            {
                "Column Field Count": total_count,
                "Table Name": "TOTAL",
            }
        )

    return table_rows

def generate_outputs(
    schema_pdf,
    cobol_pdf,
) -> None:
    total_start = perf_counter()
    step_start = perf_counter()
    timings: list[str] = []

    reset_outputs()

```

```

warnings = []

schema_path = save_uploaded_file(
    schema_pdf,
    settings.input_dir,
)

cobol_pdf_path = save_uploaded_file(
    cobol_pdf,
    settings.input_dir,
)

step_start = log_timing(
    timings,
    "Save uploaded files",
    step_start,
)

metadata_service = MetadataService()

metadata = metadata_service.build_metadata(
    schema_path,
)

step_start = log_timing(
    timings,
    "Build metadata from schema PDF",
    step_start,
)

canonical_builder = CanonicalSchemaBuilder()

canonical_schema = canonical_builder.build(
    metadata=metadata,
)

step_start = log_timing(
    timings,
    "Build canonical schema",
    step_start,
)

canonical_reconciler = CanonicalReconciler()

canonical_schema = canonical_reconciler.reconcile(
    canonical_schema=canonical_schema,
    metadata=metadata,
)

step_start = log_timing(
    timings,
    "Reconcile canonical schema",
    step_start,
)

db2_builder = DB2ModelBuilder()

db2_model = db2_builder.build(
    canonical_schema,
)

step_start = log_timing(
    timings,
    "Build DB2 model",
    step_start,
)

ddl_generator = DDLGenerator()

ddl_text = ddl_generator.generate(
    db2_model,
)

step_start = log_timing(
    timings,

```

```

        "Generate DB2 DDL",
        step_start,
    )

    excel_mapping_service = ExcelSheetMappingService()

    excel_mapping_rows = excel_mapping_service.build(
        metadata=metadata,
        db2_model=db2_model,
    )

    step_start = log_timing(
        timings,
        "Generate Excel Sheet Mapping rows",
        step_start,
    )

    phase2_metadata_generator = Phase2MetadataGenerator()

    phase2_metadata_json = phase2_metadata_generator.generate_json(
        canonical_schema=canonical_schema,
        db2_model=db2_model,
        metadata=metadata,
    )

    step_start = log_timing(
        timings,
        "Generate Phase 2 metadata JSON",
        step_start,
    )

    canonical_json = canonical_schema.model_dump_json(
        indent=2,
    )

    step_start = log_timing(
        timings,
        "Generate canonical JSON",
        step_start,
    )

    cobol_extractor = CobolPdfExtractor()

    cobol_text = cobol_extractor.extract_text(
        cobol_pdf_path,
    )

    step_start = log_timing(
        timings,
        "Extract COBOL PDF text",
        step_start,
    )

    converted_cobol = ""
    validation_messages = []

    try:
        phase2_bridge = Phase2ConversionBridge()

        converted_cobol, validation_messages = phase2_bridge.convert(
            cobol_text=cobol_text,
            canonical_json=canonical_json,
            phase2_metadata_json=phase2_metadata_json,
            ddl_text=ddl_text,
            idms_schema_text=None,
            relationship_overrides_json=None,
            target_program=None,
            use_ddl=True,
            use_idms_schema=False,
            use_overrides=False,
        )

    except Exception as exc:
        converted_cobol = ""
        validation_messages = [
            f"Phase 2 COBOL conversion failed: {exc}",

```

```

    ]

    step_start = log_timing(
        timings,
        "Phase 2 COBOL conversion",
        step_start,
    )

    relationships = metadata.relationships

    all_sets = build_sets_summary(
        metadata,
    )

    step_start = log_timing(
        timings,
        "Build relationship and set summaries",
        step_start,
    )

    output_dir = Path(
        settings.output_dir,
    )

    output_files = {}

    output_files["ddl"] = write_output_file(
        output_dir=output_dir,
        file_name="db2_ddl.sql",
        content=ddl_text,
    )

    output_files["canonical"] = write_output_file(
        output_dir=output_dir,
        file_name="canonical_model.json",
        content=canonical_json,
    )

    output_files["phase2_metadata"] = write_output_file(
        output_dir=output_dir,
        file_name=PHASE2_METADATA_FILE,
        content=phase2_metadata_json,
    )

    output_files["converted_cobol"] = write_output_file(
        output_dir=output_dir,
        file_name="converted_db2_cobol.cbl",
        content=converted_cobol,
    )

    step_start = log_timing(
        timings,
        "Write output files",
        step_start,
    )

    er_png_path = ""
    er_image_bytes = None

    try:
        er_path = output_dir / ER_DIAGRAM_FILE

        er_png_path = generate_er_diagram(
            metadata,
            str(
                er_path,
            ),
        )

        with open(
            er_png_path,
            "rb",
        ) as image_file:
            er_image_bytes = image_file.read()

        output_files["er_diagram"] = Path(

```

```

        er_png_path,
    )

except Exception as exc:
    warnings.append(
        "ER diagram was not generated. "
        "Graphviz may not be installed or configured. "
        f"Details: {exc}"
    )

step_start = log_timing(
    timings,
    "ER diagram generation",
    step_start,
)

st.session_state.generated = True
st.session_state.schema_path = schema_path
st.session_state.cobol_pdf_path = cobol_pdf_path
st.session_state.metadata = metadata
st.session_state.canonical_schema = canonical_schema
st.session_state.db2_model = db2_model
st.session_state.ddl_text = ddl_text
st.session_state.canonical_json = canonical_json
st.session_state.phase2_metadata_json = phase2_metadata_json
st.session_state.cobol_text = cobol_text
st.session_state.converted_cobol = converted_cobol
st.session_state.relationships = relationships
st.session_state.all_sets = all_sets
st.session_state.record_count = len(
    metadata.records,
)
st.session_state.set_count = len(
    all_sets,
)
st.session_state.relationship_count = len(
    relationships,
)
st.session_state.validation_messages = validation_messages
st.session_state.generation_warnings = warnings
st.session_state.er_png_path = er_png_path
st.session_state.er_image_bytes = er_image_bytes
st.session_state.output_files = output_files
st.session_state.excel_mapping_rows = excel_mapping_rows

total_elapsed = perf_counter() - total_start

timings.append(
    f"TOTAL: {total_elapsed:.2f} seconds",
)

st.session_state.generation_timings = timings

def render_download_button(
    label: str,
    path,
    file_name: str,
    mime: str,
) -> None:
    if not path:
        st.button(
            label,
            disabled=True,
            use_container_width=True,
        )
    return

with open(
    path,
    "rb",
) as file:
    st.download_button(
        label=label,
        data=file,
        file_name=file_name,

```

```

        mime=mime,
        use_container_width=True,
    )

def render_download_buttons() -> None:
    if not st.session_state.generated:
        return

    st.markdown("### Download Outputs")

    output_files = st.session_state.output_files

    col1, col2, col3, col4, col5 = st.columns(
        5,
    )

    with col1:
        render_download_button(
            label="DB2 DDL",
            path=output_files.get(
                "ddl",
            ),
            file_name="db2_ddl.sql",
            mime="text/sql",
        )

    with col2:
        render_download_button(
            label="Phase Metadata",
            path=output_files.get(
                "phase2_metadata",
            ),
            file_name=PHASE2_METADATA_FILE,
            mime="application/json",
        )

    with col3:
        render_download_button(
            label="Canonical JSON",
            path=output_files.get(
                "canonical",
            ),
            file_name="canonical_model.json",
            mime="application/json",
        )

    with col4:
        if st.session_state.er_image_bytes:
            st.download_button(
                label="ER Diagram",
                data=st.session_state.er_image_bytes,
                file_name=ER_DIAGRAM_FILE,
                mime="image/png",
                use_container_width=True,
            )
        else:
            st.button(
                "ER Diagram",
                disabled=True,
                use_container_width=True,
            )

    with col5:
        render_download_button(
            label="DB2 COBOL",
            path=output_files.get(
                "converted_cobol",
            ),
            file_name="converted_db2_cobol.cbl",
            mime="text/plain",
        )

def total_timing_only() -> str:
    for timing in st.session_state.generation_timings:

```



```

        if timing.startswith(
            "TOTAL:",
        ):
            return timing

    return ""

def get_total_generation_time() -> str:
    total_timing = total_timing_only()

    if not total_timing:
        return ""

    return total_timing.replace(
        "TOTAL:",
        "",
    ).strip()

def render_status_panel() -> None:
    if not st.session_state.generated:
        return

    total_time = get_total_generation_time()

    if total_time:
        st.success(
            f"Generation completed in {total_time}",
        )

    if st.session_state.generation_warnings:
        with st.expander(
            "Generation Warnings",
            expanded=False,
        ):
            for warning in st.session_state.generation_warnings:
                st.warning(
                    warning,
                )

    if not st.session_state.validation_messages:
        st.success(
            "COBOL conversion validation passed.",
        )
    else:
        st.warning(
            "COBOL conversion completed with validation messages. "
            "Open the Validation tab for details."
        )

def render_main_tab() -> None:
    st.markdown("### Generate Modernization Outputs")

    st.info(
        "Upload the Schema Listing PDF and COBOL Code PDF to generate DB2 "
        "DDL, metadata, ER diagram, converted DB2 COBOL, and sheet mapping."
    )

    col1, col2 = st.columns(
        2,
    )

    with col1:
        schema_pdf = st.file_uploader(
            "Schema Listing PDF",
            type=["pdf"],
            key="schema_listing_pdf",
        )

    with col2:
        cobol_pdf = st.file_uploader(
            "COBOL Code PDF",
            type=["pdf"],
            key="cobol_code_pdf",

```

```

    )

generate_clicked = st.button(
    "Generate Outputs",
    type="primary",
    use_container_width=True,
)

if generate_clicked:
    if schema_pdf is None:
        st.error(
            "Please upload the Schema Listing PDF.",
        )
        st.stop()

    if cobol_pdf is None:
        st.error(
            "Please upload the COBOL Code PDF.",
        )
        st.stop()

    with st.spinner(
        "Processing schema, generating DB2 artifacts, and converting COBOL..."
    ):
        try:
            generate_outputs(
                schema_pdf=schema_pdf,
                cobol_pdf=cobol_pdf,
            )

            st.success(
                "Generation completed successfully.",
            )

        except Exception as exc:
            st.error(
                "Generation failed.",
            )
            st.exception(
                exc,
            )
            st.stop()

if st.session_state.generated:
    st.markdown("### Output Status")

    col_a, col_b = st.columns(
        2,
    )

    with col_a:
        st.write(
            f"Schema PDF: `{st.session_state.schema_path}`",
        )

    with col_b:
        st.write(
            f"COBOL PDF: `{st.session_state.cobol_pdf_path}`",
        )

    render_status_panel()
    render_download_buttons()

def render_metadata_overview_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("## Metadata Overview")

    st.markdown("### Summary")

    col1, col2, col3, col4 = st.columns(

```

```

    4,
)

with col1:
    st.metric(
        "Records",
        st.session_state.record_count,
    )

with col2:
    st.metric(
        "Sets",
        st.session_state.set_count,
    )

with col3:
    st.metric(
        "Relationships",
        st.session_state.relationship_count,
    )

with col4:
    st.metric(
        "Sheet Mapping Rows",
        len(
            st.session_state.excel_mapping_rows,
        ),
    )

st.divider()

st.markdown("### Detected Sets")

st.json(
    st.session_state.all_sets,
)

st.divider()

st.markdown("### Detected Relationships")

relationship_rows = build_relationship_rows(
    st.session_state.relationships,
)

if relationship_rows:
    st.dataframe(
        relationship_rows,
        use_container_width=True,
        hide_index=True,
    )
else:
    st.info(
        "No relationships detected.",
    )

st.divider()

st.markdown("### Table List and Mapping Row Counts")

mapping_count_rows = build_mapping_row_count_by_table(
    rows=st.session_state.excel_mapping_rows,
    include_total=True,
)

if mapping_count_rows:
    st.dataframe(
        mapping_count_rows,
        use_container_width=True,
        hide_index=True,
    )
else:
    st.info(
        "No table mapping row counts available.",
    )

```

```

st.caption(
    "This count is based on actual Excel Sheet Mapping rows. "
    "The TOTAL matches the detailed mapping table."
)

st.divider()

st.markdown("### DB2 Physical Column Counts")

physical_count_rows = build_db2_physical_column_count_rows(
    db2_model=st.session_state.db2_model,
    include_total=True,
)

if physical_count_rows:
    st.dataframe(
        physical_count_rows,
        use_container_width=True,
        hide_index=True,
    )
else:
    st.info(
        "No DB2 physical column counts available.",
    )

st.caption(
    "This count is based on DB2 physical columns. It can differ from "
    "mapping row count when COBOL YEAR, MONTH, and DAY fields map to "
    "one DB2 DATE column."
)

def render_ddl_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("### DB2 DDL Preview")

    st.code(
        st.session_state.ddl_text,
        language="sql",
    )

def render_phase2_metadata_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("### Phase 2 Metadata Preview")

    st.code(
        st.session_state.phase2_metadata_json,
        language="json",
    )

def render_excel_sheet_mapping_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("### Excel Sheet Mapping")

    rows = st.session_state.excel_mapping_rows

    if not rows:
        st.info(
            "No mapping rows generated.",

```

```

    )
    return

st.markdown("### Sheet Map Abstract")

abstract_rows = build_mapping_row_count_by_table(
    rows=rows,
    include_total=True,
)

if abstract_rows:
    st.dataframe(
        abstract_rows,
        use_container_width=True,
        hide_index=True,
    )
else:
    st.info(
        "No table abstract rows available.",
    )

st.caption(
    "The TOTAL row matches the number of detailed mapping rows below."
)

st.markdown("### Sheet Mapping Details")

st.dataframe(
    rows,
    use_container_width=True,
    hide_index=True,
)

def render_er_diagram_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("## ER Diagram")

    if st.session_state.er_image_bytes:
        st.image(
            st.session_state.er_image_bytes,
        )
    else:
        st.warning(
            "ER diagram was not generated. Graphviz may not be installed "
            "or configured."
        )

def render_converted_cobol_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("## Converted DB2 COBOL")

    if not st.session_state.converted_cobol:
        st.warning(
            "Converted COBOL output is empty or conversion failed.",
        )

    st.text_area(
        "Final DB2 COBOL Code",
        value=st.session_state.converted_cobol,
        height=750,
    )

def render_validation_tab() -> None:

```

```

if not st.session_state.generated:
    st.info(
        "Generate outputs from the Main tab first.",
    )
    return

st.markdown("## Validation")

if st.session_state.generation_timings:
    st.markdown("### Generation Timings")

    st.code(
        "\n".join(
            st.session_state.generation_timings,
        )
    )

if st.session_state.validation_messages:
    st.markdown("### Validation Messages")

    for message in st.session_state.validation_messages:
        st.warning(
            message,
        )
else:
    st.success(
        "No warnings or validation messages.",
    )

def main() -> None:
    st.set_page_config(
        page_title="IDMS DB2 Modernizer",
        layout="wide",
    )

    initialize_directories()
    initialize_session_state()

    st.title(
        "IDMS > DB2 Modernizer",
    )

    st.caption(
        "Generate DB2 DDL, Phase 2 metadata, canonical JSON, ER diagram, "
        "converted DB2 COBOL, and Excel-style sheet mapping from uploaded PDFs."
    )

    tabs = st.tabs(
        [
            "Main",
            "Metadata Overview",
            "DB2 DDL",
            "Phase 2 Metadata",
            "Excel Sheet Mapping",
            "ER Diagram",
            "Converted DB2 COBOL",
            "Validation",
        ]
    )

    with tabs[0]:
        render_main_tab()

    with tabs[1]:
        render_metadata_overview_tab()

    with tabs[2]:
        render_ddl_tab()

    with tabs[3]:
        render_phase2_metadata_tab()

    with tabs[4]:
        render_excel_sheet_mapping_tab()

```

```

with tabs[5]:
    render_er_diagram_tab()

with tabs[6]:
    render_converted_cobol_tab()

with tabs[7]:
    render_validation_tab()

if __name__ == "__main__":
    main()

```

```

=====
FILE: src\idms_modernizer\core\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_modernizer\core\config.py
ENCODING: utf-8
=====

```

```

from pathlib import Path

from pydantic_settings import BaseSettings

```

```

class Settings(BaseSettings):

    APP_NAME: str = (
        "IDMS DB2 Modernizer"
    )

    BASE_DIR: Path = Path.cwd()

    INPUT_DIR: str = "data/input"

    OUTPUT_DIR: str = "data/output"

    TEMP_DIR: str = "data/temp"

    DEBUG: bool = True

    @property
    def input_dir(self) -> str:
        return self.INPUT_DIR

    @property
    def output_dir(self) -> str:
        return self.OUTPUT_DIR

    @property
    def temp_dir(self) -> str:
        return self.TEMP_DIR

    class Config:
        env_file = ".env"

```

```

settings = Settings()

```

```

=====
FILE: src\idms_modernizer\core\constants.py
ENCODING: utf-8
=====

```

```

SUPPORTED_FILE_TYPES = [
    "pdf"
]

```

```

MAX_UPLOAD_SIZE_MB = 100

DB2_DDL_FILE = (
    "db2_ddl.sql"
)

ER_DIAGRAM_FILE = (
    "er_diagram.png"
)

EXCEL_MODEL_FILE = (
    "db2_model.xlsx"
)

REPORT_FILE = (
    "migration_report.pdf"
)

SUPPORTED_FILE_TYPES = [
    "pdf"
]

MAX_UPLOAD_SIZE_MB = 100

DB2_DDL_FILE = (
    "db2_ddl.sql"
)

ER_DIAGRAM_FILE = (
    "er_diagram.png"
)

EXCEL_MODEL_FILE = (
    "db2_model.xlsx"
)

REPORT_FILE = (
    "migration_report.pdf"
)

CANONICAL_MODEL_FILE = (
    "canonical_model.json"
)

PHASE2_METADATA_FILE = (
    "phase2_metadata.json"
)

```

```

=====
FILE: src\idms_modernizer\core\exceptions.py
ENCODING: utf-8
=====

```

```

class ModernizerException(
    Exception
):
    pass

class PdfExtractionException(
    ModernizerException
):
    pass

class SchemaParsingException(
    ModernizerException
):
    pass

class ValidationException(
    ModernizerException
):
    pass

```



```
=====
FILE: src\idms_modernizer\core\logger.py
ENCODING: utf-8
=====
```

```
import logging

def get_logger(
    logger_name: str
) -> logging.Logger:

    logger = logging.getLogger(
        logger_name
    )

    logger.setLevel(
        logging.INFO
    )

    if not logger.handlers:

        handler = (
            logging.StreamHandler()
        )

        formatter = (
            logging.Formatter(
                "%(asctime)s | %(levelname)s | %(message)s"
            )
        )

        handler.setFormatter(
            formatter
        )

        logger.addHandler(
            handler
        )

    return logger
```

```
=====
FILE: src\idms_modernizer\core\utils.py
ENCODING: utf-8
=====
```

```
from pathlib import Path
```

```
def ensure_directory(
    directory: str
) -> None:

    Path(
        directory
    ).mkdir(
        parents=True,
        exist_ok=True
    )
```

```
def normalize_text(
    text: str
) -> str:

    return " ".join(
        text.split()
    )
```

```
=====
FILE: src\idms_modernizer\domain\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_modernizer\domain\canonical_models.py
ENCODING: utf-8
=====
```

```
from pydantic import BaseModel, Field
```

```
class CanonicalField(BaseModel):
    name: str
    datatype: str | None = None
    length: int | None = None
    scale: int | None = None
    occurs: bool = False
    occurs_max: int | None = None
```

```
class CanonicalRecord(BaseModel):
    name: str

    # Backward-compatible single PK.
    primary_key: str | None = None

    # Composite PK support.
    # When CALC points to a group, physical child fields are stored here.
    primary_keys: list[str] = Field(default_factory=list)

    record_id: str | None = None
    location_mode: str | None = None
    fields: list[CanonicalField] = Field(default_factory=list)
```

```
class CanonicalSet(BaseModel):
    name: str
    owner_record: str | None = None
    member_record: str | None = None
    order_mode: str | None = None
```

```
class CanonicalRelationship(BaseModel):
    set_name: str
    parent_record: str
    child_record: str
    cardinality: str = "1:N"
```

```
class CanonicalSchema(BaseModel):
    records: list[CanonicalRecord] = Field(default_factory=list)
    sets: list[CanonicalSet] = Field(default_factory=list)
    relationships: list[CanonicalRelationship] = Field(default_factory=list)
```

```
=====
FILE: src\idms_modernizer\domain\db2_models.py
ENCODING: utf-8
=====
```

```
from dataclasses import dataclass, field
```

```
@dataclass
class DB2Column:
    name: str
    datatype: str
    nullable: bool = True
    primary_key: bool = False
    generated: bool = False
    source_kind: str = ""
```

```

@dataclass
class DB2ForeignKey:
    column_name: str
    reference_table: str
    reference_column: str
    set_name: str = ""

@dataclass
class DB2Table:
    name: str
    columns: list[DB2Column] = field(default_factory=list)
    foreign_keys: list[DB2ForeignKey] = field(default_factory=list)
    primary_key: str | None = None
    primary_keys: list[str] = field(default_factory=list)

```

```

@dataclass
class DB2Model:
    tables: list[DB2Table] = field(default_factory=list)

```

```

=====
FILE: src\idms_modernizer\domain\document_models.py
ENCODING: utf-8
=====

```

```

from pydantic import BaseModel, Field

```

```

class DocumentLine(BaseModel):
    line_number: int
    text: str

```

```

class DocumentPage(BaseModel):
    page_number: int
    lines: list[DocumentLine] = Field(
        default_factory=list
    )

```

```

class DocumentModel(BaseModel):
    pages: list[DocumentPage] = Field(
        default_factory=list
    )

```

```

=====
FILE: src\idms_modernizer\domain\models.py
ENCODING: utf-8
=====

```

```

from pydantic import BaseModel

```

```

from idms_modernizer.domain.db2_models import Table
from idms_modernizer.domain.relationship_models import (
    Relationship
)
from idms_modernizer.domain.schema_models import (
    SchemaMetadata
)

```

```

class ConversionContext(
    BaseModel
):

    schema_metadata: (
        SchemaMetadata | None
    ) = None

    relationships: (
        list[Relationship]
    )

```

```
) = []
```

```
db2_tables: (  
    list[Table]  
) = []
```

```
=====
```

FILE: src\idms_modernizer\domain\relationship_models.py
ENCODING: utf-8

```
=====
```

```
from pydantic import BaseModel
```

```
class SetMembership(BaseModel):
```

```
    set_name: str
```

```
    role: str
```

```
class Relationship(BaseModel):
```

```
    set_name: str
```

```
    owner_record: str
```

```
    member_record: str
```

```
    cardinality: str = "1:N"
```

```
=====
```

FILE: src\idms_modernizer\domain\schema_models.py
ENCODING: utf-8

```
=====
```

```
from dataclasses import dataclass, field
```

```
from pydantic import BaseModel, Field
```

```
from idms_modernizer.domain.relationship_models import (  
    Relationship,  
    SetMembership,  
)
```

```
class DataField(BaseModel):
```

```
    name: str
```

```
    level: int | None = None
```

```
    datatype: str | None = None
```

```
    length: int | None = None
```

```
    scale: int | None = None
```

```
    picture: str | None = None
```

```
    start_position: int | None = None
```

```
    end_position: int | None = None
```

```
    basetype: str | None = None
```

```
    has_child: bool = False
```

```
    is_group: bool = False
```

```
    occurs: bool = False
```

```
    occurs_min: int | None = None
```

```
    occurs_max: int | None = None
```

```
    raw_line: str | None = None
```

```
    rest: str | None = None
```

```
@dataclass
```

```
class Record:
```

```
    name: str
```

```

# Physical / DDL-safe fields.
# Used by canonical schema, DB2 model, DDL, Phase 2 metadata, and COBOL conversion.
fields: list[DataField] = field(
    default_factory=list,
)

# Excel-only complete field list.
# Includes group fields, leaf fields, outer date groups, key groups, attributes, OCCURS groups.
mapping_fields: list[DataField] = field(
    default_factory=list,
)

set_memberships: list[SetMembership] = field(
    default_factory=list,
)

primary_key: str | None = None
cobol_zone: str | None = None

```

```

class SetDefinition(BaseModel):
    name: str
    owner_record: str | None = None
    member_record: str | None = None

```

```

class SchemaMetadata(BaseModel):
    records: list[Record] = Field(
        default_factory=list,
    )

    sets: list[SetDefinition] = Field(
        default_factory=list,
    )

    relationships: list[Relationship] = Field(
        default_factory=list,
    )

```

```

=====
FILE: src\idms_modernizer\domain\section_models.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field

```

```

@dataclass
class RecordSection:
    record_name: str
    lines: list[str] = field(
        default_factory=list
    )

```

```

=====
FILE: src\idms_modernizer\extractors\cobol_zone_extractor.py
ENCODING: utf-8
=====

```

```

import re

```

```

class CobolZoneExtractor:
    """
    Extracts the IDMS region / zone from a schema record section.

    Examples:
        WITHIN INS-DEMO-REGION OFFSET 5 PGS FOR 20 PGS
        WITHIN
        EMP-DEMO-REGION OFFSET 5 PGS FOR 45 PGS

    Output:
        INS-DEMO-REGION
        EMP-DEMO-REGION
    """

```

```
"""
```

```
DIRECT_WITHIN_PATTERN = re.compile(
    r"\bWITHIN\s+([A-Z][A-Z0-9-]*REGION)\b",
    re.IGNORECASE,
)
```

```
REGION_PATTERN = re.compile(
    r"\b([A-Z][A-Z0-9-]*REGION)\b",
    re.IGNORECASE,
)
```

```
def extract(
    self,
    lines: list[str],
) -> str | None:
    cleaned_lines = [
        self.clean_line(line)
        for line in lines
        if line and self.clean_line(line)
    ]

    if not cleaned_lines:
        return None

    joined_text = " ".join(cleaned_lines)

    direct_match = self.DIRECT_WITHIN_PATTERN.search(
        joined_text,
    )

    if direct_match:
        return direct_match.group(1).upper()

    for index, line in enumerate(cleaned_lines):
        upper_line = line.upper()

        if upper_line == "WITHIN":
            region = self.find_region_after_within(
                lines=cleaned_lines,
                start_index=index + 1,
            )

            if region:
                return region

        if "WITHIN" in upper_line:
            region = self.find_region_in_line(
                line=line,
            )

            if region:
                return region

            region = self.find_region_after_within(
                lines=cleaned_lines,
                start_index=index + 1,
            )

            if region:
                return region

    return None

def find_region_after_within(
    self,
    lines: list[str],
    start_index: int,
) -> str | None:
    for line in lines[start_index : start_index + 5]:
        region = self.find_region_in_line(
            line=line,
        )

        if region:
            return region
```

```

        return None

    def find_region_in_line(
        self,
        line: str,
    ) -> str | None:
        match = self.REGION_PATTERN.search(
            line,
        )

        if not match:
            return None

        return match.group(1).upper()

    def clean_line(
        self,
        line: str,
    ) -> str:
        if line is None:
            return ""

        cleaned = line.strip()
        cleaned = cleaned.replace("\u00a0", " ")
        cleaned = re.sub(r"\s+", " ", cleaned)

        return cleaned

```

```

=====
FILE: src\idms_modernizer\extractors\field_extractor.py
ENCODING: utf-8
=====

```

```

import re

from dataclasses import dataclass
from typing import List

from idms_modernizer.domain.schema_models import DataField

@dataclass
class FieldCandidate:
    level: int
    name: str
    rest: str
    original_line: str
    has_child: bool = False
    is_group: bool = False
    occurs: bool = False
    occurs_min: int | None = None
    occurs_max: int | None = None

class FieldExtractor:
    """
    Extracts IDMS schema fields.

    Generic behavior only:
    - No business field names are hardcoded.
    - No record names are hardcoded.
    - Physical extraction and Sheet Mapping extraction are separated.

    extract():
    - Used for physical / DDL-safe fields.
    - Returns only leaf fields.
    - Excludes FILLER by default because FILLER should not become a DB2 column.

    extract_all():
    - Used for Excel Sheet Mapping.
    - Returns all fields:
      groups, subgroups, leaves, date groups, date parts, OCCURS groups, and FILLER.
    - Includes FILLER by default so Sheet Mapping can show FILLER rows.
    """

```

```

FIELD_NAME_PATTERN = re.compile(
    r"^\s*(?P<level>0[1-9]|[1-4][0-9]|88)\s+"
    r"(?P<name>[A-Z][A-Z0-9-]*|FILLER)\b"
    r"(?P<rest>.*)$",
    re.IGNORECASE,
)

OCCURS_PATTERN = re.compile(
    r"\bOCCURS\s+"
    r"(?P<min>[0-9]+)"
    r"(?:\s+TO\s+(?P<max>[0-9]+))?",
    re.IGNORECASE,
)

def __init__(
    self,
    include_filler: bool = False,
    auto_number_filler: bool = False,
) -> None:
    self.include_filler = include_filler
    self.auto_number_filler = auto_number_filler

def extract(
    self,
    lines: List[str],
) -> List[DataField]:
    """
    Physical / DDL-safe extraction.

    FILLER is excluded here because physical DB2 columns should not be created
    from COBOL FILLER.
    """

    print("USING FIELD EXTRACTOR VERSION LEAF-ONLY-WITH-LEVEL")

    candidates = self.discover_candidates(
        lines=lines,
        include_filler=False,
    )

    leaf_candidates = self.mark_and_filter_leaf_fields(
        candidates=candidates,
    )

    return self.candidates_to_fields(
        candidates=leaf_candidates,
    )

def extract_all(
    self,
    lines: List[str],
    include_filler: bool = True,
) -> List[DataField]:
    """
    Excel Sheet Mapping extraction.

    FILLER is included here so Sheet Mapping can show FILLER rows.
    """

    print("USING FIELD EXTRACTOR VERSION ALL-FIELDS-FOR-MAPPING")

    candidates = self.discover_candidates(
        lines=lines,
        include_filler=include_filler,
    )

    self.mark_group_fields(
        candidates=candidates,
    )

    return self.candidates_to_fields(
        candidates=candidates,
    )

def discover_candidates(

```



```

self,
lines: List[str],
include_filler: bool | None = None,
) -> List[FieldCandidate]:
    candidates: List[FieldCandidate] = []

    if include_filler is None:
        include_filler = self.include_filler

    filler_index = 0

    for line in lines:
        cleaned_line = self.clean_line(
            line=line,
        )

        if not cleaned_line:
            continue

        match = self.FIELD_NAME_PATTERN.match(
            cleaned_line,
        )

        if not match:
            continue

        level = int(
            match.group("level"),
        )

        name = match.group("name").upper()
        rest = match.group("rest").strip()

        if level == 88:
            continue

        if name == "FILLER":
            if not include_filler:
                continue

            filler_index += 1

            candidate_name = name

            if self.auto_number_filler:
                candidate_name = f"FILLER_{filler_index}"
            else:
                candidate_name = name

        occurs_info = self.parse_occurs(
            text=rest,
        )

        candidate = FieldCandidate(
            level=level,
            name=candidate_name,
            rest=rest,
            original_line=cleaned_line,
            occurs=occurs_info["occurs"],
            occurs_min=occurs_info["occurs_min"],
            occurs_max=occurs_info["occurs_max"],
        )

        candidates.append(candidate)

    return candidates

def mark_group_fields(
    self,
    candidates: List[FieldCandidate],
) -> None:
    for index, candidate in enumerate(candidates):
        candidate.has_child = False
        candidate.is_group = False

        for later_candidate in candidates[index + 1:]:

```

```

        if later_candidate.level <= candidate.level:
            break

        if later_candidate.level > candidate.level:
            candidate.has_child = True
            candidate.is_group = True
            break

def mark_and_filter_leaf_fields(
    self,
    candidates: List[FieldCandidate],
) -> List[FieldCandidate]:
    self.mark_group_fields(
        candidates=candidates,
    )

    leaf_candidates: List[FieldCandidate] = []

    for candidate in candidates:
        if candidate.has_child:
            continue

        leaf_candidates.append(candidate)

    return leaf_candidates

def candidates_to_fields(
    self,
    candidates: List[FieldCandidate],
) -> List[DataField]:
    fields: List[DataField] = []

    for candidate in candidates:
        display_name = candidate.name

        if display_name.startswith("FILLER_"):
            display_name = "FILLER"

        field = DataField(
            name=display_name,
            level=candidate.level,
            rest=candidate.rest,
            raw_line=candidate.original_line,
            has_child=candidate.has_child,
            is_group=candidate.is_group,
            occurs=candidate.occurs,
            occurs_min=candidate.occurs_min,
            occurs_max=candidate.occurs_max,
        )

        fields.append(field)

    return fields

def parse_occurs(
    self,
    text: str,
) -> dict:
    match = self.OCCURS_PATTERN.search(
        text or "",
    )

    if not match:
        return {
            "occurs": False,
            "occurs_min": None,
            "occurs_max": None,
        }

    occurs_min = self.safe_int(
        value=match.group("min"),
    )

    occurs_max_raw = match.group("max")

    occurs_max = (

```

```

        self.safe_int(
            value=occurs_max_raw,
        )
        if occurs_max_raw is not None
        else occurs_min
    )

    return {
        "occurs": True,
        "occurs_min": occurs_min,
        "occurs_max": occurs_max,
    }

def clean_line(
    self,
    line: str,
) -> str:
    value = str(line or "").strip()

    if not value:
        return ""

    value = value.replace("\u00a0", " ")
    value = re.sub(r"\s+", " ", value)

    return value.strip()

def safe_int(
    self,
    value,
) -> int | None:
    try:
        if value is None:
            return None

        return int(value)
    except Exception:
        return None

```

```

=====
FILE: src\idms_modernizer\extractors\owner_member_extractor.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_modernizer.domain.relationship_models import (
    SetMembership
)

```

```
class OwnerMemberExtractor:
```

```

    SAME_LINE_PATTERN = re.compile(
        r"^\\s*([A-Z0-9\\-]+)\\s+(?:INDEX\\s+)?(OWNER|MEMBER)\\b",
        re.IGNORECASE
    )

    SET_NAME_PATTERN = re.compile(
        r"^\\s*([A-Z][A-Z0-9\\-]+)\\s*$",
        re.IGNORECASE
    )

    ROLE_PATTERN = re.compile(
        r"^\\s*(OWNER|MEMBER)\\b",
        re.IGNORECASE
    )

    IGNORED_SET_NAMES = {
        "CALC"
    }

    HEADER_WORDS = {
        "SET",
        "TYPE",
    }

```

```

        "NEXT",
        "PRIOR",
        "OWNER",
        "MEMBER"
    }

def extract(
    self,
    record_name: str,
    lines: list[str]
) -> list[SetMembership]:

    memberships = []
    seen = set()

    in_dbkey_block = False
    pending_set_name = None

    for raw_line in lines:

        line = raw_line.strip().upper()

        if not line:
            continue

        if "DBKEY POSITIONS" in line:
            in_dbkey_block = True
            pending_set_name = None
            continue

        if in_dbkey_block and line.startswith("DATA ITEM"):
            break

        if not in_dbkey_block:
            continue

        same_line_match = self.SAME_LINE_PATTERN.search(
            line
        )

        if same_line_match:

            set_name = same_line_match.group(1).upper()
            role = same_line_match.group(2).upper()

            pending_set_name = None

            if set_name in self.IGNORED_SET_NAMES:
                continue

            key = (
                set_name,
                role
            )

            if key not in seen:
                seen.add(key)

                memberships.append(
                    SetMembership(
                        set_name=set_name,
                        role=role
                    )
                )

            continue

        role_match = self.ROLE_PATTERN.search(
            line
        )

        if role_match and pending_set_name:

            set_name = pending_set_name
            role = role_match.group(1).upper()

```

```

        pending_set_name = None

        if set_name in self.IGNORED_SET_NAMES:
            continue

        key = (
            set_name,
            role
        )

        if key not in seen:
            seen.add(key)

            memberships.append(
                SetMembership(
                    set_name=set_name,
                    role=role
                )
            )

        continue

    set_name_match = self.SET_NAME_PATTERN.search(
        line
    )

    if set_name_match:

        candidate = set_name_match.group(1).upper()

        if candidate in self.HEADER_WORDS:
            continue

        if candidate in self.IGNORED_SET_NAMES:
            pending_set_name = None
            continue

        pending_set_name = candidate

    print(
        f"{record_name}: "
        f"{len(memberships)} memberships"
    )

    for membership in memberships:
        print(
            f"{record_name} -> "
            f"{membership.set_name} "
            f"{membership.role}"
        )

    return memberships

```

```

=====
FILE: src\idms_modernizer\extractors\primary_key_extractor.py
ENCODING: utf-8
=====

```

```
import re
```

```
from idms_modernizer.services.name_normalizer import NameNormalizer
```

```
class PrimaryKeyExtractor:
```

```
    """
```

```
    Extracts IDMS CALC primary key metadata.
```

```
    Generic behavior only:
```

- No hardcoded record names.
- No hardcoded key names.
- Handles listing formats with dots, spaces, and split lines.

```
    Supported examples:
```

- LOCATION MODE CALC USING EMP-ID-0415
- LOCATION MODE..... CALC USING KY-VMBFC DUPLICATES NOT ALLOWED

```

- CALC USING KY-VMBFC
- SET CONTROL ITEM FOR CALC <start> <length>
"""

CALC_PATTERN = re.compile(
    r"\bCALC\s+USING\s+([A-Z0-9-])\b",
    re.IGNORECASE,
)

CONTROL_ITEM_PATTERN = re.compile(
    r"\bSET\s+CONTROL\s+ITEM\s+FOR\s+CALC"
    r"(?:\s+(?P<start>[0-9]+)\s+(?P<length>[0-9]+))?",
    re.IGNORECASE,
)

CONTROL_ITEM_FOLLOWING_CALC_PATTERN = re.compile(
    r"\bCALC\s+(?P<start>[0-9]+)\s+(?P<length>[0-9]+)\b",
    re.IGNORECASE,
)

def extract(
    self,
    lines: list[str],
) -> str | None:
    cleaned_lines = [
        self.clean_line(line=line)
        for line in lines or []
    ]

    joined_text = " ".join(
        line
        for line in cleaned_lines
        if line
    )

    match = self.CALC_PATTERN.search(joined_text)

    if match:
        return NameNormalizer.normalize(
            match.group(1),
        )

    for line in cleaned_lines:
        match = self.CALC_PATTERN.search(line)

        if match:
            return NameNormalizer.normalize(
                match.group(1),
            )

    return None

def extract_control_item(
    self,
    lines: list[str],
) -> dict[str, int] | None:
    cleaned_lines = [
        self.clean_line(line=line)
        for line in lines or []
        if self.clean_line(line=line)
    ]

    joined_text = " ".join(cleaned_lines)

    match = self.CONTROL_ITEM_PATTERN.search(joined_text)

    if match:
        start = match.group("start")
        length = match.group("length")

        if start and length:
            return {
                "start": int(start),
                "length": int(length),
            }

```

```

match = self.CONTROL_ITEM_FOLLOWING_CALC_PATTERN.search(joined_text)

if match:
    return {
        "start": int(match.group("start")),
        "length": int(match.group("length")),
    }

return None

def clean_line(
    self,
    line: str,
) -> str:
    value = str(line or "").upper()

    value = value.replace(".", " ")
    value = value.replace("_", " ")
    value = value.replace(":", " ")
    value = value.replace("\u00a0", " ")

    value = re.sub(
        r"\s+",
        " ",
        value,
    )

    return value.strip()

```

```

=====
FILE: src\idms_modernizer\generators\ddl_generator.py
ENCODING: utf-8
=====

```

```

class DDLGenerator:
    """
    Generates DB2 DDL.

    Supports:
    - Single primary key
    - Composite primary key through table.primary_keys
    - Multiple FK columns
    - FK constraints emitted after CREATE TABLE
    """

    def generate(
        self,
        db2_model,
    ) -> str:
        ddl: list[str] = []

        for table in getattr(db2_model, "tables", []) or []:
            ddl.extend(
                self.generate_create_table(
                    table=table,
                )
            )

        foreign_key_statements = self.generate_foreign_key_statements(
            db2_model=db2_model,
        )

        if foreign_key_statements:
            ddl.append("")
            ddl.extend(foreign_key_statements)

        return "\n".join(ddl)

    def generate_create_table(
        self,
        table,
    ) -> list[str]:
        lines: list[str] = []

        table_name = getattr(table, "name", "")

```

```

lines.append(f"CREATE TABLE {table_name}")
lines.append("(")

definitions: list[str] = []
added_columns: set[str] = set()

for column in getattr(table, "columns", []) or []:
    column_name = getattr(column, "name", "")

    if not column_name:
        continue

    if column_name in added_columns:
        continue

    datatype = getattr(column, "datatype", "")

    definition = f"    {column_name} {datatype}"

    if not getattr(column, "nullable", True):
        definition += " NOT NULL"

    definitions.append(definition)
    added_columns.add(column_name)

primary_keys = self.primary_key_columns(
    table=table,
)

if primary_keys:
    definitions.append(
        f"    PRIMARY KEY ({', '.join(primary_keys)})"
    )

lines.append(",\n".join(definitions))
lines.append(");")
lines.append("")

return lines

def primary_key_columns(
    self,
    table,
) -> list[str]:
    primary_keys = list(getattr(table, "primary_keys", []) or [])

    if not primary_keys:
        single_primary_key = getattr(table, "primary_key", None)

        if single_primary_key:
            primary_keys = [single_primary_key]

    if not primary_keys:
        primary_keys = [
            getattr(column, "name", "")
            for column in getattr(table, "columns", []) or []
            if getattr(column, "primary_key", False)
        ]

    cleaned_primary_keys = []

    for primary_key in primary_keys:
        if not primary_key:
            continue

        if primary_key in cleaned_primary_keys:
            continue

        cleaned_primary_keys.append(primary_key)

    return cleaned_primary_keys

def generate_foreign_key_statements(
    self,
    db2_model,

```



```

) -> list[str]:
    statements: list[str] = []
    added_foreign_keys: set[tuple[str, str, str, str]] = set()

    for table in getattr(db2_model, "tables", []) or []:
        table_name = getattr(table, "name", "")

        if not table_name:
            continue

        for foreign_key in getattr(table, "foreign_keys", []) or []:
            child_column = self.get_foreign_key_column_name(
                foreign_key=foreign_key,
            )

            parent_table = self.get_foreign_key_reference_table(
                foreign_key=foreign_key,
            )

            parent_column = self.get_foreign_key_reference_column(
                foreign_key=foreign_key,
            )

            if not child_column or not parent_table or not parent_column:
                continue

            fk_key = (
                table_name,
                child_column,
                parent_table,
                parent_column,
            )

            if fk_key in added_foreign_keys:
                continue

            added_foreign_keys.add(fk_key)

            constraint_name = self.constraint_name(
                table_name=table_name,
                column_name=child_column,
            )

            statements.append(
                f"ALTER TABLE {table_name} "
                f"ADD CONSTRAINT {constraint_name} "
                f"FOREIGN KEY ({child_column}) "
                f"REFERENCES {parent_table} ({parent_column});"
            )

        return statements

def get_foreign_key_column_name(
    self,
    foreign_key,
) -> str:
    return (
        getattr(foreign_key, "column_name", None)
        or getattr(foreign_key, "child_column", None)
        or getattr(foreign_key, "child_fk", None)
        or getattr(foreign_key, "foreign_key", None)
        or ""
    )

def get_foreign_key_reference_table(
    self,
    foreign_key,
) -> str:
    return (
        getattr(foreign_key, "reference_table", None)
        or getattr(foreign_key, "parent_table", None)
        or getattr(foreign_key, "referenced_table", None)
        or ""
    )

def get_foreign_key_reference_column(

```

```

        self,
        foreign_key,
    ) -> str:
        return (
            getattr(foreign_key, "reference_column", None)
            or getattr(foreign_key, "parent_column", None)
            or getattr(foreign_key, "referenced_column", None)
            or getattr(foreign_key, "parent_key", None)
            or ""
        )

    def constraint_name(
        self,
        table_name: str,
        column_name: str,
    ) -> str:
        raw_name = f"FK_{table_name}_{column_name}"

        cleaned = (
            raw_name
            .replace("-", "_")
            .replace(" ", "_")
            .replace(".", "_")
            .upper()
        )

        return cleaned[:128]

```

```

=====
FILE: src\idms_modernizer\graph\relationship_resolver.py
ENCODING: utf-8
=====

```

```

from collections import defaultdict

from idms_modernizer.domain.relationship_models import (
    Relationship
)

```

```

class RelationshipResolver:

    def resolve(
        self,
        metadata
    ) -> list[Relationship]:

        owner_map = defaultdict(list)

        member_map = defaultdict(list)

        for record in metadata.records:

            for membership in record.set_memberships:

                if membership.role == "OWNER":

                    owner_map[
                        membership.set_name
                    ].append(
                        record.name
                    )

                elif membership.role == "MEMBER":

                    member_map[
                        membership.set_name
                    ].append(
                        record.name
                    )

        relationships = []

        all_sets = (
            set(owner_map.keys())

```

```

        | set(member_map.keys())
    )

    for set_name in all_sets:

        owners = owner_map.get(
            set_name,
            []
        )

        members = member_map.get(
            set_name,
            []
        )

        for owner in owners:

            for member in members:

                relationships.append(
                    Relationship(
                        set_name=set_name,
                        owner_record=owner,
                        member_record=member
                    )
                )

    return relationships

```

```

=====
FILE: src\idms_modernizer\graph\schema_graph.py
ENCODING: utf-8
=====

```

```
import networkx as nx
```

```
class SchemaGraph:
```

```

    def __init__(self):

        self.graph = nx.DiGraph()

    def add_record(
        self,
        record_name: str
    ):

        self.graph.add_node(
            record_name,
            node_type="record"
        )

    def add_relationship(
        self,
        owner: str,
        member: str,
        set_name: str
    ):

        self.graph.add_edge(
            owner,
            member,
            set_name=set_name
        )

```

```

=====
FILE: src\idms_modernizer\parsers\__init__.py
ENCODING: utf-8
=====

```

```
[Empty file]
```

```
=====
FILE: src\idms_modernizer\parsers\cobol_pdf_extractor.py
ENCODING: utf-8
=====
```

```
import re
import pdfplumber

class CobolPdfExtractor:
    """
    Extracts COBOL text from PDF.

    Supports:
    - COBOL pasted as text in PDF
    - line-preserving extraction
    - fallback to empty string if extraction fails
    """

    def extract_text(
        self,
        pdf_path: str
    ) -> str:

        lines: list[str] = []

        with pdfplumber.open(pdf_path) as pdf:
            for page in pdf.pages:
                page_text = (
                    page.extract_text()
                    or ""
                )

                for raw_line in page_text.splitlines():
                    cleaned_line = self._clean_line(
                        raw_line
                    )

                    lines.append(
                        cleaned_line
                    )

        return "\n".join(lines).strip()

    def _clean_line(
        self,
        line: str | None
    ) -> str:

        if line is None:
            return ""

        cleaned = str(line).rstrip()

        cleaned = cleaned.replace(
            "\t",
            " "
        )

        cleaned = re.sub(
            r"[\u00A0]+",
            " ",
            cleaned
        )

        return cleaned
```

```
=====
FILE: src\idms_modernizer\parsers\text_extractor.py
ENCODING: utf-8
=====
```

```
import re
import pdfplumber
```

```

from idms_modernizer.domain.document_models import (
    DocumentLine,
    DocumentModel,
    DocumentPage
)

class TextExtractor:
    """
    Plain text PDF extractor.

    Important:
    - Uses only page.extract_text().
    - Does not use extract_tables().
    - Does not use extract_words().
    - This keeps the useful rows seen in your debug, such as:
      03 EMP-FIRST-NAME-0415 DISPLAY X(10) 5 10
    """

    def extract_document(
        self,
        pdf_path: str
    ) -> DocumentModel:

        pages: list[DocumentPage] = []

        with pdfplumber.open(pdf_path) as pdf:
            for page_index, page in enumerate(pdf.pages):
                text = (
                    page.extract_text()
                    or ""
                )

                lines: list[DocumentLine] = []

                for line_index, raw_line in enumerate(
                    text.splitlines()
                ):
                    cleaned_line = self._clean_line(
                        raw_line
                    )

                    if not cleaned_line:
                        continue

                    lines.append(
                        DocumentLine(
                            line_number=line_index,
                            text=cleaned_line
                        )
                    )

                pages.append(
                    DocumentPage(
                        page_number=page_index,
                        lines=lines
                    )
                )

            return DocumentModel(
                pages=pages
            )

    def _clean_line(
        self,
        line: str | None
    ) -> str:

        if not line:
            return ""

        cleaned = str(line).strip()

        cleaned = re.sub(
            r"<[^>]+>",
            " ",

```

```

        cleaned
    )

    cleaned = cleaned.replace(
        "&nbsp;",
        " "
    )

    cleaned = cleaned.replace(
        "\t",
        " "
    )

    cleaned = cleaned.replace(
        "|",
        " "
    )

    cleaned = re.sub(
        r"\s+",
        " ",
        cleaned
    )

    return cleaned.strip()

```

```

=====
FILE: src\idms_modernizer\services\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_modernizer\services\business_datatype_inference.py
ENCODING: utf-8
=====

```

```

class BusinessDatatypeInference:
    """
    Generic datatype inference.

    This class intentionally does not inspect field names.

    Reason:
    Field-name based rules such as PHONE, ZIP, SSN, CODE, ACCOUNT,
    POLICY, or LICENSE are business/domain assumptions and should not be
    hardcoded.

    Datatype should be decided upstream from:
    - IDMS PIC clause
    - USAGE
    - length
    - scale
    """

    @staticmethod
    def infer(
        field_name: str,
        current_type: str,
    ) -> str:
        if current_type:
            return current_type.upper()

        return "VARCHAR"

```

```

=====
FILE: src\idms_modernizer\services\canonical_reconciler.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.canonical_models import (
    CanonicalSchema,

```

```

        CanonicalField
    )
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer

class CanonicalReconciler:
    """
    Reconciles canonical schema with IDMS schema metadata.

    IDMS schema is authoritative for:
    - datatype
    - length
    - scale
    - picture

    Canonical schema is authoritative for:
    - normalized record names
    - primary keys
    - relationships
    """

    def reconcile(
        self,
        canonical_schema: CanonicalSchema,
        metadata: SchemaMetadata
    ) -> CanonicalSchema:

        idms_field_lookup = self._build_idms_field_lookup(
            metadata
        )

        for record in canonical_schema.records:
            normalized_record_name = NameNormalizer.normalize(
                record.name
            )

            reconciled_fields: list[CanonicalField] = []

            for field in record.fields:
                normalized_field_name = NameNormalizer.normalize(
                    field.name
                )

                idms_field = idms_field_lookup.get(
                    (
                        normalized_record_name,
                        normalized_field_name
                    )
                )

                if idms_field is None:
                    idms_field = idms_field_lookup.get(
                        (
                            normalized_record_name,
                            self._remove_record_suffix(
                                normalized_field_name
                            )
                        )
                    )

                if idms_field is None:
                    reconciled_fields.append(field)
                    continue

                reconciled_fields.append(
                    CanonicalField(
                        name=field.name,
                        datatype=idms_field.datatype,
                        length=idms_field.length,
                        scale=idms_field.scale,
                        occurs=field.occurs,
                        occurs_max=field.occurs_max
                    )
                )
            )

```

```

        record.fields = reconciled_fields

    return canonical_schema

def _build_idms_field_lookup(
    self,
    metadata: SchemaMetadata
) -> dict[tuple[str, str], object]:

    lookup: dict[tuple[str, str], object] = {}

    for record in metadata.records:
        record_name = NameNormalizer.normalize(
            record.name
        )

        for field in record.fields:
            field_name = NameNormalizer.normalize(
                field.name
            )

            lookup[
                (
                    record_name,
                    field_name
                )
            ] = field

            lookup[
                (
                    record_name,
                    self._remove_record_suffix(field_name)
                )
            ] = field

    return lookup

def _remove_record_suffix(
    self,
    field_name: str
) -> str:

    parts = field_name.split("_")

    if (
        len(parts) > 1
        and parts[-1].isdigit()
        and len(parts[-1]) == 4
    ):
        return "_".join(parts[:-1])

    return field_name

```

```

=====
FILE: src\idms_modernizer\services\canonical_schema_builder.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.domain.canonical_models import (
    CanonicalSchema,
    CanonicalRecord,
    CanonicalField,
    CanonicalSet,
    CanonicalRelationship,
)
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.services.date_field_consolidator import DateFieldConsolidator

class CanonicalSchemaBuilder:
    """
    Builds canonical schema from IDMS metadata.

```


Generic behavior only:

- No hardcoded record names.
- No hardcoded column names.
- No hardcoded SET names.

Rules:

- Physical DB2 columns come from record.fields, after date consolidation.
 - Group detection uses mapping_fields level hierarchy.
 - CALC outer group is never a physical DB2 PK column.
 - CALC intermediate groups are not physical DB2 columns.
 - CALC physical non-date leaf children become actual composite PK columns.
 - Date group/date parts are never physical PK.
 - FILLER is never physical PK.
 - If no CALC exists, DB2MappingService creates ID_RECORD_<record_name> CHAR(20).
- """

```
def build(self, metadata) -> CanonicalSchema:
    schema = CanonicalSchema()

    for record in getattr(metadata, "records", []) or []:
        record_name = NameNormalizer.normalize(getattr(record, "name", "") or "")

        primary_key = (
            NameNormalizer.normalize(getattr(record, "primary_key", "") or "")
            if getattr(record, "primary_key", None)
            else None
        )

        canonical_record = CanonicalRecord(
            name=record_name,
            primary_key=primary_key,
            primary_keys=[],
        )

        mapping_fields = (
            getattr(record, "mapping_fields", None)
            or getattr(record, "fields", None)
            or []
        )

        group_names = self.build_group_name_set(fields=mapping_fields)
        date_names = self.build_date_name_set(fields=mapping_fields)

        physical_fields = getattr(record, "fields", []) or []

        normalized_fields = DateFieldConsolidator consolidate(
            physical_fields,
        )

        added_fields: set[str] = set()

        for field in normalized_fields:
            field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")

            if not field_name:
                continue

            if self.is_date_part_name(field_name):
                continue

            if field_name in added_fields:
                continue

            if field_name in group_names and field_name not in date_names:
                continue

            if self.is_non_physical_group_field(field=field):
                continue

            adjusted_field = self.adjust_occurs_field(field=field)

            canonical_record.fields.append(
                CanonicalField(
                    name=field_name,
                    datatype=getattr(adjusted_field, "datatype", None),
                    length=getattr(adjusted_field, "length", None),
                )
            )
```

```

        scale=getattr(adjusted_field, "scale", None),
        occurs=getattr(adjusted_field, "occurs", False),
        occurs_max=getattr(adjusted_field, "occurs_max", None),
    )
)

added_fields.add(field_name)

self.apply_primary_key_rules(
    canonical_record=canonical_record,
    record=record,
    added_fields=added_fields,
    group_names=group_names,
    date_names=date_names,
)

schema.records.append(canonical_record)

self.add_sets_and_relationships(
    schema=schema,
    metadata=metadata,
)

return schema

def apply_primary_key_rules(
    self,
    canonical_record: CanonicalRecord,
    record,
    added_fields: set[str],
    group_names: set[str],
    date_names: set[str],
) -> None:
    primary_key = canonical_record.primary_key

    if not primary_key:
        canonical_record.primary_keys = []
        return

    primary_key_fields = self.resolve_primary_key_fields(
        record=record,
        primary_key=primary_key,
        group_names=group_names,
        date_names=date_names,
    )

    if not primary_key_fields:
        canonical_record.primary_keys = []
        canonical_record.primary_key = None
        return

    canonical_primary_keys: list[str] = []

    for key_field in primary_key_fields:
        key_name = NameNormalizer.normalize(getattr(key_field, "name", "") or "")

        if not key_name:
            continue

        if key_name in canonical_primary_keys:
            continue

        if key_name in date_names:
            continue

        if key_name in group_names:
            continue

        if self.is_date_part_name(key_name):
            continue

        if self.is_date_field(key_field):
            continue

        if self.is_filler_field(key_field):
            continue

```

```

        if not self.has_physical_definition(key_field):
            continue

        canonical_primary_keys.append(key_name)

        if key_name in added_fields:
            continue

        canonical_record.fields.append(
            CanonicalField(
                name=key_name,
                datatype=self.datatype_for_key_field(key_field),
                length=self.length_for_key_field(key_field),
                scale=getattr(key_field, "scale", None),
                occurs=False,
                occurs_max=None,
            )
        )

        added_fields.add(key_name)

    canonical_record.primary_keys = canonical_primary_keys

    if canonical_primary_keys:
        canonical_record.primary_key = canonical_primary_keys[0]
    else:
        canonical_record.primary_key = None

def resolve_primary_key_fields(
    self,
    record,
    primary_key: str,
    group_names: set[str],
    date_names: set[str],
) -> list:
    mapping_fields = (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

    key_field = self.find_field_for_primary_key(
        record=record,
        primary_key=primary_key,
    )

    if key_field is None:
        return []

    key_name = NameNormalizer.normalize(getattr(key_field, "name", "") or "")

    if key_name in group_names:
        return self.child_physical_fields_for_group(
            fields=mapping_fields,
            group_field=key_field,
            group_names=group_names,
            date_names=date_names,
        )

    if key_name in date_names:
        return []

    if self.is_date_field(key_field):
        return []

    if self.is_date_part_name(key_name):
        return []

    if self.is_filler_field(key_field):
        return []

    return [key_field]

def child_physical_fields_for_group(
    self,

```

```

        fields,
        group_field,
        group_names: set[str],
        date_names: set[str],
    ) -> list:
        field_list = list(fields or [])

        if not field_list:
            return []

        group_name = NameNormalizer.normalize(getattr(group_field, "name", "") or "")
        group_level = getattr(group_field, "level", None)

        if group_level is None:
            return []

        try:
            group_level_int = int(group_level)
        except Exception:
            return []

        collecting = False
        children = []

        for field in field_list:
            field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")
            field_level = getattr(field, "level", None)

            if field_level is None:
                continue

            try:
                field_level_int = int(field_level)
            except Exception:
                continue

            if not collecting:
                if field_name == group_name:
                    collecting = True
                continue

            if field_level_int <= group_level_int:
                break

            if not field_name:
                continue

            if field_name in group_names:
                continue

            if field_name in date_names:
                continue

            if self.is_date_part_name(field_name):
                continue

            if self.is_date_field(field):
                continue

            if self.is_filler_field(field):
                continue

            if not self.has_physical_definition(field):
                continue

            children.append(field)

        return children

    def build_group_name_set(self, fields) -> set[str]:
        result: set[str] = set()
        field_list = list(fields or [])

        for index, field in enumerate(field_list):
            field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")
            field_level = getattr(field, "level", None)

```

```

        if not field_name or field_level is None:
            continue

        try:
            field_level_int = int(field_level)
        except Exception:
            continue

        for next_field in field_list[index + 1:]:
            next_level = getattr(next_field, "level", None)

            if next_level is None:
                continue

            try:
                next_level_int = int(next_level)
            except Exception:
                continue

            if next_level_int > field_level_int:
                result.add(field_name)
                break

            if next_level_int <= field_level_int:
                break

    return result

def build_date_name_set(self, fields) -> set[str]:
    result: set[str] = set()
    field_list = list(fields or [])

    for index, field in enumerate(field_list):
        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")
        field_level = getattr(field, "level", None)

        if not field_name:
            continue

        if self.is_date_part_name(field_name):
            result.add(field_name)
            continue

        if self.is_date_field(field):
            result.add(field_name)
            continue

        if field_level is None:
            continue

        try:
            field_level_int = int(field_level)
        except Exception:
            continue

    descendants = []

    for next_field in field_list[index + 1:]:
        next_level = getattr(next_field, "level", None)

        if next_level is None:
            continue

        try:
            next_level_int = int(next_level)
        except Exception:
            continue

        if next_level_int <= field_level_int:
            break

        descendants.append(next_field)

    parts_found = set()

```

```

        for descendant in descendants:
            descendant_name = NameNormalizer.normalize(
                getattr(descendant, "name", "") or ""
            )

            part = self.date_part_type_from_name(descendant_name)

            if part:
                parts_found.add(part)

        if {"YEAR", "MONTH", "DAY"}.issubset(parts_found):
            result.add(field_name)

        for descendant in descendants:
            descendant_name = NameNormalizer.normalize(
                getattr(descendant, "name", "") or ""
            )

            if self.date_part_type_from_name(descendant_name):
                result.add(descendant_name)

    return result

def date_part_type_from_name(self, field_name: str) -> str | None:
    parsed = DateFieldConsolidator.parse_date_part(field_name=field_name)

    if parsed is None:
        return None

    return parsed.get("part")

def find_field_for_primary_key(self, record, primary_key: str):
    fields = []
    fields.extend(getattr(record, "fields", []) or [])
    fields.extend(getattr(record, "mapping_fields", []) or [])

    normalized_primary_key = NameNormalizer.normalize(primary_key)
    suffix_removed_primary_key = self.remove_record_suffix(normalized_primary_key)

    for field in fields:
        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")

        if field_name == normalized_primary_key:
            return field

        if self.remove_record_suffix(field_name) == suffix_removed_primary_key:
            return field

    return None

def datatype_for_key_field(self, key_field) -> str:
    datatype = getattr(key_field, "datatype", None)

    if datatype:
        return datatype

    picture = str(getattr(key_field, "picture", "") or "").upper()

    if "X" in picture:
        return "CHAR"

    if "9" in picture:
        return "DECIMAL"

    return "CHAR"

def length_for_key_field(self, key_field) -> int | None:
    length = getattr(key_field, "length", None)

    if length:
        try:
            return int(length)
        except Exception:
            pass

    return 20

```

```

def adjust_occurs_field(self, field):
    occurs = getattr(field, "occurs", False)
    occurs_max = getattr(field, "occurs_max", None)

    if not occurs or not occurs_max:
        return field

    length = getattr(field, "length", None)

    if length is None:
        return field

    try:
        adjusted_length = int(length) * int(occurs_max)
    except Exception:
        return field

    try:
        return field.model_copy(update={"length": adjusted_length})
    except Exception:
        try:
            field.length = adjusted_length
        except Exception:
            pass

        return field

def is_non_physical_group_field(self, field) -> bool:
    if self.is_date_field(field):
        return False

    if getattr(field, "is_group", False):
        return True

    if getattr(field, "has_child", False):
        return True

    return False

def has_physical_definition(self, field) -> bool:
    if getattr(field, "picture", None):
        return True

    if getattr(field, "datatype", None):
        return True

    if getattr(field, "length", None):
        return True

    return False

def is_date_field(self, field) -> bool:
    datatype = str(getattr(field, "datatype", "") or "").upper()
    basetype = str(getattr(field, "basetype", "") or "").upper()

    field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")
    normalized_name = field_name.replace(" ", "_")

    return (
        datatype == "DATE"
        or basetype == "DATE"
        or normalized_name.endswith("_DATE")
        or normalized_name == "DATE"
        or "_DATE_" in normalized_name
    )

def is_filler_field(self, field) -> bool:
    return str(getattr(field, "name", "") or "").upper().startswith("FILLER")

def is_date_part_name(self, field_name: str) -> bool:
    return DateFieldConsolidator.parse_date_part(field_name=field_name) is not None

def remove_record_suffix(self, field_name: str) -> str:
    normalized = NameNormalizer.normalize(field_name)
    normalized = normalized.replace(" ", "_")

```

```

        return re.sub(r"_[0-9]{4}$", "", normalized)

def add_sets_and_relationships(self, schema: CanonicalSchema, metadata) -> None:
    added_relationships = set()

    for rel in getattr(metadata, "relationships", []) or []:
        set_name = getattr(rel, "set_name", None)
        owner_record = getattr(rel, "owner_record", None)
        member_record = getattr(rel, "member_record", None)

        if not set_name or not owner_record or not member_record:
            continue

        normalized_set_name = NameNormalizer.normalize(set_name)
        normalized_owner = NameNormalizer.normalize(owner_record)
        normalized_member = NameNormalizer.normalize(member_record)

        key = (
            normalized_owner,
            normalized_member,
            normalized_set_name,
        )

        if key in added_relationships:
            continue

        added_relationships.add(key)

        schema.sets.append(
            CanonicalSet(
                name=normalized_set_name,
                owner_record=normalized_owner,
                member_record=normalized_member,
            )
        )

        schema.relationships.append(
            CanonicalRelationship(
                set_name=normalized_set_name,
                parent_record=normalized_owner,
                child_record=normalized_member,
                cardinality="1:N",
            )
        )

```

```

=====
FILE: src\idms_modernizer\services\date_field_consolidator.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.services.name_normalizer import NameNormalizer

```

```

class DateFieldConsolidator:
    """
    Consolidates legacy date child fields into one DATE field.

    Supports:
    - YEAR / MONTH / DAY
    - YR / MO / DY
    - Y / M / D
    - YY / MM / DD
    - YYYY / MM / DD
    - DY / DM / DD
    """

    YEAR_PARTS = {
        "YEAR",
        "YR",
        "Y",
        "YY",
    }

```



```

        "YYYY",
        "DY",
    }

MONTH_PARTS = {
    "MONTH",
    "MON",
    "MO",
    "M",
    "MM",
    "DM",
}

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}

@staticmethod
def consolidate(
    fields: list[DataField],
) -> list[DataField]:
    result: list[DataField] = []
    consumed: set[str] = set()
    added_dates: set[str] = set()

    date_groups = DateFieldConsolidator.collect_date_groups(
        fields=fields,
    )

    for date_name, parts in date_groups.items():
        if not DateFieldConsolidator.has_complete_date_group(
            parts=parts,
        ):
            continue

        normalized_date_name = NameNormalizer.normalize(
            date_name,
        )

        if normalized_date_name in added_dates:
            continue

        template = (
            parts.get("YEAR")
            or parts.get("MONTH")
            or parts.get("DAY")
        )

        result.append(
            DataField(
                name=date_name,
                level=DateFieldConsolidator.parent_level(
                    field=template,
                ),
                datatype="DATE",
                length=None,
                scale=None,
                picture=None,
                start_position=DateFieldConsolidator.min_start_position(
                    parts=parts,
                ),
                end_position=DateFieldConsolidator.max_end_position(
                    parts=parts,
                ),
                basetype="DATE",
                has_child=True,
                is_group=True,
            )
        )

        added_dates.add(
            normalized_date_name,
        )

```

```

        for part_field in parts.values():
            consumed.add(
                NameNormalizer.normalize(
                    part_field.name,
                )
            )

    for field in fields:
        if not field or not field.name:
            continue

        normalized_name = NameNormalizer.normalize(
            field.name,
        )

        if normalized_name in consumed:
            continue

        if normalized_name in added_dates:
            continue

        result.append(
            field,
        )

    return result

@staticmethod
def collect_date_groups(
    fields: list[DataField],
) -> dict[str, dict[str, DataField]]:
    groups: dict[str, dict[str, DataField]] = {}

    for field in fields:
        parsed = DateFieldConsolidator.parse_date_part(
            field_name=field.name,
        )

        if parsed is None:
            continue

        date_name = parsed["date_name"]
        part = parsed["part"]

        if date_name not in groups:
            groups[date_name] = {}

        groups[date_name][part] = field

    return groups

@staticmethod
def parse_date_part(
    field_name: str,
) -> dict[str, str] | None:
    tokens = DateFieldConsolidator.split_tokens(
        value=field_name,
    )

    if len(tokens) < 2:
        return None

    for index, token in enumerate(tokens):
        part = DateFieldConsolidator.date_part_type(
            token=token,
            tokens=tokens,
        )

        if part is None:
            continue

        date_tokens = tokens.copy()
        date_tokens[index] = "DATE"

    return {
        "date_name": " ".join(date_tokens),
    }

```

```

        "part": part,
    }

    return None

@staticmethod
def date_part_type(
    token: str,
    tokens: list[str],
) -> str | None:
    token = token.upper()

    has_dy_dm_dd = (
        "DY" in tokens
        and "DM" in tokens
        and "DD" in tokens
    )

    if token in DateFieldConsolidator.YEAR_PARTS:
        if token == "DY" and not has_dy_dm_dd:
            return "DAY"

        return "YEAR"

    if token in DateFieldConsolidator.MONTH_PARTS:
        return "MONTH"

    if token in DateFieldConsolidator.DAY_PARTS:
        return "DAY"

    return None

@staticmethod
def has_complete_date_group(
    parts: dict[str, DataField],
) -> bool:
    return (
        "YEAR" in parts
        and "MONTH" in parts
        and "DAY" in parts
    )

@staticmethod
def parent_level(
    field: DataField | None,
) -> int | None:
    if field is None:
        return None

    level = getattr(
        field,
        "level",
        None,
    )

    if level is None:
        return None

    try:
        return max(
            int(level) - 1,
            1,
        )

    except Exception:
        return level

@staticmethod
def min_start_position(
    parts: dict[str, DataField],
) -> int | None:
    values: list[int] = []

    for field in parts.values():
        value = getattr(
            field,

```

```

        "start_position",
        None,
    )

    if value is not None:
        values.append(
            int(value),
        )

    if not values:
        return None

    return min(values)

@staticmethod
def max_end_position(
    parts: dict[str, DataField],
) -> int | None:
    values: list[int] = []

    for field in parts.values():
        value = getattr(
            field,
            "end_position",
            None,
        )

        if value is not None:
            values.append(
                int(value),
            )

    if not values:
        return None

    return max(values)

@staticmethod
def split_tokens(
    value: str,
) -> list[str]:
    normalized = NameNormalizer.normalize(
        value,
    )

    return [
        token.upper()
        for token in re.split(
            r"[\s_-]+",
            normalized,
        )
        if token
    ]

```

```

=====
FILE: src\idms_modernizer\services\db2_datatype_mapper.py
ENCODING: utf-8
=====

```

```
import re
```

```

class DB2DatatypeMapper:
    """
    Maps COBOL / IDMS PIC metadata to DB2 datatype.

    Rules:
    - PIC X(n) -> CHAR(n)
    - PIC X repeated -> CHAR(length)

    - PIC 9(n) -> DECIMAL(n)
    - PIC S9(n) -> DECIMAL(n)

    - PIC 9(n) COMP-3 -> DECIMAL(n,0)
    - PIC S9(n) COMP-3 -> DECIMAL(n,0)
    """

```

- PIC 9(n)V9(m) -> DECIMAL(n+m,m)
- PIC S9(n)V9(m) -> DECIMAL(n+m,m)
- PIC 9(n)V9(m) COMP-3 -> DECIMAL(n+m,m)
- PIC S9(n)V9(m) COMP-3 -> DECIMAL(n+m,m)
- PIC 9(n)V99 -> DECIMAL(n+2,2)
- PIC S9(n)V99 -> DECIMAL(n+2,2)
- PIC 9(n)V99 COMP-3 -> DECIMAL(n+2,2)
- PIC S9(n)V99 COMP-3 -> DECIMAL(n+2,2)

Important:

- Do not merge implied-decimal PICs.
- PIC S9(13)V9(2) COMP-3 must map to DECIMAL(15,2).
- PIC S9(13)V99 COMP-3 must map to DECIMAL(15,2).
- It must not be treated as PIC S9(15) COMP-3.
- Handles lowercase v / comp-3 and spaces around parentheses.

- DATE -> DATE
- TIMESTAMP / DATETIME -> TIMESTAMP

"""

```

DEFAULT_CHAR_LENGTH = 1
DEFAULT_DECIMAL_PRECISION = 18
DEFAULT_DECIMAL_SCALE = 0

```

```

@staticmethod
def map(field) -> str:
    datatype = (
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or ""
    )
    datatype = str(datatype or "").strip().upper()

    picture = (
        getattr(field, "picture", None)
        or getattr(field, "pic", None)
        or getattr(field, "pic_clause", None)
        or ""
    )
    picture = str(picture or "").strip()

    length = (
        getattr(field, "length", None)
        or getattr(field, "precision", None)
        or getattr(field, "storage_length", None)
        or getattr(field, "physical_length", None)
        or getattr(field, "byte_length", None)
    )

    scale = getattr(field, "scale", None)

    if datatype == "DATE":
        return "DATE"

    if datatype in {"TIMESTAMP", "DATETIME"}:
        return "TIMESTAMP"

    if picture:
        return DB2DatatypeMapper.map_picture(
            picture=picture,
            fallback_length=length,
            fallback_scale=scale,
        )

    if datatype in {"DECIMAL", "NUMERIC", "COMP", "COMP-3"}:
        precision = DB2DatatypeMapper.safe_int(
            value=length,
            default=DB2DatatypeMapper.DEFAULT_DECIMAL_PRECISION,
        )
        actual_scale = DB2DatatypeMapper.safe_int(
            value=scale,

```

```

        default=DB2DatatypeMapper.DEFAULT_DECIMAL_SCALE,
    )

    return DB2DatatypeMapper.format_decimal(
        precision=precision,
        scale=actual_scale,
        force_scale=actual_scale > 0,
    )

if datatype in {"CHAR", "VARCHAR", "DISPLAY", "TEXT", "ALPHANUMERIC"}:
    actual_length = DB2DatatypeMapper.safe_int(
        value=length,
        default=DB2DatatypeMapper.DEFAULT_CHAR_LENGTH,
    )
    return f"CHAR({actual_length})"

return f"CHAR({DB2DatatypeMapper.DEFAULT_CHAR_LENGTH})"

@staticmethod
def map_datatype(field) -> str:
    return DB2DatatypeMapper.map(field=field)

@staticmethod
def map_picture(
    picture: str,
    fallback_length=None,
    fallback_scale=None,
) -> str:
    clean = DB2DatatypeMapper.clean_picture(
        picture=picture,
    )

    if not clean:
        return f"CHAR({DB2DatatypeMapper.DEFAULT_CHAR_LENGTH})"

    clean_upper = clean.upper()

    has_comp3 = "COMP-3" in clean_upper
    has_comp = bool(re.search(r"\bCOMP\b", clean_upper)) and not has_comp3

    core = DB2DatatypeMapper.picture_core(
        picture=clean,
    )

    if DB2DatatypeMapper.picture_is_char(core):
        length = DB2DatatypeMapper.char_length(
            picture=core,
            fallback_length=fallback_length,
        )

        return f"CHAR({length})"

    if DB2DatatypeMapper.picture_is_decimal(core):
        precision, scale = DB2DatatypeMapper.decimal_precision_scale(
            picture=core,
        )

        return DB2DatatypeMapper.format_decimal(
            precision=precision,
            scale=scale,
            force_scale=True,
        )

    if DB2DatatypeMapper.picture_is_numeric(core):
        precision = DB2DatatypeMapper.numeric_precision(
            picture=core,
            fallback_length=fallback_length,
        )

    if has_comp3 or has_comp:
        return DB2DatatypeMapper.format_decimal(
            precision=precision,
            scale=0,
            force_scale=True,
        )

```

```

        return DB2DatatypeMapper.format_decimal(
            precision=precision,
            scale=0,
            force_scale=False,
        )

    if fallback_length:
        actual_length = DB2DatatypeMapper.safe_int(
            value=fallback_length,
            default=DB2DatatypeMapper.DEFAULT_CHAR_LENGTH,
        )

        return f"CHAR({actual_length})"

    return f"CHAR({DB2DatatypeMapper.DEFAULT_CHAR_LENGTH})"

@staticmethod
def clean_picture(
    picture: str,
) -> str:
    value = str(picture or "").strip().upper()

    value = value.replace("\u00a0", " ")
    value = value.replace("\t", " ")
    value = value.replace(".", "")

    value = re.sub(
        r"\bPICTURE\b",
        "",
        value,
        flags=re.IGNORECASE,
    )

    value = re.sub(
        r"\bPIC\b",
        "",
        value,
        flags=re.IGNORECASE,
    )

    value = re.sub(r"\s+", " ", value).strip()

    value = value.replace(" (", "(")
    value = value.replace("( ", "(")
    value = value.replace(" )", ")")
    value = value.replace(")", " ")

    value = re.sub(r"\s+", " ", value).strip()

    return value

@staticmethod
def picture_core(
    picture: str,
) -> str:
    value = str(picture or "").strip().upper()

    value = value.replace("\u00a0", " ")
    value = value.replace("\t", " ")

    value = value.replace("COMP-3", "")
    value = re.sub(r"\bCOMP\b", "", value)
    value = re.sub(r"\bDISPLAY\b", "", value)
    value = re.sub(r"\bUSAGE\b", "", value)
    value = re.sub(r"\bIS\b", "", value)
    value = re.sub(r"\bSIGN\b", "", value)
    value = re.sub(r"\bLEADING\b", "", value)
    value = re.sub(r"\bTRAILING\b", "", value)
    value = re.sub(r"\bSEPARATE\b", "", value)
    value = re.sub(r"\bCHARACTER\b", "", value)

    value = re.sub(
        r"\bPICTURE\b",
        "",
        value,
        flags=re.IGNORECASE,
    )

```

```

    )

    value = re.sub(
        r"\bPIC\b",
        "",
        value,
        flags=re.IGNORECASE,
    )

    value = value.replace(".", "")
    value = re.sub(r"\s+", "", value).strip()

    return value

@staticmethod
def picture_is_char(
    picture: str,
) -> bool:
    value = str(picture or "").strip().upper()

    if value.startswith("(") and value.endswith(")"):
        return DB2DatatypeMapper.extract_parenthesized_int(value) is not None

    if value and set(value) == {"X"}:
        return True

    return False

@staticmethod
def picture_is_numeric(
    picture: str,
) -> bool:
    value = str(picture or "").strip().upper()

    if not value:
        return False

    if value.startswith("S"):
        value = value[1:]

    if "V" in value:
        return False

    return DB2DatatypeMapper.is_numeric_picture_part(
        value=value,
    )

@staticmethod
def picture_is_decimal(
    picture: str,
) -> bool:
    value = str(picture or "").strip().upper()

    if not value:
        return False

    if value.startswith("S"):
        value = value[1:]

    if "V" not in value:
        return False

    before_v, after_v = value.split("V", 1)

    return (
        DB2DatatypeMapper.is_numeric_picture_part(before_v)
        and DB2DatatypeMapper.is_numeric_picture_part(after_v)
    )

@staticmethod
def char_length(
    picture: str,
    fallback_length=None,
) -> int:
    value = str(picture or "").strip().upper()

```



```

if value.startswith("X(") and value.endswith(")"):
    parsed_length = DB2DatatypeMapper.extract_parenthesized_int(value)

    if parsed_length is not None:
        return parsed_length

if value and set(value) == {"X"}:
    return len(value)

return DB2DatatypeMapper.safe_int(
    value=fallback_length,
    default=DB2DatatypeMapper.DEFAULT_CHAR_LENGTH,
)

@staticmethod
def numeric_precision(
    picture: str,
    fallback_length=None,
) -> int:
    value = str(picture or "").strip().upper()

    if value.startswith("S"):
        value = value[1:]

    if "V" in value:
        precision, _scale = DB2DatatypeMapper.decimal_precision_scale(
            picture=picture,
        )

        return precision

    precision = DB2DatatypeMapper.count_numeric_digits(
        value=value,
    )

    if precision > 0:
        return precision

    return DB2DatatypeMapper.safe_int(
        value=fallback_length,
        default=DB2DatatypeMapper.DEFAULT_DECIMAL_PRECISION,
    )

@staticmethod
def decimal_precision_scale(
    picture: str,
) -> tuple[int, int]:
    value = str(picture or "").strip().upper()

    if value.startswith("S"):
        value = value[1:]

    if "V" not in value:
        precision = DB2DatatypeMapper.numeric_precision(
            picture=value,
            fallback_length=DB2DatatypeMapper.DEFAULT_DECIMAL_PRECISION,
        )

        return precision, 0

    before_v, after_v = value.split("V", 1)

    integer_digits = DB2DatatypeMapper.count_numeric_digits(
        value=before_v,
    )
    decimal_digits = DB2DatatypeMapper.count_numeric_digits(
        value=after_v,
    )

    precision = integer_digits + decimal_digits
    scale = decimal_digits

    return precision, scale

@staticmethod
def is_numeric_picture_part(

```

```

        value: str,
    ) -> bool:
        text = str(value or "").strip().upper()

        if not text:
            return False

        if text.startswith("S"):
            text = text[1:]

        count = DB2DatatypeMapper.count_numeric_digits(
            value=text,
        )

        if count <= 0:
            return False

        remaining = DB2DatatypeMapper.remove_numeric_picture_tokens(
            value=text,
        )

        return remaining == ""

    @staticmethod
    def count_numeric_digits(
        value: str,
    ) -> int:
        text = str(value or "").strip().upper()

        if text.startswith("S"):
            text = text[1:]

        total = 0
        index = 0

        while index < len(text):
            current_character = text[index]

            if current_character not in {"9", "0"}:
                index += 1
                continue

            if index + 1 < len(text) and text[index + 1] == "(":
                close_index = text.find(")", index + 2)

                if close_index != -1:
                    number_text = text[index + 2:close_index].strip()

                    if number_text.isdigit():
                        total += int(number_text)
                        index = close_index + 1
                        continue

            total += 1
            index += 1

        return total

    @staticmethod
    def count_9_digits(
        value: str,
    ) -> int:
        return DB2DatatypeMapper.count_numeric_digits(
            value=value,
        )

    @staticmethod
    def remove_numeric_picture_tokens(
        value: str,
    ) -> str:
        text = str(value or "").strip().upper()

        if text.startswith("S"):
            text = text[1:]

        result = []

```

```

index = 0

while index < len(text):
    current_character = text[index]

    if current_character not in {"9", "0"}:
        result.append(current_character)
        index += 1
        continue

    if index + 1 < len(text) and text[index + 1] == "(":
        close_index = text.find(")", index + 2)

        if close_index != -1:
            number_text = text[index + 2:close_index].strip()

            if number_text.isdigit():
                index = close_index + 1
                continue

        index += 1

    return "".join(result)

@staticmethod
def extract_parenthesized_int(
    value: str,
) -> int | None:
    text = str(value or "").strip()

    open_index = text.find("(")

    if open_index == -1:
        return None

    close_index = text.find(")", open_index + 1)

    if close_index == -1:
        return None

    number_text = text[open_index + 1:close_index].strip()

    if not number_text.isdigit():
        return None

    return int(number_text)

@staticmethod
def format_decimal(
    precision: int,
    scale: int = 0,
    force_scale: bool = False,
) -> str:
    actual_precision = DB2DatatypeMapper.safe_int(
        value=precision,
        default=DB2DatatypeMapper.DEFAULT_DECIMAL_PRECISION,
    )
    actual_scale = DB2DatatypeMapper.safe_int(
        value=scale,
        default=DB2DatatypeMapper.DEFAULT_DECIMAL_SCALE,
    )

    if actual_precision <= 0:
        actual_precision = DB2DatatypeMapper.DEFAULT_DECIMAL_PRECISION

    if actual_scale < 0:
        actual_scale = 0

    if force_scale or actual_scale > 0:
        return f"DECIMAL({actual_precision},{actual_scale})"

    return f"DECIMAL({actual_precision})"

@staticmethod
def safe_int(
    value,

```

```

        default: int,
    ) -> int:
        try:
            if value is None:
                return default

            text = str(value).strip()

            if not text:
                return default

            return int(text)
        except Exception:
            return default

```

```

=====
FILE: src\idms_modernizer\services\db2_generation_service.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.generators.ddl_generator import (
    DDLGenerator
)

from idms_modernizer.services.db2_mapping_service import (
    DB2MappingService
)

```

```

class DB2GenerationService:

    def __init__(self):

        self.mapper = (
            DB2MappingService()
        )

        self.ddl_generator = (
            DDLGenerator()
        )

    def generate(
        self,
        canonical_schema
    ):

        db2_model = (
            self.mapper.build_db2_model(
                canonical_schema
            )
        )

        ddl = (
            self.ddl_generator.generate(
                db2_model
            )
        )

        return db2_model, ddl

```

```

=====
FILE: src\idms_modernizer\services\db2_mapping_service.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2ForeignKey,
    DB2Model,
    DB2Table,
)

```

```
from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper
```

```
print("LOADED DB2MappingService VERSION NAMING-FK-DATE-FIX-2026-07-29")
```

```
class DB2MappingService:
```

```
    """
```

```
    Builds DB2 model from canonical schema.
```

```
    Naming rules implemented:
```

```
    1. Normal COBOL business field:
```

- Start from COBOL business field.
- Normalize separators to underscore.
- Apply project abbreviations.
- Remove generic trailing qualifiers where applicable.
- Append _479<record-code>.

```
    Example:
```

```
    NR-CIO-FORM-AS in VMBSIC -> NR_CIOFMAS_479BSIC
```

```
    CT-RK-TGDSV in VMBFORM -> CT_RKTGDSV_479FORM
```

```
    NR-ID-STOCK in VMBCOUP -> NR_IDSTOCK_479COUP
```

```
    2. Date field:
```

- Outer COBOL date field becomes one DB2 DATE column.
- DB2 date name keeps DA prefix.
- Append _479<record-code>.

```
    Example:
```

```
    DA-CPTA-GDIFC -> DA_CPTA_479MBFC
```

```
    DA-UB-GDIFR -> DA_UBDATE_479MBFR
```

```
    3. Foreign key:
```

- FK column reuses exact referenced/master DB2 column name.
- No SET prefix is added to FK column names.
- Relationship/set name is metadata only.

```
    Example:
```

```
    Master PK: NR_CIOFMAS_479BSIC
```

```
    Child FK : NR_CIOFMAS_479BSIC
```

```
    4. Missing PK:
```

- If no primary key exists, create generated technical key:
 ID_RECORD_<record-code>
- Type: CHAR(20)

```
    """
```

```
    TECHNICAL_KEY_DATATYPE = "CHAR(20)"
```

```
    GENERIC_TRAILING_TOKENS = {
```

```
        "SIC",  
        "FC",  
        "GDIFC",  
        "GDIFR",  
        "GDIFAR",  
        "GDIF",  
        "GDI",  
        "REC",  
        "RECORD",
```

```
    }
```

```
    PROJECT_ABBREVIATIONS = {
```

```
        "FORM": "FM",  
        "EVPRCP": "ERCP",
```

```
    }
```

```
    DATE_TOKENS = {
```

```
        "DATE",  
        "DT",  
        "DTE",  
        "DA",  
        "YYMMDD",  
        "YYYYMMDD",  
        "YMD",
```

```
    }
```

```

YEAR_PARTS = {
    "YEAR",
    "YR",
    "Y",
    "YY",
    "YYYY",
    "DY",
}

MONTH_PARTS = {
    "MONTH",
    "MON",
    "MO",
    "M",
    "MM",
    "DM",
}

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}

def build_db2_model(
    self,
    schema: CanonicalSchema,
) -> DB2Model:
    print("USING DB2MappingService.build_db2_model VERSION NAMING-FK-DATE-FIX-2026-07-29")

    tables: list[DB2Table] = []

    for record in getattr(schema, "records", []) or []:
        record_name = getattr(record, "name", "") or ""
        table_name = self.normalize_table_name(record_name)

        table = DB2Table(
            name=table_name,
            columns=[],
            foreign_keys=[],
            primary_key=None,
            primary_keys=[],
        )

        declared_primary_keys = self.get_record_primary_keys(
            record=record,
        )

        normalized_primary_keys = []

        for primary_key in declared_primary_keys:
            normalized_primary_key = self.normalize_column_name(
                value=primary_key,
                record_name=record_name,
                field=None,
            )

            if normalized_primary_key and normalized_primary_key not in normalized_primary_keys:
                normalized_primary_keys.append(normalized_primary_key)

        table.primary_keys = normalized_primary_keys

        if table.primary_keys:
            table.primary_key = table.primary_keys[0]

        added_columns: set[str] = set()

        for field in getattr(record, "fields", []) or []:
            raw_field_name = getattr(field, "name", "") or ""

            column_name = self.normalize_column_name(
                value=raw_field_name,
                record_name=record_name,
                field=field,
            )

```

```

        if not column_name:
            continue

        if column_name in added_columns:
            continue

        is_primary_key = column_name in table.primary_keys

        column = DB2Column(
            name=column_name,
            datatype=DB2DatatypeMapper.map(field),
            nullable=not is_primary_key,
            primary_key=is_primary_key,
            generated=False,
            source_kind="COBOL",
        )

        table.columns.append(column)
        added_columns.add(column_name)

    self.ensure_record_primary_key(
        table=table,
        record_name=record_name,
    )

    tables.append(table)

model = DB2Model(
    tables=tables,
)

self.apply_relationship_foreign_keys(
    schema=schema,
    model=model,
)

return model

def apply_relationship_foreign_keys(
    self,
    schema: CanonicalSchema,
    model: DB2Model,
) -> None:
    table_lookup = self.build_table_lookup(
        model=model,
    )

    for relationship in getattr(schema, "relationships", []) or []:
        set_name = self.get_relationship_set_name(
            relationship=relationship,
        )

        owner_record = self.get_relationship_owner_record(
            relationship=relationship,
        )

        member_record = self.get_relationship_member_record(
            relationship=relationship,
        )

        if not owner_record or not member_record:
            continue

        owner_table = self.find_table_for_record(
            record_name=owner_record,
            table_lookup=table_lookup,
        )

        member_table = self.find_table_for_record(
            record_name=member_record,
            table_lookup=table_lookup,
        )

        if owner_table is None or member_table is None:
            continue

```

```

        owner_pk_columns = self.get_primary_key_columns(
            table=owner_table,
        )

    if not owner_pk_columns:
        owner_pk_columns = self.ensure_record_primary_key(
            table=owner_table,
            record_name=owner_record,
        )

    for owner_pk_column in owner_pk_columns:
        fk_column = self.ensure_fk_column_reusing_parent_name(
            child_table=member_table,
            parent_pk_column=owner_pk_column,
        )

        self.add_foreign_key_if_missing(
            child_table=member_table,
            child_column=fk_column.name,
            parent_table=owner_table,
            parent_column=owner_pk_column.name,
            set_name=set_name,
        )

def ensure_fk_column_reusing_parent_name(
    self,
    child_table: DB2Table,
    parent_pk_column: DB2Column,
) -> DB2Column:
    column_name = getattr(parent_pk_column, "name", "") or ""

    existing_column = self.find_column(
        table=child_table,
        column_name=column_name,
    )

    if existing_column is not None:
        existing_column.nullable = True
        existing_column.generated = True
        existing_column.primary_key = False
        existing_column.source_kind = "SET_FK"

        if not getattr(existing_column, "datatype", None):
            existing_column.datatype = getattr(parent_pk_column, "datatype", "") or ""

        return existing_column

    new_column = DB2Column(
        name=column_name,
        datatype=getattr(parent_pk_column, "datatype", "") or self.TECHNICAL_KEY_DATATYPE,
        nullable=True,
        primary_key=False,
        generated=True,
        source_kind="SET_FK",
    )

    child_table.columns.append(new_column)

    return new_column

def add_foreign_key_if_missing(
    self,
    child_table: DB2Table,
    child_column: str,
    parent_table: DB2Table,
    parent_column: str,
    set_name: str = "",
) -> None:
    if self.foreign_key_exists(
        child_table=child_table,
        child_column=child_column,
        parent_table=parent_table,
        parent_column=parent_column,
    ):
        return

```



```

        child_table.foreign_keys.append(
            DB2ForeignKey(
                column_name=child_column,
                reference_table=parent_table.name,
                reference_column=parent_column,
                set_name=self.normalize_name(set_name),
            )
        )
    )

def foreign_key_exists(
    self,
    child_table: DB2Table,
    child_column: str,
    parent_table: DB2Table,
    parent_column: str,
) -> bool:
    normalized_child_column = self.normalize_name(child_column)
    normalized_parent_table = self.normalize_name(parent_table.name)
    normalized_parent_column = self.normalize_name(parent_column)

    for foreign_key in getattr(child_table, "foreign_keys", []) or []:
        existing_child_column = self.normalize_name(
            getattr(foreign_key, "column_name", "") or ""
        )
        existing_parent_table = self.normalize_name(
            getattr(foreign_key, "reference_table", "") or ""
        )
        existing_parent_column = self.normalize_name(
            getattr(foreign_key, "reference_column", "") or ""
        )

        if (
            existing_child_column == normalized_child_column
            and existing_parent_table == normalized_parent_table
            and existing_parent_column == normalized_parent_column
        ):
            return True

    return False

def ensure_record_primary_key(
    self,
    table: DB2Table,
    record_name: str,
) -> list[DB2Column]:
    primary_key_columns: list[DB2Column] = []

    for primary_key_name in list(getattr(table, "primary_keys", []) or []):
        column = self.find_column(
            table=table,
            column_name=primary_key_name,
        )

        if column is None:
            continue

        column.nullable = False
        column.primary_key = True

        if not getattr(column, "source_kind", ""):
            column.source_kind = "COBOL"

        primary_key_columns.append(column)

    if primary_key_columns:
        table.primary_keys = [
            column.name
            for column in primary_key_columns
        ]
        table.primary_key = table.primary_keys[0]

    return primary_key_columns

generated_pk_name = self.generated_primary_key_name(
    record_name=record_name,

```

```

)

existing_generated = self.find_column(
    table=table,
    column_name=generated_pk_name,
)

if existing_generated is not None:
    existing_generated.nullable = False
    existing_generated.primary_key = True
    existing_generated.generated = True
    existing_generated.source_kind = "GENERATED PK"

    table.primary_keys = [existing_generated.name]
    table.primary_key = existing_generated.name

    return [existing_generated]

generated_column = DB2Column(
    name=generated_pk_name,
    datatype=self.TECHNICAL_KEY_DATATYPE,
    nullable=False,
    primary_key=True,
    generated=True,
    source_kind="GENERATED PK",
)

table.columns.insert(0, generated_column)
table.primary_keys = [generated_column.name]
table.primary_key = generated_column.name

return [generated_column]

def generated_primary_key_name(
    self,
    record_name: str,
) -> str:
    record_code = self.record_code(record_name)

    if record_code:
        return f"ID_RECORD_{record_code}"

    table_name = self.normalize_table_name(record_name)

    if table_name:
        return f"ID_RECORD_{table_name}"

    return "ID_RECORD"

def get_primary_key_columns(
    self,
    table: DB2Table,
) -> list[DB2Column]:
    primary_key_names = list(getattr(table, "primary_keys", []) or [])

    if not primary_key_names and getattr(table, "primary_key", None):
        primary_key_names = [table.primary_key]

    if not primary_key_names:
        primary_key_names = [
            getattr(column, "name", "") or ""
            for column in getattr(table, "columns", []) or []
            if getattr(column, "primary_key", False)
        ]

    result: list[DB2Column] = []

    for primary_key_name in primary_key_names:
        column = self.find_column(
            table=table,
            column_name=primary_key_name,
        )

        if column is not None:
            result.append(column)

```

```

        return result

def find_column(
    self,
    table: DB2Table,
    column_name: str,
) -> DB2Column | None:
    normalized_column_name = self.normalize_name(column_name)

    if not normalized_column_name:
        return None

    for column in getattr(table, "columns", []) or []:
        current_name = self.normalize_name(
            getattr(column, "name", "") or ""
        )

        if current_name == normalized_column_name:
            return column

    return None

def build_table_lookup(
    self,
    model: DB2Model,
) -> dict[str, DB2Table]:
    lookup: dict[str, DB2Table] = {}

    for table in getattr(model, "tables", []) or []:
        table_name = getattr(table, "name", "") or ""
        normalized_table_name = self.normalize_name(table_name)

        if normalized_table_name:
            lookup[normalized_table_name] = table

        suffix_removed = self.remove_record_suffix(
            normalized_table_name,
        )

        if suffix_removed:
            lookup[suffix_removed] = table

    return lookup

def find_table_for_record(
    self,
    record_name: str,
    table_lookup: dict[str, DB2Table],
) -> DB2Table | None:
    normalized_record_name = self.normalize_table_name(record_name)

    if normalized_record_name in table_lookup:
        return table_lookup[normalized_record_name]

    suffix_removed = self.remove_record_suffix(
        normalized_record_name,
    )

    if suffix_removed in table_lookup:
        return table_lookup[suffix_removed]

    for table_key, table in table_lookup.items():
        if self.remove_record_suffix(table_key) == suffix_removed:
            return table

    return None

def normalize_table_name(
    self,
    value: str | None,
) -> str:
    return self.to_db2_name(value or "")

def normalize_column_name(
    self,
    value: str | None,

```

```

        record_name: str | None,
        field=None,
    ) -> str:
        raw_value = str(value or "").strip()

        if not raw_value:
            return ""

        if field is not None and self.is_date_field(field=field):
            return self.date_column_name(
                field_name=raw_value,
                record_name=record_name,
            )

        return self.normal_business_column_name(
            field_name=raw_value,
            record_name=record_name,
        )

    def normal_business_column_name(
        self,
        field_name: str,
        record_name: str | None,
    ) -> str:
        base = self.business_field_base_name(
            field_name=field_name,
        )

        record_code = self.record_code(record_name)

        if record_code:
            return f"{base}_479{record_code}"

        return base

    def date_column_name(
        self,
        field_name: str,
        record_name: str | None,
    ) -> str:
        base = self.date_field_base_name(
            field_name=field_name,
        )

        record_code = self.record_code(record_name)

        if record_code:
            return f"{base}_479{record_code}"

        return base

    def business_field_base_name(
        self,
        field_name: str,
    ) -> str:
        normalized = self.to_db2_name(field_name)

        normalized = self.remove_record_suffix(normalized)

        parts = [
            part
            for part in normalized.split("_")
            if part
        ]

        if not parts:
            return normalized

        parts = self.remove_generic_trailing_tokens(
            parts=parts,
        )

        parts = [
            self.PROJECT_ABBREVIATIONS.get(part, part)
            for part in parts
        ]

```

```

    if len(parts) == 1:
        return parts[0]

    first = parts[0]
    rest = "".join(parts[1:])

    if rest:
        return f"{first}_{rest}"

    return first

def date_field_base_name(
    self,
    field_name: str,
) -> str:
    normalized = self.to_db2_name(field_name)

    normalized = self.remove_record_suffix(normalized)

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    if parts[0] == "DA":
        main_parts = parts[1:]
    else:
        main_parts = parts

    if not main_parts:
        return "DA"

    if main_parts[0] in {"UB", "UE"}:
        return f"DA_{main_parts[0]}DATE"

    main_parts = self.apply_date_project_abbreviations(
        parts=main_parts,
    )

    main_parts = self.remove_generic_trailing_tokens(
        parts=main_parts,
    )

    if not main_parts:
        return "DA"

    if len(main_parts) == 1:
        main = main_parts[0]
    else:
        main = "".join(main_parts)

    return f"DA_{main}"

def apply_date_project_abbreviations(
    self,
    parts: list[str],
) -> list[str]:
    output: list[str] = []

    for part in parts:
        if part == "EVPRCP":
            output.append("ERCP")
            continue

        if part == "GDIFAR":
            output.append("GD")
            continue

        if part in {"GDIFC", "GDIFR"}:
            continue

```

```

        output.append(
            self.PROJECT_ABBREVIATIONS.get(part, part)
        )

    return output

def remove_generic_trailing_tokens(
    self,
    parts: list[str],
) -> list[str]:
    output = list(parts or [])

    while output and output[-1] in self.GENERIC_TRAILING_TOKENS:
        output.pop()

    return output

def record_code(
    self,
    record_name: str | None,
) -> str:
    normalized = self.to_db2_name(record_name or "")
    compact = re.sub(r"^[A-Z0-9]", "", normalized)

    if not compact:
        return ""

    if len(compact) <= 4:
        return compact

    return compact[-4:]

def is_date_field(
    self,
    field,
) -> bool:
    datatype = str(
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or "",
    ).strip().upper()

    basetype = str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or "",
    ).strip().upper()

    if datatype == "DATE" or basetype == "DATE":
        return True

    field_name = getattr(field, "name", "") or ""

    if self.is_date_like_name(field_name):
        picture = str(
            getattr(field, "picture", None)
            or getattr(field, "pic", None)
            or getattr(field, "pic_clause", None)
            or "",
        ).strip()

        if self.is_yyyymmdd_picture(picture):
            return True

    return False

def is_yyyymmdd_picture(
    self,
    picture: str,
) -> bool:
    if not picture:
        return False

    clean = DB2DatatypeMapper.clean_picture(
        picture=picture,

```

```

    )
    core = DB2DatatypeMapper.picture_core(
        picture=clean,
    )

    return core in {
        "9(8)",
        "S9(8)",
        "99999999",
        "S99999999",
    }

def is_date_like_name(
    self,
    value: str,
) -> bool:
    parts = self.name_parts(value)

    if not parts:
        return False

    if parts[0] == "DA":
        return True

    if any(part in self.DATE_TOKENS for part in parts):
        return True

    compact = "".join(parts)

    return any(token in compact for token in self.DATE_TOKENS)

def is_date_part_name(
    self,
    value: str,
) -> bool:
    parts = self.name_parts(value)

    for part in parts:
        if part in self.YEAR_PARTS:
            return True

        if part in self.MONTH_PARTS:
            return True

        if part in self.DAY_PARTS:
            return True

    return False

def get_record_primary_keys(
    self,
    record,
) -> list[str]:
    primary_keys: list[str] = []

    explicit_primary_keys = getattr(record, "primary_keys", None)

    if explicit_primary_keys:
        if isinstance(explicit_primary_keys, list):
            primary_keys.extend(explicit_primary_keys)
        else:
            primary_keys.append(explicit_primary_keys)

    primary_key = getattr(record, "primary_key", None)

    if primary_key:
        primary_keys.append(primary_key)

    cleaned: list[str] = []

    for primary_key_value in primary_keys:
        normalized = self.normalize_name(
            primary_key_value,
        )

        if not normalized:

```

```

        continue

    if normalized in cleaned:
        continue

    cleaned.append(normalized)

return cleaned

def get_relationship_set_name(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("set_name")
            or relationship.get("set")
            or relationship.get("name")
            or "",
        )

    return str(
        getattr(relationship, "set_name", None)
        or getattr(relationship, "set", None)
        or getattr(relationship, "name", None)
        or "",
    )

def get_relationship_owner_record(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("owner_record")
            or relationship.get("parent_record")
            or relationship.get("owner")
            or relationship.get("parent")
            or "",
        )

    return str(
        getattr(relationship, "owner_record", None)
        or getattr(relationship, "parent_record", None)
        or getattr(relationship, "owner", None)
        or getattr(relationship, "parent", None)
        or "",
    )

def get_relationship_member_record(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("member_record")
            or relationship.get("child_record")
            or relationship.get("member")
            or relationship.get("child")
            or "",
        )

    return str(
        getattr(relationship, "member_record", None)
        or getattr(relationship, "child_record", None)
        or getattr(relationship, "member", None)
        or getattr(relationship, "child", None)
        or "",
    )

def normalize_name(
    self,
    value: str | None,
) -> str:
    return self.to_db2_name(value or "")

```



```

def to_db2_name(
    self,
    value: str | None,
) -> str:
    text = str(value or "").strip().upper()

    text = text.replace("\u00a0", " ")
    text = text.replace("\t", " ")

    text = re.sub(r"^[A-Z0-9]+", "_", text)
    text = re.sub(r"_"+, "_", text)
    text = text.strip("_")

    return text

def name_parts(
    self,
    value: str | None,
) -> list[str]:
    normalized = self.to_db2_name(value or "")

    return [
        part
        for part in normalized.split("_")
        if part
    ]

def remove_record_suffix(
    self,
    name: str | None,
) -> str:
    value = str(name or "").strip().upper()

    value = re.sub(r"[_-\s]+[0-9]{4}$", "", value)
    value = re.sub(r"[0-9]{4}$", "", value)
    value = re.sub(r"[_-\s]+$", "", value)

    return value

```

```

=====
FILE: src\idms_modernizer\services\db2_model_builder.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import DB2Model
from idms_modernizer.services.db2_mapping_service import DB2MappingService

```

```

class DB2ModelBuilder:

    def __init__(self):
        self.mapping_service = DB2MappingService()

    def build(
        self,
        schema: CanonicalSchema
    ) -> DB2Model:

        return self.mapping_service.build_db2_model(
            schema
        )

```

```

=====
FILE: src\idms_modernizer\services\db2_model_service.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2Model,
    DB2Table
)

```

```
class DB2ModelService:
```

```
    def build(
        self,
        metadata
    ) -> DB2Model:

        tables = []

        for record in metadata.records:

            columns = []

            for field in record.fields:

                columns.append(
                    DB2Column(
                        name=field.name,
                        datatype="VARCHAR(255)"
                    )
                )

            tables.append(
                DB2Table(
                    name=record.name,
                    columns=columns,
                    foreign_keys=[]
                )
            )

        return DB2Model(
            tables=tables
        )
```

```
=====
FILE: src\idms_modernizer\services\db2_name_normalizer.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.services.name_normalizer import NameNormalizer
```

```
class DB2NameNormalizer:
```

```
    REMOVE_RECORD_SUFFIX = False
```

```
    @staticmethod
```

```
    def normalize_record_name(
```

```
        name: str | None
```

```
    ) -> str:
```

```
        return NameNormalizer.normalize(name)
```

```
    @staticmethod
```

```
    def normalize_column_name(
```

```
        name: str | None
```

```
    ) -> str:
```

```
        normalized = NameNormalizer.normalize(name)
```

```
        if not DB2NameNormalizer.REMOVE_RECORD_SUFFIX:
            return normalized
```

```
        parts = normalized.split("_")
```

```
        if (
            len(parts) > 1
            and parts[-1].isdigit()
            and len(parts[-1]) == 4
        ):
            return "_".join(parts[:-1])
```

```
        return normalized
```

```
=====
FILE: src\idms_modernizer\services\document_segmenter.py
ENCODING: utf-8
=====
```

```
import re

from idms_modernizer.domain.section_models import (
    RecordSection
)

class DocumentSegmenter:

    RECORD_PATTERN = re.compile(
        r"RECORD\s+NAME.*?([A-Z0-9\-]+)",
        re.IGNORECASE
    )

    def segment(
        self,
        document
    ) -> list[RecordSection]:

        sections = []

        current_section = None

        for page in document.pages:

            for line in page.lines:

                text = line.text.strip()

                record_match = (
                    self.RECORD_PATTERN.search(
                        text
                    )
                )

                if record_match:

                    if current_section:

                        sections.append(
                            current_section
                        )

                    current_section = (
                        RecordSection(
                            record_name=record_match.group(1),
                            lines=[]
                        )
                    )

                elif current_section:

                    current_section.lines.append(
                        text
                    )

            if current_section:

                sections.append(
                    current_section
                )

        return sections
```

```
=====
FILE: src\idms_modernizer\services\er_diagram_generator.py
ENCODING: utf-8
=====
```

```
import os
from pathlib import Path
from graphviz import Digraph

from idms_modernizer.domain.schema_models import (
    SchemaMetadata
)

def generate_er_diagram(
    metadata: SchemaMetadata,
    output_path: str
) -> str:
    """Generate a simple ER diagram PNG from the canonical metadata.

    Nodes are records and edges are relationships (labeled with set name
    and cardinality if present).
    Returns the path to the generated PNG file.
    """

    out = Path(output_path)
    out.parent.mkdir(parents=True, exist_ok=True)
    os.environ["PATH"] += (
        os.pathsep +
        r"C:\Program Files\Graphviz\bin"
    )

    dot = Digraph("er", format="png")
    dot.attr(rankdir="LR")
    dot.attr("node", shape="box")

    # Add record nodes
    for record in metadata.records:
        # simple label: record name
        dot.node(record.name, label=record.name)

    # Add relationships
    for rel in metadata.relationships:
        owner = rel.owner_record
        member = rel.member_record
        label = rel.set_name
        if getattr(rel, "cardinality", None):
            label = f"{label}\n{rel.cardinality}"

        dot.edge(owner, member, label=label)

    # Render to the requested path (without extension)
    filename_no_ext = str(out.with_suffix(""))

    try:
        dot.render(
            filename=filename_no_ext,
            cleanup=True
        )
    except Exception as ex:
        raise RuntimeError(
            "Graphviz not installed. "
            "Install Graphviz and add "
            "'C:\\Program Files\\Graphviz\\bin' "
            "to PATH."
        ) from ex

    # Graphviz writes the .png next to filename_no_ext
    png_path = filename_no_ext + ".png"

    return png_path
```

```
=====
FILE: src\idms_modernizer\services\excel_sheet_mapping_service.py
ENCODING: utf-8
=====
```

```
import re

from idms_modernizer.domain.db2_models import (
    DB2Model,
    DB2Table,
    DB2Column,
)
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper

print("LOADED ExcelSheetMappingService VERSION FULL-LATEST-CALC-PIC-DATE-FK-NAMING-2026-07-29")

class ExcelSheetMappingService:
    """
    Builds Excel Sheet Mapping rows.

    Rules:
    - Always add level 01 record row if missing.
    - Include all outer/group rows from mapping_fields.
    - Group rows do not create DB2 physical field names.
    - Outer CALC group does not show CALC.
    - Inner groups under CALC also do not show CALC or PK.
    - Only elementary non-date descendants of CALC show CALC and PK.
    - Date groups and date parts inside CALC show neither CALC nor PK.
    - FILLER rows are COBOL-only.
    - Outer date group shows DB2 record and DATE datatype.
    - Date parts remain visible but DB2 mapping stays blank.
    - Sheet Mapping DB2 datatype is calculated directly from IDMS PIC Clause.
    - Field end position = STRT + LGTH - 1.
    - SET/FK rows reuse referenced/master DB2 column names.
    - DB2 field names use project naming rules and _479<record-code>.
    """

    COLUMNS = [
        "Cobol Record IDMS",
        "Cobol Zone",
        "IDMS Key",
        "IDMS PIC Clause",
        "Length of Field Bytes",
        "Field end position",
        "DB2 Key",
        "New DB2 Record",
        "New DB2 Field name",
        "New DB2 Data Type",
        "Hopex Expression TypeRemark",
        "Relation",
        "Reference Field Name (CopyBook) ",
        "Reference Field PIC Clause",
        "Cross Application DB2 Field Name",
        "Cross Appln DB2 Data Type",
        "Basetype",
    ]

    YEAR_PARTS = {
        "YEAR",
        "YR",
        "Y",
        "YY",
        "YYYY",
        "DY",
    }

    MONTH_PARTS = {
        "MONTH",
        "MON",
        "MO",
        "M",
        "MM",
    }
```

```

        "DM",
    }

    DAY_PARTS = {
        "DAY",
        "D",
        "DD",
    }

    DATE_TOKENS = {
        "DATE",
        "DT",
        "DTE",
        "DA",
        "YYMMDD",
        "YYYYMMDD",
        "YMD",
    }

    DATE_CHILD_TOKENS = YEAR_PARTS | MONTH_PARTS | DAY_PARTS

    PROJECT_ABBREVIATIONS = {
        "FORM": "FM",
        "EVPRCP": "ERCP",
    }

    GENERIC_TRAILING_TOKENS = {
        "SIC",
        "FC",
        "GDIFC",
        "GDIFR",
        "GDIFAR",
        "GDIF",
        "GDI",
        "REC",
        "RECORD",
    }

    DATE_TRAILING_TOKENS = {
        "SIC",
        "FC",
        "GDIFC",
        "GDIFR",
        "GDIF",
        "GDI",
    }

    def build(
        self,
        metadata: SchemaMetadata,
        db2_model: DB2Model,
    ) -> list[dict[str, str]]:
        print("USING ExcelSheetMappingService.build VERSION FULL-LATEST-CALC-PIC-DATE-FK-NAMING-2026-07-29")

        rows: list[dict[str, str]] = []

        table_lookup = self.build_table_lookup(
            db2_model=db2_model,
        )

        relationship_lookup = self.build_relationship_lookup(
            metadata=metadata,
        )

        for record in getattr(metadata, "records", []) or []:
            mapping_fields = (
                getattr(record, "mapping_fields", None)
                or getattr(record, "fields", None)
                or []
            )

            if not self.has_level_01_row(
                record=record,
                mapping_fields=mapping_fields,
            ):
                rows.append(

```

```

        self.build_record_level_01_row(
            record=record,
        )
    )

    table = self.find_table_for_record(
        record_name=getattr(record, "name", "") or "",
        table_lookup=table_lookup,
    )

    rows.extend(
        self.build_record_mapping_rows(
            record=record,
            table=table,
            relationship_lookup=relationship_lookup,
        )
    )

    rows.extend(
        self.build_generated_pk_rows(
            record=record,
            table=table,
        )
    )

    rows.extend(
        self.build_audit_rows(
            record=record,
            table=table,
        )
    )

    rows.extend(
        self.build_set_fk_rows_for_record(
            metadata=metadata,
            record=record,
            table=table,
            table_lookup=table_lookup,
        )
    )

    rows = self.force_pic_based_datatypes(
        rows=rows,
    )

    return rows

def build_record_mapping_rows(
    self,
    record,
    table: DB2Table | None,
    relationship_lookup: dict[str, str],
) -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []

    mapping_fields = (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

    column_lookup = (
        self.build_column_lookup(table=table)
        if table is not None
        else {}
    )

    date_scope = self.build_date_scope(
        fields=mapping_fields,
    )

    date_group_info = self.collect_date_group_info(
        fields=mapping_fields,
    )

    calc_scope = self.build_calc_scope(

```

```

        record=record,
        fields=mapping_fields,
    )

for field in mapping_fields:
    if self.is_filler_field(field=field):
        rows.append(
            self.build_filler_row(
                record=record,
                field=field,
            )
        )
        continue

    is_group = self.is_group_field(
        field=field,
    )

    is_outer_date = self.is_outer_date_group(
        field=field,
        date_group_info=date_group_info,
    )

    is_date_child = self.is_date_child(
        field=field,
        date_scope=date_scope,
    )

    is_direct_date = self.is_direct_date_field(
        field=field,
    )

    is_date = (
        is_outer_date
        or is_date_child
        or is_direct_date
    )

    calc_status = self.calc_status_for_field(
        field=field,
        calc_scope=calc_scope,
    )

    db2_column = self.find_column_for_field(
        record=record,
        field=field,
        column_lookup=column_lookup,
    )

    db2_key = self.db2_key_label(
        table=table,
        column=db2_column,
    )

    if calc_status == "ROOT":
        db2_key = ""

    if is_group:
        db2_key = ""

    if calc_status == "DESCENDANT" and not is_date and not is_group:
        db2_key = self.add_pk_to_db2_key(
            db2_key=db2_key,
        )

    if is_date:
        db2_key = ""

    row = self.empty_row()

    row["Cobol Record IDMS"] = getattr(record, "name", "") or ""
    row["Cobol Zone"] = self.format_cobol_zone(field=field)
    row["IDMS Key"] = self.idms_key_label(
        calc_status=calc_status,
        is_date=is_date,
        is_group=is_group,

```



```

        db2_key=db2_key,
    )
    row["IDMS PIC Clause"] = self.get_field_picture(field=field)
    row["Length of Field Bytes"] = self.to_string(
        self.get_field_length(field=field)
    )
    row["Field end position"] = self.to_string(
        self.get_field_end_position(field=field)
    )
    row["DB2 Key"] = db2_key

    if is_outer_date:
        row["New DB2 Record"] = self.db2_record_name(
            record=record,
            table=table,
        )
        row["New DB2 Field name"] = self.convert_date_field_to_db2_field_name(
            record=record,
            field=field,
        )
        row["New DB2 Data Type"] = "DATE"
        row["DB2 Key"] = ""

    elif is_date_child:
        row["New DB2 Record"] = ""
        row["New DB2 Field name"] = ""
        row["New DB2 Data Type"] = ""
        row["DB2 Key"] = ""

    elif is_direct_date:
        row["New DB2 Record"] = self.db2_record_name(
            record=record,
            table=table,
        )
        row["New DB2 Field name"] = self.convert_date_field_to_db2_field_name(
            record=record,
            field=field,
        )
        row["New DB2 Data Type"] = "DATE"
        row["DB2 Key"] = ""

    elif calc_status == "ROOT":
        row["New DB2 Record"] = ""
        row["New DB2 Field name"] = ""
        row["New DB2 Data Type"] = ""
        row["DB2 Key"] = ""

    elif is_group:
        row["New DB2 Record"] = ""
        row["New DB2 Field name"] = ""
        row["New DB2 Data Type"] = ""
        row["DB2 Key"] = ""

    elif self.should_emit_db2_field(field=field):
        row["New DB2 Record"] = self.db2_record_name(
            record=record,
            table=table,
        )

        if db2_column is not None and getattr(db2_column, "name", None):
            row["New DB2 Field name"] = getattr(db2_column, "name", "") or ""
        else:
            row["New DB2 Field name"] = self.convert_cobol_zone_to_db2_field_name(
                record=record,
                field=field,
            )

        row["New DB2 Data Type"] = self.infer_db2_datatype_from_field(
            field=field,
        )

    row["Hopex Expression TypeRemark"] = self.hopex_remark(
        field=field,
    )
    row["Relation"] = relationship_lookup.get(
        NameNormalizer.normalize(getattr(record, "name", "") or ""),

```

```

        "",
    )
    row["Basetype"] = self.get_field_basetype(
        field=field,
    )

    rows.append(row)

return rows

def build_calc_scope(
    self,
    record,
    fields,
) -> dict[str, str]:
    scope: dict[str, str] = {}

    primary_key = getattr(record, "primary_key", None)

    if not primary_key:
        return scope

    normalized_primary_key = NameNormalizer.normalize(primary_key)
    suffix_removed_primary_key = self.remove_record_suffix(
        normalized_primary_key,
    )

    mapping_fields = list(fields or [])

    key_field = None
    key_index = None

    for index, field in enumerate(mapping_fields):
        field_name = NameNormalizer.normalize(
            getattr(field, "name", "") or ""
        )

        if not field_name:
            continue

        if field_name == normalized_primary_key:
            key_field = field
            key_index = index
            break

        if self.remove_record_suffix(field_name) == suffix_removed_primary_key:
            key_field = field
            key_index = index
            break

    if key_field is None or key_index is None:
        if normalized_primary_key:
            scope[normalized_primary_key] = "DESCENDANT"
        return scope

    key_name = NameNormalizer.normalize(
        getattr(key_field, "name", "") or ""
    )

    if not key_name:
        return scope

    key_is_group = self.is_group_field(
        field=key_field,
    )

    key_level = getattr(key_field, "level", None)

    if not key_is_group:
        scope[key_name] = "DESCENDANT"
        return scope

    scope[key_name] = "ROOT"

    if key_level is None:
        return scope

```

```

try:
    key_level_int = int(key_level)
except Exception:
    return scope

for child_field in mapping_fields[key_index + 1:]:
    child_name = NameNormalizer.normalize(
        getattr(child_field, "name", "") or ""
    )
    child_level = getattr(child_field, "level", None)

    if child_level is None:
        continue

    try:
        child_level_int = int(child_level)
    except Exception:
        continue

    if child_level_int <= key_level_int:
        break

    if child_name:
        scope[child_name] = "DESCENDANT"

return scope

def calc_status_for_field(
    self,
    field,
    calc_scope: dict[str, str],
) -> str:
    field_name = NameNormalizer.normalize(
        getattr(field, "name", "") or ""
    )

    if not field_name:
        return ""

    if field_name in calc_scope:
        return calc_scope[field_name]

    suffix_removed = self.remove_record_suffix(field_name)

    for key, value in calc_scope.items():
        if self.remove_record_suffix(key) == suffix_removed:
            return value

    return ""

def idms_key_label(
    self,
    calc_status: str,
    is_date: bool,
    is_group: bool,
    db2_key: str,
) -> str:
    if is_date:
        return ""

    if is_group:
        return ""

    labels: list[str] = []

    if calc_status == "DESCENDANT":
        labels.append("CALC")

    db2_key_parts = [
        part.strip().upper()
        for part in str(db2_key or "").split("/")
        if part.strip()
    ]

    if "FK" in db2_key_parts:

```

```

        labels.append("SET")

    output: list[str] = []

    for label in labels:
        if label not in output:
            output.append(label)

    return "; ".join(output)

def add_pk_to_db2_key(
    self,
    db2_key: str,
) -> str:
    parts = [
        part.strip().upper()
        for part in str(db2_key or "").split("/")
        if part.strip()
    ]

    if "PK" not in parts:
        parts.insert(0, "PK")

    ordered_parts: list[str] = []

    if "PK" in parts:
        ordered_parts.append("PK")

    if "FK" in parts:
        ordered_parts.append("FK")

    return "/".join(ordered_parts)

def force_pic_based_datatypes(
    self,
    rows: list[dict[str, str]],
) -> list[dict[str, str]]:
    corrected_rows: list[dict[str, str]] = []

    for row in rows or []:
        pic_clause = str(row.get("IDMS PIC Clause", "") or "").strip()
        db2_record = str(row.get("New DB2 Record", "") or "").strip()
        db2_field = str(row.get("New DB2 Field name", "") or "").strip()
        current_type = str(row.get("New DB2 Data Type", "") or "").strip().upper()

        if current_type == "DATE":
            corrected_rows.append(row)
            continue

        if pic_clause and db2_record and db2_field:
            row["New DB2 Data Type"] = DB2DatatypeMapper.map_picture(
                picture=pic_clause,
                fallback_length=None,
                fallback_scale=None,
            )

        corrected_rows.append(row)

    return corrected_rows

def build_table_lookup(
    self,
    db2_model: DB2Model,
) -> dict[str, DB2Table]:
    lookup: dict[str, DB2Table] = {}

    if db2_model is None:
        return lookup

    for table in getattr(db2_model, "tables", []) or []:
        table_name = getattr(table, "name", "") or ""

        for key in self.name_lookup_keys(table_name):
            lookup[key] = table

    return lookup

```

```

def build_column_lookup(
    self,
    table: DB2Table,
) -> dict[str, DB2Column]:
    lookup: dict[str, DB2Column] = {}

    for column in getattr(table, "columns", []) or []:
        column_name = getattr(column, "name", "") or ""

        for key in self.name_lookup_keys(column_name):
            lookup[key] = column

    return lookup

def build_relationship_lookup(
    self,
    metadata: SchemaMetadata,
) -> dict[str, str]:
    lookup: dict[str, str] = {}

    for relationship in getattr(metadata, "relationships", []) or []:
        owner_record = self.get_relationship_owner_record(relationship=relationship)
        member_record = self.get_relationship_member_record(relationship=relationship)
        set_name = self.get_relationship_set_name(relationship=relationship)

        for record_name in [owner_record, member_record]:
            normalized_record = NameNormalizer.normalize(record_name)

            if not normalized_record:
                continue

            existing = lookup.get(normalized_record, "")

            if existing:
                if set_name and set_name not in existing:
                    lookup[normalized_record] = f"{existing}; {set_name}"
            else:
                lookup[normalized_record] = set_name

    return lookup

def build_date_scope(
    self,
    fields,
) -> dict[int, bool]:
    scope: dict[int, bool] = {}
    active_date_levels: list[int] = []

    for field in fields or []:
        level = self.safe_int(getattr(field, "level", None), default=0)

        active_date_levels = [
            current_level
            for current_level in active_date_levels
            if current_level < level
        ]

        if active_date_levels:
            scope[id(field)] = True

        if self.is_potential_outer_date(field=field):
            active_date_levels.append(level)

    return scope

def collect_date_group_info(
    self,
    fields,
) -> dict[str, dict]:
    info: dict[str, dict] = {}

    for index, field in enumerate(fields or []):
        if not self.is_group_field(field=field):
            continue

```

```

        if not self.is_potential_outer_date(field=field):
            continue

        if not self.group_has_ymd_children(
            field=field,
            fields=fields,
            start_index=index,
        ):
            continue

        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")

        if field_name:
            info[field_name] = {"field": field}

    return info

def group_has_ymd_children(
    self,
    field,
    fields,
    start_index: int,
) -> bool:
    parent_level = self.safe_int(getattr(field, "level", None), default=0)

    found_year = False
    found_month = False
    found_day = False

    for child in list(fields or [])[start_index + 1:]:
        child_level = self.safe_int(getattr(child, "level", None), default=0)

        if child_level <= parent_level:
            break

        child_name = getattr(child, "name", "") or ""
        parts = self.name_parts(child_name)

        if any(part in self.YEAR_PARTS for part in parts):
            found_year = True

        if any(part in self.MONTH_PARTS for part in parts):
            found_month = True

        if any(part in self.DAY_PARTS for part in parts):
            found_day = True

    return found_year and found_month and found_day

def is_outer_date_group(
    self,
    field,
    date_group_info: dict[str, dict],
) -> bool:
    if not self.is_group_field(field=field):
        return False

    field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")

    return field_name in date_group_info

def is_date_child(
    self,
    field,
    date_scope: dict[int, bool],
) -> bool:
    if id(field) not in date_scope:
        return False

    return self.is_date_part_name(field_name=getattr(field, "name", "") or "")

def is_direct_date_field(
    self,
    field,
) -> bool:
    if self.is_group_field(field=field):

```

```

        return False

    datatype = str(
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or "",
    ).strip().upper()

    basetype = str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or "",
    ).strip().upper()

    if datatype == "DATE" or basetype == "DATE":
        return not self.is_date_part_name(field_name=getattr(field, "name", "") or "")

    pic_clause = self.get_field_picture(field=field)

    if self.is_date_like_name(getattr(field, "name", "") or "") and self.is_yyyymmdd_picture(pic_clause):
        return True

    return False

def is_potential_outer_date(
    self,
    field,
) -> bool:
    field_name = getattr(field, "name", "") or ""

    if self.is_date_like_name(field_name):
        return True

    datatype = str(
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or "",
    ).strip().upper()

    basetype = str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or "",
    ).strip().upper()

    return datatype == "DATE" or basetype == "DATE"

def is_date_part_name(
    self,
    field_name: str,
) -> bool:
    parts = self.name_parts(field_name or "")

    return any(part in self.DATE_CHILD_TOKENS for part in parts)

def is_yyyymmdd_picture(
    self,
    pic_clause: str,
) -> bool:
    clean = DB2DatatypeMapper.clean_picture(pic_clause)
    core = DB2DatatypeMapper.picture_core(clean)

    return core in {"9(8)", "S9(8)", "99999999", "S99999999"}

def is_date_like_name(
    self,
    value: str,
) -> bool:
    parts = self.name_parts(value or "")

    if any(part in self.DATE_TOKENS for part in parts):
        return True

    if parts and parts[0] == "DA":

```

```

        return True

compact = "".join(parts)

return any(token in compact for token in self.DATE_TOKENS)

def find_table_for_record(
    self,
    record_name: str,
    table_lookup: dict[str, DB2Table],
) -> DB2Table | None:
    for key in self.name_lookup_keys(record_name):
        if key in table_lookup:
            return table_lookup[key]

    return None

def find_column_for_field(
    self,
    record,
    field,
    column_lookup: dict[str, DB2Column],
) -> DB2Column | None:
    field_name = getattr(field, "name", "") or ""

    if not field_name:
        return None

    possible_names = [
        field_name,
        self.remove_record_suffix(field_name),
        self.cobol_field_base_name(field_name=field_name),
        self.date_field_base_name(field_name=field_name),
        self.convert_cobol_zone_to_db2_field_name(record=record, field=field),
        self.convert_date_field_to_db2_field_name(record=record, field=field),
    ]

    for name in possible_names:
        column = self.find_matching_column(
            normalized_column_name=name,
            column_lookup=column_lookup,
        )

        if column is not None:
            return column

    return None

def find_matching_column(
    self,
    normalized_column_name: str,
    column_lookup: dict[str, DB2Column],
) -> DB2Column | None:
    for key in self.name_lookup_keys(normalized_column_name):
        if key in column_lookup:
            return column_lookup[key]

    suffix_removed = self.remove_record_suffix(normalized_column_name)

    for key in self.name_lookup_keys(suffix_removed):
        if key in column_lookup:
            return column_lookup[key]

    target_suffix_removed = self.remove_record_suffix(
        self.to_db2_name(normalized_column_name)
    )

    for current_name, column in column_lookup.items():
        if self.remove_record_suffix(current_name) == target_suffix_removed:
            return column

    return None

def build_filler_row(
    self,
    record,

```



```

        field,
    ) -> dict[str, str]:
        row = self.empty_row()

        row["Cobol Record IDMS"] = getattr(record, "name", "") or ""
        row["Cobol Zone"] = self.format_cobol_zone(field=field)
        row["IDMS PIC Clause"] = self.get_field_picture(field=field)
        row["Length of Field Bytes"] = self.to_string(self.get_field_length(field=field))
        row["Field end position"] = self.to_string(self.get_field_end_position(field=field))

        return row

def build_generated_pk_rows(
    self,
    record,
    table: DB2Table | None,
) -> list[dict[str, str]]:
    if table is None:
        return []

    rows: list[dict[str, str]] = []

    for column in getattr(table, "columns", []) or []:
        if getattr(column, "source_kind", "") != "GENERATED PK":
            continue

        row = self.empty_row()
        row["Cobol Record IDMS"] = getattr(record, "name", "") or ""
        row["DB2 Key"] = "PK"
        row["New DB2 Record"] = getattr(table, "name", "") or ""
        row["New DB2 Field name"] = getattr(column, "name", "") or ""
        row["New DB2 Data Type"] = self.get_db2_datatype(db2_column=column)
        rows.append(row)

    return rows

def build_audit_rows(
    self,
    record,
    table: DB2Table | None,
) -> list[dict[str, str]]:
    if table is None:
        return []

    rows: list[dict[str, str]] = []
    record_name = getattr(record, "name", "") or ""
    normalized_record_name = self.to_db2_name(record_name)

    audit_fields = [
        (f"TS_CREATE_{normalized_record_name}", "TIMESTAMP"),
        (f"TS_UPDATE_{normalized_record_name}", "TIMESTAMP"),
        (f"ID_USERID_{normalized_record_name}", "CHAR(8)"),
    ]

    for field_name, datatype in audit_fields:
        row = self.empty_row()
        row["Cobol Record IDMS"] = record_name
        row["New DB2 Record"] = getattr(table, "name", "") or ""
        row["New DB2 Field name"] = field_name
        row["New DB2 Data Type"] = datatype
        rows.append(row)

    return rows

def build_set_fk_rows_for_record(
    self,
    metadata: SchemaMetadata,
    record,
    table: DB2Table | None,
    table_lookup: dict[str, DB2Table],
) -> list[dict[str, str]]:
    if table is None:
        return []

    rows: list[dict[str, str]] = []
    emitted: set[tuple[str, str, str]] = set()

```

```

rows.extend(
    self.build_set_fk_rows_from_db2_foreign_keys(
        record=record,
        table=table,
        emitted=emitted,
    )
)

rows.extend(
    self.build_set_fk_rows_from_metadata_relationships(
        metadata=metadata,
        record=record,
        member_table=table,
        table_lookup=table_lookup,
        emitted=emitted,
    )
)

return rows

def build_set_fk_rows_from_db2_foreign_keys(
    self,
    record,
    table: DB2Table,
    emitted: set[tuple[str, str, str, str]],
) -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []
    cobol_record_name = getattr(record, "name", "") or ""

    column_lookup = self.build_column_lookup(table=table)

    for foreign_key in getattr(table, "foreign_keys", []) or []:
        fk_column_name = self.get_foreign_key_column_name(foreign_key=foreign_key)

        if not fk_column_name:
            continue

        fk_column = self.find_matching_column(
            normalized_column_name=fk_column_name,
            column_lookup=column_lookup,
        )

        if fk_column is None:
            continue

        fk_datatype = self.get_db2_datatype(db2_column=fk_column)
        set_name = getattr(foreign_key, "set_name", "") or ""

        key = (
            NameNormalizer.normalize(cobol_record_name),
            NameNormalizer.normalize(getattr(table, "name", "") or ""),
            NameNormalizer.normalize(getattr(fk_column, "name", "") or ""),
            fk_datatype,
        )

        if key in emitted:
            continue

        emitted.add(key)

        row = self.empty_row()
        row["Cobol Record IDMS"] = cobol_record_name
        row["IDMS Key"] = "SET"
        row["DB2 Key"] = "FK"
        row["New DB2 Record"] = getattr(table, "name", "") or ""
        row["New DB2 Field name"] = getattr(fk_column, "name", "") or ""
        row["New DB2 Data Type"] = fk_datatype
        row["Relation"] = set_name
        rows.append(row)

    return rows

def build_set_fk_rows_from_metadata_relationships(
    self,
    metadata: SchemaMetadata,

```

```

record,
member_table: DB2Table,
table_lookup: dict[str, DB2Table],
emitted: set[tuple[str, str, str, str]],
) -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []

    cobol_record_name = getattr(record, "name", "") or ""
    normalized_current_record = NameNormalizer.normalize(cobol_record_name)
    normalized_current_table = NameNormalizer.normalize(getattr(member_table, "name", "") or "")

    for relationship in getattr(metadata, "relationships", []) or []:
        set_name = self.get_relationship_set_name(relationship=relationship)
        owner_record_name = self.get_relationship_owner_record(relationship=relationship)
        member_record_name = self.get_relationship_member_record(relationship=relationship)

        if not set_name:
            continue

        normalized_member_record = NameNormalizer.normalize(member_record_name)

        if normalized_member_record not in {normalized_current_record, normalized_current_table}:
            continue

        owner_table = self.find_table_for_record(
            record_name=owner_record_name,
            table_lookup=table_lookup,
        )

        owner_pk_column = self.find_primary_key_column(table=owner_table)

        if owner_pk_column is None:
            continue

        fk_column_name = getattr(owner_pk_column, "name", "") or ""
        fk_datatype = self.get_db2_datatype(db2_column=owner_pk_column)

        key = (
            NameNormalizer.normalize(cobol_record_name),
            NameNormalizer.normalize(getattr(member_table, "name", "") or ""),
            NameNormalizer.normalize(fk_column_name),
            fk_datatype,
        )

        if key in emitted:
            continue

        emitted.add(key)

        row = self.empty_row()
        row["Cobol Record IDMS"] = cobol_record_name
        row["IDMS Key"] = "SET"
        row["DB2 Key"] = "FK"
        row["New DB2 Record"] = getattr(member_table, "name", "") or ""
        row["New DB2 Field name"] = fk_column_name
        row["New DB2 Data Type"] = fk_datatype
        row["Relation"] = set_name
        rows.append(row)

    return rows

def find_primary_key_column(
    self,
    table: DB2Table | None,
) -> DB2Column | None:
    if table is None:
        return None

    primary_key_names = list(getattr(table, "primary_keys", []) or [])

    if not primary_key_names and getattr(table, "primary_key", None):
        primary_key_names = [table.primary_key]

    normalized_primary_keys = {
        NameNormalizer.normalize(primary_key)
        for primary_key in primary_key_names
    }

```

```

        if primary_key
    }

    for column in getattr(table, "columns", []) or []:
        column_name = NameNormalizer.normalize(getattr(column, "name", "") or "")

        if column_name in normalized_primary_keys:
            return column

        if getattr(column, "primary_key", False):
            return column

    columns = list(getattr(table, "columns", []) or [])

    return columns[0] if columns else None

def get_foreign_key_column_name(
    self,
    foreign_key,
) -> str:
    if isinstance(foreign_key, dict):
        return str(
            foreign_key.get("column_name")
            or foreign_key.get("column")
            or foreign_key.get("fk_column")
            or foreign_key.get("foreign_key_column")
            or "",
        )

    return str(
        getattr(foreign_key, "column_name", None)
        or getattr(foreign_key, "column", None)
        or getattr(foreign_key, "fk_column", None)
        or getattr(foreign_key, "foreign_key_column", None)
        or "",
    )

def get_relationship_set_name(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("set_name")
            or relationship.get("set")
            or relationship.get("name")
            or "",
        )

    return str(
        getattr(relationship, "set_name", None)
        or getattr(relationship, "set", None)
        or getattr(relationship, "name", None)
        or "",
    )

def get_relationship_owner_record(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("owner_record")
            or relationship.get("parent_record")
            or relationship.get("owner")
            or relationship.get("parent")
            or "",
        )

    return str(
        getattr(relationship, "owner_record", None)
        or getattr(relationship, "parent_record", None)
        or getattr(relationship, "owner", None)
        or getattr(relationship, "parent", None)
        or "",
    )

```

```

def get_relationship_member_record(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("member_record")
            or relationship.get("child_record")
            or relationship.get("member")
            or relationship.get("child")
            or "",
        )

    return str(
        getattr(relationship, "member_record", None)
        or getattr(relationship, "child_record", None)
        or getattr(relationship, "member", None)
        or getattr(relationship, "child", None)
        or "",
    )

def get_db2_datatype(
    self,
    db2_column: DB2Column | None,
) -> str:
    if db2_column is None:
        return ""

    return str(
        getattr(db2_column, "datatype", None)
        or getattr(db2_column, "data_type", None)
        or getattr(db2_column, "type", None)
        or "",
    ).strip().upper()

def db2_key_label(
    self,
    table: DB2Table | None,
    column: DB2Column | None,
) -> str:
    if table is None or column is None:
        return ""

    labels: list[str] = []

    if self.is_primary_key_column(table=table, column=column):
        labels.append("PK")

    if self.is_foreign_key_column(table=table, column=column):
        labels.append("FK")

    return "/".join(labels)

def is_primary_key_column(
    self,
    table: DB2Table,
    column: DB2Column,
) -> bool:
    if getattr(column, "primary_key", False):
        return True

    column_name = NameNormalizer.normalize(getattr(column, "name", "") or "")
    primary_keys = list(getattr(table, "primary_keys", []) or [])

    if not primary_keys and getattr(table, "primary_key", None):
        primary_keys = [table.primary_key]

    normalized_primary_keys = {
        NameNormalizer.normalize(primary_key)
        for primary_key in primary_keys
        if primary_key
    }

    return column_name in normalized_primary_keys

```

```

def is_foreign_key_column(
    self,
    table: DB2Table,
    column: DB2Column,
) -> bool:
    column_name = NameNormalizer.normalize(getattr(column, "name", "") or "")

    for foreign_key in getattr(table, "foreign_keys", []) or []:
        fk_column_name = NameNormalizer.normalize(self.get_foreign_key_column_name(foreign_key))

        if fk_column_name == column_name:
            return True

    source_kind = str(getattr(column, "source_kind", "") or "").upper()

    return source_kind in {"SET_FK", "FK", "FOREIGN KEY"}

def should_emit_db2_field(
    self,
    field,
) -> bool:
    if self.is_filler_field(field=field):
        return False

    if self.is_group_field(field=field):
        return False

    if self.is_date_part_name(field_name=getattr(field, "name", "") or ""):
        return False

    picture = self.get_field_picture(field=field)

    if picture:
        return True

    datatype = str(getattr(field, "datatype", "") or "").upper()

    return datatype in {"CHAR", "VARCHAR", "DECIMAL", "NUMERIC", "DATE"}

def is_filler_field(
    self,
    field,
) -> bool:
    return NameNormalizer.normalize(getattr(field, "name", "") or "") == "FILLER"

def is_group_field(
    self,
    field,
) -> bool:
    return bool(getattr(field, "has_child", False) or getattr(field, "is_group", False))

def db2_record_name(
    self,
    record,
    table: DB2Table | None,
) -> str:
    if table is not None and getattr(table, "name", None):
        return getattr(table, "name", "") or ""

    return self.to_db2_name(getattr(record, "name", "") or "")

def convert_cobol_zone_to_db2_field_name(
    self,
    record,
    field,
) -> str:
    field_name = getattr(field, "name", "") or ""
    base = self.cobol_field_base_name(field_name=field_name)
    record_code = self.record_code(record_name=getattr(record, "name", "") or "")

    if record_code:
        return f"{base}_479{record_code}"

    return base

def convert_date_field_to_db2_field_name(

```

```

        self,
        record,
        field,
    ) -> str:
        field_name = getattr(field, "name", "") or ""
        base = self.date_field_base_name(field_name=field_name)
        record_code = self.record_code(record_name=getattr(record, "name", "") or "")

        if record_code:
            return f"{base}_479{record_code}"

        return base

def cobol_field_base_name(
    self,
    field_name: str,
) -> str:
    normalized = self.to_db2_name(field_name)
    normalized = self.remove_record_suffix(normalized)

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    parts = self.remove_generic_trailing_tokens(parts=parts)

    parts = [
        self.PROJECT_ABBREVIATIONS.get(part, part)
        for part in parts
    ]

    if len(parts) == 1:
        return parts[0]

    first = parts[0]
    rest = "".join(parts[1:])

    if rest:
        return f"{first}_{rest}"

    return first

def date_field_base_name(
    self,
    field_name: str,
) -> str:
    normalized = self.to_db2_name(field_name)
    normalized = self.remove_record_suffix(normalized)

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    if parts[0] == "DA":
        main_parts = parts[1:]
    else:
        main_parts = parts

    if not main_parts:
        return "DA"

    if main_parts[0] in {"UB", "UE"}:
        return f"DA_{main_parts[0]}DATE"

    main_parts = self.apply_date_project_abbreviations(
        parts=main_parts,

```

```

    )

    main_parts = self.remove_generic_trailing_tokens(
        parts=main_parts,
    )

    if not main_parts:
        return "DA"

    if len(main_parts) == 1:
        main = main_parts[0]
    else:
        main = "".join(main_parts)

    return f"DA_{main}"

def apply_date_project_abbreviations(
    self,
    parts: list[str],
) -> list[str]:
    output: list[str] = []

    for part in parts:
        if part == "EVPRCP":
            output.append("ERCP")
            continue

        if part == "GDIFAR":
            output.append("GD")
            continue

        if part in {"GDIFC", "GDIFR"}:
            continue

        output.append(
            self.PROJECT_ABBREVIATIONS.get(part, part)
        )

    return output

def remove_generic_trailing_tokens(
    self,
    parts: list[str],
) -> list[str]:
    output = list(parts or [])

    while output and output[-1] in self.GENERIC_TRAILING_TOKENS:
        output.pop()

    return output

def record_code(
    self,
    record_name: str,
) -> str:
    normalized = self.to_db2_name(record_name)
    compact = re.sub(r"^[A-Z0-9]", "", normalized)

    if len(compact) <= 4:
        return compact

    return compact[-4:]

def get_field_picture(
    self,
    field,
) -> str:
    if self.is_group_field(field=field):
        return ""

    text = str(
        getattr(field, "picture", None)
        or getattr(field, "pic", None)
        or getattr(field, "pic_clause", None)
        or "",
    ).strip()

```



```

    if not text:
        return ""

    if text.upper().startswith("PIC "):
        return text

    if text.upper().startswith("PICTURE "):
        return "PIC " + text[8:].strip()

    return f"PIC {text}"

def get_field_length(
    self,
    field,
):
    return (
        getattr(field, "length", None)
        or getattr(field, "storage_length", None)
        or getattr(field, "physical_length", None)
        or getattr(field, "byte_length", None)
        or getattr(field, "bytes", None)
    )

def get_field_end_position(
    self,
    field,
):
    start = (
        getattr(field, "start_position", None)
        or getattr(field, "start", None)
    )

    length = self.get_field_length(field=field)

    try:
        if start is not None and length not in ("", None):
            return int(start) + int(length) - 1
    except Exception:
        pass

    end_position = (
        getattr(field, "end_position", None)
        or getattr(field, "field_end_position", None)
        or getattr(field, "end", None)
    )

    if end_position is not None:
        return end_position

    return ""

def infer_db2_datatype_from_field(
    self,
    field,
) -> str:
    pic_clause = self.get_field_picture(field=field)

    if pic_clause:
        return DB2DatatypeMapper.map_picture(
            picture=pic_clause,
            fallback_length=None,
            fallback_scale=getattr(field, "scale", None),
        )

    datatype = str(getattr(field, "datatype", "") or "").upper()

    if datatype == "DATE":
        return "DATE"

    if datatype in {"TIMESTAMP", "DATETIME"}:
        return "TIMESTAMP"

    length = self.get_field_length(field=field) or 1

    if datatype in {"CHAR", "VARCHAR", "DISPLAY", "TEXT"}:

```

```

        return f"CHAR({int(length)})"

    if datatype in {"DECIMAL", "NUMERIC"}:
        scale = getattr(field, "scale", None)

        if scale is not None and int(scale) > 0:
            return f"DECIMAL({int(length)},{int(scale)})"

        return f"DECIMAL({int(length)})"

    return ""

def get_field_basetype(
    self,
    field,
) -> str:
    return str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or getattr(field, "datatype", None)
        or "",
    )

def hopex_remark(
    self,
    field,
) -> str:
    if getattr(field, "occurs", False):
        occurs_min = getattr(field, "occurs_min", None)
        occurs_max = getattr(field, "occurs_max", None)

        if occurs_min is not None and occurs_max is not None:
            if occurs_min == occurs_max:
                return f"OCCURS {occurs_max}"

            return f"OCCURS {occurs_min} TO {occurs_max}"

    return ""

def has_level_01_row(
    self,
    record,
    mapping_fields,
) -> bool:
    record_name = NameNormalizer.normalize(getattr(record, "name", "") or "")

    for field in mapping_fields or []:
        level = getattr(field, "level", None)

        try:
            if int(level) != 1:
                continue
        except Exception:
            continue

        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")

        if field_name == record_name:
            return True

        if self.remove_record_suffix(field_name) == self.remove_record_suffix(record_name):
            return True

    return False

def build_record_level_01_row(
    self,
    record,
) -> dict[str, str]:
    row = self.empty_row()

    record_name = getattr(record, "name", "") or ""

    row["Cobol Record IDMS"] = record_name
    row["Cobol Zone"] = self.format_cobol_level_and_name(level=1, name=record_name)

```

```

        return row

def name_lookup_keys(
    self,
    value: str,
) -> list[str]:
    keys: list[str] = []

    candidates = [
        value,
        NameNormalizer.normalize(value or ""),
        self.to_db2_name(value or ""),
        self.remove_record_suffix(value or ""),
        self.remove_record_suffix(NameNormalizer.normalize(value or "")),
        self.remove_record_suffix(self.to_db2_name(value or "")),
    ]

    for candidate in candidates:
        candidate_text = str(candidate or "").strip()

        if not candidate_text:
            continue

        normalized_name = NameNormalizer.normalize(candidate_text)
        db2_name = self.to_db2_name(candidate_text)

        for key in [candidate_text, normalized_name, db2_name]:
            key = str(key or "").strip()

            if key and key not in keys:
                keys.append(key)

    return keys

def format_cobol_zone(
    self,
    field,
) -> str:
    level = getattr(field, "level", None)
    name = getattr(field, "name", "") or ""

    return self.format_cobol_level_and_name(level=level, name=name)

def format_cobol_level_and_name(
    self,
    level,
    name: str,
) -> str:
    level_text = self.to_string(level)

    if level_text:
        try:
            level_text = f"{int(level_text):02d}"
        except Exception:
            pass

    if level_text and name:
        return f"{level_text} {name}"

    if name:
        return str(name)

    return level_text

def remove_record_suffix(
    self,
    name: str,
) -> str:
    value = str(name or "").strip().upper()
    value = re.sub(r"[_\-\s]{4}$", "", value)
    value = re.sub(r"[0-9]{4}$", "", value)
    value = re.sub(r"[_\-\s]+$", "", value)

    return value

def to_db2_name(

```

```

        self,
        value: str,
    ) -> str:
        normalized = str(value or "").upper().strip()
        normalized = re.sub(r"^[A-Z0-9]+", "_", normalized)
        normalized = re.sub(r"_+", "_", normalized)
        normalized = normalized.strip("_")

        return normalized

    def name_parts(
        self,
        value: str,
    ) -> list[str]:
        normalized = self.to_db2_name(value or "")

        return [part for part in normalized.split("_") if part]

    def empty_row(
        self,
    ) -> dict[str, str]:
        return {column: "" for column in self.COLUMNS}

    def to_string(
        self,
        value,
    ) -> str:
        if value is None:
            return ""

        return str(value)

    def safe_int(
        self,
        value,
        default: int = 0,
    ) -> int:
        try:
            if value is None:
                return default

            text = str(value).strip()

            if not text:
                return default

            return int(text)
        except Exception:
            return default

```

```

=====
FILE: src\idms_modernizer\services\metadata_builder.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.domain.document_models import (
    DocumentModel
)

from idms_modernizer.domain.schema_models import (
    DataField,
    Record,
    SchemaMetadata,
)

class MetadataBuilder:

    RECORD_PATTERN = re.compile(
        r"RECORD\s+NAME.*?([A-Z][A-Z0-9\-\-]+)",
        re.IGNORECASE
    )

```

```

FIELD_PATTERN = re.compile(
    r"^\d{2}\s+([A-Z][A-Z0-9-]+)",
    re.IGNORECASE
)

def build(
    self,
    document: DocumentModel
) -> SchemaMetadata:

    metadata = SchemaMetadata()

    current_record = None

    seen_records = set()

    for page in document.pages:

        for line in page.lines:

            text = line.text.strip()

            if not text:
                continue

            #
            # RECORD DETECTION
            #

            record_match = (
                self.RECORD_PATTERN.search(
                    text
                )
            )

            if record_match:

                record_name = (
                    record_match.group(1)
                    .strip()
                    .upper()
                )

                if (
                    record_name
                    not in seen_records
                ):

                    current_record = Record(
                        name=record_name
                    )

                    metadata.records.append(
                        current_record
                    )

                    seen_records.add(
                        record_name
                    )

                continue

            #
            # FIELD DETECTION
            #

            if current_record:

                field_match = (
                    self.FIELD_PATTERN.match(
                        text
                    )
                )

                if field_match:

```

```

        field_name = (
            field_match.group(1)
            .strip()
            .upper()
        )

        current_record.fields.append(
            DataField(
                name=field_name
            )
        )

#
# DEBUG OUTPUT
#

print(
    "\n===== RECORDS ====="
)

for record in metadata.records:

    print(
        f"{record.name} -> "
        f"{len(record.fields)} fields"
    )

print(
    "=====\n"
)

return metadata

```

```

=====
FILE: src\idms_modernizer\services\metadata_service.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.parsers.text_extractor import TextExtractor
from idms_modernizer.services.document_segmenter import DocumentSegmenter
from idms_modernizer.extractors.owner_member_extractor import OwnerMemberExtractor
from idms_modernizer.extractors.field_extractor import FieldExtractor
from idms_modernizer.extractors.cobol_zone_extractor import CobolZoneExtractor
from idms_modernizer.services.schema_picture_enricher import SchemaPictureEnricher
from idms_modernizer.services.relationship_resolver import RelationshipResolver
from idms_modernizer.domain.schema_models import SchemaMetadata, Record
from idms_modernizer.extractors.primary_key_extractor import PrimaryKeyExtractor

```

```

print("LOADED MetadataService VERSION ENRICH-BOTH-FIELDS-MAPPING-FIELDS-2026-07-30")

```

```

class MetadataService:
    """
    Builds SchemaMetadata from the schema PDF.

    Critical rule:
    - record.fields is used for physical DB2 / DDL / canonical schema.
    - record.mapping_fields is used for Excel Sheet Mapping.
    - BOTH must be enriched with:
      - PIC clause
      - datatype
      - length
      - scale
      - start_position
      - end_position
      - basetype

    Root cause fixed:
    - Excel Sheet Mapping reads record.mapping_fields first.
    - If mapping_fields is not enriched, Excel will show missing/wrong
      IDMS PIC Clause even if record.fields is correct.
    """

    def __init__(self):

```

```

self.extractor = TextExtractor()
self.segmenter = DocumentSegmenter()
self.field_extractor = FieldExtractor()
self.picture_enricher = SchemaPictureEnricher()
self.membership_extractor = OwnerMemberExtractor()
self.zone_extractor = CobolZoneExtractor()
self.relationship_resolver = RelationshipResolver()
self.primary_key_extractor = PrimaryKeyExtractor()

def build_metadata(
    self,
    pdf_path: str,
) -> SchemaMetadata:
    print("USING MetadataService.build_metadata VERSION ENRICH-BOTH-FIELDS-MAPPING-FIELDS-2026-07-30")

    document = self.extractor.extract_document(
        pdf_path,
    )

    sections = self.segmenter.segment(
        document,
    )

    metadata = SchemaMetadata()

    for section in sections:
        record = Record(
            name=section.record_name,
        )

        record.cobol_zone = self.zone_extractor.extract(
            section.lines,
        )

        physical_fields = self.field_extractor.extract(
            section.lines,
        )

        mapping_fields = self.field_extractor.extract_all(
            section.lines,
            include_filler=True,
        )

        print("=" * 100)
        print(f"METADATA DEBUG RECORD: {record.name}")
        print("=" * 100)
        print(f"RAW SECTION LINE COUNT: {len(section.lines or [])}")
        print(f"PHYSICAL FIELD COUNT BEFORE ENRICH: {len(physical_fields or [])}")
        print(f"MAPPING FIELD COUNT BEFORE ENRICH: {len(mapping_fields or [])}")

        record.fields = self.picture_enricher.enrich(
            fields=physical_fields,
            lines=section.lines,
            debug_label=f"{record.name} PHYSICAL",
        )

        record.mapping_fields = self.picture_enricher.enrich(
            fields=mapping_fields,
            lines=section.lines,
            debug_label=f"{record.name} MAPPING",
        )

        record.set_memberships = self.membership_extractor.extract(
            section.record_name,
            section.lines,
        )

        record.primary_key = self.primary_key_extractor.extract(
            section.lines,
        )

        print("=" * 100)
        print(f"FIELDS EXTRACTED FOR {record.name} - PHYSICAL")
        print("=" * 100)

        for field in record.fields or []:

```

```

        print(
            "PHYSICAL_FIELD_DEBUG",
            "name=", getattr(field, "name", None),
            "level=", getattr(field, "level", None),
            "datatype=", getattr(field, "datatype", None),
            "length=", getattr(field, "length", None),
            "scale=", getattr(field, "scale", None),
            "picture=", getattr(field, "picture", None),
            "start=", getattr(field, "start_position", None),
            "end=", getattr(field, "end_position", None),
            "basetype=", getattr(field, "basetype", None),
            "group=", getattr(field, "is_group", None),
            "has_child=", getattr(field, "has_child", None),
        )

    print("=" * 100)
    print(f"FIELDS EXTRACTED FOR {record.name} - MAPPING")
    print("=" * 100)

    for field in record.mapping_fields or []:
        print(
            "MAPPING_FIELD_DEBUG",
            "name=", getattr(field, "name", None),
            "level=", getattr(field, "level", None),
            "datatype=", getattr(field, "datatype", None),
            "length=", getattr(field, "length", None),
            "scale=", getattr(field, "scale", None),
            "picture=", getattr(field, "picture", None),
            "start=", getattr(field, "start_position", None),
            "end=", getattr(field, "end_position", None),
            "basetype=", getattr(field, "basetype", None),
            "group=", getattr(field, "is_group", None),
            "has_child=", getattr(field, "has_child", None),
        )

    print(
        f"MetadataService: {record.name} "
        f"PK={record.primary_key} "
        f"ZONE={record.cobol_zone} "
        f"PHYSICAL_FIELDS={len(record.fields or [])} "
        f"MAPPING_FIELDS={len(record.mapping_fields or [])}"
    )

    metadata.records.append(record)

metadata.relationships = self.relationship_resolver.resolve(
    metadata,
)

print(
    f"Relationships Found: {len(metadata.relationships)}"
)

return metadata

```

```

=====
FILE: src\idms_modernizer\services\name_normalizer.py
ENCODING: utf-8
=====

```

```
import re
```

```
class NameNormalizer:
```

```

    @staticmethod
    def normalize(
        name: str | None
    ) -> str:

```

```

        if not name:
            return ""

```

```

        normalized = name.strip().upper()
        normalized = normalized.replace("-", "_")

```



```

normalized = re.sub(
    r"^[^A-Z0-9_]",
    "_",
    normalized
)
normalized = re.sub(
    r"_{2,}",
    "_",
    normalized
)

return normalized.strip("_")

```

```

=====
FILE: src\idms_modernizer\services\phase2_conversion_bridge.py
ENCODING: utf-8
=====

```

```

class Phase2ConversionBridge:
    """
    Bridge from Phase 1 outputs to the Phase 2 converter.

    Import is intentionally lazy so Phase 1 UI can still load even if
    Phase 2 package has a missing or broken module.
    """

    def convert(
        self,
        cobol_text: str,
        canonical_json: str,
        phase2_metadata_json: str,
        ddl_text: str,
        idms_schema_text: str | None = None,
        relationship_overrides_json: str | None = None,
        target_program: str | None = None,
        use_ddl: bool = True,
        use_idms_schema: bool = False,
        use_overrides: bool = False
    ) -> tuple[str, list[str]]:

        if not cobol_text or not cobol_text.strip():
            raise ValueError(
                "COBOL code is required for Phase 2 conversion."
            )

        try:
            from idms_db2_converter.services.conversion_service import (
                ConversionService
            )

        except Exception as exc:
            raise RuntimeError(
                "Phase 2 converter package could not be loaded. "
                "Check src/idms_db2_converter files. "
                f"Details: {exc}"
            ) from exc

        if not canonical_json or not canonical_json.strip():
            canonical_json = '{"records": [], "sets": [], "relationships": []}'

        if not phase2_metadata_json:
            phase2_metadata_json = ""

        if not ddl_text:
            ddl_text = ""

        if not idms_schema_text:
            idms_schema_text = ""

        if not relationship_overrides_json:
            relationship_overrides_json = ""

        service = ConversionService()

        converted, validation_messages = service.convert_retrieval(

```

```

        cobol_text=cobol_text,
        canonical_json=canonical_json,
        phase2_metadata_json=phase2_metadata_json,
        ddl_text=ddl_text if use_ddl else None,
        idms_schema_text=(
            idms_schema_text
            if use_idms_schema
            else None
        ),
        relationship_overrides_json=(
            relationship_overrides_json
            if use_overrides and relationship_overrides_json.strip()
            else None
        ),
        target_program=(
            target_program.strip()
            if target_program and target_program.strip()
            else None
        )
    )

    return converted, validation_messages

```

```

=====
FILE: src\idms_modernizer\services\phase2_metadata_generator.py
ENCODING: utf-8
=====

```

```

import json
import re

from datetime import datetime
from typing import Any

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import DB2Model, DB2Table, DB2Column
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer

```

```

class Phase2MetadataGenerator:
    VERSION = "1.0"

    DATE_PARTS = {
        "YEAR": {
            "substring_start": 3,
            "substring_length": 2,
        },
        "MONTH": {
            "substring_start": 6,
            "substring_length": 2,
        },
        "DAY": {
            "substring_start": 9,
            "substring_length": 2,
        },
    }

    YEAR_PARTS = {
        "YEAR",
        "YR",
        "Y",
        "YY",
        "YYYY",
        "DY",
    }

    MONTH_PARTS = {
        "MONTH",
        "MON",
        "MO",
        "M",
        "MM",
        "DM",
    }

```

```

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}

def generate_json(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata,
) -> str:
    payload = self.generate_payload(
        canonical_schema=canonical_schema,
        db2_model=db2_model,
        metadata=metadata,
    )

    return json.dumps(
        payload,
        indent=2,
    )

def generate_payload(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata,
) -> dict[str, Any]:
    db2_table_lookup = self.build_db2_table_lookup(
        db2_model=db2_model,
    )

    canonical_record_lookup = self.build_canonical_record_lookup(
        canonical_schema=canonical_schema,
    )

    records_payload = self.build_records_payload(
        db2_model=db2_model,
    )

    record_table_map = self.build_record_table_map(
        metadata=metadata,
        db2_table_lookup=db2_table_lookup,
    )

    field_map = self.build_field_map(
        metadata=metadata,
        db2_table_lookup=db2_table_lookup,
        record_table_map=record_table_map,
    )

    calc_key_map = self.build_calc_key_map(
        canonical_schema=canonical_schema,
        metadata=metadata,
        record_table_map=record_table_map,
    )

    relationships_payload = self.build_relationships_payload(
        canonical_schema=canonical_schema,
        db2_table_lookup=db2_table_lookup,
    )

    set_ordering_map = self.build_set_ordering_map(
        relationships=relationships_payload,
    )

    nullable_fk_map = self.build_nullable_fk_map(
        db2_model=db2_model,
    )

    table_key_map = self.build_table_key_map(
        db2_model=db2_model,
    )

```

```

date_part_map = self.build_date_part_map(
    metadata=metadata,
    db2_table_lookup=db2_table_lookup,
    record_table_map=record_table_map,
)

navigation_intent = self.build_navigation_intent(
    relationships=relationships_payload,
)

output_semantics = self.build_output_semantics()

validation_messages = self.build_validation_messages(
    canonical_schema=canonical_schema,
    db2_model=db2_model,
    metadata=metadata,
    field_map=field_map,
)

return {
    "version": self.VERSION,
    "generated_at": datetime.utcnow().isoformat() + "Z",
    "records": records_payload,
    "record_table_map": record_table_map,
    "field_map": field_map,
    "calc_key_map": calc_key_map,
    "relationships": relationships_payload,
    "set_ordering_map": set_ordering_map,
    "nullable_fk_map": nullable_fk_map,
    "table_key_map": table_key_map,
    "date_part_map": date_part_map,
    "navigation_intent": navigation_intent,
    "output_semantics": output_semantics,
    "validation_messages": validation_messages,
    "canonical_record_lookup_count": len(canonical_record_lookup),
}

def build_records_payload(
    self,
    db2_model: DB2Model,
) -> list[dict[str, Any]]:
    records_payload: list[dict[str, Any]] = []

    for table in getattr(db2_model, "tables", []) or []:
        fields_payload: list[dict[str, Any]] = []

        for column in getattr(table, "columns", []) or []:
            datatype, length, scale = self.parse_db2_datatype(
                getattr(column, "datatype", ""),
            )

            fields_payload.append(
                {
                    "name": getattr(column, "name", ""),
                    "column": getattr(column, "name", ""),
                    "datatype": datatype,
                    "length": length,
                    "scale": scale,
                    "nullable": getattr(column, "nullable", True),
                    "primary_key": getattr(column, "primary_key", False),
                    "generated": getattr(column, "generated", False),
                    "source_kind": getattr(column, "source_kind", ""),
                }
            )

        primary_keys = list(getattr(table, "primary_keys", []) or [])

        if not primary_keys and getattr(table, "primary_key", None):
            primary_keys = [table.primary_key]

        records_payload.append(
            {
                "name": getattr(table, "name", ""),
                "table": getattr(table, "name", ""),
                "primary_key": primary_keys[0] if primary_keys else None,
                "primary_keys": primary_keys,
            }
        )

```

```

        "fields": fields_payload,
    }
)

return records_payload

def build_table_key_map(
    self,
    db2_model: DB2Model,
) -> dict[str, Any]:
    table_key_map: dict[str, Any] = {}

    for table in getattr(db2_model, "tables", []) or []:
        table_name = NameNormalizer.normalize(getattr(table, "name", "") or "")

        primary_keys = list(getattr(table, "primary_keys", []) or [])

        if not primary_keys and getattr(table, "primary_key", None):
            primary_keys = [table.primary_key]

        foreign_keys = []

        for foreign_key in getattr(table, "foreign_keys", []) or []:
            foreign_keys.append(
                {
                    "column_name": getattr(foreign_key, "column_name", "") or "",
                    "reference_table": getattr(foreign_key, "reference_table", "") or "",
                    "reference_column": getattr(foreign_key, "reference_column", "") or "",
                    "set_name": getattr(foreign_key, "set_name", "") or "",
                }
            )

        if table_name:
            table_key_map[table_name] = {
                "primary_keys": primary_keys,
                "foreign_keys": foreign_keys,
            }

    return table_key_map

def build_db2_table_lookup(
    self,
    db2_model: DB2Model,
) -> dict[str, DB2Table]:
    lookup: dict[str, DB2Table] = {}

    for table in getattr(db2_model, "tables", []) or []:
        table_name = NameNormalizer.normalize(
            getattr(table, "name", "") or ""
        )

        if table_name:
            lookup[table_name] = table

        suffix_removed = self.remove_record_suffix(table_name)

        if suffix_removed:
            lookup[suffix_removed] = table

    return lookup

def build_canonical_record_lookup(
    self,
    canonical_schema: CanonicalSchema,
) -> dict[str, Any]:
    lookup: dict[str, Any] = {}

    for record in getattr(canonical_schema, "records", []) or []:
        record_name = NameNormalizer.normalize(
            getattr(record, "name", "") or ""
        )

        if record_name:
            lookup[record_name] = record

        suffix_removed = self.remove_record_suffix(record_name)

```

```

        if suffix_removed:
            lookup[suffix_removed] = record

    return lookup

def build_record_table_map(
    self,
    metadata: SchemaMetadata,
    db2_table_lookup: dict[str, DB2Table],
) -> dict[str, str]:
    record_table_map: dict[str, str] = {}

    for record in getattr(metadata, "records", []) or []:
        record_name = NameNormalizer.normalize(
            getattr(record, "name", "") or "",
        )

        if not record_name:
            continue

        table = db2_table_lookup.get(record_name)

        if table is None:
            suffix_removed = self.remove_record_suffix(record_name)
            table = db2_table_lookup.get(suffix_removed)

        record_table_map[record_name] = (
            getattr(table, "name", "") if table is not None else record_name
        )

    return record_table_map

def build_field_map(
    self,
    metadata: SchemaMetadata,
    db2_table_lookup: dict[str, DB2Table],
    record_table_map: dict[str, str],
) -> dict[str, dict[str, Any]]:
    field_map: dict[str, dict[str, Any]] = {}

    for record in getattr(metadata, "records", []) or []:
        idms_record_name = NameNormalizer.normalize(
            getattr(record, "name", "") or "",
        )

        if not idms_record_name:
            continue

        table_name = record_table_map.get(
            idms_record_name,
            idms_record_name,
        )

        table = db2_table_lookup.get(
            NameNormalizer.normalize(table_name),
        )

        if table is None:
            continue

        column_lookup = self.build_column_lookup(
            table=table,
        )

        source_fields = (
            getattr(record, "mapping_fields", None)
            or getattr(record, "fields", None)
            or []
        )

        for field in source_fields:
            legacy_field_name = NameNormalizer.normalize(
                getattr(field, "name", "") or "",
            )

```

```

        if not legacy_field_name:
            continue

        column = column_lookup.get(legacy_field_name)

        if column is None:
            suffix_removed = self.remove_record_suffix(legacy_field_name)
            column = column_lookup.get(suffix_removed)

        if column is None:
            continue

        field_map[legacy_field_name] = {
            "record": idms_record_name,
            "table": getattr(table, "name", ""),
            "column": getattr(column, "name", ""),
            "host": self.host_variable_name(
                record=idms_record_name,
                column=getattr(column, "name", ""),
            ),
        }

    return field_map

def build_column_lookup(
    self,
    table: DB2Table,
) -> dict[str, DB2Column]:
    lookup: dict[str, DB2Column] = {}

    for column in getattr(table, "columns", []) or []:
        column_name = NameNormalizer.normalize(
            getattr(column, "name", "") or "",
        )

        if column_name:
            lookup[column_name] = column

    suffix_removed = self.remove_record_suffix(column_name)

    if suffix_removed:
        lookup[suffix_removed] = column

    return lookup

def build_calc_key_map(
    self,
    canonical_schema: CanonicalSchema,
    metadata: SchemaMetadata,
    record_table_map: dict[str, str],
) -> dict[str, dict[str, Any]]:
    calc_key_map: dict[str, dict[str, Any]] = {}

    for record in getattr(metadata, "records", []) or []:
        record_name = NameNormalizer.normalize(
            getattr(record, "name", "") or "",
        )

        primary_keys = []

        explicit_primary_keys = getattr(record, "primary_keys", None)

        if explicit_primary_keys:
            if isinstance(explicit_primary_keys, list):
                primary_keys.extend(explicit_primary_keys)
            else:
                primary_keys.append(explicit_primary_keys)

        primary_key = getattr(record, "primary_key", None)

        if primary_key:
            primary_keys.append(primary_key)

        cleaned_primary_keys = []

        for primary_key_value in primary_keys:

```

```

        normalized_primary_key = NameNormalizer.normalize(
            primary_key_value,
        )

        if not normalized_primary_key:
            continue

        if normalized_primary_key in cleaned_primary_keys:
            continue

        cleaned_primary_keys.append(normalized_primary_key)

    if not cleaned_primary_keys:
        continue

    calc_key_map[record_name] = {
        "record": record_name,
        "table": record_table_map.get(record_name, record_name),
        "primary_key": cleaned_primary_keys[0],
        "primary_keys": cleaned_primary_keys,
    }

    return calc_key_map

def build_relationships_payload(
    self,
    canonical_schema: CanonicalSchema,
    db2_table_lookup: dict[str, DB2Table],
) -> list[dict[str, Any]]:
    relationships_payload: list[dict[str, Any]] = []

    for relationship in getattr(canonical_schema, "relationships", []) or []:
        parent_record = NameNormalizer.normalize(
            getattr(relationship, "parent_record", None)
            or getattr(relationship, "owner_record", None)
            or "",
        )

        child_record = NameNormalizer.normalize(
            getattr(relationship, "child_record", None)
            or getattr(relationship, "member_record", None)
            or "",
        )

        set_name = NameNormalizer.normalize(
            getattr(relationship, "set_name", None)
            or getattr(relationship, "name", None)
            or "",
        )

        parent_table = db2_table_lookup.get(parent_record)
        child_table = db2_table_lookup.get(child_record)

        parent_keys = []
        child_fks = []

        if parent_table is not None:
            parent_keys = list(getattr(parent_table, "primary_keys", []) or [])

            if not parent_keys and getattr(parent_table, "primary_key", None):
                parent_keys = [parent_table.primary_key]

        if child_table is not None:
            for foreign_key in getattr(child_table, "foreign_keys", []) or []:
                fk_set_name = NameNormalizer.normalize(
                    getattr(foreign_key, "set_name", "") or "",
                )

                if set_name and fk_set_name and fk_set_name != set_name:
                    continue

                child_fks.append(
                    getattr(foreign_key, "column_name", "") or "",
                )

        relationships_payload.append(

```



```

        {
            "set_name": set_name,
            "parent_record": parent_record,
            "child_record": child_record,
            "owner_record": parent_record,
            "member_record": child_record,
            "parent_table": getattr(parent_table, "name", "") if parent_table else "",
            "child_table": getattr(child_table, "name", "") if child_table else "",
            "parent_key": parent_keys[0] if parent_keys else None,
            "parent_keys": parent_keys,
            "child_fk": child_fks[0] if child_fks else None,
            "child_fks": child_fks,
            "order_by": getattr(relationship, "order_by", []) or [],
        }
    )

    return relationships_payload

def build_set_ordering_map(
    self,
    relationships: list[dict[str, Any]],
) -> dict[str, dict[str, Any]]:
    set_ordering_map: dict[str, dict[str, Any]] = {}

    for relationship in relationships:
        set_name = relationship.get("set_name") or ""

        if not set_name:
            continue

        set_ordering_map[set_name] = {
            "parent_record": relationship.get("parent_record") or "",
            "child_record": relationship.get("child_record") or "",
            "parent_key": relationship.get("parent_key"),
            "parent_keys": relationship.get("parent_keys") or [],
            "child_fk": relationship.get("child_fk"),
            "child_fks": relationship.get("child_fks") or [],
            "order_by": relationship.get("order_by") or [],
        }

    return set_ordering_map

def build_nullable_fk_map(
    self,
    db2_model: DB2Model,
) -> dict[str, Any]:
    nullable_fk_map: dict[str, Any] = {}

    for table in getattr(db2_model, "tables", []) or []:
        column_lookup = {
            getattr(column, "name", ""): column
            for column in getattr(table, "columns", []) or []
        }

        for foreign_key in getattr(table, "foreign_keys", []) or []:
            fk_column_name = getattr(foreign_key, "column_name", "")
            fk_column = column_lookup.get(fk_column_name)

            key = f"{getattr(table, 'name', '')}.{fk_column_name}"

            nullable_fk_map[key] = {
                "nullable": (
                    getattr(fk_column, "nullable", True)
                    if fk_column is not None
                    else True
                ),
                "set_name": getattr(foreign_key, "set_name", "") or "",
                "reference_table": getattr(foreign_key, "reference_table", "") or "",
                "reference_column": getattr(foreign_key, "reference_column", "") or "",
            }

    return nullable_fk_map

def build_date_part_map(
    self,
    metadata: SchemaMetadata,

```

```

    db2_table_lookup: dict[str, DB2Table],
    record_table_map: dict[str, str],
) -> dict[str, dict[str, Any]]:
    date_part_map: dict[str, dict[str, Any]] = {}

    for record in getattr(metadata, "records", []) or []:
        idms_record_name = NameNormalizer.normalize(
            getattr(record, "name", "") or "",
        )

        if not idms_record_name:
            continue

        table_name = record_table_map.get(
            idms_record_name,
            idms_record_name,
        )

        table = db2_table_lookup.get(
            NameNormalizer.normalize(table_name),
        )

        if table is None:
            continue

        date_columns = {
            NameNormalizer.normalize(getattr(column, "name", "") or ""): column
            for column in getattr(table, "columns", []) or []
            if str(getattr(column, "datatype", "")).upper() == "DATE"
        }

        source_fields = (
            getattr(record, "mapping_fields", None)
            or getattr(record, "fields", None)
            or []
        )

        for field in source_fields:
            legacy_field_name = NameNormalizer.normalize(
                getattr(field, "name", "") or "",
            )

            parsed = self.parse_date_part(
                field_name=legacy_field_name,
            )

            if parsed is None:
                continue

            part = parsed["part"]
            date_key = parsed["date_key"]

            date_column = date_columns.get(date_key)

            if date_column is None:
                suffix_removed = self.remove_record_suffix(date_key)
                date_column = date_columns.get(suffix_removed)

            if date_column is None:
                continue

            date_part_map[legacy_field_name] = {
                "record": idms_record_name,
                "table": getattr(table, "name", ""),
                "column": getattr(date_column, "name", ""),
                "host": self.host_variable_name(
                    record=idms_record_name,
                    column=getattr(date_column, "name", ""),
                ),
                "substring_start": self.DATE_PARTS[part]["substring_start"],
                "substring_length": self.DATE_PARTS[part]["substring_length"],
                "part": part,
            }

    return date_part_map

```

```

def parse_date_part(
    self,
    field_name: str,
) -> dict[str, Any] | None:
    candidates = self.parse_date_part_candidates(
        field_name=field_name,
    )

    if not candidates:
        return None

    return candidates[0]

def parse_date_part_candidates(
    self,
    field_name: str,
) -> list[dict[str, Any]]:
    normalized = NameNormalizer.normalize(field_name or "")

    if not normalized:
        return []

    tokens = [
        token
        for token in re.split(r"[\s_]+", normalized)
        if token
    ]

    candidates = []

    for index, token in enumerate(tokens):
        part = self.date_part_type(
            token=token,
            tokens=tokens,
        )

        if part is None:
            continue

        date_tokens = tokens.copy()
        date_tokens[index] = "DATE"

        date_key = "_".join(date_tokens)

        candidates.append(
            {
                "date_key": date_key,
                "part": part,
                "tokens": date_tokens,
            }
        )

    return candidates

def date_part_type(
    self,
    token: str,
    tokens: list[str],
) -> str | None:
    token = token.upper()

    has_dy_dm_dd = (
        "DY" in tokens
        and "DM" in tokens
        and "DD" in tokens
    )

    if token in {"YEAR", "YR", "Y", "YY", "YYYY"}:
        return "YEAR"

    if token in {"MONTH", "MON", "MO", "M", "MM", "DM"}:
        return "MONTH"

    if token in {"DAY", "D", "DD"}:
        return "DAY"

```

```

        if token == "DY":
            if has_dy_dm_dd:
                return "YEAR"

            return "DAY"

    return None

def build_navigation_intent(
    self,
    relationships: list[dict[str, Any]],
) -> dict[str, Any]:
    navigation_intent: dict[str, Any] = {}

    for relationship in relationships:
        set_name = relationship.get("set_name") or ""

        if not set_name:
            continue

        navigation_intent[set_name] = {
            "access_pattern": "SET_NAVIGATION",
            "parent_record": relationship.get("parent_record"),
            "child_record": relationship.get("child_record"),
            "parent_key": relationship.get("parent_key"),
            "parent_keys": relationship.get("parent_keys") or [],
            "child_fk": relationship.get("child_fk"),
            "child_fks": relationship.get("child_fks") or [],
            "order_by": relationship.get("order_by") or [],
        }

    return navigation_intent

def build_output_semantics(
    self,
) -> dict[str, Any]:
    return {
        "generated_by": "idms-db2-modernizer-phase1",
        "usage": "IDMS retrieval COBOL to DB2 embedded SQL conversion",
        "physical_types_source": "DB2_DDL",
        "logical_mapping_source": "PHASE1_CANONICAL_METADATA",
        "db2_key_values": ["PK", "FK", "PK/FK"],
        "generated_primary_key_rule": "ID_RECORD_<record_name>",
        "generated_primary_key_datatype": "CHAR(20)",
        "foreign_key_rule": "Use SET relationship and owner primary key columns only.",
        "character_rule": "CHAR by default; VARCHAR only when length equals 100.",
        "numeric_rule": "Numeric COBOL PIC maps to DECIMAL.",
        "date_rule": "Date part recognition remains generic for YEAR, YR, Y, YY, YYYY, DY, MONTH, MON, MO, M, MM, D
M, DAY, D, DD.",
    }

def build_validation_messages(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata,
    field_map: dict[str, dict[str, Any]],
) -> list[str]:
    messages: list[str] = []

    if not getattr(canonical_schema, "records", []) or []:
        messages.append("No canonical records found.")

    if not getattr(db2_model, "tables", []) or []:
        messages.append("No DB2 tables found.")

    if not getattr(metadata, "records", []) or []:
        messages.append("No IDMS metadata records found.")

    if not field_map:
        messages.append("No field map entries generated.")

    for table in getattr(db2_model, "tables", []) or []:
        primary_keys = list(getattr(table, "primary_keys", []) or [])

        if not primary_keys and getattr(table, "primary_key", None):

```

```

        primary_keys = [table.primary_key]

    if not primary_keys:
        messages.append(f"Table {table.name} has no primary key.")
        continue

    column_names = {
        NameNormalizer.normalize(getattr(column, "name", "") or "")
        for column in getattr(table, "columns", []) or []
    }

    for primary_key in primary_keys:
        if NameNormalizer.normalize(primary_key) not in column_names:
            messages.append(
                f"Table {table.name} primary key {primary_key} is not present as a DB2 column."
            )

    return messages

def parse_db2_datatype(
    self,
    datatype: str,
) -> tuple[str, int | None, int | None]:
    value = (datatype or "").strip().upper()

    decimal_match = re.match(
        r"^(DECIMAL|NUMERIC)$($(\d+)(?:, \s*(\d+))?$)$",
        value,
    )

    if decimal_match:
        return (
            "DECIMAL",
            int(decimal_match.group(2)),
            int(decimal_match.group(3) or 0),
        )

    char_match = re.match(
        r"^(VARCHAR|CHAR|CHARACTER)$($(\d+))$",
        value,
    )

    if char_match:
        return (
            "VARCHAR" if char_match.group(1) == "VARCHAR" else "CHAR",
            int(char_match.group(2)),
            None,
        )

    if value == "DATE":
        return "DATE", None, None

    if value == "TIMESTAMP":
        return "TIMESTAMP", None, None

    return value or "CHAR", None, None

def host_variable_name(
    self,
    record: str,
    column: str,
) -> str:
    record_name = NameNormalizer.normalize(record or "")
    column_name = NameNormalizer.normalize(column or "")
    return f"HV-{{record_name}}-{{column_name}}"

def remove_record_suffix(
    self,
    value: str,
) -> str:
    text = NameNormalizer.normalize(value or "")

    if not text:
        return ""

    text = text.replace(" ", "_")

```

```

return re.sub(
    r"_[0-9]{4}$",
    "",
    text,
)

```

```

=====
FILE: src\idms_modernizer\services\relationship_resolver.py
ENCODING: utf-8
=====

```

```

from collections import defaultdict

```

```

from idms_modernizer.domain.relationship_models import (
    Relationship,
)

```

```

class RelationshipResolver:

```

```

    """

```

```

    Resolves IDMS SET owner/member relationships from record set memberships.

```

```

    Generic behavior only:

```

- No hardcoded SET names.
 - No hardcoded record names.
 - OWNER and MEMBER are read from schema listing memberships.
- ```

 """

```

```

 def resolve(

```

```

 self,
 metadata,

```

```

) -> list[Relationship]:

```

```

 owner_map: dict[str, list[str]] = defaultdict(list)
 member_map: dict[str, list[str]] = defaultdict(list)

```

```

 for record in getattr(metadata, "records", []) or []:
 record_name = getattr(record, "name", "") or ""

```

```

 if not record_name:
 continue

```

```

 for membership in getattr(record, "set_memberships", []) or []:

```

```

 set_name = (
 getattr(membership, "set_name", None)
 or getattr(membership, "name", None)
 or ""
)

```

```

 role = (
 getattr(membership, "role", None)
 or getattr(membership, "relation_type", None)
 or getattr(membership, "type", None)
 or ""
)

```

```

 set_name = str(set_name or "").strip().upper()
 role = str(role or "").strip().upper()

```

```

 if not set_name:
 continue

```

```

 if not role:
 continue

```

```

 if set_name == "CALC":
 continue

```

```

 if role == "OWNER":
 if record_name not in owner_map[set_name]:
 owner_map[set_name].append(record_name)

```

```

 elif role == "MEMBER":
 if record_name not in member_map[set_name]:
 member_map[set_name].append(record_name)

```

```

relationships: list[Relationship] = []

all_sets = set(owner_map.keys()) | set(member_map.keys())

for set_name in sorted(all_sets):
 owners = owner_map.get(set_name, [])
 members = member_map.get(set_name, [])

 if not owners:
 continue

 if not members:
 continue

 for owner in owners:
 for member in members:
 if not owner or not member:
 continue

 if owner == member:
 continue

 relationships.append(
 Relationship(
 set_name=set_name,
 owner_record=owner,
 member_record=member,
 cardinality="1:N",
)
)

metadata.relationships = relationships

return relationships

```

```

=====
FILE: src\idms_modernizer\services\schema_picture_enricher.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.services.name_normalizer import NameNormalizer

```

```
print("LOADED SchemaPictureEnricher VERSION FINAL-ROBUST-PIC-PARENS-V-2026-07-30")
```

```
class SchemaPictureEnricher:
```

```
 """
```

```
 Enriches DataField objects using schema listing lines.
```

```
 Rules implemented:
```

- Build Excel IDMS PIC Clause from schema PICTURE + USAGE.
- DISPLAY X(n) -> PIC X(n)
- DISPLAY 9(n) -> PIC 9(n)
- DISPLAY 9(n)V9(m) -> PIC 9(n)V9(m)
- DISPLAY S9(n)V9(m) -> PIC S9(n)V9(m)
- COMP-3 9(n) -> PIC 9(n) COMP-3
- COMP-3 S9(n) -> PIC S9(n) COMP-3
- COMP-3 S9(n)V9(m) -> PIC S9(n)V9(m) COMP-3
- COMP-3 S9(n)V99 -> PIC S9(n)V9(2) COMP-3
- COMP 9(n) -> PIC 9(n) COMP
- Group items do not get PIC.
- OCCURS does not change PIC.
- Keys do not change PIC.
- END = STRT + LGTH - 1.

```
 Critical:
```

- Do not merge implied decimal fields.
- PIC S9(13)V9(2) COMP-3 must remain PIC S9(13)V9(2) COMP-3.
- PIC 9(8) COMP-3 must remain PIC 9(8) COMP-3.
- Do not reconstruct from LGTH when full PIC is present.

"""

```
FIELD_START_PATTERN = re.compile(
 r"^\s*(?P<level>0[1-9]|[1-4][0-9]|88)\s+"
 r"(?P<name>[A-Z][A-Z0-9-]*|FILLER)\b"
 r"(?P<rest>.*)$",
 re.IGNORECASE,
)

USAGE_PATTERN = re.compile(
 r"\b(?:USAGE\s+IS\s+)?(?:P<usage>DISPLAY|COMP-3|COMP)\b",
 re.IGNORECASE,
)

START_LENGTH_PATTERN = re.compile(
 r"\b(?:P<start>[0-9]+)\s+(?P<length>[0-9]+)\s*$",
 re.IGNORECASE,
)
```

```
COMPACT_PIC_PATTERNS = [
 re.compile(r"S?9$\d+$V9$\d+$", re.IGNORECASE),
 re.compile(r"S?9$\d+$V[90]+", re.IGNORECASE),
 re.compile(r"S?[90]+V9$\d+$", re.IGNORECASE),
 re.compile(r"S?[90]+V[90]+", re.IGNORECASE),
 re.compile(r"S?9$\d+$", re.IGNORECASE),
 re.compile(r"S?[90]+", re.IGNORECASE),
 re.compile(r"X$\d+$", re.IGNORECASE),
 re.compile(r"X+", re.IGNORECASE),
]
```

```
DATE_NAME_TOKENS = {
 "DA",
 "DATE",
 "DT",
 "DTE",
 "YYMMDD",
 "YYYYMMDD",
 "YMD",
}
```

```
YEAR_PARTS = {
 "YEAR",
 "YR",
 "Y",
 "YY",
 "YYYY",
 "DY",
}
```

```
MONTH_PARTS = {
 "MONTH",
 "MON",
 "MO",
 "M",
 "MM",
 "DM",
}
```

```
DAY_PARTS = {
 "DAY",
 "D",
 "DD",
}
```

```
def enrich(
 self,
 fields: list[DataField],
 lines: list[str],
 debug_label: str = "",
) -> list[DataField]:
 print(
 "USING SchemaPictureEnricher.enrich VERSION FINAL-ROBUST-PIC-PARENS-V-2026-07-30",
 f"debug_label={debug_label}",
)

 normalized_lines = self.normalize_lines(
```



```

 lines=lines,
)

 print(
 "SCHEMA_ENRICH_DEBUG_LINES",
 f"debug_label={debug_label}",
 f"line_count={len(normalized_lines)}",
)

 field_blocks = self.build_field_blocks(
 normalized_lines=normalized_lines,
 debug_label=debug_label,
)

 enriched_fields: list[DataField] = []

 for field in fields or []:
 field_name = getattr(field, "name", "") or ""

 block = self.find_block_for_field(
 field_name=field_name,
 field_blocks=field_blocks,
)

 if not block:
 print(
 "SCHEMA_ENRICH_NO_BLOCK",
 f"debug_label={debug_label}",
 f"field={repr(field_name)}",
)

 enriched_fields.append(
 self.copy_field_with_defaults(
 field=field,
)
)
 continue

 enriched = self.enrich_field(
 field=field,
 block=block,
 debug_label=debug_label,
)

 enriched_fields.append(enriched)

 enriched_fields = self.enrich_group_positions_from_children(
 fields=enriched_fields,
 debug_label=debug_label,
)

 return enriched_fields

def normalize_lines(
 self,
 lines: list[str],
) -> list[str]:
 result: list[str] = []

 for line in lines or []:
 if line is None:
 continue

 value = getattr(line, "text", None)

 if value is None:
 value = str(line)

 value = str(value)
 value = value.replace("\t", " ")
 value = value.replace("\u00a0", " ")
 value = value.replace("PICTURE .", "PICTURE")
 value = value.replace("PIC .", "PIC")
 value = re.sub(r"<[^>]+>", " ", value)
 value = re.sub(r"\s+", " ", value).strip()

```

```

 if value:
 result.append(value)

 return result

def build_field_blocks(
 self,
 normalized_lines: list[str],
 debug_label: str = "",
) -> dict[str, str]:
 starts: list[tuple[int, str]] = []

 for index, line in enumerate(normalized_lines or []):
 match = self.FIELD_START_PATTERN.match(line)

 if not match:
 continue

 field_name = match.group("name") or ""

 starts.append(
 (
 index,
 field_name.upper(),
)
)

 print(
 "SCHEMA_BLOCK_STARTS_DEBUG",
 f"debug_label={debug_label}",
 f"starts_count={len(starts)}",
 f"starts={starts[:80]}",
)

 blocks: dict[str, str] = {}

 for index, item in enumerate(starts):
 start_index, field_name = item

 if index + 1 < len(starts):
 end_index = starts[index + 1][0]
 else:
 end_index = len(normalized_lines)

 block = " ".join(
 normalized_lines[start_index:end_index],
)

 normalized_field_name = NameNormalizer.normalize(
 field_name,
)

 if normalized_field_name:
 blocks[normalized_field_name] = block

 suffix_removed = self.remove_record_suffix(
 value=normalized_field_name,
)

 if suffix_removed:
 blocks.setdefault(
 suffix_removed,
 block,
)

 print(
 "SCHEMA_BLOCK_DEBUG",
 f"debug_label={debug_label}",
 f"field={repr(normalized_field_name)}",
 f"block={repr(block)}",
)

 return blocks

def find_block_for_field(
 self,

```

```

 field_name: str,
 field_blocks: dict[str, str],
) -> str:
 normalized_name = NameNormalizer.normalize(
 field_name or "",
)

 if not normalized_name:
 return ""

 if normalized_name in field_blocks:
 return field_blocks[normalized_name]

 suffix_removed = self.remove_record_suffix(
 value=normalized_name,
)

 if suffix_removed in field_blocks:
 return field_blocks[suffix_removed]

 for block_name, block in field_blocks.items():
 if self.remove_record_suffix(block_name) == suffix_removed:
 return block

 return ""

def enrich_field(
 self,
 field: DataField,
 block: str,
 debug_label: str = "",
) -> DataField:
 field_name = getattr(field, "name", "") or ""

 if not block:
 return self.copy_field_with_defaults(field=field)

 parsed_line = self.parse_schema_block(
 block=block,
 field_name=field_name,
 debug_label=debug_label,
)

 usage = parsed_line["usage"]
 start_position = parsed_line["start_position"]
 storage_length = parsed_line["storage_length"]
 picture_core = parsed_line["picture_core"]

 is_group = bool(
 getattr(field, "has_child", False)
 or getattr(field, "is_group", False)
)

 if is_group:
 return self.make_field(
 source=field,
 datatype=getattr(field, "datatype", None),
 length=storage_length or getattr(field, "length", None),
 scale=getattr(field, "scale", None),
 picture=None,
 start_position=start_position or getattr(field, "start_position", None),
 end_position=self.calculate_end_position(
 start_position=start_position,
 storage_length=storage_length,
 fallback=getattr(field, "end_position", None),
),
 basetype=getattr(field, "basetype", None),
)

 rendered_picture = self.render_cobol_pic_clause(
 picture=picture_core,
 usage=usage,
 storage_length=storage_length,
)

 datatype, logical_length, scale = self.derive_type_from_picture(

```

```

 picture=rendered_picture,
 usage=usage,
 storage_length=storage_length,
 fallback_datatype=getattr(field, "datatype", None),
)

 if self.is_date_field(
 field=field,
 picture=rendered_picture,
 storage_length=storage_length,
 datatype=datatype,
):
 datatype = "DATE"
 scale = None

 end_position = self.calculate_end_position(
 start_position=start_position,
 storage_length=storage_length,
 fallback=getattr(field, "end_position", None),
)

 actual_length = storage_length

 if actual_length is None:
 actual_length = logical_length

 print(
 "SCHEMA_ENRICH_RESULT",
 f"debug_label={debug_label}",
 f"field={repr(field_name)}",
 f"usage={repr(usage)}",
 f"picture_core={repr(picture_core)}",
 f"rendered_picture={repr(rendered_picture)}",
 f"datatype={repr(datatype)}",
 f"length={actual_length}",
 f"scale={scale}",
 f"start={start_position}",
 f"end={end_position}",
)

 return self.make_field(
 source=field,
 datatype=datatype,
 length=actual_length,
 scale=scale,
 picture=rendered_picture,
 start_position=start_position,
 end_position=end_position,
 basetype=self.basetype_for_datatype(
 datatype=datatype,
),
)

def parse_schema_block(
 self,
 block: str,
 field_name: str,
 debug_label: str = "",
) -> dict:
 text = str(block or "").strip()

 usage = self.extract_usage(
 block=text,
)

 start_position, storage_length = self.extract_start_and_length(
 block=text,
)

 body_without_positions = self.remove_trailing_start_and_length(
 value=text,
)

 rest = self.extract_rest_after_level_and_name(
 block=body_without_positions,
)

```

```

picture_area = self.remove_usage_words(
 value=rest,
)

picture_area = self.remove_schema_noise(
 value=picture_area,
)

picture_core = self.extract_picture_core(
 value=picture_area,
)

print(
 "SCHEMA_PARSE_DEBUG",
 f"debug_label={debug_label}",
 f"field={repr(field_name)}",
 f"block={repr(block)}",
 f"usage={repr(usage)}",
 f"start={start_position}",
 f"length={storage_length}",
 f"rest={repr(rest)}",
 f"picture_area={repr(picture_area)}",
 f"picture_core={repr(picture_core)}",
)

return {
 "usage": usage,
 "start_position": start_position,
 "storage_length": storage_length,
 "picture_core": picture_core,
}

def extract_rest_after_level_and_name(
 self,
 block: str,
) -> str:
 match = self.FIELD_START_PATTERN.match(
 block or "",
)

 if not match:
 return str(block or "")

 return str(match.group("rest") or "").strip()

def extract_usage(
 self,
 block: str,
) -> str:
 upper = str(block or "").upper()

 if "COMP-3" in upper:
 return "COMP-3"

 if re.search(r"\bCOMP\b", upper):
 return "COMP"

 usage_match = self.USAGE_PATTERN.search(
 block or "",
)

 if usage_match:
 return usage_match.group("usage").upper()

 return "DISPLAY"

def extract_start_and_length(
 self,
 block: str,
) -> tuple[int | None, int | None]:
 match = self.START_LENGTH_PATTERN.search(
 block or "",
)

 if not match:

```

```

 return None, None

 return int(match.group("start")), int(match.group("length"))

def remove_trailing_start_and_length(
 self,
 value: str,
) -> str:
 return re.sub(
 r"\b[0-9]+\s+[0-9]+\s*$",
 "",
 str(value or ""),
).strip()

def remove_usage_words(
 self,
 value: str,
) -> str:
 text = str(value or "")

 text = re.sub(r"\bUSAGE\s+IS\s+COMP-3\b", " ", text, flags=re.IGNORECASE)
 text = re.sub(r"\bUSAGE\s+IS\s+COMP\b", " ", text, flags=re.IGNORECASE)
 text = re.sub(r"\bUSAGE\s+IS\s+DISPLAY\b", " ", text, flags=re.IGNORECASE)
 text = re.sub(r"\bCOMP-3\b", " ", text, flags=re.IGNORECASE)
 text = re.sub(r"\bCOMP\b", " ", text, flags=re.IGNORECASE)
 text = re.sub(r"\bDISPLAY\b", " ", text, flags=re.IGNORECASE)

 return re.sub(r"\s+", " ", text).strip()

def remove_schema_noise(
 self,
 value: str,
) -> str:
 text = str(value or "")

 text = re.sub(r"\bOCCURS\b.*$", " ", text, flags=re.IGNORECASE)
 text = re.sub(r"\bREDEFINES\b.*$", " ", text, flags=re.IGNORECASE)
 text = text.replace(".", " ")
 text = text.replace("\u00a0", " ")
 text = text.replace("\t", " ")

 return re.sub(r"\s+", " ", text).strip()

def extract_picture_core(
 self,
 value: str,
) -> str | None:
 original = str(value or "").upper()
 compact = re.sub(r"\s+", "", original)

 compact = compact.replace("PICTURE", "")
 compact = compact.replace("PIC", "")
 compact = compact.replace(".", "")

 candidates: list[tuple[int, int, str]] = []

 for pattern in self.COMPACT_PIC_PATTERNS:
 for match in pattern.finditer(compact):
 raw = match.group(0)
 core = self.clean_picture_core(raw)

 if not core:
 continue

 score = self.picture_score(
 core=core,
)

 candidates.append(
 (
 score,
 match.start(),
 core,
)
)

```

```

 if not candidates:
 return None

 candidates.sort(
 key=lambda item: (item[0], item[1]),
)

 return candidates[-1][2]

def picture_score(
 self,
 core: str,
) -> int:
 text = str(core or "").upper()

 score = 0

 if "V" in text:
 score += 1000

 if "(" in text and ")" in text:
 score += 100

 if text.startswith("S"):
 score += 10

 if text.startswith("X"):
 score += 5

 score += len(text)

 return score

def clean_picture_core(
 self,
 value: str | None,
) -> str | None:
 if value is None:
 return None

 text = str(value).upper()
 text = text.replace("PICTURE", "")
 text = text.replace("PIC", "")
 text = text.replace(".", "")
 text = text.replace("\u00a0", " ")
 text = text.replace("\t", " ")
 text = re.sub(r"\s+", "", text)

 return text.strip() or None

def render_cobol_pic_clause(
 self,
 picture: str | None,
 usage: str,
 storage_length: int | None,
) -> str | None:
 if not picture:
 return None

 normalized_picture = self.normalize_picture_preserving_v(
 picture=picture,
 storage_length=storage_length,
)

 if not normalized_picture:
 return None

 usage_upper = str(usage or "DISPLAY").upper()

 if usage_upper in {"COMP", "COMP-3"}:
 if usage_upper not in normalized_picture.upper():
 normalized_picture = f"{normalized_picture} {usage_upper}"

 if normalized_picture.upper().startswith("PIC "):
 return normalized_picture

```

```

 return f"PIC {normalized_picture}"

def normalize_picture_preserving_v(
 self,
 picture: str | None,
 storage_length: int | None = None,
) -> str | None:
 if not picture:
 return None

 core = self.picture_core(picture)

 if not core:
 return None

 if "V" in core:
 before_v, after_v = core.split("V", 1)

 before_v_normalized = self.normalize_9_part(
 part=before_v,
 storage_length=None,
)
 after_v_normalized = self.normalize_9_part(
 part=after_v,
 storage_length=None,
)

 return f"{before_v_normalized}V{after_v_normalized}"

 if core.startswith("X"):
 return self.normalize_x_part(
 part=core,
 storage_length=storage_length,
)

 return self.normalize_9_part(
 part=core,
 storage_length=storage_length,
)

def normalize_x_part(
 self,
 part: str,
 storage_length: int | None = None,
) -> str:
 text = str(part or "").upper()

 if text.startswith("X(") and text.endswith(")"):
 return text

 if set(text) == {"X"}:
 if len(text) == 1 and storage_length and int(storage_length) > 1:
 return f"X({int(storage_length)})"

 if len(text) == 1:
 return "X"

 return f"X({len(text)})"

 return text

def normalize_9_part(
 self,
 part: str,
 storage_length: int | None = None,
) -> str:
 text = str(part or "").upper()

 signed = False

 if text.startswith("S"):
 signed = True
 text = text[1:]

 if text.startswith("9(") and text.endswith(")"):
 return f"{'S' if signed else ''}{text}"

```



```

count = self.count_9_digits(
 value=text,
)

if count <= 0:
 return f"{'S' if signed else ''}{text}"

if count == 1 and storage_length and int(storage_length) > 1:
 return f"{'S' if signed else ''}9({int(storage_length)})"

if count == 1:
 return f"{'S' if signed else ''}9"

return f"{'S' if signed else ''}9({count})"

def derive_type_from_picture(
 self,
 picture: str | None,
 usage: str,
 storage_length: int | None,
 fallback_datatype: str | None,
) -> tuple[str | None, int | None, int | None]:
 if not picture:
 return fallback_datatype, storage_length, None

 core = self.picture_core(
 picture=picture,
)

 if not core:
 return fallback_datatype, storage_length, None

 if self.picture_is_character(core):
 length = self.character_length(
 picture=core,
)
 return "CHAR", length, None

 if self.picture_is_decimal(core):
 precision, scale = self.decimal_precision_scale(
 picture=core,
)
 return "DECIMAL", precision, scale

 if self.picture_is_numeric(core):
 precision = self.numeric_precision(
 picture=core,
)
 return "DECIMAL", precision, 0

 return fallback_datatype, storage_length, None

def picture_core(
 self,
 picture: str | None,
) -> str:
 text = str(picture or "").upper()
 text = text.replace("PICTURE", "")
 text = text.replace("PIC", "")
 text = text.replace("COMP-3", "")
 text = re.sub(r"\bCOMP\b", "", text)
 text = text.replace("DISPLAY", "")
 text = text.replace("USAGE", "")
 text = text.replace("IS", "")
 text = text.replace(".", "")
 text = re.sub(r"\s+", "", text)

 return text.strip()

def picture_is_character(
 self,
 picture: str,
) -> bool:
 text = str(picture or "").upper()

```

```

 if text.startswith("X(") and text.endswith(")"):
 return self.extract_parenthesized_int(text) is not None

 return bool(text) and set(text) == {"X"}

def character_length(
 self,
 picture: str,
) -> int:
 text = str(picture or "").upper()

 if text.startswith("X(") and text.endswith(")"):
 parsed = self.extract_parenthesized_int(text)

 if parsed is not None:
 return parsed

 return text.count("X")

def picture_is_numeric(
 self,
 picture: str,
) -> bool:
 text = str(picture or "").upper()

 if text.startswith("S"):
 text = text[1:]

 return "9" in text and "V" not in text

def picture_is_decimal(
 self,
 picture: str,
) -> bool:
 text = str(picture or "").upper()

 if text.startswith("S"):
 text = text[1:]

 return "V" in text and "9" in text

def decimal_precision_scale(
 self,
 picture: str,
) -> tuple[int, int]:
 text = str(picture or "").upper()

 if text.startswith("S"):
 text = text[1:]

 before_v, after_v = text.split("V", 1)

 integer_digits = self.count_9_digits(
 value=before_v,
)
 decimal_digits = self.count_9_digits(
 value=after_v,
)

 return integer_digits + decimal_digits, decimal_digits

def numeric_precision(
 self,
 picture: str,
) -> int:
 text = str(picture or "").upper()

 if text.startswith("S"):
 text = text[1:]

 return self.count_9_digits(
 value=text,
)

def count_9_digits(
 self,

```

```

 value: str,
) -> int:
 text = str(value or "").upper()

 if text.startswith("S"):
 text = text[1:]

 total = 0
 index = 0

 while index < len(text):
 if text[index] not in {"9", "0"}:
 index += 1
 continue

 if index + 1 < len(text) and text[index + 1] == "(":
 close_index = text.find(")", index + 2)

 if close_index != -1:
 number_text = text[index + 2:close_index]

 if number_text.isdigit():
 total += int(number_text)
 index = close_index + 1
 continue

 total += 1
 index += 1

 return total

def extract_parenthesized_int(
 self,
 value: str,
) -> int | None:
 text = str(value or "")

 open_index = text.find("(")

 if open_index == -1:
 return None

 close_index = text.find(")", open_index + 1)

 if close_index == -1:
 return None

 number_text = text[open_index + 1:close_index].strip()

 if not number_text.isdigit():
 return None

 return int(number_text)

def calculate_end_position(
 self,
 start_position: int | None,
 storage_length: int | None,
 fallback=None,
) -> int | None:
 try:
 if start_position is not None and storage_length is not None:
 return int(start_position) + int(storage_length) - 1
 except Exception:
 pass

 try:
 if fallback is not None:
 return int(fallback)
 except Exception:
 pass

 return None

def is_date_field(
 self,

```

```

 field: DataField,
 picture: str | None,
 storage_length: int | None,
 datatype: str | None,
) -> bool:
 datatype_upper = str(datatype or "").upper()

 if datatype_upper == "DATE":
 return True

 field_name = NameNormalizer.normalize(
 getattr(field, "name", "") or "",
)

 parts = [
 part
 for part in field_name.replace("-", " ").replace("_", " ").split()
 if part
]

 if "DATE" in parts or "DA" in parts:
 if storage_length == 8:
 return True

 core = self.picture_core(
 picture=picture,
)

 if core in {"9(8)", "S9(8)", "99999999", "S99999999"}:
 if field_name.startswith("DA ") or " DATE" in field_name:
 return True

 return False

def basetype_for_datatype(
 self,
 datatype: str | None,
) -> str | None:
 datatype_upper = str(datatype or "").upper()

 if datatype_upper == "DATE":
 return "DATE"

 if datatype_upper in {"DECIMAL", "NUMERIC", "INTEGER"}:
 return "NUMERIC"

 if datatype_upper in {"CHAR", "VARCHAR"}:
 return "CHAR"

 return datatype

def enrich_group_positions_from_children(
 self,
 fields: list[DataField],
 debug_label: str = "",
) -> list[DataField]:
 output = list(fields or [])

 for index, field in enumerate(output):
 is_group = bool(
 getattr(field, "has_child", False)
 or getattr(field, "is_group", False)
)

 if not is_group:
 continue

 level = getattr(field, "level", None)

 try:
 parent_level = int(level)
 except Exception:
 continue

 child_starts: list[int] = []
 child_ends: list[int] = []

```

```

for child in output[index + 1:]:
 child_level = getattr(child, "level", None)

 try:
 current_level = int(child_level)
 except Exception:
 continue

 if current_level <= parent_level:
 break

 start = getattr(child, "start_position", None)
 end = getattr(child, "end_position", None)

 if start is not None:
 try:
 child_starts.append(int(start))
 except Exception:
 pass

 if end is not None:
 try:
 child_ends.append(int(end))
 except Exception:
 pass

 if not child_starts and not child_ends:
 continue

 group_start = min(child_starts) if child_starts else getattr(field, "start_position", None)
 group_end = max(child_ends) if child_ends else getattr(field, "end_position", None)

 group_length = getattr(field, "length", None)

 try:
 if group_start is not None and group_end is not None:
 group_length = int(group_end) - int(group_start) + 1
 except Exception:
 pass

 output[index] = self.make_field(
 source=field,
 datatype=getattr(field, "datatype", None),
 length=group_length,
 scale=getattr(field, "scale", None),
 picture=None,
 start_position=group_start,
 end_position=group_end,
 basetype=getattr(field, "basetype", None),
)

return output

def copy_field_with_defaults(
 self,
 field: DataField,
) -> DataField:
 return self.make_field(
 source=field,
 datatype=getattr(field, "datatype", None),
 length=getattr(field, "length", None),
 scale=getattr(field, "scale", None),
 picture=getattr(field, "picture", None),
 start_position=getattr(field, "start_position", None),
 end_position=getattr(field, "end_position", None),
 basetype=getattr(field, "basetype", None),
)

def make_field(
 self,
 source: DataField,
 datatype: str | None,
 length: int | None,
 scale: int | None,
 picture: str | None,

```

```

 start_position: int | None,
 end_position: int | None,
 basetype: str | None,
) -> DataField:
 data = source.model_dump()

 data.update(
 {
 "datatype": datatype,
 "length": length,
 "scale": scale,
 "picture": picture,
 "start_position": start_position,
 "end_position": end_position,
 "basetype": basetype,
 }
)

 return DataField(**data)

def remove_record_suffix(
 self,
 value: str,
) -> str:
 text = str(value or "").strip().upper()
 text = re.sub(r"[\s_-]+[0-9]{4}$", "", text)
 text = re.sub(r"[0-9]{4}$", "", text)
 text = re.sub(r"[\s_-]+$", "", text)

 return text

```

```

=====
FILE: src\idms_modernizer\services\validation_service.py
ENCODING: utf-8
=====

```

```

class ValidationService:

 def validate_metadata(
 self,
 metadata
) -> list[str]:

 errors = []

 record_names = {
 record.name
 for record in metadata.records
 }

 for rel in metadata.relationships:

 if rel.owner_record not in record_names:

 errors.append(
 f"Missing Owner Record: {rel.owner_record}"
)

 if rel.member_record not in record_names:

 errors.append(
 f"Missing Member Record: {rel.member_record}"
)

 return errors

```

```

=====
FILE: src\idms_modernizer\tests__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: src\idms_modernizer\tests\test_date_field_consolidator.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.services.date_field_consolidator import (
 DateFieldConsolidator
)
```

```
from idms_modernizer.domain.schema_models import (
 DataField
)
```

```
def test_consolidate_year_month_day():
 fields = [
 DataField(name="START_YEAR"),
 DataField(name="START_MONTH"),
 DataField(name="START_DAY")
]
```

```
 out = DateFieldConsolidator.consolidate(fields)
```

```
 assert len(out) == 1
 assert out[0].name == "START_DATE"
 assert out[0].datatype == "DATE"
```

```
def test_consolidate_variants():
 fields = [
 DataField(name="FINISH_YR"),
 DataField(name="FINISH_MO"),
 DataField(name="FINISH_DY")
]
```

```
 out = DateFieldConsolidator.consolidate(fields)
```

```
 assert len(out) == 1
 assert out[0].name == "FINISH_DATE"
 assert out[0].datatype == "DATE"
```

```
def test_preserve_existing_full_date():
 fields = [
 DataField(name="BIRTH_DATE", datatype="DATE"),
 DataField(name="BIRTH_YEAR"),
 DataField(name="BIRTH_MONTH")
]
```

```
 out = DateFieldConsolidator.consolidate(fields)
```

```
 # Should keep the existing BIRTH_DATE and not create duplicate
 names = [f.name for f in out]
 assert "BIRTH_DATE" in names
 assert all(n.startswith("BIRTH_") for n in names)
```

```
def test_keep_year_only():
 fields = [
 DataField(name="CLAIM_YEAR")
]
```

```
 out = DateFieldConsolidator.consolidate(fields)
```

```
 assert len(out) == 1
 assert out[0].name == "CLAIM_YEAR"
```

```
=====
FILE: src\idms_modernizer\tests\test_diagram_parser.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.parsers.pdf_diagram_parser import (
 PdfDiagramParser
)
```

```

def test_diagram_parser():

 parser = (
 PdfDiagramParser()
)

 relationships = (
 parser.parse(
 """
 DEPARTMENT
 EMPLOYEE
 """
)
)

 assert (
 len(relationships)
 == 1
)

 assert (
 relationships[0].owner
 == "DEPARTMENT"
)

=====
FILE: src\idms_modernizer\tests\test_field_extractor.py
ENCODING: utf-8
=====

import re
import sys
from dataclasses import dataclass, asdict
from typing import List, Optional, Set

----- PDF READER -----
def read_pdf_lines(pdf_path: str) -> List[str]:
 """
 Extract text lines from a PDF file using pdfminer.six.
 Returns a list of lines.
 """
 try:
 from pdfminer.high_level import extract_text
 except ImportError as e:
 raise RuntimeError(
 "pdfminer.six is required. Install it with:\n\n"
 " pip install pdfminer.six\n\n"
) from e

 text = extract_text(pdf_path) or ""
 # Split into lines while preserving basic line structure
 lines = text.splitlines()
 # Trim trailing spaces for consistency
 return [ln.rstrip("\r\n") for ln in lines]

----- DATA TYPES -----
@dataclass(frozen=True)
class DataField:
 name: str
 datatype: str
 length: Optional[int]
 scale: Optional[int]
 picture: str

----- EXTRACTOR -----
class FieldExtractor:
 """
 Robust parser for IDMS-like listing lines.

 - Extracts PICTURE regardless of position and with/without PIC/PICTURE keyword

```



```

- Recognizes DISPLAY, COMP, COMP-3 in any order relative to PICTURE
- Skips FILLER items by default (configurable)
- Removes duplicate names
- Maps:
 * X -> CHAR(1)
 * X(n) -> VARCHAR(n)
 * DISPLAY + 9/S9[/ (n)] -> INTEGER(length = digits)
 * COMP/COMP-3 + 9/S9[/ (n)] -> DECIMAL(precision = digits)
 * S9(n)V9(m)/9(n)V9(m) -> DECIMAL(precision = n+m, scale = m)
"""

def __init__(
 self,
 require_explicit_length: bool = False,
 include_filler: bool = False,
 auto_number_filler: bool = True,
):
 self.require_explicit_length = require_explicit_length
 self.include_filler = include_filler
 self.auto_number_filler = auto_number_filler

Start: <level> <name>
_R_START = re.compile(r"^\s*(\d{2})\s+([A-Z0-9\-\+])\b", re.IGNORECASE)

USAGE anywhere (allow "USAGE IS ...")
_R_USAGE = re.compile(r"\b(?:USAGE\s+IS\s+)?(DISPLAY|COMP-3|COMP)\b", re.IGNORECASE)

PICTURE anywhere; optional PIC/PICTURE keyword; tolerate spaces; no trailing \b
_R_PIC_ANYWHERE = re.compile(
 r"""
 (?:\b(?:PIC|PICTURE)\s*)?
 (
 S\s*9\s*$\s*\d+\s*$\s*V\s*9\s*$\s*\d+\s*$ | # S9(n)V9(m) / 9(n)V9(m)
 S\s*9\s*$\s*\d+\s*$ | # S9(n)
 9\s*$\s*\d+\s*$ | # 9(n)
 S\s*9 | # S9
 9 | # 9
 X\s*$\s*\d+\s*$ | # X(n)
 X | # X
)
 """,
 re.IGNORECASE | re.VERBOSE,
)

Helpers (operate on normalized picture like X(30), 9(7), S9(7)V9(2))
_R_CHAR = re.compile(r"^\s*X(\d+)\s*$", re.IGNORECASE) # X(n)
_R_INT = re.compile(r"^\s*9(?:\s*(\d+)\s*)?\s*$", re.IGNORECASE) # 9 / 9(n) / S9 / S9(n)
_R_DEC = re.compile(r"^\s*9\s*(\d+)\s*$\s*V9\s*(\d+)\s*$", re.IGNORECASE) # S9(n)V9(m) / 9(n)V9(m)

def extract(self, lines: List[str]) -> List[DataField]:
 seen: Set[str] = set()
 out: List[DataField] = []
 filler_idx = 0

 for raw in lines:
 line = raw.strip()
 if not line:
 continue

 m_start = self._R_START.match(line)
 if not m_start:
 continue

 _, name = m_start.groups()
 name_up = name.upper()

 # Handle FILLER items
 if name_up.startswith("FILLER"):
 if not self.include_filler:
 continue
 if self.auto_number_filler:
 filler_idx += 1
 name_up = f"FILLER_{filler_idx}"

 if name_up in seen:
 continue

```

```

USAGE default to DISPLAY if absent
m_usage = self._R_USAGE.search(line)
usage = m_usage.group(1).upper() if m_usage else "DISPLAY"

PICTURE capture
m_pic = self._R_PIC_ANYWHERE.search(line)
if not m_pic:
 continue

Normalize: remove spaces inside the picture and uppercase
pic = re.sub(r"\s+", "", m_pic.group(1).upper())

Optional filter: require explicit length (skip X, 9, S9)
if self.require_explicit_length and pic in {"X", "9", "S9"}:
 continue

dtype: str = "VARCHAR"
length: Optional[int] = None
scale: Optional[int] = None

Character
if pic == "X":
 dtype, length = "CHAR", 1
elif self._R_CHAR.fullmatch(pic):
 length = int(self._R_CHAR.fullmatch(pic).group(1))
 dtype = "VARCHAR"

Decimal with V
elif self._R_DEC.fullmatch(pic):
 int_len, frac_len = map(int, self._R_DEC.fullmatch(pic).groups())
 length = int_len + frac_len
 scale = frac_len
 dtype = "DECIMAL"

Integer-like 9/S9 with optional (n)
elif self._R_INT.fullmatch(pic):
 digits_str = self._R_INT.fullmatch(pic).group(1)
 digits = int(digits_str) if digits_str else 1
 length = digits
 dtype = "INTEGER" if usage == "DISPLAY" else "DECIMAL"
else:
 # Unrecognized picture; skip
 continue

seen.add(name_up)
out.append(
 DataField(
 name=name_up,
 datatype=dtype,
 length=length,
 scale=scale,
 picture=pic,
)
)

return out

```

```

----- DEMO (PDF PATH) -----

```

```

if __name__ == "__main__":

```

```

 # Option A: Provide the PDF file path here (edit this string)

```

```

 pdf_path = r"C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\src\idms_modernizer\data\IDMS_Schema_Listing.pdf"

```

```

 # Option B: Or pass the path as first CLI argument

```

```

 if len(sys.argv) > 1:

```

```

 pdf_path = sys.argv[1]

```

```

 # Read lines from PDF

```

```

 lines = read_pdf_lines(pdf_path)

```

```

 # Configure extractor as needed

```

```

 extractor = FieldExtractor(

```

```

 require_explicit_length=False, # set True to skip X / 9 / S9 without (n)

```

```

 include_filler=False, # set True to include FILLER items
)

```

```

 auto_number_filler=True
)

 fields = extractor.extract(lines)
 for f in fields:
 print(asdict(f))

=====
FILE: src\idms_modernizer\tests\test_normalization_mapping.py
ENCODING: utf-8
=====

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.services.date_field_consolidator import DateFieldConsolidator
from idms_modernizer.services.db2_mapping_service import DB2MappingService

def test_name_normalizer_removes_suffix_and_normalizes():
 assert (
 NameNormalizer.normalize(
 "CLAIM-YEAR-0405"
)
 == "CLAIM_YEAR"
)
 assert (
 NameNormalizer.normalize(
 "EMP-ID 0415"
)
 == "EMP_ID"
)
 assert (
 NameNormalizer.normalize(
 " CLAIM--MONTH-0405 "
)
 == "CLAIM_MONTH"
)

def test_date_field_consolidator_consolidates_suffixed_date_fields():
 fields = [
 DataField(name="CLAIM-YEAR-0405"),
 DataField(name="CLAIM-MONTH-0405"),
 DataField(name="CLAIM-DAY-0405"),
 DataField(name="OTHER-FIELD-0405"),
]

 consolidated = DateFieldConsolidator.consolidate(
 fields
)

 assert any(
 field.name == "CLAIM_DATE"
 and field.datatype == "DATE"
 for field in consolidated
)
 assert any(
 field.name == "OTHER-FIELD-0405"
 and field.datatype is None
 for field in consolidated
)
 assert len(consolidated) == 2

def test_db2_mapping_service_preserves_date_and_varchar_length():
 mapper = DB2MappingService()

 date_column = DataField(
 name="CLAIM_DATE",
 datatype="DATE"
)
 assert mapper.map_datatype(date_column) == "DATE"

 timestamp_column = DataField(
 name="EVENT_TS",

```

```

 datatype="TIMESTAMP"
)
 assert mapper.map_datatype(timestamp_column) == "TIMESTAMP"

 string_column = DataField(
 name="NAME",
 datatype="DISPLAY",
 length=40,
)
 assert mapper.map_datatype(string_column) == "VARCHAR(40)"

```

```

=====
FILE: src\idms_modernizer\tests\test_schema_parser.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.parsers.pdf_schema_parser import (
 PdfSchemaParser
)

```

```

def test_schema_parser():

```

```

 sample_text = """

 RECORD NAME..... EMPLOYEE

 EMP-ID-0415

 EMP-FIRST-NAME-0415

 EMP-LAST-NAME-0415

 RECORD NAME..... DEPARTMENT

 DEPT-ID-0416

 DEPT-NAME-0416

 """

```

```

 parser = (
 PdfSchemaParser()
)

```

```

 metadata = (
 parser.parse(
 sample_text
)
)

```

```

 assert (
 len(metadata.records)
 == 2
)

```

```

 assert (
 metadata.records[0].name
 == "EMPLOYEE"
)

```

```

 assert (
 len(
 metadata.records[0].fields
)
 == 3
)

```

```

=====
FILE: src\idms_modernizer\ui__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: src\idms_modernizer\ui\metadata_page.py
ENCODING: utf-8
=====
```

```
import streamlit as st

def render_metadata(
 metadata
):
 st.subheader(
 "Detected Records"
)

 for record in metadata.records:

 st.markdown(
 f"### {record.name}"
)

 st.write(
 f"Fields: {len(record.fields)}"
)

 field_names = [
 field.name
 for field in record.fields
]

 st.dataframe(
 field_names
)
```

```
=====
FILE: src\idms_modernizer\ui\phase2_converter_panel.py
ENCODING: utf-8
=====
```

```
import json
from pathlib import Path
from typing import Any

import streamlit as st

from idms_modernizer.parsers.cobol_pdf_extractor import CobolPdfExtractor
from idms_modernizer.services.phase2_conversion_bridge import (
 Phase2ConversionBridge
)

try:
 from idms_db2_converter.exceptions import ConversionError
except Exception:
 class ConversionError(Exception):
 pass

class Phase2ConverterPanel:
 """
 Streamlit panel for running Phase 2 conversion inside Phase 1 app.

 Inputs:
 - COBOL text, COBOL PDF, or COBOL text file
 - Canonical JSON generated by Phase 1
 - Phase 2 metadata JSON generated by Phase 1
 - DB2 DDL generated by Phase 1
 - Optional IDMS schema text
 - Optional relationship overrides

 Important:
 - No hardcoded relationship overrides.
 - Relationship override JSON is dynamically generated from Phase 2 metadata
```

```

 when the user chooses to use overrides.
 """

```

```

def render(
 self,
 canonical_json: str,
 phase2_metadata_json: str,
 ddl_text: str,
 idms_schema_text: str | None = None
) -> None:

 st.header(
 "Phase 2 - IDMS Retrieval COBOL to DB2 COBOL"
)

 st.markdown(
 """

```

Use this section to convert an IDMS retrieval COBOL program into DB2 embedded SQL COBOL using the Phase 1 outputs generated above.

```

 """

)

 with st.expander(
 "Phase 1 Inputs Passed to Phase 2",
 expanded=False
):
 st.subheader(
 "Canonical JSON"
)

 st.code(
 canonical_json or "",
 language="json"
)

 st.subheader(
 "Phase 2 Metadata JSON"
)

 st.code(
 phase2_metadata_json or "",
 language="json"
)

 st.subheader(
 "DB2 DDL"
)

 st.code(
 ddl_text or "",
 language="sql"
)

 st.subheader(
 "COBOL Input"
)

 cobol_text = self._render_cobol_input()

 st.subheader(
 "Conversion Settings"
)

 target_program = st.text_input(
 "Target PROGRAM-ID",
 value="",
 placeholder="Example: DEPTDB2"
)

 col1, col2, col3 = st.columns(3)

 with col1:
 use_ddl = st.checkbox(
 "Use DB2 DDL",
 value=True,
 help="Recommended. DDL should be the physical source of truth."

```

```

)

with col2:
 use_phase2_metadata = st.checkbox(
 "Use Phase 2 Metadata",
 value=True,
 help="Recommended. Uses Phase 1 generated mapping metadata."
)

with col3:
 use_idms_schema = st.checkbox(
 "Use IDMS Schema Text",
 value=False,
 help="Optional. Usually not required if canonical, DDL, and Phase 2 metadata are provided."
)

use_overrides = st.checkbox(
 "Use relationship key overrides",
 value=False,
 help="Optional. Dynamically generated from Phase 2 metadata when enabled."
)

relationship_overrides_json = ""

if use_overrides:
 relationship_overrides_json = self._render_relationship_overrides(
 phase2_metadata_json=phase2_metadata_json
)

phase2_idms_schema_text = ""

if use_idms_schema:
 phase2_idms_schema_text = st.text_area(
 "Paste IDMS schema/subschema listing text",
 height=300,
 value=idms_schema_text or ""
)

convert_clicked = st.button(
 "Convert to DB2 COBOL",
 type="primary"
)

if convert_clicked:
 self._run_conversion(
 cobol_text=cobol_text,
 canonical_json=canonical_json,
 phase2_metadata_json=(
 phase2_metadata_json
 if use_phase2_metadata
 else ""
),
 ddl_text=(
 ddl_text
 if use_ddl
 else ""
),
 idms_schema_text=(
 phase2_idms_schema_text
 if use_idms_schema
 else None
),
 relationship_overrides_json=(
 relationship_overrides_json
 if use_overrides
 else None
),
 target_program=target_program,
 use_ddl=use_ddl,
 use_idms_schema=use_idms_schema,
 use_overrides=use_overrides
)

def _render_cobol_input(
 self
) -> str:

```

```

input_mode = st.radio(
 "Choose COBOL input method",
 [
 "Paste COBOL",
 "Upload COBOL PDF",
 "Upload COBOL Text File"
],
 horizontal=True
)

cobol_text = ""

if input_mode == "Paste COBOL":
 cobol_text = st.text_area(
 "Paste IDMS COBOL retrieval program",
 height=420,
 placeholder="Paste DEPTRPT or another IDMS retrieval program here..."
)

elif input_mode == "Upload COBOL PDF":
 cobol_text = self._load_cobol_from_pdf()

elif input_mode == "Upload COBOL Text File":
 cobol_text = self._load_cobol_from_text_file()

return cobol_text

def _load_cobol_from_pdf(
 self
) -> str:

 cobol_text = ""

 cobol_pdf = st.file_uploader(
 "Upload COBOL PDF",
 type=[
 "pdf"
],
 key="phase2_cobol_pdf"
)

 if cobol_pdf is None:
 return cobol_text

 temp_path = self._save_uploaded_file(
 uploaded_file=cobol_pdf,
 folder="data/temp"
)

 extractor = CobolPdfExtractor()

 try:
 cobol_text = extractor.extract_text(
 temp_path
)

 st.success(
 "COBOL PDF extracted successfully."
)

 cobol_text = st.text_area(
 "Extracted COBOL text",
 value=cobol_text,
 height=420
)

 except Exception as ex:
 st.error(
 f"Failed to extract COBOL PDF: {ex}"
)

 return cobol_text

def _load_cobol_from_text_file(
 self

```



```

) -> str:

cobol_text = ""

cobol_file = st.file_uploader(
 "Upload COBOL source file",
 type=[
 "cbl",
 "cob",
 "txt"
],
 key="phase2_cobol_text"
)

if cobol_file is None:
 return cobol_text

try:
 cobol_text = cobol_file.read().decode(
 "utf-8",
 errors="ignore"
)

 st.success(
 "COBOL text file loaded successfully."
)

 cobol_text = st.text_area(
 "Loaded COBOL text",
 value=cobol_text,
 height=420
)

except Exception as ex:
 st.error(
 f"Failed to read COBOL file: {ex}"
)

return cobol_text

def _render_relationship_overrides(
 self,
 phase2_metadata_json: str
) -> str:

```

```

 st.markdown(
 """
Relationship overrides are generated from the current Phase 2 metadata.
You can edit them before conversion if needed.
"""
)

```

```

 generated_overrides = self._build_overrides_from_phase2_metadata(
 phase2_metadata_json
)

 if not generated_overrides:
 st.info(
 "No complete relationship overrides could be generated from Phase 2 metadata."
)

 generated_overrides = {
 "relationships": []
 }

 return st.text_area(
 "Relationship key overrides JSON",
 height=300,
 value=json.dumps(
 generated_overrides,
 indent=2
)
)

def _build_overrides_from_phase2_metadata(
 self,

```

```

 phase2_metadata_json: str
) -> dict[str, Any]:

 if not phase2_metadata_json or not phase2_metadata_json.strip():
 return {
 "relationships": []
 }

 try:
 payload = json.loads(
 phase2_metadata_json
)

 except json.JSONDecodeError:
 return {
 "relationships": []
 }

 relationships = payload.get(
 "relationships",
 []
)

 override_relationships = []

 for relationship in relationships:
 set_name = self._get_first_non_empty(
 relationship,
 [
 "set_name",
 "name",
 "relationship"
]
)

 parent_record = self._get_first_non_empty(
 relationship,
 [
 "parent_record",
 "parent_table"
]
)

 child_record = self._get_first_non_empty(
 relationship,
 [
 "child_record",
 "child_table"
]
)

 parent_key = self._get_first_non_empty(
 relationship,
 [
 "parent_key",
 "parent_pk"
]
)

 child_fk = self._get_first_non_empty(
 relationship,
 [
 "child_fk",
 "foreign_key"
]
)

 order_by = relationship.get(
 "order_by",
 []
)

 if not set_name:
 continue

 if not parent_record:

```

```

 continue

 if not child_record:
 continue

 item = {
 "set_name": self._to_idms_name(set_name),
 "parent_record": self._normalize_name(parent_record),
 "child_record": self._normalize_name(child_record)
 }

 if parent_key:
 item[
 "parent_key"
] = self._normalize_name(parent_key)

 if child_fk:
 item[
 "child_fk"
] = self._normalize_name(child_fk)

 if order_by:
 item[
 "order_by"
] = [
 self._normalize_name(value)
 for value in order_by
 if value
]

 override_relationships.append(
 item
)

 return {
 "relationships": override_relationships
 }

def _run_conversion(
 self,
 cobol_text: str,
 canonical_json: str,
 phase2_metadata_json: str,
 ddl_text: str,
 idms_schema_text: str | None,
 relationship_overrides_json: str | None,
 target_program: str | None,
 use_ddl: bool,
 use_idms_schema: bool,
 use_overrides: bool
) -> None:

 if not cobol_text or not cobol_text.strip():
 st.error(
 "COBOL code is required."
)
 st.stop()

 bridge = Phase2ConversionBridge()

 try:
 converted, validation_messages = bridge.convert(
 cobol_text=cobol_text,
 canonical_json=canonical_json,
 phase2_metadata_json=phase2_metadata_json,
 ddl_text=ddl_text,
 idms_schema_text=idms_schema_text,
 relationship_overrides_json=relationship_overrides_json,
 target_program=target_program,
 use_ddl=use_ddl,
 use_idms_schema=use_idms_schema,
 use_overrides=use_overrides
)

 st.success(
 "Phase 2 conversion completed."
)

```

```

)

 if validation_messages:
 st.warning(
 "Conversion completed with validation messages."
)

 for message in validation_messages:
 st.write(
 f"- {message}"
)

 else:
 st.info(
 "Validation passed."
)

 st.subheader(
 "Final DB2 COBOL"
)

 st.text_area(
 "Copy converted DB2 COBOL",
 value=converted,
 height=700
)

except ConversionError as exc:
 st.error(
 "Phase 2 conversion failed."
)

 st.code(
 str(exc),
 language="text"
)

except Exception as exc:
 st.error(
 "Unexpected Phase 2 conversion error."
)

 st.code(
 str(exc),
 language="text"
)

def _save_uploaded_file(
 self,
 uploaded_file,
 folder: str
) -> str:

 target_folder = Path(
 folder
)

 target_folder.mkdir(
 parents=True,
 exist_ok=True
)

 target_path = (
 target_folder
 / uploaded_file.name
)

 with open(
 target_path,
 "wb"
) as file:
 file.write(
 uploaded_file.getbuffer()
)

 return str(

```

```

 target_path
)

def _get_first_non_empty(
 self,
 source: dict[str, Any],
 keys: list[str]
) -> str | None:

 for key in keys:
 value = source.get(
 key
)

 if value is None:
 continue

 value = str(
 value
).strip()

 if value:
 return value

 return None

def _normalize_name(
 self,
 value: str | None
) -> str:

 if value is None:
 return ""

 return str(
 value
).strip().upper().replace(
 "-",
 "_"
)

def _to_idms_name(
 self,
 value: str | None
) -> str:

 if value is None:
 return ""

 return str(
 value
).strip().upper().replace(
 "-",
 "_"
)

```

```

=====
FILE: src\idms_modernizer\ui\schema_listing_only_app.py
ENCODING: utf-8
=====

```

```

from pathlib import Path
from tempfile import NamedTemporaryFile

import streamlit as st

from idms_modernizer.core.config import settings
from idms_modernizer.core.utils import ensure_directory
from idms_modernizer.services.metadata_service import MetadataService
from idms_modernizer.services.canonical_schema_builder import (
 CanonicalSchemaBuilder,
)
from idms_modernizer.services.canonical_reconciler import (
 CanonicalReconciler,
)

```

```

from idms_modernizer.services.db2_model_builder import DB2ModelBuilder
from idms_modernizer.generators.ddl_generator import DDLGenerator
from idms_modernizer.services.excel_sheet_mapping_service import (
 ExcelSheetMappingService,
)

```

```

def initialize_directories() -> None:
 ensure_directory(
 settings.output_dir,
)

```

```

def initialize_session_state() -> None:
 defaults = {
 "schema_only_generated": False,
 "schema_only_error": "",
 "schema_only_schema_path": "",
 "schema_only_metadata": None,
 "schema_only_canonical_schema": None,
 "schema_only_db2_model": None,
 "schema_only_ddl_text": "",
 "schema_only_excel_mapping_rows": [],
 }

```

```

 for key, value in defaults.items():
 if key not in st.session_state:
 st.session_state[key] = value

```

```

def reset_schema_only_outputs() -> None:
 st.session_state.schema_only_generated = False
 st.session_state.schema_only_error = ""
 st.session_state.schema_only_schema_path = ""
 st.session_state.schema_only_metadata = None
 st.session_state.schema_only_canonical_schema = None
 st.session_state.schema_only_db2_model = None
 st.session_state.schema_only_ddl_text = ""
 st.session_state.schema_only_excel_mapping_rows = []

```

```

def save_uploaded_schema_file(
 uploaded_file,
) -> str:
 suffix = Path(
 uploaded_file.name,
).suffix or ".pdf"

 with NamedTemporaryFile(
 delete=False,
 suffix=suffix,
) as temp_file:
 temp_file.write(
 uploaded_file.getbuffer(),
)

 return temp_file.name

```

```

def generate_schema_outputs(
 schema_pdf,
) -> None:
 reset_schema_only_outputs()

 try:
 schema_path = save_uploaded_schema_file(
 uploaded_file=schema_pdf,
)

 metadata_service = MetadataService()

 metadata = metadata_service.build_metadata(
 pdf_path=schema_path,
)

 canonical_builder = CanonicalSchemaBuilder()

```

```

canonical_schema = canonical_builder.build(
 metadata=metadata,
)

canonical_reconciler = CanonicalReconciler()

canonical_schema = canonical_reconciler.reconcile(
 canonical_schema=canonical_schema,
 metadata=metadata,
)

db2_builder = DB2ModelBuilder()

db2_model = db2_builder.build(
 canonical_schema,
)

ddl_generator = DDLGenerator()

ddl_text = ddl_generator.generate(
 db2_model,
)

excel_mapping_service = ExcelSheetMappingService()

excel_mapping_rows = excel_mapping_service.build(
 metadata=metadata,
 db2_model=db2_model,
)

st.session_state.schema_only_schema_path = schema_path
st.session_state.schema_only_metadata = metadata
st.session_state.schema_only_canonical_schema = canonical_schema
st.session_state.schema_only_db2_model = db2_model
st.session_state.schema_only_ddl_text = ddl_text
st.session_state.schema_only_excel_mapping_rows = excel_mapping_rows
st.session_state.schema_only_generated = True

except Exception as exc:
 st.session_state.schema_only_generated = False
 st.session_state.schema_only_error = str(
 exc,
)
)

def build_mapping_row_count_by_table(
 rows: list[dict],
 include_total: bool = True,
) -> list[dict[str, int | str]]:
 """
 Builds table-level counts from the actual Excel Sheet Mapping rows.

 This count matches the detailed mapping table because it counts the
 generated mapping rows, not only DB2 physical columns.

 Output column order:
 - Column Field Count
 - Table Name
 """
 table_counts: dict[str, int] = {}

 for row in rows or []:
 table_name = (
 row.get("New DB2 Record")
 or row.get("Cobol Record IDMS")
 or ""
)

 table_name = str(
 table_name,
).strip()

 if not table_name:
 continue

```

```

 table_counts[table_name] = table_counts.get(
 table_name,
 0,
) + 1

output_rows: list[dict[str, int | str]] = []

total_count = 0

for table_name in sorted(table_counts):
 count = table_counts[table_name]

 output_rows.append(
 {
 "Column Field Count": count,
 "Table Name": table_name,
 }
)

 total_count += count

if include_total:
 output_rows.append(
 {
 "Column Field Count": total_count,
 "Table Name": "TOTAL",
 }
)

return output_rows

def build_db2_physical_column_count_rows(
 db2_model,
 include_total: bool = True,
) -> list[dict[str, int | str]]:
 """
 Builds table-level physical DB2 column counts.

 This is separate from the mapping row count because DATE child fields
 can create multiple mapping rows but only one DB2 physical DATE column.
 """
 table_rows: list[dict[str, int | str]] = []

 if db2_model is None:
 return table_rows

 total_count = 0

 for table in getattr(
 db2_model,
 "tables",
 [],
) or []:
 table_name = getattr(
 table,
 "name",
 "",
)

 columns = getattr(
 table,
 "columns",
 [],
) or []

 count = len(
 columns,
)

 table_rows.append(
 {
 "Column Field Count": count,
 "Table Name": table_name,
 }
)

```



```

 total_count += count

 if include_total:
 table_rows.append(
 {
 "Column Field Count": total_count,
 "Table Name": "TOTAL",
 }
)

 return table_rows

def render_main_tab() -> None:
 st.markdown("## Schema Listing Only")

 st.info(
 "Upload only the Schema Listing PDF to generate Metadata Overview, "
 "DB2 DDL, and Excel Sheet Mapping."
)

 schema_pdf = st.file_uploader(
 "Schema Listing PDF",
 type=["pdf"],
 key="schema_only_schema_listing_pdf",
)

 generate_clicked = st.button(
 "Generate Schema Outputs",
 type="primary",
 use_container_width=True,
)

 if generate_clicked:
 if schema_pdf is None:
 st.error(
 "Please upload a Schema Listing PDF."
)
 return

 with st.spinner(
 "Processing schema listing and generating outputs..."
):
 generate_schema_outputs(
 schema_pdf=schema_pdf,
)

 if st.session_state.schema_only_generated:
 st.success(
 "Schema outputs generated successfully."
)
 else:
 st.error(
 "Schema output generation failed."
)

 if st.session_state.schema_only_error:
 st.error(
 st.session_state.schema_only_error,
)

 if st.session_state.schema_only_generated:
 st.markdown("### Output Status")

 metadata = st.session_state.schema_only_metadata
 db2_model = st.session_state.schema_only_db2_model
 mapping_rows = st.session_state.schema_only_excel_mapping_rows

 col1, col2, col3, col4 = st.columns(
 4,
)

 with col1:
 st.metric(
 "Records",

```

```

 len(
 metadata.records,
)
 if metadata is not None
 else 0,
)

with col2:
 st.metric(
 "Relationships",
 len(
 metadata.relationships,
)
 if metadata is not None
 else 0,
)

with col3:
 st.metric(
 "DB2 Tables",
 len(
 db2_model.tables,
)
 if db2_model is not None
 else 0,
)

with col4:
 st.metric(
 "Sheet Mapping Rows",
 len(
 mapping_rows,
)
 if mapping_rows is not None
 else 0,
)

def render_metadata_overview_tab() -> None:
 if not st.session_state.schema_only_generated:
 st.info(
 "Generate outputs from the Main tab first."
)
 return

 metadata = st.session_state.schema_only_metadata
 db2_model = st.session_state.schema_only_db2_model
 mapping_rows = st.session_state.schema_only_excel_mapping_rows

 if metadata is None:
 st.info(
 "No metadata available."
)
 return

 st.markdown("## Metadata Overview")

 st.markdown("### Records")

 record_rows = []

 for record in metadata.records:
 record_rows.append(
 {
 "Record": record.name,
 "Cobol Zone": record.cobol_zone or "",
 "Primary Key": record.primary_key or "",
 "Field Count": len(
 record.fields,
),
 "Set Membership Count": len(
 record.set_memberships,
),
 }
)
)

```

```

st.dataframe(
 record_rows,
 use_container_width=True,
 hide_index=True,
)

st.divider()

st.markdown("### Table List and Mapping Row Counts")

mapping_count_rows = build_mapping_row_count_by_table(
 rows=mapping_rows,
 include_total=True,
)

if mapping_count_rows:
 st.dataframe(
 mapping_count_rows,
 use_container_width=True,
 hide_index=True,
)
else:
 st.info(
 "No table mapping row counts available."
)

st.caption(
 "This count is based on the actual Excel Sheet Mapping rows, "
 "so the TOTAL matches the detailed mapping table."
)

st.divider()

st.markdown("### DB2 Physical Column Counts")

physical_column_count_rows = build_db2_physical_column_count_rows(
 db2_model=db2_model,
 include_total=True,
)

if physical_column_count_rows:
 st.dataframe(
 physical_column_count_rows,
 use_container_width=True,
 hide_index=True,
)
else:
 st.info(
 "No DB2 physical column counts available."
)

st.caption(
 "This count is based on DB2 physical columns. It can be lower than "
 "the mapping row count when COBOL YEAR, MONTH, and DAY fields map "
 "to one DB2 DATE column."
)

st.divider()

st.markdown("### Records and Fields")

record_names = [
 record.name
 for record in metadata.records
]

if not record_names:
 st.info(
 "No records found."
)
 return

selected_record = st.selectbox(
 "Select a record",
 options=record_names,
)

```

```

selected_record_obj = None

for record in metadata.records:
 if record.name == selected_record:
 selected_record_obj = record
 break

if selected_record_obj is not None:
 st.write(
 f"Fields found: `{len(selected_record_obj.fields)}`"
)

 field_rows = []

 for field in selected_record_obj.fields:
 field_rows.append(
 {
 "Field": field.name,
 "Level": field.level,
 "Datatype": field.datatype,
 "Length": field.length,
 "Scale": field.scale,
 "Picture": field.picture,
 "Start Position": field.start_position,
 "End Position": field.end_position,
 "Basetype": field.basetype,
 }
)

 st.dataframe(
 field_rows,
 use_container_width=True,
 hide_index=True,
)

with st.expander(
 "View all records and fields as JSON",
 expanded=False,
):
 st.json(
 [
 {
 "record": record.name,
 "cobol_zone": record.cobol_zone,
 "primary_key": record.primary_key,
 "fields": [
 field.model_dump()
 for field in record.fields
],
 "set_memberships": [
 {
 "set_name": getattr(
 membership,
 "set_name",
 "",
),
 "relation_type": getattr(
 membership,
 "relation_type",
 "",
),
 "owner_record": getattr(
 membership,
 "owner_record",
 "",
),
 "member_record": getattr(
 membership,
 "member_record",
 "",
),
 }
 for membership in record.set_memberships
],
 }
],
)

```

```

 for record in metadata.records
]
)

def render_ddl_tab() -> None:
 if not st.session_state.schema_only_generated:
 st.info(
 "Generate outputs from the Main tab first."
)
 return

 st.markdown("## DB2 DDL")

 ddl_text = st.session_state.schema_only_ddl_text

 if not ddl_text:
 st.info(
 "No DDL generated."
)
 return

 st.code(
 ddl_text,
 language="sql",
)

def render_excel_sheet_mapping_tab() -> None:
 if not st.session_state.schema_only_generated:
 st.info(
 "Generate outputs from the Main tab first."
)
 return

 st.markdown("## Excel Sheet Mapping")

 rows = st.session_state.schema_only_excel_mapping_rows

 if not rows:
 st.info(
 "No mapping rows generated."
)
 return

 st.markdown("### Sheet Map Abstract")

 abstract_rows = build_mapping_row_count_by_table(
 rows=rows,
 include_total=True,
)

 if abstract_rows:
 st.dataframe(
 abstract_rows,
 use_container_width=True,
 hide_index=True,
)
 else:
 st.info(
 "No table abstract rows available."
)

 st.caption(
 "The TOTAL row matches the number of detailed mapping rows below."
)

 st.markdown("### Sheet Mapping Details")

 st.dataframe(
 rows,
 use_container_width=True,
 hide_index=True,
)

```

```

def main() -> None:
 st.set_page_config(
 page_title="Schema Listing Mapping",
 layout="wide",
)

 initialize_directories()
 initialize_session_state()

 st.title(
 "Schema Listing Mapping"
)

 st.caption(
 "Upload Schema Listing PDF only and generate DB2 DDL plus "
 "Excel-style mapping."
)

 tabs = st.tabs(
 [
 "Main",
 "Metadata Overview",
 "DB2 DDL",
 "Excel Sheet Mapping",
]
)

 with tabs[0]:
 render_main_tab()

 with tabs[1]:
 render_metadata_overview_tab()

 with tabs[2]:
 render_ddl_tab()

 with tabs[3]:
 render_excel_sheet_mapping_tab()

if __name__ == "__main__":
 main()

```

```

=====
FILE: src\idms_modernizer\ui\upload_page.py
ENCODING: utf-8
=====

```

```

from pathlib import Path

import streamlit as st

from idms_modernizer.core.config import (
 settings
)

```

```

def render_upload_page():

 st.subheader(
 "Upload IDMS Schema PDF"
)

 uploaded_file = (
 st.file_uploader(
 "Schema PDF",
 type=["pdf"]
)
)

 if uploaded_file:

 save_path = (
 Path(
 settings.input_dir
)
)

```

```

)
 / uploaded_file.name
)

 with open(
 save_path,
 "wb"
) as file:

 file.write(
 uploaded_file.read()
)

 st.success(
 f"Saved: {save_path}"
)

 return str(save_path)

return None

```

```

=====
FILE: src\tools\audit_hardcoding.py
ENCODING: utf-8
=====

```

```

import re
from pathlib import Path

```

```

SEARCH_TERMS = [
 "0400",
 "0405",
 "0410",
 "0415",
 "0420",
 "0425",
 "0430",
 "0435",
 "0440",
 "0445",
 "0450",
 "0455",
 "0460",
 "DEPARTMENT",
 "EMPLOYEE",
 "EMPOSITION",
 "JOB",
 "OFFICE",
 "COVERAGE",
 "EXPERTISE",
 "NON-HOSP",
 "SKILL",
 "STRUCTURE",
 "DEPT-EMPLOYEE",
 "EMP-EMPOSITION",
 "JOB-EMPOSITION",
 "OFFICE-EMPLOYEE",
 "NEW-DEPT-STORE-DEPARTMENT",
 "OLD-DEPT-ERASE-DEPARTMENT",
 "OLD-LASTNAME-MODIFY-EMPLOYEE",
]

```

```

SKIP_DIRS = {
 ".git",
 ".venv",
 "__pycache__",
 "data",
 "output",
 "temp",
 "logs",
 "tests",
}

```

```

SKIP_FILES = {
 "audit_hardcoding.py",
 "installed.txt",
}

INCLUDE_SUFFIXES = {
 ".py",
}

FALSE_POSITIVE_SUBSTRINGS = {
 "STRUCTUREDBLOCKS",
 "CLASS STRUCTUREDBLOCKS",
 "FROM IDMS_DB2_CONVERTER.TRANSFORMERS.STRUCTURED_BLOCKS",
 "IDMS_DB2_CONVERTER.TRANSFORMERS.STRUCTURED_BLOCKS",
}

def should_skip(
 path: Path,
) -> bool:
 if path.name in SKIP_FILES:
 return True

 parts = set(
 part.lower()
 for part in path.parts
)

 if parts.intersection(SKIP_DIRS):
 return True

 if path.suffix.lower() not in INCLUDE_SUFFIXES:
 return True

 return False

def strip_docstrings(
 text: str,
) -> str:
 text = re.sub(
 r'(""".*?""")',
 "",
 text,
 flags=re.DOTALL,
)

 text = re.sub(
 r'('''.*?''')',
 "",
 text,
 flags=re.DOTALL,
)

 return text

def is_likely_comment_line(
 line: str,
) -> bool:
 stripped = line.strip()

 if not stripped:
 return True

 return stripped.startswith(
 (
 "#",
 "*",
 "- ",
)
)

```



```

def contains_false_positive(
 line: str,
) -> bool:
 upper_line = line.upper()

 for value in FALSE_POSITIVE_SUBSTRINGS:
 if value in upper_line:
 return True

 return False

def term_pattern(
 term: str,
) -> re.Pattern:
 escaped = re.escape(
 term,
)

 if re.fullmatch(
 r"\d+",
 term,
):
 return re.compile(
 rf"(?<!\d){escaped}(?!\\d)",
 re.IGNORECASE,
)

 return re.compile(
 rf"(?<![A-Z0-9_-]){escaped}(?![A-Z0-9_-])",
 re.IGNORECASE,
)

def main() -> None:
 root = Path.cwd()

 findings: list[str] = []

 compiled_terms = [
 (
 term,
 term_pattern(term),
)
 for term in SEARCH_TERMS
]

 for path in root.rglob("*"):
 if not path.is_file():
 continue

 if should_skip(path):
 continue

 try:
 text = path.read_text(
 encoding="utf-8",
 errors="ignore",
)
 except Exception:
 continue

 text_without_docstrings = strip_docstrings(
 text,
)

 for line_number, line in enumerate(
 text_without_docstrings.splitlines(),
 start=1,
):
 if is_likely_comment_line(
 line,
):
 continue

```

```

 if contains_false_positive(
 line,
):
 continue

 for term, pattern in compiled_terms:
 if pattern.search(line):
 findings.append(
 f"{path}:{line_number}: {line.strip()}"
)

 if not findings:
 print(
 "No hardcoded demo terms found in scanned Python logic."
)
 return

 print(
 "Potential hardcoded demo terms found in Python logic:"
)

 print()

 for item in findings:
 print(
 item,
)

if __name__ == "__main__":
 main()

```

```

=====
FILE: test
ENCODING: utf-8
=====

```

```

digraph G { A -> B }

```

```

=====
FILE: uv.lock
ENCODING: utf-8
=====

```

```

version = 1
revision = 3
requires-python = ">=3.12"
resolution-markers = [
 "python_full_version >= '3.15' and sys_platform == 'win32'",
 "python_full_version == '3.14.*' and sys_platform == 'win32'",
 "python_full_version >= '3.15' and sys_platform == 'emscripten'",
 "python_full_version == '3.14.*' and sys_platform == 'emscripten'",
 "python_full_version >= '3.15' and sys_platform == 'darwin'",
 "python_full_version >= '3.15' and platform_machine == 'aarch64' and sys_platform == 'linux'",
 "(python_full_version >= '3.15' and platform_machine != 'aarch64' and sys_platform == 'linux') or (python_full_vers
ion >= '3.15' and sys_platform != 'darwin' and sys_platform != 'emscripten' and sys_platform != 'linux' and sys_platfor
m != 'win32')",
 "python_full_version == '3.14.*' and sys_platform == 'darwin'",
 "python_full_version == '3.14.*' and platform_machine == 'aarch64' and sys_platform == 'linux'",
 "(python_full_version == '3.14.*' and platform_machine != 'aarch64' and sys_platform == 'linux') or (python_full_ve
rsion == '3.14.*' and sys_platform != 'darwin' and sys_platform != 'emscripten' and sys_platform != 'linux' and sys_pla
tform != 'win32')",
 "python_full_version < '3.14' and sys_platform == 'win32'",
 "python_full_version < '3.14' and sys_platform == 'emscripten'",
 "python_full_version < '3.14' and sys_platform == 'darwin'",
 "python_full_version < '3.14' and platform_machine == 'aarch64' and sys_platform == 'linux'",
 "(python_full_version < '3.14' and platform_machine != 'aarch64' and sys_platform == 'linux') or (python_full_versi
on < '3.14' and sys_platform != 'darwin' and sys_platform != 'emscripten' and sys_platform != 'linux' and sys_platform
!= 'win32')",
]

```

```

[[package]]
name = "altair"
version = "6.2.1"

```

```

source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "jinja2" },
 { name = "jsonschema" },
 { name = "narwhals" },
 { name = "packaging" },
 { name = "typing-extensions", marker = "python_full_version < '3.15'" },
]
sdist = { url = "https://files.pythonhosted.org/packages/86/97/9a0dc61efd4f2dee29cb6d8edbbacdb789ce48cbffd98efa2b3ab145b297/altair-6.2.1.tar.gz", hash = "sha256:ca0298fa20b1a4fae22eff8847b95f74912bd90544013ad36af192119883ea64", size = 766468, upload-time = "2026-06-05T16:23:36.57Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/dc/78/b556548d92b9e29ae68a86e7b41688820900809e189e39caf308c7d44a3/altair-6.2.1-py3-none-any.whl", hash = "sha256:bf2fee3733c3a31a588e45b857a2495a88d506970deb87f74e1613f0247446b1", size = 797498, upload-time = "2026-06-05T16:23:34.799Z" },
]

[[package]]
name = "annotated-types"
version = "0.7.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/ee/67/531ea369ba64dcff5ec9c3402f9f51bf748cec26dde048a2f973a4eea7f5/annotated_types-0.7.0.tar.gz", hash = "sha256:aff07c09a53a08bc8cfcc9c85b05f1aa9a2a6f23728d790723543408344ce89", size = 16081, upload-time = "2024-05-20T21:33:25.928Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/78/b6/6307fbef88d9b5ee7421e68d78a9f162e0da4900bc5f5793f6d3d0e34fb8/annotated_types-0.7.0-py3-none-any.whl", hash = "sha256:1f02e8b43a8fbcc3f3e0d4f0f4bfc8131bcb4eebe8849b8e5c773f3a1c582a53", size = 13643, upload-time = "2024-05-20T21:33:24.1Z" },
]

[[package]]
name = "anyio"
version = "4.14.0"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "idna" },
 { name = "typing-extensions", marker = "python_full_version < '3.13'" },
]
sdist = { url = "https://files.pythonhosted.org/packages/1c/b5/001890774a9552aff22502b8da382593109ce0c95314abaebbb116567545/anyio-4.14.0.tar.gz", hash = "sha256:b47c1f9ccf73e67021df785332508f99379c68fa7d0684e8e3492cb1d4b23f89", size = 253586, upload-time = "2026-06-15T22:00:49.021Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/ba/16/9826f089383c593cdfc4a6e5aca94d9e91ae1692c57af82c3b2aa5e810f7/anyio-4.14.0-py3-none-any.whl", hash = "sha256:dd9b7a2a9799ed6552fde617b2c5df02b7fdd7d88392cf4810e51bae46164d9", size = 123506, upload-time = "2026-06-15T22:00:47.595Z" },
]

[[package]]
name = "attrs"
version = "26.1.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/9a/8e/82a0fe20a541c03148528be8cac2408564a6c9a0cc7e9171802bc1d26985/attrs-26.1.0.tar.gz", hash = "sha256:d03ceb89cb322a8fd706d4fb91940737b6642aa36998fe130a9bc96c985eff32", size = 952055, upload-time = "2026-03-19T14:22:25.026Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/64/b4/17d4b0b2a2dc85a6df63d1157e028ed19f90d4cd97c36717afef2bc2f395/attrs-26.1.0-py3-none-any.whl", hash = "sha256:c647aa4a12dfbad9333ca4e71fe62ddc36f4e63b2d260a37a8b83d2f043ac309", size = 67548, upload-time = "2026-03-19T14:22:23.645Z" },
]

[[package]]
name = "blinker"
version = "1.9.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/21/28/9b3f50ce0e048515135495f198351908d99540d69bfdc8c1d15b73dc55ce/blinker-1.9.0.tar.gz", hash = "sha256:b4ce2265a7abece45e7cc896e98dbebe6cead56bcf805a3d23136d145f5445bf", size = 22460, upload-time = "2024-11-08T17:25:47.436Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/10/cb/f2ad4230dc2eb1a74edf38f1a38b9b52277f75bef262d8908e60d957e13c/blinker-1.9.0-py3-none-any.whl", hash = "sha256:ba0efaa9080b619ff2f3459d1d500c57bddea4a6b424b60a91141db6fd2f08bc", size = 8458, upload-time = "2024-11-08T17:25:46.184Z" },
]

[[package]]
name = "cachetools"
version = "7.1.4"

```

```

source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/f4/8b/0d3945a13955303b81272f759a0331e54c5c793da455e6f5706b89d2639c/cachetools-7.1.4.tar.gz", hash = "sha256:437f55a4e0c1b01a4f3077cc470e6991d47430970e36fbc77e2be0df4fc1cd6", size = 40085, upload-time = "2026-05-21T22:40:43.376Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/8c/7b/1fc1c09cc0756cf25861a3be10565915953876da48bb228fb9a672b20a42/cachetools-7.1.4-py3-none-any.whl", hash = "sha256:323dc4127934744db5b54eb4924482d7edafbf9554e820d1531c2e08c0e4ef54", size = 16761, upload-time = "2026-05-21T22:40:41.845Z" },
]

[[package]]
name = "certifi"
version = "2026.6.17"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/c9/c7/424b75da314c1045981bd9777432fad05a9e0c69daa4ed7e308bbaff4e405/certifi-2026.6.17.tar.gz", hash = "sha256:024c88eeec92ca068db80f02b8b07c9cef7b9fe261d1d535abfd5abd6f6af432", size = 134594, upload-time = "2026-06-17T10:31:07.894Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/ef/2f/c5464532e965badff2f4c4c1a3a83f5697f0d7c407ed0cda44aaa99bb451/certifi-2026.6.17-py3-none-any.whl", hash = "sha256:2227dcbaafe0d2f59279d1762dddc37783ed4354594f194ffc31d20f41fc3db", size = 133289, upload-time = "2026-06-17T10:31:06.348Z" },
]

[[package]]
name = "cffi"
version = "2.0.0"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "pycparser", marker = "implementation_name != 'PyPy'" },
]
sdist = { url = "https://files.pythonhosted.org/packages/eb/56/b1ba7935a17738ae8453301356628e8147c79dbb825bcb373dc7401f9846/cffi-2.0.0.tar.gz", hash = "sha256:44d1b5909021139fe36001ae048dbdde8214afa20200eda0f64c068cac5d5529", size = 523588, upload-time = "2025-09-08T23:24:04.541Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/ea/47/4f61023ea636104d4f16ab488e268b93008c3d0bb76893b1b31db1f96802/cffi-2.0.0-cp312-cp312-macosx_10_13_x86_64.whl", hash = "sha256:6d02d6655b0e54f54c4ef0b94eb6be0607b70853c45ce98bd278dc7de718be5d", size = 185271, upload-time = "2025-09-08T23:22:44.795Z" },
 { url = "https://files.pythonhosted.org/packages/df/a2/781b623f57358e360d62cdd7a8c681f074a71d445418a776eef0aadb4ab4/cffi-2.0.0-cp312-cp312-macosx_11_0_arm64.whl", hash = "sha256:8eca2a813c1cb7ad4fb74d368c2ffbbb4789d377ee5bb8df983732c0dee76c", size = 181048, upload-time = "2025-09-08T23:22:45.938Z" },
 { url = "https://files.pythonhosted.org/packages/ff/df/a4f0fbd47331ceeba3d37c2e51e9dfc9722498becbeec2bd8bc856c9538a/cffi-2.0.0-cp312-cp312-manylinux1_i686.manylinux2014_i686.manylinux2_17_i686.manylinux2_5_i686.whl", hash = "sha256:21d1152871b019407d8ac3985f6775c079416c282e431a4da6afe7aefd2bccbe", size = 212529, upload-time = "2025-09-08T23:22:47.349Z" },
 { url = "https://files.pythonhosted.org/packages/d5/72/12b5f8d3865bf0f87cf1404d8c374e7487dcf097a1c91c436e72e6badd83/cffi-2.0.0-cp312-cp312-manylinux2014_aarch64.manylinux2_17_aarch64.whl", hash = "sha256:b21e08af67b8a103c71a250401c78d5e0893beff75e28c53c98f4de42f774062", size = 220097, upload-time = "2025-09-08T23:22:48.677Z" },
 { url = "https://files.pythonhosted.org/packages/c2/95/7a135d52a50dfa7c882ab0ac17e8dc11cec9d55d2c18dda414c051c5e69e/cffi-2.0.0-cp312-cp312-manylinux2014_ppc64le.manylinux2_17_ppc64le.whl", hash = "sha256:1e3a615586f05fc4065a8b22b8152f0c1b00cdbc60596d187c2a74f9e3036e4e", size = 207983, upload-time = "2025-09-08T23:22:50.06Z" },
 { url = "https://files.pythonhosted.org/packages/3a/c8/15cb9ada8895957ea171c62dc78ff3e99159ee7adb13c0123c001a2546c1/cffi-2.0.0-cp312-cp312-manylinux2014_s390x.manylinux2_17_s390x.whl", hash = "sha256:81afed14892743bbe14dacb9e36d9e0e504cd204e0b165062c488942b9718037", size = 206519, upload-time = "2025-09-08T23:22:51.364Z" },
 { url = "https://files.pythonhosted.org/packages/78/2d/7fa73dfa841b5ac06c7b8855cfc18622132e365f5b81d02230333ff26e9e/cffi-2.0.0-cp312-cp312-manylinux2014_x86_64.manylinux2_17_x86_64.whl", hash = "sha256:3e17ed538242334bf70832644a32a7a3e3d83b57567f9fd60a26257e992b79ba", size = 219572, upload-time = "2025-09-08T23:22:52.902Z" },
 { url = "https://files.pythonhosted.org/packages/07/e0/267e57e387b4ca276b90f0434ff88b2c2241ad72b16d31836addfd6031b/cffi-2.0.0-cp312-cp312-musllinux_1_2_aarch64.whl", hash = "sha256:3925dd22fa2b7699ed2617149842d2e6adde22b262fcbfada50e3d195e4b3a94", size = 222963, upload-time = "2025-09-08T23:22:54.518Z" },
 { url = "https://files.pythonhosted.org/packages/b6/75/1f2747525e06f53efbd878f4d03bac5b859cbc11c633d0fb81432d98a795/cffi-2.0.0-cp312-cp312-musllinux_1_2_x86_64.whl", hash = "sha256:2c8f814d84194c9ea681642fd164267891702542f028a15fc97d4674b6206187", size = 221361, upload-time = "2025-09-08T23:22:55.867Z" },
 { url = "https://files.pythonhosted.org/packages/7b/2b/2b6435f76bfeb6bbf055596976da087377ede68df465419d192acf00c437/cffi-2.0.0-cp312-cp312-win32.whl", hash = "sha256:da902562c3e9c550df360bfa53c035b2f241fed6d9aef119048073680ace4a18", size = 172932, upload-time = "2025-09-08T23:22:57.188Z" },
 { url = "https://files.pythonhosted.org/packages/f8/ed/13bd4418627013bec4ed6e54283b1959cf6db888048c7cf4b4c3b5b36002/cffi-2.0.0-cp312-cp312-win_amd64.whl", hash = "sha256:da6824880ad6320861f129cd9c1bf96ca849a2771a59e0344e88681905916f5", size = 183557, upload-time = "2025-09-08T23:23:00.879Z" },
 { url = "https://files.pythonhosted.org/packages/95/31/9f7f93ad2f8eff1dbc1c3656d7ca5bdf8fb52c9d786b4dcf19b2d02217fa/cffi-2.0.0-cp312-cp312-win_arm64.whl", hash = "sha256:4671d9dd5ec934cb9a73e7ee9676f9362aba54f7f34910956b84d727b0d73fb6", size = 177762, upload-time = "2025-09-08T23:22:59.668Z" },
 { url = "https://files.pythonhosted.org/packages/4b/8d/a0a47a0c9e413a658623d014e91e74a50cdd2c423f7ccfd44086ef767f90/cffi-2.0.0-cp313-cp313-macosx_10_13_x86_64.whl", hash = "sha256:00bdf7acc5f795150faa6957054fbbca2439db2f775ce831222b66f192f03beb", size = 185230, upload-time = "2025-09-08T23:23:00.879Z" },
 { url = "https://files.pythonhosted.org/packages/4a/d2/a6c0296814556c8ee32009d9c2ad4f85f2707cdecfd7727951ec228005d/cffi-2.0.0-cp313-cp313-macosx_11_0_arm64.whl", hash = "sha256:45d5e886156860dc35862657e1494b9bae8dfa63bf56796f2fb56e16

```

79fc0bca", size = 181043, upload-time = "2025-09-08T23:23:02.231Z" },  
 { url = "https://files.pythonhosted.org/packages/b0/1e/d22cc63332bd59b06481ceaac49d6c507598642e2230f201649058a7e704/cffi-2.0.0-cp313-cp313-manylinux1\_i686.manylinux2014\_i686.manylinux\_2\_17\_i686.manylinux\_2\_5\_i686.whl", hash = "sha256:07b271772c100085dd28b74fa0cd81c8fb1a3ba18b21e03d7c27f3436a10606b", size = 212446, upload-time = "2025-09-08T23:23:03.472Z" },  
 { url = "https://files.pythonhosted.org/packages/a9/f5/a2c23eb03b61a0b8747f211eb716446c826ad66818ddc7810cc2cc19b3f2/cffi-2.0.0-cp313-cp313-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.whl", hash = "sha256:d48a880098c96020b02d5a1f7d9251308510ce8858940e6fa99ece33f610838b", size = 220101, upload-time = "2025-09-08T23:23:04.792Z" },  
 { url = "https://files.pythonhosted.org/packages/f2/7f/e6647792f2c5850d634695bc0e6ab4111ae88e9981d35ac269956605feba/cffi-2.0.0-cp313-cp313-manylinux2014\_ppc64le.manylinux\_2\_17\_ppc64le.whl", hash = "sha256:f93fd8e5c80a4aa1f424d6173f14a892044054871c771f8566e4008eaa359d2", size = 207948, upload-time = "2025-09-08T23:23:06.127Z" },  
 { url = "https://files.pythonhosted.org/packages/cb/1e/a5a1bd6f1fb30f22573f76533de12a00bf274abdc55c8edab639078abb6/cffi-2.0.0-cp313-cp313-manylinux2014\_s390x.manylinux\_2\_17\_s390x.whl", hash = "sha256:dd4f05f54a52fb558f1ba9f528228066954fee3ebe629fc1660d874d040ae5a3", size = 206422, upload-time = "2025-09-08T23:23:07.753Z" },  
 { url = "https://files.pythonhosted.org/packages/98/df/0a1755e750013a2081e863e7cd37e0cdd02664372c754e5560099eb7aa44/cffi-2.0.0-cp313-cp313-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.whl", hash = "sha256:c8d3b5532fc71b7a77c09192b4a5a200ea992702734a2e9279a37f2478236f26", size = 219499, upload-time = "2025-09-08T23:23:09.648Z" },  
 { url = "https://files.pythonhosted.org/packages/50/e1/a969e687fcf9ea58e6e2a928ad5e2dd88cc126f0ab477e9971f2309b57c/cffi-2.0.0-cp313-cp313-musllinux\_1\_2\_aarch64.whl", hash = "sha256:d9b29c1f0ae438d5ee9acb31cadee00a58c46cc9c0b2f9038c6b0b3470877a8c", size = 222928, upload-time = "2025-09-08T23:23:10.928Z" },  
 { url = "https://files.pythonhosted.org/packages/36/54/0362578dd2c9e557a28ac77698ed67323ed5b9775ca9d3fe73fe191bb5d8/cffi-2.0.0-cp313-cp313-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:6d50360be4546678fc1b79ffe7a66265e28667840010348dd69a314145807a1b", size = 221302, upload-time = "2025-09-08T23:23:12.422Z" },  
 { url = "https://files.pythonhosted.org/packages/eb/6d/bf9bda840d5f1dfdbf0fec87fbbdb64a918a69bca42cfa0ba7b137c48cb8/cffi-2.0.0-cp313-cp313-win32.whl", hash = "sha256:74a03b9698e198d47562765773b4a8309919089150a0bb17d829ad7b44b60d27", size = 172909, upload-time = "2025-09-08T23:23:14.322Z" },  
 { url = "https://files.pythonhosted.org/packages/37/18/6519e1ee6f5a1e579e04b9ddb6f1676c17368a7aba48299c3759bbc3c8b3/cffi-2.0.0-cp313-cp313-win\_amd64.whl", hash = "sha256:19f705ada2530c1167abacb171925dd886168931e0a7b78f5bffcfae5c6b5be75", size = 183402, upload-time = "2025-09-08T23:23:15.535Z" },  
 { url = "https://files.pythonhosted.org/packages/cb/0e/02ceeec9a7d6ee63bb596121c2c8e9b3a9e150936f4fbef6ca1943e6137c/cffi-2.0.0-cp313-cp313-win\_arm64.whl", hash = "sha256:256f80b80ca3853f90c21b23ee78cd008713787b1b1e93eae9f3d6a7134abd91", size = 177780, upload-time = "2025-09-08T23:23:16.761Z" },  
 { url = "https://files.pythonhosted.org/packages/92/c4/3ce07396253a83250ee98564f8d7e9789fab8e58858f35d07a9a2c78de9f/cffi-2.0.0-cp314-cp314-macosx\_10\_13\_x86\_64.whl", hash = "sha256:fc33c5141b55ed366cfaad382df24fe7dcdb686de5be719b207bb248e3053dc5", size = 185320, upload-time = "2025-09-08T23:23:18.087Z" },  
 { url = "https://files.pythonhosted.org/packages/59/dd/27e9fa567a23931c838c6b02d0764611c62290062a6d4e8ff7863daf9730/cffi-2.0.0-cp314-cp314-macosx\_11\_0\_arm64.whl", hash = "sha256:c654de545946e0db659b3400168c9ad31b5d29593291482c43e3564effbcee13", size = 181487, upload-time = "2025-09-08T23:23:19.622Z" },  
 { url = "https://files.pythonhosted.org/packages/d6/43/0e822876f87ea8a4ef95442c3d766a06a51fc5298823f884ef87aaad168c/cffi-2.0.0-cp314-cp314-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.whl", hash = "sha256:24b6f81f1983e6df8db3adc38562c83f7d4a0c36162885ec7f7b77c7dcbec97b", size = 220049, upload-time = "2025-09-08T23:23:20.853Z" },  
 { url = "https://files.pythonhosted.org/packages/b4/89/76799151d9c2d2d1ead63c2429da9ea9d7aac304603de0c6e8764e6e8e70/cffi-2.0.0-cp314-cp314-manylinux2014\_ppc64le.manylinux\_2\_17\_ppc64le.whl", hash = "sha256:12873ca6cb9b0f0d3a0da705d6086fe911591737a59f28b7936bdfed27c0d47c", size = 207793, upload-time = "2025-09-08T23:23:22.08Z" },  
 { url = "https://files.pythonhosted.org/packages/bb/dd/3465b14bb9e24ee24cb88c9e3730f6de63111fffe513492bf8c808a3547e/cffi-2.0.0-cp314-cp314-manylinux2014\_s390x.manylinux\_2\_17\_s390x.whl", hash = "sha256:d9b97165e8aed9272a6bb17c01e3cc5871a594a446ebdc996e2397a1c1ea8ef", size = 206300, upload-time = "2025-09-08T23:23:23.314Z" },  
 { url = "https://files.pythonhosted.org/packages/47/d9/d83e293854571c877a92da46fdec39158f8d7e68da75bf73581225d28e90/cffi-2.0.0-cp314-cp314-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.whl", hash = "sha256:afb8db5439b81cf9c9d0c80404b60c3cc9c3add93e114dcae767f1477cb53775", size = 219244, upload-time = "2025-09-08T23:23:24.541Z" },  
 { url = "https://files.pythonhosted.org/packages/2b/0f/1f177e3683aeadd2bb00f7679a16451d302c436b5cbf2505f0ea8146ef59e/cffi-2.0.0-cp314-cp314-musllinux\_1\_2\_aarch64.whl", hash = "sha256:737fe7d37e1a1bffe70bd5754ea763a62a066dc5913ca57e957824b72a85e205", size = 222828, upload-time = "2025-09-08T23:23:26.143Z" },  
 { url = "https://files.pythonhosted.org/packages/c6/0f/cafacedb4b040e3119dcb32fed8bdef8dfe94da653155f9d0b9dc660166e/cffi-2.0.0-cp314-cp314-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:38100abb9d1b1435bc4cc340bb4489635dc2f0da7456590877030c9b3d40b0c1", size = 220926, upload-time = "2025-09-08T23:23:27.873Z" },  
 { url = "https://files.pythonhosted.org/packages/3e/aa/df335faa45b395396fcb03de2dfcab242cd61a9900e914fe682a59170b1/cffi-2.0.0-cp314-cp314-win32.whl", hash = "sha256:087067fa8953339c723661eda6b54bc98c5625757ea62e95eb4898ad5e776e9f", size = 175328, upload-time = "2025-09-08T23:23:44.61Z" },  
 { url = "https://files.pythonhosted.org/packages/bb/92/882c2d30831744296ce713f0feb4c1cd30f346ef747b530b5318715cc367/cffi-2.0.0-cp314-cp314-win\_amd64.whl", hash = "sha256:203a48d1fb583fc7d78a4c6655692963b860a417c0528492a6bc21f1aaefab25", size = 185650, upload-time = "2025-09-08T23:23:45.848Z" },  
 { url = "https://files.pythonhosted.org/packages/9f/2c/98ece204b9d35a7366b5b2c6539c350313ca13932143e79dc133ba757104/cffi-2.0.0-cp314-cp314-win\_arm64.whl", hash = "sha256:dbd5c7a25a7cb98f5ca55d258b103a2054f859a46ae11aaf23134f9cc0d356ad", size = 180687, upload-time = "2025-09-08T23:23:47.105Z" },  
 { url = "https://files.pythonhosted.org/packages/3e/61/c768e4d548bfa607abcda77423448df8c471f25dbe64fb2ef6d555eae006/cffi-2.0.0-cp314-cp314t-macosx\_10\_13\_x86\_64.whl", hash = "sha256:9a67fc9e8eb39039280526379b3a70023d77caec1852002b4da7e8b270c4dd9", size = 188773, upload-time = "2025-09-08T23:23:29.347Z" },  
 { url = "https://files.pythonhosted.org/packages/2c/ea/5f76bce7cf6fcd0ab1a1058b5af899bfef198bea4d5686da88471ea0336/cffi-2.0.0-cp314-cp314t-macosx\_11\_0\_arm64.whl", hash = "sha256:7a66c7204d8869299919db4d5069a82f1561581af12b11b3c9f48c584eb8743d", size = 185013, upload-time = "2025-09-08T23:23:30.63Z" },  
 { url = "https://files.pythonhosted.org/packages/be/b4/c56878d0d1755cf9caa54ba71e5d049479c52f9e4afc230f06822162ab2f/cffi-2.0.0-cp314-cp314t-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.whl", hash = "sha256:7cc09976e8b56f8cebd752f7113ad07752461f48a58cbb644139015ac24954c", size = 221593, upload-time = "2025-09-08T23:23:31.91Z" },  
 { url = "https://files.pythonhosted.org/packages/e0/0d/eb704606dfe8033e7128df5e90fee946bbcb64a04fcdad97321309004000/cffi-2.0.0-cp314-cp314t-manylinux2014\_ppc64le.manylinux\_2\_17\_ppc64le.whl", hash = "sha256:92b68146a71df78564e4ef48af17

```

551a5ddd142e5190cdf2c5624d0c3ff5b2e8", size = 209354, upload-time = "2025-09-08T23:23:33.214Z" },
 { url = "https://files.pythonhosted.org/packages/d8/19/3c435d727b368ca475fb8742ab97c9cb13a0de600ce86f62eab7fa3ee60
/cffi-2.0.0-cp314-cp314t-manylinux2014_s390x.manylinux_2_17_s390x.whl", hash = "sha256:b1e74d11748e7e98e2f426ab176d4ed7
20a64412b6a15054378afdb71e0f37dc", size = 208480, upload-time = "2025-09-08T23:23:34.495Z" },
 { url = "https://files.pythonhosted.org/packages/d0/44/681604464ed9541673e486521497406fadcc15b5217c3e326b061696899a
/cffi-2.0.0-cp314-cp314t-manylinux2014_x86_64.manylinux_2_17_x86_64.whl", hash = "sha256:28a3a209b96630bca57cce802da70c
266eb08c6e97e5afd61a75611ee6c64592", size = 221584, upload-time = "2025-09-08T23:23:36.096Z" },
 { url = "https://files.pythonhosted.org/packages/25/8e/342a504ff018a2825d395d44d63a767dd8ebc927ebda557fecdac3ac33a
/cffi-2.0.0-cp314-cp314t-musllinux_1_2_aarch64.whl", hash = "sha256:7553fb2090d71822f02c629afe6042c299edf91ba1bf9495116
5613553984512", size = 224443, upload-time = "2025-09-08T23:23:37.328Z" },
 { url = "https://files.pythonhosted.org/packages/e1/5e/b666bacbbc60fbf415ba9988324a132c9a7a0448a9a8f125074671c0f2c3
/cffi-2.0.0-cp314-cp314t-musllinux_1_2_x86_64.whl", hash = "sha256:6c6c373cfc5c83a975506110d17457138c8c63016b563cc9ed6e
056a82f13ce4", size = 223437, upload-time = "2025-09-08T23:23:38.945Z" },
 { url = "https://files.pythonhosted.org/packages/a0/1d/ec1a60bd1a10daa292d3cd6bb0b359a81607154fb8165f3ec95fe003b85c
/cffi-2.0.0-cp314-cp314t-win32.whl", hash = "sha256:1fc9ea04857caf665289b7a75923f2c6ed559b8298a1b8c49e59f7dd95c8481e",
size = 180487, upload-time = "2025-09-08T23:23:40.423Z" },
 { url = "https://files.pythonhosted.org/packages/bf/41/4c1168c74fac325c0c8156f04b6749c8b6a8f405bbf91413ba088359f60d
/cffi-2.0.0-cp314-cp314t-win_amd64.whl", hash = "sha256:d68b6cef7827e8641e8ef16f4494edda8b36104d79773a334beaa1e3521430f
6", size = 191726, upload-time = "2025-09-08T23:23:41.742Z" },
 { url = "https://files.pythonhosted.org/packages/ae/3a/dbeec9d1ee0844c679f6bb5d6ad4e9f198b1224f4e7a32825f47f6192b0c
/cffi-2.0.0-cp314-cp314t-win_arm64.whl", hash = "sha256:0a1527a803f0a659de1af2e1fd700213caba79377e27e4693648c2923da066f
9", size = 184195, upload-time = "2025-09-08T23:23:43.004Z" },
]

[[package]]
name = "charset-normalizer"
version = "3.4.7"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/e7/a1/67fe25fac3c7642725500a3f6cfe5821ad557c3abb11c9d20d12c700
8d3e/charset_normalizer-3.4.7.tar.gz", hash = "sha256:ae89db9e5f98a11a4bf50407d4363e7b09b31e55bc117b4f7d80aab97ba009e5"
, size = 144271, upload-time = "2026-04-02T09:28:39.342Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/0c/eb/4fc8d0a7110eb5fc9cc161723a34a8a6c200ce3b4fbf681bc86feee22308
/charset_normalizer-3.4.7-cp312-cp312-macosx_10_13_universal2.whl", hash = "sha256:eca9705049ad3c7345d574e3510665cb2cf8
44c2f2dcfe675332677f081cbd46", size = 311328, upload-time = "2026-04-02T09:26:24.331Z" },
 { url = "https://files.pythonhosted.org/packages/f8/e3/0fad706008ac9d7b9b5be6dc767c05f9d3e5df51744ce4cc9605de7b9f4
/charset_normalizer-3.4.7-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash =
"sha256:6178f72c5508bfc5fd446a5905e698c6212932f25bcdd4b47a757a50605a90e2", size = 208061, upload-time = "2026-04-02T09:
26:25.568Z" },
 { url = "https://files.pythonhosted.org/packages/42/f0/3dd1045c47f4a4604df85ec18ad093912ae1344ac706993aff91d38773a2
/charset_normalizer-3.4.7-cp312-cp312-manylinux2014_ppc64le.manylinux_2_17_ppc64le.manylinux_2_28_ppc64le.whl", hash =
"sha256:e1421b502d83040e6d7fb2fb18dff63957f720da3d77b2fbd3187ceb63755d7b", size = 229031, upload-time = "2026-04-02T09:
26:26.865Z" },
 { url = "https://files.pythonhosted.org/packages/dc/67/675a46eb016118a2fbde5a277a5d15f4f69d5f3f5f338e5ee2f8948fcf43
/charset_normalizer-3.4.7-cp312-cp312-manylinux2014_s390x.manylinux_2_17_s390x.manylinux_2_28_s390x.whl", hash = "sha25
6:edac0f1ab77644605be2cbbba52e6b7f630731fc42b34cb0f634be1a6eface56a", size = 225239, upload-time = "2026-04-02T09:26:28.
044Z" },
 { url = "https://files.pythonhosted.org/packages/4b/f8/d0118a2f5f23b02cd166fa385c60f9b0d4f9194f574e2b31cef350ad7223
/charset_normalizer-3.4.7-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl", hash = "sh
a256:5649fd1c7bade02f320a462fdefd0b4bd3ce036065836d4f42e0de958038e116", size = 216589, upload-time = "2026-04-02T09:26:
29.239Z" },
 { url = "https://files.pythonhosted.org/packages/b1/f1/6d2b0b261b6c4ceef0fcb0d17a01cc5bc53586c2d4796fa04b5c540bc13d
/charset_normalizer-3.4.7-cp312-cp312-manylinux_2_31_armv7l.whl", hash = "sha256:203104ed3e428044fd943bc4bf45fa73c07303
91f9621e37fe39ecf477b128cb", size = 202733, upload-time = "2026-04-02T09:26:30.5Z" },
 { url = "https://files.pythonhosted.org/packages/6f/c0/7b1f943f7e87cc3db9626ba17807d042c38645f0a1d4415c7a1aafb5591f
/charset_normalizer-3.4.7-cp312-cp312-manylinux_2_31_riscv64.manylinux_2_39_riscv64.whl", hash = "sha256:298930cec56029
e05497a76988377cbd7457ba864beee9a2ad7e844fe74cd1f1", size = 212652, upload-time = "2026-04-02T09:26:31.709Z" },
 { url = "https://files.pythonhosted.org/packages/38/dd/5a9ab159fe45c6e72079398f277b7d2b523e7f716acc489726115a910097
/charset_normalizer-3.4.7-cp312-cp312-musllinux_1_2_aarch64.whl", hash = "sha256:708838739abf24b2ceb208d0e22403dd018fae
ef86ddac04319a62ae884c4f15", size = 211229, upload-time = "2026-04-02T09:26:33.282Z" },
 { url = "https://files.pythonhosted.org/packages/d5/ff/531a1cad5ca855d1c1a8b69cb71abfd6d85c0291580146fda7c82857caa1
/charset_normalizer-3.4.7-cp312-cp312-musllinux_1_2_armv7l.whl", hash = "sha256:0f7eb884681e3938906ed0434f20c63046eacd0
111c4ba96f27b76084cd679f5", size = 203552, upload-time = "2026-04-02T09:26:34.845Z" },
 { url = "https://files.pythonhosted.org/packages/c1/4c/a5fb52d528a8ca41f7598cb619409ece30a169fbd9fcdce592e53b46c3a6
/charset_normalizer-3.4.7-cp312-cp312-musllinux_1_2_ppc64le.whl", hash = "sha256:4dc1e73c36828f982bfe79fadf5919923f8a6f
4df2860804db9a98c48824ce8d", size = 230806, upload-time = "2026-04-02T09:26:36.152Z" },
 { url = "https://files.pythonhosted.org/packages/59/7a/071feed8124111a32b316b33ae4de83d36923039ef8cf48120266844285b
/charset_normalizer-3.4.7-cp312-cp312-musllinux_1_2_riscv64.whl", hash = "sha256:aed52fea0513bac0ccde438c188c8a471c4e0f
457c2dd20cdbf6ea7a450046c7", size = 212316, upload-time = "2026-04-02T09:26:37.672Z" },
 { url = "https://files.pythonhosted.org/packages/fd/35/f7dba3994312d7ba508e041eaac39a36b120f32d4c8662b8814dab876431
/charset_normalizer-3.4.7-cp312-cp312-musllinux_1_2_s390x.whl", hash = "sha256:fea24543955a6a729c45a73fe90e08c743f0b333
4bbf3201e6c4bc1b0c7fa464", size = 227274, upload-time = "2026-04-02T09:26:38.93Z" },
 { url = "https://files.pythonhosted.org/packages/8a/2d/a572df5c9204ab7688ec1edc895a73ebded3b023bb07364710b05dd1c9be
/charset_normalizer-3.4.7-cp312-cp312-musllinux_1_2_x86_64.whl", hash = "sha256:bb6d88045545b26da47aa879dd4a89a71d1dce0
f0e549b1abcb31dfe4a8eac49", size = 218468, upload-time = "2026-04-02T09:26:40.17Z" },
 { url = "https://files.pythonhosted.org/packages/86/eb/890922a8b03a568ca2f336c36585a4713c55d4d67bf0f0c78924be6315ca

```

/charset\_normalizer-3.4.7-cp312-cp312-win32.whl", hash = "sha256:2257141f39fe5a3fdf38aeccae4b953e5f3b3324f4ff0daf9f15b8518666a2c", size = 148460, upload-time = "2026-04-02T09:26:41.416Z" },

{ url = "https://files.pythonhosted.org/packages/35/d9/0e7dffa06c5ab081f75b1b786f0aefc88365825dfcd0ac544bdb7b2b6853/charset\_normalizer-3.4.7-cp312-cp312-win\_amd64.whl", hash = "sha256:5ed6ab538499c8644b8a3e18debabacd7ce684f3fa91cf867521a7a0279cab2d6", size = 159330, upload-time = "2026-04-02T09:26:42.554Z" },

{ url = "https://files.pythonhosted.org/packages/9e/5d/481bcc2a7c88ea6b0878c299547843b2521ccbc40980cb406267088bc701/charset\_normalizer-3.4.7-cp312-cp312-win\_arm64.whl", hash = "sha256:56be790f86bfb2c98fb742ce566dfb4816e5a83384616ab59c49e0604d49c51d", size = 147828, upload-time = "2026-04-02T09:26:44.075Z" },

{ url = "https://files.pythonhosted.org/packages/c1/3b/66777e39d3ae1ddc77ee606be4ec6d8cbd4c801f65e5a1b6f2b11b8346dd/charset\_normalizer-3.4.7-cp313-cp313-macosx\_10\_13\_universal2.whl", hash = "sha256:f496c9c3cc02230093d8330875c4c3cdfc3b73612a5fd921c65d39cbcef08063", size = 309627, upload-time = "2026-04-02T09:26:45.198Z" },

{ url = "https://files.pythonhosted.org/packages/2e/4e/b7f84e617b4854ade48a1b7915c8ccfadeba444d2a18c291f696e37f0d3b/charset\_normalizer-3.4.7-cp313-cp313-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.manylinux\_2\_28\_aarch64.whl", hash = "sha256:0ea948db76d31190bf08bd371623927ee1339d5f2a0b4b1b4a4439a65298703c", size = 207008, upload-time = "2026-04-02T09:26:46.824Z" },

{ url = "https://files.pythonhosted.org/packages/c4/bb/ec73c0257c9e11b268f018f068f5d00aa0ef8c8b09f7753ebd5f2880e248/charset\_normalizer-3.4.7-cp313-cp313-manylinux2014\_ppc64le.manylinux\_2\_17\_ppc64le.manylinux\_2\_28\_ppc64le.whl", hash = "sha256:a277ab8928b9f299723bc1a2dabb1265911b1a76341f90a510368ca44ad9ab66", size = 228303, upload-time = "2026-04-02T09:26:48.397Z" },

{ url = "https://files.pythonhosted.org/packages/85/fb/32d1f5033484494619f701e719429c69b766bfc4dbc61aa9e9c8c166528b/charset\_normalizer-3.4.7-cp313-cp313-manylinux2014\_s390x.manylinux\_2\_17\_s390x.manylinux\_2\_28\_s390x.whl", hash = "sha256:3bec022aec2c514d9cf199522a802bd007cd588ab17ab252f20f9c34d067c18", size = 224282, upload-time = "2026-04-02T09:26:49.684Z" },

{ url = "https://files.pythonhosted.org/packages/fa/07/330e3a0dda4c404d6da83b327270906e9654a24f6c546dc886a0eb0fffb23/charset\_normalizer-3.4.7-cp313-cp313-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.manylinux\_2\_28\_x86\_64.whl", hash = "sha256:e044c39e41b92c845bc815e5ae4230804e8e7bc29e399b0437d6422d92809dd", size = 215595, upload-time = "2026-04-02T09:26:50.915Z" },

{ url = "https://files.pythonhosted.org/packages/e3/7c/fc890655786e423f02556e0216d4b8c6bcb6bdfa890160dc66bf52dee468/charset\_normalizer-3.4.7-cp313-cp313-manylinux\_2\_31\_armv7l.whl", hash = "sha256:f495a1652cf3fbab2eb0639776dad966c2fb874d79d87ca07f9d5f95b8bd215", size = 201986, upload-time = "2026-04-02T09:26:52.197Z" },

{ url = "https://files.pythonhosted.org/packages/d8/97/bfb18b3db2aed3b90cf54dc292ad79fdd5ad65c4eae454099475cbeadd0d/charset\_normalizer-3.4.7-cp313-cp313-manylinux\_2\_31\_riscv64.manylinux\_2\_39\_riscv64.whl", hash = "sha256:e712b419df8ba5e42b226c510472b37bd57b38e897d3eca5e8cfd410a29fa859", size = 211711, upload-time = "2026-04-02T09:26:53.49Z" },

{ url = "https://files.pythonhosted.org/packages/6f/a5/a581c13798546a7fd557c82614a5c65a13df2157e9ad6373166d2a3e645d/charset\_normalizer-3.4.7-cp313-cp313-musllinux\_1\_2\_aarch64.whl", hash = "sha256:7804338df6fcc08105c7745f1502ba68d900f45fd770d5bdd5288ddccb8a42d8", size = 210036, upload-time = "2026-04-02T09:26:54.975Z" },

{ url = "https://files.pythonhosted.org/packages/8c/bf/b3ab5bcb478ea4193d517644b0fb2bf5497fbceaa7a1bc0f4d5b50953861/charset\_normalizer-3.4.7-cp313-cp313-musllinux\_1\_2\_armv7l.whl", hash = "sha256:481551899c856c704d58119b5025793fa6730ad3a3571971af568f66d2424bb5", size = 202998, upload-time = "2026-04-02T09:26:56.303Z" },

{ url = "https://files.pythonhosted.org/packages/e7/4e/23efd79b65d314fa320ec6017b4b5834d5c12a58ba4610aa353af2e2f577/charset\_normalizer-3.4.7-cp313-cp313-musllinux\_1\_2\_ppc64le.whl", hash = "sha256:f59099f9b66f0d7145115e6f80dd8b1d847176df89b234a5a6b3f00437aa0832", size = 230056, upload-time = "2026-04-02T09:26:57.554Z" },

{ url = "https://files.pythonhosted.org/packages/b9/9f/1e1941bc3f0e01df116e68dc37a55c4d249df5e6fa77f008841aef68264f/charset\_normalizer-3.4.7-cp313-cp313-musllinux\_1\_2\_riscv64.whl", hash = "sha256:f59ad4c0e8f6bba240a9bb85504faa1ab438237199d4cce5f622761507b8f6a6", size = 211537, upload-time = "2026-04-02T09:26:58.843Z" },

{ url = "https://files.pythonhosted.org/packages/80/0f/088cbb3020d44428964a6c97fe1edfb1b9550396bf6d278330281e8b709c/charset\_normalizer-3.4.7-cp313-cp313-musllinux\_1\_2\_s390x.whl", hash = "sha256:3dedcc22d73ec993f42055eff4fcfed9318d1eeb9a6606c55892a26964964e48", size = 226176, upload-time = "2026-04-02T09:27:00.437Z" },

{ url = "https://files.pythonhosted.org/packages/6a/9f/130394f9bbe06f4f63e22641d32fc9b202b7e251c9aef4db044324dac493/charset\_normalizer-3.4.7-cp313-cp313-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:64f02c6841d7d83f832cd97ccf8eb8a906d06eb95d5276069175c696b024b60a", size = 217723, upload-time = "2026-04-02T09:27:02.021Z" },

{ url = "https://files.pythonhosted.org/packages/73/55/c469897448a06e49f8fa03f6caae97074fde823f432a98f979cc42b90e69/charset\_normalizer-3.4.7-cp313-cp313-win32.whl", hash = "sha256:4042d5c8f957e15221d423ba781e85d553722fc4113f523f2feb7b188cc34c5e", size = 148085, upload-time = "2026-04-02T09:27:03.192Z" },

{ url = "https://files.pythonhosted.org/packages/5d/78/1b74c5bbb3f99b77a1715c91b3e0b5bdb6fe302d95ace4f5b1bec37b0167/charset\_normalizer-3.4.7-cp313-cp313-win\_amd64.whl", hash = "sha256:3946fa46a0cf3e4c8cb1cc52f56bb536310d34f25f01ca9b6c16afa767dab110", size = 158819, upload-time = "2026-04-02T09:27:04.454Z" },

{ url = "https://files.pythonhosted.org/packages/68/86/46bd42279d323deb8687c4a5a811fd548cb7d1de10cf6535d099877a9a9f/charset\_normalizer-3.4.7-cp313-cp313-win\_arm64.whl", hash = "sha256:80d04837f55fc81da168b98de4f4b797ef07fc8a79ab71c6e9b9c4dd662b15b", size = 147915, upload-time = "2026-04-02T09:27:05.971Z" },

{ url = "https://files.pythonhosted.org/packages/97/c8/c67cb8c70e19ef1960b97b22ed2a1567711de46c4ddf19799923adc836c2/charset\_normalizer-3.4.7-cp314-cp314-macosx\_10\_15\_universal2.whl", hash = "sha256:c36c333c39be2dbca264d780333c896ab8fa7d4d6f0ab7edb7dfd7aea6e98c0", size = 309234, upload-time = "2026-04-02T09:27:07.194Z" },

{ url = "https://files.pythonhosted.org/packages/99/85/c091fdee33f20de70d6c8b522743b6f831a2f1cd3ff86de4c6a827c48a76/charset\_normalizer-3.4.7-cp314-cp314-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.manylinux\_2\_28\_aarch64.whl", hash = "sha256:1c2aed2e5e41f24ea8ef1590b8e848a79b56f3a5564a65ceec43c9d692dc7d8a", size = 208042, upload-time = "2026-04-02T09:27:08.749Z" },

{ url = "https://files.pythonhosted.org/packages/87/1c/ab2ce611b984d2fd5d86a5a8a19c1ae26acac6bad967da4967562c75114d/charset\_normalizer-3.4.7-cp314-cp314-manylinux2014\_ppc64le.manylinux\_2\_17\_ppc64le.manylinux\_2\_28\_ppc64le.whl", hash = "sha256:54523e136b894806c0fa0bc7b1b50c32c186f2fcee897a495406bb6e311d2b", size = 228706, upload-time = "2026-04-02T09:27:09.951Z" },

{ url = "https://files.pythonhosted.org/packages/a8/29/2b1d2cb00bf085f59d29eb773ce58ec2d325430f8c216804a0a5cd83cbca/charset\_normalizer-3.4.7-cp314-cp314-manylinux2014\_s390x.manylinux\_2\_17\_s390x.manylinux\_2\_28\_s390x.whl", hash = "sha256:715479b9a2802ecac752a3b0efa2b0b60285cf962ee38414211abdfccc233b41", size = 224727, upload-time = "2026-04-02T09:27:11.175Z" },

{ url = "https://files.pythonhosted.org/packages/47/5c/032c2d5a07fe4d4855fea851209cca2b6f03ebeb6d4e3afdb3358386a684

/charset\_normalizer-3.4.7-cp314-cp314-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.manylinux\_2\_28\_x86\_64.whl", hash = "sha256:bd6c2a1c7573c64738d716488d2cdd3c00e340e4835707d8fdb8dc1a66ef164e", size = 215882, upload-time = "2026-04-02T09:27:12.446Z" },

{ url = "https://files.pythonhosted.org/packages/2c/c2/356065d5a8b78ed04499cae5f339f091946a6a74f91e03476c33f0ab7100/charset\_normalizer-3.4.7-cp314-cp314-manylinux\_2\_31\_armv7l.whl", hash = "sha256:c45e9440fb78f8ddabc7f14b68f936737a121355bf59f3907f4e17721b9d1aae", size = 200860, upload-time = "2026-04-02T09:27:13.721Z" },

{ url = "https://files.pythonhosted.org/packages/0c/cd/a32a84217ced5039f53b29f460962abb2d4420def55afabe45b1c3c7483d/charset\_normalizer-3.4.7-cp314-cp314-manylinux\_2\_31\_riscv64.manylinux\_2\_39\_riscv64.whl", hash = "sha256:3534e7dcdbcf757da6b85a0bbf5b6868786d5982dd959b065e65481644817a18", size = 211564, upload-time = "2026-04-02T09:27:15.272Z" },

{ url = "https://files.pythonhosted.org/packages/44/86/58e6f13ce26cc3b8f4a36b94a0f22ae2f00a72534520f4ae6857c4b81f89/charset\_normalizer-3.4.7-cp314-cp314-musllinux\_1\_2\_aarch64.whl", hash = "sha256:e8ac484bf18ce6975760921bb6148041faa8fe0547200386ea0b525b2d7bf7b", size = 211276, upload-time = "2026-04-02T09:27:16.834Z" },

{ url = "https://files.pythonhosted.org/packages/8f/fe/d17c32cd72e17e155e06883efa84514ca375f8a528ba2546bee73fc4df81/charset\_normalizer-3.4.7-cp314-cp314-musllinux\_1\_2\_armv7l.whl", hash = "sha256:a5fe03b42827c13cdccd08e6c0247b6a6d4b5e3cdc53fd1749f5896adcdc2356", size = 201238, upload-time = "2026-04-02T09:27:18.229Z" },

{ url = "https://files.pythonhosted.org/packages/6a/29/f33daa50b06525a237451cdbc6c9da366c381a3dadcd833fa5676bc468b3/charset\_normalizer-3.4.7-cp314-cp314-musllinux\_1\_2\_ppc64le.whl", hash = "sha256:2d6eb928e13016cea4f1f21d1e10c1cebd5a421bc57ddfb5b1142ae3f86824fab", size = 230189, upload-time = "2026-04-02T09:27:19.445Z" },

{ url = "https://files.pythonhosted.org/packages/b6/6e/52c84015394a6a0bdcd435210a7e944c5f94ea1055f5cc5d56c5fe368e7b/charset\_normalizer-3.4.7-cp314-cp314-musllinux\_1\_2\_riscv64.whl", hash = "sha256:e74327fb75de8986940def6e8dee4f127cc9752bee7355bb323c5b2659b6d46", size = 211352, upload-time = "2026-04-02T09:27:20.79Z" },

{ url = "https://files.pythonhosted.org/packages/8c/d7/4353be581b37303fb9198bf1da3cf8f09c1082561e8e922aa7b39bf9fe8/charset\_normalizer-3.4.7-cp314-cp314-musllinux\_1\_2\_s390x.whl", hash = "sha256:d6038d37043bcd98a66e68d3aa2b6a35505dc01328cd65217cefe82f25def44", size = 227024, upload-time = "2026-04-02T09:27:22.063Z" },

{ url = "https://files.pythonhosted.org/packages/30/45/99d18aa925bd1740098ccd3060e238e21115fffbfdbcf8f3ece837d0ace6c/charset\_normalizer-3.4.7-cp314-cp314-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:7579e913a5339fb8fa133f6bbcfdb8e6749696206cf05acd0bca71ab1436d8e72", size = 217869, upload-time = "2026-04-02T09:27:23.486Z" },

{ url = "https://files.pythonhosted.org/packages/5c/05/5ee478aa53f4bb7996482153d4bfe1b89e0f087f0ab6b294fcf92d595873/charset\_normalizer-3.4.7-cp314-cp314-win32.whl", hash = "sha256:5b77459df20e08151cd6f8b9ef8ef1f961ef73d85c21a555c7eed5b79410ec10", size = 148541, upload-time = "2026-04-02T09:27:25.146Z" },

{ url = "https://files.pythonhosted.org/packages/48/77/72dcb0921b2ce86420b2d79d454c7022bf5be40202a2a07906b9f2a35c97/charset\_normalizer-3.4.7-cp314-cp314-win\_amd64.whl", hash = "sha256:92a0a01ead5e668468e952e4238cccd7c537364eb7d851ab144ab6627dbbe12f", size = 159634, upload-time = "2026-04-02T09:27:26.642Z" },

{ url = "https://files.pythonhosted.org/packages/c6/a3/c2369911cd72f02386e4e340770f6e158c7980267da16af8f668217abaa0/charset\_normalizer-3.4.7-cp314-cp314-win\_arm64.whl", hash = "sha256:67f6279d125ca0046a7fd386d01b311c6363844deac3e5b069b514ba3e63c246", size = 148384, upload-time = "2026-04-02T09:27:28.271Z" },

{ url = "https://files.pythonhosted.org/packages/94/09/7e8a7f73d24dba1f0035fbbf014d2c36828fc1bf9c88f84093e57d315935/charset\_normalizer-3.4.7-cp314t-cp314t-macosx\_10\_15\_universal2.whl", hash = "sha256:effcf3f449787117233702311a1b7d8f59cb a9ced946ba727bdc329ec69028e24", size = 330133, upload-time = "2026-04-02T09:27:29.474Z" },

{ url = "https://files.pythonhosted.org/packages/8d/da/96975ddb11f8e977f706f45cddd8540fd8242f71ecdb5d18a80723dcf62c/charset\_normalizer-3.4.7-cp314-cp314t-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.manylinux\_2\_28\_aarch64.whl", hash = "sha256:fbccdc05410c9ee21bbf16a35f4c1d16123dcdeb8a1d38f33654fa21d0234f79", size = 216257, upload-time = "2026-04-02T09:27:30.793Z" },

{ url = "https://files.pythonhosted.org/packages/e5/e8/1d63bf8ef2d388e95c64b2098f45f84758f6d102a087552da1485912637b/charset\_normalizer-3.4.7-cp314-cp314t-manylinux2014\_ppc64le.manylinux\_2\_17\_ppc64le.manylinux\_2\_28\_ppc64le.whl", hash = "sha256:733784b6d6def852c814bce5f318d25da2ee65dd4839a0718641c696e09a2960", size = 234851, upload-time = "2026-04-02T09:27:32.44Z" },

{ url = "https://files.pythonhosted.org/packages/9b/40/e5ff04233e70da2681fa43969ad6f66ca5611d7e669be0246c4c7aaf6dc8/charset\_normalizer-3.4.7-cp314-cp314t-manylinux2014\_s390x.manylinux\_2\_17\_s390x.manylinux\_2\_28\_s390x.whl", hash = "sha256:a89c23ef8d2c6b27fd200a42aa4ac72786e7c60d40efdc76e6011260b6e949c4", size = 233393, upload-time = "2026-04-02T09:27:34.03Z" },

{ url = "https://files.pythonhosted.org/packages/be/c1/06c6c49d5a5450f76899992f1ee40b41d076aee9279b49cf9974d2f313d5/charset\_normalizer-3.4.7-cp314-cp314t-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.manylinux\_2\_28\_x86\_64.whl", hash = "sha256:6c114670c45346afedc0d947faf3c7f701051d2518b943679c8ff88befe14f8e", size = 223251, upload-time = "2026-04-02T09:27:35.369Z" },

{ url = "https://files.pythonhosted.org/packages/2b/9f/f2ff16fb050946169e3e1f82134d107e5d4ae72647ec8a1b1446c148480f/charset\_normalizer-3.4.7-cp314-cp314t-manylinux\_2\_31\_armv7l.whl", hash = "sha256:a180c5e59792af262bf263b21a3c49353f25945d8d9f70628e73de370d55e1e1", size = 206609, upload-time = "2026-04-02T09:27:36.661Z" },

{ url = "https://files.pythonhosted.org/packages/69/d5/a527c0cd8d64d2eab7459784fb4169a0ac76e5a6fc5237337982fd61347e/charset\_normalizer-3.4.7-cp314-cp314t-manylinux\_2\_31\_riscv64.manylinux\_2\_39\_riscv64.whl", hash = "sha256:3c9a494bc5ec77d43cea229c4f6db1e4d8fe7e1bbffa8b6f0f0032430ff8ab44", size = 220014, upload-time = "2026-04-02T09:27:38.019Z" },

{ url = "https://files.pythonhosted.org/packages/7e/80/8a7b8104a3e203074dc9aa2c613d4b726c0e136bad1cc734594b02867972/charset\_normalizer-3.4.7-cp314-cp314t-musllinux\_1\_2\_aarch64.whl", hash = "sha256:8d828b6667a32a728a1ad1d93957cdf37489c57b97ae6c4de2860fa749b8fc1e", size = 218979, upload-time = "2026-04-02T09:27:39.37Z" },

{ url = "https://files.pythonhosted.org/packages/02/9a/b759b503d507f375b2b5c153e4d2ee0a75aa215b7f2489cf314f4541f2c0/charset\_normalizer-3.4.7-cp314-cp314t-musllinux\_1\_2\_armv7l.whl", hash = "sha256:cf1493cd8607bec4d8a7b9b004e699fcf8f9103a9284cc94962cb73d20f9d4a3", size = 209238, upload-time = "2026-04-02T09:27:40.722Z" },

{ url = "https://files.pythonhosted.org/packages/3c/4e/0f3f5d47b86bdb79256e7290b26ac847a2832d9a4033f7eb2cd4bcf4bb5b/charset\_normalizer-3.4.7-cp314-cp314t-musllinux\_1\_2\_ppc64le.whl", hash = "sha256:0c96c3b819b5c3e9e165495db84d41914d6894d55181d2d108cc1a69bfc9cce0", size = 236110, upload-time = "2026-04-02T09:27:42.33Z" },

{ url = "https://files.pythonhosted.org/packages/96/23/bce28734eb3ed2c91dcf93abeb8a5cf393a7b2749725030bb630e554fdd8/charset\_normalizer-3.4.7-cp314-cp314t-musllinux\_1\_2\_riscv64.whl", hash = "sha256:752a45dc4a6934060b3b0dab47e04edc3326575f82be64bc4fc293914566503e", size = 219824, upload-time = "2026-04-02T09:27:43.924Z" },

{ url = "https://files.pythonhosted.org/packages/2c/6f/6e897c6984cc4d41af319b077f2f600fc8214eb2fe2d6bcb79141b882400/charset\_normalizer-3.4.7-cp314-cp314t-musllinux\_1\_2\_s390x.whl", hash = "sha256:8778f0c7a52e56f75d12dae53ae320fae900a8b9b4164b981b9c5ce059cd1fcb", size = 233103, upload-time = "2026-04-02T09:27:45.348Z" },



```

 { url = "https://files.pythonhosted.org/packages/76/22/ef7bd0fe480a0ae9b656189ec00744b60933f68b4f42a7bb06589f6f576a
/charset_normalizer-3.4.7-cp314-cp314t-muslinux_1_2_x86_64.whl", hash = "sha256:ce3412f8e1e31eb81ea42f4169ed94861c56e6
43189e1e75f0041f3fe7020abe", size = 225194, upload-time = "2026-04-02T09:27:46.706Z" },
 { url = "https://files.pythonhosted.org/packages/c5/a7/0e0ab3e0b5bc1219bd80a6a0d4d72ca74d9250cb2382b7c699c147e06017
/charset_normalizer-3.4.7-cp314-cp314t-win32.whl", hash = "sha256:c03a41a8784091e67a39648f70c5f97b5b6a37f216896d44d2cdc
b82615339a0", size = 159827, upload-time = "2026-04-02T09:27:48.053Z" },
 { url = "https://files.pythonhosted.org/packages/7a/1d/29d32e0fb40864b1f878c7f5a0b343ae676c6e2b271a2d55cc3a152391da
/charset_normalizer-3.4.7-cp314-cp314t-win_amd64.whl", hash = "sha256:03853ed82eeebbce3c2abfdbc98c96dc205f32a79627688ac
9a27370ea61a49c", size = 174168, upload-time = "2026-04-02T09:27:49.795Z" },
 { url = "https://files.pythonhosted.org/packages/de/32/d92444ad05c7a6e41fb2036749777c163baf7a0301a040cb672d6b2b1ae9
/charset_normalizer-3.4.7-cp314-cp314t-win_arm64.whl", hash = "sha256:c35abb8bfff0185efac5878da64c45dafd2b37fb0383add1b
e155a763c1f083d", size = 153018, upload-time = "2026-04-02T09:27:51.116Z" },
 { url = "https://files.pythonhosted.org/packages/db/8f/61959034484a4a7c527811f4721e75d02d653a35afb0b6054474d8185d4c
/charset_normalizer-3.4.7-py3-none-any.whl", hash = "sha256:3dce51d0f5e7951f8bb4900c257dad282f49190fdbebecdd4ba99bcc41fe
f404d", size = 61958, upload-time = "2026-04-02T09:28:37.794Z" },
]

[[package]]
name = "click"
version = "8.4.1"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "colorama", marker = "sys_platform == 'win32'" },
]
sdist = { url = "https://files.pythonhosted.org/packages/9b/98/518d8e5081007684232226f475082b30087d0f585e8457db08729825
9f49/click-8.4.1.tar.gz", hash = "sha256:918b5633eddf6b41c32d4f454bf0de810065c74e3f7dbf8ee5452f8be88d3e96", size = 3530
07, upload-time = "2026-05-22T04:08:37.769Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/c7/0d/67e5b4109ea4a837e80daa87c2c696711955e40449a97e8926672534def2
/click-8.4.1-py3-none-any.whl", hash = "sha256:482be17c6991b8c19c5429a1e995d9b0efdbb63172824c41f99965dc0ade8ec2", size
= 116639, upload-time = "2026-05-22T04:08:35.26Z" },
]

[[package]]
name = "colorama"
version = "0.4.6"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/d8/53/6f443c9a4a8358a93a6792e2acffbd9d5cb0a5cfd8802644b7b1c9a
02e4/colorama-0.4.6.tar.gz", hash = "sha256:08695f5cb7ed6e0531a20572697297273c47b8cae5a63ffcd6d6ed5c201be6e44", size = 2
7697, upload-time = "2022-10-25T02:36:22.414Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/d1/d6/3965ed04c63042e047cb6a3e6ed1a63a35087b6a609aa3a15ed8ac56c221
/colorama-0.4.6-py2.py3-none-any.whl", hash = "sha256:4f1d9991f5acc0ca119f9d443620b77f9d6b33703e51011c16baf57afb285fc6"
, size = 25335, upload-time = "2022-10-25T02:36:20.889Z" },
]

[[package]]
name = "cryptography"
version = "49.0.0"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "cffi", marker = "platform_python_implementation != 'PyPy'" },
]
sdist = { url = "https://files.pythonhosted.org/packages/1f/99/d1c90d6041656cc6ee229dc99cd67fd0cd5aec3c5f7d72fffc27cc75
0054/cryptography-49.0.0.tar.gz", hash = "sha256:f89660a348f4f78a92366240a61404e337586ef7f5909a2fef59ca8ef505493", siz
e = 854345, upload-time = "2026-06-12T20:02:30.512Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/9b/22/adf66990e63584a68dfb50c24f48a125c07b1699899381c8151e63ed458c
/cryptography-49.0.0-cp311-abi3-macosx_11_0_arm64.whl", hash = "sha256:966fe0e9c67490071f14c0d2b1cb2dfb3023c5ce39457343
931415f08382f2db", size = 4032100, upload-time = "2026-06-12T20:02:32.143Z" },
 { url = "https://files.pythonhosted.org/packages/09/41/3797cfaf69cae04a13ee78ebd83f0678d9c02b4779d21ce24445326f1a69
/cryptography-49.0.0-cp311-abi3-manylinux2014_aarch64.whl", hash = "sha256:36d1709f992593689b45b
da411498d62c6e365f2ca00b84657d4dadd24de16db", size = 4692978, upload-time = "2026-06-12T20:01:21.305Z" },
 { url = "https://files.pythonhosted.org/packages/e6/8b/43011f7ebe515a8aa20d61f290a326cd890c2e738e16e59eaff8d9c3a412
/cryptography-49.0.0-cp311-abi3-manylinux2014_x86_64.whl", hash = "sha256:0e959b578856a3924bc0cbb
710fc12c387b9412a951389f3ca61704a9e25f325", size = 4716422, upload-time = "2026-06-12T20:01:48.566Z" },
 { url = "https://files.pythonhosted.org/packages/4a/91/01ce7303a4579e6d3a6abef01bd322848e9ea7a219adcabc5048b9033571
/cryptography-49.0.0-cp311-abi3-manylinux_2_28_aarch64.whl", hash = "sha256:53ecce2e23f7169b6117e99fc8a944e5e50f79e6975
8a83b52a00cb98ab2b2d2", size = 4700503, upload-time = "2026-06-12T20:02:47.091Z" },
 { url = "https://files.pythonhosted.org/packages/62/99/a2c95cf8293f07491e9e27c20cc4dcd18176d944e674679adeb1d0173fd6
/cryptography-49.0.0-cp311-abi3-manylinux_2_28_ppc64le.whl", hash = "sha256:2eda353d8a27bcbcaa4cbcd18994a74ab4d19a2ca89
7db188ea269ab9b71419b", size = 5309779, upload-time = "2026-06-12T20:02:08.987Z" },
 { url = "https://files.pythonhosted.org/packages/20/2c/0622f20ff02b2ef32558733443805dc82fd4c275be01b2d19d14676f3a1b
/cryptography-49.0.0-cp311-abi3-manylinux_2_28_x86_64.whl", hash = "sha256:2afe9051da7ae7bd5905da5a949280c7d2bb75682e18
8f650a9d0f2756b834c6", size = 4749683, upload-time = "2026-06-12T20:02:03.335Z" },
 { url = "https://files.pythonhosted.org/packages/a3/5b/c5246635d5fd3b64e0d45ae10e99fd32fe9676a79915ccfe5a61ba9af1a5

```

/cryptography-49.0.0-cp311-abi3-manylinux\_2\_31\_armv7l.whl", hash = "sha256:0b82e28ee398a386f0807bba7884d30f25218855690f45115831bcc5d90822c", size = 4337874, upload-time = "2026-06-12T20:02:54.323Z" },  
 { url = "https://files.pythonhosted.org/packages/6d/88/05563c7fe2e914e87d1a536d06fe83e66b4e1d95cb593e05aea375531da8/cryptography-49.0.0-cp311-abi3-manylinux\_2\_34\_aarch64.whl", hash = "sha256:ccac2bfeb3c306b862133e3bb71f3f6ee8bb525240089b2d952e4144b3a6d5da7", size = 4700283, upload-time = "2026-06-12T20:01:34.822Z" },  
 { url = "https://files.pythonhosted.org/packages/c4/b6/d7696e4e890d6ae1469935164c9e5215c557671cb78d6e3f458ccceaa632/cryptography-49.0.0-cp311-abi3-manylinux\_2\_34\_ppc64le.whl", hash = "sha256:d0527ce944105f257f605a827d6bead966c752038b6e8656abb9c5edee6fc68", size = 5265844, upload-time = "2026-06-12T20:01:24.09Z" },  
 { url = "https://files.pythonhosted.org/packages/a9/3c/f3ad17eecc1a57b0ba236dc01cf90e783c51f4a2f35f64777cc4f47a184b2/cryptography-49.0.0-cp311-abi3-manylinux\_2\_34\_x86\_64.whl", hash = "sha256:cbc77da8c523d5abd028635ba850a6966fccee2c82e2b65a41d1d8afe0f98be9", size = 4749290, upload-time = "2026-06-12T20:01:30.848Z" },  
 { url = "https://files.pythonhosted.org/packages/4f/01/339573cf1023163a400b0b5d16f6d507de413b9f60be6fd1b77feeaf6737/cryptography-49.0.0-cp311-abi3-musllinux\_1\_2\_aarch64.whl", hash = "sha256:b87e65d263b3e5d3bb92a57e2a6638e2f31110fa7aa890c7b2dbba42248d0a3f", size = 4834612, upload-time = "2026-06-12T20:01:29.246Z" },  
 { url = "https://files.pythonhosted.org/packages/71/fd/577302e213a1be9468f92d1afef66fc1ef83d516819d9992ca547f592bd/cryptography-49.0.0-cp311-abi3-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:66ec79c3904820572d7e987abdf304281f141d37ad9a489b8e97066e7b9b6459", size = 4980804, upload-time = "2026-06-12T20:01:42.853Z" },  
 { url = "https://files.pythonhosted.org/packages/1f/09/f42b1d190c5ba75f72062a387f8030d1d75f6ab035788f1d9c4b01de6525/cryptography-49.0.0-cp311-abi3-win\_amd64.whl", hash = "sha256:e5dfc1e64de5677cec922ffa8da89c546d0415bf6efdf081842e5d44c84e1f0e", size = 3810026, upload-time = "2026-06-12T20:02:39.262Z" },  
 { url = "https://files.pythonhosted.org/packages/ec/9e/db72b3ae7fc9cfad53e630e56c6ae83b9b6ff0bf3718ff8012d20b3aabf/cryptography-49.0.0-cp314-cp314t-macosx\_11\_0\_arm64.whl", hash = "sha256:73a205dce83953d131a4aa1e0fd917a2fd1c5b1eef251e9d7152efefcbf5caf7", size = 4013892, upload-time = "2026-06-12T20:02:10.735Z" },  
 { url = "https://files.pythonhosted.org/packages/86/12/c48a424f38db03027be9f7ed5c7dc5de9933dbee992865f98b13727a009d/cryptography-49.0.0-cp314-cp314t-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.whl", hash = "sha256:196ecd6a36e4e9aa10270393bb98d8df88fccee0bf1e5128b91ae4eb4375896d", size = 4678835, upload-time = "2026-06-12T20:02:48.743Z" },  
 { url = "https://files.pythonhosted.org/packages/68/78/8a3ad4653662c93fc44dc4e5d8fd374c25c42e07b34bbfbaddf49cf57a5a8/cryptography-49.0.0-cp314-cp314t-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.whl", hash = "sha256:7abcee80084cda3f7691f3eb1ce480d8df49cec637b429aa35986c1de71738aa", size = 4697239, upload-time = "2026-06-12T20:02:56.03Z" },  
 { url = "https://files.pythonhosted.org/packages/a8/b2/2193fc74f81aee4f9b6273313b73b5176718932ed8f2e4b03fa040480a6/cryptography-49.0.0-cp314-cp314t-manylinux\_2\_28\_aarch64.whl", hash = "sha256:4ae387c9cb68ea569ca17e490d66d8142b81c3cc814bf179974b7d146e490bbb", size = 4685593, upload-time = "2026-06-12T20:02:50.666Z" },  
 { url = "https://files.pythonhosted.org/packages/47/f1/1d3eaa243bfc5de4a187b22aa8c048b3e4980bfb830ac46e6bac2e66947/cryptography-49.0.0-cp314-cp314t-manylinux\_2\_28\_ppc64le.whl", hash = "sha256:f37d847238971164fdb68ade6f6574aecc9c0af714190e2083429ff68f4ce9d", size = 5289961, upload-time = "2026-06-12T20:01:46.468Z" },  
 { url = "https://files.pythonhosted.org/packages/58/39/2d51306721330c486495853eda1c567880ff036de15a14c4b74f399934af/cryptography-49.0.0-cp314-cp314t-manylinux\_2\_28\_x86\_64.whl", hash = "sha256:c2bc30226390d60ea19d9f82b19db005fe0452154a23c1c410c12ea801e43561", size = 4731145, upload-time = "2026-06-12T20:02:16.832Z" },  
 { url = "https://files.pythonhosted.org/packages/17/50/983e838c7fd0d87fd8c969bcd328edaf5f756e38df5281637424c155873/cryptography-49.0.0-cp314-cp314t-manylinux\_2\_31\_armv7l.whl", hash = "sha256:07cab27cc7b7e0fd28e5e26bb9eeedde5c135c868b46de4a27845abe94af6122", size = 4321719, upload-time = "2026-06-12T20:02:52.611Z" },  
 { url = "https://files.pythonhosted.org/packages/a7/f5/8f571d7e27c55bce9f76f026143bcb1e040a4233149ecca0bea5fa5dd5f7/cryptography-49.0.0-cp314-cp314t-manylinux\_2\_34\_aarch64.whl", hash = "sha256:b20133d204d2bb56ba047642199603876c872026ca53e79c35b83772ab2cc505", size = 4685209, upload-time = "2026-06-12T20:02:07.282Z" },  
 { url = "https://files.pythonhosted.org/packages/e7/84/0e27016a6fc5a0886f797018b26aa42f40c09a82332bfff77822a451deaaa/cryptography-49.0.0-cp314-cp314t-manylinux\_2\_34\_ppc64le.whl", hash = "sha256:b970c6da94d5bb18629db453d14f2a1300f6bf59b61e9b82377931ef95504866", size = 5246285, upload-time = "2026-06-12T20:01:32.439Z" },  
 { url = "https://files.pythonhosted.org/packages/11/2d/5e1fb307cb5931881516b464c98774b3f2c36b5d4bb9a2830253cf553cad/cryptography-49.0.0-cp314-cp314t-manylinux\_2\_34\_x86\_64.whl", hash = "sha256:d8ecde755e2e91bf773fc94e8c9d730cd7f2007004cb492263a794ec3899a1c8", size = 4730441, upload-time = "2026-06-12T20:02:01.469Z" },  
 { url = "https://files.pythonhosted.org/packages/e4/c0/bff5a02ee731d207d6a1ed51732549d8c53d2bc8da1d10ec6f2844201d68/cryptography-49.0.0-cp314-cp314t-musllinux\_1\_2\_aarch64.whl", hash = "sha256:e3fb64c420688e5319ae25113a354015abbd8dffbfbc41781a1ea66fc7622ac3", size = 4815869, upload-time = "2026-06-12T20:01:36.574Z" },  
 { url = "https://files.pythonhosted.org/packages/b9/26/814681d14248d95d73d5c3eea0c39a94eb8302df966f670a2c60de90974b/cryptography-49.0.0-cp314-cp314t-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:32703d93296f5c1f4b53349ad3a250c2cae0fdec3a3dd5d47e616d8d616af27", size = 4960948, upload-time = "2026-06-12T20:02:18.688Z" },  
 { url = "https://files.pythonhosted.org/packages/4c/fe/93ecac273d3738939d023612ad12cca9a3740a5345d69fda04134c43fd96/cryptography-49.0.0-cp314-cp314t-win\_amd64.whl", hash = "sha256:33cd0565932807baddb67b96dbee92f2c374b5c89dee09fd74079aeb8c8dba61", size = 3799153, upload-time = "2026-06-12T20:01:39.059Z" },  
 { url = "https://files.pythonhosted.org/packages/19/2a/5bb823f5bedcf80718cea7fbc95ec5515cca3769633c4b01a32be7f30e7c/cryptography-49.0.0-cp39-abi3-macosx\_11\_0\_arm64.whl", hash = "sha256:ec5e529fb80935c94fe7b729f9972b50e351a0e6b50aa294fd5cabb109fcc29a", size = 4025947, upload-time = "2026-06-12T20:01:25.745Z" },  
 { url = "https://files.pythonhosted.org/packages/3d/df/40577043ca124e17012f408dddaeb213b856336ac82ddb3bc915f39e29f/cryptography-49.0.0-cp39-abi3-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.whl", hash = "sha256:f78ff2c9ed8dc2d036b0f4d640e22522213d047c1b14e61205a7e55c80a494d4", size = 4692429, upload-time = "2026-06-12T20:01:53.628Z" },  
 { url = "https://files.pythonhosted.org/packages/2c/99/2d13299eb3dd27b02dcfaafcc91d6b5cb3329f7cbdd6d8f51921acd566c1a/cryptography-49.0.0-cp39-abi3-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.whl", hash = "sha256:35b151772baff2c74cba7fa290ceaff4c3b11c0c881eb93eb5dbcb05a7cfbba18", size = 4700968, upload-time = "2026-06-12T20:02:45.383Z" },  
 { url = "https://files.pythonhosted.org/packages/a5/4d/9c0cd02f95e2602dd5e563da149ee0830abef3537b8b34dc56281ebe27a/cryptography-49.0.0-cp39-abi3-manylinux\_2\_28\_aarch64.whl", hash = "sha256:0f21641cf4b30fca7aee061ced0ec7ad7b073518088b7c9969a297c0ae796c69", size = 4697758, upload-time = "2026-06-12T20:01:41.13Z" },  
 { url = "https://files.pythonhosted.org/packages/24/01/186c825898477d77e2324d5360fefe622ff1d8d1963ec0554e2cada8ec77/cryptography-49.0.0-cp39-abi3-manylinux\_2\_28\_ppc64le.whl", hash = "sha256:9e82dcc8e56052715fb18b2429e3bca4823b1629136a2084fc45a9a5cecb9b64", size = 5298863, upload-time = "2026-06-12T20:02:25.579Z" },  
 { url = "https://files.pythonhosted.org/packages/b8/7b/62cbbab75d0659865bf0273790031544a0b16c8072d258f9428dc8d190dc/cryptography-49.0.0-cp39-abi3-manylinux\_2\_28\_x86\_64.whl", hash = "sha256:6f2debedf9ca60cf1d5bd466475638af5130f89965605

```

cd818484d19987d3a21", size = 4735983, upload-time = "2026-06-12T20:01:50.14Z" },
 { url = "https://files.pythonhosted.org/packages/6c/72/3e798c064bc39e471008075d0f9bc9daf77a80879c092e4a8e170c585ed4/cryptography-49.0.0-cp39-abi3-manylinux_2_31_armv7l.whl", hash = "sha256:8c25ceb16df5b9435f3f6a9829204985b0e0cbee3b48aacd432c7d2c850b44d9", size = 4334173, upload-time = "2026-06-12T20:01:44.743Z" },
 { url = "https://files.pythonhosted.org/packages/f0/ee/6fca21d1ac73e06f8bef71940abfd4d2f6472b4bca284d770f32bd4086f6/cryptography-49.0.0-cp39-abi3-manylinux_2_34_aarch64.whl", hash = "sha256:28d8b15e6275f12c8a207dc309dfa957903c927d08d0cc937ee3f63f200693cc", size = 4697298, upload-time = "2026-06-12T20:02:20.918Z" },
 { url = "https://files.pythonhosted.org/packages/67/d0/a5fcd3515f0bae49a7b6d0413cc1bdccdcc1fc0047037a0d480642cdc5d6/cryptography-49.0.0-cp39-abi3-manylinux_2_34_ppc64le.whl", hash = "sha256:6fc361c34fb6aac015ce19435876635e5c6d21db31998b0920f675f131e043b8", size = 5254338, upload-time = "2026-06-12T20:02:22.737Z" },
 { url = "https://files.pythonhosted.org/packages/a0/84/84fe36f19caf857d61cb7fc9c63035a47ffabd84ea12d1d393148efa3615/cryptography-49.0.0-cp39-abi3-manylinux_2_34_x86_64.whl", hash = "sha256:2400ef9c9e2299a25614eb1dea3db54a69b1349efd043bfac9c67630d136df36", size = 4735650, upload-time = "2026-06-12T20:02:41.389Z" },
 { url = "https://files.pythonhosted.org/packages/6c/a0/db537264e234f7273a73ec020873d6d6b39df8a53db78b550ca8320440e/cryptography-49.0.0-cp39-abi3-musllinux_1_2_aarch64.whl", hash = "sha256:67e1d20ad9ef3a563c59ef22e7a8a0b8210bd26604369ea4a30a7c66aefe504e", size = 4834820, upload-time = "2026-06-12T20:01:51.847Z" },
 { url = "https://files.pythonhosted.org/packages/93/77/8df9eb486495979bccecd1062e2eaf435250e84437040295b57d09048b0b/cryptography-49.0.0-cp39-abi3-musllinux_1_2_x86_64.whl", hash = "sha256:42b0684e0e40cf26122427802486f6d93aea593612603a94fbf260c7eb1e9c1b", size = 4967968, upload-time = "2026-06-12T20:02:12.524Z" },
 { url = "https://files.pythonhosted.org/packages/c2/e6/f60198ea8d9dfa15fff9ed4ca02ce362f6eadd9ba757dcc50634c4257b63/cryptography-49.0.0-cp39-abi3-win_amd64.whl", hash = "sha256:026ac7423e6fa66872d3bf889be5974507da3944f866f704fa200eada cd00001", size = 3785547, upload-time = "2026-06-12T20:02:26.847Z" },
]

[[package]]
name = "gitdb"
version = "4.0.12"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "smmap" },
]
sdist = { url = "https://files.pythonhosted.org/packages/72/94/63b0fc47eb32792c7ba1fe1b694daec9a63620db1e313033d18140c2320a/gitdb-4.0.12.tar.gz", hash = "sha256:5ef71f855d191a3326fcfbc0d5da835f26b13fbcba60c32c21091c349ffdb571", size = 394684, upload-time = "2025-01-02T07:20:46.413Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/a0/61/5c78b91c3143ed5c14207f463aecfc8f9dbb5092fb2869baf37c273b2705/gitdb-4.0.12-py3-none-any.whl", hash = "sha256:67073e15955400952c6565cc3e707c554a4eea2e428946f7a4c162fab9bd9bcbf", size = 62794, upload-time = "2025-01-02T07:20:43.624Z" },
]

[[package]]
name = "gitpython"
version = "3.1.50"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "gitdb" },
]
sdist = { url = "https://files.pythonhosted.org/packages/33/f6/354ae6491228b5eb40e10d89c4d13c651fe1cf7556e35ebdded50cff57ce/gitpython-3.1.50.tar.gz", hash = "sha256:80da2d12504d52e1f998772dc5baf6e553f8d2fcef1fcc226c9d9a2ee3372dcc", size = 219798, upload-time = "2026-05-06T04:01:26.571Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/20/7a/1c6e3562dfd8950adbb11ffbc65d21e7c89d01a6e4f137fa981056de25c5/gitpython-3.1.50-py3-none-any.whl", hash = "sha256:d352abe2908d07355014abdd21ddf798c2a961469239afec4962e9da884858f9", size = 212507, upload-time = "2026-05-06T04:01:23.799Z" },
]

[[package]]
name = "graphviz"
version = "0.21"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/f8/b3/3ac91e9be6b761a4b30d66ff165e54439dcd48b83f4e20d644867215f6ca/graphviz-0.21.tar.gz", hash = "sha256:20743e7183be82aaaa8ad6c93f8893c923bd6658a04c32ee115edb3c8a835f78", size = 200434, upload-time = "2025-06-15T09:35:05.824Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/91/4c/e0ce1ef95d4000ebc1c11801f9b944fa5910ecc15b5e351865763d8657f8/graphviz-0.21-py3-none-any.whl", hash = "sha256:54f33de9f4f911d7e84e4191749cac8cc5653f815b06738c54bd9a15ab8b1e42", size = 47300, upload-time = "2025-06-15T09:35:04.433Z" },
]

[[package]]
name = "h11"
version = "0.16.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/01/ee/02a2c011bdab74c6fb3c75474d40b3052059d95df7e73351460c8588d963/h11-0.16.0.tar.gz", hash = "sha256:4e35b956cf45792e4caa5885e69fba00bdbbc6ffafbfba020300e549b208ee5fff1", size = 101250, upload-time = "2025-04-24T03:35:25.427Z" }

```

```
wheels = [
 { url = "https://files.pythonhosted.org/packages/04/4b/29cac41a4d98d144bf5f6d33995617b185d14b22401f75ca86f384e87ff1
/h11-0.16.0-py3-none-any.whl", hash = "sha256:63cf8bbe7522de3b6f5932fda1d9c2772064ffb3dae62d55932da54b31cb6c86", size =
37515, upload-time = "2025-04-24T03:35:24.344Z" },
]

[[package]]
name = "httptools"
version = "0.8.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/43/e5/d471fcb0e14523fe1c3f4ba58ca52480e7bd70ad7109a3846bc75892
f7fb/httptools-0.8.0.tar.gz", hash = "sha256:6b2a32f18d97e16e90827d7a819ffa8dbd8cc245fc4e1fa9d1095b54ef4bd999", size =
271342, upload-time = "2026-05-25T22:17:48.841Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/14/88/1d21a36da8f5cb0fa49eafd4b169eba5608d57e75bbcf61845cbc6243216
/httptools-0.8.0-cp312-cp312-macosx_10_13_universal2.whl", hash = "sha256:880490234c10f70a9830743097e8958d6e4b9f5a0ffc2
4515023afeef984054d", size = 208247, upload-time = "2026-05-25T22:17:07.843Z" },
 { url = "https://files.pythonhosted.org/packages/a5/42/cc4feea2945cb3051038f090c9b36bd5b8a9d7f5a894a506a8983e33fd1c
/httptools-0.8.0-cp312-cp312-macosx_11_0_arm64.whl", hash = "sha256:5931891fb7b441b8a3853cf1b85c82c903defce084dd5f6771c
a46e31bf862c5", size = 113064, upload-time = "2026-05-25T22:17:09.136Z" },
 { url = "https://files.pythonhosted.org/packages/e3/a6/febbb8b8db0f58b38e44ad6cb946e6a255ae49b55f2e8543408fb7501ccd
/httptools-0.8.0-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl", hash = "sha256:b15fc622
b0f869d19207c4089a501d9bcc63ca5e071ffdd2f03f922df882dcb2", size = 523851, upload-time = "2026-05-25T22:17:10.106Z" },
 { url = "https://files.pythonhosted.org/packages/b7/e4/f90a0df0b83beff265b7e3b65f2a4cefd95792d4be0ac3e16049f2acd3c2
/httptools-0.8.0-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:4
25f83884fd6343828d8c565f046cb72b6d19063f6924093e11bcd8e1548cd09", size = 518842, upload-time = "2026-05-25T22:17:11.218
Z" },
 { url = "https://files.pythonhosted.org/packages/9e/2d/0c9ac76dd2c893841fbf6498d6accec4f2442e1b7067f6e3e316a80e494e8
/httptools-0.8.0-cp312-cp312-musllinux_1_2_aarch64.whl", hash = "sha256:ef7c3c97f4311c7be57e2986629df89d49cb434dbff78ea
fcd48c2bffb986b15a", size = 501238, upload-time = "2026-05-25T22:17:12.728Z" },
 { url = "https://files.pythonhosted.org/packages/ca/42/906adc91ae3a5fa9c59c0a2f21c139725bd7e5b41ae6acd485cd14123ebf
/httptools-0.8.0-cp312-cp312-musllinux_1_2_x86_64.whl", hash = "sha256:a1afd7c9fbff0d9f5d489c4ce2768bd09c84a46ddefc7161
e6aa82ae35c85745", size = 509567, upload-time = "2026-05-25T22:17:13.842Z" },
 { url = "https://files.pythonhosted.org/packages/05/0b/4240efeb672751ee5b9b380cb0e3fcd050bc05f68adc7a8aefc4fcd9a69a
/httptools-0.8.0-cp312-cp312-win_amd64.whl", hash = "sha256:cd96f29b4bab1d42fa6e3d008711c75e0f79e94e06827330160e3a30422
7f150", size = 90918, upload-time = "2026-05-25T22:17:15.155Z" },
 { url = "https://files.pythonhosted.org/packages/5e/e5/8cfcabc5546e8022f168be28bcdaa128a240a0befdd03b59d558b4f18bd6
/httptools-0.8.0-cp313-cp313-macosx_10_13_universal2.whl", hash = "sha256:614ceea8ea606848bece2338ac03b3ce5324bcb4be8dc
7d377ed708012fa4db8", size = 205148, upload-time = "2026-05-25T22:17:16.333Z" },
 { url = "https://files.pythonhosted.org/packages/2a/0e/0fb14848c19a686c8062ff9067c1a48793e3224b47bc5b201535b6036fce
/httptools-0.8.0-cp313-cp313-macosx_11_0_arm64.whl", hash = "sha256:2d689918c15a013c65ef52d9fd495d766893ab831a2c8d89f2a
c5940a5df847c", size = 111368, upload-time = "2026-05-25T22:17:17.586Z" },
 { url = "https://files.pythonhosted.org/packages/2e/1b/46f1cecfc06b9bbde8e4b8c88034ac7908989e5ff7a3a388ef38392949c1f
/httptools-0.8.0-cp313-cp313-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl", hash = "sha256:eb3028cc
a2fc0a6d720e52ef61d8ebb62fcfbef1de56874546d858d3f25a26b7", size = 486447, upload-time = "2026-05-25T22:17:18.564Z" },
 { url = "https://files.pythonhosted.org/packages/77/00/258bfc0837221f81d9725c45f9b948a6a6b2994a147a4fb66e85100c668f
/httptools-0.8.0-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:8
8bdd940f2b5d487b4d032c6afa5489a7dc4694410d43de3c38c4fb3af0dc45d", size = 482448, upload-time = "2026-05-25T22:17:19.912
Z" },
 { url = "https://files.pythonhosted.org/packages/04/ab/d1cef3b5523f4d272a70f42a776c3169a2dddf3a54de4b2ce4a36341528
/httptools-0.8.0-cp313-cp313-musllinux_1_2_aarch64.whl", hash = "sha256:6a43c9dd399758ccc0531acb0a3c4a6c299ee893ee9400e
9c893b7bdcfae0681", size = 464460, upload-time = "2026-05-25T22:17:20.882Z" },
 { url = "https://files.pythonhosted.org/packages/ce/48/5d1d072442277bb2b3434e0e60690b8e8c23840ef7de8b6ea54040a536d3
/httptools-0.8.0-cp313-cp313-musllinux_1_2_x86_64.whl", hash = "sha256:0770728beb05094c809b98e814eddf5fef69d26ad7d21185
f2f6d5884a0ba683", size = 471312, upload-time = "2026-05-25T22:17:22.085Z" },
 { url = "https://files.pythonhosted.org/packages/0d/66/b96623b27e51a68199ef4efdda0613cced9233fe3062ac74e50749c5ad37
/httptools-0.8.0-cp313-cp313-win_amd64.whl", hash = "sha256:7685df791fad561384bfb139e77fde27a1ff93134e016f95a0db424ffb
f77b1", size = 90117, upload-time = "2026-05-25T22:17:23.074Z" },
 { url = "https://files.pythonhosted.org/packages/1a/12/fa3fbf5f9517b273edea2dc982aa82a8c634091e67c590792b729017bc6f
/httptools-0.8.0-cp314-cp314-macosx_10_13_universal2.whl", hash = "sha256:de242a49b5d18e0a8776e654e9f6bf6d89f3875a5c35b
425a0e7ce940feb3fd6", size = 206183, upload-time = "2026-05-25T22:17:24.004Z" },
 { url = "https://files.pythonhosted.org/packages/30/fc/5e7c4cb443370f2090a3aba0453a07384d29ff66b7435bb90e77e1037599
/httptools-0.8.0-cp314-cp314-macosx_11_0_arm64.whl", hash = "sha256:159e9ab5f701ccd42e555a12f1ad8ff69702910fc1c996cf2bb
66e5fcb7a231b", size = 112079, upload-time = "2026-05-25T22:17:25.216Z" },
 { url = "https://files.pythonhosted.org/packages/ba/53/771bd891eb0f236f32145d6a1775777ec85745f3cc983a1f23d1a3b8ddfe
/httptools-0.8.0-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl", hash = "sha256:c4a9f170
7e4823d54dfec6c33fa3697d302aed536ed352a7ebb5a061ddb869d0", size = 481596, upload-time = "2026-05-25T22:17:26.186Z" },
 { url = "https://files.pythonhosted.org/packages/62/42/94e15bc68ce3d423243c45d7f1b0c7561f13844f97dc52ae23182fb65628
/httptools-0.8.0-cp314-cp314-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:d
76ad7b951387e3632c8716a9bb03ac5b45c5f16119aa409db0459520887944e", size = 480865, upload-time = "2026-05-25T22:17:27.542
Z" },
 { url = "https://files.pythonhosted.org/packages/1c/7c/fe2980fc03723272e30f135b62360b075f513dfe7cc73aef36c7f04012bd
/httptools-0.8.0-cp314-cp314-musllinux_1_2_aarch64.whl", hash = "sha256:a3b7387147361c3fd47a0bde763c5c91b5b4cd4dc9989b8
ece84ff436c99843b", size = 463189, upload-time = "2026-05-25T22:17:28.546Z" },
 { url = "https://files.pythonhosted.org/packages/15/1b/47fc5fff68acd1bfa20b4734059c9a06cadb88119dcd5258b5b0d21d91c8
/httptools-0.8.0-cp314-cp314-musllinux_1_2_x86_64.whl", hash = "sha256:f256d6ce930c52ca1cb2a960b7da03548c454e7d28b06059
```

```

ad41bfe789036ce0", size = 466610, upload-time = "2026-05-25T22:17:29.816Z" },
 { url = "https://files.pythonhosted.org/packages/60/bd/07b13c93ffd9bec9546e0d43f8e19378dd696dbd278511406bc07371ef1f
/httptools-0.8.0-cp314-cp314-win_amd64.whl", hash = "sha256:19d1ee275bb59ba2643ba9a3a1e51cc0c788caf2b8df506368e03f56fdd
08527", size = 92705, upload-time = "2026-05-25T22:17:31.133Z" },
 { url = "https://files.pythonhosted.org/packages/fd/c4/121648f68ce066d7bd762d6b6d97e620847642d38d54f3d90ff11d947629
/httptools-0.8.0-cp314-cp314t-macosx_11_13_universal2.whl", hash = "sha256:de1ed58a974e75d56560acc7e7fed01a454994429456
f65209789992e41f2568", size = 215023, upload-time = "2026-05-25T22:17:32.401Z" },
 { url = "https://files.pythonhosted.org/packages/b9/b0/312a062ae741ae3e8baa8c8bf20be81b2e67337b259ab4349bec7b6142e
/httptools-0.8.0-cp314-cp314t-macosx_11_0_arm64.whl", hash = "sha256:e93c227b595c6926c1acee96891dd9da4be338cfbe82e5cd3b
b9d8dd7dc4ac0b", size = 117405, upload-time = "2026-05-25T22:17:33.742Z" },
 { url = "https://files.pythonhosted.org/packages/fc/37/fccd705f795386bb05bf413012fecff2a33e5aa8c2f069096de3e9fd8702
/httptools-0.8.0-cp314-cp314t-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl", hash = "sha256:2a021c3
a8e65cc125390d72f59b968afca3bdcaff25bd67965e0a055a14946ca", size = 558497, upload-time = "2026-05-25T22:17:34.732Z" },
 { url = "https://files.pythonhosted.org/packages/bd/39/f172e803576de35f5ba77ff417cf0e34429d35dc014deef15afa337a72c
/httptools-0.8.0-cp314-cp314t-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:
48774d39cbb70e2b1f71f88852a3087ae1d3a1eb80482bb48c13067ab080c14f", size = 571585, upload-time = "2026-05-25T22:17:35.81
3Z" },
 { url = "https://files.pythonhosted.org/packages/3e/b9/f5564760af99f3dbbf3f9104dc00e5da27e96cf433c6bdcf77617f70bf3f
/httptools-0.8.0-cp314-cp314t-musllinux_1_2_aarch64.whl", hash = "sha256:88eead8ec8680a9f146c655bc88445a325bd7921cfd819
4c7337e9467282427d", size = 543297, upload-time = "2026-05-25T22:17:37.08Z" },
 { url = "https://files.pythonhosted.org/packages/99/67/8d9f2c313618e161b82f3873188e7196126da1d6e29688df40eb3997c77a
/httptools-0.8.0-cp314-cp314t-musllinux_1_2_x86_64.whl", hash = "sha256:2c032fa028f46871ec7e1fc59fc15e8023eab3e6bbe6ece
786a1611719a5d081", size = 539535, upload-time = "2026-05-25T22:17:38.032Z" },
 { url = "https://files.pythonhosted.org/packages/48/63/b906c01e53f50d432c0defe43ce52764a111dc1bdd028bafbeb54dcfd008
/httptools-0.8.0-cp314-cp314t-win_amd64.whl", hash = "sha256:384c17174464c8e873398b7af24f0b1f44d992c820328413951a625323
155d77", size = 108209, upload-time = "2026-05-25T22:17:39.473Z" },
]

```

```

[[package]]
name = "idms-db2-modernizer"
version = "0.1.0"
source = { virtual = "." }
dependencies = [
 { name = "graphviz" },
 { name = "networkx" },
 { name = "pdfplumber" },
 { name = "pydantic" },
 { name = "pydantic-settings" },
 { name = "streamlit" },
]

```

```

[package.dev-dependencies]
dev = [
 { name = "pdfminer-six" },
 { name = "pytest" },
]

```

```

[package.metadata]
requires-dist = [
 { name = "graphviz", specifier = ">=0.21" },
 { name = "networkx", specifier = ">=3.6.1" },
 { name = "pdfplumber", specifier = ">=0.11.10" },
 { name = "pydantic", specifier = ">=2.13.4" },
 { name = "pydantic-settings", specifier = ">=2.14.1" },
 { name = "streamlit", specifier = ">=1.58.0" },
]

```

```

[package.metadata.requires-dev]
dev = [
 { name = "pdfminer-six", specifier = ">=20250506" },
 { name = "pytest", specifier = ">=9.1.0" },
]

```

```

[[package]]
name = "idna"
version = "3.18"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/cd/63/9496c57188a2ee585e0f1db071d75089a11e98aa86eb99d9d7618fc1
edge/idna-3.18.tar.gz", hash = "sha256:ffb385a7e039654cef1ab9ef32c6fafe283c0c0467bba1d9029738ce4a14a848", size = 196711
, upload-time = "2026-06-02T14:34:07.794Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/1e/5e/d4e9f1a599fb8e573b7b87160658329fbf28d19eac2718f51fc3def3aa5a
/idna-3.18-py3-none-any.whl", hash = "sha256:7f952cbe720b688055e3f87de14f5c3e5fdaa8bc3928985c4077ca689de849a2", size =
65455, upload-time = "2026-06-02T14:34:06.319Z" },
]

```

```

[[package]]
name = "iniconfig"
version = "2.3.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/72/34/14ca021ce8e5dfedc35312d08ba8bf51 added999c576889fc2c24cb97f4f10/iniconfig-2.3.0.tar.gz", hash = "sha256:c76315c77db068650d49c5b56314774a7804df16fee4402c1f19d6d15d8c4730", size = 20503, upload-time = "2025-10-18T21:55:43.219Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/cb/b1/3846dd7f199d53cb17f49cba7e651e9ce294d8497c8c150530ed11865bb8/iniconfig-2.3.0-py3-none-any.whl", hash = "sha256:f631c04d2c48c52b84d0d0549c99ff3859c98df65b3101406327ecc7d53fbf12", size = 7484, upload-time = "2025-10-18T21:55:41.639Z" },
]

[[package]]
name = "itsdangerous"
version = "2.2.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/9c/cb/8ac0172223afbcb63986cc25049b154ecfb5e85932587206f42317be31d/itsdangerous-2.2.0.tar.gz", hash = "sha256:e0050c0b7da1eea53ffaf149c0cfbb5c6e2e2b69c4bef22c81fa6eb73e5f6173", size = 54410, upload-time = "2024-04-16T21:28:15.614Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/04/96/92447566d16df59b2a776c0fb82dbc4d9e07cd95062562af01e408583fc4/itsdangerous-2.2.0-py3-none-any.whl", hash = "sha256:c6242fc49e35958c8b15141343aa660db5fc54d4f13a1db01a3f5891b98700ef", size = 16234, upload-time = "2024-04-16T21:28:14.499Z" },
]

[[package]]
name = "jinja2"
version = "3.1.6"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "markupsafe" },
]
sdist = { url = "https://files.pythonhosted.org/packages/df/bf/f7da0350254c0ed7c72f3e33cef02e048281fec7ecec5f032d4aac52226b/jinja2-3.1.6.tar.gz", hash = "sha256:0137fb05990d35f1275a587e9aee6d56da821fc83491a0fb838183be43f66d6d", size = 245115, upload-time = "2025-03-05T20:05:02.478Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/62/a1/3d680cbfd5f4b8f15abc1d571870c5fc3e594bb582bc3b64ea099db13e56/jinja2-3.1.6-py3-none-any.whl", hash = "sha256:85ece4451f492d0c13c5dd7c13a64681a86afae63a5f347908daf103ce6d2f67", size = 134899, upload-time = "2025-03-05T20:05:00.369Z" },
]

[[package]]
name = "jsonschema"
version = "4.26.0"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "attrs" },
 { name = "jsonschema-specifications" },
 { name = "referencing" },
 { name = "rpds-py" },
]
sdist = { url = "https://files.pythonhosted.org/packages/b3/fc/e067678238fa451312d4c62bf6e6cf5ec56375422aee02f9cb5f909b3047/jsonschema-4.26.0.tar.gz", hash = "sha256:0c26707e2efad8aa1bfc5b7ce170f3fcccc2e4918ff85989ba9ffa9facb2be326", size = 366583, upload-time = "2026-01-07T13:41:07.246Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/69/90/f63fb5873511e014207a475e2bb4e8b2e570d655b00ac19a9a0ca0a385ee/jsonschema-4.26.0-py3-none-any.whl", hash = "sha256:d489f15263b8d200f8387e64b4c3a75f06629559fb73deb8dfdb525f2dab50ce", size = 90630, upload-time = "2026-01-07T13:41:05.306Z" },
]

[[package]]
name = "jsonschema-specifications"
version = "2025.9.1"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "referencing" },
]
sdist = { url = "https://files.pythonhosted.org/packages/19/74/a633ee74eb36c44aa6d1095e7cc5569bebf04342ee146178e2d36600708b/jsonschema_specifications-2025.9.1.tar.gz", hash = "sha256:b540987f239e745613c7a9176f3edb72b832a4ac465cf02712288397832b5e8d", size = 32855, upload-time = "2025-09-08T01:34:59.186Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/41/45/1a4ed80516f02155c51f51e8cedb3c1902296743db0bbc66608a0db2814f/jsonschema_specifications-2025.9.1-py3-none-any.whl", hash = "sha256:98802fee3a11ee76ecaca44429fda8a41bfff98b00a0f2838151b113f210cc6fe", size = 18437, upload-time = "2025-09-08T01:34:57.871Z" },
]

```

```

[[package]]
name = "markupsafe"
version = "3.0.3"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/7e/99/7690b6d4034fffd95959cbe0c02de8deb3098cc577c67bb6a24fe5d7caa7/markupsafe-3.0.3.tar.gz", hash = "sha256:722695808f4b6457b320fdc131280796bdcbe04ab50fe1795cd540799ebe1698", size = 80313, upload-time = "2025-09-27T18:37:40.426Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/5a/72/147da192e38635ada20e0a2e1a51cf8823d2119ce8883f7053879c2199b5/markupsafe-3.0.3-cp312-cp312-macosx_10_13_x86_64.whl", hash = "sha256:d53197da72cc091b024dd97249dfc7794d6a56530370992a5e1a08983ad9230e", size = 11615, upload-time = "2025-09-27T18:36:30.854Z" },
 { url = "https://files.pythonhosted.org/packages/9a/81/7e4e08678a1f98521201c3079f77db69fb552acd56067661f8c2f534a718/markupsafe-3.0.3-cp312-cp312-macosx_11_0_arm64.whl", hash = "sha256:1872df69a4de6aeadd3491198eaf13810b565bdbc3ae2dc8780f14458ec73ce", size = 12020, upload-time = "2025-09-27T18:36:31.971Z" },
 { url = "https://files.pythonhosted.org/packages/1e/2c/799f4742efc39633a1b54a92eec4082e4f815314869865d876824c257c1e/markupsafe-3.0.3-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:3a7e8ae81ae39e62a41ec302f972ba6ae23a5c5396c8e60113e9066ef893da0d", size = 24332, upload-time = "2025-09-27T18:36:32.813Z" },
 { url = "https://files.pythonhosted.org/packages/3c/2e/8d0c2ab90a8c1d9a24f0399058ab8519a3279d1bd4289511d74e909f060e/markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:d6dd0be5b5b189d31db7cda48b91d7e0a9795f31430b7f271219ab30fd3ac9d", size = 22947, upload-time = "2025-09-27T18:36:33.862Z" },
 { url = "https://files.pythonhosted.org/packages/2c/54/887f3092a85238093a0b2154bd629c89444f395618842e8b0c41783898ea/markupsafe-3.0.3-cp312-cp312-manylinux_2_31_riscv64.manylinux_2_39_riscv64.whl", hash = "sha256:94c6f0bb423f739146aec64595853541634bde58b2135f27f61c1ffd1cd4d16a", size = 21962, upload-time = "2025-09-27T18:36:35.099Z" },
 { url = "https://files.pythonhosted.org/packages/c9/2f/336b8c7b6f4a4d95e91119dc8521402461b74a485558d8f238a68312f11c/markupsafe-3.0.3-cp312-cp312-musllinux_1_2_aarch64.whl", hash = "sha256:be8813b57049a7dc738189df53d69395eba14fb99345e0a5994914a3864c8a4b", size = 23760, upload-time = "2025-09-27T18:36:36.001Z" },
 { url = "https://files.pythonhosted.org/packages/32/43/67935f2b7e4982ff50a4d169b724d74b62a3964bc1a9a527f5ac4f1ee2b/markupsafe-3.0.3-cp312-cp312-musllinux_1_2_riscv64.whl", hash = "sha256:83891d0e9fb81a825d9a6d61e3f07550ca70a076484292a70fde82c4b807286f", size = 21529, upload-time = "2025-09-27T18:36:36.906Z" },
 { url = "https://files.pythonhosted.org/packages/89/e0/4486f11e51bbb8b0c041098859e869e304d1c261e59244baa3d295d47b7/markupsafe-3.0.3-cp312-cp312-musllinux_1_2_x86_64.whl", hash = "sha256:77f0643abe7495da77fb436f50f8dab76dbc6e5fd25d39589a0f1fe6548bfa2b", size = 23015, upload-time = "2025-09-27T18:36:37.868Z" },
 { url = "https://files.pythonhosted.org/packages/2f/e1/78ee7a023dac597a5825441ebd17170785a9dab23de95d2c7508ade94e0e/markupsafe-3.0.3-cp312-cp312-win32.whl", hash = "sha256:d88b440e37a16e651bda4c7c2b930eb586fd15ca7406cb39e211fcff3bf3017d", size = 14540, upload-time = "2025-09-27T18:36:38.761Z" },
 { url = "https://files.pythonhosted.org/packages/aa/5b/bec5aa9bbbbb2c946ca2733ef9c4ca91c91b6a24580193e891b5f7d8e8e1e/markupsafe-3.0.3-cp312-cp312-win_amd64.whl", hash = "sha256:26a5784ded40c9e318cfc2bdb30fe164bdb8665ded9cd64d500a34fb42067b1c", size = 15105, upload-time = "2025-09-27T18:36:39.701Z" },
 { url = "https://files.pythonhosted.org/packages/e5/f1/216fc1bbfd74011693a4fd837e7026152e89c4bcf3e77b6692fba9923123/markupsafe-3.0.3-cp312-cp312-win_arm64.whl", hash = "sha256:35add3b638a5d900e807944a078b51922212fb3dedb01633a8defc4b01a3c85f", size = 13906, upload-time = "2025-09-27T18:36:40.689Z" },
 { url = "https://files.pythonhosted.org/packages/38/2f/907b9c7bbba283e68f20259574b13d005c121a0fa4c175f9bed27c4597ff/markupsafe-3.0.3-cp313-cp313-macosx_10_13_x86_64.whl", hash = "sha256:e1cf1972137e83c5d4c136c43ced9ac51d0e124706ee1c8a8532c1287fa8795", size = 11622, upload-time = "2025-09-27T18:36:41.777Z" },
 { url = "https://files.pythonhosted.org/packages/9c/d9/5f7756922cdd676869eca1c4e3c0cd0df60ed30199ffd775e319089cb3ed/markupsafe-3.0.3-cp313-cp313-macosx_11_0_arm64.whl", hash = "sha256:116bb52f642a37c115f517494ea5feb03889e04df47eef5b130b1808ce7c219", size = 12029, upload-time = "2025-09-27T18:36:43.257Z" },
 { url = "https://files.pythonhosted.org/packages/00/07/575a68c754943058c78f30db02ee03a64b3c638586fba6add56830b30a3/markupsafe-3.0.3-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:133a43e73a802c5562be9bbcd03d090aa5a1fe899db609c29e8c8d815c5f6de6", size = 24374, upload-time = "2025-09-27T18:36:44.508Z" },
 { url = "https://files.pythonhosted.org/packages/a9/21/9b05698b46f218fc0e118e1f8168395c65c8a2c750ae2bab54fc4bd4e0e8/markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:ccfcd093f13f0f0b7fdd0f198b90053bf7b2f02a3927a30e63f3ccc9df56b676", size = 22980, upload-time = "2025-09-27T18:36:45.385Z" },
 { url = "https://files.pythonhosted.org/packages/7f/71/544260864f893f18b6827315b988c146b559391e6e7e8f7252839b1b846a/markupsafe-3.0.3-cp313-cp313-manylinux_2_31_riscv64.manylinux_2_39_riscv64.whl", hash = "sha256:509fa21c6deb7a7a273d629cf5ec029bc209d1a51178615ddf718f5918992ab9", size = 21990, upload-time = "2025-09-27T18:36:46.916Z" },
 { url = "https://files.pythonhosted.org/packages/c2/28/b50fc2f74d1ad761af2f5dcce7492648b983d00a65b8c0eb457c82ebbe/markupsafe-3.0.3-cp313-cp313-musllinux_1_2_aarch64.whl", hash = "sha256:a4afe79fb3de0b7097d81da19090f4df4f8d3a2b3adaa8764138aac2e44f3af1", size = 23784, upload-time = "2025-09-27T18:36:47.884Z" },
 { url = "https://files.pythonhosted.org/packages/ed/76/104b2aa106a208da8b17a2fb72e033a5a9d7073c68f7e508b94916ed47a9/markupsafe-3.0.3-cp313-cp313-musllinux_1_2_riscv64.whl", hash = "sha256:795e7751525cae078558e679d646ae45574b47ed6e7771863fcc079a6171a0fc", size = 21588, upload-time = "2025-09-27T18:36:48.82Z" },
 { url = "https://files.pythonhosted.org/packages/b5/99/16a5eb2d140087ebd97180d95249b00a03aa87e29cc224056274f2e45fd6/markupsafe-3.0.3-cp313-cp313-musllinux_1_2_x86_64.whl", hash = "sha256:8485f406a96febb5140bfeca44a73e3ce5116b2501ac54fe953e488fb1d03b12", size = 23041, upload-time = "2025-09-27T18:36:49.797Z" },
 { url = "https://files.pythonhosted.org/packages/19/bc/e7140ed90c5d61d77cea142eed9f9c303f4c4806f60a1044c13e3f1471d0/markupsafe-3.0.3-cp313-cp313-win32.whl", hash = "sha256:bdd37121970bf8be76c5fb069c7751683bdf373db1ed6c010162b2a130248ed", size = 14543, upload-time = "2025-09-27T18:36:51.584Z" },
 { url = "https://files.pythonhosted.org/packages/05/73/c4abe620b841b6b791f2edc248f556900667a5a1cf023a6646967ae98335/markupsafe-3.0.3-cp313-cp313-win_amd64.whl", hash = "sha256:9a1abfdcd021a164803f4d485104931fb8f8c1efd55bc6b748d2f5774e78b62c5", size = 15113, upload-time = "2025-09-27T18:36:52.537Z" },

```

```
{ url = "https://files.pythonhosted.org/packages/f0/3a/fa34a0f7cfef23cf9500d68cb7c32dd64ffd58a12b09225fb03dd37d5b80/markupsafe-3.0.3-cp313-cp313-win_arm64.whl", hash = "sha256:7e68f88e5b8799aa49c85cd116c932a1ac15caaa3f5db09087854d218359e485", size = 13911, upload-time = "2025-09-27T18:36:53.513Z" },
{ url = "https://files.pythonhosted.org/packages/e4/d7/e05cd7efe43a88a17a37b3ae96e79a19e846f3f456fe79c57ca61356ef01/markupsafe-3.0.3-cp313-cp313t-macosx_10_13_x86_64.whl", hash = "sha256:218551f6df4868a8d527e3062d0fb968682fe92054e89978594c28e642c43a73", size = 11658, upload-time = "2025-09-27T18:36:54.819Z" },
{ url = "https://files.pythonhosted.org/packages/99/9e/e412117548182ce2148bdeacdda3bb494260c0b0184360fe0d56389b523b/markupsafe-3.0.3-cp313t-cp313t-macosx_11_0_arm64.whl", hash = "sha256:3524b778fe5cfc3452a09d31e7b5adeffeea8c5be1d43c4f810ba09f2ceb29d37", size = 12066, upload-time = "2025-09-27T18:36:55.714Z" },
{ url = "https://files.pythonhosted.org/packages/bc/e6/fa0ffcd717ef64a5108eaa7b4f5ed28d56122c9a6d70ab8b72f9f715c80/markupsafe-3.0.3-cp313-cp313t-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:4e885a3d1efa2eadc93c894a21770e4bc67899e3543680313b09f139e149ab19", size = 25639, upload-time = "2025-09-27T18:36:56.908Z" },
{ url = "https://files.pythonhosted.org/packages/96/ec/2102e881fe9d25fc16cb4b25d5f5cde50970967ffa5dddafdb771237062d/markupsafe-3.0.3-cp313-cp313t-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:8709b08f4a89aa7586de0aadcd8da56180242ee0ada3999749b183aa23df95025", size = 23569, upload-time = "2025-09-27T18:36:57.913Z" },
{ url = "https://files.pythonhosted.org/packages/4b/30/6f2fce1f1f205fc9323255b216ca8a235b15860c34b6798f810f05828e32/markupsafe-3.0.3-cp313-cp313t-manylinux_2_31_riscv64.manylinux_2_39_riscv64.whl", hash = "sha256:b8512a91625c9b3da6f127803b166b629725e68af71f8184ae7e7d54686a56d6", size = 23284, upload-time = "2025-09-27T18:36:58.833Z" },
{ url = "https://files.pythonhosted.org/packages/58/47/4a0ccca4ab9f5dcb6f79c0236d954acb38220721e704223a8aafa38b5c8/markupsafe-3.0.3-cp313-cp313t-musllinux_1_2_aarch64.whl", hash = "sha256:9b79b7a16f7fedff2495d684f2b59b0457c3b493778c9eed3111be64d58279f", size = 24801, upload-time = "2025-09-27T18:36:59.739Z" },
{ url = "https://files.pythonhosted.org/packages/6a/70/3780e9b72180b6fecb83a4814d84c3bf4b4ae4bf0b19c27196104149734c/markupsafe-3.0.3-cp313-cp313t-musllinux_1_2_riscv64.whl", hash = "sha256:12c63dfb4a98206f045aa9563db46507995f7ef6d83b2f68eda65c307c6829eb", size = 22769, upload-time = "2025-09-27T18:37:00.719Z" },
{ url = "https://files.pythonhosted.org/packages/98/c5/c03c7f4125180fc215220c035beac6b9cb684bc7a067c84fc69414d315f5/markupsafe-3.0.3-cp313-cp313t-musllinux_1_2_x86_64.whl", hash = "sha256:8f71bc33915be5186016f675cd83a1e08523649b0e33efdb898db577ef5bb009", size = 23642, upload-time = "2025-09-27T18:37:01.673Z" },
{ url = "https://files.pythonhosted.org/packages/80/d6/2d1b89f6ca4bfff1036499b1e29a1d02d282259f3681540e16563f27ebc23/markupsafe-3.0.3-cp313-cp313t-win32.whl", hash = "sha256:69c0b73548bc525c8cb9a251cddf1931d1db4d2258e9599c28c07ef3580ef354", size = 14612, upload-time = "2025-09-27T18:37:02.639Z" },
{ url = "https://files.pythonhosted.org/packages/2b/98/e48a4bfba0a0ffcf9925fe2d69240bfaa19c6f7507b8cd09c70684a53c1e/markupsafe-3.0.3-cp313-cp313t-win_amd64.whl", hash = "sha256:1b4b79e8ebf6b55351f0d91fe80f893b4743f104bff22e90697db1590e47a218", size = 15200, upload-time = "2025-09-27T18:37:03.582Z" },
{ url = "https://files.pythonhosted.org/packages/0e/72/e3cc540f351f316e9ed0f092757459afbc595824ca724cbc5a5d4263713f/markupsafe-3.0.3-cp313-cp313t-win_arm64.whl", hash = "sha256:ad2cf8aa28b8c020ab2fc8287b0f823d0a7d8630784c31e9ee5edea20f406287", size = 13973, upload-time = "2025-09-27T18:37:04.929Z" },
{ url = "https://files.pythonhosted.org/packages/33/8a/8e42d838cd89b7dde187011e97fe6c3af66d8c044997d2183fbd6d31352/markupsafe-3.0.3-cp314-cp314-macosx_10_13_x86_64.whl", hash = "sha256:eea9599de571d72e2daf60164784109f19978b327a3910d3e9de8c97b5b70cfe", size = 11619, upload-time = "2025-09-27T18:37:06.342Z" },
{ url = "https://files.pythonhosted.org/packages/b5/64/7660f8a4a8e53c924d0fa05dc3a55c9cee10bbd82b11c5afb27d44b096ce/markupsafe-3.0.3-cp314-cp314-macosx_11_0_arm64.whl", hash = "sha256:c47a551199eb8eb2121d4f0f15ae0f923d31350ab9280078d1e5f12b249e0026", size = 12029, upload-time = "2025-09-27T18:37:07.213Z" },
{ url = "https://files.pythonhosted.org/packages/da/ef/e648bffd021127bef5fa12e1720ffed0c6cbb8310c8d9bea7266337ff06de/markupsafe-3.0.3-cp314-cp314-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:f34c41761022dd093b4b6896d4810782ffbbabe30f2d443ff5f083e0cbbb8c737", size = 24408, upload-time = "2025-09-27T18:37:09.572Z" },
{ url = "https://files.pythonhosted.org/packages/41/3c/a36c2450754618e62008bf7435ccb0f88053e07592e6028a34776213d877/markupsafe-3.0.3-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:457a69a9577064c05a97c41f4e65148652db078a3a509039e64d3467b9ef97", size = 23005, upload-time = "2025-09-27T18:37:10.58Z" },
{ url = "https://files.pythonhosted.org/packages/bc/20/b7fdf89a8456b099837cd1dc21974632a02a999ec9bf7ca3e490aacd98e7/markupsafe-3.0.3-cp314-cp314-manylinux_2_31_riscv64.manylinux_2_39_riscv64.whl", hash = "sha256:e8afcf3f2ccfa24215f8cb28dcf43f0113ac3c37c2f0f0806d8c70e4228c5cf4d", size = 22048, upload-time = "2025-09-27T18:37:11.547Z" },
{ url = "https://files.pythonhosted.org/packages/9a/a7/591f592afdc734f4db08a75793a55d7fbcc6902a723ae4cfcfbab61010cc5/markupsafe-3.0.3-cp314-cp314-musllinux_1_2_aarch64.whl", hash = "sha256:ec15a59cf5af7be74194f7ab02d0f59a62bdcf1a537677ce67a2537c9b87fcda", size = 23821, upload-time = "2025-09-27T18:37:12.48Z" },
{ url = "https://files.pythonhosted.org/packages/7d/33/45b24e4f44195b26521bc6f1a82197118f74df348556594bd2262bda1038/markupsafe-3.0.3-cp314-cp314-musllinux_1_2_riscv64.whl", hash = "sha256:0eb9ff8191e8498cca014656ae6b8d61f39da5f95b488805da4bb029ccbbfbaf", size = 21606, upload-time = "2025-09-27T18:37:13.485Z" },
{ url = "https://files.pythonhosted.org/packages/ff/0e/53dfaca23a69fbbfbf17a4b64072090e70717344c52eaaaa9c5ddff1e5f0/markupsafe-3.0.3-cp314-cp314-musllinux_1_2_x86_64.whl", hash = "sha256:2713baf880df847f2bece4230d4d094280f4e67b1e813eec43b4c0e144a34ffe", size = 23043, upload-time = "2025-09-27T18:37:14.408Z" },
{ url = "https://files.pythonhosted.org/packages/46/11/f333a06fc16236d5238bfe74daccbca41459dcd8d1fa952e8fbd5dccb70/markupsafe-3.0.3-cp314-cp314-win32.whl", hash = "sha256:729586769a26dbceff69f7a7dbbf59ab6572b99d94576a5592625d5b411576b9", size = 14747, upload-time = "2025-09-27T18:37:15.36Z" },
{ url = "https://files.pythonhosted.org/packages/28/52/182836104b33b444e400b14f797212f720cbc9ed6ba34c800639d154e821/markupsafe-3.0.3-cp314-cp314-win_amd64.whl", hash = "sha256:bdc919ead48f234740ad807933cdf545180bfbe9342c2bb451556db2ed958581", size = 15341, upload-time = "2025-09-27T18:37:16.496Z" },
{ url = "https://files.pythonhosted.org/packages/6f/18/acf23e91bd94fd7b3031558b1f013adfa21a8e407a3fdb32745538730382/markupsafe-3.0.3-cp314-cp314-win_arm64.whl", hash = "sha256:5a7d5dc5140555cf21a6fefbdfb8723f06fcd2f63ef108f2854de715e4422cb4", size = 14073, upload-time = "2025-09-27T18:37:17.476Z" },
{ url = "https://files.pythonhosted.org/packages/3c/f0/57689aa4076e1b43b15fdfa646b04653969d50cf30c32a102762be2485da/markupsafe-3.0.3-cp314-cp314t-macosx_10_13_x86_64.whl", hash = "sha256:1353ef0cb1b138e1907ae78e2f6c63ff67501122006b0f9abad68fda5f4ffc6ab", size = 11661, upload-time = "2025-09-27T18:37:18.453Z" },
```



```

 { url = "https://files.pythonhosted.org/packages/89/c3/2e67a7ca217c6912985ec766c6393b636fb0c234443ff9d91404dc4c79f/markupsafe-3.0.3-cp314-cp314t-macosx_11_0_arm64.whl", hash = "sha256:1085e7fbddd3be5f89cc898938f42c0b3c711fdcb37d75221de2666af647c175", size = 12069, upload-time = "2025-09-27T18:37:19.332Z" },
 { url = "https://files.pythonhosted.org/packages/f0/00/be561dce4e6ca66b15276e184ce4b8aec61fe83662cce2f7d72bd3249d28/markupsafe-3.0.3-cp314-cp314t-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:1b52b4fb9d9f4eb9ae465f8d0c228a00624de2334f216f178a995ccdcf82c4634", size = 25670, upload-time = "2025-09-27T18:37:20.245Z" },
 { url = "https://files.pythonhosted.org/packages/50/09/c419f6f5a92e5fadde27efd190eca90f05e1261b10dbd8cbbcb39cd8ea1dc/markupsafe-3.0.3-cp314-cp314t-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:fe51ac40f757d41b7c48425901843666a6677e3e8eb0abcf09e4ba6e664f50", size = 23598, upload-time = "2025-09-27T18:37:21.177Z" },
 { url = "https://files.pythonhosted.org/packages/22/44/a0681611106e0b2921b3033fc19bc53323e0b50bc70cfffdd19f7d679bb66/markupsafe-3.0.3-cp314-cp314t-manylinux_2_31_riscv64.manylinux_2_39_riscv64.whl", hash = "sha256:f190daf0f1f3c72eac4efd5c430a8de82489d9cfff23c364c3ea822545032993e", size = 23261, upload-time = "2025-09-27T18:37:22.167Z" },
 { url = "https://files.pythonhosted.org/packages/5f/57/1b0b3f100259dc9fffe780cfb60d4be71375510e435efec3d116b6436d43/markupsafe-3.0.3-cp314-cp314t-musllinux_1_2_aarch64.whl", hash = "sha256:e56b7d45a839a697b5eb268c82a71bd8c7f6c94d6fd50c3d577fa39a9f1409f5", size = 24835, upload-time = "2025-09-27T18:37:23.296Z" },
 { url = "https://files.pythonhosted.org/packages/26/6a/4bf6d0c97c4920f1597cc14dd720705eca0bf7c787aebc6bb4d1bead5388/markupsafe-3.0.3-cp314-cp314t-musllinux_1_2_riscv64.whl", hash = "sha256:f3e98bb3798ead92273dc0e5fd0f31ade220f59a266ff4d8a4f6065e0a3ce0523", size = 22733, upload-time = "2025-09-27T18:37:24.237Z" },
 { url = "https://files.pythonhosted.org/packages/14/c7/ca723101509b518797fedc2fdf79ba57f886b4aca8a7d31857ba3ee8281f/markupsafe-3.0.3-cp314-cp314t-musllinux_1_2_x86_64.whl", hash = "sha256:5678211cb9333a6468fb8d8be0305520aa073f50d17f089b5b4b477ea6e67fdc", size = 23672, upload-time = "2025-09-27T18:37:25.271Z" },
 { url = "https://files.pythonhosted.org/packages/fb/df/5bd7a48c256faecd1d36edc13133e51397e41b73bb77e1a69deab746ebac/markupsafe-3.0.3-cp314-cp314t-win32.whl", hash = "sha256:915c04ba3851909ce68ccc2b8e2cd691618c4dc4c4232fb7982bca3f41fd8c3d", size = 14819, upload-time = "2025-09-27T18:37:26.285Z" },
 { url = "https://files.pythonhosted.org/packages/1a/8a/0402ba61a2f16038b48b39bccca271134be00c5c9f0f623208399333c448/markupsafe-3.0.3-cp314-cp314t-win_amd64.whl", hash = "sha256:4faaffd047e07c38848ce017e8725090413cd80cbc23d86e55c587bf979e579c9", size = 15426, upload-time = "2025-09-27T18:37:27.316Z" },
 { url = "https://files.pythonhosted.org/packages/70/bc/6f1c2f612465f5fa89b95bead1f44dc6b07670fd42891d8fcd5d039f4f4/markupsafe-3.0.3-cp314-cp314t-win_arm64.whl", hash = "sha256:32001d6a8fc98c8cb5c947787c5d08b0a50663d139f1305bac5885d98d9b40fa", size = 14146, upload-time = "2025-09-27T18:37:28.327Z" },
]

[[package]]
name = "narwhals"
version = "2.22.1"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/62/3c/c4ef2164a71c1a63d7f1ae411c4082c5fa872405106db60a4b7114989ad7/narwhals-2.22.1.tar.gz", hash = "sha256:d62920805a0a43b7ff8b54b0c0d3142d796f8a9301836ada37e573d6a33cbcd9", size = 647493, upload-time = "2026-06-05T12:34:34.051Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/48/ca/36339329c4604adbcc99c899b7eb1ce1a555c499b6a6860757dc9bfed36d/narwhals-2.22.1-py3-none-any.whl", hash = "sha256:60567d774edf77db53906f89d9fbd164e66e56d66d388e1e6990f17ac33cfb53", size = 454815, upload-time = "2026-06-05T12:34:32.289Z" },
]

[[package]]
name = "networkx"
version = "3.6.1"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/6a/51/63fe664f3908c97be9d2e4f1158eb633317598cfa6e1fc14af5383f17512/networkx-3.6.1.tar.gz", hash = "sha256:26b7c357acc0c8cde558ad486283728b65b6a95d85ee1cd66bafab4c8168509", size = 2517025, upload-time = "2025-12-08T17:02:39.908Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/9e/c9/b2622292ea83fbb4ec318f5b9ab867d0a28ab43c5717bb85b0a5f6b3b0a4/networkx-3.6.1-py3-none-any.whl", hash = "sha256:d47fbf302e7d9cbbb9e255a0d267983d2aa476bac30e90dfbe5669bd57f3762", size = 2068504, upload-time = "2025-12-08T17:02:38.159Z" },
]

[[package]]
name = "numpy"
version = "2.4.6"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/d0/ad/fed0499ce6a338d2a03ebae59cd15093910c8875328855781952abf6c2fe/numpy-2.4.6.tar.gz", hash = "sha256:f3a3570c4a2a16746ac2c31a7c7c7b0c186b95ce902e33db6f28094ed7387dda", size = 20735807, upload-time = "2026-05-18T23:37:14.07Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/95/2a/3d7b5ac8aac24feaf9ad7ed58f45b0bbc06d37e4338ae84c9f2298b570f9/numpy-2.4.6-cp312-cp312-macosx_10_13_x86_64.whl", hash = "sha256:001fbb8e08d942dd57599e781f2472269ee7f2755fae407b4f67b2f0b17da3f1", size = 16689119, upload-time = "2026-05-18T23:33:54.065Z" },
 { url = "https://files.pythonhosted.org/packages/ea/12/92c4c131527599e8288d6918e88d88726f84d805d784b771f32408aaef/numpy-2.4.6-cp312-cp312-macosx_11_0_arm64.whl", hash = "sha256:ebfb099f8dcf083deef3ac1ca4c1503f387cf76296fcb3816b66f5ecb5f54fdb", size = 14699246, upload-time = "2026-05-18T23:33:57.621Z" },
 { url = "https://files.pythonhosted.org/packages/ad/fe/c0a6b7b2ca128a8fb228575147073b660656734b8ebe4d76c8fd748dcc79/numpy-2.4.6-cp312-cp312-macosx_14_0_arm64.whl", hash = "sha256:3213d622a0283a39a93d188f3cf72b26862df52fbb4ca3697f51705"

```

016523d41", size = 5204410, upload-time = "2026-05-18T23:34:00.302Z" },  
 { url = "https://files.pythonhosted.org/packages/f3/d4/9770d14ba719432bb90a421bfd443872ed0f70f7264b64bec12ea363d5fd/numpy-2.4.6-cp312-cp312-macosx\_14\_0\_x86\_64.whl", hash = "sha256:357cc07a6d7b0b182ff02249616a03742827ebbb1277546b5c7cd7f7620a45698", size = 6551240, upload-time = "2026-05-18T23:34:02.852Z" },  
 { url = "https://files.pythonhosted.org/packages/c9/c6/50a46a6205feba2343f1d6d17438107c5dc491ed1c736e6ea68689fd906b/numpy-2.4.6-cp312-cp312-manylinux\_2\_27\_aarch64.manylinux\_2\_28\_aarch64.whl", hash = "sha256:5f9fb9157b4ce2971008323afe46053787b526ef624fea915b261468a8421a0f", size = 15671012, upload-time = "2026-05-18T23:34:05.485Z" },  
 { url = "https://files.pythonhosted.org/packages/99/60/14115e6364fa676c5397c2ad3004e527e9aa487abf5d0706ec81bbd08529/numpy-2.4.6-cp312-cp312-manylinux\_2\_27\_x86\_64.manylinux\_2\_28\_x86\_64.whl", hash = "sha256:90f9849678c75fe7afa2d348ac842c168b0a4d3d61919687216dfc547976d853", size = 16645538, upload-time = "2026-05-18T23:34:09.265Z" },  
 { url = "https://files.pythonhosted.org/packages/ae/c5/693cbe59e57db94d2231fa519ca3978dc9e19da5a8f088588f5c6e947ff2/numpy-2.4.6-cp312-cp312-musllinux\_1\_2\_aarch64.whl", hash = "sha256:c1a2af6c6ef86344a6b0db6b97834208bf598db514f2b155042439b62605601a", size = 17020706, upload-time = "2026-05-18T23:34:13.053Z" },  
 { url = "https://files.pythonhosted.org/packages/ef/fc/85b7c4eff9b4966ade25c2273cf7e7012e92366c032058653934b37de044/numpy-2.4.6-cp312-cp312-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:e5805d5a22fd19c8ccff10a9561f9df94436b0545619ea579db2d3c35294bce2", size = 18368541, upload-time = "2026-05-18T23:34:17.024Z" },  
 { url = "https://files.pythonhosted.org/packages/f6/81/e1b27545deedce7f4a0b348618c6b62d74e36a4dc9ccdd42f3eb2f85eee32/numpy-2.4.6-cp312-cp312-win32.whl", hash = "sha256:e3eeb0aabdb6d5ce64faae67e9935203a6991b4bc2a485a767fbafbc25125f45", size = 5962825, upload-time = "2026-05-18T23:34:20.3Z" },  
 { url = "https://files.pythonhosted.org/packages/ab/ca/feab00bd44aa5fe1ad2c18f08b4d3bb92e26484b0b1d1443897809ed528c/numpy-2.4.6-cp312-cp312-win\_amd64.whl", hash = "sha256:d8e8286dd7cea7895157318d1b91cdacac64c479f3c3bc8dce548331728484751", size = 12321687, upload-time = "2026-05-18T23:34:23.095Z" },  
 { url = "https://files.pythonhosted.org/packages/63/cf/5a6d34850a39d1093558564f77ee8e8e0bee5061151b8f05a55711001ec7/numpy-2.4.6-cp312-cp312-win\_arm64.whl", hash = "sha256:4081eb135ac24158bd51cdfbfef16f1c64df7063b1143f24731387137c092bec8", size = 10221482, upload-time = "2026-05-18T23:34:25.876Z" },  
 { url = "https://files.pythonhosted.org/packages/fb/82/bdad26d7438c6791ca31b7c024ca37c1eab8b726ba236129005cd4a06e45/numpy-2.4.6-cp313-cp313-macosx\_10\_13\_x86\_64.whl", hash = "sha256:511dbaf848decaaf4b4ca48032619fb3138710c4bf7da7617765edad1ef96b0", size = 16684648, upload-time = "2026-05-18T23:34:29.41Z" },  
 { url = "https://files.pythonhosted.org/packages/1b/30/a80189bcc7f5e4258b3fbc3968d909d1756f54d023299ecc39ad6fdb9ef8/numpy-2.4.6-cp313-cp313-macosx\_11\_0\_arm64.whl", hash = "sha256:bf162abab1c1a736333192707cef898e735a5ca00f38f27eedf44b39d9e85eb", size = 14693902, upload-time = "2026-05-18T23:34:33.013Z" },  
 { url = "https://files.pythonhosted.org/packages/97/12/70b5d0d7c15e1ebb8a6a84a8caa1d19e181d84fb58bb6d70aca29099dec1/numpy-2.4.6-cp313-cp313-macosx\_14\_0\_arm64.whl", hash = "sha256:043191bfa8eab18c776647b62723ac9dddece59743b13f49b2016094129c2b3f", size = 5198992, upload-time = "2026-05-18T23:34:36.132Z" },  
 { url = "https://files.pythonhosted.org/packages/ba/8c/ebd2a8f8a83541f8d38cc5667e8c2b69cccfcd30da6e45693e8158857d44b/numpy-2.4.6-cp313-cp313-macosx\_14\_0\_x86\_64.whl", hash = "sha256:6180d8b35af935aed8ece3a85e0a43f87393ae0ac87c8d2c8bd2c993f7270ef3", size = 6546944, upload-time = "2026-05-18T23:34:38.484Z" },  
 { url = "https://files.pythonhosted.org/packages/bb/c5/7b863a97a1671a0338f4253bd3b5a3d3852f0692dae91711c9f4a10e787/numpy-2.4.6-cp313-cp313-manylinux\_2\_27\_aarch64.manylinux\_2\_28\_aarch64.whl", hash = "sha256:72fbe16c6fac95aedf5937fa873445cec2110be35d8a4e9433d7501fd98dae6b", size = 15669392, upload-time = "2026-05-18T23:34:41.257Z" },  
 { url = "https://files.pythonhosted.org/packages/a5/9d/3584b9984ca4c047aea75214ce1a4c473d849bd71b604264b7f5653f8a8/numpy-2.4.6-cp313-cp313-manylinux\_2\_27\_x86\_64.manylinux\_2\_28\_x86\_64.whl", hash = "sha256:a7830bab239b79cda9c08c2da014761cafb48da6150e1da17ac06283f43b6089", size = 16633220, upload-time = "2026-05-18T23:34:45.075Z" },  
 { url = "https://files.pythonhosted.org/packages/05/ae/7c67fba23bd98cae7c99261f3a16072ade14813486b0282cb29846de832/numpy-2.4.6-cp313-cp313-musllinux\_1\_2\_aarch64.whl", hash = "sha256:ef4aea96ce4d3b074422cb4f2f64e2216bf9e213004bb58ecfdf50ea02ea8eb9a", size = 17020800, upload-time = "2026-05-18T23:34:49.065Z" },  
 { url = "https://files.pythonhosted.org/packages/d9/5d/3b6725cb31d983c5e66916f5d36f6d7e5521129e4c4404d64f918292a5b6/numpy-2.4.6-cp313-cp313-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:dfa20cc6ca228e6b155b11da03825975ce66aea520985dbbddd0f2a5a495c605", size = 18357600, upload-time = "2026-05-18T23:34:52.709Z" },  
 { url = "https://files.pythonhosted.org/packages/f7/da/2cc6c2fe8898de01d90c75c5f5f914a23daf99e3e0f59516a08760c8b5/numpy-2.4.6-cp313-cp313-win32.whl", hash = "sha256:56b39e5e0622a09a25bf5baf62f4bcf0cb8a41ae6e2819cf49bbbc5a74c083f91", size = 5961134, upload-time = "2026-05-18T23:34:55.618Z" },  
 { url = "https://files.pythonhosted.org/packages/b5/cd/9cc4dc876fb065d5c220aae4d5e14826b2715331bb7618ce1fb07a679d99/numpy-2.4.6-cp313-cp313-win\_amd64.whl", hash = "sha256:c4fc99836233ea196540b17ab0983aff60ed07941751930f5f4d05bc3b3b7359", size = 12318598, upload-time = "2026-05-18T23:34:58.928Z" },  
 { url = "https://files.pythonhosted.org/packages/39/1e/c0bcbaf18694116485fe28fd1be698c278fcd4141c5b0e53a2aed8b12a8/numpy-2.4.6-cp313-cp313-win\_arm64.whl", hash = "sha256:a7c711e21628b52034bb5ab8d1bce291f752fcc5e92accc615778acee1ff4778", size = 10222272, upload-time = "2026-05-18T23:35:02.167Z" },  
 { url = "https://files.pythonhosted.org/packages/63/6d/cc5619247c8f4204e507f5883528372e4ac4bb189e579fb859a12e480b1f/numpy-2.4.6-cp313-cp313-macosx\_11\_0\_arm64.whl", hash = "sha256:112b06a867b235ef466ed3508ddf0238050df9cf727cafb5301ac385b899189a1", size = 14821197, upload-time = "2026-05-18T23:35:05.468Z" },  
 { url = "https://files.pythonhosted.org/packages/00/58/f1c39161c87d9e9bed660f1ed4bafcf0e403d5ec9650b6dd77aead07d489b/numpy-2.4.6-cp313-cp313-macosx\_14\_0\_arm64.whl", hash = "sha256:eaf7fa2de5c0b8ae6ff8e9bea2ccd725e980541244521d8d4b5f3354a27babe", size = 5326287, upload-time = "2026-05-18T23:35:08.693Z" },  
 { url = "https://files.pythonhosted.org/packages/af/57/3917ab0fd97f271a8694513581b8a36c655f111c446852c302f04ccdb6fc/numpy-2.4.6-cp313-cp313-macosx\_14\_0\_x86\_64.whl", hash = "sha256:7265a2f3d436e54ef9f2b52b5c937e6be778781bd97a590319d7348f1c1ca997", size = 6646763, upload-time = "2026-05-18T23:35:11.459Z" },  
 { url = "https://files.pythonhosted.org/packages/eb/0f/037e64c494b67581ae18193d770adef354c41f3f2c8ebf865602d949bf8f/numpy-2.4.6-cp313-cp313-manylinux\_2\_27\_aarch64.manylinux\_2\_28\_aarch64.whl", hash = "sha256:f74a575920ab21fe304421a3fc28793d82e299cae9eccb37084e9fc7f3617c20", size = 15728070, upload-time = "2026-05-18T23:35:14.79Z" },  
 { url = "https://files.pythonhosted.org/packages/21/a6/5d2bae9c9542eb4df16dc9c46dc79c186e9bad53805dfa5399a6023c6db0/numpy-2.4.6-cp313-cp313-manylinux\_2\_27\_x86\_64.manylinux\_2\_28\_x86\_64.whl", hash = "sha256:ede83e07a75dd06bc501566c1eca2afcd0d61677c1472ac9ad93fdee6e638a48d", size = 16681752, upload-time = "2026-05-18T23:35:18.836Z" },  
 { url = "https://files.pythonhosted.org/packages/92/14/23d1dfb410ae362cd59ce53e936b1513d545eb40db3949ced632e19a459e/numpy-2.4.6-cp313-cp313-musllinux\_1\_2\_aarch64.whl", hash = "sha256:68bb27509ac1b9a3443094260f6326150663b06abe40b73a2f81160623da5b67", size = 17086024, upload-time = "2026-05-18T23:35:22.52Z" },

```
{ url = "https://files.pythonhosted.org/packages/4b/6e/23595a2c642cdf3bc567877064bdd7f91c8b0038a4453cf2daf7248eafe9
/numpy-2.4.6-cp313-cp313t-musllinux_1_2_x86_64.whl", hash = "sha256:a0df0043bdb289bde1f62da130d20df23d58b45429f752bc7a8
fc5325a225ecd", size = 18403398, upload-time = "2026-05-18T23:35:26.398Z" },
{ url = "https://files.pythonhosted.org/packages/8a/90/0ac3bc947217e66dec77e7cbc6a1979d1af70b6461b82f620d3bccd5e4c8
/numpy-2.4.6-cp313-cp313t-win32.whl", hash = "sha256:29a287e0cf63ff528da061de6b9f64a4618da591ca1046aafc54062e40ca7eab",
size = 6084971, upload-time = "2026-05-18T23:35:29.387Z" },
{ url = "https://files.pythonhosted.org/packages/77/71/5673e351671a1d2bd6063b91b44f70c0affea7d1516fa7a6572941ba4aa1
/numpy-2.4.6-cp313-cp313t-win_amd64.whl", hash = "sha256:25c692919ac5a0f1f70a3bfc6d2745b24fd095c353d50812637d6fcab442e
75", size = 12458532, upload-time = "2026-05-18T23:35:32.175Z" },
{ url = "https://files.pythonhosted.org/packages/3f/88/19d3503c5046e688f049274b27a3ef3d771152fa80d3ba3d01a3dfff61abe
/numpy-2.4.6-cp313-cp313t-win_arm64.whl", hash = "sha256:1e978ec1e8bd0e0e4de6bb75de9d30cbb74db6b6a2bb727618613703ca0167
dd", size = 10291881, upload-time = "2026-05-18T23:35:35.465Z" },
{ url = "https://files.pythonhosted.org/packages/f8/91/3ab2044d05fd16d343c5ac2e69b127f1b2854040dd20b193257c78028bd3
/numpy-2.4.6-cp314-cp314-macosx_10_15_x86_64.whl", hash = "sha256:06ca2f61ec4385a07a6977c55ba998a4466c123642ba3a32694d31
28fce18c079", size = 16683458, upload-time = "2026-05-18T23:35:38.353Z" },
{ url = "https://files.pythonhosted.org/packages/8e/62/764ce66fa4147ae6d73071a3abf804ffe06f174618697c571acd726a7c9
/numpy-2.4.6-cp314-cp314-macosx_11_0_arm64.whl", hash = "sha256:38efbc8de75c7a0fc1ac190162d892787f3f47b57cc291231aafec3
6b80982b7", size = 14704559, upload-time = "2026-05-18T23:35:42.14Z" },
{ url = "https://files.pythonhosted.org/packages/60/61/23f27c172f022e04025b7dc2367f4d63c1a398120607ec896228649a6f48
/numpy-2.4.6-cp314-cp314-macosx_14_0_arm64.whl", hash = "sha256:d581b735e177fddcde6fed8e7e8880a3fb6ee4e3653a3ac6af01c6f
4c03effc5", size = 5209716, upload-time = "2026-05-18T23:35:45.377Z" },
{ url = "https://files.pythonhosted.org/packages/03/71/21cf70dc6ea3e3ac95fc53a265b2fc248b981f0194ceb5b475271b8809d
/numpy-2.4.6-cp314-cp314-macosx_14_0_x86_64.whl", hash = "sha256:0a041d3d761dc3c35cc56ce0351506a02bcb25f7b169f65243514
1a17db9096", size = 6543947, upload-time = "2026-05-18T23:35:47.926Z" },
{ url = "https://files.pythonhosted.org/packages/d5/91/64288395ee1799bd2e0b04a305dce966da90c961e1f3fe982a05ee1c036
/numpy-2.4.6-cp314-cp314-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:40fdc1ae7125e518ea98e53e69a
4ebc27e1fd50510c47b7ea130cf21e5e1d42b", size = 15685197, upload-time = "2026-05-18T23:35:50.863Z" },
{ url = "https://files.pythonhosted.org/packages/f3/eb/ebffaa97dc55502df69584a8f0dcf07f69a3e0b3e2323670a2722db9aa39
/numpy-2.4.6-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:a2c306dea656c12c68f51f4cea133
cbe78ca7435eb28c735eac1d3ebe73be6e8", size = 16638245, upload-time = "2026-05-18T23:35:54.752Z" },
{ url = "https://files.pythonhosted.org/packages/b8/0b/54f9da33128d7e350fab89c7455902eeae70349ee52bddd448dc4a576f45
/numpy-2.4.6-cp314-cp314-musllinux_1_2_aarch64.whl", hash = "sha256:33111801a01c12a8a1e3721f0a9232f8cfc8ae2c6b7098167e6
f623c6073f402", size = 17036587, upload-time = "2026-05-18T23:35:58.353Z" },
{ url = "https://files.pythonhosted.org/packages/b6/f0/fdebc1052db1cc37c64beb22072d67cd6d1c71adca1299f53dec2b5e20d3
/numpy-2.4.6-cp314-cp314-musllinux_1_2_x86_64.whl", hash = "sha256:ae506e902902557576a26ff33eda8695e7ecb3cb36c3b573a07
65dee114ebdb", size = 18363226, upload-time = "2026-05-18T23:36:02.845Z" },
{ url = "https://files.pythonhosted.org/packages/aa/b4/298628d98c72b57e57f7165ae6a481a1deaf6f3c28262a6e4c739c275930
/numpy-2.4.6-cp314-cp314-win32.whl", hash = "sha256:aaf159caa35993cb1f56fb9b8e4610d35758e7ca005412eb1daa856a78c9c4b1",
size = 6010196, upload-time = "2026-05-18T23:36:05.92Z" },
{ url = "https://files.pythonhosted.org/packages/df/ac/46de6dda46478f7942f839e094970be2d4a861e005c4b3bf07c92e291a09
/numpy-2.4.6-cp314-cp314-win_amd64.whl", hash = "sha256:b507f5c4c1d508876d1819b6bf9a49d365b96320b5d4993426b33a23ca4b826
1", size = 12450334, upload-time = "2026-05-18T23:36:09.107Z" },
{ url = "https://files.pythonhosted.org/packages/78/92/b8b798ac784102c0da830d2257d59358e3d3d90d1e2b3f2575dad976c5cf
/numpy-2.4.6-cp314-cp314-win_arm64.whl", hash = "sha256:6f41ae150c4e32db4f3310cdaf64b1593a03dbabe29eec77fc9b50fe64061df
6", size = 10495678, upload-time = "2026-05-18T23:36:12.766Z" },
{ url = "https://files.pythonhosted.org/packages/30/34/ec28d1aa8115971537c01469ab2011ee96827930f0a124de1000cc2a7ed7
/numpy-2.4.6-cp314-cp314t-macosx_11_0_arm64.whl", hash = "sha256:ec3d2cfe132e7d51f44a832b303895e6f2d499c5e74dfdbd06ee2
46147a304a", size = 14823672, upload-time = "2026-05-18T23:36:16.473Z" },
{ url = "https://files.pythonhosted.org/packages/16/bd/f6d1fede4e54e8042a7ff97bb495510f3c220f94bcd9e8b228e87c92cc0d
/numpy-2.4.6-cp314-cp314t-macosx_14_0_arm64.whl", hash = "sha256:e3e5193ef5a3dc73bccee507f7dc2c90dbb76c42df8d8fae3d1067
a583df579e", size = 5328731, upload-time = "2026-05-18T23:36:19.767Z" },
{ url = "https://files.pythonhosted.org/packages/f4/f0/e105b9e2fd728a9910103884dec6951d9dd73896b914a98d9a231de02ee
/numpy-2.4.6-cp314-cp314t-macosx_14_0_x86_64.whl", hash = "sha256:17f9ade344e7d9b464a084d69bcf18fc691cb1db67c62ed80820b
f4926d78f0e", size = 6649805, upload-time = "2026-05-18T23:36:22.266Z" },
{ url = "https://files.pythonhosted.org/packages/82/dd/1206a7ca6ab15e3f02069707ca96222e202af681bb73756da7527f3cb837
/numpy-2.4.6-cp314-cp314t-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:9cd5ff25db4e7ba6a375693b3
fc0fc1791ec636c17db3720da19bde7180ec43", size = 15730496, upload-time = "2026-05-18T23:36:25.713Z" },
{ url = "https://files.pythonhosted.org/packages/51/e7/38d3ea825dcab85a591734dec2f6c67caa7c8367d374df1a1c3842f9b07
/numpy-2.4.6-cp314-cp314t-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:7d92c3819208a60205a12a245c91
ad70cb0a85336659b19b834205573ac8456e", size = 16679616, upload-time = "2026-05-18T23:36:29.652Z" },
{ url = "https://files.pythonhosted.org/packages/93/b7/caabfd53edf663e0b4eb74d7d405d83baef09eb5e83bcd32d601d72b93e
/numpy-2.4.6-cp314-cp314t-musllinux_1_2_aarch64.whl", hash = "sha256:e85b752a1e912b70eaad4fafbd4d1238007ab221de2009b9a2
f5aef7461239895", size = 17085145, upload-time = "2026-05-18T23:36:33.449Z" },
{ url = "https://files.pythonhosted.org/packages/f9/45/68d7c33a6bcf3e5aa3bdbd57a367e6f615286dfd6482f97e8ffeb734306e
/numpy-2.4.6-cp314-cp314t-musllinux_1_2_x86_64.whl", hash = "sha256:29cb7f67d10b479ff07c17d33e39f78c07f71c40ef30d63c153
d340e96cd3fb4", size = 18403813, upload-time = "2026-05-18T23:36:37.369Z" },
{ url = "https://files.pythonhosted.org/packages/9c/50/0753655aa844c99cd9e018aac76f130f1bd81d881bb74bc0aef5d73a8ba
/numpy-2.4.6-cp314-cp314t-win32.whl", hash = "sha256:260a5d70215b61ab4fadf5c7baacd64821842975eea312125ed3c39a6391b063",
size = 6156982, upload-time = "2026-05-18T23:36:40.817Z" },
{ url = "https://files.pythonhosted.org/packages/b2/d4/7c67becf668f973cb490cec3e98dfd799d866f9c989a54d355672cfa0db6
/numpy-2.4.6-cp314-cp314t-win_amd64.whl", hash = "sha256:81a1cca95ed5bb92aa8b10dd2cdc9a0d3853a50fad926c28b5d7e8ea543896
27", size = 12638908, upload-time = "2026-05-18T23:36:43.996Z" },
{ url = "https://files.pythonhosted.org/packages/43/bb/e1c71a4295b1b1d1393d50dbb4f2a36283c6859d9d3892e84f00ec5a91d5
/numpy-2.4.6-cp314-cp314t-win_arm64.whl", hash = "sha256:0c9136e14ed34a9e343a31c533d78a9813a69a3148332bce5e9821cb2f996e
66", size = 10565867, upload-time = "2026-05-18T23:36:47.114Z" },
]
```

```

[[package]]
name = "packaging"
version = "26.2"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/d7/f1/e7a6dd94a8d4a5626c03e4e99c87f241ba9e350cd9e6d75123f992427270/packaging-26.2.tar.gz", hash = "sha256:ff452ff5a3e828ce110190feff1178bb1f2ea2281fa2075aad987c2fb221661", size = 228134, upload-time = "2026-04-24T20:15:23.917Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/df/b2/87e62e8c3e2f4b32e5fe99e0b86d576da1312593b39f47d8ceef365e95ed/packaging-26.2-py3-none-any.whl", hash = "sha256:5fc45236b9446107ff2415ce77c807cee2862cb6fac22b8a73826d0693b0980e", size = 100195, upload-time = "2026-04-24T20:15:22.081Z" },
]

[[package]]
name = "pandas"
version = "3.0.3"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "numpy" },
 { name = "python-dateutil" },
 { name = "tzdata", marker = "sys_platform == 'emscripten' or sys_platform == 'win32'" },
]
sdist = { url = "https://files.pythonhosted.org/packages/f8/87/4341c6252d1c47b08768c3d25ac487362bf403f0313ddae4a2a26c9b1b4c/pandas-3.0.3.tar.gz", hash = "sha256:696a4a00a2a2a35d4e5deb3fc946641b96c944f02230e4f76137fe35d806c4fc", size = 4651414, upload-time = "2026-05-11T18:54:29.212Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/24/f1/392f8c5bfc16f66a0d2d41561c01627c228fe7ed2a0d056ef11315042570/pandas-3.0.3-cp312-cp312-macosx_10_13_x86_64.whl", hash = "sha256:fed2ff7fd9779120e388e285fc029bd5cf9490cdd2e4166a9ee22c0e49a9ab09", size = 10357846, upload-time = "2026-05-11T18:52:36.143Z" },
 { url = "https://files.pythonhosted.org/packages/cf/3d/b16412745651e855f357e5e66930248688378853a6e2698a214e331fba1f/pandas-3.0.3-cp312-cp312-macosx_11_0_arm64.whl", hash = "sha256:b168fc218fd80a6cbdbdbc1a97ddc7889ed057d7eb45f50d866cea5b5f39904c4", size = 9899550, upload-time = "2026-05-11T18:52:38.976Z" },
 { url = "https://files.pythonhosted.org/packages/31/a8/fa2535168fffcdf67f4f6de28d2dd903a747ca7c8ea6989451aaeb3a92f/pandas-3.0.3-cp312-cp312-manylinux_2_24_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:0383c72c75cdcca61a9e116e61143902dbfd08bfff356829c2f6d1cf40a9ca8c", size = 10412965, upload-time = "2026-05-11T18:52:41.915Z" },
 { url = "https://files.pythonhosted.org/packages/65/b6/09b01cdbc15224e2850365192d17b7bdebb8b8dbd8780ed221fcd0d9a515/pandas-3.0.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:6dc0b3fd2169c9157deed50b4d519553a3655c8c6a96027136d654592be973a9", size = 10894600, upload-time = "2026-05-11T18:52:45.022Z" },
 { url = "https://files.pythonhosted.org/packages/c9/a4/2eb28f2fccb4ced4a2c79ab2a5dee9ade1ebf44922ebad6fea158c9f95d4/pandas-3.0.3-cp312-cp312-musllinux_1_2_aarch64.whl", hash = "sha256:7e65d5407dc0b394f509699650e4a2ec01c0514f21850f453fa60f3be79a5dbf", size = 11422824, upload-time = "2026-05-11T18:52:48.058Z" },
 { url = "https://files.pythonhosted.org/packages/f8/45/830bb57f533a4604b355e07edcb8ea18cf88b5f94e5fca92f27052d7c597/pandas-3.0.3-cp312-cp312-musllinux_1_2_x86_64.whl", hash = "sha256:557409bcb4178e70ee8d9ddb494798e51ebf6ea59330f6be22c51bab2a7db6c762c5d79d305c", size = 11950889, upload-time = "2026-05-11T18:52:50.905Z" },
 { url = "https://files.pythonhosted.org/packages/b9/c5/fc1b368f303087d20e8c9bf3d6ceb186263cfac0ade735cd938538bea839/pandas-3.0.3-cp312-cp312-win_amd64.whl", hash = "sha256:c7be265b62cef88e253a941e4698604973736dcfe242fdb5198f0f7bc473cdcc", size = 9755463, upload-time = "2026-05-11T18:52:53.386Z" },
 { url = "https://files.pythonhosted.org/packages/86/bd/fda8f9705b1b09c6ebe14bfc0fa0e4ec8584d54ea673628f157ff55131af/pandas-3.0.3-cp312-cp312-win_arm64.whl", hash = "sha256:557409bcb4178e70ee8d9ddb494798e51ebf6ea59330f6be22c51bab2a7db6c49", size = 9066158, upload-time = "2026-05-11T18:52:56.038Z" },
 { url = "https://files.pythonhosted.org/packages/c5/90/62d8302883c44308c477e222c3daf7c813a34c8e96985882fbd53d964352/pandas-3.0.3-cp313-cp313-macosx_10_13_x86_64.whl", hash = "sha256:67b3b64c11910cfa29f4e94a14d3bfff9ee693b6fc76055e7cad549cee0aee5fa", size = 10331071, upload-time = "2026-05-11T18:52:58.838Z" },
 { url = "https://files.pythonhosted.org/packages/7f/ae/6a6493c783a101f165e4356953ba3c74d6f77f0042fa7d753da9dfbb640c/pandas-3.0.3-cp313-cp313-macosx_11_0_arm64.whl", hash = "sha256:39436b377d56d2a2e52d0395bdbbee171f01068e99af5250509aceeb929f765c7", size = 9875690, upload-time = "2026-05-11T18:53:01.431Z" },
 { url = "https://files.pythonhosted.org/packages/62/7c/5df8e9f56c69a2769f9e9382a5ef8f2658c007e376434e1e2cbb57ad895f/pandas-3.0.3-cp313-cp313-manylinux_2_24_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:d4be06d68f9ddcfc645b87534911da79a8fbfffc7573c80e0edcf42a5020624d8", size = 10381634, upload-time = "2026-05-11T18:53:04.393Z" },
 { url = "https://files.pythonhosted.org/packages/99/68/1237369725aa617bb358263d535803e3053fdb593513ec5ed9c9896b5b6/pandas-3.0.3-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:a4eeb6830daf35a71cc09649bd823e2b542dac246cdee9614c6e4bd65028cd6a", size = 10891243, upload-time = "2026-05-11T18:53:07.643Z" },
 { url = "https://files.pythonhosted.org/packages/25/93/77d108e8af7222b4a503ebde0e30215b1c2e4f8e53a526431890f22d5586/pandas-3.0.3-cp313-cp313-musllinux_1_2_aarch64.whl", hash = "sha256:1928e07221f82db493cd4af1e23c1bfc5a524a19a4699887975bfff68f49a72bfb", size = 11388659, upload-time = "2026-05-11T18:53:10.634Z" },
 { url = "https://files.pythonhosted.org/packages/d0/bd/eff5b4399f332ac386c853f6cd2bd3fa2ca0061b9f36ecd9c4d7c4265649/pandas-3.0.3-cp313-cp313-musllinux_1_2_x86_64.whl", hash = "sha256:51b1fe551acb77dac643c6fda86084d8d446c10fe64b06a9cc29c4cc8540e7f2", size = 11942880, upload-time = "2026-05-11T18:53:13.536Z" },
 { url = "https://files.pythonhosted.org/packages/2c/20/559ace4200982c3887d0b86bfd0d856a2143ef8ddab63cc07934951a964c/pandas-3.0.3-cp313-cp313-win_amd64.whl", hash = "sha256:a82d532a3351d435432cd913edbccaf8b8e01d4dd0e5ced5a8d2e8ecd94c7e44", size = 9757091, upload-time = "2026-05-11T18:53:16.306Z" },
 { url = "https://files.pythonhosted.org/packages/3a/66/69055a09fe200f29f922a3eeec4804611900b95f52d932ece3393c3c0c19/pandas-3.0.3-cp313-cp313-win_arm64.whl", hash = "sha256:275c14e0fce14a2ec20eee474aeed305478ea3c1e6f6a9d8fe219a165542717e", size = 9057282, upload-time = "2026-05-11T18:53:18.768Z" },
 { url = "https://files.pythonhosted.org/packages/57/0e/efe801b0e6811e8e650cd21b7f2608e30f08a7067e2bf6e8752b0d56ee3c

```

```
/pandas-3.0.3-cp313-cp313t-macosx_10_13_x86_64.whl", hash = "sha256:46997386d528eb40376ecd6b033cf4a8a1e5282580f68f43de875b78cba2199d", size = 10767016, upload-time = "2026-05-11T18:53:21.227Z" },
{ url = "https://files.pythonhosted.org/packages/ea/dc/eb55135a1d5f0f9519f28da1f609a206d2cad1f9c35c32d51e38dd7261ae/pandas-3.0.3-cp313-cp313t-macosx_11_0_arm64.whl", hash = "sha256:261e308dfb22448384b7580cf719d2f998fe2966c92893c3e77d14008af1f066", size = 10420210, upload-time = "2026-05-11T18:53:23.982Z" },
{ url = "https://files.pythonhosted.org/packages/c6/3e/b1d5d955ce33ffecb407465a60bc32769d74fcf68224b7ae67ae11d4dea4/pandas-3.0.3-cp313-cp313t-manylinux_2_24_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:dd1a5d1def6a46002e964510bd6c67c368aa0951df5d1d9f8365336f5a1f490cd", size = 10336126, upload-time = "2026-05-11T18:53:26.731Z" },
{ url = "https://files.pythonhosted.org/packages/f5/76/a01261711ab60a22d71b862f0de20e4c504bf80457270ad8cb42110f6abc/pandas-3.0.3-cp313-cp313t-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:d72828c20c6d6e83e1e22a6a3b47b326b71664112fa9705dcbccfd7a39b62085", size = 10728051, upload-time = "2026-05-11T18:53:29.125Z" },
{ url = "https://files.pythonhosted.org/packages/e9/21/ea191195e587b18cf682e97f433f81b2d0f9e341380e80a3e0d6e4403c8e/pandas-3.0.3-cp313-cp313t-musllinux_1_2_aarch64.whl", hash = "sha256:d26cbe1fcfc12e8fd900e2454163e466b2d3af84f7c75481df7683ffc073d870", size = 11350796, upload-time = "2026-05-11T18:53:32.056Z" },
{ url = "https://files.pythonhosted.org/packages/64/69/f0eaf54939f0e8c6768fd06be9af2cef9b36048b96dfb9e1b2c685a807e/pandas-3.0.3-cp313-cp313t-musllinux_1_2_x86_64.whl", hash = "sha256:3e91cec1879ada0624fc3dc9953c5cbd60208e59c0db28f540c5d6d47502422f", size = 11799741, upload-time = "2026-05-11T18:53:34.985Z" },
{ url = "https://files.pythonhosted.org/packages/45/a4/865e0e510cae5fc2194de4db28be638952de942571ba9125934fd9c01d47/pandas-3.0.3-cp313-cp313t-win_amd64.whl", hash = "sha256:08d789b41f87e0905880e293cedf6197ce71fe67cc081358b1e148a491b9bd13", size = 10499958, upload-time = "2026-05-11T18:53:37.857Z" },
{ url = "https://files.pythonhosted.org/packages/86/54/effdcc30fff7a08037889200e148ebe94c16c4f653be078c7b3675955df1/pandas-3.0.3-cp314-cp314-macosx_10_15_x86_64.whl", hash = "sha256:3650109c0f22879df8bd6179ab9ee3d7f1d1d4e7e0094a3f0032d9f51e2e64ac", size = 10336065, upload-time = "2026-05-11T18:53:41.099Z" },
{ url = "https://files.pythonhosted.org/packages/68/10/bf2d6738d72748b961a3751ab89522d58c54efc36a8e1a12161216cd45cf/pandas-3.0.3-cp314-cp314-macosx_11_0_arm64.whl", hash = "sha256:bab900348131a7db1f69a7309ef141fd5680f1487094193bcbbbb61791573bf8f", size = 9926101, upload-time = "2026-05-11T18:53:43.515Z" },
{ url = "https://files.pythonhosted.org/packages/ae/e9/e35cf11c8a136e757b956f5f0efddca50aecde85ea055f1898dfc68262f3/pandas-3.0.3-cp314-cp314-manylinux_2_24_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:ba7e08b9ac1d54569cd1e256e3668975ed624d6826f7b68df0342b012007bddb", size = 10457553, upload-time = "2026-05-11T18:53:46.394Z" },
{ url = "https://files.pythonhosted.org/packages/58/3b/1cdec6772bdbaf7b25dab360c59f03cadf05492dd724c6540af905389b07/pandas-3.0.3-cp314-cp314-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:9d71c63ae4ebdbf70209742096f1fc46a83a0613c99d4b23766cced9ff8cd62a", size = 10914065, upload-time = "2026-05-11T18:53:49.134Z" },
{ url = "https://files.pythonhosted.org/packages/c4/c2/1ef64445fcd72e3627bceec77e3560636f87ddce4ed841afe76b83b5bf9/pandas-3.0.3-cp314-cp314-musllinux_1_2_aarch64.whl", hash = "sha256:e3a2ec42c98ffa2565a67e08e218d06d72576d758d90facb7c00805194d8f360", size = 11459188, upload-time = "2026-05-11T18:53:52.527Z" },
{ url = "https://files.pythonhosted.org/packages/7e/49/4d8d4f42c9c9c4adc7a1870f269c02cbd6cd40d059622c06fb298addcbad/pandas-3.0.3-cp314-cp314-musllinux_1_2_x86_64.whl", hash = "sha256:335f62418ed562cfc3c49e9e196375c28b729dcecf8543abf4f9438e381bf3c76", size = 11982966, upload-time = "2026-05-11T18:53:55.043Z" },
{ url = "https://files.pythonhosted.org/packages/38/55/792619469bab9882d8bbd5865d45a72f6478762d04a9af4bf0d08c503e95/pandas-3.0.3-cp314-cp314-win_amd64.whl", hash = "sha256:3c20a521bbb85902f79f7270c80a59e1b5452d96d170c034f207181870f97ac5", size = 9876755, upload-time = "2026-05-11T18:53:58.067Z" },
{ url = "https://files.pythonhosted.org/packages/2a/af/33c469653b0ba03b50c3a98192d4c07f0c75c66b263ceb097fce0ee97d31/pandas-3.0.3-cp314-cp314-win_arm64.whl", hash = "sha256:a2d2dff8a04f3917b55ab3910c32990f8ddf7eacea114947838cefa976a68977", size = 9198658, upload-time = "2026-05-11T18:54:00.733Z" },
{ url = "https://files.pythonhosted.org/packages/a2/fa/b8c257bd76b8bd060c3a9151c1fca05e9b9c5e3af5d0f549c0356f6d143d/pandas-3.0.3-cp314-cp314t-macosx_10_15_x86_64.whl", hash = "sha256:0d589105b3c14645af1738ff279b2995102d8f7a03b0a66dc8d95550eb513e04", size = 10787242, upload-time = "2026-05-11T18:54:03.564Z" },
{ url = "https://files.pythonhosted.org/packages/54/eb/f19206ffb0bf1919002969aa448b4702c6594845156a6f8050674855aac3/pandas-3.0.3-cp314-cp314t-macosx_11_0_arm64.whl", hash = "sha256:13fc1e853d9e04743d11ba75a985ccbc2a317fe07d8af61e445a6fd24dacd6a6", size = 10436369, upload-time = "2026-05-11T18:54:06.311Z" },
{ url = "https://files.pythonhosted.org/packages/fd/24/c7c39fb4fe22b71a0c2d78bf0c585c600092d85f94f086d2b3b2f6ca27e2/pandas-3.0.3-cp314-cp314t-manylinux_2_24_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:819959dab7bbdb049c15623fbac4e29a191b9528160a61fb1032242d8ced2d9c", size = 10358306, upload-time = "2026-05-11T18:54:09.085Z" },
{ url = "https://files.pythonhosted.org/packages/16/ec/dd2a9eb7fa1204df88c0864164e35b228ac581062ac612ba0a67fd812e4c/pandas-3.0.3-cp314-cp314t-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:60ae316d3fd75d1858d450d0db0103ea2be3e7d4a95ec2f064f7e2ae63f7b028", size = 10758394, upload-time = "2026-05-11T18:54:11.956Z" },
{ url = "https://files.pythonhosted.org/packages/95/6e/00c61ea8e85b4f6d8d35e11852a1a4998fc7fafc91c6a602d1cc9c972d64/pandas-3.0.3-cp314-cp314t-musllinux_1_2_aarch64.whl", hash = "sha256:bd3a518890b400d32f9023722dc9a9a5c969f00b415419a3c06c043f09bb5d7d", size = 11375717, upload-time = "2026-05-11T18:54:14.539Z" },
{ url = "https://files.pythonhosted.org/packages/31/89/8f1c268969fac43688d65fd92e67df24bd128d53cb4d2eee534cd307399/pandas-3.0.3-cp314-cp314t-musllinux_1_2_x86_64.whl", hash = "sha256:9c39be2d709d01fa972a0cabcc522389fcce4f3969332ba25a7d6c5802cf976a", size = 11828897, upload-time = "2026-05-11T18:54:17.146Z" },
{ url = "https://files.pythonhosted.org/packages/56/3b/e7d20dea247a3e6dc0bd8a6953854afbedc03951def4e7371e05e7263e25/pandas-3.0.3-cp314-cp314t-win_amd64.whl", hash = "sha256:4db8c527972a821cf5286b40ccc57642a39bc62e62022b42f99f8a67fca8c3a1", size = 10900855, upload-time = "2026-05-11T18:54:19.722Z" },
{ url = "https://files.pythonhosted.org/packages/0f/54/68a0978d1ef8502b8492099beaa6e7a0c1b32e3b5d4f677f5810cb08711c/pandas-3.0.3-cp314-cp314t-win_arm64.whl", hash = "sha256:b2c95f8bfc1ee412bf482605d7bfd30c12d1d26bd59fdd91efeef1d4718decbl", size = 9466464, upload-time = "2026-05-11T18:54:22.754Z" },
]
```

```
[[package]]
name = "pdfminer-six"
version = "20260107"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "charset-normalizer" },
```

```

 { name = "cryptography" },
]
sdist = { url = "https://files.pythonhosted.org/packages/34/a4/5cec1112009f0439a5ca6afa8ace321f0ab2f48da3255b7a1c895301
4670/pdfminer_six-20260107.tar.gz", hash = "sha256:96bfd431e3577a55a0efd25676968ca4ce8fd5b53f14565f85716fff363889602", s
ize = 8512094, upload-time = "2026-01-07T13:29:12.937Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/20/8b/28c4eae9d6b036a52cb44720408f26b1a143ca9bce76cc19e8f5de00ab4
/pdfminer_six-20260107-py3-none-any.whl", hash = "sha256:366585ba97e80dffaf8f00cebe303d2f381884d8637af4ce422f1df3ef38111
a9", size = 6592252, upload-time = "2026-01-07T13:29:10.742Z" },
]

[[package]]
name = "pdfplumber"
version = "0.11.10"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "pdfminer-six" },
 { name = "pillow" },
 { name = "pypdfium2" },
]
sdist = { url = "https://files.pythonhosted.org/packages/05/56/6f450312ba05a27d7713b73857c1a25100dbda04fbc1331b13fb227a
607d/pdfplumber-0.11.10.tar.gz", hash = "sha256:b95b2d28c66efb0a794a83b88c6c6aea5987532a445d20a1cbbcfa657022e6e57", size
= 102892, upload-time = "2026-06-15T03:31:31.035Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/a2/9a/07d658e1e7fad860f1c541ab941348125dbdab773be3a0afaf32361866c7
/pdfplumber-0.11.10-py3-none-any.whl", hash = "sha256:7741ea81bf165b474b153e6789d10d18e06b6ddcf3ec84289c3ef2fed6802580"
, size = 60047, upload-time = "2026-06-15T03:31:29.702Z" },
]

[[package]]
name = "pillow"
version = "12.2.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/8c/21/c2bccd5906101a30244eaffc1b6e6ce71a31bd0742a01eb89e660ebf
ac2d/pillow-12.2.0.tar.gz", hash = "sha256:a830b1a40919539d07806aa58e1b114df53ddd43213d9c8b75847eee6c0182b5", size = 46
987819, upload-time = "2026-04-01T14:46:17.687Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/58/be/7482c8a5ebbbbc6470b3eb791812fff7d5e0216c2be3827b30b8bb6603ed
/pillow-12.2.0-cp312-cp312-macosx_10_13_x86_64.whl", hash = "sha256:2d192a155bbcec180f8564f693e6fd9bcccff5a7af9b32e24bf
8c9c69dbad6b5", size = 5308279, upload-time = "2026-04-01T14:43:13.246Z" },
 { url = "https://files.pythonhosted.org/packages/d8/95/0a351b9289c2b5cbde0bacd4a83ebc44023e835490a727b2a3bd60ddc0f4
/pillow-12.2.0-cp312-cp312-macosx_11_0_arm64.whl", hash = "sha256:f3f40b3c5a968281fd507d519e444c35f0fff171237f4fdde090dd
60699458421", size = 4695490, upload-time = "2026-04-01T14:43:15.584Z" },
 { url = "https://files.pythonhosted.org/packages/de/af/4e8e6869cbcd569d43c416fad3dc4ecb944cb5d9492defaed89ddd6fe871
/pillow-12.2.0-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch64.whl", hash = "sha256:03e7e372d5240cc23e9f07deca
4d775c0817bffc641b01e9c3af208dbd300987", size = 6284462, upload-time = "2026-04-01T14:43:18.268Z" },
 { url = "https://files.pythonhosted.org/packages/e9/9e/c05e19657fd57841e476be1ab46c4d501bffbadafbdc31a6d665f8b737b6
/pillow-12.2.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl", hash = "sha256:b86024e52a1b269467a802258c25
521e6d742349d760728092e1bc2d135b4d76", size = 8094744, upload-time = "2026-04-01T14:43:20.716Z" },
 { url = "https://files.pythonhosted.org/packages/2b/54/1789c455ed10176066b6e7e6da1b01e50e36f94ba584dc68d9eebf9156d
/pillow-12.2.0-cp312-cp312-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:7371b48c4fa448d20d2714c9a
1f775a81155050d383333e0a6c15b1123dda005", size = 6398371, upload-time = "2026-04-01T14:43:23.443Z" },
 { url = "https://files.pythonhosted.org/packages/43/e3/fdc657359e919462369869f1c9f0e973f353f9a9ee295a39b1fea8ee1a77
/pillow-12.2.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl", hash = "sha256:62f5409336adb0663b7caa0da5c
7d9e7bdbaae9ce761d34669420c2a801b2780", size = 7087215, upload-time = "2026-04-01T14:43:26.758Z" },
 { url = "https://files.pythonhosted.org/packages/8b/f8/2f6825e441d5b1959d2ca5adec984210f1ec086435b0ed5f52c19b3b8a6e
/pillow-12.2.0-cp312-cp312-musllinux_1_2_aarch64.whl", hash = "sha256:01afa7cf67f74f09523699b4e88c73fb55c13346d212a59a2
db1f86b0a63e8c5", size = 6509783, upload-time = "2026-04-01T14:43:29.56Z" },
 { url = "https://files.pythonhosted.org/packages/67/f9/029a27095ad20f854f9dba026b3ea6428548316e057e6fc3545409e86651
/pillow-12.2.0-cp312-cp312-musllinux_1_2_x86_64.whl", hash = "sha256:fc3d34d4a8fbec3e88a79b92e5465e0f9b842b628675850d86
0b8bd300b159f5", size = 7212112, upload-time = "2026-04-01T14:43:32.091Z" },
 { url = "https://files.pythonhosted.org/packages/be/42/025cfe05d1be22dbfdb4f264fe9de1ccda83f66e4fc3aac94748e784af04
/pillow-12.2.0-cp312-cp312-win32.whl", hash = "sha256:58f62cc0f00fd29e64b29f4fd923ffdb3859c9f9e6105bfc37ba1d08994e8940"
, size = 6378489, upload-time = "2026-04-01T14:43:34.601Z" },
 { url = "https://files.pythonhosted.org/packages/5d/7b/25a221d2c761c6a8ae21bfa3874988ff2583e19cf8a27bf2fee358df7942
/pillow-12.2.0-cp312-cp312-win_amd64.whl", hash = "sha256:7f84204dee22a783350679a0333981df803dac21a0190d706a50475e361c9
3f5", size = 7084129, upload-time = "2026-04-01T14:43:37.213Z" },
 { url = "https://files.pythonhosted.org/packages/0f/98/f3a6657ecb698c937f6c76ee564882945f29b79bad496abcba0e84659ec5
/pillow-12.2.0-cp312-cp312-win_arm64.whl", hash = "sha256:af73337013e0b3b46f175e79492d96845b16126ddf79c438d7ea7ff27783a
414", size = 2463612, upload-time = "2026-04-01T14:43:39.421Z" },
 { url = "https://files.pythonhosted.org/packages/4a/01/53d10cf0dbad820a8db274d259a37ba50b88b24768ddccccc07355382d5ad
d9b2d31ca61267f4c", size = 4100837, upload-time = "2026-04-01T14:43:41.506Z" },
 { url = "https://files.pythonhosted.org/packages/0f/98/f3a6657ecb698c937f6c76ee564882945f29b79bad496abcba0e84659ec5
/pillow-12.2.0-cp313-cp313-ios_13_0_arm64_iphoneos.whl", hash = "sha256:8297651f5b5679c19968abefd6bb84d95fe30ef712eb1b2
d9b2d31ca61267f4c", size = 4100837, upload-time = "2026-04-01T14:43:41.506Z" },
 { url = "https://files.pythonhosted.org/packages/0f/98/f3a6657ecb698c937f6c76ee564882945f29b79bad496abcba0e84659ec5
/pillow-12.2.0-cp313-cp313-ios_13_0_arm64_iphonesimulator.whl", hash = "sha256:50d8520da2a6ce0af445fa6d648c4273c3eeefbc
32d7ce049f22e8b5c3daecc2", size = 4176528, upload-time = "2026-04-01T14:43:43.773Z" },
]

```

{ url = "https://files.pythonhosted.org/packages/69/bc/8986948f05e3ea490b8442ea1c1d4d990b24a7e43d8a51b2c7d8b1dced36/pillow-12.2.0-cp313-cp313-ios\_13\_0\_x86\_64\_iphonesimulator.whl", hash = "sha256:766cef22385fa1091258ad7e6216792b156dc16d8d3fa607e7545b2b72061f1c", size = 3640401, upload-time = "2026-04-01T14:43:45.87Z" },

{ url = "https://files.pythonhosted.org/packages/34/46/6c717baadcd62bc8ed51d238d521ab651eaa74838291bda1f86fe1f864c9/pillow-12.2.0-cp313-cp313-macosx\_10\_13\_x86\_64.whl", hash = "sha256:5d2fd0fa6b5d9d1de415060363433f28da8b1526c1c129020435e186794b3795", size = 5308094, upload-time = "2026-04-01T14:43:48.438Z" },

{ url = "https://files.pythonhosted.org/packages/71/43/905a14a8b17fdb1ccbc58d282454490662d2cb89a6bfec26af6d3520da5ec/pillow-12.2.0-cp313-cp313-macosx\_11\_0\_arm64.whl", hash = "sha256:56b25336f502b6ed02e889f4ece894a72612fe885889a6e8c4c80239ff6e5f5f", size = 4695402, upload-time = "2026-04-01T14:43:51.292Z" },

{ url = "https://files.pythonhosted.org/packages/73/dd/42107efcb777b16fa0393317eac58f5b5cf30e8392e266e76e51cff28c3d/pillow-12.2.0-cp313-cp313-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.whl", hash = "sha256:f1c943e96e85df3d3478f7b691f229887e143f81fedab9b20205349ab04d73ed", size = 6280005, upload-time = "2026-04-01T14:43:54.242Z" },

{ url = "https://files.pythonhosted.org/packages/a8/68/b93e09e5e8549019e61acfa49f65b1a8530765a7f812c77a7461bca7e4494/pillow-12.2.0-cp313-cp313-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.whl", hash = "sha256:03f6fab9219220f041c74aeaa2939ff062bd5c364ba9ce037197f4c6d498cd9", size = 8090669, upload-time = "2026-04-01T14:43:57.335Z" },

{ url = "https://files.pythonhosted.org/packages/4b/6e/3ccb54ce8ec4ddd1accd2d89004308b7b0b21c4ac3d20fa70af4760a4330/pillow-12.2.0-cp313-cp313-manylinux\_2\_27\_aarch64.manylinux\_2\_28\_aarch64.whl", hash = "sha256:5cdfebd752ec52bf5bb4e35d9c64b40826bc5b40a13df7c3cda20a2c03a0f5ed", size = 6395194, upload-time = "2026-04-01T14:43:59.864Z" },

{ url = "https://files.pythonhosted.org/packages/67/ee/21d4e8536afd1a328f01b359b4d3997b291ffd35a237c877b331c1c3b71c/pillow-12.2.0-cp313-cp313-manylinux\_2\_27\_x86\_64.manylinux\_2\_28\_x86\_64.whl", hash = "sha256:eedf4b74eda2b5a4b2b2fb4c006d6295df3bf29e459e198c90ea48e130dc75c3", size = 7082423, upload-time = "2026-04-01T14:44:02.74Z" },

{ url = "https://files.pythonhosted.org/packages/78/5f/e9f86ab0146464e8c133fe85df987ed9e77e08b29d8d35f9f9f4d6f917ba/pillow-12.2.0-cp313-cp313-musllinux\_1\_2\_aarch64.whl", hash = "sha256:00a2865911330191c0b818c59103b58a5e697cae67042366970a6b6f1b20b7f9", size = 6505667, upload-time = "2026-04-01T14:44:05.381Z" },

{ url = "https://files.pythonhosted.org/packages/ed/1e/409007f56a2fdce61584fd3acbc2bbc259857d555196cedcad68c015c82/pillow-12.2.0-cp313-cp313-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:1e175442ed87f4912397c6d35a0db6a7b52592156014706f17658ff58bbf795", size = 7208580, upload-time = "2026-04-01T14:44:08.39Z" },

{ url = "https://files.pythonhosted.org/packages/23/c4/7349421080b12fb35414607b8871e9534546c128a11965fd4a7002ccfbee/pillow-12.2.0-cp313-cp313-win32.whl", hash = "sha256:144748b3af2d1b358d41286056d0003f47cb339b8c43a9ea42f5fea4d8c66b6e", size = 6375896, upload-time = "2026-04-01T14:44:11.197Z" },

{ url = "https://files.pythonhosted.org/packages/3f/82/8a3739a5e470b3c6cbb1d21d315800d8e16bff503d1f16b03a4ec3212786/pillow-12.2.0-cp313-cp313-win\_amd64.whl", hash = "sha256:390ede346628ccc626e5730107cde16c42d3836b89662a115a921f28440e6a3b", size = 7081266, upload-time = "2026-04-01T14:44:13.947Z" },

{ url = "https://files.pythonhosted.org/packages/c3/25/f968f618a062574294592f668218f8af564830ccebddd1fa6200f598e65c5/pillow-12.2.0-cp313-cp313-win\_arm64.whl", hash = "sha256:8023abc91fba39036dbce14a7d6535632f99c0b857807cbbbf21ecc9f4717f06", size = 2463508, upload-time = "2026-04-01T14:44:16.312Z" },

{ url = "https://files.pythonhosted.org/packages/4d/a4/b342930964e3cb4dce5038ae34b0eab4653334995336cd486c5a8c25a00c/pillow-12.2.0-cp313-cp313t-macosx\_10\_13\_x86\_64.whl", hash = "sha256:042db20a421b9bafec4b84a8b6e444686bd9d836c7fd24542db3e7df7baad9b", size = 5309927, upload-time = "2026-04-01T14:44:18.89Z" },

{ url = "https://files.pythonhosted.org/packages/9f/de/23198e0a65a9cf06123f5435a5d95cea62a365697f8f03d134d3f3a96151/pillow-12.2.0-cp313-cp313t-macosx\_11\_0\_arm64.whl", hash = "sha256:dd025009355c926a84a612fecf58bb315a3f6814b17ead51a8e48d3823d9087f", size = 4698624, upload-time = "2026-04-01T14:44:21.115Z" },

{ url = "https://files.pythonhosted.org/packages/01/a6/1265e977f17d93ea37aa28aa81bad4fa597933879fac2520d24e021c8da3/pillow-12.2.0-cp313-cp313t-manylinux2014\_aarch64.manylinux\_2\_17\_aarch64.whl", hash = "sha256:88ddbc66737e277852913bd1e07c150cc7bb124539f94c4e2df5344494e0a612", size = 6321252, upload-time = "2026-04-01T14:44:23.663Z" },

{ url = "https://files.pythonhosted.org/packages/3c/83/5982eb4a285967baa70340320be9f88e57665a387e3a53a7f0db8231a0cd/pillow-12.2.0-cp313-cp313t-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.whl", hash = "sha256:d362d1878f00c142b7e1a16e6e5e780f02be8195123f164edf7eddd911ee7c", size = 8126550, upload-time = "2026-04-01T14:44:26.772Z" },

{ url = "https://files.pythonhosted.org/packages/4e/48/6ffc514adce69f050d0753b1a18fd920fce8cac87620d5a31231b04bfc5/pillow-12.2.0-cp313-cp313t-musllinux\_2\_27\_aarch64.manylinux\_2\_28\_aarch64.whl", hash = "sha256:2c727a6d53cb0018aadd8018c2b938376af27914a68a492f59dfcaca650d5eea", size = 6433114, upload-time = "2026-04-01T14:44:29.615Z" },

{ url = "https://files.pythonhosted.org/packages/36/a3/f9a77144231fb8d40ee27107b4463e205fa4677e2ca2548e14da5cf18dce/pillow-12.2.0-cp313-cp313t-manylinux\_2\_27\_x86\_64.manylinux\_2\_28\_x86\_64.whl", hash = "sha256:efd8c21c98c5cc60653bcb311bef2ce0401642b7ce9d09e03a7da87c878289d4", size = 7115667, upload-time = "2026-04-01T14:44:32.773Z" },

{ url = "https://files.pythonhosted.org/packages/c1/fc/ac4ee3041e7d5a565e1c4fd72a113f03b6394cc72ab7089d27608f8aacbb/pillow-12.2.0-cp313-cp313t-musllinux\_1\_2\_aarch64.whl", hash = "sha256:9f08483a632889536b8139663db60f6724bfc443c96f1b18855860d7d5c0fd4", size = 6538966, upload-time = "2026-04-01T14:44:35.252Z" },

{ url = "https://files.pythonhosted.org/packages/c0/a8/27fb307055087f3668f6d0a8ccb636e7431d56ed0750e07a60547b1e083e/pillow-12.2.0-cp313-cp313t-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:dac8d77255a37e81a2efcbdf1c05f1c15ee82200e6c240d7e127e25e365c39ea", size = 7238241, upload-time = "2026-04-01T14:44:37.875Z" },

{ url = "https://files.pythonhosted.org/packages/ad/4b/926ab182c07fccae9fcb120043464e1ff1564775ec8864f21a0ebce6ac25/pillow-12.2.0-cp313-cp313t-win32.whl", hash = "sha256:ee3120ae9dfff32f121610bb08e4313be87e03efeadfc6c0d18f89127e24d0c24", size = 6379592, upload-time = "2026-04-01T14:44:40.336Z" },

{ url = "https://files.pythonhosted.org/packages/c2/c4/f9e476451a098181b30050cc4c9a3556b64c02cf6497ea421ac047e89e4b/pillow-12.2.0-cp313-cp313t-win\_amd64.whl", hash = "sha256:325ca0528c6788d2a6c3d40e3568639398137346c3d6e66bb61db96b9651c98", size = 7085542, upload-time = "2026-04-01T14:44:43.251Z" },

{ url = "https://files.pythonhosted.org/packages/00/a4/285f12aeacbe2d6dc36c407dfbbe9e96d4a80b0fb710a337f6d2ad978c75/pillow-12.2.0-cp313-cp313t-win\_arm64.whl", hash = "sha256:2e5a76d03a6c6dcef67edabda7a52494afa4035021a79c8558e14af25313d453", size = 2465765, upload-time = "2026-04-01T14:44:45.996Z" },

{ url = "https://files.pythonhosted.org/packages/bf/98/4595daa2365416a86cb0d495248a393dfc84e96d62ad080c8546256cb9c0/pillow-12.2.0-cp314-cp314-ios\_13\_0\_arm64\_iphoneos.whl", hash = "sha256:3adc9215e8be0448ed6e814966ecf3d9952f0ea40eb14e89a102b87f450660d8", size = 4100848, upload-time = "2026-04-01T14:44:48.48Z" },

{ url = "https://files.pythonhosted.org/packages/0b/79/40184d64cf89f6663e18dfcf7ca21aae2491fff1a16127681bf1fa9b8cf/pillow-12.2.0-cp314-cp314-ios\_13\_0\_arm64\_iphonesimulator.whl", hash = "sha256:6a9adfc6d24b10f89588096364cc726174118c62130c817c2837c60cf08a392b", size = 4176515, upload-time = "2026-04-01T14:44:51.353Z" },

{ url = "https://files.pythonhosted.org/packages/b0/63/703f86fd4c422a9cf722833670f4f71418fb116b2853ff7da722ea43f184

```
/pillow-12.2.0-cp314-cp314-ios_13_0_x86_64_iphonesimulator.whl", hash = "sha256:6a6e67ea2e6feda684ed370f9a1c52e7a243631c025ba42149a2cc5934dec295", size = 3640159, upload-time = "2026-04-01T14:44:53.588Z" },
 { url = "https://files.pythonhosted.org/packages/71/e0/fb22f797187d0be2270f83500aab851536101b254bfa1eae10795709d283/pillow-12.2.0-cp314-cp314-macosx_10_15_x86_64.whl", hash = "sha256:2bb4a8d594eacdfc59d9e5ad972aa8afdd48d584ffd5f13a937a664c3e7db0ed", size = 5312185, upload-time = "2026-04-01T14:44:56.039Z" },
 { url = "https://files.pythonhosted.org/packages/ba/8c/1a9e46228571de18f8e28f16fabdfc20212a5d019f3e3303452b3f0a580d/pillow-12.2.0-cp314-cp314-macosx_11_0_arm64.whl", hash = "sha256:80b2da48193b2f33ed0c32c38140f9d3186583ce7d516526d462645fd98660ae", size = 4695386, upload-time = "2026-04-01T14:44:58.663Z" },
 { url = "https://files.pythonhosted.org/packages/70/62/98f6b7f0c88b9addd0e87c217ded307b36be024d4ff8869a812b241d1345/pillow-12.2.0-cp314-cp314-manylinux2014_aarch64_manylinux_2_17_aarch64.whl", hash = "sha256:22db17c68434de69d8ecfc2fe821569195c0c373b25ccc9cbdacf2c6e53c601", size = 6280384, upload-time = "2026-04-01T14:45:01.5Z" },
 { url = "https://files.pythonhosted.org/packages/5e/03/688747d2e91cfbe0e64f316cd2e8005698f76ada3130d0194664174fa5de/pillow-12.2.0-cp314-cp314-manylinux2014_x86_64_manylinux_2_17_x86_64.whl", hash = "sha256:7b14cc0106cd9aecd6a15dd6903840a058b4700fcb817687d0ee4fc8b6e389be", size = 8091599, upload-time = "2026-04-01T14:45:04.5Z" },
 { url = "https://files.pythonhosted.org/packages/f6/35/577e22b936fcd66537329b33af0b4ccfefaebd8aec04b266528cddb33c/pillow-12.2.0-cp314-cp314-manylinux_2_27_aarch64_manylinux_2_28_aarch64.whl", hash = "sha256:8cbeb542b2ebc6fcdacabf8ac8c1a97c9b3ad3927d46b8723f9d4f033288a0f", size = 6396021, upload-time = "2026-04-01T14:45:07.117Z" },
 { url = "https://files.pythonhosted.org/packages/11/8d/d2532ad2a603ca2b93ad9f5135732124e57811d0168155852f37fbce2458/pillow-12.2.0-cp314-cp314-manylinux_2_27_x86_64_manylinux_2_28_x86_64.whl", hash = "sha256:4bf0d7bc812fbd20395212969e41931001fd59eb55a60658b0e5710872e95286", size = 7083360, upload-time = "2026-04-01T14:45:09.763Z" },
 { url = "https://files.pythonhosted.org/packages/5e/26/d325f9f56c7e039034897e7380e9cc202b1e368bf0d4d4cbe6a441f02885/pillow-12.2.0-cp314-cp314-musllinux_1_2_aarch64.whl", hash = "sha256:9aba9a17b623ef750a4d11b742cbafffeb48a869821252b30ee21b5e91392c50", size = 6507628, upload-time = "2026-04-01T14:45:12.378Z" },
 { url = "https://files.pythonhosted.org/packages/5f/f7/769d5632ffb0988f1c5e7660b3e731e30f7f8ec4318e94d0a5d674eb65a4/pillow-12.2.0-cp314-cp314-musllinux_1_2_x86_64.whl", hash = "sha256:deede7c263feb25dba4e82ea23058a235dcc2fe1f6021025dc71f2b618e26104", size = 7209321, upload-time = "2026-04-01T14:45:15.122Z" },
 { url = "https://files.pythonhosted.org/packages/6a/7a/c253e3c645cd47f1aceea6a8bacdba9991bf45bb7dfe927f7c893e89c93c/pillow-12.2.0-cp314-cp314-win32.whl", hash = "sha256:632ff19b2778e43162304d50da0181ce24ac5bb8180122cbe1bf4673428328c7", size = 6479723, upload-time = "2026-04-01T14:45:17.797Z" },
 { url = "https://files.pythonhosted.org/packages/cd/8b/601e6566b957ca50e28725cb6c355c59c2c8609751efbecd980db44e0349/pillow-12.2.0-cp314-cp314-win_amd64.whl", hash = "sha256:4e6c62e9d237e9b65fac06857d511e90d8461a32adcc1b9065ea0c0fa3a28150", size = 7217400, upload-time = "2026-04-01T14:45:20.529Z" },
 { url = "https://files.pythonhosted.org/packages/d6/94/220e46c73065c3e2951bb91c11a1fb636c8c9ad427ac3ce7d7f3359b9b2f/pillow-12.2.0-cp314-cp314-win_arm64.whl", hash = "sha256:b1c1fbd8a5a1af3412a0810d060a78b5136ec0836c8a4ef9aa11807f2a22f4e1", size = 2554835, upload-time = "2026-04-01T14:45:23.162Z" },
 { url = "https://files.pythonhosted.org/packages/b6/ab/1b426a3974cb0e7da5c29ccff4807871d48110933a57207b5a676cccc155/pillow-12.2.0-cp314-cp314t-macosx_10_15_x86_64.whl", hash = "sha256:57850958fe9c751670e49b2cecf6294acc99e562531f4bd317fa5ddee2068463", size = 5314225, upload-time = "2026-04-01T14:45:25.637Z" },
 { url = "https://files.pythonhosted.org/packages/19/1e/dce46f371be2438eefce2a1960ee2a243bbe5e961890146d2dee1ff0f12/pillow-12.2.0-cp314-cp314t-macosx_11_0_arm64.whl", hash = "sha256:d5d38f1411c0ed9f97bcb49b7bd59b6b7c314e0e27420e34d99d844b9ce3b6f3", size = 4698541, upload-time = "2026-04-01T14:45:28.355Z" },
 { url = "https://files.pythonhosted.org/packages/55/c3/7fbecf70adb3a0c33b77a300dc52e424dc22ad8cdc06557a2e49523b703d/pillow-12.2.0-cp314-cp314t-manylinux2014_aarch64_manylinux_2_17_aarch64.whl", hash = "sha256:5c0a9f29ca8e79f09de89293f82fc9b0270bb4af1d58bc98f540cc4aedf03166", size = 6322251, upload-time = "2026-04-01T14:45:30.924Z" },
 { url = "https://files.pythonhosted.org/packages/1c/3c/7fbc17cfb7e4fe0ef1642e0abc17fc6c94c9f7a16be41498e12e2ba60408/pillow-12.2.0-cp314-cp314t-manylinux2014_x86_64_manylinux_2_17_x86_64.whl", hash = "sha256:1610dd6c61621ae1cf811bef44d77e149ce3f7b95afe66a4512f8c59f25d9ebe", size = 8127807, upload-time = "2026-04-01T14:45:33.908Z" },
 { url = "https://files.pythonhosted.org/packages/ff/c3/a8ae14d6defd2e448493ff512fae903b1e9bd40b72efb6ec55ce0048c8ce/pillow-12.2.0-cp314-cp314t-manylinux_2_27_aarch64_manylinux_2_28_aarch64.whl", hash = "sha256:0a34329707af4f73cf1782a36cd2289c036880654a2c11f027bce9052d35dd", size = 6433935, upload-time = "2026-04-01T14:45:36.623Z" },
 { url = "https://files.pythonhosted.org/packages/6e/32/2880fb3a074847ac159d8f902cb43278a61e85f681661e7419e6596803ed/pillow-12.2.0-cp314-cp314t-manylinux_2_27_x86_64_manylinux_2_28_x86_64.whl", hash = "sha256:8e9c4f5b3c546fa3458a29ab22646c1c6c787ea8f5ef51300e5a60300736905e", size = 7116720, upload-time = "2026-04-01T14:45:39.258Z" },
 { url = "https://files.pythonhosted.org/packages/46/87/495cc9c30e0129501643f24d320076f4cc54f718341df18cc70ec94c44e1/pillow-12.2.0-cp314-cp314t-musllinux_1_2_aarch64.whl", hash = "sha256:f0b43ee2f06b41473269765c2feae53fc2e2fbf96e5e22ca94fb5ad677856f06", size = 6540498, upload-time = "2026-04-01T14:45:41.879Z" },
 { url = "https://files.pythonhosted.org/packages/18/53/773f5edca692009d883a72211b60fdaf8871cbef075eaa9d577f0a2f989e/pillow-12.2.0-cp314-cp314t-musllinux_1_2_x86_64.whl", hash = "sha256:f278f034eb75b4e8a13a54a876cc4a5ab39173d2cdd93a638e1b467fc545ac43", size = 7239413, upload-time = "2026-04-01T14:45:44.705Z" },
 { url = "https://files.pythonhosted.org/packages/c9/e4/4b64a97d71b2a83158134abbb2f5bd3f8a2ea691361282f010998f339ec7/pillow-12.2.0-cp314-cp314t-win32.whl", hash = "sha256:6bb77b2dcb06b20f9f4b4a8454caa581cd4dd0643a08bacf821216a16d9c8354", size = 6482084, upload-time = "2026-04-01T14:45:47.568Z" },
 { url = "https://files.pythonhosted.org/packages/ba/13/306d275efd3a3453f72114b7431c877d10b1154014c1ebbedd067770d629/pillow-12.2.0-cp314-cp314t-win_amd64.whl", hash = "sha256:6562ace0d3fb5f20ed7290f1f929cae41b25ae29528f2af1722966a0a02e2aa1", size = 7225152, upload-time = "2026-04-01T14:45:50.032Z" },
 { url = "https://files.pythonhosted.org/packages/ff/6e/cf826fae916b8658848d7b9f38d88da6396895c676e8086fc0988073aaf8/pillow-12.2.0-cp314-cp314t-win_arm64.whl", hash = "sha256:aa88ccfe4e32d362816319ed727a004423aab09c5cea43c01a4b435643fa34eb", size = 2556579, upload-time = "2026-04-01T14:45:52.529Z" },
]
```

```
[[package]]
name = "pluggy"
version = "1.6.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/f9/e2/3e91f31a7d2b083fe6ef3fa267035b518369d9511ffab804f839851d2779/pluggy-1.6.0.tar.gz", hash = "sha256:7dcc130b76258d33b90f61b658791dede3486c3e6bf003ee5c9bfb396dd22f3", size = 694
```



```

12, upload-time = "2025-05-15T12:30:07.975Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/54/20/4d324d65cc6d9205fadedc306948156824eb9f0ee1633355a8f7ec5c66bf
/pluggy-1.6.0-py3-none-any.whl", hash = "sha256:e920276dd6813095e9377c0bc5566d94c932c33b27a3e3945d8389c374dd4746", size
= 20538, upload-time = "2025-05-15T12:30:06.134Z" },
]

[[package]]
name = "protobuf"
version = "7.35.1"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/da/01/9ef0afd7999eb9badb3a768b4aedd78c86d4c65cfaf1958ab276199e
76b4/protobuf-7.35.1.tar.gz", hash = "sha256:ce115a26fe0c39a2c29973d914d327e516a6455464489fe3cd1e51a1b354f81a", size =
458717, upload-time = "2026-06-11T21:55:40.257Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/10/03/8aeeb7458d22546bf64b5250ca1daeb5ff757d900e8e4a7476c6f0db843e
/protobuf-7.35.1-cp310-abi3-macosx_10_9_universal2.whl", hash = "sha256:24f857477359a85c0c235261b8ba905fd51b2562f4a64ca
1df5473f29850cbf6", size = 433226, upload-time = "2026-06-11T21:55:31.719Z" },
 { url = "https://files.pythonhosted.org/packages/37/4b/dfb89eb0e652a1ff073c39a59fb5e3a83cfe9b57a2c83fa6d78270101767
/protobuf-7.35.1-cp310-abi3-manylinux2014_aarch64.whl", hash = "sha256:11d6b0ec246892d85215b0a13ca6e0233cf5284b68f0ac02
646427f4ff88a799", size = 328847, upload-time = "2026-06-11T21:55:34.035Z" },
 { url = "https://files.pythonhosted.org/packages/0f/58/dc12f2cd484951524af6e3382c785869b9b3fb5e52ee95ae23add53ee8f9
/protobuf-7.35.1-cp310-abi3-manylinux2014_s390x.whl", hash = "sha256:b73f9489a4b8b1c9cb1f8ed951c736392592edb24b9d6819f3
6d2e10b171d5b4", size = 344030, upload-time = "2026-06-11T21:55:34.941Z" },
 { url = "https://files.pythonhosted.org/packages/e4/be/5b3cfe508bfab6761414ff944e3366eb13be4fd71efcd69450f89ba39f43
/protobuf-7.35.1-cp310-abi3-manylinux2014_x86_64.whl", hash = "sha256:74758715c53d7158fb76caf4f0cfdacc5329a4b1bb994f865
d6cf302d413a1c4", size = 327130, upload-time = "2026-06-11T21:55:35.921Z" },
 { url = "https://files.pythonhosted.org/packages/d8/bc/6d6c7ba8709c85f8f2c390b2b118d6fb08a783676a572271851bf45a7d22
/protobuf-7.35.1-cp310-abi3-win32.whl", hash = "sha256:353652e4efd0bca5b5fc2656abf8307ef351f0cf938c9eba09f0e09c20a25c30
", size = 428945, upload-time = "2026-06-11T21:55:37.034Z" },
 { url = "https://files.pythonhosted.org/packages/0a/19/8d0cb6f20a1ef7b18f1c8986ad5783f22f84cce39c6ce9a6e645ea55192e
/protobuf-7.35.1-cp310-abi3-win_amd64.whl", hash = "sha256:230a75ddfc2de4806e56696ce9640c1cdfdb6543b7cfce98d42a4c0a0e7b
db87", size = 439996, upload-time = "2026-06-11T21:55:38.123Z" },
 { url = "https://files.pythonhosted.org/packages/19/c7/5f7c636ec43e0c545e28d1fdb71990108306f7bdc89f069ba97e428e7f
/protobuf-7.35.1-py3-none-any.whl", hash = "sha256:4bc97768d8fe4ad6743c8a19403e314511ed9f6d13205b687e52421c023ac1b9", s
ize = 171659, upload-time = "2026-06-11T21:55:39.155Z" },
]

[[package]]
name = "pyarrow"
version = "24.0.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/91/13/13e1069b351bdc3881266e11147ffccf687505dbb0ea74036237f5d4
54a5/pyarrow-24.0.0.tar.gz", hash = "sha256:85fe721a14dd823aca09127acbb06c3ca723efbd436c004f16bca601b04dcc83", size = 1
180261, upload-time = "2026-04-21T10:51:25.837Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/b4/a9/9686d9f07837f91f775e8932659192e02c74f9d8920524b480b85212cc68
/pyarrow-24.0.0-cp312-cp312-macosx_12_0_arm64.whl", hash = "sha256:6233c9ed9ab9d1db47de57d9753256d9dcffbf42db341576099f
0fd9f6bf4810", size = 34981559, upload-time = "2026-04-21T10:47:22.17Z" },
 { url = "https://files.pythonhosted.org/packages/80/b6/0ddf0e9b6ead3474ab087ae598c76b031fc45532bf6a63f3a553440fb258
/pyarrow-24.0.0-cp312-cp312-macosx_12_0_x86_64.whl", hash = "sha256:f7616236ec1bc2b15bfdec22a71ab38851c86f8f05ff64f379e
1278cf20c634a", size = 36663654, upload-time = "2026-04-21T10:47:28.315Z" },
 { url = "https://files.pythonhosted.org/packages/7c/3b/926382efe8ce27ba729071d3566ade6dfb86bdf112f366000196b2f5780a
/pyarrow-24.0.0-cp312-cp312-manylinux_2_28_aarch64.whl", hash = "sha256:1617043b99bd33e5318ae18eb2919af09c71322ef1ca465
66cdafcc6e6712fb66", size = 45679394, upload-time = "2026-04-21T10:47:34.821Z" },
 { url = "https://files.pythonhosted.org/packages/b3/7a/829f7d9dfd37c207206081d6dad474d81dde29952401f07f2ba507814818
/pyarrow-24.0.0-cp312-cp312-manylinux_2_28_x86_64.whl", hash = "sha256:6165461f55ef6314f026de6638d661188e3455d3ec498345
56a0ebbbdbace18bb", size = 48863122, upload-time = "2026-04-21T10:47:42.056Z" },
 { url = "https://files.pythonhosted.org/packages/5f/e8/f88ce625fe8babaae64e8db2d417c7653adb3019b08aae85c5ed787dc816
/pyarrow-24.0.0-cp312-cp312-musllinux_1_2_aarch64.whl", hash = "sha256:3b13dedfe76a0ad2d1d859b0811b538274a9e9d93a0bcb05c
f59333ab4980cc7e", size = 49376032, upload-time = "2026-04-21T10:47:48.967Z" },
 { url = "https://files.pythonhosted.org/packages/36/7a/82c363caa145ffff88fb475da50d3bf52bb024f61917be5424c3392eaf878
/pyarrow-24.0.0-cp312-cp312-musllinux_1_2_x86_64.whl", hash = "sha256:25ea65d868eb04015cd18e6df2f9e98f07e5bda2abefabcb8
8fce39a947716f6", size = 51929490, upload-time = "2026-04-21T10:47:55.981Z" },
 { url = "https://files.pythonhosted.org/packages/66/1c/e3e72c8014ad2743ca64a701652c733cc5cbceee15c0463a32a8c55518d9e
/pyarrow-24.0.0-cp312-cp312-win_amd64.whl", hash = "sha256:295f0a7f2e242dabd513737cf076007dc5b2d59237e3eca37b05c0c6446f
3826", size = 27355660, upload-time = "2026-04-21T10:48:01.718Z" },
 { url = "https://files.pythonhosted.org/packages/6f/d3/a1abf004482026ddc17f4503db227787fa3cfe41ec5091ff20e4fea55e57
/pyarrow-24.0.0-cp313-cp313-macosx_12_0_arm64.whl", hash = "sha256:02b001b3ed4723caa44f6cd1af2d5c86aa2cf9971dacc2ffa55b
21237713dfba", size = 34976759, upload-time = "2026-04-21T10:48:07.258Z" },
 { url = "https://files.pythonhosted.org/packages/4f/4a/34f0a36d28a2dd32225301b79daad44e243dc1a2bb77d43b60749be255c4
/pyarrow-24.0.0-cp313-cp313-macosx_12_0_x86_64.whl", hash = "sha256:04920d6a71aab08a0417709efce97d45ea8e6fb733d9ca9ecf
fb13c67839f68", size = 36658471, upload-time = "2026-04-21T10:48:13.347Z" },
 { url = "https://files.pythonhosted.org/packages/1f/78/543b94712ae8bb1a6023bcc1ac1a740fbff8286747c289cd9468fced2a5
/pyarrow-24.0.0-cp313-cp313-manylinux_2_28_aarch64.whl", hash = "sha256:a964266397740257f16f7bb2e4f08a0c81454004beab8ff
59dd531b73610e9f2", size = 45675981, upload-time = "2026-04-21T10:48:20.201Z" },
]

```

```
{ url = "https://files.pythonhosted.org/packages/84/9f/8fb7c222b100d314137fa40ec050de56cd8c6d957d1cfff685ce72f15b17
/pyarrow-24.0.0-cp313-cp313-manylinux_2_28_x86_64.whl", hash = "sha256:6f066b179d68c413374294bc1735f68475457c933258df59
4443bb9d88ddc2a0", size = 48859172, upload-time = "2026-04-21T10:48:27.541Z" },
{ url = "https://files.pythonhosted.org/packages/a7/d3/1ea72538e6c8b3b475ed78d1049a2c518e655761ea50fe1171fc855fcab7
/pyarrow-24.0.0-cp313-cp313-musllinux_1_2_aarch64.whl", hash = "sha256:1183baeb14c5f587b1ec52831e665718ce632caab84b7cd6
b85fd44f96114495", size = 49385733, upload-time = "2026-04-21T10:48:34.72Z" },
{ url = "https://files.pythonhosted.org/packages/c3/be/c3d8b06a1ba35f2260f8e1f771abbee7d5e345c0937aab90675706b1690a
/pyarrow-24.0.0-cp313-cp313-musllinux_1_2_x86_64.whl", hash = "sha256:806f24b4085453c197a5078218d1ee08783ebbbba271badd15
3d1ae22a3ee804f", size = 51934335, upload-time = "2026-04-21T10:48:42.099Z" },
{ url = "https://files.pythonhosted.org/packages/9c/62/89e07a1e7329d2cde3e3c6994ba0839a24977a2beda8be6005ea3d860b99
/pyarrow-24.0.0-cp313-cp313-win_amd64.whl", hash = "sha256:e4505fc6583f7b05ab854934896bcac8253b04ac1171a77dfb73efef9207
6d91", size = 27271748, upload-time = "2026-04-21T10:49:42.532Z" },
{ url = "https://files.pythonhosted.org/packages/17/1a/cff3a59f80b5b1658549d46611b67163f65e0664431c076ad728bf9d5af4
/pyarrow-24.0.0-cp313-cp313t-macosx_12_0_arm64.whl", hash = "sha256:1a4e45017efbf115032e4475ee876d525e0e36c742214fbe405
332480ecd6275", size = 35238554, upload-time = "2026-04-21T10:48:48.526Z" },
{ url = "https://files.pythonhosted.org/packages/a8/99/cce0f42a327bfef2c420fb6078a3eb834826e5d6697bf3009fe11d2ad051
/pyarrow-24.0.0-cp313-cp313t-macosx_12_0_x86_64.whl", hash = "sha256:7986f1fa71cee060ad00758bcc79d3a93bab8559bf978fab9e
53472a2e25a17b", size = 36782301, upload-time = "2026-04-21T10:48:55.181Z" },
{ url = "https://files.pythonhosted.org/packages/2a/66/8e560d5ff6793ca29aca213c53eec0dd482dd46cb93b2819e5aab52e4252
/pyarrow-24.0.0-cp313-cp313t-manylinux_2_28_aarch64.whl", hash = "sha256:d3e0b61e8efb24ed38898e5cdc5fffa9124be48008d40
1a1f8071500494ae42", size = 45721929, upload-time = "2026-04-21T10:49:03.676Z" },
{ url = "https://files.pythonhosted.org/packages/27/0c/a26e2550d030716e078d9f16eb74973cbf0b33b672884e9f9da1c83b871
/pyarrow-24.0.0-cp313-cp313t-manylinux_2_28_x86_64.whl", hash = "sha256:55a3bc1e3df3b5567b7d27ef551b2283f0c68a5e86f1cd5
6abc569da4f31335b", size = 48825365, upload-time = "2026-04-21T10:49:11.714Z" },
{ url = "https://files.pythonhosted.org/packages/5f/eb/771f9ecbc0c65e73fe9dcccdd1717901b9594f08c4515d000c7c62df573811
/pyarrow-24.0.0-cp313-cp313t-musllinux_1_2_aarch64.whl", hash = "sha256:641f795b361874ac9da5294f8f443dfdbec355cf2bd9e3b
8d97aaac2306b9b37", size = 49451819, upload-time = "2026-04-21T10:49:21.747Z" },
{ url = "https://files.pythonhosted.org/packages/48/da/61ae89a88732f5a785646f3ec6125dbb640fa98a540eb2b9889caa561403
/pyarrow-24.0.0-cp313-cp313t-musllinux_1_2_x86_64.whl", hash = "sha256:8adc8e6ce5fccf5dc707046ae4914fd537def529709cc0d2
85d37a7f9cd442ca", size = 51909252, upload-time = "2026-04-21T10:49:31.164Z" },
{ url = "https://files.pythonhosted.org/packages/cb/1a/8dd5cacf7b66573fa91c03d06d213356ad4edd71813aa75e08ce2b3a844
/pyarrow-24.0.0-cp313-cp313t-win_amd64.whl", hash = "sha256:9b18371ad2f44044b81a8d23bc2d8a9b6a6226dca775e8e16cfee640473
d6c5d", size = 27388127, upload-time = "2026-04-21T10:49:37.334Z" },
{ url = "https://files.pythonhosted.org/packages/ad/80/d022a34ff05d2cbedd8ccf841fc1f532ecfa9eb5ed1711b56d0e0ea71fc9
/pyarrow-24.0.0-cp314-cp314-macosx_12_0_arm64.whl", hash = "sha256:1cc9057f0319e26333b357e17f3c2c022f1a83739b48a88b25bf
d5fa2dc18838", size = 35007997, upload-time = "2026-04-21T10:49:48.796Z" },
{ url = "https://files.pythonhosted.org/packages/1a/ff/f01485fda6f4e5d441afb8dd5e7681e4db18826c1e271852f5d3957d6a80
/pyarrow-24.0.0-cp314-cp314-macosx_12_0_x86_64.whl", hash = "sha256:e6f1278eae4785b6db21229374a1c9e54ec7c549de5d1efc9630
b6207de7e170b", size = 36678720, upload-time = "2026-04-21T10:49:55.858Z" },
{ url = "https://files.pythonhosted.org/packages/9e/c2/2d2d5fea814237923f71b36495211f20b43a1576f9a4d6da7e751a64ec6f
/pyarrow-24.0.0-cp314-cp314-manylinux_2_28_aarch64.whl", hash = "sha256:adbbdc55506cbdabb830890444fb856bf0b060c46c6f80
26c6c2f2cf86ae795", size = 45741852, upload-time = "2026-04-21T10:50:04.624Z" },
{ url = "https://files.pythonhosted.org/packages/8e/3a/28ba9c1c1ebdbb5f1b94dfebb46f207e52e6a554b7fe4132540fde29a3a0
/pyarrow-24.0.0-cp314-cp314-manylinux_2_28_x86_64.whl", hash = "sha256:ae8a1145af31d903fa9bb166824d7abe9b4681a000b0159c
9fb99c11bc11ad26", size = 48889852, upload-time = "2026-04-21T10:50:12.293Z" },
{ url = "https://files.pythonhosted.org/packages/df/51/4a389acfd31dca009f8fb82d7f510bb4130f2b3a8e18cf00194d0687d8ac
/pyarrow-24.0.0-cp314-cp314-musllinux_1_2_aarch64.whl", hash = "sha256:d7027eba1df3b2069e2e8d80f644fa0918b68c46432af3d0
88ddd390d063ecde", size = 49445207, upload-time = "2026-04-21T10:50:20.677Z" },
{ url = "https://files.pythonhosted.org/packages/19/4b/0bab2b23d2ae901b1b9a03c0efd4b2d070256f8ce3fc43f6e58c167b2081
/pyarrow-24.0.0-cp314-cp314-musllinux_1_2_x86_64.whl", hash = "sha256:e56a1ffe9bf7b727432b89104cc0849c21582949dd7bdc34
f17b2001a351a76", size = 51954117, upload-time = "2026-04-21T10:50:29.14Z" },
{ url = "https://files.pythonhosted.org/packages/29/88/f4e9145da0417b3d2c12035a8492b35ff4a3dbbc653e614fcfb51d9dedb38
/pyarrow-24.0.0-cp314-cp314-win_amd64.whl", hash = "sha256:38be1808cdd068605b787e6ca9119b27eb275a0234e50212c3492331680c
3b1e", size = 28001155, upload-time = "2026-04-21T10:51:22.337Z" },
{ url = "https://files.pythonhosted.org/packages/79/4f/46a49a63f43526da895b1a45bbb51d5baf8e4d77159f8528fc3e5490007f
/pyarrow-24.0.0-cp314-cp314t-macosx_12_0_arm64.whl", hash = "sha256:418e48ce50a45a6ac73c454677203a9c75c966cb1e92ca3370
959185f197a05", size = 35250387, upload-time = "2026-04-21T10:50:35.552Z" },
{ url = "https://files.pythonhosted.org/packages/a0/da/d5e0cd5ef00796922404806d5f00325cdadc3441ce2c13fe7115f2df9a64
/pyarrow-24.0.0-cp314-cp314t-macosx_12_0_x86_64.whl", hash = "sha256:2f16197705a230a78270cdd4ea8a1d57e86b2fdcbcb34a1f6ae
bc72e65c986f9a", size = 36797102, upload-time = "2026-04-21T10:50:42.417Z" },
{ url = "https://files.pythonhosted.org/packages/34/c7/5904145b0a593a05236c882933d439b5720f0a145381179063722fbfc123
/pyarrow-24.0.0-cp314-cp314t-manylinux_2_28_aarch64.whl", hash = "sha256:fb24ac194bfc5e86839d7dcd52092ee31e5fe6733fe11f
5e3b06ef0812b20072", size = 45745118, upload-time = "2026-04-21T10:50:49.324Z" },
{ url = "https://files.pythonhosted.org/packages/13/d3/cc42fe166d1c6e4d5b80e530b7949104d10e17508a90ae202dac205ce2a
/pyarrow-24.0.0-cp314-cp314t-manylinux_2_28_x86_64.whl", hash = "sha256:9700ebd9a51f5895ce75ff4ac4b3c47a7d4b42bc618be8e
713e5d56bacf5f931", size = 48844765, upload-time = "2026-04-21T10:50:55.579Z" },
{ url = "https://files.pythonhosted.org/packages/b0/49/942c3b79878ba928324d1e17c274ed84581db8c0a749b24bcf4cbdf15bd3
/pyarrow-24.0.0-cp314-cp314t-musllinux_1_2_aarch64.whl", hash = "sha256:d8dd2768da81d3ee08cfea9b597f4abb4e8e1dc8ae7e20
4b608d23a0d3ab699", size = 49471890, upload-time = "2026-04-21T10:51:02.439Z" },
{ url = "https://files.pythonhosted.org/packages/76/97/ff71431000a75d84135a1ace5ca4ba11726a231a8007bbb320a4c54075d5
/pyarrow-24.0.0-cp314-cp314t-musllinux_1_2_x86_64.whl", hash = "sha256:61a3d7eaa97a14768b542f3d284dc6400dd2470d9f080708
b13cd46b6ae18136", size = 51932250, upload-time = "2026-04-21T10:51:10.576Z" },
{ url = "https://files.pythonhosted.org/packages/51/be/6f79d55816d5c22557cf27533543d5d70dfe692adfbec4b99f2760674f38
/pyarrow-24.0.0-cp314-cp314t-win_amd64.whl", hash = "sha256:c91d00057f23b8d353039520dc3a6c09d8608164c692e9f59a175a42b2a
e9c19", size = 28131282, upload-time = "2026-04-21T10:51:16.815Z" },
]
```

```

[[package]]
name = "pycparser"
version = "3.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/1b/7d/92392ff7815c21062bea51aa7b87d45576f649f16458d78b7cf94b9a
b2e6/pycparser-3.0.tar.gz", hash = "sha256:600f49d217304a5902ac3c37e1281c9fe94e4d0489de643a9504c5cdfdfc6b29", size = 10
3492, upload-time = "2026-01-21T14:26:51.89Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/0c/c3/44f3fbbfa403ea2a7c779186dc20772604442dde72947e7d01069cbe98e3
/pycparser-3.0-py3-none-any.whl", hash = "sha256:b727414169a36b7d524c1c3e31839a521725078d7b2ff038656844266160a992", siz
e = 48172, upload-time = "2026-01-21T14:26:50.693Z" },
]

[[package]]
name = "pydantic"
version = "2.13.4"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "annotated-types" },
 { name = "pydantic-core" },
 { name = "typing-extensions" },
 { name = "typing-inspection" },
]
sdist = { url = "https://files.pythonhosted.org/packages/18/a5/b60d21ac674192f8ab0ba4e9fd860690f9b4a6e51ca5df118733b487
d8d6/pydantic-2.13.4.tar.gz", hash = "sha256:c40756b57adaa8b1efeece5c196f3f3b7c435f90e84ea7f443901bec8099ef6", size =
844775, upload-time = "2026-05-06T13:43:05.343Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/fd/7b/122376b1fd3c62c1ed9dc80c931ace4844b3c55407b6fb2d199377c9736f
/pydantic-2.13.4-py3-none-any.whl", hash = "sha256:45a282cde31d808236fd7ea9d919b128653c8b38b393d1c4ab335c62924d9aba", s
ize = 472262, upload-time = "2026-05-06T13:43:02.641Z" },
]

[[package]]
name = "pydantic-core"
version = "2.46.4"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "typing-extensions" },
]
sdist = { url = "https://files.pythonhosted.org/packages/9d/56/921726b776ace8d8f5db44c4ef961006580d91dc52b803c489fafd1a
a249/pydantic_core-2.46.4.tar.gz", hash = "sha256:62f875393d7f270851f20523dd2e29f082bcc82292d66db2b64ea71f64b6e1c1", si
ze = 471464, upload-time = "2026-05-06T13:37:06.98Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/ce/8c/af022f0af448d7747c5154288d46b5f2bc5f17366eaa0e23e9aa04d59f3b
/pydantic_core-2.46.4-cp312-cp312-macosx_10_12_x86_64.whl", hash = "sha256:3245406455a5d98187ec35530fd772b1d799b2666798
0872c8d4614991e2c4a2", size = 2106158, upload-time = "2026-05-06T13:38:57.215Z" },
 { url = "https://files.pythonhosted.org/packages/19/95/6195171e385007300f0f5574592e467c568becce2d937a0b6804f218bc49
/pydantic_core-2.46.4-cp312-cp312-macosx_11_0_arm64.whl", hash = "sha256:962ccbab7b642487b1d8b7df90ef677e03134cf1fd8880
bf698649b22a69371f", size = 1951724, upload-time = "2026-05-06T13:37:02.697Z" },
 { url = "https://files.pythonhosted.org/packages/8e/bc/f47d1ff9cbb1620e1b5b697eef06010035735f07820180e74178226b27b3
/pydantic_core-2.46.4-cp312-cp312-manylinux_2_17_aarch64.manylinux2014_aarch64.whl", hash = "sha256:8233f2947cf85404441
fd7e0085f53b10c93e0ee78611099b5c7237e36aacbf7", size = 1975742, upload-time = "2026-05-06T13:37:09.448Z" },
 { url = "https://files.pythonhosted.org/packages/5b/11/9b9a5b0306345664a2da6410877af6e8082481b5884b3ddd78d47c6013ce
/pydantic_core-2.46.4-cp312-cp312-manylinux_2_17_armv7l.manylinux2014_armv7l.whl", hash = "sha256:3a233125ac121aa3ffba9
a2b59edfca4985a76092dc8279586ab4b71390875e7", size = 2052418, upload-time = "2026-05-06T13:37:38.234Z" },
 { url = "https://files.pythonhosted.org/packages/f1/b7/a65fec226f5d78fc39f4a13c4cc0c768c22b113438f60c14adc9d2865038
/pydantic_core-2.46.4-cp312-cp312-manylinux_2_17_ppc64le.manylinux2014_ppc64le.whl", hash = "sha256:5b712b53160b79a5850
310b912a5ef8e57e56947c8ad690c227f5c9d7e561712", size = 2232274, upload-time = "2026-05-06T13:38:27.753Z" },
 { url = "https://files.pythonhosted.org/packages/68/f0/92039db98b907ef49269a8271f67db9cb78ae2fc68062ef7e4e77adb5f61
/pydantic_core-2.46.4-cp312-cp312-manylinux_2_17_s390x.manylinux2014_s390x.whl", hash = "sha256:9401557acd873c3a7f3eb93
83edef8ac4968f9510e340f4808d427e75667e7b4", size = 2309940, upload-time = "2026-05-06T13:38:05.353Z" },
 { url = "https://files.pythonhosted.org/packages/5f/97/2aab507d3d00ca626e8e57c1eac6a79e4e5fbcc63eb99733ff55d1717f65
/pydantic_core-2.46.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl", hash = "sha256:926c9541b14b12b1681dc
a8a0b75feb510b06c6341b70a8e500c2fdcff837cce", size = 2094516, upload-time = "2026-05-06T13:39:10.577Z" },
 { url = "https://files.pythonhosted.org/packages/22/37/a8aca44d40d737dde2bc05b3c6c07dff0de07ce6f82e9f3167aeaf4d5dea
/pydantic_core-2.46.4-cp312-cp312-manylinux_2_31_riscv64.whl", hash = "sha256:56cb4851bc3fd117eddcf4fe66afd750a50274b
0da8e22be256d10e5611987", size = 2136854, upload-time = "2026-05-06T13:40:22.59Z" },
 { url = "https://files.pythonhosted.org/packages/24/99/fcef1b79238c06a8cbec70819ac722ba76e02bc8ada9b0fd66eba40da01b
/pydantic_core-2.46.4-cp312-cp312-manylinux_2_5_i686.manylinux1_i686.whl", hash = "sha256:c68fcd102d71ea85c5b2dfac3f4f8
476eff42a9e078fd5faefffd6d145063536b", size = 2180306, upload-time = "2026-05-06T13:40:10.666Z" },
 { url = "https://files.pythonhosted.org/packages/ae/6c/fc44000918855b42779d007ae63b0532794739027b2f417321cddbc44f6a
/pydantic_core-2.46.4-cp312-cp312-musllinux_1_1_aarch64.whl", hash = "sha256:b2f69dec1725e79a012d920df1707de5caf7ed5e08
f3be4435e25803efc47458", size = 2190044, upload-time = "2026-05-06T13:40:43.231Z" },
 { url = "https://files.pythonhosted.org/packages/6b/65/d9cad9cf1920d7a127ad2edba16c1db7916e59719285cd6c94600b0080ba
/pydantic_core-2.46.4-cp312-cp312-musllinux_1_1_armv7l.whl", hash = "sha256:8d0820e8192167f80d88d64038e609c31452eeca865

```

b4e1d9950a27a4609b00b", size = 2329133, upload-time = "2026-05-06T13:39:57.365Z" },

{ url = "https://files.pythonhosted.org/packages/d0/cf/c873d91679f3a30bcf5e7ac280ce5573483e72295307685120d0d5ad3416/pydantic\_core-2.46.4-cp312-cp312-musllinux\_1\_1\_x86\_64.whl", hash = "sha256:fbdb89b3e1c94a30cc5edf477c6e6a5dc4d8f84665b455c27582f211a1c72c", size = 2374464, upload-time = "2026-05-06T13:38:06.976Z" },

{ url = "https://files.pythonhosted.org/packages/47/bd/6f2fc8188f31bf10590f1e98e7b306336161fac930a8c514cd7bd828c7dc/pydantic\_core-2.46.4-cp312-cp312-win32.whl", hash = "sha256:9aa768456404a8bf48a4406685ac2bec8e72b62c69313734fa3b73cf33b3a894", size = 1974823, upload-time = "2026-05-06T13:40:47.985Z" },

{ url = "https://files.pythonhosted.org/packages/40/8c/985c1d41ea1107c2534abd9870e4ed5c8e7669b5c308297835c001e7a1c4/pydantic\_core-2.46.4-cp312-cp312-win\_amd64.whl", hash = "sha256:e9c26f834c65f5752f3f06cb08cb86a913ceb7274d0db6e267808a708b46bc89", size = 2072919, upload-time = "2026-05-06T13:39:21.153Z" },

{ url = "https://files.pythonhosted.org/packages/c4/ba/f463d006e0c47373ca7ec5e1a261c59dc01ef4d62b2657af925fb0deeee3a/pydantic\_core-2.46.4-cp312-cp312-win\_arm64.whl", hash = "sha256:4fc73cb559bdb54b1134a706a2802a4cddd27a0633f5abb7e53056268751ac6a", size = 2027604, upload-time = "2026-05-06T13:39:03.753Z" },

{ url = "https://files.pythonhosted.org/packages/51/a2/5d30b469c5267a17b39dec53208222f76a8d351dfac4af661888c5aee77d/pydantic\_core-2.46.4-cp313-cp313-macosx\_10\_12\_x86\_64.whl", hash = "sha256:5d5902252db0d3cedf8d4a1bc68f70eeb430f7e4c7104c8c476753519b423008", size = 2106306, upload-time = "2026-05-06T13:37:48.029Z" },

{ url = "https://files.pythonhosted.org/packages/c1/81/4fa520eaffa8bd7d1525e644cd6d39e7d60b1592bc5b516693c7340b50f1/pydantic\_core-2.46.4-cp313-cp313-macosx\_11\_0\_arm64.whl", hash = "sha256:c94f0688e7b8d0a67abf40e57a7eaaecd17cc9586706a31b76c031f63df052b4", size = 1951906, upload-time = "2026-05-06T13:37:17.012Z" },

{ url = "https://files.pythonhosted.org/packages/03/d5/fd02da45b659668b05923b17ba3a0100a0a3d5541e3bd8fcc4ecb711309e/pydantic\_core-2.46.4-cp313-cp313-manylinux\_2\_17\_aarch64.manylinux2014\_aarch64.whl", hash = "sha256:f027324c56cd5406ca49c124b0db10e56c69064fec039acc571c29020cc87c76", size = 1976802, upload-time = "2026-05-06T13:37:35.113Z" },

{ url = "https://files.pythonhosted.org/packages/21/f2/95727e1368be3d3ed485eaab7adbd7dda408f33f7a36e8b48e0144002b91/pydantic\_core-2.46.4-cp313-cp313-manylinux\_2\_17\_armv7l.manylinux2014\_armv7l.whl", hash = "sha256:e739fee756ba1010f8bccb534252e85a35fe45ae92c295a06059ce58b74ccd3", size = 2052446, upload-time = "2026-05-06T13:37:12.313Z" },

{ url = "https://files.pythonhosted.org/packages/9c/86/5d99feea3f77c7234b8718075b23db11532773c1a0dbd9b9490215dc2eeb/pydantic\_core-2.46.4-cp313-cp313-manylinux\_2\_17\_ppc64le.manylinux2014\_ppc64le.whl", hash = "sha256:9d56801be94b86a9da183e5f3766e6310752b99ff647e38b09a9500d88e46e76", size = 2232757, upload-time = "2026-05-06T13:39:01.149Z" },

{ url = "https://files.pythonhosted.org/packages/d2/3a/508ac615935ef57588cf6d9e9b91309fdc2da751af865e02a9098de88258c/pydantic\_core-2.46.4-cp313-cp313-manylinux\_2\_17\_s390x.manylinux2014\_s390x.whl", hash = "sha256:2412e734dcb48da14d4e4006b82b46b74f2518b8a26ee7e58c6844a6cd6d03c4", size = 2309275, upload-time = "2026-05-06T13:37:41.406Z" },

{ url = "https://files.pythonhosted.org/packages/07/f8/41db9de19d7987d6b04715a02b3b40aea467000275d9d758ffaa31af7d50/pydantic\_core-2.46.4-cp313-cp313-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl", hash = "sha256:9551187363ffcc0de2a00b2e47c25aeab1020b69b668762966df15fc5659dd5a", size = 2094467, upload-time = "2026-05-06T13:39:18.847Z" },

{ url = "https://files.pythonhosted.org/packages/2c/e2/f35033184cb11d0052daf4416e8e10a502ea2ac006fc4f459aee872727d1/pydantic\_core-2.46.4-cp313-cp313-manylinux\_2\_31\_riscv64.whl", hash = "sha256:0186750b482eefa11d7f435892b09c5c606193ef3375bcf94aa00ae6bfb66262", size = 2134417, upload-time = "2026-05-06T13:40:17.944Z" },

{ url = "https://files.pythonhosted.org/packages/7e/7b/6ceeb1cc90e193862f444ebe373d8df613f0a82572dde03fb10734c6c71/pydantic\_core-2.46.4-cp313-cp313-manylinux\_2\_5\_i686.manylinux1\_i686.whl", hash = "sha256:5855698a4856556d86e8e6cd8434bc3ac0314ee8e12089ae0e143f64c6256e4e", size = 2179782, upload-time = "2026-05-06T13:40:32.618Z" },

{ url = "https://files.pythonhosted.org/packages/5a/f2/c8d7773ede6af08036423a00ae0ceffce266c3c52a096c435d68c896083f/pydantic\_core-2.46.4-cp313-cp313-musllinux\_1\_1\_aarch64.whl", hash = "sha256:cbaf13819775b7f769bf4a1f066cb6df7a28d4480081a589828ef190226881cd", size = 2188782, upload-time = "2026-05-06T13:36:51.018Z" },

{ url = "https://files.pythonhosted.org/packages/59/31/0c864784e31f09f05cdd87606f08923b9c9e7f6e51dd27f20f62f975ce9f/pydantic\_core-2.46.4-cp313-cp313-musllinux\_1\_1\_armv7l.whl", hash = "sha256:633147d34cf4550417f12e2b1a0383973bdf5cddfde212cb09e9a581cf10820be", size = 2328334, upload-time = "2026-05-06T13:40:37.764Z" },

{ url = "https://files.pythonhosted.org/packages/c2/eb/4f6c8a41efa30baa755590f4141abf3a8c370fab610915733e74134a7270/pydantic\_core-2.46.4-cp313-cp313-musllinux\_1\_1\_x86\_64.whl", hash = "sha256:82cf5301172168103724d49a1444d3378cb20cdee30b116a1bd6031236298a5d", size = 2372986, upload-time = "2026-05-06T13:39:34.152Z" },

{ url = "https://files.pythonhosted.org/packages/5b/24/b375a480d313860c299764bfe9f349a3dc9108b3adc0d7f0d786492ebf/pydantic\_core-2.46.4-cp313-cp313-win32.whl", hash = "sha256:9fa8ae11da9e2b3126c6426f147e0fba88d96d65921799bb30c6abd1cb2c97fb", size = 1973693, upload-time = "2026-05-06T13:37:55.072Z" },

{ url = "https://files.pythonhosted.org/packages/7e/e8/cff247591966f2d22ec8c003cd7587e27b7ba7b81ab2fb888e3ab75dc285/pydantic\_core-2.46.4-cp313-cp313-win\_amd64.whl", hash = "sha256:6b3ace8194b0e5204818c92802dcdca7fc6d88aabb799d7c795540d9cd6d292", size = 2071819, upload-time = "2026-05-06T13:38:49.139Z" },

{ url = "https://files.pythonhosted.org/packages/c6/1a/f4aee670d5670e9e148e0c82c7db98d780be566c6e6a97ee8035528ca0b3/pydantic\_core-2.46.4-cp313-cp313-win\_arm64.whl", hash = "sha256:184c081504d17f1c1066e430e117142b2c77d9448a97f7b65c6ac9fd9aee238d", size = 2027411, upload-time = "2026-05-06T13:40:45.796Z" },

{ url = "https://files.pythonhosted.org/packages/8d/74/228a26ddad29c6672b805d9fd78e8d251cd04004fa7eed0e622096cd0250/pydantic\_core-2.46.4-cp314-cp314-macosx\_10\_12\_x86\_64.whl", hash = "sha256:428e04521a40150c85216fc8b85e8d39fece235a9cf5e383761238c7fa9b96fb", size = 2102079, upload-time = "2026-05-06T13:38:41.019Z" },

{ url = "https://files.pythonhosted.org/packages/ad/1f/8970b150a4b4365623ae00fc88603491f763c627311ae8031e3111356d6e/pydantic\_core-2.46.4-cp314-cp314-macosx\_11\_0\_arm64.whl", hash = "sha256:23ace664830ee0bfe014a0c7bc248b1f7f25ed7ad103852c317624a1083af462", size = 1952179, upload-time = "2026-05-06T13:36:59.812Z" },

{ url = "https://files.pythonhosted.org/packages/95/30/5211a831ae054928054b2f79731661087a2bc5c01e825c672b3a4a8f1b3e/pydantic\_core-2.46.4-cp314-cp314-manylinux\_2\_17\_aarch64.manylinux2014\_aarch64.whl", hash = "sha256:ce5c1d2a8b27468f433ca974829c44060b8097eedc39933e3c206a90ee49c4a9", size = 1978926, upload-time = "2026-05-06T13:37:39.933Z" },

{ url = "https://files.pythonhosted.org/packages/57/ef/6896687331eb67adeef047db3c2e8788fc65a7fd9c9e2b46b7744fe245/pydantic\_core-2.46.4-cp314-cp314-manylinux\_2\_17\_armv7l.manylinux2014\_armv7l.whl", hash = "sha256:7283d57845ecf5a163403eb0702dfc220cc4fbdd18919cb5ccea4f95ee1cdab4", size = 2046785, upload-time = "2026-05-06T13:38:01.995Z" },

{ url = "https://files.pythonhosted.org/packages/60/d9/6715260422ff50a2109878fd24d948a6c3446bb2664f34ee78cd972b3acd/pydantic\_core-2.46.4-cp314-cp314-manylinux\_2\_17\_ppc64le.manylinux2014\_ppc64le.whl", hash = "sha256:8daafc69c93ee8a0204506a3b6b30f586ef54028f52ae65c4cfc5184fd5914", size = 2228733, upload-time = "2026-05-06T13:40:50.371Z" },

{ url = "https://files.pythonhosted.org/packages/18/ae/fdb2f64316afca925640f8e70bb1a564b0ec2721c1389e25b8eb4bf9a299/pydantic\_core-2.46.4-cp314-cp314-manylinux\_2\_17\_s390x.manylinux2014\_s390x.whl", hash = "sha256:cd2213145bcc2ba85884d0a63d222fece9209678f77b9b4d76f054c561adb28", size = 2307534, upload-time = "2026-05-06T13:37:21.531Z" },

{ url = "https://files.pythonhosted.org/packages/89/1d/8eff589b45bb8190a9d12c49cfad0f176a5cbd1534908a6b5125e2886239/pydantic\_core-2.46.4-cp314-cp314-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl", hash = "sha256:7a5f930472650a82629163023e630d160863fce524c616f4e5186e5de9d9a49b", size = 2099732, upload-time = "2026-05-06T13:39:31.942Z" },

{ url = "https://files.pythonhosted.org/packages/06/d5/ee5a3366637fee41dee51a1fc91562dcf12ddbc68fda34e6b253da2324bb/pydantic\_core-2.46.4-cp314-cp314-manylinux\_2\_31\_riscv64.whl", hash = "sha256:c1b3f518abeca3aa13c712fd202306e145abf59a18b094a6baf2d2bbf59192c", size = 2129627, upload-time = "2026-05-06T13:37:25.033Z" },

{ url = "https://files.pythonhosted.org/packages/94/33/2414be571d2c6a6c4d08be21f9292b6d3fdb08949a97b6dfe985017821db/pydantic\_core-2.46.4-cp314-cp314-manylinux\_2\_5\_i686.manylinux1\_i686.whl", hash = "sha256:1a7dd0b3ee80d90150e3495a3a13a34dbcbfd4f012996a6a1d8900e91b5c0fb", size = 2179141, upload-time = "2026-05-06T13:37:14.046Z" },

{ url = "https://files.pythonhosted.org/packages/7b/79/7daa95be995be0eccc4cf75064cb33f9bbbf3fe0158caf2f0d4a996a5c7/pydantic\_core-2.46.4-cp314-cp314-musllinux\_1\_1\_aarch64.whl", hash = "sha256:3fb702cd90b0446a3a1c5e470bfa0dd23c0233b676a9099ddcc964fa6ca13898", size = 2184325, upload-time = "2026-05-06T13:36:53.615Z" },

{ url = "https://files.pythonhosted.org/packages/9f/cb/d0a382f5c0de8a222dc61c65348e0ce831b1f68e0a018450d31c2cace3a5/pydantic\_core-2.46.4-cp314-cp314-musllinux\_1\_1\_armv7l.whl", hash = "sha256:b8458003118a712e66286df6a707db01c52c0f52f7db8e4a38f0da1d3b94fc4e", size = 2323990, upload-time = "2026-05-06T13:40:29.971Z" },

{ url = "https://files.pythonhosted.org/packages/05/db/d9ba624cc4a5aced1598e88c04fddb8310c8a69b9d38b9a3d39ce3a61ed7/pydantic\_core-2.46.4-cp314-cp314-musllinux\_1\_1\_x86\_64.whl", hash = "sha256:372429a130e469c9cd698925ce5fc50940b7a1336b0d82038e63d5bb4c4edc519", size = 2369978, upload-time = "2026-05-06T13:37:23.027Z" },

{ url = "https://files.pythonhosted.org/packages/f2/20/d15df15ba918c423461905802bfd2981c3af0bfa0e40d05e13edbf48bc3/pydantic\_core-2.46.4-cp314-cp314-win32.whl", hash = "sha256:85bb3611ff1802f3ee7fdd7dbff26b56f343fb432d57a4728fdd49b6ef35e2f4", size = 1966354, upload-time = "2026-05-06T13:38:03.499Z" },

{ url = "https://files.pythonhosted.org/packages/fc/b6/6b8de4c0a7d7ab3004c439c80c5c1e0a3e8d78bbae19379b01960383d9e5/pydantic\_core-2.46.4-cp314-cp314-win\_amd64.whl", hash = "sha256:811ff8e9c313ab425368bcb36e5c4ebd7108c2bbf4e4089c9fbb0b01eff63fac", size = 2072238, upload-time = "2026-05-06T13:39:40.807Z" },

{ url = "https://files.pythonhosted.org/packages/32/36/51eb763beec1f4cf59b1db243a7dcc39cbb41230f050a09b9d69faaf0a48/pydantic\_core-2.46.4-cp314-cp314-win\_arm64.whl", hash = "sha256:bfec22eab3c8cc2ceec0248aec886624116dc079afa027ecc8ad4a7e62010f8a", size = 2018251, upload-time = "2026-05-06T13:37:26.722Z" },

{ url = "https://files.pythonhosted.org/packages/e8/91/855af51d625b23aa987116a19e231d2aaef9c4a415273ddc189b79a45fee/pydantic\_core-2.46.4-cp314-cp314t-macosx\_10\_12\_x86\_64.whl", hash = "sha256:af8244b2bef6aaad6d92cda81372de7f8c8d36c9f0c3ea36e827c60e7d9467a0", size = 2099593, upload-time = "2026-05-06T13:39:47.682Z" },

{ url = "https://files.pythonhosted.org/packages/fb/1b/8784a54c65edb5f49f0a14d6977cf1b209bba85a4c77445b255c2de58ab3/pydantic\_core-2.46.4-cp314-cp314t-macosx\_11\_0\_arm64.whl", hash = "sha256:5a4330cdbc57162e4b3aa30f3588ba752257694c9c9be3e7ebb11b4aca659b5d", size = 1935226, upload-time = "2026-05-06T13:40:40.428Z" },

{ url = "https://files.pythonhosted.org/packages/e8/e7/1955d28d1afc56dd4b3ad7cc0cf39df1b9852964cf16e5d13912756d6d6b/pydantic\_core-2.46.4-cp314-cp314t-manylinux\_2\_17\_aarch64.manylinux2014\_aarch64.whl", hash = "sha256:29c61fc04a3d840155ff08e475a04809278972fe6aef51e2720554e96367e34b", size = 1974605, upload-time = "2026-05-06T13:37:32.029Z" },

{ url = "https://files.pythonhosted.org/packages/93/e2/3fedbf0ba7a22850e6e9fd78117f1c0f10f950182344d8a6c535d468fdd8/pydantic\_core-2.46.4-cp314-cp314t-manylinux\_2\_17\_armv7l.manylinux2014\_armv7l.whl", hash = "sha256:c50f2528cf200c5eed56faf3f4e22fcd5f38c157a8b78576e6ba3168ec35f000", size = 2030777, upload-time = "2026-05-06T13:38:55.239Z" },

{ url = "https://files.pythonhosted.org/packages/f8/61/46be275fcaaba0b4f5b9669dd852267ce1ff616592dccf7a7845588df091/pydantic\_core-2.46.4-cp314-cp314t-manylinux\_2\_17\_ppc64le.manylinux2014\_ppc64le.whl", hash = "sha256:0cbe8b01f948de4286c74cdd6c667aceb38f5c1e26f0693b3983d9d74887c65e", size = 2236641, upload-time = "2026-05-06T13:37:08.096Z" },

{ url = "https://files.pythonhosted.org/packages/60/db/12e93e46a8bac9988be3c016860f83293daea8c716c029c9ace279036f2f/pydantic\_core-2.46.4-cp314-cp314t-manylinux\_2\_17\_s390x.manylinux2014\_s390x.whl", hash = "sha256:617d7e2ca7dcb8c5cf6bcb8c59b8832c94b36196bbf1cbd1bfb56ed341905edd", size = 2286404, upload-time = "2026-05-06T13:40:20.221Z" },

{ url = "https://files.pythonhosted.org/packages/e2/4a/4d8b19008f38d31c53b8219cfedc2e3d5de5fe99d90076b7e767de29274f/pydantic\_core-2.46.4-cp314-cp314t-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl", hash = "sha256:7027560ee92211647d0d34e3f7cd6f50da56399d26a9c8ad0da286d3869a5f3f", size = 2109219, upload-time = "2026-05-06T13:38:12.153Z" },

{ url = "https://files.pythonhosted.org/packages/88/70/3cbc40978fefb7bb09c6708d40d4ad1a5d70fd7213c3d17f971de868ec1f/pydantic\_core-2.46.4-cp314-cp314t-manylinux\_2\_31\_riscv64.whl", hash = "sha256:f99626688942fb746e545232e7726926f3be91b5975f8b55327665fafda991c7", size = 2110594, upload-time = "2026-05-06T13:40:02.971Z" },

{ url = "https://files.pythonhosted.org/packages/9d/20/b8d36736216e29491125531685b2f9e61aa5b4b2599893f8268551da3338/pydantic\_core-2.46.4-cp314-cp314t-manylinux\_2\_5\_i686.manylinux1\_i686.whl", hash = "sha256:fc3e9034a63de20e15e8ade85358bc6efc614008cab72898b4b4952bea0509ff", size = 2159542, upload-time = "2026-05-06T13:39:27.506Z" },

{ url = "https://files.pythonhosted.org/packages/1d/a2/367df868eb584dacf6bf82a389272406d7178e301c4ac82545ab98bc2dd9/pydantic\_core-2.46.4-cp314-cp314t-musllinux\_1\_1\_aarch64.whl", hash = "sha256:97e7cf2be5c77b7d1a9713a05605d49460d02c6078d38d8bef3cbe323c548424", size = 2168146, upload-time = "2026-05-06T13:38:31.93Z" },

{ url = "https://files.pythonhosted.org/packages/c1/b8/4460f77f7e201893f649a29ab355ddddd3beee8a97bcb1a320db414f9a06e/pydantic\_core-2.46.4-cp314-cp314t-musllinux\_1\_1\_armv7l.whl", hash = "sha256:3bf92c5d0e00fefaab325a4d27828fe6b6e2a21848686b5b60d2d9eeb09d76c6", size = 2306309, upload-time = "2026-05-06T13:37:44.717Z" },

{ url = "https://files.pythonhosted.org/packages/64/c4/be2639293acd87cd8dbcc41a73cee9b2ebf996fe6d892a1a74e88ad3f7/pydantic\_core-2.46.4-cp314-cp314t-musllinux\_1\_1\_x86\_64.whl", hash = "sha256:3ecbc122d18468d06ca279dc26a8c2e2d5acb10943bb35e36ae92096dc3b5565", size = 2369736, upload-time = "2026-05-06T13:37:05.645Z" },

{ url = "https://files.pythonhosted.org/packages/30/a6/9f9f380dbb301f67023bf8f707aaa75daadf84f7152d95c410fd7e81d994/pydantic\_core-2.46.4-cp314-cp314t-win32.whl", hash = "sha256:e846ae7835bf0703ae43f534ab79a867146dadd59dc9ca5c8b53d5c8f7c9ef02", size = 1955575, upload-time = "2026-05-06T13:38:51.116Z" },

{ url = "https://files.pythonhosted.org/packages/40/1f/f1eb9eb350e795d1af8586289746f5c5677d16043040d63710e22abc43c9/pydantic\_core-2.46.4-cp314-cp314t-win\_amd64.whl", hash = "sha256:2108ba5c1c1eca18030634489dc54484414ee36357f2f9f780b93e7ddbb44b5", size = 2051624, upload-time = "2026-05-06T13:38:21.672Z" },

{ url = "https://files.pythonhosted.org/packages/f6/d2/42dd53d0a85c27606f316d3aa5d2869c4e8470a5ed6dec30e4a1abe19192/pydantic\_core-2.46.4-cp314-cp314t-win\_arm64.whl", hash = "sha256:4fcbe087dbc2068af7eda3aa87634eba216dbda64d1ae73c8684b621d33f6596", size = 2017325, upload-time = "2026-05-06T13:40:52.723Z" },

{ url = "https://files.pythonhosted.org/packages/9d/1d/8987ad40f65ae1432753072f214fb5c74fe47ffbd0698bb9cbbb585664f8/pydantic\_core-2.46.4-graalpy312-graalpy250\_312\_native-macosx\_10\_12\_x86\_64.whl", hash = "sha256:1d8ba486450b14f3b1d63bc521d410ec7565e52f887b9fb671791886436a42f7", size = 2095527, upload-time = "2026-05-06T13:39:52.283Z" },

{ url = "https://files.pythonhosted.org/packages/64/d3/84c282a7eee1d3ac4c0377546ef5a1ea436ce26840d9ac3b7ed54a377507

```

/pydantic_core-2.46.4-graalpy312-graalpy250_312_native-macosx_11_0_arm64.whl", hash = "sha256:3009f12e4e90b7f88b4f9adb1
b0c4a3d58fe7820f3238c190047209d148026df", size = 1936024, upload-time = "2026-05-06T13:40:15.671Z" },
 { url = "https://files.pythonhosted.org/packages/d7/ca/eac61596cdeb4d7e174d3dc0bd8a6238f14f75f97a24e7b7db4c7e7340a0
/pydantic_core-2.46.4-graalpy312-graalpy250_312_native-manylinux_2_17_aarch64.manylinux2014_aarch64.whl", hash = "sha25
6:ad785e92e6dc634c2155edc8bd6b64957ab844541bcb96a1366c202951ae526", size = 1990696, upload-time = "2026-05-06T13:38:34
.717Z" },
 { url = "https://files.pythonhosted.org/packages/fa/c3/7c8b240552251faf6b3a957db200fcfbcc36763c050428b601e0c9b83b
/pydantic_core-2.46.4-graalpy312-graalpy250_312_native-manylinux_2_17_x86_64.manylinux2014_x86_64.whl", hash = "sha256:
00c603d540afdd6b80eb39f078f33ebd46211f02f33e34a32d9f053bba711de0", size = 2147590, upload-time = "2026-05-06T13:39:29.8
83Z" },
]

[[package]]
name = "pydantic-settings"
version = "2.14.1"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "pydantic" },
 { name = "python-dotenv" },
 { name = "typing-inspection" },
]
sdist = { url = "https://files.pythonhosted.org/packages/07/60/1d1e59c9c90d54591469ada7d268251f71c24bdb765f1a8a832cee8c
6653/pydantic_settings-2.14.1.tar.gz", hash = "sha256:e874d3bec7e787b0c9958277956ed9b4dd5de6a80e162188fdaff7c5e26fd5fa"
, size = 235551, upload-time = "2026-05-08T13:40:06.542Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/ae/8d/f1af3832f5e6eb13ba94ee809e72b8ecb5eef226d27ee0bef7d963d943c7
/pydantic_settings-2.14.1-py3-none-any.whl", hash = "sha256:6e3c7edfd8277687cdc598f56e5cfff0e9bfff0910a3749deaa8d4401c3a
2b9de", size = 60964, upload-time = "2026-05-08T13:40:04.958Z" },
]

[[package]]
name = "pydeck"
version = "0.9.2"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "jinja2" },
 { name = "numpy" },
]
sdist = { url = "https://files.pythonhosted.org/packages/eb/df/4e9e7f20f8034a37c6571c93809f6d22388c39978c98d174d656c1a1
8fd2/pydeck-0.9.2.tar.gz", hash = "sha256:c10d9035e81ead6385264cac8d19402471f6866a15ca1f7df1400f52142bcf87", size = 584
9672, upload-time = "2026-04-16T18:30:30.089Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/88/24/b30ee7d723100fd822de1bb4c0adea62f3419884a75a536f35f355d1e7c0
/pydeck-0.9.2-py2.py3-none-any.whl", hash = "sha256:8213dfeacc5f6bfe6825f61c8ee34e3850e8a31fc43924379ec98edb34a75b25",
size = 11305615, upload-time = "2026-04-16T18:30:28.133Z" },
]

[[package]]
name = "pygments"
version = "2.20.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/c3/b2/bc9c9196916376152d65522fdcebac55e66de6603a76a02bca1b641
4f6c/pygments-2.20.0.tar.gz", hash = "sha256:6757cd03768053ff99f3039c1a36d6c0aa0b263438fcab17520b30a303a82b5f", size =
4955991, upload-time = "2026-03-29T13:29:33.898Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/f4/7e/a72dd26f3b0f4f2bf1dd8923c85f7ceb43172af56d63c7383eb62b332364
/pygments-2.20.0-py3-none-any.whl", hash = "sha256:81a9e26dd42fd28a23a2d169d86d7ac03b46e2f8b59ed4698fb4785f946d0176", s
ize = 1231151, upload-time = "2026-03-29T13:29:30.038Z" },
]

[[package]]
name = "pypdfium2"
version = "5.10.1"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/1d/78/d9b45abb97a3686643f7c6472a5f7688f2013a373226121dc76b9deb
bacf/pypdfium2-5.10.1.tar.gz", hash = "sha256:f257d2011eb43c846b7e9f5a802e28646b29732763e4a35dd6ca76f9be580538", size =
272963, upload-time = "2026-06-15T10:09:16.179Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/98/49/0b85fcc0d236582143a25cf275b0b4f5d786f51eb07a89ec9c79d43efe18
/pypdfium2-5.10.1-py3-none-android_23_arm64_v8a.whl", hash = "sha256:13abf7a9f5e0ddbec8bbcccea5f13ae5abe8a298ea219e125b
0fc24c1d2171b4", size = 3409176, upload-time = "2026-06-15T10:08:36.017Z" },
 { url = "https://files.pythonhosted.org/packages/20/d2/2f522c5b2ad5166edf256bb4dbab97de5d07b2573f8a5630ddfdcd1f8d4ee
/pypdfium2-5.10.1-py3-none-android_23_armeabi_v7a.whl", hash = "sha256:a0dc52b56631e2f7edcdb22bae3b155aa840bec32c5bd057
81e90baaecee88e7", size = 2866175, upload-time = "2026-06-15T10:08:37.955Z" },
 { url = "https://files.pythonhosted.org/packages/a7/be/477548c026c2badfdbf4afc3358b7135121fb5bec2e2effcb67e3a674d0b
/pypdfium2-5.10.1-py3-none-macosx_12_0_arm64.whl", hash = "sha256:ebb9e63f92d15fc41b359fe7a187233dfae37548800e1fa09cb2f

```

```

c466ac89951", size = 3621427, upload-time = "2026-06-15T10:08:39.969Z" },
 { url = "https://files.pythonhosted.org/packages/8c/9b/0131c7f711b62c6edd6b200e9eb6340be6de4f6dc5baae625e3394d8d5fb
 /pypdfium2-5.10.1-py3-none-macosx_12_0_x86_64.whl", hash = "sha256:d04f2050b6b32bb18624688b600543342a4ab3aacf64bf66521a
 5af72bbc7de1", size = 3682825, upload-time = "2026-06-15T10:08:42.181Z" },
 { url = "https://files.pythonhosted.org/packages/9d/3d/bddfceb6e67e54d6dd1ab6c0f1feff796a89596e40f6345a3b4cc6a3d408
 /pypdfium2-5.10.1-py3-none-manylinux_2_17_aarch64.manylinux2014_aarch64.whl", hash = "sha256:0de53d2710ca9509fc2812340a
 cc57c3f043697609068592d87de654f8cabf44", size = 3682206, upload-time = "2026-06-15T10:08:44.087Z" },
 { url = "https://files.pythonhosted.org/packages/58/08/dedeb25c6645fc8a5eda24f54e0b1d083b7334ababf5f518bb939d729cc0
 /pypdfium2-5.10.1-py3-none-manylinux_2_17_armv7l.manylinux2014_armv7l.whl", hash = "sha256:10a26ce04795f8ec079e81c707fc
 b5737061e8a78025babc3d6e36642e9c903a", size = 3413720, upload-time = "2026-06-15T10:08:45.739Z" },
 { url = "https://files.pythonhosted.org/packages/cc/48/69e1fc8b1216005243c6415183bbf6de1cda3f5a06758b3fa4a26a7385c6
 /pypdfium2-5.10.1-py3-none-manylinux_2_17_i686.manylinux2014_i686.whl", hash = "sha256:be6d2a8d1bcfd777188e7aa55a25c83f
 34d54e8350ad8810fb32017787d9b0d9", size = 3812913, upload-time = "2026-06-15T10:08:47.45Z" },
 { url = "https://files.pythonhosted.org/packages/72/7f/132455a58ad736d76815c6cd1307532c3f433299d945b9d2f8cc2387c309
 /pypdfium2-5.10.1-py3-none-manylinux_2_17_ppc64le.manylinux2014_ppc64le.whl", hash = "sha256:cf4d2527f79f31c550490cc74c
 9f32e19385a944630a3ef4cd4d9b6f961fbf77", size = 4223220, upload-time = "2026-06-15T10:08:49.099Z" },
 { url = "https://files.pythonhosted.org/packages/bf/8c/4d5804eca598bbe894e0a9a510807e221c1623e7276ce6b68fa2660dc933
 /pypdfium2-5.10.1-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl", hash = "sha256:3ba127750bc3f4161461538d532d
 74491cd976f584f1753a6cee9cb821338ec1", size = 3738950, upload-time = "2026-06-15T10:08:50.821Z" },
 { url = "https://files.pythonhosted.org/packages/d1/7f/baac59bf14ff914d97789ee0368c22ae233aa857aa9c0726bcb515dbc4d7
 /pypdfium2-5.10.1-py3-none-manylinux_2_27_s390x.manylinux_2_28_s390x.whl", hash = "sha256:80f30517ee089dfbbc6e9de6da365
 b6e8c0ce8f80c40b7d4025a3b1d3bbd8a70", size = 4029869, upload-time = "2026-06-15T10:08:52.494Z" },
 { url = "https://files.pythonhosted.org/packages/d0/d7/a5d58a0bcbca31a0e37ed636a76ef3d2d215733f28af61637287d061b0c54
 /pypdfium2-5.10.1-py3-none-manylinux_2_34_riscv64.manylinux_2_39_riscv64.whl", hash = "sha256:ad5f5de15febb788c6eb3853e
 58aceda9cd8c5187c92472abaecf9558deb0cf", size = 3990927, upload-time = "2026-06-15T10:08:54.555Z" },
 { url = "https://files.pythonhosted.org/packages/da/1a/98eebd14b36812176297cf765d504ebee0c982f895c8a3a9fbd2717797de
 /pypdfium2-5.10.1-py3-none-musllinux_1_2_aarch64.whl", hash = "sha256:8947fa3cd808da33960bdf8ff9e5247aa94ccd94f0b09bb34
 02e99498d83bcc9", size = 4989624, upload-time = "2026-06-15T10:08:56.284Z" },
 { url = "https://files.pythonhosted.org/packages/1c/20/f2e124d607b8bb90a9f1ce976afff38c70e215cd7ea86af784cb2e8a19dd
 /pypdfium2-5.10.1-py3-none-musllinux_1_2_armv7l.whl", hash = "sha256:139a6387a3a2652f288e53164268bb03af4fa221d78484ee18
 407053a60082a3", size = 4535124, upload-time = "2026-06-15T10:08:58.214Z" },
 { url = "https://files.pythonhosted.org/packages/59/ef/469ea87f668a32ff3280ea15e522e3a7858d2c80f1ecc320ece1244624c9
 /pypdfium2-5.10.1-py3-none-musllinux_1_2_i686.whl", hash = "sha256:172ff3e10358d66456e27fb0b8b5098e28ec24e51072eb4b5b86
 077d550e21bb", size = 5229373, upload-time = "2026-06-15T10:09:00.702Z" },
 { url = "https://files.pythonhosted.org/packages/c3/48/c7ed3001f0c5e28114c98bf918c8121e422a65ba7323e1e12f3f28e2d278
 /pypdfium2-5.10.1-py3-none-musllinux_1_2_ppc64le.whl", hash = "sha256:09fda0609dc4749c9865a0315c447d6f2583a580693bcabd6
 9980d3fbb22ad51", size = 5140010, upload-time = "2026-06-15T10:09:02.734Z" },
 { url = "https://files.pythonhosted.org/packages/b2/bc/00b731bfc1fdc0f3c7d91108fceb54978ff4f5a92336820e5ed6c12535ff
 /pypdfium2-5.10.1-py3-none-musllinux_1_2_riscv64.whl", hash = "sha256:6b01adcf9aaf7a635a59bed5687fd4fd7b0da292664f050d
 4ebd2bfa5c70584", size = 4643310, upload-time = "2026-06-15T10:09:04.794Z" },
 { url = "https://files.pythonhosted.org/packages/9a/e8/3ad242233f657c19092a8c83684c8154b8d02a826535a6a2591d91aa5dde
 /pypdfium2-5.10.1-py3-none-musllinux_1_2_s390x.whl", hash = "sha256:610e14c37d2090b826dccc0604fc7e7612c0ce591190780ae22
 8abdb9abb971e", size = 5087879, upload-time = "2026-06-15T10:09:06.861Z" },
 { url = "https://files.pythonhosted.org/packages/80/3f/9d0747a02ac3021ed3db7ac27c5187d97e78b0253a4bdfbf38666968474
 /pypdfium2-5.10.1-py3-none-musllinux_1_2_x86_64.whl", hash = "sha256:f2803020952afa57e1e148adc19369b835154e1fa251a1ca85
 008ab6466a710f", size = 5047369, upload-time = "2026-06-15T10:09:08.883Z" },
 { url = "https://files.pythonhosted.org/packages/9e/41/7d5187e9527eae81890a3b13193442112ef788d61ddaa68a6d59ac447bec
 /pypdfium2-5.10.1-py3-none-win32.whl", hash = "sha256:8702bb4f01ddfc8e7757b41b4c2c8392ac17c9f0234476e1e69672ea7c6d6aa0",
 size = 3680056, upload-time = "2026-06-15T10:09:10.95Z" },
 { url = "https://files.pythonhosted.org/packages/16/1d/c62bd59dd8345cc4b640f942f465633f6b07b859d01ddb648610a7bf5c7c
 /pypdfium2-5.10.1-py3-none-win_amd64.whl", hash = "sha256:58da5b51fb7884c7d21a05062ab13edb011d1a08dfd9694f3d5d685df6279
 6b9", size = 3812105, upload-time = "2026-06-15T10:09:12.684Z" },
 { url = "https://files.pythonhosted.org/packages/10/d5/21bac39125df8a93e99c04583486b58a62b5997d6b3541e3ad0f69053392
 /pypdfium2-5.10.1-py3-none-win_arm64.whl", hash = "sha256:e3301c2f7a66fb8cb57dba857d0c9e90215e178f6602a87c5a306cd98513d
 ab8", size = 3600043, upload-time = "2026-06-15T10:09:14.606Z" },
]

[[package]]
name = "pytest"
version = "9.1.0"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "colorama", marker = "sys_platform == 'win32'" },
 { name = "iniconfig" },
 { name = "packaging" },
 { name = "pluggy" },
 { name = "pygments" },
]

sdist = { url = "https://files.pythonhosted.org/packages/84/0e/b5858858d74958632c49b72cb25a3976ff9f632397626715be71c89d
3971/pytest-9.1.0.tar.gz", hash = "sha256:41dd9148c08072446394cefd3d79701701335a9f4cae69ba92e39f6c7f5c061c", size = 163
4181, upload-time = "2026-06-13T18:52:45.983Z" }

wheels = [
 { url = "https://files.pythonhosted.org/packages/8b/5a/ba30a81239b909821b3153e303e7def45178bf353da4f72380e6c5e8793b
 /pytest-9.1.0-py3-none-any.whl", hash = "sha256:8ebb0e7888bdf2bdfc602ec51f8f62d50200af37356c74e503c79a94f5c81f32", size
 = 386453, upload-time = "2026-06-13T18:52:44.045Z" },
]

```

```

[[package]]
name = "python-dateutil"
version = "2.9.0.post0"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "six" },
]
sdist = { url = "https://files.pythonhosted.org/packages/66/c0/0c8b6ad9f17a802ee498c46e004a0eb49bc148f2fd230864601a86dcf6db/python-dateutil-2.9.0.post0.tar.gz", hash = "sha256:37dd54208da7e1cd875388217d5e00ebd4179249f90fb72437e91a35459a0ad3", size = 342432, upload-time = "2024-03-01T18:36:20.211Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/ec/57/56b9bcc3c9c6a792fcbaf139543cee77261f3651ca9da0c93f5c1221264b/python_dateutil-2.9.0.post0-py2.py3-none-any.whl", hash = "sha256:a8b2bc7bffa282281c8140a97d3aa9c14da0b136dfe83f850eea9a5f7470427", size = 229892, upload-time = "2024-03-01T18:36:18.57Z" },
]

[[package]]
name = "python-dotenv"
version = "1.2.2"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/82/ed/0301aeeac3e5353ef3d94b6ec08bbcabd04a72018415dcb29e588514bba8/python_dotenv-1.2.2.tar.gz", hash = "sha256:2c371a91fbd7ba082c2c1dc1f8bf89ca22564a087c2c287cd9b662adde799cf3", size = 50135, upload-time = "2026-03-01T16:00:26.196Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/0b/d7/1959b9648791274998a9c3526f6d0ec8fd2233e4d4acce81bbae76b44b2a/python_dotenv-1.2.2-py3-none-any.whl", hash = "sha256:1d8214789a24de455a8b8bd8ae6fe3c6b69a5e3d64aa8a8e5d68e694bbcb285a", size = 22101, upload-time = "2026-03-01T16:00:25.09Z" },
]

[[package]]
name = "python-multipart"
version = "0.0.32"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/5b/42/55c32bb9b12693c092ad250a0e82edb5b31ddeda6eb772de5f308b3804ad/python_multipart-0.0.32.tar.gz", hash = "sha256:be54b7f3fa167bb83e4fcd936b887b708f4e57fe75911c02aebf53efaf8d938e", size = 46881, upload-time = "2026-06-04T16:18:58.647Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/e1/04/e8135ebd1ad02c56ec633277529b2602ff99ff634be76cdba5744cf554fd/python_multipart-0.0.32-py3-none-any.whl", hash = "sha256:ff6d3f776f16878c894e52e107296ffc890e913c611b1a4ec6c44e2821fe2e23", size = 30042, upload-time = "2026-06-04T16:18:57.319Z" },
]

[[package]]
name = "referencing"
version = "0.37.0"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "attrs" },
 { name = "rpds-py" },
 { name = "typing-extensions", marker = "python_full_version < '3.13'" },
]
sdist = { url = "https://files.pythonhosted.org/packages/22/f5/df4e9027acead3ecc63e50fe1e36aca1523e1719559c499951bb4b53188f/referencing-0.37.0.tar.gz", hash = "sha256:44aefc3142c5b842538163acb373e24cce6632bd54bdb01b21ad5863489f50d8", size = 78036, upload-time = "2025-10-13T15:30:48.871Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/2c/58/ca301544e1fa93ed4f80d724bf5b194f6e4b945841c5bfd555878eea9fcb/referencing-0.37.0-py3-none-any.whl", hash = "sha256:381329a9f99628c9069361716891d34ad94af76e461dcb0335825aecc7692231", size = 26766, upload-time = "2025-10-13T15:30:47.625Z" },
]

[[package]]
name = "requests"
version = "2.34.2"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "certifi" },
 { name = "charset-normalizer" },
 { name = "idna" },
 { name = "urllib3" },
]
sdist = { url = "https://files.pythonhosted.org/packages/ac/c3/e2a2b89f2d3e2179abd6d00ebd70bff6273f37fb3e0cc209f48b39d00cbf/requests-2.34.2.tar.gz", hash = "sha256:f288924cae4e29463698d6d60bc6a4da69c89185ad1e0bcc4104f584e960b9ed", size = 142856, upload-time = "2026-05-14T19:25:27.735Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/a0/f4/c67b0b3f1b9245e8d266f0f112c500d50e5b4e83cb6f3b71b6528104182a

```



```

/requests-2.34.2-py3-none-any.whl", hash = "sha256:2a0d60c172f83ac6ab31e4554906c0f3b3588d37b5cb939b1c061f4907e278e0", s
ize = 73075, upload-time = "2026-05-14T19:25:26.443Z" },
]

[[package]]
name = "rpds-py"
version = "2026.5.1"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/2e/43/25a8dcd3feedd735039a8f0b5b7e3b118232b5eae288c4fd9ab200d4
1094/rpds_py-2026.5.1.tar.gz", hash = "sha256:07b24fea40541e28570e5b795a4a38fbdc12550c06bd0748005ecc8116ca256", size =
64459, upload-time = "2026-05-28T12:02:13.232Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/d4/e7/a78582dc57caa592dcc7d4fb69b61390561e908eb3d2f5df5928a8e354c0
/rpds_py-2026.5.1-cp312-cp312-macosx_10_12_x86_64.whl", hash = "sha256:3abe24a6657adcfa645d718063a5fa5103ecc71ddb726d7
8af8f9368018ff1d", size = 353040, upload-time = "2026-05-28T11:59:12.531Z" },
 { url = "https://files.pythonhosted.org/packages/a3/43/35e3f136343aef451e545ce8c38d36c2f93c0ed88703db8b64ba2b205c68
/rpds_py-2026.5.1-cp312-cp312-macosx_11_0_arm64.whl", hash = "sha256:58b1d94308dd0b1982f61f2eb54bf92997c9ece8a8093ef01
4250f4a517906c", size = 345775, upload-time = "2026-05-28T11:59:13.827Z" },
 { url = "https://files.pythonhosted.org/packages/20/e1/0f2160c5982d3157734d5cb3ed63d8b2d583a73c9864f77b666449f32cf8
/rpds_py-2026.5.1-cp312-cp312-manylinux_2_17_aarch64.manylinux2014_aarch64.whl", hash = "sha256:0fa92420128dadce7f54bd7
3ba1825a273e9268fe9e35dbdf7e6362890efa4e08", size = 376329, upload-time = "2026-05-28T11:59:15.271Z" },
 { url = "https://files.pythonhosted.org/packages/d0/11/ee0ba42aff83bf4effdbbc576673c6be64c5e173978c3f6d537e94482f77d
/rpds_py-2026.5.1-cp312-cp312-manylinux_2_17_armv7l.manylinux2014_armv7l.whl", hash = "sha256:ca653c6546386227cd9800d1b
ef6a348099ac8db4250341da6d90f663d6dfcb", size = 383539, upload-time = "2026-05-28T11:59:16.665Z" },
 { url = "https://files.pythonhosted.org/packages/11/df/d94aa6a499d4ac40afe2d7620f2c597fd3c0f182e854ad7cf3f596a81cb6
/rpds_py-2026.5.1-cp312-cp312-manylinux_2_17_ppc64le.manylinux2014_ppc64le.whl", hash = "sha256:66c93681c4729e4e3ecba31
b8179fae083ff3118841672835140338b4b9867c1", size = 494674, upload-time = "2026-05-28T11:59:17.991Z" },
 { url = "https://files.pythonhosted.org/packages/1f/75/33d30f43bb2f458de11979486a591b1bf6e5651765ed1704c6197c2dc773
/rpds_py-2026.5.1-cp312-cp312-manylinux_2_17_s390x.manylinux2014_s390x.whl", hash = "sha256:40fff257542e04796880e011e15c
d4dc21c2599975df2aaa8f2c8495ca574e1a5", size = 389268, upload-time = "2026-05-28T11:59:19.434Z" },
 { url = "https://files.pythonhosted.org/packages/f4/1e/2c9096fc19d5fd084b0184ca2b651e659aa0a37e6fdbecf6ece47f147fe1
/rpds_py-2026.5.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl", hash = "sha256:b6825cc329b290e93c5f6a9be
2393118a763f6ccf6abdb83704e0c102ca583644", size = 376280, upload-time = "2026-05-28T11:59:21Z" },
 { url = "https://files.pythonhosted.org/packages/b9/e5/61ec9f8be8211ea7f48448195549e4aaf02004083475493b0e137702ecb2
/rpds_py-2026.5.1-cp312-cp312-manylinux_2_31_riscv64.whl", hash = "sha256:de42116e69cb53b911cc34aee5ab98f36c597b8225450
45d49e938818b99e5e4", size = 387233, upload-time = "2026-05-28T11:59:22.454Z" },
 { url = "https://files.pythonhosted.org/packages/0d/ca/bcec1005c4f4a234f92a29078631fee49206c7265ccae966f18fd332e80e
/rpds_py-2026.5.1-cp312-cp312-manylinux_2_5_i686.manylinux1_i686.whl", hash = "sha256:c0f920015df2a504bebaba6d4c31ccf3f
cf942f92655c086da30b671aad19aa6", size = 405009, upload-time = "2026-05-28T11:59:23.845Z" },
 { url = "https://files.pythonhosted.org/packages/72/e6/4d5718c5cf26c522dc7c9999e238da1e77380b81d0c5d1df11e271ddfeb1
/rpds_py-2026.5.1-cp312-cp312-musllinux_1_2_aarch64.whl", hash = "sha256:0408a24e44feb919423cd6d9da677cb5cddb894d2ca9e7
63967d156d9c60fab4", size = 553113, upload-time = "2026-05-28T11:59:25.184Z" },
 { url = "https://files.pythonhosted.org/packages/d4/25/2ee807bdb3e1f0b7eddf7782acd5665a8b5205a331a7d7244a52c4812fd9
/rpds_py-2026.5.1-cp312-cp312-musllinux_1_2_i686.whl", hash = "sha256:cea68bcd53467561ae2f96a6bdad1544299ba97b5b0ddcd5a
c3d376e5c781c24", size = 618838, upload-time = "2026-05-28T11:59:26.749Z" },
 { url = "https://files.pythonhosted.org/packages/6a/c1/7d4c26f167f8c41501cc073d30ee22082b16ce358cf5b00ec97cbc7804ea
/rpds_py-2026.5.1-cp312-cp312-musllinux_1_2_x86_64.whl", hash = "sha256:4be8b1d2a705cc37d08256004e1d07de143fa0075c8e85a
3df020b776f62b732", size = 582436, upload-time = "2026-05-28T11:59:28.11Z" },
 { url = "https://files.pythonhosted.org/packages/04/1d/9d12b0a337bab46f4769f8857f4007e3b2d639e14f9a44a0efe157696e64
/rpds_py-2026.5.1-cp312-cp312-win32.whl", hash = "sha256:6736718bd44fc49cbcb538ba30516fdebef161522acefb739657d48b97bd864f
ed", size = 212734, upload-time = "2026-05-28T11:59:29.689Z" },
 { url = "https://files.pythonhosted.org/packages/c5/93/e4116f2de7f56bc7406a76033dc501811ddeb22b7f056b92d632871ebb0c
/rpds_py-2026.5.1-cp312-cp312-win_amd64.whl", hash = "sha256:0a7d1eec967df0e9b22614a5e177622e0c89611d03727fa0cb48e45028
907870", size = 229045, upload-time = "2026-05-28T11:59:31.033Z" },
 { url = "https://files.pythonhosted.org/packages/cb/53/6c3419d85eb2ec5938a37627c585b42d76a63bb731d6e42ed4b079ebf486
/rpds_py-2026.5.1-cp312-cp312-win_arm64.whl", hash = "sha256:1841d067089e117142d79b98aa0df2f08b52f2ecc1819dd2700636c0db
74a473", size = 223967, upload-time = "2026-05-28T11:59:32.318Z" },
 { url = "https://files.pythonhosted.org/packages/6c/32/14c961ad295f490eb0849ada8b79683e93a59b9de3afdd983eaf55fa6867
/rpds_py-2026.5.1-cp313-cp313-macosx_10_12_x86_64.whl", hash = "sha256:efef4ac29c6ff495531eb17ee705b62841ecaa291b7c7077
e848ea03e237164d", size = 352787, upload-time = "2026-05-28T11:59:33.655Z" },
 { url = "https://files.pythonhosted.org/packages/ca/bb/d1b85117967c11191441a7274ae616c65d93901d082c588f89a50a8da5ae
/rpds_py-2026.5.1-cp313-cp313-macosx_11_0_arm64.whl", hash = "sha256:c39f5b67a8a2e67179ada2a954227d670fe65fa9098457f698
f56ddf248709b3", size = 345179, upload-time = "2026-05-28T11:59:35Z" },
 { url = "https://files.pythonhosted.org/packages/7c/46/d84105f062e626a1b233f863907288a4708c2d833b8b4c6fb2764bc080c0
/rpds_py-2026.5.1-cp313-cp313-manylinux_2_17_aarch64.manylinux2014_aarch64.whl", hash = "sha256:b5c30f3f04eef4fbd362226
a6f31d7c8895ca4fbb6e0b790f6890a98d8da8559", size = 376173, upload-time = "2026-05-28T11:59:36.43Z" },
 { url = "https://files.pythonhosted.org/packages/e2/ae/469d7959ce5b1201e1de135dc735b86db3b35dd0d1734f6a44246d5f061c
/rpds_py-2026.5.1-cp313-cp313-manylinux_2_17_armv7l.manylinux2014_armv7l.whl", hash = "sha256:277f6c82f0580848796c7ecc8
a7173aa3bfb928e4ff831261c2f60a81dc270db", size = 383162, upload-time = "2026-05-28T11:59:37.995Z" },
 { url = "https://files.pythonhosted.org/packages/dc/a2/57853d31a1116a561aa072794602ad3f6341e18d70a8523f1bd5b9fc1e5a
/rpds_py-2026.5.1-cp313-cp313-manylinux_2_17_ppc64le.manylinux2014_ppc64le.whl", hash = "sha256:63c2c4c213f1a4e3f3de28e
cab029dbdee976324e729c0d7a55211be72576b02", size = 495093, upload-time = "2026-05-28T11:59:39.453Z" },
 { url = "https://files.pythonhosted.org/packages/99/63/3a8eabcd9314b7daf5c65f451d2c33d989235cd8a5762186cf2c3f5a4f8
/rpds_py-2026.5.1-cp313-cp313-manylinux_2_17_s390x.manylinux2014_s390x.whl", hash = "sha256:3350ec808fb538fe71a1f94dfaa
0e29c598dfad805ce49f0caec5ae3183c652b", size = 389829, upload-time = "2026-05-28T11:59:40.896Z" },
 { url = "https://files.pythonhosted.org/packages/4b/25/05678d97fc25e2622df14dc530fb82023174ecffff6733991ed0d78f167bd

```

/rpds\_py-2026.5.1-cp313-cp313-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl", hash = "sha256:b1b964e3ab599e718dc46c018d104b1ebc007cbc6567d827c94a687fca56d77e", size = 374786, upload-time = "2026-05-28T11:59:42.626Z" },  
 { url = "https://files.pythonhosted.org/packages/88/d1/8c90b6431e50a3b91b284a5c7c8c0c4f9c006444d90477a740d6e0f9c694/rpds\_py-2026.5.1-cp313-cp313-manylinux\_2\_31\_riscv64.whl", hash = "sha256:19cb09fab7b7f9c96b2a6e28f2e34b72a3705ff27b37ed b77455316e5d3f3dc9b", size = 386920, upload-time = "2026-05-28T11:59:44.124Z" },  
 { url = "https://files.pythonhosted.org/packages/ff/99/4638f672ab356682d633ee0da9255f5b67ce6efd0b85eb94ad3e255e65a5/rpds\_py-2026.5.1-cp313-cp313-manylinux\_2\_5\_i686.manylinux1\_i686.whl", hash = "sha256:abe76bcdba31e576cb83eeb8797aa0d88 2b738fef6dc65d0601fc753806a5b46", size = 405059, upload-time = "2026-05-28T11:59:47.177Z" },  
 { url = "https://files.pythonhosted.org/packages/66/3f/3546524b6eb4cc2e1f363a3d638fa52f6c24faae3500c25fb488b02f1740/rpds\_py-2026.5.1-cp313-cp313-musllinux\_1\_2\_aarch64.whl", hash = "sha256:8bfff073db3899158fff55ebf57b113a67030af26f80a1 8978f9f0aa60250ddf", size = 553030, upload-time = "2026-05-28T11:59:48.603Z" },  
 { url = "https://files.pythonhosted.org/packages/c6/c3/7b3388c796fc471bd17194242d4dc1a7608567c0fa422bcc1c5e79f9c1e/rpds\_py-2026.5.1-cp313-cp313-musllinux\_1\_2\_i686.whl", hash = "sha256:8ba264fa49b666cd9cc56bf34ec7002fb3d27a4ae5bcb4d 43d0d18feb1bb6f", size = 618975, upload-time = "2026-05-28T11:59:50.314Z" },  
 { url = "https://files.pythonhosted.org/packages/61/1e/a3cb07f2795075d1d88efddae2f541359fde5f08c81ee114c29c2949c90a/rpds\_py-2026.5.1-cp313-cp313-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:4860b603ddda0475a8885499b3729e90229d480105b4265 1962a5397d995fa89", size = 581178, upload-time = "2026-05-28T11:59:51.673Z" },  
 { url = "https://files.pythonhosted.org/packages/a1/74/e758c03a5ef46f04c37f2651a2893db846d569ba8a7bca469d4b58939bcd/rpds\_py-2026.5.1-cp313-cp313-win32.whl", hash = "sha256:7944270ae71383f6e2657dd7d5ce4eeb4ac2d0059a6738f0510583d462ab48 42", size = 212481, upload-time = "2026-05-28T11:59:53.148Z" },  
 { url = "https://files.pythonhosted.org/packages/70/ec/a2aca432db9c7359b40fa393eeaaa0d166c2f70175be956e75fa24197c44/rpds\_py-2026.5.1-cp313-cp313-win\_amd64.whl", hash = "sha256:88647f43a73c4e01be19b04ceef0c8d3a1958153604d13c773becd8016 f2a0cf", size = 228519, upload-time = "2026-05-28T11:59:54.505Z" },  
 { url = "https://files.pythonhosted.org/packages/29/60/a73bfdd45b096574556acf303bbd9fa9eed36ca8a818b514e2a5d5fe2b9d/rpds\_py-2026.5.1-cp313-cp313-win\_arm64.whl", hash = "sha256:453895624ecf7db7063b1004e44037522bbaef9ff6a945e59bc71662d7 a03abd", size = 223446, upload-time = "2026-05-28T11:59:56.081Z" },  
 { url = "https://files.pythonhosted.org/packages/18/e2/408105fd611823f00882aea810f3989a30d26b1bab8b6beb20f98c724e0e/rpds\_py-2026.5.1-cp313-cp313t-macosx\_10\_12\_x86\_64.whl", hash = "sha256:b4e4bc98639ec915f512fde3aa7a95e041d95d9c3cc86e ea841fa63cb1e8600", size = 355287, upload-time = "2026-05-28T11:59:57.448Z" },  
 { url = "https://files.pythonhosted.org/packages/8d/58/5c4a43436843c90d0f6dd19f82c200c80e3843ca9fa07b237623327f6d384/rpds\_py-2026.5.1-cp313-cp313t-macosx\_11\_0\_arm64.whl", hash = "sha256:cacedb7a6e167680acba45ad5716e89067d225dc80da0d704 0cae8c81d4572fa", size = 347033, upload-time = "2026-05-28T11:59:58.881Z" },  
 { url = "https://files.pythonhosted.org/packages/fb/c2/1a71acdaca4f4e259b10278fb87b039ded3cf80041bcd89dd8a3ea702ded6/rpds\_py-2026.5.1-cp313-cp313t-manylinux\_2\_17\_aarch64.manylinux2014\_aarch64.whl", hash = "sha256:68700371c5d7ae1412862d dfa719090925c93ecf351c566d66f09d04b136ea00", size = 376891, upload-time = "2026-05-28T12:00:00.516Z" },  
 { url = "https://files.pythonhosted.org/packages/c2/c8/535f3d9b65addd8e28aa87b83c6e526799c3717a88273db8ea795beee7a/rpds\_py-2026.5.1-cp313-cp313t-manylinux\_2\_17\_armv7l.manylinux2014\_armv7l.whl", hash = "sha256:296c799becfa849c779c8725 494fe9ed94959ed88678df4364b058465bad7f0", size = 385646, upload-time = "2026-05-28T12:00:02.394Z" },  
 { url = "https://files.pythonhosted.org/packages/1c/91/dc033f313345c354ade914dbe73cddb90b615a4409ea02430d5356794f3d8/rpds\_py-2026.5.1-cp313-cp313t-manylinux\_2\_17\_ppc64le.manylinux2014\_ppc64le.whl", hash = "sha256:d3858b908218ee108d0bbf b2095ccc237648053c9bf98affad7cb079acaf1d97", size = 498830, upload-time = "2026-05-28T12:00:04.189Z" },  
 { url = "https://files.pythonhosted.org/packages/27/fc/90fcbcea459dbb8ddc18a2e0fd1de9412b48bc84ffff2db771cf714bacfd6/rpds\_py-2026.5.1-cp313-cp313t-manylinux\_2\_17\_s390x.manylinux2014\_s390x.whl", hash = "sha256:4fb8d2e7cb2f850b169806d61d 1b991738acac96500a75c30f49caf064ce7cef", size = 392830, upload-time = "2026-05-28T12:00:05.797Z" },  
 { url = "https://files.pythonhosted.org/packages/b2/1d/46cd11a228c9750684a798d98f878be6f614aa762438da7378f035e79e35/rpds\_py-2026.5.1-cp313-cp313t-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl", hash = "sha256:27b74c10ed6a8f190f4287f5 3bcfea348b92a84a9c9f70d30183d1e6172d580d", size = 379613, upload-time = "2026-05-28T12:00:07.433Z" },  
 { url = "https://files.pythonhosted.org/packages/24/4a/d9b0c6af3a1de03eb93741bbe8be2bdce84d8fda8224f3005451d86df389/rpds\_py-2026.5.1-cp313-cp313t-manylinux\_2\_31\_riscv64.whl", hash = "sha256:b9a6528956191c48c52294a592dbd4a8386d70488bdb 5c0efcb6966466cd83", size = 388183, upload-time = "2026-05-28T12:00:09.227Z" },  
 { url = "https://files.pythonhosted.org/packages/c5/b4/db7aaabdda6d020a9cf87d981bcc2f57a434c7dec60ecfc2ab3dd50b20351/rpds\_py-2026.5.1-cp313-cp313t-manylinux\_2\_5\_i686.manylinux1\_i686.whl", hash = "sha256:af03e34e860047bc7a352b842856fcf7 8798fbb81132cc98bd2f907ab4eb9cd2", size = 408578, upload-time = "2026-05-28T12:00:10.779Z" },  
 { url = "https://files.pythonhosted.org/packages/08/d6/070f6a41cbb343e2ac4171859bf3f3623e0ab002f72619d6d505313ec2de/rpds\_py-2026.5.1-cp313-cp313t-musllinux\_1\_2\_aarch64.whl", hash = "sha256:fae6e836d10abbe191d557d33bd58bd5987725fe63aa1 eef557d230209855bd", size = 553573, upload-time = "2026-05-28T12:00:12.443Z" },  
 { url = "https://files.pythonhosted.org/packages/75/ab/1a71ea3589c4345dac0a0518f0e6a031cb42689277851b683c46d27463a5/rpds\_py-2026.5.1-cp313-cp313t-musllinux\_1\_2\_i686.whl", hash = "sha256:fc0c0f878ea770a0a8a462456c5ad36fc9fe6358e6b76fda dc7f17575e0b8bf1", size = 620861, upload-time = "2026-05-28T12:00:14.09Z" },  
 { url = "https://files.pythonhosted.org/packages/8a/22/9bf80a56069c0c443fcfefac639a86a744550a2898817a6dfd3e26654924/rpds\_py-2026.5.1-cp313-cp313t-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:e0b360f316d966b048b085857630b3cc51f3db2f07b06f 440eac8f695374d1e3", size = 585633, upload-time = "2026-05-28T12:00:15.66Z" },  
 { url = "https://files.pythonhosted.org/packages/da/68/3b2c0a75c9e04125696f84ebdbbf304acf5a40b58ba4481cbb98a922c3ba/rpds\_py-2026.5.1-cp313-cp313t-win32.whl", hash = "sha256:a2999883eedf72fdb7520b92c7d4ec2572a71ff40239377aa604cc529eec afc", size = 210074, upload-time = "2026-05-28T12:00:17.291Z" },  
 { url = "https://files.pythonhosted.org/packages/e7/8b/609157d5a25d37d4f29f92840ba531f416907c34ae5c5739dd21fc2bef98/rpds\_py-2026.5.1-cp313-cp313t-win\_amd64.whl", hash = "sha256:e07be2a9d7122bd6e82dea89814ef8dc893feb1aae97fec1630f3263b bb30e55", size = 228635, upload-time = "2026-05-28T12:00:18.73Z" },  
 { url = "https://files.pythonhosted.org/packages/d4/6f/19c19184b590d8de87e712e4abe4b3875771eff60216fb6153cf6665c68/rpds\_py-2026.5.1-cp314-cp314-macosx\_10\_12\_x86\_64.whl", hash = "sha256:1f2c391c3059798093b65df23aca2cac150460ae9c630d99 dec83d703d9485b9", size = 349756, upload-time = "2026-05-28T12:00:20.217Z" },  
 { url = "https://files.pythonhosted.org/packages/e5/60/a06fe7da34eca79dacbf958a2ba0c6eea85bc2b29de20080bf40f72f66fa/rpds\_py-2026.5.1-cp314-cp314-macosx\_11\_0\_arm64.whl", hash = "sha256:413b424f7c4ee65ab5e5be91f5731be0f8b41a1ee2b12dfe81 0d716312e95a78", size = 343831, upload-time = "2026-05-28T12:00:21.711Z" },  
 { url = "https://files.pythonhosted.org/packages/bf/ec/b233b97b90e2a6ef6ca8ad386ee284968e74bcfe113b3f1a8d9036429a9/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_aarch64.manylinux2014\_aarch64.whl", hash = "sha256:2c595a1d9255dce0599e131

30d1440ab2506654f2b50294226ee06402f8fef63", size = 375127, upload-time = "2026-05-28T12:00:23.326Z" },  
 { url = "https://files.pythonhosted.org/packages/14/7f/e00aae54067f2b488c4637961d5f58204d470795fc791085fa3f15060d2e/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_armv7l.manylinux2014\_armv7l.whl", hash = "sha256:1c27c5f6102eac8c03e7595a00827a53b271ba40a53b59ff8709170e0855ea4a", size = 379034, upload-time = "2026-05-28T12:00:24.89Z" },  
 { url = "https://files.pythonhosted.org/packages/be/cc/423999bbb8ae8dc93c77fc1d5e984ade5eb89d237d3bb884ccfa72ae2890/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_ppc64le.manylinux2014\_ppc64le.whl", hash = "sha256:6c7fc61d44cacecaf3aea542b0e053db77972a4573eceda16fb2b399161195", size = 490823, upload-time = "2026-05-28T12:00:26.676Z" },  
 { url = "https://files.pythonhosted.org/packages/0f/aa/c671bf660f12e68d3c52ff86c7066ed1372df5a0f4f2ff584e419b8207e7/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_s390x.manylinux2014\_s390x.whl", hash = "sha256:2c817a189d4ee14290420e5ff051e4dd6baa13f3edf84685071dee07a6d538ee", size = 388144, upload-time = "2026-05-28T12:00:28.577Z" },  
 { url = "https://files.pythonhosted.org/packages/19/c8/d63bb75b68afe77b229e3021c6031bcfa01da5db5b0e69d0d10f9ba679a7/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl", hash = "sha256:21846aac0ed2e0589f38c12dc44e77bb64e494b77leadbcf169cba00566ba7ba", size = 371959, upload-time = "2026-05-28T12:00:30.304Z" },  
 { url = "https://files.pythonhosted.org/packages/82/35/c51122014d8274ff37dc606d60049c3db7d83da02b5b282511e5a906a9a6/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_31\_riscv64.whl", hash = "sha256:b317c87a13f769a4e787819bd508aaa5d69aa09b0880de9af6d3a8a54571cdec", size = 383558, upload-time = "2026-05-28T12:00:31.764Z" },  
 { url = "https://files.pythonhosted.org/packages/e3/f9/2790cb99c136a5363acdeacf5c27c56f3de0d4118a1f48fca83404c99c89/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_5\_i686.manylinux1\_i686.whl", hash = "sha256:ce87129d9f2c14fa6c4a8601fb80eb4488c80d38a20cd13758ef11123e14995d", size = 402789, upload-time = "2026-05-28T12:00:33.247Z" },  
 { url = "https://files.pythonhosted.org/packages/e5/1b/e4fb584f8c75d35c38150ff6a332cda949e6f97acba1f4fd123b14ab56fe/rpds\_py-2026.5.1-cp314-cp314-musllinux\_1\_2\_aarch64.whl", hash = "sha256:9cddb6c1207d284d94fd1530ad5f7fbd797fe7c4b8704ba85f49414f2557e7d", size = 551405, upload-time = "2026-05-28T12:00:34.819Z" },  
 { url = "https://files.pythonhosted.org/packages/d8/f7/a6731b4216cb3793ea1af5391da240f5683dacc0d13e034fe5fc3503f240/rpds\_py-2026.5.1-cp314-cp314-musllinux\_1\_2\_i686.whl", hash = "sha256:4e237e139f94d3c036fd28eb9f564c99055476ff4ff05cd42be55ce349b5aa02", size = 616975, upload-time = "2026-05-28T12:00:36.268Z" },  
 { url = "https://files.pythonhosted.org/packages/2c/ea/2e051a81d95d8e63f4b35a1c463a87e8766bc3d083c067c5df6b6f220747/rpds\_py-2026.5.1-cp314-cp314-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:ed0954b524873214369184a9c82b0eaa45a3fbb9a798cd95b17e0d98499e7ea0", size = 578701, upload-time = "2026-05-28T12:00:37.82Z" },  
 { url = "https://files.pythonhosted.org/packages/65/56/b5f6fdb2083e32bca8a8993d89e70db114b4756c9e2c38421328126689d2/rpds\_py-2026.5.1-cp314-cp314-win32.whl", hash = "sha256:2d88621d6a7d4dfa633d21abe90f280bb205274e16b1d1e61c6ad4640b2453b7", size = 209806, upload-time = "2026-05-28T12:00:39.492Z" },  
 { url = "https://files.pythonhosted.org/packages/fb/80/65a5aa96c155e611d1ed844e4e1f57f3e36b021f396d9f8585d756e6b90d/rpds\_py-2026.5.1-cp314-cp314-win\_amd64.whl", hash = "sha256:cef8ac28d26f4dda3533060c20fbf80a325458fa9fd23ea72a73cdfa8e978838", size = 225985, upload-time = "2026-05-28T12:00:40.94Z" },  
 { url = "https://files.pythonhosted.org/packages/27/7c/ad185212e87b05f196daef92bc5f3caf07298eb47c295b5585c3dd3093ac/rpds\_py-2026.5.1-cp314-cp314-win\_arm64.whl", hash = "sha256:eaeea962c68cdc68d4a533ba985ab8e9484277910bbfaa2ab3ef7732667bfed8", size = 221219, upload-time = "2026-05-28T12:00:43.15Z" },  
 { url = "https://files.pythonhosted.org/packages/23/58/e14ae18759020334646b031e708ab4158d653a938822bfb7b95ef2e93aa3/rpds\_py-2026.5.1-cp314-cp314-macosx\_10\_12\_x86\_64.whl", hash = "sha256:21942f52dbbd5f8758bf021213d28bd45c39e873e65e2407faf5f1846f5761ad", size = 352148, upload-time = "2026-05-28T12:00:44.638Z" },  
 { url = "https://files.pythonhosted.org/packages/31/9b/5f4a1e2f960bca3ac5d052b139dd31eed97b259f9d909173821760d542e8/rpds\_py-2026.5.1-cp314-cp314-macosx\_11\_0\_arm64.whl", hash = "sha256:f414556f6e3958300ff941e40c9f97e3dc9774ddd1b3434c475d73dd354bbd3", size = 345196, upload-time = "2026-05-28T12:00:46.14Z" },  
 { url = "https://files.pythonhosted.org/packages/1a/71/1d9574d6a2fa20ab60eaa55c7467f5aa20c0bc770f341a05f09c0876f59e2/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_aarch64.manylinux2014\_aarch64.whl", hash = "sha256:ef1013a8625c74043210190b246f5b1551e09757c1f356c6e4160ef96c5bc081", size = 374981, upload-time = "2026-05-28T12:00:47.531Z" },  
 { url = "https://files.pythonhosted.org/packages/0c/9a/37e99f4915a80aa71670263c1267f7ae0af95f53a3f61e6c3bdc016d4515/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_armv7l.manylinux2014\_armv7l.whl", hash = "sha256:cc68e231a77a5f0d774ae278a1f8e55c0456501820847c1e4efb3829f3441df6", size = 379961, upload-time = "2026-05-28T12:00:49.216Z" },  
 { url = "https://files.pythonhosted.org/packages/a8/ff/6e73f74b89d2e0715e0fc86b7dde893f9a61ae2f9b256ff3bdfc41ac4e94/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_ppc64le.manylinux2014\_ppc64le.whl", hash = "sha256:9baffb505aff33acc69b422a19f77806680f3c8632227d79f48de8a810d1c2c5", size = 495965, upload-time = "2026-05-28T12:00:51.111Z" },  
 { url = "https://files.pythonhosted.org/packages/ea/e0/425faba25f59d74d4638b267f7c7a80e8649d2ef4db10a19b0c4a71e6e6f/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_s390x.manylinux2014\_s390x.whl", hash = "sha256:b8d2f912928d426e8cfa396f7f3f8d29a59e6689c86dcca3c420730c1096322b", size = 389526, upload-time = "2026-05-28T12:00:52.77Z" },  
 { url = "https://files.pythonhosted.org/packages/c6/76/7a41960e3fddae47fab43a28684d5da981401dfdd88253de0944148654cb/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl", hash = "sha256:90f628283be835db980c941767d41c9a27b5239e54ba0a9c1335247e82406964", size = 376190, upload-time = "2026-05-28T12:00:54.215Z" },  
 { url = "https://files.pythonhosted.org/packages/27/60/5f38dc70824fc6951b51d35377e577a3a3a4c81a6769cc5a2de25ebe0ad1/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_31\_riscv64.whl", hash = "sha256:1ebb2f0ab7e161323995a72de805170e0203df0c3dd22e1ef1cd1fdd90bd7a131", size = 383921, upload-time = "2026-05-28T12:00:55.673Z" },  
 { url = "https://files.pythonhosted.org/packages/60/1a/d60a38caa1505f4b9483c3fbbd12c94e1079154f4f401a6da96f7e77621/rpds\_py-2026.5.1-cp314-cp314-manylinux\_2\_5\_i686.manylinux1\_i686.whl", hash = "sha256:f3dfd3d16ded76f1f8c9cdebd0e1ea55f4f4c23b812de189814da7cf229c22a81", size = 404766, upload-time = "2026-05-28T12:00:57.518Z" },  
 { url = "https://files.pythonhosted.org/packages/87/ff/602fd3f174d6425f0bce05ad0dfbec0e96b38d0f7d08a79af5aa20083885/rpds\_py-2026.5.1-cp314-cp314-musllinux\_1\_2\_aarch64.whl", hash = "sha256:9af8905b8f854990e40d5206aa5ac58d9b0fe0b7f351ff2bb086c20f6c8c6a47", size = 551343, upload-time = "2026-05-28T12:00:58.978Z" },  
 { url = "https://files.pythonhosted.org/packages/b8/c1/1be13327acdbead3eca1fde03b6a34dbb011f1e864e217f0d32cc1779a7f/rpds\_py-2026.5.1-cp314-cp314-musllinux\_1\_2\_i686.whl", hash = "sha256:036a36a87fb1cd3b214d11c4b3c4f7d2ddad933625dca1c900b56a057c07740a", size = 618502, upload-time = "2026-05-28T12:01:00.656Z" },  
 { url = "https://files.pythonhosted.org/packages/f3/d7/afb49b49d7f2be8b7ba1a9f0977fa5168003437b93086726f066544e8351/rpds\_py-2026.5.1-cp314-cp314-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:62ae3853454fe9ef283a03c96c2d835d39e84b14643a9d62c82ef0fb87d02ca", size = 581916, upload-time = "2026-05-28T12:01:02.22Z" },  
 { url = "https://files.pythonhosted.org/packages/25/d1/dbef8c1f8a10f07be6b2b5f054e20099fd9924b3ec001b8f0b6ac7813a85/rpds\_py-2026.5.1-cp314-cp314-win32.whl", hash = "sha256:6c3d771a46ec18b12af06ce36243a9a80b07a5d0515236332d90863ca8bb326a", size = 207855, upload-time = "2026-05-28T12:01:03.821Z" },

{ url = "https://files.pythonhosted.org/packages/2a/72/bfa4e61ab8e7dc1c8adf397e05e6cbdd4239357bd72b248d3de662f23915/rpds\_py-2026.5.1-cp314-win\_amd64.whl", hash = "sha256:c93c629be4636cf54337bd5f06c104d55e42ced54d681f6fe21ae510a65116f6", size = 225422, upload-time = "2026-05-28T12:01:05.194Z" },

{ url = "https://files.pythonhosted.org/packages/27/3a/7b5da92b640f67b6717ccafc83cdd06bfa7ff2395c3685c68922bb54d703/rpds\_py-2026.5.1-cp315-cp315-macosx\_10\_12\_x86\_64.whl", hash = "sha256:3574b55c604b8f75dacb007136508bbcd0b406e626301778096a133327e7f2fb", size = 349576, upload-time = "2026-05-28T12:01:06.722Z" },

{ url = "https://files.pythonhosted.org/packages/d7/8a/2aafd7ad355a1bd48ca76e2262b74b15e6432b5a1efe150efd4d779cd55d/rpds\_py-2026.5.1-cp315-cp315-macosx\_11\_0\_arm64.whl", hash = "sha256:94068eb3ae6d43f5a786b7db96a406a34e6d5c24489feef32fd6e8946ea7b291", size = 343640, upload-time = "2026-05-28T12:01:08.441Z" },

{ url = "https://files.pythonhosted.org/packages/f7/7d/6c9523c1abbe840a1b7fba3c516d48e1d3487cc80fea4366c4071cf56784/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_aarch64.manylinux2014\_aarch64.whl", hash = "sha256:f3a5b10e8ce894825f380a8f1b6444cf73c294dfea62afbb2d13e3a9e630cec1", size = 375322, upload-time = "2026-05-28T12:01:09.934Z" },

{ url = "https://files.pythonhosted.org/packages/5a/5d/0b7b03fb1dc509321f01de3149784ab773e34c8573022029af8076afcb9c/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_armv7l.manylinux2014\_armv7l.whl", hash = "sha256:fc09f82e63d4bcd58149572f857a431bae851dc747e313c3b5bdf7abb907fda8", size = 379066, upload-time = "2026-05-28T12:01:11.48Z" },

{ url = "https://files.pythonhosted.org/packages/d7/e2/8ef6012999ebf1cb1c22f876d9ce5e63d960fd4631d2af3202d3f480aa25/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_ppc64le.manylinux2014\_ppc64le.whl", hash = "sha256:e10464d17df3b582745c25cec695cb9558bca2c6b6db631aeef787fc72c767b2", size = 494586, upload-time = "2026-05-28T12:01:13.051Z" },

{ url = "https://files.pythonhosted.org/packages/80/af/1eeb029bec67582c226b7809172207cd005073af4ebd906e65ff494f4983/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_s390x.manylinux2014\_s390x.whl", hash = "sha256:ba05adbff15d994c38ec0b7ab32e858e5110c21e9009a00a86545fd220f84e038", size = 388415, upload-time = "2026-05-28T12:01:14.631Z" },

{ url = "https://files.pythonhosted.org/packages/18/23/ffbe10711c4d766c1cab0557d6906c074f795814863c67b351355d29354a/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl", hash = "sha256:77c004fdc7b891967106f78ddfd7b076bf6e813c6139c6ffff6aed3bcaa960b26", size = 372427, upload-time = "2026-05-28T12:01:16.153Z" },

{ url = "https://files.pythonhosted.org/packages/bd/3a/30ba4a6ad457e5b070c18d742a33fb77d8d922b565cc881f8a5313d63bfe/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_31\_riscv64.whl", hash = "sha256:83bcf894486c9d78dd290d3c0124ff6dd8875d3025e2090a8ec49fcc37c55fdd", size = 383615, upload-time = "2026-05-28T12:01:17.809Z" },

{ url = "https://files.pythonhosted.org/packages/d3/69/62e242b53ce39c0814bd24e1a6e6eba6c92be716277745f317f9540a2e7b/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_5\_i686.manylinux1\_i686.whl", hash = "sha256:c3df104083952a0e0c6f10de33e440eab e98fb6317d23e1a58c68f6df08d01b9", size = 402786, upload-time = "2026-05-28T12:01:19.419Z" },

{ url = "https://files.pythonhosted.org/packages/38/c1/a770b9c186928a1ed0f7e6d7ae50e7f3950ed23e3f9e366dbc8e38cb55de/rpds\_py-2026.5.1-cp315-cp315-musllinux\_1\_2\_aarch64.whl", hash = "sha256:980450826cf22e133c57e0835070bdd0dd3f73b9b708c3ce223def2cb9469e14", size = 551583, upload-time = "2026-05-28T12:01:21.013Z" },

{ url = "https://files.pythonhosted.org/packages/21/7c/68e8579b95375b70d2a963103c42e705856cdb98569258bd807f4423891c/rpds\_py-2026.5.1-cp315-cp315-musllinux\_1\_2\_i686.whl", hash = "sha256:205dde846f24332ab0c1188699a043b8d165b79bb84529ce272c45048ff6be01", size = 616941, upload-time = "2026-05-28T12:01:22.548Z" },

{ url = "https://files.pythonhosted.org/packages/70/a1/a6135aed5730ff03ab957182259987ac11e55fb392a28dc6f0592048a280/rpds\_py-2026.5.1-cp315-cp315-musllinux\_1\_2\_x86\_64.whl", hash = "sha256:3966b82dd563176396df030f3dd52a6e54cb69b718e95e78bd555ed3d1e0185d", size = 578349, upload-time = "2026-05-28T12:01:24.118Z" },

{ url = "https://files.pythonhosted.org/packages/09/6e/f24201a76a84e6c49d0bdfdfcb735210e21701e9b21c5bfc0ba497dd62f6/rpds\_py-2026.5.1-cp315-cp315-win32.whl", hash = "sha256:7818f8d0a415be74d2be3590b0a1c1f463a642f4d0217e7d10602dceef5b79aa", size = 209922, upload-time = "2026-05-28T12:01:25.522Z" },

{ url = "https://files.pythonhosted.org/packages/9e/e4/966bc240bb0485fc265278f6de44d05834bf0b3618886e0b22e33d54c49a/rpds\_py-2026.5.1-cp315-cp315-win\_amd64.whl", hash = "sha256:b3cc20cd0800af78fd0fac68086e28c1856cec51ea528bb81ea851aa40d39325", size = 226003, upload-time = "2026-05-28T12:01:27.062Z" },

{ url = "https://files.pythonhosted.org/packages/5c/5c/a15a59269cd5e74472734516c73795c15eccfc841b3d4b0228c3f53f19d0/rpds\_py-2026.5.1-cp315-cp315-win\_arm64.whl", hash = "sha256:3609e9939a8a76cd904cf98a3f1f13b5dc7e150adeaee89e0ea09652ea213e16", size = 221245, upload-time = "2026-05-28T12:01:28.51Z" },

{ url = "https://files.pythonhosted.org/packages/e0/22/135ce03804e179a71ceb13be095deda4a279bc88f7a6b8fa161c5ad44e12/rpds\_py-2026.5.1-cp315-cp315-macosx\_10\_12\_x86\_64.whl", hash = "sha256:5d333a7127d4b307601ac37792bee01bb95c867cbfacf21b6375b804d6bbd723", size = 352015, upload-time = "2026-05-28T12:01:30.214Z" },

{ url = "https://files.pythonhosted.org/packages/3b/5f/f1f6d2652eb9d848f6eb369d8db83a2da6249bb49ad2c2a48f45d54538d3/rpds\_py-2026.5.1-cp315-cp315-macosx\_11\_0\_arm64.whl", hash = "sha256:b5f077b44a4f7808520f66dae234988d867deb9aed9be5da057ce9ba831b2a41", size = 345016, upload-time = "2026-05-28T12:01:31.656Z" },

{ url = "https://files.pythonhosted.org/packages/88/66/b74182775691ea2290c99e52ac8d5db844e56fbec90ce421f107658c8314/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_aarch64.manylinux2014\_aarch64.whl", hash = "sha256:55d8f9b7b78c9538fc9e04e82ec0e888ff9cc3cfcfad152c77e57cd09351a98a", size = 374775, upload-time = "2026-05-28T12:01:33.136Z" },

{ url = "https://files.pythonhosted.org/packages/ff/8f/15e5a61d9f0a43902d36561d4f07cae6ae9f4716be825159fd72717f33af/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_armv7l.manylinux2014\_armv7l.whl", hash = "sha256:e3a8ae58895ac107ed934a6bf51e5846f95c53b9b940c2c6d310838fd5846358", size = 380270, upload-time = "2026-05-28T12:01:34.574Z" },

{ url = "https://files.pythonhosted.org/packages/02/c3/f859b12763a80540cdf2af0f15b19904cf756a71d7bdd3f82ff3e5b1bbf9/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_ppc64le.manylinux2014\_ppc64le.whl", hash = "sha256:0957cf3c2b8632ec7aaebf febea8005b353cc2a237b6e2ae3c2cac0820704cfb", size = 495285, upload-time = "2026-05-28T12:01:36.127Z" },

{ url = "https://files.pythonhosted.org/packages/1c/c7/ff27c2ac8411d30b03b1829fd88cae8dad1a4d0da48dd25e57c4038042e6/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_s390x.manylinux2014\_s390x.whl", hash = "sha256:c396c1304de421050b3681ea70f371874b54d41b0151e96109758144c231e30b", size = 389581, upload-time = "2026-05-28T12:01:37.635Z" },

{ url = "https://files.pythonhosted.org/packages/6e/67/fe92ee32a6cc05c77228a2f8b1762e7124f386ec20ff83d0757b762d58d0/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl", hash = "sha256:aad1bfff7f666b9598e573815affd666aac6a13a585dde336f843e3350c7fad", size = 376041, upload-time = "2026-05-28T12:01:39.307Z" },

{ url = "https://files.pythonhosted.org/packages/f8/91/b4d6685c27aba55bd82f25b278be8237038117d05f9659a6213ad3408130/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_31\_riscv64.whl", hash = "sha256:656a042550878f12d45752452d47094b7cfe5ad1e9d7b87b5a22ad3ae5ff8015", size = 383946, upload-time = "2026-05-28T12:01:41.043Z" },

{ url = "https://files.pythonhosted.org/packages/bd/79/2c1d832a53c8e0f8e98fc970ec257b950fec4d4f62be2ab7182b500a0cbcb/rpds\_py-2026.5.1-cp315-cp315-manylinux\_2\_5\_i686.manylinux1\_i686.whl", hash = "sha256:73c4bd4f70294737b5206a3e8e30ccadbf8a60301831c8ea23eec5dbeea1ecfa", size = 405526, upload-time = "2026-05-28T12:01:43.032Z" },

{ url = "https://files.pythonhosted.org/packages/78/c4/c98117b03c6a8581ab2c2dfccfe9a5ad82bd8128a3c28b46a6ad2d97c393

```

/rpds_py-2026.5.1-cp315-cp315t-musllinux_1_2_aarch64.whl", hash = "sha256:43bca78665423cabae77146f2fe7ce55272b6c8d55d82
cca83effd42c7e13972", size = 551165, upload-time = "2026-05-28T12:01:44.648Z" },
{ url = "https://files.pythonhosted.org/packages/3b/c1/bc479ca069200af730881b1bd525e3114b2b391a351509fcb1b772f28086
/rpds_py-2026.5.1-cp315-cp315t-musllinux_1_2_i686.whl", hash = "sha256:42d0f20e85e549c870749d0e247f0c10d318a45b7e9676d5
75d2dc04a1b2e66", size = 618778, upload-time = "2026-05-28T12:01:46.337Z" },
{ url = "https://files.pythonhosted.org/packages/77/65/38ab2f90df44c2febfb63cc10ced40763d9b4bc94d173e734528663fe7f5
/rpds_py-2026.5.1-cp315-cp315t-musllinux_1_2_x86_64.whl", hash = "sha256:b1be5c35683684d5331b93600c210e8367c254683d8a6d
f6bd21bd2da3a334fb", size = 581839, upload-time = "2026-05-28T12:01:48.109Z" },
{ url = "https://files.pythonhosted.org/packages/15/2d/ce1f605fe036aadd460e5822e578c6c7ec3a860936cca37d6e0f299daa77
/rpds_py-2026.5.1-cp315-cp315t-win32.whl", hash = "sha256:75808f6c38ce7749bb68cc2770161aae5045e6c6f6781a9782e74b9330439
9df", size = 207866, upload-time = "2026-05-28T12:01:49.648Z" },
{ url = "https://files.pythonhosted.org/packages/79/cb/966040123eb102371559746908ef2c9471f4d43e17ec9a645a2258dab64b
/rpds_py-2026.5.1-cp315-cp315t-win_amd64.whl", hash = "sha256:90bd6630002a1c7f09e7843dd79f0d24f3d2897cc25a753480917865d
14f15b3", size = 225441, upload-time = "2026-05-28T12:01:51.408Z" },
]

```

```

[[package]]
name = "six"
version = "1.17.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/94/e7/b2c673351809dca68a0e064b6af791aa332cf192da575fd474ed7d6f
16a2/six-1.17.0.tar.gz", hash = "sha256:ff70335d468e7eb6ec65b95b99d3a2836546063f63acc5171de367e834932a81", size = 34031
, upload-time = "2024-12-04T17:35:28.174Z" }
wheels = [
{ url = "https://files.pythonhosted.org/packages/b7/ce/149a00dd41f10bc29e5921b496af8b574d8413afcd5e30dfa0ed46c2cc5e
/six-1.17.0-py2.py3-none-any.whl", hash = "sha256:4721f391ed90541fddacab5acf947aa0d3dc7d27b2e1e8eda2be8970586c3274", si
ze = 11050, upload-time = "2024-12-04T17:35:26.475Z" },
]

```

```

[[package]]
name = "smmap"
version = "5.0.3"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/1f/ea/49c993d6dfdd7338c9b1000a0f36817ed7ec84577ae2e52f890d1a4f
f909/smmap-5.0.3.tar.gz", hash = "sha256:4d9debb8b99007ae47165abc08670bd74cb74b5227dda7f643eccc4e9eb5642c", size = 2250
6, upload-time = "2026-03-09T03:43:26.12Z" }
wheels = [
{ url = "https://files.pythonhosted.org/packages/c1/d4/59e74daffcb57a07668852eeeb6035af9f32cbfd7a1d2511f17d2fe6a738
/smmap-5.0.3-py3-none-any.whl", hash = "sha256:c106e05d5a61449cf6ba9a1e650227ecfb141590d2a98412103ff35d89fc7b2f", size
= 24390, upload-time = "2026-03-09T03:43:24.361Z" },
]

```

```

[[package]]
name = "starlette"
version = "1.3.1"
source = { registry = "https://pypi.org/simple" }
dependencies = [
{ name = "anyio" },
{ name = "typing-extensions", marker = "python_full_version < '3.13'" },
]
sdist = { url = "https://files.pythonhosted.org/packages/eb/e3/7c1dc7381d9f8ab7d854328ebfa884e62cb3f3d8549ddfd37c7814f4
2afa/starlette-1.3.1.tar.gz", hash = "sha256:05d0213193f2fbaae60e2ecb593b4add4262ad4e46536b54abe36f11a71724e0", size =
2703240, upload-time = "2026-06-12T09:23:11.602Z" }
wheels = [
{ url = "https://files.pythonhosted.org/packages/ec/bb/2799cc2ede3ed41131f8975621e7213dfc7ef4acbbaadfa440f32500c370
/starlette-1.3.1-py3-none-any.whl", hash = "sha256:c7372aae11c3c3f26a42df7bd626cec2f47d03483d261d369516a615a53714c6", s
ize = 73632, upload-time = "2026-06-12T09:23:10.017Z" },
]

```

```

[[package]]
name = "streamlit"
version = "1.58.0"
source = { registry = "https://pypi.org/simple" }
dependencies = [
{ name = "altair" },
{ name = "anyio" },
{ name = "blinker" },
{ name = "cachetools" },
{ name = "click" },
{ name = "gitpython" },
{ name = "httptools" },
{ name = "itsdangerous" },
{ name = "numpy" },
{ name = "packaging" },
{ name = "pandas" },
{ name = "pillow" },

```

```

 { name = "protobuf" },
 { name = "pyarrow" },
 { name = "pydeck" },
 { name = "python-multipart" },
 { name = "requests" },
 { name = "starlette" },
 { name = "tenacity" },
 { name = "toml" },
 { name = "typing-extensions" },
 { name = "uvicorn" },
 { name = "watchdog", marker = "sys_platform != 'darwin'" },
 { name = "websockets" },
]
sdist = { url = "https://files.pythonhosted.org/packages/91/74/20dac6d6200d6ec0e1c230fb8eeb6a1a423645eacb76e8d802adfc246456/streamlit-1.58.0.tar.gz", hash = "sha256:78a22e7085b053af7ce544442bf4b670771e68c509ba1bdaa056ba0708f49c3d", size = 8721149, upload-time = "2026-05-28T18:02:44.606Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/df/84/14c36a92fb24f8e1cea452f53b0744b5da69d52cdd2fe22e71e6fbf765d5/streamlit-1.58.0-py3-none-any.whl", hash = "sha256:4ca8a7afc5bd16a5f176ccf4be1e34e8121cad0240becd127fb58a103ea3178d", size = 9219185, upload-time = "2026-05-28T18:02:41.993Z" },
]

[[package]]
name = "tenacity"
version = "9.1.4"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/47/c6/ee486fd809e357697ee8a44d3d69222b344920433d3b6666ccd9b374630c/tenacity-9.1.4.tar.gz", hash = "sha256:adb31d4c263f2bd041081ab33b498309a57c77f9acf2db65aadf0898179cf93a", size = 49413, upload-time = "2026-02-07T10:45:33.841Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/d7/c1/eb8f9debc45d3b7918a32ab756658a0904732f75e555402972246b0b8e71/tenacity-9.1.4-py3-none-any.whl", hash = "sha256:6095a360c919085f28c6527de529e76a06ad89b23659fa881ae0649b867a9d55", size = 28926, upload-time = "2026-02-07T10:45:32.24Z" },
]

[[package]]
name = "toml"
version = "0.10.2"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/be/ba/1f744cdc819428fc6b5084ec34d9b30660f6f9daaf70eead706e3203ec3c/toml-0.10.2.tar.gz", hash = "sha256:b3bda1d108d5dd99f4a20d24d9c348e91c4db7ab1b749200bde2f839ccbe68f", size = 22253, upload-time = "2020-11-01T01:40:22.204Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/44/6f/7120676b6d73228c96e17f1f794d8ab046fc910d781c8d151120c3f1569e/toml-0.10.2-py2.py3-none-any.whl", hash = "sha256:806143ae5bfb6a3c6e736a764057db0e6a0e05e338b5630894a5f779cabb4f9b", size = 16588, upload-time = "2020-11-01T01:40:20.672Z" },
]

[[package]]
name = "typing-extensions"
version = "4.15.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/72/94/1a15dd82efb362ac84269196e94cf00f187f7ed21c242792a923cdb1c61f/typing_extensions-4.15.0.tar.gz", hash = "sha256:0cea48d173cc12fa28ecabc3b837ea3cf6f38c6d1136f85cbaaf598984861466", size = 109391, upload-time = "2025-08-25T13:49:26.313Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/18/67/36e9267722cc04a6b9f15c7f3441c2363321a3ea07da7ae0c0707beeb2a9c/typing_extensions-4.15.0-py3-none-any.whl", hash = "sha256:f0fa19c6845758ab08074a0cfa8b7aecb71c999ca73d62883bc25cc018c4e548", size = 44614, upload-time = "2025-08-25T13:49:24.86Z" },
]

[[package]]
name = "typing-inspection"
version = "0.4.2"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "typing-extensions" },
]
sdist = { url = "https://files.pythonhosted.org/packages/55/e3/70399cb7dd41c10ac53367ae42139cf4b1ca5f36bb3dc6c9d33acdb43655/typing_inspection-0.4.2.tar.gz", hash = "sha256:ba561c48a67c5958007083d386c3295464928b01faa735ab8547c5692e87f464", size = 75949, upload-time = "2025-10-01T02:14:41.687Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/dc/9b/47798a6c91d8bdb567fe2698fe81e0c6b7cb7ef4d13da4114b41d239f65d/typing_inspection-0.4.2-py3-none-any.whl", hash = "sha256:4ed1cacbdc298c220f1bd249ed5287caa16f34d44ef4e9c3d0cbad5b521545e7", size = 14611, upload-time = "2025-10-01T02:14:40.154Z" },
]

```

```

[[package]]
name = "tzdata"
version = "2026.2"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/ba/19/1b9b0e29f30c6d35cb345486df41110984ea67ae69dddbc0e8a10099
9493/tzdata-2026.2.tar.gz", hash = "sha256:9173fde7d80d9018e02a662e168e5a2d04f87c41ea174b139fbef642eda62d10", size = 19
8254, upload-time = "2026-04-24T15:22:08.651Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/ce/e4/dccd7f47c4b64213ac01ef921a1337ee6e30e8c6466046018326977efd95
/tzdata-2026.2-py2.py3-none-any.whl", hash = "sha256:bbe9af844f658da81a5f95019480da3a89415801f6cc966806612cc7169bffe7",
size = 349321, upload-time = "2026-04-24T15:22:05.876Z" },
]

[[package]]
name = "urllib3"
version = "2.7.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/53/0c/06f8b233b8fd13b9e5ee11424ef85419ba0d8ba0b3138bf360be2ff5
6953/urllib3-2.7.0.tar.gz", hash = "sha256:231e0ec3b63ceb14667c67be60f2f2c40a518cb38b03af60abc813da26505f4c", size = 43
1284, upload-time = "2026-05-07T16:13:18.596Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/7f/3e/5db95bcf282c52709639744ca2a8b149baccf648e39c8cc87553df9eae0c
/urllib3-2.7.0-py3-none-any.whl", hash = "sha256:9fb4c81ebbb1ce9531cce37674bbc6f1360472bc18ca9a553ede278ef7276897", siz
e = 131087, upload-time = "2026-05-07T16:13:17.151Z" },
]

[[package]]
name = "uvicorn"
version = "0.49.0"
source = { registry = "https://pypi.org/simple" }
dependencies = [
 { name = "click" },
 { name = "h11" },
]
sdist = { url = "https://files.pythonhosted.org/packages/c4/1f/fa18009dea8469069cca78a4e877a008ab78f08b064bfc9ab8915790
77ff/uvicorn-0.49.0.tar.gz", hash = "sha256:ebf4271aa580d9de97f93192d4595176df6e91f9aae919ca73e4fc07df1e66a3", size = 9
1284, upload-time = "2026-06-03T22:01:30.448Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/88/fa/e1388bbcf24ef3274f45c0c1c7b501fd14971037c1b6ee23610553307497
/uvicorn-0.49.0-py3-none-any.whl", hash = "sha256:ba3d14c3ee7e41c6c654c46c9eb489d33213cdd30aa1696eab1374337c13f68f", si
ze = 71376, upload-time = "2026-06-03T22:01:29.037Z" },
]

[[package]]
name = "watchdog"
version = "6.0.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/db/7d/7f3d619e951c88ed75c6037b246ddcf2d322812ee8ea189be8951172
1d54/watchdog-6.0.0.tar.gz", hash = "sha256:9ddf7c82fda3ae8e24decda1338ede66e1c99883db93711d8fb941eaa2d8c282", size = 1
31220, upload-time = "2024-11-01T14:07:13.037Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/a9/c7/ca4bf3e518cb57a686b2feb4f55a1892fd9a3dd13f470fca14e00f80ea36
/watchdog-6.0.0-py3-none-manylinux2014_aarch64.whl", hash = "sha256:7607498efa04a3542ae3e05e64da8202e58159aa1fa4acddf76
78d34a35d4f13", size = 79079, upload-time = "2024-11-01T14:06:59.472Z" },
 { url = "https://files.pythonhosted.org/packages/5c/51/d46dc9332f9a647593c947b4b88e2381c8dfc0942d15b8edc0310fa4abb1
/watchdog-6.0.0-py3-none-manylinux2014_armv7l.whl", hash = "sha256:9041567ee8953024c83343288ccc458fd0a2d811d6a0fd68c4c2
2609e3490379", size = 79078, upload-time = "2024-11-01T14:07:01.431Z" },
 { url = "https://files.pythonhosted.org/packages/d4/57/04edbf5e169cd318d5f07b4766fee38e825d64b6913ca157ca32d1a42267
/watchdog-6.0.0-py3-none-manylinux2014_i686.whl", hash = "sha256:82dc3e3143c7e38ec49d61af98d6558288c415eac98486a5c58172
6e0737c00e", size = 79076, upload-time = "2024-11-01T14:07:02.568Z" },
 { url = "https://files.pythonhosted.org/packages/ab/cc/da8422b300e13cb187d2203f20b9253e91058aaf7db65b74142013478e66
/watchdog-6.0.0-py3-none-manylinux2014_ppc64.whl", hash = "sha256:212ac9b8bf1161dc91bd09c048048a95ca3a4c4f5e5d4a7d1b1a7
d5752a7f96f", size = 79077, upload-time = "2024-11-01T14:07:03.893Z" },
 { url = "https://files.pythonhosted.org/packages/2c/3b/b8964e04ae1a025c44ba8e4291f86e97fac443bca31de8bd98d3263d2fcf
/watchdog-6.0.0-py3-none-manylinux2014_ppc64le.whl", hash = "sha256:e3df4cbb9a450c6d49318f6d14f4bbc80d763fa587ba46ec86f
99f9e6876bb26", size = 79078, upload-time = "2024-11-01T14:07:05.189Z" },
 { url = "https://files.pythonhosted.org/packages/62/ae/a696eb424bedff7407801c257d4b1afda455fe40821a2be430e173660e81
/watchdog-6.0.0-py3-none-manylinux2014_s390x.whl", hash = "sha256:2cce7cfc2008eb51feb6aab51251fd79b85d9894e98ba847408f6
62b3395ca3c", size = 79077, upload-time = "2024-11-01T14:07:06.376Z" },
 { url = "https://files.pythonhosted.org/packages/b5/e8/dbf020b4d98251a9860752a094d09a65e1b436ad181faf929983f697048f
/watchdog-6.0.0-py3-none-manylinux2014_x86_64.whl", hash = "sha256:20ffe5b202af80ab4266dcd3e91aae72bf2da48c0d33bdb15c66
658e685e94e2", size = 79078, upload-time = "2024-11-01T14:07:07.547Z" },
 { url = "https://files.pythonhosted.org/packages/07/f6/d0e5b343768e8bcb4cda79f0f2f5501bf26177ecd5651f84c07567461cf
/watchdog-6.0.0-py3-none-win32.whl", hash = "sha256:07df1fdd701c5d4c8e55ef6cf55b8f0120fe1aef7ef39a1c6fc6bc2e606d517a",
size = 79065, upload-time = "2024-11-01T14:07:09.525Z" },
]

```

```

 { url = "https://files.pythonhosted.org/packages/db/d9/c495884c6e548fce18a8f40568ff120bc3a4b7b99813081c8ac0c936fa64
/watchdog-6.0.0-py3-none-win_amd64.whl", hash = "sha256:cbafb470cf848d93b5d013e2ecb245d4aa1c8fd0504e863ccefa32445359d68
0", size = 79070, upload-time = "2024-11-01T14:07:10.686Z" },
 { url = "https://files.pythonhosted.org/packages/33/e8/e40370e6d74ddb447f002a32919d91310d6074130fe4e17dabcafc15cbf1
/watchdog-6.0.0-py3-none-win_ia64.whl", hash = "sha256:a1914259fa9e1454315171103c6a30961236f508b9b623eae470268bbcc6a22f
", size = 79067, upload-time = "2024-11-01T14:07:11.845Z" },
]

[[package]]
name = "websockets"
version = "16.0"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/packages/04/24/4b2031d72e840ce4c1ccb255f693b15c334757fc50023e4db9537080
b8c4/websockets-16.0.tar.gz", hash = "sha256:5f6261a5e56e8d5c42a4497b364ea24d94d9563e8fbd44e78ac40879c60179b5", size =
179346, upload-time = "2026-01-10T09:23:47.181Z" }
wheels = [
 { url = "https://files.pythonhosted.org/packages/84/7b/bac442e6b96c9d25092695578dda82403c77936104b5682307bd4deb1ad4
/websockets-16.0-cp312-cp312-macosx_10_13_universal2.whl", hash = "sha256:71c989cbf3254fbd5e84d3bfff31e4da39c43f884e64f2
551d14bb3c186230f00", size = 177365, upload-time = "2026-01-10T09:22:46.787Z" },
 { url = "https://files.pythonhosted.org/packages/b0/fe/136ccece61bd690d9c1f715baaeefd953bb2360134de73519d5df19d29ca
/websockets-16.0-cp312-cp312-macosx_10_13_x86_64.whl", hash = "sha256:8b6e209ffee39ff1b6d0fa7bfe6de950c60dfb91b8fced1
7da4ee539121a79", size = 175038, upload-time = "2026-01-10T09:22:47.999Z" },
 { url = "https://files.pythonhosted.org/packages/40/1e/9771421ac2286eaa9b5b8575b0cb701ae3663abf8b5e1f64f1fd90d0a673
/websockets-16.0-cp312-cp312-macosx_11_0_arm64.whl", hash = "sha256:86890e837d61574c92a97496d590968b23c2ef0aeb8a9bc9421
d174cd378ae39", size = 175328, upload-time = "2026-01-10T09:22:49.809Z" },
 { url = "https://files.pythonhosted.org/packages/18/29/71729b4671f21e1eaa5d6573031ab810ad2936c8175f03f97f3ff164c802
/websockets-16.0-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl", hash = "sha256:9b5aca38
b67492ef518a8ab76851862488a478602229112c4b0d58d63a7a4d5c", size = 184915, upload-time = "2026-01-10T09:22:51.071Z" },
 { url = "https://files.pythonhosted.org/packages/97/bb/21c36b7dbbafcf85d2d480cd65df02a1dc93bf76d97147605a8e27ff9409d
/websockets-16.0-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:e
0334872c0a37b606418ac52f6ab9cfd17317ac26365f7f65e203e2d0d0d359f", size = 186152, upload-time = "2026-01-10T09:22:52.224
Z" },
 { url = "https://files.pythonhosted.org/packages/4a/34/9bf8df0cf88fa7bfe36678dc7b02970c9a7d5e065a3099292db87b1be2
/websockets-16.0-cp312-cp312-musllinux_1_2_aarch64.whl", hash = "sha256:a0b31e0b424cc6b5a04b8838bbaec1688834b2383256688
cf47eb97412531da1", size = 185583, upload-time = "2026-01-10T09:22:53.443Z" },
 { url = "https://files.pythonhosted.org/packages/47/88/4dd516068e1a3d6ab3c7c183288404cd424a9a02d585efbac226cb61ff2d
/websockets-16.0-cp312-cp312-musllinux_1_2_x86_64.whl", hash = "sha256:485c49116d0af10ac698623c513c1cc01c9446c058a4e61e
3bf6c19dff7335a2", size = 184880, upload-time = "2026-01-10T09:22:55.033Z" },
 { url = "https://files.pythonhosted.org/packages/91/d6/7d4553ad4bf1c0421e1ebd4b18de5d9098383b5caa1d937b63df8d04b565
/websockets-16.0-cp312-cp312-win32.whl", hash = "sha256:eaded469f5e5b7294e2bdca0ab06becb6756ea86894a47806456089298813c8
9", size = 178261, upload-time = "2026-01-10T09:22:56.251Z" },
 { url = "https://files.pythonhosted.org/packages/c3/f0/f3a17365441ed1c27f850a80b2bc680a0fa9505d733fe152fdf5e98c1c0b
/websockets-16.0-cp312-cp312-win_amd64.whl", hash = "sha256:5569417dc80977fc8c2d43a86f78e0a5a22fee17565d78621b6bb264a11
5d4ea", size = 178693, upload-time = "2026-01-10T09:22:57.478Z" },
 { url = "https://files.pythonhosted.org/packages/cc/9c/baa8456050d1c1b08dd0ec7346026668cbc6f145ab4e314d707bb845bf0d
/websockets-16.0-cp313-cp313-macosx_10_13_universal2.whl", hash = "sha256:878b336ac47938b474c8f982ac2f7266a540adc3fa4ad
74ae96fea9823a02cc9", size = 177364, upload-time = "2026-01-10T09:22:59.333Z" },
 { url = "https://files.pythonhosted.org/packages/7e/0c/8811fc53e9bcff68fe7de2bcbe75116a8d959ac699a3200f4847a8925210
/websockets-16.0-cp313-cp313-macosx_10_13_x86_64.whl", hash = "sha256:52a0fec0e6c8d9a784c2c78276a48a2bdf099e4ccc2a4cad5
3b27718dbfd0230", size = 175039, upload-time = "2026-01-10T09:23:01.171Z" },
 { url = "https://files.pythonhosted.org/packages/aa/82/39a5f910cb99ec0b59e482971238c845af9220d3ab9fa76dd9162cda9d62
/websockets-16.0-cp313-cp313-macosx_11_0_arm64.whl", hash = "sha256:e6578ed5b6981005df1860a56e3617f14a6c307e6a71b4fff8c
48fdc50f3ed2c", size = 175323, upload-time = "2026-01-10T09:23:02.341Z" },
 { url = "https://files.pythonhosted.org/packages/bd/28/0a25ee5342eb5d5f297d992a77e56892ecb65e7854c7898fb7d35e9b33bd
/websockets-16.0-cp313-cp313-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl", hash = "sha256:95724e63
8f0f9c350bb1c2b0a7ad0e83d9cc0c9259f3ea94e40d7b02a2179ae5", size = 184975, upload-time = "2026-01-10T09:23:03.756Z" },
 { url = "https://files.pythonhosted.org/packages/f9/66/27ea52741752f5107c2e41fda05e8395a682a1e11c4e592a809a90c6a506
/websockets-16.0-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:c
0204dc62a89dc9d50d682412c10b3542d748260d743500a85c13cd1ee4bde82", size = 186203, upload-time = "2026-01-10T09:23:05.01Z
" },
 { url = "https://files.pythonhosted.org/packages/37/e5/8e32857371406a757816a2b471939d51c463509be73fa538216ea52b792a
/websockets-16.0-cp313-cp313-musllinux_1_2_aarch64.whl", hash = "sha256:52ac480f44d32970d66763115edea932f1c5b1312de36df
06d6b219f6741eed8", size = 185653, upload-time = "2026-01-10T09:23:06.301Z" },
 { url = "https://files.pythonhosted.org/packages/9b/67/f926bac29882894669368dc73f4da900fcdf47955d0a0185d60103df5737
/websockets-16.0-cp313-cp313-musllinux_1_2_x86_64.whl", hash = "sha256:6e5a82b677f8f6f59e8dfc34ec06ca6b5b48bc4fcd3a46ac
d093694cc2c24d8f", size = 184920, upload-time = "2026-01-10T09:23:07.492Z" },
 { url = "https://files.pythonhosted.org/packages/3c/a1/3d6ccdd125b0a42a311bcd15a7f705d688f73b2a22d8cf1c0875d35d34a
/websockets-16.0-cp313-cp313-win32.whl", hash = "sha256:abf050a199613f64c886ea10f38b47770a65154dc37181bfaaff70c160f45315
a", size = 178255, upload-time = "2026-01-10T09:23:09.245Z" },
 { url = "https://files.pythonhosted.org/packages/6b/ae/90366304d7c2ce80f9b826096a9e9048b4bb760e44d3b873bb272cba696b
/websockets-16.0-cp313-cp313-win_amd64.whl", hash = "sha256:3425ac5cf448801335d6fdc7ae1eb22072055417a96cc6b31b3861f455f
bc156", size = 178689, upload-time = "2026-01-10T09:23:10.483Z" },
 { url = "https://files.pythonhosted.org/packages/f3/1d/e88022630271f5bd349ed82417136281931e558d628dd52c4d8621b4a0b2
/websockets-16.0-cp314-cp314-macosx_10_15_universal2.whl", hash = "sha256:8cc451a50f2aee53042ac52d2d053d08bf89bcb31ae79
9cb4487587661c038a0", size = 177406, upload-time = "2026-01-10T09:23:12.178Z" },
 { url = "https://files.pythonhosted.org/packages/f2/78/e63be1bf0724eeb4616efb1ae1c9044f7c3953b7957799abb5915bffd38e

```



```
/websockets-16.0-cp314-cp314-macosx_10_15_x86_64.whl", hash = "sha256:daa3b6ff70a9241cf6c7fc9e949d41232d9d7d26fd3522b1a
d2b4d62487e9904", size = 175085, upload-time = "2026-01-10T09:23:13.511Z" },
 { url = "https://files.pythonhosted.org/packages/bb/f4/d3c9220d818ee955ae390cf319a7c7a467beceb24f05ee7aaaa2414345ba
/websockets-16.0-cp314-cp314-macosx_11_0_arm64.whl", hash = "sha256:fd3cb4adb94a2a6e2b7c0d8d05cb94e6f1c81a0cf9dc2694fb6
5c7e8d94c42e4", size = 175328, upload-time = "2026-01-10T09:23:14.727Z" },
 { url = "https://files.pythonhosted.org/packages/63/bc/d3e208028de777087e6fb2b122051a6ff7bbcca0d6df9d9c2bf1dd869ae9
/websockets-16.0-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl", hash = "sha256:781caf5e
8eee67f663126490c2f96f40906594cb86b408a703630f95550a8c3e", size = 185044, upload-time = "2026-01-10T09:23:15.939Z" },
 { url = "https://files.pythonhosted.org/packages/ad/6e/9a0927ac24bd33a0a9af834d89e0abc7cfd8e13bed17a86407a66773cc0e
/websockets-16.0-cp314-cp314-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:c
aab51a72c51973ca21fa8a18bd8165e1a0183f1ac7066a182ff27107b71e1a4", size = 186279, upload-time = "2026-01-10T09:23:17.148
Z" },
 { url = "https://files.pythonhosted.org/packages/b9/ca/bf1c68440d7a868180e11be653c85959502efd3a709323230314fda6e0b3
/websockets-16.0-cp314-cp314-musllinux_1_2_aarch64.whl", hash = "sha256:19c4dc84098e523fd63711e563077d39e90ec6702aff4b5
d9e344a60cb3c0cb1", size = 185711, upload-time = "2026-01-10T09:23:18.372Z" },
 { url = "https://files.pythonhosted.org/packages/c4/f8/fdc34643a989561f217bb477cbc47a3a07212cbda91c0e44389c43c296ebf
/websockets-16.0-cp314-cp314-musllinux_1_2_x86_64.whl", hash = "sha256:a5e18a238a2b2249c9a9235466b90e96ae4795672598a587
72dd806edc7ac6d3", size = 184982, upload-time = "2026-01-10T09:23:19.652Z" },
 { url = "https://files.pythonhosted.org/packages/dd/d1/574fa27e233764dbac9c52730d63fcf2823b16f0856b3329fc6268d6ae4f
/websockets-16.0-cp314-cp314-win32.whl", hash = "sha256:a069d734c4a043182729edd3e9f247c3b2a4035415a9172fd0f1b71658a320a
8", size = 177915, upload-time = "2026-01-10T09:23:21.458Z" },
 { url = "https://files.pythonhosted.org/packages/8a/f1/ae6b937bf3126b5134ce1f482365fde31a357c784ac51852978768b5eff4
/websockets-16.0-cp314-cp314-win_amd64.whl", hash = "sha256:c0ee0e63f23914732c6d7e0cce24915c48f3f1512ec1d079ed01fc629da
b269d", size = 178381, upload-time = "2026-01-10T09:23:22.715Z" },
 { url = "https://files.pythonhosted.org/packages/06/9b/f791d1db48403e1f0a27577a6beb37afae94254a8c6f08be4a23e4930bc0
/websockets-16.0-cp314-cp314t-macosx_10_15_universal2.whl", hash = "sha256:a35539cacc3febb22b8f4d4a99cc79b104226a756aa7
400adc722e83b0d03244", size = 177737, upload-time = "2026-01-10T09:23:24.523Z" },
 { url = "https://files.pythonhosted.org/packages/bd/40/53ad02341fa33b3ce489023f635367a4ac98b73570102ad2cdd770dacc9a
/websockets-16.0-cp314-cp314t-macosx_10_15_x86_64.whl", hash = "sha256:b784ca5de850f4ce93ec85d3269d24d4c82f22b7212023c9
74c401d4980ebc5e", size = 175268, upload-time = "2026-01-10T09:23:25.781Z" },
 { url = "https://files.pythonhosted.org/packages/74/9b/6158d4e459b984f949dcbbb0c5d270154c7618e11c01029b9bbd1bb4c4f9
/websockets-16.0-cp314-cp314t-macosx_11_0_arm64.whl", hash = "sha256:569d01a4e7fba956c5ae4fc988f0d4e187900f5497ce46339c
996dbf24f17641", size = 175486, upload-time = "2026-01-10T09:23:27.033Z" },
 { url = "https://files.pythonhosted.org/packages/e5/2d/7583b30208b639c8090206f95073646c2c9ffdd66f44df967981a64f849ad
/websockets-16.0-cp314-cp314t-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl", hash = "sha256:50f23cd
d8343b984957e4077839841146f67a3d31ab0d00e6b824e74c5b2f6e8", size = 185331, upload-time = "2026-01-10T09:23:28.259Z" },
 { url = "https://files.pythonhosted.org/packages/45/b0/cce3784eb519b7b5ad680d14b9673a31ab8dcb7aad8b64d81709d2430aa8
/websockets-16.0-cp314-cp314t-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl", hash = "sha256:
152284a83a00c59b759697b7f9e9cddf4e3c7861dd0d964b472b70f78f89e80e", size = 186501, upload-time = "2026-01-10T09:23:29.44
92Z" },
 { url = "https://files.pythonhosted.org/packages/19/60/b8ebe4c7e89fb5f6cdf080623c9d92789a53636950f7abacfc33fe2b3135
/websockets-16.0-cp314-cp314t-musllinux_1_2_aarch64.whl", hash = "sha256:bc59589ab64b0022385f429b94697348a6a234e8ce2254
4e3681b2e9331b5944", size = 186062, upload-time = "2026-01-10T09:23:31.368Z" },
 { url = "https://files.pythonhosted.org/packages/88/a8/a080593f89b0138b6cba1b28f8df5673b5506f72879322288b031337c0b8
/websockets-16.0-cp314-cp314t-musllinux_1_2_x86_64.whl", hash = "sha256:32da954ffa2814258030e5a57bc73a3635463238e797c73
75dc8091327434206", size = 185356, upload-time = "2026-01-10T09:23:32.627Z" },
 { url = "https://files.pythonhosted.org/packages/c2/b6/b9afed2afaddaf5ebb2afa801abf4b0868f42f8539bfe4b071b5266c9fe
/websockets-16.0-cp314-cp314t-win32.whl", hash = "sha256:5a4b4cc550cb665dd8a47f868c8d04c8230f857363ad3c9caf7a0c3bf8c61c
a6", size = 178085, upload-time = "2026-01-10T09:23:33.816Z" },
 { url = "https://files.pythonhosted.org/packages/9f/3e/28135a24e384493fa804216b79a6a6759a38cc4ff59118787b9fb693df93
/websockets-16.0-cp314-cp314t-win_amd64.whl", hash = "sha256:b14dc141ed6d2dde437cddb216004bcac6a1df0935d79656387bd41632
ba0bbd", size = 178531, upload-time = "2026-01-10T09:23:35.016Z" },
 { url = "https://files.pythonhosted.org/packages/6f/28/258ebab549c2bf3e64d2b0217b973467394a9cea8c42f70418ca2c5d0d2e
/websockets-16.0-py3-none-any.whl", hash = "sha256:1637db62fad1dc833276dded54215f2c7fa46912301a24bd94d45d46a011ceec", s
ize = 171598, upload-time = "2026-01-10T09:23:45.395Z" },
]
```

#### EXPORT SUMMARY

```
Source folder: C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master
Total files found: 115
Text/code files processed: 104
Files skipped: 11
```

#### SKIPPED FILES

```
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\data\input\IDMS_DB2_Code_09_07_2026.pdf - Binary file s
kipped
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\data\input\IDMS_Schema_Diagram.pdf - Binary file skippe
d
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\data\input\IDMS_Schema_Listing.pdf - Binary file skippe
d
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\data\input\Retrieval_COBOL_code.pdf - Binary file skipp
ed
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\data\input\Schema_Listing.pdf - Binary file skipped
```

C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\data\input\Update\_COBOL\_code.pdf - Binary file skipped  
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\data\output\er\_diagram.png - Binary file skipped  
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\data\temp\diagram\_page\_0.png - Binary file skipped  
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\src\idms\_modernizer\data\IDMS Diagram.pdf - Binary file skipped  
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\src\idms\_modernizer\data\IDMS\_Schema\_Listing.pdf - Binary file skipped  
C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\src\idms\_modernizer\data\Retrieval.pdf - Binary file skipped